多模态大模型

讲义

原理、方法与研究实践

精简版 v1.0

作者：刘佳宇

2026 年 10 月

前言与使用方法

本讲义面向已修过机器学习、深度学习、线性代数与概率统计的研究生。讨论从观测如何变为可学习的表示开始，依次进入跨模态目标、模型连接与融合、数据和训练，再到能力验证、部署和交互。学习的重点是根据任务说明模型为什么这样设计，并能用实验检验这种设计。

贯穿全书的例子是视觉问答：一幅图像经过怎样的预处理与编码，哪些视觉信息到达语言模型，回答位置如何获得监督，怎样检查答案确实依赖图像。这个简单入口可以延伸到文档、视频、语音、动作和外部工具。遇到一种新架构时，可沿着同一条路径追踪输入、状态、损失和验证协议。

正文分为五篇。第一篇建立表示与接口，第二篇讨论对齐、架构和生成，第三篇落实数据与训练，第四篇建立能力诊断与实验方法，第五篇处理系统成本、持续交互和研究实践。第 4 章的传感器专题、第 8 章的生成专题，以及第 17 章的具身和世界模型部分可以按研究方向选读。附录用于查符号、补先修和检查实验记录。

每章给出学习目标、必要的算例以及四道习题。题目区分定义、推导、诊断和实验设计；附录提供解题要点。三项综合实践是作业设计，书中没有将未经运行的设计写成实验结果。实施时可以使用小模型、冻结主干或离线特征，先验证机制，再增加规模。

阅读顺序 首次学习建议先读第 1–3 章，再读第 5–7 章，接着学习第 9–15 章。进行系统实验时加入第 16–18 章；生成与具身方向分别补第 8 章和第 4 章。48 学时课程需要从这些单元中选择核心阅读，不能把全书全部专题都作为逐页必读内容。

记号与范围 公式中的维度、单位、索引和条件在使用处说明。模型隐藏维度与输入的原始维度分别记载；同一字母在不同局部问题中可以有不同含义，不能跨章直接代入。所有计算成本都需要注明所含模块、序列长度及训练或推理阶段。数字算例只解释定义，除明确注明来自论文者外，不代表推荐超参数或性能结论。

版本与来源 本版根据原 21 章讲义重新组织。重复定义与通用小结合并，必要技术内容按其主要用途迁移。来源、迁移表、表格重排及公式恢复的记录存放在工程的 `editorial/` 目录。原稿中依相邻文字整理的表格不作为论文实验数据转载。本版机制图为教学示意，文献导读指向可核验的原始论文；新的实证结论仍需独立实验支持。

目录

前言与使用方法 i

第一部分　问题与表示 1

第一章　多模态学习的问题、目标与先修约定 2
1.1　观察、任务与信息来源 . . . 2
1.2　从表示到条件预测 . . . 5
1.3　序列、概率与注意力约定 . . . 6
1.4　研究问题与阅读路线 . . . 10
习题与讨论 . . . 10

第二章　视觉表示与信息粒度 11
2.1　图像接口与预处理 . . . 11
2.2　从图像块到视觉序列 . . . 12
2.3　空间结构与特征粒度 . . . 17
2.4　高分辨率与坐标恢复 . . . 22
2.5　编码器的预训练与迁移 . . . 26
习题与讨论 . . . 29

第三章　音频与视频的时序表示 30
3.1　音频的时间轴与时频接口 . . . 30
3.2　连续与离散音频表示 . . . 36
3.3　语音预训练与非语言声学信息 . . . 42
3.4　视频采样与时空编码 . . . 47
3.5　长视频的信息组织 . . . 55
3.6　音视频同步与对应关系 . . . 60
习题与讨论 . . . 63

第四章　空间、动作与异步传感器表示 65
4.1　三维观测与坐标变换 . . . 65
4.2　姿态、运动与动作接口 . . . 71
4.3　惯性与机器人状态 . . . 76
4.4　事件流的生成与编码 . . . 82
4.5　时间对齐与空间标定 . . . 88
习题与讨论 . . . 93

第二部分　对齐、架构与生成 94

第五章　跨模态对齐与学习目标 95
5.1　对应关系与表示空间 . . . 95
5.2　双向对比目标与梯度 . . . 98
5.3　批内负样本与监督噪声 . . . 99
5.4　检索与零样本分类 . . . 100
5.5　匹配、掩码与条件生成 . . . 100
5.6　联合学习与知识迁移 . . . 107
习题与讨论 . . . 112

第六章 视觉语言模型的连接与条件注入 **113**
6.1 统一的数据流与接口 . . . 113
6.2 逐位置映射与梯度传播 . . . 116
6.3 查询式视觉压缩 . . . 117
6.4 交叉注意力与条件访问 . . . 124
6.5 三种经典架构 . . . 131
6.6 架构选择的诊断方法 . . . 136
习题与讨论 . . . 138

第七章 多模态融合与统一架构 **139**
7.1 融合发生在哪里 . . . 139
7.2 序列组织与交互代价 . . . 145
7.3 参数共享与信息保留 . . . 151
7.4 条件路由与专家机制 . . . 158
7.5 统一输入输出与训练目标 . . . 163
7.6 理解与生成的结构取舍 . . . 171
习题与讨论 . . . 175

第八章 条件生成与理解生成协同 **176**
8.1 自回归、扩散与流匹配 . . . 176
8.2 生成表示与条件机制 . . . 179
8.3 潜空间图像生成与控制 . . . 187
8.4 视频的运动与长时一致性 . . . 190
8.5 语音与音视频生成 . . . 194
习题与讨论 . . . 201

第三部分 数据与训练 202

第九章 多模态数据与监督设计 **203**
9.1 样本结构与监督粒度 . . . 203
9.2 来源、去重与质量控制 . . . 209
9.3 配对、时间与区域标注 . . . 214
9.4 指令与合成数据 . . . 219
9.5 数据混合与采样 . . . 225
9.6 许可、隐私与可追溯性 . . . 231
习题与讨论 . . . 236

第十章 预训练流程与优化 **237**
10.1 训练阶段与参数范围 . . . 237
10.2 任务混合与损失统计 . . . 240
10.3 优化器与有效批量 . . . 245
10.4 训练资源、规模与分布式优化 . . . 247
10.5 恢复、监控与故障定位 . . . 254
10.6 收敛与训练结果诊断 . . . 257
习题与讨论 . . . 259

第十一章 指令微调与偏好优化 **260**
11.1 指令样本与监督位置 . . . 260
11.2 多模态指令与证据利用 . . . 266
11.3 低秩适配与模块选择 . . . 271
11.4 奖励建模与策略优化 . . . 277
11.5 直接偏好优化与多模态反馈 . . . 281
11.6 能力保持与分布偏差 . . . 284
习题与讨论 . . . 293

第四部分　能力与验证 294

第十二章　感知、文档理解与细粒度定位 295
12.1 任务输出与监督粒度 . . . 295
12.2 文字、版面与图表 . . . 304
12.3 区域预测与语言条件定位 . . . 311
12.4 时间片段与音视频事件 . . . 317
12.5 证据缺失与感知结果校验 . . . 324
习题与讨论 . . . 330

第十三章　多模态推理与证据利用 331
13.1 问题分解与证据依赖 . . . 331
13.2 关系、数量与图表求解 . . . 335
13.3 时序关系与因果线索 . . . 344
13.4 空间约束与动态状态 . . . 349
13.5 推理时计算与外部验证 . . . 356
习题与讨论 . . . 363

第十四章　评测协议与研究实验设计 364
14.1 估计什么、如何划分 . . . 364
14.2 公平基线、消融与统计 . . . 368
14.3 感知、检索与定位指标 . . . 373
14.4 时序、音频与生成评价 . . . 381
14.5 复杂任务与裁判 . . . 388
14.6 污染、设置与分组结果 . . . 395
习题与讨论 . . . 404

第十五章　幻觉、鲁棒性与可靠性 405
15.1 证据不一致的类型与成因 . . . 405
15.2 缺失、扰动、错配与分布变化 . . . 410
15.3 不确定性与置信度校准 . . . 417
15.4 失败检测、拒答与风险控制 . . . 420
15.5 证据引用与外部校验 . . . 424
15.6 攻击、不可信输入与系统边界 . . . 428
习题与讨论 . . . 433

第五部分　系统与研究训练 435

第十六章　推理效率与模型部署 436
16.1 端到端成本与关键路径 . . . 436
16.2 上下文、视觉预算与缓存 . . . 441
16.3 低精度与批处理 . . . 445
16.4 服务接口、调度与监控 . . . 449
16.5 端侧部署与任务拆分 . . . 454
习题与讨论 . . . 459

第十七章　多模态智能体、持续交互与具身行动 461
17.1 闭环、状态与动作接口 . . . 461
17.2 工具调用与结果验证 . . . 465
17.3 检索、记忆与执行恢复 . . . 472
17.4 流式观测与持续状态 . . . 484
17.5 语言条件策略与具身数据 . . . 489
17.6 世界模型与闭环验证 . . . 494
17.7 执行成本与权限边界 . . . 497
习题与讨论 . . . 503

第十八章 综合案例与研究实践 **505**
18.1 案例一：多模态文档助手 505
18.2 案例二：持续音视频事件理解 509
18.3 案例三：语言条件具身操作 511
18.4 行业差异与研究问题 515
18.5 从复现到可检验的贡献 518
习题与讨论 519

第六部分 附录 **520**

附录 A 先修知识与实现速查 **521**
A.1 向量、形状与归一化 521
A.2 概率、交叉熵与监督位置 521
A.3 注意力、残差与梯度 521
A.4 优化、资源与复现 522
A.5 卷积与尺度的必要背景 522

附录 B 符号、单位与接口约定 **523**
B.1 常见记号 523
B.2 每个接口必须记录的内容 523

附录 C 习题解答要点 **524**
C.1 第 1 章 524
C.2 第 2 章 524
C.3 第 3 章 524
C.4 第 4 章 525
C.5 第 5 章 525
C.6 第 6 章 525
C.7 第 7 章 526
C.8 第 8 章 526
C.9 第 9 章 526
C.10 第 10 章 526
C.11 第 11 章 527
C.12 第 12 章 527
C.13 第 13 章 527
C.14 第 14 章 528
C.15 第 15 章 528
C.16 第 16 章 528
C.17 第 17 章 529
C.18 第 18 章 529

附录 D 实践协议与复现记录 **530**
D.1 六项机制实践 530
D.2 每次实验的最小记录 530
D.3 三项综合案例的验收 531
D.4 来源与本版整理范围 531

第一部分

问题与表示

第一章　多模态学习的问题、目标与先修约定

一张图像能否帮助模型回答问题，取决于任务需要什么证据、编码器保留了什么，以及训练目标奖励了什么。把图像转换成与词嵌入同维的向量，只完成了计算接口的连接；证据是否被保留、是否被使用，仍需训练与验证。全书沿着这条链路展开。

学习目标　给定一个多模态任务，说明其观测、输出、监督和验证对象；写出条件预测目标；区分表示接口、语义对应与信息融合。

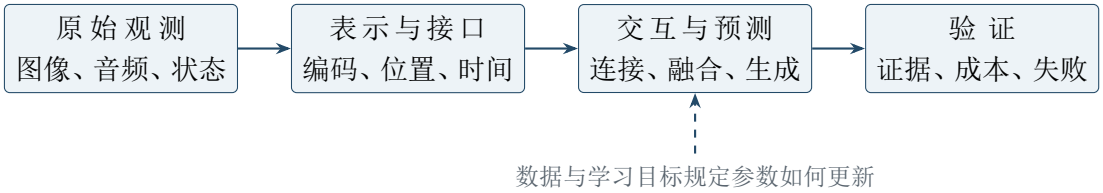


图 1.1　从观测到验证的教学主线。训练与推理共用表示和预测模块，监督及参数更新发生在训练阶段。

1.1　观察、任务与信息来源

将一次任务写为观测 $X = (X^{(1)}, \ldots, X^{(M)})$、问题或条件 q、目标 y。模态标识指出数据的物理来源与表示规则；一个文件可以包含多个模态，同一模态也可以有不同编码。例如，视频中的帧、音轨与字幕应分别保留时间与来源，不能因它们封装在同一文件中而混为一种输入。

1.1.1　模态的定义与常见模态类型

模态　模态（Modality）是指信息被感知、获取、表示或表达的一种形式或通道。在人工智能领域，模态通常用于区分不同类型的数据及其所承载的信息，例如文本、图像、音频、视频、三维空间数据和传感器信号等。

不同模态往往具有不同的数据结构、统计特征和信息表达方式。例如，文本通常由离散的词元序列构成，图像通常表现为二维像素网格，音频通常表现为连续的时间信号，而视频则可以看作具有时间顺序的图像帧序列。这些差异决定了不同模态在数据处理、特征提取和模型建模方面具有各自的特点。

模态不等同于数据文件格式，也不完全等同于传感器类型。同一种模态可以具有不同的表示形式，例如图像可以用原始像素、视觉特征或视觉 Token 表示；同一种传

感器也可以产生不同形式的数据。同样，多个摄像头虽然可以提供来自不同视角的观测，但它们仍然可能属于相同的视觉模态。

因此，理解模态的关键，不只是判断数据采用什么格式存储，而是分析数据从何而来、以什么形式组织，以及它能够提供什么类型的信息。

多模态的基本含义 当一个系统需要处理两种或两种以上不同模态的信息，并利用它们之间的关联完成理解、预测或生成等任务时，就涉及多模态学习（Multimodal Learning）。例如：

- 图像与文本结合，可以让模型根据自然语言指令回答图像相关问题。
- 视频与音频结合，可以同时利用画面和声音识别事件。
- 摄像头与激光雷达结合，可以同时利用视觉语义与几何距离信息。
- 图像与事件相机数据结合，可以利用图像中的外观信息和事件流中的高时间分辨率变化信息。

从形式上，可以将一个多模态输入表示为：

$$\mathcal{X} = \{X^{(1)}, X^{(2)}, \ldots, X^{(M)}\} \tag{1.1}$$

其中，M 表示参与任务的模态数量，$X^{(m)}$ 表示第 m 种模态的观测数据。模型根据具体任务，利用这些输入产生输出：

$$Y = f\left(X^{(1)}, X^{(2)}, \ldots, X^{(M)}\right) \tag{1.2}$$

这里的 f 表示相应的学习模型，Y 可以是分类结果、文本回答、空间预测，也可以是图像、音频或其他形式的生成结果。多模态并不意味着必须将所有模态转换为相同的数据形式，也不意味着所有模态必须在同一时刻输入模型。不同模态可以保留各自的表示方式，通过适当的机制建立关联并协同工作。

同时，**多模态学习与多模态大模型并不是同一个概念。**多模态学习是一个更广泛的研究领域，其中既包括针对特定任务设计的小型融合模型，也包括利用大规模数据进行预训练、能够执行多种任务的多模态基础模型。多模态大模型是在这一发展脉络下形成的重要技术方向，后续章节将进一步讨论其架构和训练方法。

1.1.2 多模态信息的互补性、冗余性与异构性

多模态学习的价值并不只是同时接收多种数据，而是利用不同信息来源之间的关系，提高系统对复杂场景的理解能力。不同模态之间通常同时存在互补性、冗余性和异构性。这三个特征共同决定了多模态信息融合的潜在收益及其技术难点。

互补性：不同模态提供不同的信息 多模态信息的互补性（Complementarity），是指不同模态能够提供彼此无法完全替代的信息，从而共同支持更完整的场景理解。例如，在自动驾驶场景中，摄像头图像能够提供道路标线、交通标志、物体颜色和纹理等丰富

的视觉语义信息；激光雷达则能够提供较为直接的距离测量和三维几何结构。两者关注的信息有所不同，因此具有互补性。

在音视频理解任务中，视频可以提供人物动作和场景变化，音频则可以提供语音内容、环境声音和声学事件。即使画面中没有明确呈现某个事件，声音也可能提供重要线索。互补性是多模态融合的重要动机之一。理想情况下，模型能够从不同模态中提取有用信息，并根据任务需求将它们结合起来，从而获得单一模态难以达到的效果。

但是，互补性并不意味着简单地增加模态就一定能够提升性能。如果某个模态与任务关系较弱、质量较差，或者模型无法有效利用其信息，那么额外输入也可能带来噪声和计算开销。

冗余性：不同模态可能表达相同或相近的信息 多模态信息的冗余性（Redundancy），是指不同模态之间可能包含部分重复、重叠或高度相关的信息。例如，在视频会议中，语音内容与字幕通常表达相近的语言语义；在道路场景中，同一个车辆可能同时出现在摄像头图像和激光雷达点云中；在视频中，相邻帧也可能包含大量相似的场景信息。

冗余信息并不一定是无用信息。适当的冗余可以为模型提供交叉验证的机会，提高系统在部分观测不可靠时的稳定性。例如，当语音识别结果不够清晰时，字幕或画面中的其他线索可能帮助模型修正判断。另一方面，冗余信息也可能造成不必要的计算开销。如果模型重复处理大量高度相似的信息，却没有提取新的有效内容，就可能降低计算效率。因此，多模态系统不仅需要利用互补信息，也需要判断哪些信息具有重复性，以及何时有必要保留、压缩或筛选这些信息。

异构性：不同模态具有不同的数据结构与统计特征 多模态信息的异构性（Heterogeneity），是指不同模态在数据形式、统计分布、时间尺度、空间结构和语义表达方式等方面存在显著差异。这种差异主要体现在以下几个方面。

第一，数据结构不同。文本通常表现为离散 Token 序列，图像表现为二维像素网格，音频表现为连续时间信号，而点云由具有空间坐标的点集合构成。不同数据结构通常需要不同的编码与处理方式。

第二，统计特征不同。不同模态的数据分布、噪声形式和特征尺度存在差异。例如，图像可能受到光照和遮挡的影响，音频可能受到环境噪声的影响，点云可能存在稀疏性和测量误差。

第三，时间尺度不同。静态图像通常描述某一时刻的场景，视频由离散帧构成，音频和某些传感器数据则可能具有更高的时间采样频率。不同模态之间不一定天然同步。

第四，空间结构与语义粒度不同。图像特征通常与二维空间位置相关，点云具有三维几何结构，文本则主要通过语言符号表达语义。即使不同模态描述同一个对象，其信息粒度与组织方式也可能不同。

第五，信息可靠性不同。同一场景中的不同模态可能受到不同因素的干扰。例如，低照度会影响摄像头成像，而强烈的背景噪声可能影响语音识别。不同模态的可靠性也会随环境和任务发生变化。

这些差异意味着，不同模态的数据不能总是直接拼接或简单相加。模型需要建立

适当的表示与关联机制，才能有效利用不同模态之间的关系。

1.2 从表示到条件预测

对于生成回答的模型，基本对象是 $p_\theta(y \mid X, q)$。训练用观测和目标构造损失，推理根据可获得的观测生成输出。若目标为序列 $y = (y_1, \ldots, y_L)$，则

$$p_\theta(y \mid X, q) = \prod_{t=1}^{L} p_\theta(y_t \mid y_{<t}, X, q). \tag{1.3}$$

该分解规定预测顺序，没有规定观测必须怎样编码，也没有保证生成内容忠于观测。后续各章分别解决这些问题。

1.2.1 从任务专用模型到通用多模态大模型

多模态大模型的基本思路 多模态大模型（Multimodal Large Models）通常**指通过大规模数据和模型训练，能够处理两种或两种以上模态，并支持较广泛理解、推理或生成任务的模型**。其中，**视觉语言大模型是目前常见的一类代表**。它们可以接收图像和文本，通过联合建模完成图像问答、图像描述、文档理解和视觉推理等任务。

更广泛的多模态模型还可以处理音频、视频、空间数据或其他类型的信息，并在一定条件下实现多模态输入与多模态输出。与传统任务专用系统相比，多模态大模型通常具有以下特点：

- **任务覆盖范围更广**。同一个模型可以通过指令或上下文处理多种相关任务，而不必为每个任务单独设计完整模型。
- **更强调大规模预训练**。模型通过大规模数据学习通用表示和任务相关能力，再通过后训练或适配提升特定能力。
- **更加重视语言交互**。许多多模态大模型利用语言作为统一的交互接口，使用户能够通过自然语言提出问题、提供约束或描述目标。
- **具有更强的组合式应用潜力**。模型可以结合检索、工具调用和外部系统完成复杂任务。
- **需要更复杂的训练与评测**。模型能力涉及多种模态和任务，不能只通过单一指标或少量演示判断其实际水平。

这些特点是总体趋势，并不意味着每一个多模态大模型都具备上述全部能力。具体能力取决于模型架构、训练数据、训练方法和实际部署条件。

从外接式架构到原生多模态建模 多模态大模型的实现并不只有一种技术路线。一类常见方案是采用**外接式架构**：视觉、音频或其他模态分别由**专用编码器处理**，再通过**投影模块、交叉注意力或其他连接机制**接入语言模型。这种方式能够利用已有的预训练模型，具有较好的模块化特点，也便于在现有语言模型基础上逐步增加新的模态能力。

另一类路线更加重视**原生多模态建模**，即在模型架构和训练过程中直接考虑多种

模态的联合处理，使模型能够在更加统一的机制下接收和生成不同类型的信息。不过，“原生多模态”并不存在适用于所有模型的单一严格定义。不同模型在模态编码器、Token 表示、融合位置、训练目标和输出方式上可能具有很大差异。

外接式架构并不必然缺乏通用性，原生多模态架构也不必然在所有任务上更优。两者在模型复用、信息交互、训练成本和扩展能力方面各有取舍，具体优劣需要通过实际任务和公平实验判断。

通用多模态大模型并不意味着完全通用 从任务专用模型走向多模态大模型，并不意味着专用模型已经失去价值，也不意味着通用模型能够在所有任务上取代专用系统。对于边界清晰、实时性要求高或需要严格控制计算资源的任务，专用模型仍然可能具有明显优势。对于需要自然语言交互、跨任务迁移、复杂信息整合或多种模态联合理解的任务，多模态大模型则具有较强的应用潜力。

同时，多模态大模型仍然可能出现视觉细节遗漏、空间关系判断错误、视频时间顺序理解不足、模态冲突处理不当以及输出缺乏可靠证据等问题。因此，判断一个模型是否适合某项任务，需要综合考虑任务准确率、鲁棒性、推理成本、响应延迟、数据要求和部署条件，而不能只看参数规模或模型是否具有通用交互能力。

1.2.2 跨模态对齐与信息融合

跨模态对齐与信息融合密切相关，但并不是同一个概念。

跨模态对齐主要关注不同模态之间如何建立对应关系。

例如，判断一张图像和一段文字是否描述相同的内容，或者确定文本中的某个对象对应图像中的哪个区域。

信息融合（Multimodal Fusion）则主要关注如何综合利用不同模态的信息。

例如，在视觉问答任务中，模型既需要理解用户提出的问题，又需要利用图像中的相关信息，最终生成符合输入内容的回答。因此，对齐可以为融合提供重要基础，但并不是所有融合方法都必须首先显式学习一个共享语义空间。一些模型可以通过联合训练和跨模态注意力直接学习信息交互，而不需要事先构建完全独立的对齐模型。同样，跨模态对齐效果较好，也不意味着模型已经能够有效完成所有复杂融合与推理任务。

1.3 序列、概率与注意力约定

本书以行表示 Token，$Z \in \mathbb{R}^{N\times d}$ 中 N 为长度、d 为隐藏维度；批量维需要时显式写出。注意力的 Softmax 沿键的位置计算。因果遮罩中的可见性以信息在决策时是否可用为准：时间索引靠前和实际已经到达是两个不同条件。编码器可以同时读取整幅静态图像，在线系统则不能读取尚未到达的视频帧。

1.3.1 自回归序列建模的基本思想

自回归语言模型的概率分解 对于长度为 T 的 Token 序列：$X = (x_1, x_2, \ldots, x_T)$ 根据概率论中的链式法则，可以将序列的联合概率分解为：

$$\prod_{t=1}^{T} P(x_t \mid x_1, \ldots, x_{t-1}) \tag{1.4}$$

也可以简写为：

$$\prod_{t=1}^{T} P(x_t \mid x_{<t}) \tag{1.5}$$

其中，$x_{<t}$ 表示第 t 个 Token 之前的所有 Token。这个公式是自回归语言建模的重要基础。它说明，一个完整序列的概率可以分解为一系列条件概率的乘积。在语言生成过程中，模型每次根据已有上下文预测下一个 Token，再将新生成的 Token 加入上下文，继续后续预测。这种机制使语言模型能够生成长度可变的文本序列。

自回归训练的损失函数 自回归语言模型通常通过最小化正确目标 Token 的负对数概率进行训练。对于长度为 T 的目标序列，一种常见损失形式为：

$$-\frac{1}{T}\sum_{t=1}^{T} \log P_\theta(x_t \mid x_{<t}) \tag{1.6}$$

其中：

- T 表示参与损失计算的目标 Token 数量。
- θ 表示模型参数。
- $P_\theta(x_t \mid x_{<t})$ 表示模型在已有上下文条件下，为正确 Token 分配的概率。

如果模型对正确 Token 分配较高概率，损失通常较小；反之，损失较大。在实际训练中，模型可以利用交叉熵损失实现这一目标，并根据需要对填充位置或其他不参与训练的位置进行掩码处理。在指令微调等场景中，模型可能只对回答部分计算损失，而不对所有输入提示 Token 计算损失。因此，损失计算范围应当与具体训练任务保持一致。

Teacher Forcing **与训练并行化** 自回归模型在生成时通常需要逐个产生 Token，但训练过程并不一定需要完全按照这种顺序逐个执行前向计算。在常见的 Teacher Forcing 训练方式中，模型直接使用真实历史 Token 作为输入，学习预测对应位置的下一个 Token。例如：输入：<BOS> / 今天 / 天气 / 很 / 好目标：今天 / 天气 / 很 / 好 / <EOS>

由于训练时整个输入序列已经已知，Transformer 可以在满足因果约束的前提下，并行计算多个位置的隐藏状态与预测结果。这与逐 Token 生成时不断将新输出加入上下文的过程不同。因此，需要区分两个概念：

训练阶段：可以利用完整的已知输入序列和因果注意力机制，并行计算多个位置的下一 Token 预测损失。

生成阶段：由于未来生成的 Token 尚未确定，通常需要按照自回归顺序逐步生成新的 Token。

这种差异对于理解大语言模型的训练效率和推理延迟非常重要。

因果约束与未来信息屏蔽 自回归语言模型要求预测当前位置的下一个 Token 时，不能提前利用尚未出现的未来内容。因此，采用 Transformer 架构的自回归模型通常需要使用因果注意力掩码（Causal Attention Mask）。例如，对于包含四个位置的输入序列，允许访问的位置关系可以表示为：

其中，✓ 表示当前位置可以利用相应位置的信息，× 表示不能访问。这样，模型在第一个位置不能提前看到第二个位置的实际输入内容，而在第四个位置可以使用前四个位置的历史信息。这种下三角形式的访问约束，是因果 Transformer 的重要特征。

因果掩码与用于忽略 PAD Token 的填充掩码不是同一个概念。两者可以在实际模型中共同使用，但解决的问题不同。关于注意力分数、因果掩码如何参与计算，以及如何构建多头注意力，将在 2.3 节进一步展开。

1.3.2 注意力机制与自注意力

Query、Key 和 Value 的含义 在 Transformer 中，注意力机制通常采用 Query、Key 和 Value 三类向量，分别记为 Q、K 和 V。可以通过信息检索的类比理解它们：Query（查询向量）：表示当前位置希望从其他位置获得什么类型的信息。Key（键向量）：用于描述各个候选位置，使模型能够判断它们与 Query 的匹配程度。Value（值向量）：表示各个候选位置实际提供给注意力输出的信息。

例如，在处理文本中的一个代词时，Query 可以参与寻找与该代词相关的上下文位置，Key 用于计算不同位置的匹配分数，Value 则提供被加权聚合的特征。不过，Q、K 和 V 并不是人工编写的自然语言查询、关键词或数据库字段，而是模型通过可学习线性变换生成的连续向量。假设输入表示为：

$$X \in \mathbb{R}^{n \times d_{\text{model}}} \tag{1.7}$$

那么可以分别通过三个投影矩阵得到：$Q = XW_Q$，$K = XW_K$，$V = XW_V$，其中：

$$W_Q \in \mathbb{R}^{d_{\text{model}} \times d_k} \tag{1.8}$$

$$W_K \in \mathbb{R}^{d_{\text{model}} \times d_k} \tag{1.9}$$

$$W_V \in \mathbb{R}^{d_{\text{model}} \times d_v} \tag{1.10}$$

这些投影矩阵通常是可学习参数，在训练过程中通过梯度优化逐步调整。因此，同一份输入可以被投影为承担不同计算角色的 Q、K 和 V。

缩放点积注意力 Transformer 中最经典的注意力形式是缩放点积注意力（Scaled Dot-Product Attention）。计算 Query 与所有 Key 之间的点积：$S = QK^T$，其中，S 是注意力分数矩阵。如果 Query 和 Key 分别具有 n_q 和 n_k 个位置，那么注意力分数矩阵的形状为：

$$S \in \mathbb{R}^{n_q \times n_k} \tag{1.11}$$

矩阵中的第 i 行、第 j 列表示第 i 个 Query 与第 j 个 Key 的匹配分数。为了控制点积数值的尺度，Transformer 通常将分数除以 $\sqrt{d_k}$，其中 d_k 为 Query 和 Key 的特征维度。随后，通过 Softmax 将分数转换为归一化权重：

$$A = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) \tag{1.12}$$

使用注意力权重对 Value 进行加权聚合：$O = AV$ 将上述过程合并，可以得到经典的缩放点积注意力公式：

$$\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{1.13}$$

Softmax 通常沿着 Key 所在的维度进行，因此每个 Query 对所有允许访问的 Key 所分配的注意力权重之和为 1。这一公式是理解 Transformer 的关键。它表明，注意力机制通过计算 Query 和 Key 之间的相似性来确定信息聚合权重，再利用这些权重对 Value 进行加权求和，从而形成新的特征表示。

为什么需要除以 $\sqrt{d_k}$ 缩放点积注意力中的分母 $\sqrt{d_k}$ 并不是任意设置的。假设 Query 和 Key 向量各维度近似独立，并且具有均值为 0、方差为 1 的分布，那么它们的点积可以写为：

$$\sum_{i=1}^{d_k} q_i k_i \tag{1.14}$$

在相应的简化统计假设下，该点积的方差与 d_k 成正比。随着向量维度增加，点积的典型数值尺度可能增大。如果直接将较大的分数输入 Softmax，输出概率可能变得非常集中，使部分位置接近 1，其他位置接近 0。这种现象可能使相关梯度变得很小，从而影响优化过程。通过除以 $\sqrt{d_k}$，可以在上述统计假设下控制点积分数的尺度，改善训练时的数值表现。

这并不意味着缩放操作能够在所有情况下保证训练稳定，而是缩放点积注意力中的一种重要数值设计。

双向注意力与因果注意力 根据任务需求，自注意力还可以采用不同的访问范围。双向自注意力（Bidirectional Self-Attention）通常允许每个位置访问序列中其他允许参与计算的位置，包括前面和后面的 Token。这种形式适用于需要综合整段输入信息的任务，例如文本编码和某些视觉表示学习。因果自注意力（Causal Self-Attention）则要求当前位置不能访问未来 Token，只能利用当前及此前允许访问的位置。

在自回归语言模型中，因果注意力能够防止模型在训练时直接读取需要预测的未来内容。通常可以通过注意力掩码实现这一约束：

$$A = \mathrm{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right) \tag{1.15}$$

其中，M 是加性注意力掩码。对于不允许访问的位置，可以将对应掩码值设为负无穷，使其在 Softmax 后获得零权重。对于允许访问的位置，掩码值通常设为零。这里的因果掩码与上一节介绍的自回归序列建模直接对应。模型可以在训练时并行计算多个位置，同时保证每个位置不使用未来 Token 的信息。

1.4 研究问题与阅读路线

一个可检验的研究问题应当同时给出任务、变量和判据。例如，研究视觉压缩时，应固定编码器、数据、问题与解码设置，改变压缩位置或预算，分别检查计数、文字和空间关系等能力，再记录编码、预填充和解码的成本。只报告一个平均分，无法说明哪些信息被保留。

阅读架构论文时，可按四个问题作笔记：输入各有什么形状；信息在哪里相遇；哪些参数从哪项损失获得梯度；哪个实验能够排除更简单的解释。后文的习题、消融和综合案例都使用这套提问方式。

习题与讨论

1. 为图像问答、跨模态检索和语言条件动作分别写出观测、条件、输出与监督来源。指出哪些变量在测试时不可获得。
2. 说明“同一隐藏维度”“同一语义空间”“联合表示”三个说法各自约束了什么，并给出互不等价的理由。
3. 若模型在无图像输入时仍能答对某类问题，应怎样设计对照，区分数据中的语言先验与视觉信息的作用？
4. 画出一个视觉问答模型的训练图，标出损失位置、可训练参数和必须保留的梯度路径。

第二章　视觉表示与信息粒度

视觉编码器决定后续模型能够接触哪些空间信息。全局语义、局部文字、对象数量和精确坐标对表示的要求不同；较强的图像分类特征不必然适合所有视觉语言任务。讨论编码器时，应先确定任务所需粒度，再检查分辨率、下采样、位置与输出层。

学习目标　计算视觉序列长度并追踪张量形状；解释位置与尺度的作用；通过受控实验选择编码器、分辨率和输出粒度。

2.1　图像接口与预处理

训练与推理必须使用相容的颜色、尺寸和归一化约定。若预处理改变了图像几何关系，还应保存从处理后坐标到原始坐标的映射。文档问答和区域定位中的错误，常从这一步开始，而非来自语言解码。

2.1.1　图像预处理、归一化与数据增强

预处理决定编码器实际看到的输入，应与权重对应的处理器一起记录。解码后先核对颜色通道、方向和数据排列，再进行几何变换与数值处理。RGB 与 BGR 调换可能不改变形状，却改变了各通道的含义；把灰度重复到三个通道也不会产生新的颜色证据。

尺寸与几何　直接缩放、保持比例后填充、裁剪和切块提供不同视野。直接改变宽高比例会改变几何关系；裁剪可能去掉问题需要的对象；填充区域应在位置或有效范围中标记。插值能改变采样网格，不能恢复原图没有的真实细节。需要输出坐标的任务，必须保存这些操作的逆向映射。

数值范围与标准化　先按配置将像素转换为浮点范围，再按通道处理，例如

$$\widehat{x}_c = \frac{x'_c - \mu_c}{\sigma_c}, \qquad \sigma_c > 0. \tag{2.1}$$

x'_c 是范围转换后的值，μ_c, σ_c 是对应配置的通道常数。它们不是全部视觉模型通用的参数。范围缩放、固定常数标准化与网络内部的 LayerNorm 是不同操作；重复执行处理器已完成的归一化会改变输入分布。

增强与监督一致性　训练增强应保留或同步更新监督。裁剪后，区域坐标和可见对象可能改变；水平翻转会改变“左边”与“右边”；文档旋转或颜色扰动也可能改变任务证据。是否采用增强，应由任务与标签的等变关系决定，不能只看图像是否仍然自然。

训练中的随机变换与测试处理应分别记录。比较模型时，如果预处理同时改变，性能差异便不只来自编码器。固定权重、用同一组原图对照处理前后视野、坐标和数值范围，是成本较低而有判别力的诊断。

2.2 从图像块到视觉序列

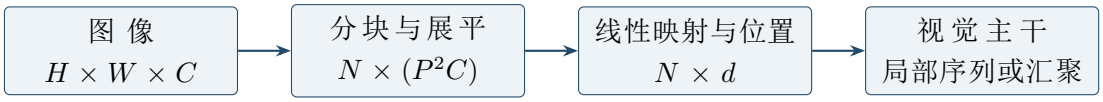


图 2.1 ViT 输入接口。$N = HW/P^2$ 要求尺寸可被块边长整除；填充、重叠或动态分块会改变计数。

2.2.1 Patch Embedding

设图像 $I \in \mathbb{R}^{H\times W\times C}$，以边长 P 的非重叠方块划分。若 H, W 均可被 P 整除，块数为 $N = (H/P)(W/P)$。每块展平为 $p_i \in \mathbb{R}^{P^2C}$，经同一个线性映射得到

$$z_i = p_i E + b, \qquad E \in \mathbb{R}^{P^2C\times d}, \quad b \in \mathbb{R}^d. \tag{2.2}$$

堆叠后 $Z \in \mathbb{R}^{N\times d}$。$P^2C$ 是原始块维度，d 是模型隐藏维度，二者无需相等。224×224 的 RGB 图像按 16×16 分块时，$N = 196$、原始块维度为 768；隐藏维度是否也取 768 是另一项配置。

以核大小和步幅均为 P 的卷积，可以实现同类的非重叠块映射。步幅小于核大小时会出现重叠，长度与边界公式随填充改变。块是规则采样单元，可能跨越对象边界，不应当作天然的对象分割。

减小块大小可提高空间采样密度，也增加序列和后续交互成本。较粗块中的细节能否保留，取决于映射和任务；不能只凭块数断言每个细小对象都已被编码。

2.2.2 视觉 Token 序列与位置编码

二维图像如何变成一维序列 经过 Patch Embedding 后，一张图像可以表示为：

$$Z \in \mathbb{R}^{N\times D} \tag{2.3}$$

其中，每个位置对应一个图像 Patch 的特征向量。例如，14×14 的 Patch 网格可以按照从左到右、从上到下的顺序展平为：$[z_1, z_2, \ldots, z_{196}]$，这是一种常见的行优先（Row-major）排列方式。

但这里存在一个问题：图像原本具有明确的二维空间结构，而单纯将所有 Patch 特征排列成序列，并不能充分告诉模型不同 Patch 原来位于图像中的哪个位置。例如，一个来自图像左上角的 Patch 和一个来自右下角的 Patch，可能包含相似的纹理。如

果模型不知道它们在原始图像中的位置，就可能难以正确理解场景布局。因此，视觉 Token 序列通常还需要加入位置相关信息。

可学习位置嵌入 经典 ViT[1] 通常采用可学习的位置嵌入。可以将每个序列位置对应的位置向量作为模型参数，在训练过程中进行优化。例如，对于 196 个 Patch，可以学习：

$$E_{\text{pos}} \in \mathbb{R}^{196\times D} \tag{2.4}$$

的位置嵌入矩阵。如果额外加入一个分类 Token，那么位置嵌入数量也相应增加。虽然图像原本具有二维结构，但经典 ViT 可以直接为展平后的一维序列位置学习嵌入。这种方法并不要求显式地将横坐标与纵坐标分别编码。不过，使用一维序列位置嵌入并不意味着模型天然具备所有二维几何关系。模型仍然需要通过数据和训练目标学习位置之间的空间联系。

二维位置表示与相对位置机制 随着视觉 Transformer 的发展，研究者提出了更多适合视觉数据的位置表示方法。例如，可以分别描述 Patch 在高度和宽度方向上的位置，再将两类信息结合。也可以使用相对位置机制，让注意力计算直接考虑两个 Patch 之间的空间距离或相对位置关系。例如，Patch A 位于 Patch B 的左侧，与 Patch A 位于 Patch B 的上方，是两种不同的二维空间关系。

一些视觉 Transformer 通过相对位置偏置、二维位置编码或其他方式对这类关系进行建模。后续的视觉与多模态 Transformer 还可能引入旋转位置编码及其二维扩展形式。这些设计的共同目的，是使模型在处理视觉 Token 时，能够更加有效地利用图像的空间结构。不过，不同位置编码方法的具体实现和适用条件可能不同。

本节主要需要理解：**位置表示是将二维图像结构引入 Transformer 的重要机制，但它并不直接等同于模型已经掌握了真实三维空间或对象之间的全部几何关系。**

CLS Token 在经典 ViT 图像分类架构中，通常会在 Patch Token 序列前额外加入一个可学习的分类 Token，记为：z_{cls} 也常被称为 CLS Token 或 Class Token。其基本思路，是为整个图像提供一个专门用于聚合全局信息的序列位置。例如，原本的 Patch Token 序列为：$[z_1, z_2, \ldots, z_{196}]$，加入 CLS Token 后：

$$[z_{\text{cls}}, z_1, z_2, \ldots, z_{196}] \tag{2.5}$$

序列长度变为：197CLS Token 可以通过 Transformer 中的自注意力机制，与其他视觉 Token 进行信息交互。经过多层 Transformer 计算后，模型可以使用最终 CLS 位置的隐藏状态作为图像级特征：

$$h_{\text{cls}} \in \mathbb{R}^{D} \tag{2.6}$$

再通过分类头预测图像类别。例如：

$$\hat{y} = W_{\text{head}} h_{\text{cls}} + b \tag{2.7}$$

CLS Token 通常是一个可学习向量，而不是从图像中截取的某个特殊区域。它也不意味着 CLS Token 天然包含完整的图像信息，其全局表示能力需要通过训练获得。此外，并不是所有 ViT 都必须使用 CLS Token。一些架构采用平均池化、注意力池化或其他聚合方式得到全局图像特征。

全局视觉表示与 Patch 级表示 ViT 可以产生不同粒度的视觉特征。例如，经过 Transformer Encoder 后，输出可以写为：

$$H = [h_{\text{cls}}, h_1, \ldots, h_N] \tag{2.8}$$

其中：

- h_{cls} 可以用作图像级表示。
- h_i 是第 i 个 Patch 对应的上下文特征。

如果任务是图像分类或图像级检索，可以使用 CLS 特征或其他池化后的全局特征。如果任务涉及目标定位、区域分析或密集预测，则可能需要保留 Patch 级特征。例如：

$$H_{\text{patch}} \in \mathbb{R}^{N \times D} \tag{2.9}$$

可以根据原始 Patch 网格重新组织为：

$$F \in \mathbb{R}^{D \times H_p \times W_p} \tag{2.10}$$

其中：$H_p = H/P, \qquad W_p = W/P$，这使视觉 Token 可以重新形成具有二维空间结构的特征图。不过，恢复 Patch 网格并不意味着已经恢复原始图像的全部像素细节。如果下游任务需要高精度分割或边界定位，可能还需要更高分辨率的特征、上采样模块或其他空间恢复机制。

不同图像分辨率下的位置处理 经典 ViT 的位置嵌入可能是按照特定训练分辨率学习的。例如，模型训练时使用 224×224 图像和 16×16 Patch，对应 14×14 的 Patch 网格。如果推理时将图像改为 384×384，那么 Patch 网格变为：24×24

Patch 数量也从 196 增加到：576，因此，原本按照 196 个 Patch 学习的位置嵌入，不能简单地按原样与 576 个 Patch 特征相加。一种常见处理方式，是将原始二维 Patch 位置网格进行适当插值，再得到与新 Patch 网格对应的位置嵌入。例如，可以保留 CLS Token 对应的位置参数，对其余 Patch 位置嵌入进行二维插值。

但位置嵌入能够完成插值，并不意味着模型一定能够在新分辨率下保持原有性能。模型是否能够适应更高分辨率，还与预训练条件、视觉 Token 数量和后续网络结构有

关。因此，实际使用预训练 ViT 时，应遵循对应模型的输入分辨率和位置编码处理策略。

2.2.3 自注意力与全局视觉建模

视觉自注意力的基本计算 假设视觉 Token 序列为：

$$X \in \mathbb{R}^{N \times D} \tag{2.11}$$

通过三个可学习投影矩阵生成：$Q = XW_Q$，$K = XW_K$，$V = XW_V$，随后，使用缩放点积注意力计算视觉 Token 之间的信息交互：

$$\text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{2.12}$$

该公式与第一章介绍的 Transformer 注意力机制一致。区别主要在于输入信息的来源。在语言模型中，Token 通常对应文本切分单元。在经典 ViT 中，Token 通常对应经过投影的图像 Patch。因此，每个视觉 Token 可以通过 Query 与其他 Token 的 Key 计算匹配分数，再根据注意力权重聚合相应的 Value 信息。

对于标准全连接自注意力，一个 Patch 可以在单层中直接与图像中其他允许访问的 Patch 建立联系。这为模型利用较大范围的视觉上下文提供了计算基础。

视觉 Transformer Block 经典 ViT 通常通过堆叠多个 Transformer Encoder Block 构建深层视觉特征。每个 Block 通常包含：

- 多头自注意力。
- 前馈网络（FFN 或 MLP）。
- 残差连接。
- 归一化。

对于一个典型的 Pre-LN Block，可以表示为：

$$Y = X + \text{MHA}(\text{LN}(X)) \tag{2.13}$$

$$Z = Y + \text{MLP}(\text{LN}(Y)) \tag{2.14}$$

其中：

- MHA 负责不同视觉 Token 之间的信息交互。
- MLP 负责对每个 Token 的特征进行非线性变换。
- 残差连接与归一化有助于网络优化和信息传播。

上式只是常见的 Transformer Block 形式，不意味着所有视觉 Transformer 都采用完全相同的归一化与残差顺序。ViT 经过多个 Block 后，可以得到更加丰富的上下文

视觉表示。对于采用固定 Patch 网格的经典 ViT，Transformer Block 通常保持 Token 序列长度不变。

但在分层视觉 Transformer 中，也可以通过 Patch Merging 等机制逐步减少 Token 数量，并改变通道维度。

ViT 的注意力与图像空间结构 虽然 ViT 将图像表示为一维 Token 序列，但这些 Token 最初来自二维 Patch 网格。因此，视觉自注意力需要与位置机制共同作用，才能有效利用图像的空间关系。例如，模型需要识别两个 Patch 是否相邻、一个对象的不同局部区域如何组成整体，以及不同对象之间具有怎样的相对位置。这些信息可能通过 Patch 内容、位置编码以及不同层的注意力与特征变换共同体现。

但不能将注意力矩阵简单理解为真实的物体分割结果或空间关系图。例如，某个注意力头在一张图像中对汽车区域分配较高权重，不一定意味着它已经准确识别了汽车边界，更不能直接证明模型建立了完整的三维对象表示。因此，注意力可视化可以帮助观察模型的部分信息交互行为，但通常不能独立作为解释模型决策机制的充分证据。

全局自注意力的计算成本 ViT 的一个重要限制，是标准全连接自注意力的计算成本与 Token 数量有关。对于包含 N 个视觉 Token、隐藏维度为 D 的序列，标准密集注意力的主要 Token 交互计算复杂度可以表示为：$O(N^2D)$，如果模型还包含线性投影和 FFN，则还需要考虑相应的特征变换成本。对于固定 Patch 大小 P，有：

$$N = \frac{HW}{P^2} \tag{2.15}$$

因此，当图像的宽度和高度同时增加一倍时，在不改变 Patch 大小的情况下，视觉 Token 数量会变为原来的 4 倍。标准全连接注意力的 Query-Key 分数矩阵元素数量，则会增加到原来的：$4^2 = 16$ 倍。例如：可以看到，随着图像分辨率提高，视觉 Token 数量增长很快。这里只分析 Patch Token，没有计入可能存在的 CLS Token 等特殊位置。

因此，在高分辨率图像和长视频任务中，直接采用全连接注意力可能带来较高的计算与显存开销。这也是后续视觉 Token 压缩、局部注意力和分层结构研究的重要动机。

局部窗口注意力与分层 ViT 为了改善高分辨率图像处理效率，一些视觉 Transformer 不再要求每一层中的每个 Token 都与全部其他 Token 执行注意力计算。例如，Swin Transformer 使用局部窗口注意力，并通过窗口移动机制建立不同窗口之间的联系。这种设计使模型能够在局部范围内计算注意力，再通过层间的信息交换扩大有效的交互范围。

同时，分层结构还可以通过 Patch Merging 等机制，逐步降低空间分辨率并增加通道维度，形成多尺度视觉特征。因此，并不是所有 ViT 都必须采用全局、等分辨率的 Token 交互方式。需要区分：

经典 ViT：通常使用固定 Patch 序列和全连接自注意力。

局部或窗口化 ViT：可以限制单层注意力范围，以降低计算成本。

分层 ViT：可以在不同网络阶段改变 Token 数量和特征分辨率。

这些路线在全局信息交互、局部结构建模、多尺度表示和计算效率方面具有不同取舍。具体的多尺度视觉特征将在 3.4 节进一步讨论。

2.3 空间结构与特征粒度

2.3.1 全局特征与局部特征

视觉特征的粒度 视觉特征的粒度（Granularity），主要描述一个特征表示对应多大范围的信息，或者具有多精细的描述能力。例如，一个向量可以描述整张图像，也可以描述图像中的某个对象、局部区域，甚至某个像素附近的视觉信息。根据表示范围，可以将视觉特征概念性地划分为：

图像级特征（Image-level Features）：主要描述整张图像的整体内容。

对象级特征（Object-level Features）：主要描述某个目标实例或语义对象。

区域级特征（Region-level Features）：主要描述图像中的某个局部区域。

Patch 级特征（Patch-level Features）：对应通过图像分块获得的局部视觉单元。

像素级特征（Pixel-level Features）：对应原始图像或特定输出网格中的细粒度空间位置。

这些概念并不是互斥的。一个对象可能覆盖多个 Patch，一个区域可能包含多个对象，而某个 Patch 特征也可能通过上下文建模包含其覆盖区域之外的信息。因此，特征粒度既涉及表示与图像空间的对应方式，也涉及模型如何组织和聚合视觉信息。

全局视觉特征 全局视觉特征（Global Visual Features）通常用于概括整张图像的主要内容。例如，对于一张公园照片，全局特征可能包含与草地、树木、人物和整体场景相关的信息。在图像分类任务中，模型通常希望利用全局特征判断图像的主要类别。在图像检索任务中，也可以将不同图像映射为全局向量，再通过向量相似度比较图像内容。假设视觉编码器输出空间特征图：

$$F \in \mathbb{R}^{C\times H'\times W'} \tag{2.16}$$

可以通过全局平均池化得到：

$$g_c = \frac{1}{H'W'}\sum_{u=1}^{H'}\sum_{v=1}^{W'} F_{c,u,v} \tag{2.17}$$

最终得到：

$$\mathbf{g} \in \mathbb{R}^{C} \tag{2.18}$$

其中，$\mathbf{g}$ 可以用作图像级表示。对于 ViT，也可以使用 CLS Token、平均池化或其

他聚合机制获得全局图像特征。因此，全局特征并不对应某一种固定的网络结构，而是根据整个图像的信息生成的整体表示。

局部视觉特征 局部视觉特征（Local Visual Features）主要描述图像中特定空间区域的信息。例如，一张街道图像可以包含多个局部区域：某一区域包含车辆，另一区域包含行人，还有区域包含交通标志或道路边界。如果希望模型分别识别这些内容，就需要保留与不同空间位置相关的视觉表示。在 CNN 中，局部特征通常表现为具有空间网格结构的特征图：

$$F \in \mathbb{R}^{C\times H'\times W'} \tag{2.19}$$

其中，每个空间位置对应一个 C 维特征向量。在 ViT 中，局部特征通常可以表现为 Patch Token：

$$Z \in \mathbb{R}^{N\times D} \tag{2.20}$$

如果这些 Patch 按照规则网格排列，并且保留了对应关系，那么可以进一步将 Token 序列重新组织为空间特征图。CNN 特征图上的某个位置，并不只包含该位置单个像素的信息，而是对应一定范围的感受野。同样，ViT 的某个 Patch Token 在经过多层自注意力后，也可能聚合来自其他 Patch 的上下文信息。因此，局部特征的空间对应关系，并不等于它的信息来源被严格限制在某个固定局部区域中。

Patch 级特征与对象级特征 Patch 级特征与对象级特征是两个容易混淆的概念。在经典 ViT 中，Patch 通常通过固定网格划分得到。例如，将图像切分为 16×16 的非重叠图像块。这种划分方式不依赖真实对象边界。例如，一个 Patch 可能覆盖一部分汽车和一部分道路，而一辆完整汽车也可能跨越多个 Patch。因此，Patch Token 并不天然等同于对象 Token。

对象级特征则通常需要进一步根据目标位置、区域候选或学习到的查询表示，将属于同一对象的信息进行聚合。例如，在目标检测中，可以根据预测的目标框提取相应区域特征，也可以利用对象查询机制获得与不同目标相关的表示。对象级特征同样不是绝对固定的概念。不同检测或分割架构可能采用不同的对象表示和聚合机制。从教学角度，可以将两者的区别概括为：Patch 级表示主要按照预设空间划分组织信息。

对象级表示则希望围绕具有一定语义或实例意义的目标组织信息。这一差异对视觉语言模型非常重要，因为自然语言描述往往以对象、属性和关系为基本组成，而不直接以规则图像 Patch 为单位。

不同视觉任务需要不同粒度的特征 不同视觉任务对特征粒度的需求存在明显差异。例如：

图像分类通常关注整张图像的主要语义，可以使用全局特征。

图像检索通常需要构建适合相似度比较的图像级表示，但细粒度检索可能进一步需要局部信息。

目标检测需要识别多个对象并预测其位置，因此通常需要空间特征和对象级表示。

语义分割需要对图像中不同位置进行类别预测，因此需要保留或重建空间密集的特征。

视觉问答的需求更加多样。有些问题只需要整体场景语义，有些问题则需要识别小目标、读取文字或判断多个对象之间的关系。

例如：问题 A：“这张图片主要展示了什么场景？”模型可能主要利用全局语义信息。问题 B：“画面右下角的标志是什么？”模型需要利用对应区域的局部细节。问题 C：“红色车辆左边有几名行人？”模型还需要识别多个对象、建立空间关联并执行数量判断。因此，视觉任务的复杂性不仅体现在需要识别多少对象，也体现在需要使用什么粒度的信息完成判断。

2.3.2 多尺度特征与层级特征

多尺度与层级特征并不完全相同 多尺度特征和层级特征（Hierarchical Features）经常同时出现，但两者并不完全等价。

多尺度特征主要强调表示所对应的空间分辨率、采样尺度或感受范围不同。

层级特征则主要强调特征经过网络不同层次的变换，可能具有不同的抽象程度和语义性质。

例如，在典型 CNN 中，浅层特征通常具有较高空间分辨率，深层特征通常具有较低空间分辨率。因此，它们既可能属于不同尺度，也可能具有不同的语义层次。但是，二者不必然绑定。例如，经典 ViT 可以在多个 Transformer Block 中保持相同的 Patch 数量和空间网格大小。此时，不同层的隐藏状态可能具有不同的语义性质，却不一定具有不同的空间分辨率。

反过来，通过对同一张特征图执行不同程度的插值或池化，可以得到不同空间分辨率的表示，但这并不意味着它们已经学习到了不同层级的语义知识。因此，在分析视觉模型时，需要分别关注特征的空间尺度与表示抽象程度。

特征金字塔的机制 特征金字塔（Feature Pyramid）是一类组织多尺度视觉特征的重要方法。它的基本思想，是将图像或网络中的不同尺度表示联系起来，使模型能够利用不同空间范围的信息。早期方法可以通过构建不同图像分辨率的图像金字塔，在多个尺度上分别进行特征提取。而在深度学习中，也可以利用神经网络不同层输出的特征，构建多尺度表示。

例如，特征金字塔网络（Feature Pyramid Network，FPN）通过自顶向下路径和横向连接等机制，将较深层的语义信息与较浅层的空间信息结合起来。在一种简化的形式中，可以表示为：

$$P_l = \mathrm{Conv}_{1\times1}(C_l) + \mathrm{Upsample}(P_{l+1}) \tag{2.21}$$

其中：

- C_l 表示主干网络第 l 层的特征。
- P_{l+1} 表示更高层特征金字塔的输出。
- Upsample 表示将高层特征调整到相应空间尺寸。

实际 FPN 通常还包含额外的卷积和特征处理操作。这一结构体现了一个重要思想：通过跨层连接，将高层语义信息传递到较高分辨率的空间特征中。但不同层特征的简单相加或拼接，并不保证一定获得更好的融合效果。特征尺度、通道分布和训练目标都可能影响实际结果。

为什么小目标通常需要较高分辨率特征 假设一张图像中的某个远处交通标志，在原始图像中只占据 12×12 个像素。如果视觉编码器采用 16 倍空间下采样，那么该目标在相应特征网格中可能只对应很少的位置，甚至不足以形成一个完整的独立特征单元。此时，如果模型没有通过其他机制保留相关细节，就可能难以准确识别该目标。相反，较高分辨率的特征图能够保留更多空间采样位置，通常更有利于小目标定位和精细边界预测。

但高分辨率特征也意味着更多存储与计算开销。因此，在目标检测和分割中，需要根据目标尺度和计算预算合理选择特征分辨率。同时，也不能简单认为提高分辨率就一定能够解决小目标识别问题。图像清晰度、目标对比度、预训练数据及检测解码器等因素同样重要。

ViT 中的层级特征 经典 ViT 通常将输入图像转换为固定数量的 Patch Token，并在多个 Transformer Block 中保持序列长度不变。例如，对于 224×224 图像和 16×16 Patch：$N = 196$，如果模型包含 12 个 Transformer Block，那么每一层通常都可以输出：

$$H_l \in \mathbb{R}^{196\times D} \tag{2.22}$$

形式的 Patch 特征，暂不计入可能存在的特殊 Token。虽然不同层的 Token 数量相同，但经过不断的信息交互和非线性变换，其表示内容可能发生变化。例如，部分较浅层可能更关注局部外观和结构，而部分较深层可能更加适合表达对象及场景语义。这种变化并不一定严格遵循浅层只表示纹理、深层只表示语义的固定模式。

因此，ViT 也具有层级特征，只是经典 ViT 的层级变化主要表现为特征内容和抽象程度的变化，而不是每一层都改变空间分辨率。

分层视觉 Transformer 为了更加高效地处理高分辨率图像，并构建多尺度表示，研究者提出了分层视觉 Transformer。例如，Swin Transformer 通过局部窗口注意力和 Patch Merging 等机制，在不同网络阶段改变视觉 Token 的数量与通道维度。通过逐步合并相邻 Token，模型可以降低空间分辨率，并建立不同尺度的特征表示。这种设计使视觉 Transformer 更容易与检测、分割等需要多尺度特征的任务结合。

此外，也存在通过卷积式特征提取模块、池化或其他 Token 聚合机制建立层级结构的视觉模型。因此，多尺度和层级表示并不是 CNN 独有的概念，现代视觉 Transformer 同样可以采用相应设计。

2.3.3 空间信息与语义信息

空间细节与语义抽象之间的权衡 在视觉编码过程中，模型通常需要对局部信息进行聚合，形成更丰富的上下文和语义表示。但随着特征聚合或空间降采样，某些细粒度位置信息可能被弱化。例如，一个 CNN 可以通过连续下采样，将原始图像逐渐转换为低分辨率特征图。在较深的网络层中，特征可能包含更广范围的上下文，但单个空间位置对应的原始图像范围也可能更大。因此，深层特征虽然可能更适合图像分类，却未必能够直接给出精确的像素级对象边界。

在 ViT 中，如果使用较大的 Patch，每个视觉 Token 也会覆盖较大的图像区域。经过自注意力后，Token 特征可能包含全局语义信息，但其与原始像素之间的对应关系仍然受到 Patch 粒度和网络结构影响。因此，视觉表示需要在语义抽象与空间定位能力之间进行权衡。

语义强不等于空间精确 考虑两个视觉任务。任务 A:“判断图像中是否存在一辆汽车。”任务 B:“准确标出图像中每辆汽车的像素级边界。”任务 A 可以主要依赖图像级或对象级语义特征。任务 B 则需要更加精细的空间定位和区域边界信息。一个视觉编码器即使能够准确完成任务 A，也不能据此断言它具备完成任务 B 所需的表示能力。例如，经过全局平均池化得到的特征可以在一定程度上总结图像内容，但池化之后不再直接保留各个空间位置与特征之间的一一对应关系。

如果希望进一步进行精确的空间预测，通常需要访问池化之前的空间特征，或者通过其他结构建立定位能力。因此，评价视觉特征时，需要区分语义判别能力与空间定位能力。

空间特征如何支持密集预测 对于语义分割任务，模型需要为不同像素或空间位置生成预测。假设 Backbone 输出：

$$F \in \mathbb{R}^{C\times H'\times W'} \tag{2.23}$$

其中，H' 和 W' 是特征图的空间尺寸。模型可以进一步通过卷积、上采样或其他解码模块，生成与原始图像空间对应的预测结果：

$$Y \in \mathbb{R}^{K\times H\times W} \tag{2.24}$$

其中，K 表示语义类别数量。输出中的每个空间位置，对应不同类别的预测分数。空间上采样本身并不能自动恢复所有已丢失的图像细节。例如，如果编码阶段已经难以区分某个小目标与背景，仅通过简单插值放大特征图，并不能保证恢复正确边界。因此，密集预测模型通常需要结合较高分辨率特征、跨层融合或其他细节恢复机制。

空间位置、几何关系与真实场景理解 二维空间特征能够帮助模型判断图像中的对象位置和相对布局，但复杂场景理解还可能涉及真实世界的几何关系。例如，当用户询问：“这两辆汽车之间的实际距离是多少？”仅凭图像中的像素距离，通常不能直接得到真实世界中的物理距离。这是因为透视投影会受到相机焦距、观察距离、姿态和其

他成像因素影响。因此，对于深度估计、三维定位和空间测量等任务，模型通常需要利用额外的几何约束、相机标定信息、多视角观测或通过数据学习得到的先验。

这说明，视觉语义理解与几何空间理解虽然相关，但并不等同。一个能够正确描述场景的视觉语言模型，也不一定能够可靠完成精确的三维测量或几何推断。

2.4 高分辨率与坐标恢复

算例：分辨率预算

对边长 336、块边长 14 的正方形图像，非重叠分块产生 $24^2 = 576$ 个块。若两边都增大一倍，块数变为 2304；标准全连接注意力的块间配对数增为原来的 16 倍。这个比例只描述该注意力项，不能直接当作端到端延迟的倍率。切块后还要把局部坐标还原到整图；否则模型即使找到了文字或对象，也可能输出错误位置。

2.4.1 图像缩放、裁剪与切块

直接缩放 直接缩放（Resize）是最简单的图像尺寸处理方法。例如，将一张 1024×768 的图像缩放为 224×224，再输入视觉编码器。其优点是实现简单，输出尺寸固定，便于批量计算。但是，如果原始图像的宽高比与目标尺寸不同，直接缩放可能引起几何变形。例如，一个原本接近圆形的对象，可能因为非等比例缩放而表现为椭圆。

此外，缩小图像时需要重新采样像素，原始图像中的高频细节可能被削弱甚至消失。对于普通图像分类任务，这种损失有时可以接受；但对于细小文字识别、小目标检测和精细空间分析，则可能影响模型表现。因此，直接缩放适用于能够容忍一定细节损失的任务，但并不适合所有高分辨率视觉输入。

等比例缩放与填充 为了避免图像几何形变，可以采用保持宽高比的缩放方式。例如，对于一张 1024×768 的图像，如果目标画布为 224×224，可以先将其等比例缩小到 224×168，再通过填充形成 224×224 的输入。这种方式通常称为 Letterbox。

其基本思想是保留原始图像的相对形状，再通过填充满足网络输入尺寸要求。相比直接拉伸，Letterbox 可以减少几何形变，但也可能引入额外的空白区域。对于采用规则 Patch 划分的视觉编码器，填充区域可能占据部分 Patch 或视觉 Token，从而产生不包含真实场景信息的计算开销。因此，保持宽高比虽然具有优势，但还需要考虑填充比例和 Token 利用效率。对于非常狭长的文档、网页或全景图像，这一问题尤其明显。

裁剪与局部细节保留 裁剪（Crop）是指从原始图像中提取某个局部区域。例如，可以从一张 1024×1024 的图像中截取一个 256×256 的区域，再将该区域输入视觉编码器。与直接缩小整张图像相比，裁剪能够以更高的局部采样精度保留相应区域中的视觉细节。

例如，对于较小的交通标志，如果先将整张高分辨率图像缩小到 224×224，标志

可能变得难以识别。但如果先截取包含交通标志的局部区域，再使用较高的局部输入分辨率，模型可能获得更加清晰的目标特征。因此，裁剪可以帮助模型集中处理局部细节。不过，裁剪也存在明显局限：它可能移除区域之外的重要上下文。例如，如果只保留一个交通标志的局部图像，模型可能难以判断该标志所处的道路环境。

因此，裁剪与全局信息保留之间存在权衡。

切块与滑动窗口 对于分辨率较高的图像，可以将整张图像划分为多个较小的子图像。这种处理方式通常称为切块（Tiling）或基于滑动窗口的处理。例如，一张 448×448 的图像，可以被划分为四张 224×224 的子图像。每张子图像分别经过视觉编码器，随后再将它们的特征进行组合。需要特别区分两种不同层次的图像切分：

图像级切块（Image Tiling）：将原始高分辨率图像划分为多个子图像，每个子图像通常独立经过视觉编码器。

Patch Embedding：在视觉编码器内部，将某张输入图像划分为 Patch，并将 Patch 映射为视觉 Token。

例如，448×448 的图像可以先被切成四张 224×224 子图像，每张子图像再通过 16×16 的 Patch Embedding 产生 196 个视觉 Token。因此，图像级切块与 ViT 内部的 Patch 划分并不是同一层次的操作。

重叠切块 如果多个子图像完全不重叠，某些对象可能恰好位于切块边界。例如，一辆汽车可能被分成左右两个局部区域，使每个子图像都只包含不完整的对象。为了减轻这种问题，可以在相邻切块之间设置一定的重叠区域。假设切块尺寸为 P，滑动步幅为 S。当：$S < P$ 时，相邻切块之间通常存在重叠。重叠切块能够增加目标跨边界时被完整观察到的机会，但也会导致部分图像区域被重复处理，从而增加计算量。

此外，如果不同切块分别产生目标检测结果，还可能需要对重复目标进行合并或去重。因此，重叠比例需要根据目标尺度、任务要求和计算预算合理设置。

切块后的空间坐标如何恢复 图像切块之后，每张子图像具有自己的局部坐标系统。例如，一个目标在局部子图像中的位置为：(u, v)，如果该切块在原始图像中的左上角位置为：(x_0, y_0) 且切块没有发生额外的缩放，那么目标在原图中的坐标为：$x = x_0 + u$，$y = y_0 + v$，如果切块还经过了缩放，则需要进一步根据缩放比例还原坐标。例如，可以写为：$x = x_0 + s_x u$，$y = y_0 + s_y v$，

其中，s_x 和 s_y 分别表示从处理后的局部坐标恢复到原图坐标的尺度系数。因此，在检测、定位、分割等任务中，高分辨率切块不仅需要管理视觉内容，还需要维护切块位置、缩放比例和对应的空间变换关系。否则，模型即使正确识别了局部对象，也可能无法准确恢复其在完整图像中的位置。

裁剪与切块的信息局限 裁剪和切块虽然能够保留局部细节，但可能破坏完整的场景上下文。例如，一张图像中存在两个相距较远的对象，它们分别位于不同切块中。如果视觉编码器独立处理每个切块，那么单个子图像的特征不一定包含另一切块的信息。后续模型需要进一步通过跨切块的信息交互，才能建立两者之间的关系。此外，切块顺

序也不天然等同于原图空间关系。

例如，将多个切块的视觉 Token 简单拼接成一维序列，并不保证语言模型能够自动理解这些 Token 在原始图像中的二维位置。因此，合理的切块方案可能还需要提供切块顺序、空间位置编码、坐标信息或全局图像上下文。裁剪与切块的价值在于提高局部细节的可见性，但不能忽略它们对整体空间结构和跨区域关系的影响。

2.4.2 多裁剪与动态分辨率

全局视图与局部视图 多裁剪方案可以同时构建全局视图和局部视图。

全局视图（Global View）通常是整张图像经过适当缩放后的表示，用于保留整体场景和布局。

局部视图（Local Views）则通过多个裁剪或切块保留局部区域的视觉细节。

例如，一张 448×448 的图像可以构建：一个 224×224 的全局缩略图。四个 224×224 的局部切块。随后，将五个视图分别送入视觉编码器。

如果每个 224×224 视图经过 16×16 Patch Embedding 得到 196 个 Patch Token，那么：$N_{\text{global}} = 196$，

$$N_{\text{local}} = 4 \times 196 = 784 \tag{2.25}$$

在未进行额外压缩、忽略特殊 Token 的情况下，组合后的 Token 数量为：

$$N_{\text{total}} = 196 + 784 = 980 \tag{2.26}$$

这种方式能够同时保留整体图像信息和局部细节，但也需要处理更多视觉 Token。五个视图独立经过视觉编码器，与将整张 448×448 图像作为一个输入进行全局自注意力编码，在计算方式和特征内容上并不等价。前者的局部视图通常首先独立编码，而后者可以在视觉编码阶段直接建立跨区域信息联系。

AnyRes 与动态高分辨率处理 在视觉语言大模型中，一类具有代表性的高分辨率处理思路是 AnyRes。例如，LLaVA-NeXT 使用动态高分辨率处理策略，根据输入图像的尺寸和宽高比选择合适的切块网格，使模型能够结合全局视图与局部图像信息。这种方法的核心目的，是减少固定低分辨率输入造成的细节损失，同时尽量复用已有的视觉编码模块。不过，AnyRes 并不是唯一的动态分辨率实现方式。

这里需要特别区分：图像切块式的动态高分辨率处理，并不等同于视觉编码器原生支持任意空间尺寸。在切块式方案中，视觉编码器仍然可以使用固定大小的子图像作为输入。而在原生动态分辨率方案中，视觉编码器本身可以处理不同空间尺寸和不同数量的视觉 Token。

动态分辨率 动态分辨率（Dynamic Resolution）通常是指模型能够根据输入图像的尺寸、宽高比、任务或资源约束，调整实际使用的图像采样尺度和视觉 Token 数量。传统固定分辨率模型通常将输入调整为预设尺寸，例如：224×224 此时，采用固定 Patch

大小的 ViT 通常得到固定数量的视觉 Token。而动态分辨率模型允许不同图像使用不同的处理尺寸。

例如，一张较简单的低分辨率图像可以使用较少 Token；一张包含大量文字或局部细节的高分辨率图像，则可能使用更多 Token。因此，动态分辨率可以使模型更加灵活地适应不同输入。动态分辨率不意味着必须完整保留原始图像中的所有像素，也不意味着图像分辨率可以无限增加。实际系统通常仍然需要设置像素数量、Patch 数量或视觉 Token 数量的上下限。

原生分辨率视觉编码 原生分辨率（Native Resolution）视觉建模强调尽量避免将所有输入图像强制缩放到同一个固定尺寸。例如，NaViT 采用能够处理不同分辨率和宽高比的视觉 Transformer 设计，并通过 Patch n' Pack 等策略，提高不同长度图像序列的训练利用效率。这种思路的优势之一，是能够根据输入图像实际形状组织视觉 Token，减少不必要的固定尺寸变换。

例如，对于一张非常宽的图像，可以保留较接近原始宽高比的 Patch 网格，而不必强制将其变为正方形。不过，当不同样本产生不同数量的视觉 Token 时，模型还需要解决变长序列批处理、位置编码和有效计算组织等问题。因此，原生分辨率处理不仅是取消 Resize，还涉及视觉编码器与训练系统的适配设计。

动态视觉 Token 数量 对于固定 Patch 大小 P 的规则网格，在不考虑额外特殊 Token 的情况下，可以近似表示为：

$$\frac{H\prime}{P} \times \frac{W\prime}{P} \tag{2.27}$$

其中，H' 和 W' 是经过必要处理后的实际输入尺寸。如果使用支持变长视觉序列的模型，那么不同输入图像可以产生不同的视觉 Token 数量。例如，Qwen2-VL 引入了动态分辨率视觉处理机制，使不同尺寸图像能够转换为不同数量的视觉 Token，并结合多模态位置编码处理图像、视频及文本中的位置信息。

实际模型可能进一步使用 Patch 合并、空间压缩或其他机制，因此最终传递给语言模型的 Token 数量，不一定严格等于视觉编码器最初产生的 Patch 数量。这也引出了一个重要区别：

视觉编码器内部处理的 Token 数量，与语言模型实际接收的视觉 Token 数量，可能并不相同。

动态分辨率与位置编码 当图像尺寸发生变化时，视觉 Token 的空间网格也可能变化。例如，14×14 的 Patch 网格与 28×28 的 Patch 网格，包含不同数量的视觉位置。如果模型采用固定长度的可学习绝对位置嵌入，就可能需要插值或其他方式适配新的网格尺寸。一些支持动态分辨率的视觉模型，则会使用适合不同空间尺度和位置网格的位置编码机制。

此外，如果多个局部切块被拼接为统一的视觉 Token 序列，还需要考虑不同切块之间的相对位置关系。因此，动态分辨率不仅要求模型能够处理变长序列，还要求位置机制能够有效描述变化后的二维空间结构。否则，即使图像细节被保留下来，模型

也可能难以准确判断不同视觉区域之间的空间关系。

2.5 编码器的预训练与迁移

预训练目标决定特征受到什么约束，输出层选择决定哪些信息交给下游。比较编码器时，应固定后续连接器和训练协议，分别观察全局语义、密集位置和文字等任务；不能从一个分类结果推断全部迁移能力。

2.5.1 自监督与视觉表征学习

视觉对比学习 视觉对比学习的基本思想，是让模型学习区分哪些图像表示应该接近、哪些表示应该远离。例如，对同一张图像执行两次不同的数据增强：$v_1 = T_1(I)$，$v_2 = T_2(I)$，其中，T_1 和 T_2 表示两种随机增强操作。

随后，通过视觉编码器得到：$z_1 = f_\theta(v_1)$，$z_2 = f_\theta(v_2)$ 由于两个视图来自同一张原始图像，可以将其视为一个正样本对。训练目标通常希望它们的表示具有较高相似度。同时，还可以利用其他图像构造负样本，使不同样本的表示具有更强的可区分性。一种常见的对比损失可以表示为：

$$-\log \frac{\exp(\mathrm{sim}(z, z^+)/\tau)}{\exp(\mathrm{sim}(z, z^+)/\tau) + \sum_j \exp(\mathrm{sim}(z, z_j^-)/\tau)} \tag{2.28}$$

其中：

- z 表示锚点特征。
- z^+ 表示正样本特征。
- z_j^- 表示负样本特征。
- sim 表示相似度函数。
- τ 表示温度参数。

通过这种方式，模型可以在不使用人工类别标签的情况下学习视觉表示。SimCLR、MoCo 等方法是视觉对比学习的重要代表。

对比学习的关键问题 对比学习的效果受到增强策略和正负样本组织方式影响。例如，对同一张图像进行随机裁剪时，不同视图可能只包含部分相同内容。模型需要学习哪些变化不应该改变其主要表示。但是，过强的增强可能破坏原有语义。例如，一张包含多种对象的图像，如果两个随机裁剪区域分别只包含不同对象，将它们直接视为完全对应的正样本，可能产生不适合某些细粒度任务的学习约束。

此外，不同原始图像也可能属于相同语义类别。如果简单将它们视为负样本，可能出现语义相近样本被错误推远的问题。因此，自监督对比学习并不只是设计一个损失函数，还需要合理构造训练视图和样本关系。

掩码图像建模 掩码图像建模（Masked Image Modeling，MIM）是另一类具有代表性的自监督方法。其基本思想是：遮挡图像的一部分，让模型利用剩余可见区域预测被遮挡的信息。例如，将一张图像划分为 196 个 Patch，随机遮挡其中部分 Patch。模型只能观察未被遮挡的区域，再根据上下文预测缺失内容。这种方法与自然语言中的 Masked Language Modeling 具有一定相似性。

但文本 Token 通常对应离散符号，而图像 Patch 包含连续像素或特征，因此视觉掩码任务的目标形式可能有所不同。模型可以预测原始像素，也可以预测经过离散化或特征编码后的目标表示。

MAE：**通过重建学习视觉表示** Masked Autoencoders[2]（MAE）是视觉掩码建模的重要代表。MAE 的一个主要特点，是采用非对称的编码器—解码器结构。其处理过程可以概括为：原始图像 → 随机遮挡大部分 Patch → Encoder 处理可见 Patch → Decoder 重建缺失区域。

例如，在经典 MAE 的代表性配置中，可以随机遮挡约 75% 的图像 Patch。MAE 的 Encoder 通常只处理可见 Patch，而不是将所有被遮挡位置的 Mask Token 一起输入编码器。随后，轻量级 Decoder 利用编码器输出以及相应的掩码位置表示，预测原始图像中被遮挡的内容。一种简化的重建损失可以表示为：

$$\frac{1}{|\mathcal{M}|}\sum_{i\in\mathcal{M}}|\hat{p}_i - p_i|_2^2 \tag{2.29}$$

其中：

- $\mathcal{M}$ 表示被遮挡的 Patch 集合。
- p_i 表示目标 Patch。
- $\hat{p}_i$ 表示对应的重建预测。

实际 MAE 实现可以采用特定的像素标准化和损失处理方式。MAE 的重要意义在于，模型可以通过恢复缺失图像内容，学习图像中的结构与上下文规律。不过，像素重建目标不保证获得的特征在所有语义任务中都最优。其迁移效果仍然取决于预训练和下游任务设置。

DINO：**无标签自蒸馏** DINO[3]（Self-Distillation with No Labels）是视觉自监督学习中的另一条重要技术路线。它采用教师—学生结构，通过不同图像视图之间的预测一致性学习特征。例如，对于同一张图像，可以构造多个不同的裁剪视图。学生网络处理相应视图，并尝试匹配教师网络产生的目标分布。

教师网络通常不直接通过相同方式进行梯度更新，而是利用学生网络参数的指数移动平均（Exponential Moving Average，EMA）进行更新。可以表示为：

$$m\theta_{t-1}^{teacher} + (1-m)\theta_t^{student} \tag{2.30}$$

其中，m 是动量系数。这种机制使教师网络的变化相对平滑，从而为学生网络提

供训练目标。教师—学生结构并不意味着教师必须是一个已经由人工标签训练好的模型。在 DINO 中，教师和学生可以通过自监督训练共同演化。此外，DINO 还需要相应的归一化、温度和防止表示坍塌的训练设计，而不是只依靠简单的特征匹配。

为什么需要防止表示坍塌 在某些不使用显式负样本的自监督方法中，如果只要求不同视图的表示一致，就可能出现一种退化情况：模型将所有输入图像映射为几乎相同的向量。例如：$f(I_1) = f(I_2) = \cdots = f(I_N) = c$，在这种情况下，不同视图之间确实具有很高的一致性，但模型失去了区分不同图像的能力。这种现象称为表示坍塌（Representation Collapse）。

因此，自监督学习不仅需要鼓励相关视图之间的一致性，还需要通过适当的机制避免模型产生无信息的常量表示。不同方法采取的策略可能不同，例如对比学习中的负样本、教师—学生结构中的特定归一化与中心化策略，以及其他特征分布约束。

DINOv2 与 DINOv3：通用视觉表征的发展 DINO 后续发展出了 DINOv2[4] 和 DINOv3[5] 等视觉预训练模型。DINOv2 进一步强调大规模数据整理、模型训练和通用视觉特征的学习，使预训练特征能够用于图像级与像素级等不同任务。其重要特点之一，是在不针对每个下游任务重新训练整个 Backbone 的条件下，仍然能够提供具有较强迁移价值的视觉表示。DINOv3 继续扩展了自监督视觉预训练的规模，并重点改善高质量密集特征的学习与保持。

例如，DINOv3 引入 Gram Anchoring 等训练机制，以缓解长时间大规模训练过程中密集特征质量退化的问题。这些工作说明，视觉自监督学习的研究目标已经不再局限于提高图像分类准确率，还逐渐关注同一个预训练 Backbone 在不同数据分布和密集视觉任务中的可复用性。但不同版本 DINO 模型的具体架构、训练配置与特征提取接口可能不同。

例如，同样称为 DINO 系列模型，也不能假定它们具有完全相同的输出层级、特征维度或空间分辨率。此外，DINO 系列主要强调视觉自监督特征学习，不应直接认为它们原生具备与自然语言文本进行开放词汇相似度比较的能力。

I-JEPA 与特征空间预测 除了像素重建和视图一致性学习，研究者还探索了直接预测视觉特征的方法。I-JEPA（Image-based Joint-Embedding Predictive Architecture）是这一技术路线的代表。其基本思想是：利用图像中某些可见区域的表示，预测其他目标区域在特征空间中的表示。例如：从输入图像中选择一个具有一定上下文范围的区域。随后，使用上下文编码器提取该区域的表示。

预测目标图像块对应的高层特征，而不是直接重建其原始 RGB 像素。这一思路强调在表示空间中建立预测关系，使模型学习能够描述图像结构和语义关联的特征。预测特征不意味着模型天然具有真实世界的因果理解或物理预测能力。其具体表示质量仍然取决于预测目标、特征编码器和训练设置。

2.5.2 视觉编码器的选择与评估

编码器选择要与任务和适配预算对应。整体语义可以先测冻结特征与线性探测；文字、小目标、定位及空间关系还应测密集输出。若允许微调，应固定数据、优化预算和任务头，分别记录可训练模块与实际成本。

两类比较支持不同结论。控制变量实验尽量统一架构、处理和训练设置，用来分析预训练目标或特征设计；系统选型允许各模型使用合理配置，比较给定约束下的最终表现。前者强调机制，后者强调可用系统，不能混用其结论。

记录中至少包括权重与处理器版本、输入分辨率、特征输出层、汇聚方式、任务头、冻结范围和评价协议。分类分数、参数量或单个排行榜都不足以决定密集视觉任务的选择；同样，短预算适配结果也不能代表充分训练后的性能。

一个实用流程是先确定信息粒度，选取几种不同监督目标或结构的编码器，在统一下游接口上做冻结评价，再逐步加入轻量适配与微调。每次扩大自由度，都应通过消融确认收益来自哪个环节。

习题与讨论

1. 推导一般长方形图像的非重叠 Patch 计数，并说明尺寸不能整除时至少两种处理方案如何改变坐标约定。
2. 若切块左上角为 (x_0, y_0)，局部结果已按缩放系数 s 归一到输入像素，写出恢复到原图像素坐标的变换。
3. 为什么全局平均池化不适合直接恢复所有对象的位置？区分表示中有无位置线索与输出是否带坐标。
4. 设计一个固定总视觉 Token 预算的实验，比较整图缩放与全局加局部切块。写出控制变量和分组指标。

文献导读 ViT 用于追踪分块到序列的完整接口；MAE 与 DINOv2 用于比较重建和表征学习的监督差异。先读其目标与评价协议，再比较编码器输出。[1, 2, 4]

第三章　音频与视频的时序表示

音频和视频都具有时间结构，但其局部表示、采样率和监督粒度不同。联合建模应以真实时间为连接条件，同时保留各模态的处理机制。把两个序列截成相同长度，不能建立它们在物理时间上的对应。

学习目标　由采样参数计算时间位置和序列长度；区分连续与离散表示；分析帧采样、音频下采样和摘要对事件证据的影响。

3.1　音频的时间轴与时频接口

以秒和赫兹记录物理时间，以整数索引记录数组位置。STFT 帧、编码器输出步与音频 Token 通常具有不同时间间隔；对齐前需把它们映射回同一时间轴，并记录居中窗口、补零和下采样约定。

3.1.1　波形、采样率、位深与声道

音频波形　波形（Waveform）是音频信号随时间变化的表示形式。在波形图中，横轴通常表示时间，纵轴表示信号幅值。例如，一个人说话时，波形可能在发声阶段表现出明显振动，在停顿阶段接近静音。通过观察波形，可以初步判断声音的持续时间、振幅变化和某些瞬态事件。但仅通过波形通常难以直接判断复杂的声音内容。例如，不同语音音素可能具有复杂而不同的频率结构，但在整体波形中未必容易直接区分。

因此，除了时域波形，还需要利用频域和时频分析方法提取更丰富的声学信息。

常见音频采样率　常见的音频采样率包括：这些只是典型使用习惯，并不表示某种任务只能采用特定采样率。例如，现代语音模型可能支持 16 kHz、24 kHz 或其他采样率，具体取决于训练数据和模型输入规范。

采样率与奈奎斯特频率　采样率决定了离散信号能够表示的频率范围。对于满足带限条件的连续信号，经典采样定理指出，如果采样频率足够高，可以根据离散采样值恢复原始信号。通常要求：$f_s > 2f_{\max}$，其中，$f_{\max}$ 表示信号中的最高频率。采样率一半对应的频率称为奈奎斯特频率（Nyquist Frequency）：

$$f_N = \frac{f_s}{2} \tag{3.1}$$

例如，16 kHz 采样率对应的奈奎斯特频率为：$f_N = 8000$ Hz，这意味着，在理想采样条件下，采样前需要适当限制高于该范围的频率成分，避免其造成混叠。采样率并不直接等于音频所包含的最高实际频率，也不能保证所有低于奈奎斯特频率的内容

都被高质量记录。实际效果还受到麦克风、滤波器和其他采集环节影响。

混叠 混叠（Aliasing）是指采样率不足时，不同连续信号的频率成分可能在离散采样结果中变得无法区分。例如，一个高于采样系统奈奎斯特频率的正弦信号，可能在采样后表现为较低频率的错误成分。因此，音频采集系统通常需要使用适当的抗混叠滤波器（Anti-Aliasing Filter），在采样前限制过高的频率成分。同样，当将高采样率音频转换为低采样率音频时，也应当通过合适的重采样算法进行滤波和重新采样。

例如，将 48 kHz 音频转换为 16 kHz，并不是简单地将文件中的采样率数字改为 16000。如果仅修改采样率元数据而不处理波形，音频播放速度和音高等都会发生变化。因此，在使用预训练音频编码器时，必须确认实际波形数据与其声明的采样率一致。

3.1.2 短时傅里叶变换与频谱

分帧与帧移 在 STFT 中，首先需要确定分析窗口的长度和相邻窗口之间的移动距离。

帧长（Frame Length）描述每次分析使用多少采样点或多长时间的信号。

帧移（Hop Length）描述相邻分析窗口之间移动多少采样点。

例如，假设采样率为 16 kHz。如果帧长为 25 ms，那么对应的采样点数为：$L = 16000 \times 0.025 = 400$，如果帧移为 10 ms，那么：$H = 16000 \times 0.010 = 160$，因此，每次可以使用约 25 ms 的音频进行频谱分析，并每隔 10 ms 产生一个新的分析位置。由于：$H < L$ 相邻窗口之间存在重叠。这种分帧方式能够使音频特征在时间上更加连续。

25 ms 和 10 ms 是语音处理中常见的示例设置，而不是所有音频模型都必须使用的参数。

加窗 将连续信号直接截取为有限长度片段，相当于施加了一个有限时间窗口。如果简单使用矩形窗口，可能产生明显的频谱泄漏（Spectral Leakage）。因此，在 STFT 中通常会对每一帧乘以窗函数。例如，常见的 Hann 窗可以表示为：

$$\frac{1}{2}\left(1 - \cos\frac{2\pi n}{L-1}\right) \tag{3.2}$$

其中：$0 \le n < L$，这是对称 Hann 窗的一种定义。实际 FFT 分析库也可能使用周期性 Hann 窗，其离散定义略有不同。加窗的主要作用是减轻有限时间截断带来的频谱泄漏。但窗函数会影响频谱主瓣宽度和旁瓣特性，因此并不存在对所有任务都最优的窗口。常见的窗口类型包括 Hann、Hamming 和矩形窗等。

STFT **的数学形式** 对于离散信号 (x[n])，可以将第 m 帧的 STFT 定义为：

$$\sum_{n=0}^{L-1} x[n + mH]w[n]e^{-j2\pi kn/N_{\text{FFT}}} \tag{3.3}$$

其中：

- m 表示时间帧编号。

- k 表示频率索引。
- L 表示窗口长度。
- H 表示帧移。
- (w[n]) 表示窗函数。
- N_{FFT} 表示执行离散傅里叶变换时使用的点数。

这里采用从第 (mH) 个采样点开始截取窗口的写法。实际软件实现也可能使用以时间帧中心为基准的对齐方式。STFT 输出通常为复数矩阵，其中包含每个时间帧、每个频率位置的幅度与相位信息。因此，STFT 可以看作同时具有时间维度和频率维度的音频表示。

FFT 点数与频率分辨率 在 STFT 中，N_{FFT} 决定频域离散采样的点数。例如，对于实值音频，如果使用 512 点 FFT，通常只需要保留非负频率部分：

$$N_f = \left\lfloor \frac{512}{2} \right\rfloor + 1 = 257 \tag{3.4}$$

因此，得到 257 个非负频率位置。相邻频率位置的间隔为：

$$\Delta f = \frac{f_s}{N_{\text{FFT}}} \tag{3.5}$$

如果：

$$f_s = 16000, \qquad N_{\text{FFT}} = 512 \tag{3.6}$$

那么：$\Delta f = 31.25$ HzFFT 频率采样间隔不等同于真正区分相近频率的物理分辨能力。例如，对一个只有 25 ms 的窗口执行 512 点 FFT，可能需要进行补零，但补零只是使频谱采样更加密集，并没有增加原始时间窗口内的观测信息。因此，不能认为只要将 FFT 点数设置得足够大，就能够无限提高真实频率分辨能力。

时频分辨率的权衡 STFT 的一个基本特点，是时间分辨率与频率分辨能力之间存在权衡。如果使用较短的分析窗口，模型可以更加精细地观察时间上的快速变化。例如，短窗口可能更适合捕捉某些瞬态声音。但短窗口包含的周期观测较少，通常不利于区分非常接近的频率成分。相反，如果使用较长的窗口，可以观察更多信号周期，通常有助于频率分析，但可能模糊快速变化发生的具体时间。因此：

短窗口：更有利于时间定位，频率分辨能力通常较弱。

长窗口：更有利于频率区分，但时间定位通常较粗。

帧移主要决定相邻时频帧之间的采样密度和计算量，并不直接决定单个窗口本身的频率分辨能力。因此，在设计 STFT 参数时，需要分别考虑帧长、帧移与 FFT 点数的作用。

频谱图 将每个时间帧的频谱沿时间方向排列，可以得到时频表示。如果采用幅度：$S(m,k) = |X(m,k)|$，则得到幅度谱图。如果采用幅度平方：$P(m,k) = |X(m,k)|^2$，则

得到功率谱图。频谱图（Spectrogram）通常使用二维图像形式展示这些结果。其中：

- 横轴表示时间。
- 纵轴表示频率。
- 颜色或亮度表示相应时间和频率位置的幅值或功率。

例如，对于一个先出现低频声音、后出现高频声音的信号，时频图可以更加直观地展示频率成分随时间变化的过程。

3.1.3 Mel 频谱与常用音频特征

为什么需要 Mel 频谱 STFT 可以生成线性频率轴上的时频表示。例如，使用 512 点 FFT、16 kHz 采样率时，可以获得 257 个非负频率位置。这些频率位置在 Hz 尺度上等间隔排列。但是，人类对频率间隔的感知并不完全遵循线性 Hz 尺度。例如，在较低频率区域，相对较小的频率变化可能具有明显的听觉差异；而在较高频率区域，相似的感知差异可能对应更大的 Hz 变化范围。

因此，研究者提出了 Mel 频率尺度，用于近似描述人类对音高间隔的感知特性。Mel 频谱（Mel Spectrogram）通过将线性频率谱映射到 Mel 频带，构建一种更加适合多种音频学习任务的时频表示。Mel 频谱并不是完整的人类听觉模型，也不意味着它能够准确描述所有响度、音色和空间听觉现象。

Mel 频率尺度 Mel 频率尺度是一种经过心理声学研究建立的非线性频率尺度。一种常见的 HTK 风格近似公式为：

$$2595 \log_{10}\left(1 + \frac{f}{700}\right) \tag{3.7}$$

其中：

- f 表示以 Hz 为单位的频率。
- mf 表示对应的 Mel 值。

这一函数能够将线性 Hz 频率转换为非线性的 Mel 尺度。Mel 尺度并不只有这一种数学实现。例如，一些音频库还采用 Slaney 风格的 Mel 尺度，在低频区域使用分段线性形式，在较高频率区域使用对数形式。因此，在实际模型中，需要明确使用哪一种 Mel 频率映射方式，而不能认为所有标记为 Mel Spectrogram 的特征必然完全一致。

Log-Mel 频谱 在实际语音和音频模型中，经常使用 Log-Mel Spectrogram，即对 Mel 频谱进一步进行对数变换。例如：

$$\log(S_{\text{mel}} + \epsilon) \tag{3.8}$$

其中，ϵ 是一个较小的正数。如果使用分贝形式，也可以采用：

$$10 \log_{10} \left(\frac{S_{\text{mel}}}{P_{\text{ref}}} \right) \tag{3.9}$$

并进行适当的数值下限处理。这种对数变换能够压缩数值动态范围，使不同强度的频谱信息更容易共同参与后续计算。例如，较强声学成分与较弱声学成分之间的数值差异，经过对数映射后可能不再像线性功率尺度上那样悬殊。因此，Log-Mel 特征广泛用于语音识别、音频事件分类、说话人相关任务和其他音频模型。但 Log-Mel 并不是原始声音的无损表示。

在计算过程中，频率信息被 Mel 滤波器组合并，而仅使用功率谱时还会舍弃 STFT 相位信息。因此，从 Log-Mel 直接恢复原始音频通常需要额外的重建模型或近似算法，无法保证无损恢复。

Log-Mel 的典型张量形状 假设输入为 1 秒、16 kHz 的单声道音频，使用以下参数：

- 窗口长度：400 采样点，即 25 ms。
- 帧移：160 采样点，即 10 ms。
- FFT 点数：512。
- Mel 频带数量：80。
- 使用中心填充。

在一种常见实现中：原始波形为：

$$x \in \mathbb{R}^{16000} \tag{3.10}$$

经过 STFT 得到：

$$X \in \mathbb{C}^{257 \times 101} \tag{3.11}$$

计算功率谱后：

$$P \in \mathbb{R}^{257 \times 101} \tag{3.12}$$

经过 Mel 滤波器组：

$$S_{\text{mel}} \in \mathbb{R}^{80 \times 101} \tag{3.13}$$

最后进行对数变换：

$$S_{\text{logmel}} \in \mathbb{R}^{80 \times 101} \tag{3.14}$$

可以看到，一维时间波形经过一系列变换后，成为具有时间与频率特征维度的二维矩阵。这种表示可以进一步作为 CNN、Transformer 或其他音频编码器的输入。最终时间帧数仍然取决于具体实现的中心填充与边界处理配置。

频谱质心与其他统计特征 除了频谱图和 Mel 特征，还可以提取一些较为紧凑的声学统计量。例如，频谱质心（Spectral Centroid）可以表示频谱能量或幅值分布的中心位置。一种常见定义为：

$$C = \frac{\sum_k f_k A[k]}{\sum_k A[k]} \tag{3.15}$$

其中：

- f_k 表示第 k 个频率位置。
- (A[k]) 表示该位置的谱幅值。

在某些声音分析场景中，频谱质心可以作为描述声音明亮程度的一种辅助特征。此外，还可以使用频谱带宽、频谱滚降点、短时能量和过零率等特征。这些统计特征通常具有解释直观、计算成本较低等特点。但它们只能描述声音的部分属性，通常不能替代通过大规模训练学习得到的完整音频表示。

从 Log-Mel 特征到音频 Token 在现代音频模型中，Log-Mel 频谱通常并不是最终表示，而是音频编码器的输入。例如，可以将 Log-Mel 特征表示为：

$$S \in \mathbb{R}^{M \times T} \tag{3.16}$$

其中，M 表示 Mel 频带数，T 表示时间帧数。随后，可以通过卷积网络提取局部时频特征，也可以通过 Transformer 对时间序列或时频 Patch 进行建模。例如，使用卷积层可以将：

$$S \in \mathbb{R}^{80 \times T} \tag{3.17}$$

转换为：

$$Z \in \mathbb{R}^{T' \times D} \tag{3.18}$$

形式的音频特征序列。其中：

- T' 表示编码器输出的时间位置数量。
- D 表示音频特征维度。

这些音频特征可以进一步被组织为音频 Token，参与语音识别、音频理解或多模态语言模型的计算。音频 Token 并不一定是离散符号。与经典 ViT[1] 的视觉 Token 类似，音频编码器也可以直接输出连续向量序列。另一方面，某些音频生成模型还会使用神经音频编解码器，将音频映射为离散的声学 Token。因此，“音频 Token”可以指连续特征序列中的基本单元，也可以指某些模型中经过量化的离散表示，具体含义取决于架构。

这些不同形式将在后续章节进一步介绍。

音频特征中的信息损失 不同音频表示在突出某些信息的同时，也可能舍弃其他信息。例如：原始波形保留数字化采样后的时间幅值序列。复数 STFT 保留短时频率系数的幅值和相位信息。功率谱主要关注频率能量分布，不直接保留相位。Mel 频谱进一步将线性频率轴上的信息聚合为较少的 Mel 频带。Log-Mel 通过对数映射调整数值尺度，但不能恢复 Mel 聚合过程丢失的频率细节。MFCC 通过 DCT 等操作进一步改变和压缩特征形式。

因此，音频特征提取并不是无条件增加信息，而是在不同任务需求下重新组织和突出特定信息。这一点与视觉特征提取中的空间降采样、Patch Embedding 和 Token 压缩具有一定相似性。从多模态大模型的角度来看，关键问题是：

如何将原始声音转化为适合模型学习和推理的表示，同时尽可能保留目标任务所需要的语言、声学和时间信息？

3.2 连续与离散音频表示

3.2.1 音频编码器与隐空间表示

音频编码器 音频编码器（Audio Encoder）是指将原始波形或预处理后的音频特征转换为学习表示的神经网络模块。例如，可以将其表示为：$Z_A = f_\theta(X_A)$，其中：

- X_A 表示输入音频或声学特征。
- f_θ 表示音频编码器。
- Z_A 表示编码器输出的隐空间表示。
- θ 表示编码器参数。

假设输入为一段 Log-Mel 特征：

$$X_A \in \mathbb{R}^{T\times 80} \tag{3.19}$$

经过编码后，可以得到：

$$Z_A \in \mathbb{R}^{T'\times D} \tag{3.20}$$

其中：

- T' 是输出时间位置数量。
- D 是编码器的隐藏特征维度。

在许多模型中：$T' < T$ 即编码器通过时间降采样减少序列长度。但并不是所有模型都必须降低时间分辨率。是否降采样取决于模型架构和任务需要。

隐空间表示 隐空间表示（Latent Representation）是神经网络经过特征变换后得到的内部表示。例如，模型可以将原始音频从较高采样率的波形，逐步变换为长度较短、维度较高的特征向量序列。这些特征不一定具有直接的人类可解释含义，但可能包含对

目标任务有用的信息。例如，在语音识别模型中，部分隐藏表示可能有助于区分不同音素和词语。在环境声音分类模型中，隐藏表示可能包含不同声学事件的特征。

在说话人识别模型中，模型则可能学习与说话人身份相关的表示。因此，隐空间表示的语义性质不是固定的，而是由训练数据、训练目标和网络架构共同塑造。隐藏维度较高并不意味着特征一定包含更多有用信息，也不意味着每个维度能够对应某种明确的声学属性。

基于卷积的音频编码器 卷积神经网络不仅能够用于图像，也能够用于音频特征提取。对于原始波形，可以使用一维卷积（1D Convolution）沿时间轴提取局部信号特征。例如：

$$x \in \mathbb{R}^N \tag{3.21}$$

可以经过多层一维卷积，转换为：

$$Z \in \mathbb{R}^{T' \times D} \tag{3.22}$$

这种设计能够利用卷积核学习局部时间结构，并通过步幅或池化降低时间序列长度。对于 Log-Mel 频谱，也可以使用二维卷积（2D Convolution），在时间与频率两个方向上提取局部时频模式。例如：

$$S \in \mathbb{R}^{80 \times T} \tag{3.23}$$

可以作为二维特征图输入卷积网络。音频频谱的两个坐标轴分别代表频率和时间，它们的物理意义与普通图像的二维空间坐标不同。因此，音频卷积网络的核大小、步幅及特征聚合方式，应当考虑时间和频率两种维度的差异。

HuBERT：通过隐藏单元预测学习语音表示 HuBERT[6]（Hidden-Unit BERT）是另一类代表性的语音自监督学习方法。它通过聚类等方式，为音频时间位置构造离散伪标签，并利用掩码预测目标训练模型。例如，可以先使用某种声学特征及聚类算法，为不同音频帧分配聚类编号：$c_t \in 1, 2, \ldots, K$，随后，模型在输入音频的部分位置被遮挡时，预测相应的隐藏单元类别。通过这种方式，模型不需要依赖人工转写文本，就能够学习音频的上下文结构。

这些聚类编号并不天然对应汉字、词语或具有明确语言学含义的音素。它们主要是通过无监督或迭代学习得到的离散目标。因此，HuBERT 中的离散聚类标签，与后面介绍的用于音频重建的神经编解码 Token，虽然形式上都可以是整数序列，但目标和用途不同。

音频编码器如何接入语言模型 在多模态大模型中，一种常见方式是使用独立的音频编码器提取声学表示，再通过连接模块将其提供给语言模型。例如：$Z_A = f_A(x)$，其中：

$$Z_A \in \mathbb{R}^{T' \times D_A} \tag{3.24}$$

如果语言模型隐藏维度为 D_L，可以通过一个 Projector 进行映射：$Z'_A = P(Z_A)$ 得到：

$$Z'_A \in \mathbb{R}^{T' \times D_L} \tag{3.25}$$

随后，可以通过统一序列建模、交叉注意力或其他方式，使语言模型利用音频信息。不过，音频特征与语言模型的隐藏维度相同，并不意味着二者已经完成语义对齐。例如，一段音频中可能包含语音内容、语调和环境噪声，而文本回答只需要其中一部分信息。因此，模型仍然需要通过相应的数据与训练目标，学习如何从音频表示中提取和使用任务相关的信息。具体的语音语言联合建模方法将在后续章节进一步介绍。

3.2.2 音频离散化与音频 Token

为什么需要音频离散化 虽然连续声学特征能够保留丰富的信息，但在某些生成和压缩任务中，直接建模高采样率波形可能具有较高的序列计算成本。例如，一秒 24 kHz 音频包含 24000 个采样点。如果生成模型需要逐采样点预测完整波形，就可能面对非常长的输出序列。因此，研究者探索先使用音频编码器将波形压缩为较短的潜在表示，再通过离散量化，将连续潜在特征转换为有限词表中的编号。这样，音频就可以表示为一组离散 Token。

例如：$C = (c_1, c_2, \ldots, c_T)$，其中：$c_t \in 0, 1, \ldots, K-1$ 表示某个时间位置被分配到的离散码本索引。这些索引可以进一步通过嵌入层转换为向量，也可以作为序列生成模型的预测目标。离散化不是音频理解或生成的唯一方式。一些模型可以直接在连续表示空间中预测或生成音频特征。

向量量化的机制 向量量化（Vector Quantization，VQ）是一种将连续向量映射为有限离散码本的方法。假设音频编码器产生一个连续特征向量：

$$z_t \in \mathbb{R}^D \tag{3.26}$$

同时，建立一个包含 K 个向量的码本：$E = e_1, e_2, \ldots, e_K$，其中：

$$e_k \in \mathbb{R}^D \tag{3.27}$$

对于输入向量 z_t，可以选择距离最近的码本向量：

$$c_t = \underset{k}{\operatorname{argmin}} \|z_t - e_k\|_2^2 \tag{3.28}$$

其中，c_t 是一个离散索引。量化后的向量为：$z_t^q = e_{c_t}$，因此，连续向量被转换为有限码本中的一个向量，同时可以使用对应的整数编号表示。这种最近邻量化只是常见方法之一。实际量化器还可能采用其他距离度量、软分配或可学习机制。

码本与音频 Token 的关系 量化码本可以理解为一组可学习的声学表示原型。例如，如果码本包含：$K = 1024$ 个向量，那么每个音频特征位置可以被分配一个取值范围为：$0 \sim 1023$ 的整数编号。这些编号可以作为离散音频 Token。但这里需要区分：

Token ID 是一个离散编号。

码本向量是该编号对应的连续特征。

例如，编号 17 可以对应一个 D 维码本向量，但编号本身并不是声音波形。只有在具备相应的音频解码器时，才能进一步尝试从量化后的潜在特征重建声音。因此，离散音频 Token 并不一定能够独立转换为可听音频。

残差向量量化 如果只使用一个码本向量表示每个时间位置，可能难以在有限码本大小下保留丰富的声学细节。因此，神经音频编解码器经常采用残差向量量化（Residual Vector Quantization，RVQ）。其基本思想是使用多个量化阶段，逐步编码前面阶段尚未表示的残差信息。例如，初始特征为：

$$r_t^{(0)} = z_t \tag{3.29}$$

第一阶段选择最接近的码本向量：

$$q_t^{(1)} = Q_1(r_t^{(0)}) \tag{3.30}$$

并得到剩余残差：

$$r_t^{(1)} = r_t^{(0)} - q_t^{(1)} \tag{3.31}$$

第二阶段继续量化：

$$q_t^{(2)} = Q_2(r_t^{(1)}) \tag{3.32}$$

依此类推。经过 M 个量化阶段后，可以得到近似重建特征：

$$\hat{z}_t = \sum_{m=1}^{M} q_t^{(m)} \tag{3.33}$$

每个量化阶段可以产生一个离散索引：

$$c_t^{(m)} \tag{3.34}$$

因此，一个时间位置可能对应多个离散码本索引，而不是单独一个 Token。这种设计使模型能够逐步增加表示精细程度，但也增加了离散索引数量和相应的建模需求。

离散 Token 的码率 音频 Token 的数量与码率可以建立联系。假设：

- 每秒产生 R 个潜在时间帧。

- 每帧使用 M 个码本索引。
- 每个码本包含 K 个候选向量。

如果各码本大小相同，且使用固定长度的二进制索引，那么理想情况下，码率可以近似表示为：

$$B \approx RM \log_2 K \tag{3.35}$$

其中，B 的单位为 bit/s。例如：$R = 50$ 即每秒 50 个潜在时间帧。如果：$M = 4,\quad K = 1024$，则每个码本索引理论上需要：

$$\log_2 1024 = 10 \tag{3.36}$$

bit。因此：$B \approx 50 \times 4 \times 10$，$B \approx 2000$ bit/s 即约 2 kbps。这里的计算只考虑固定码本索引的理论存储量，不包括编码头、元数据、纠错、熵编码及其他实际系统开销。此外，真实模型的码本数量、帧率和索引组织方式可能不同，因此不能将这一公式直接作为某个具体音频编解码器的实际码率。

3.2.3 音频片段、时间戳与长音频处理

为什么长音频需要特殊处理 音频与静态图像的重要区别之一，是音频信息沿时间轴持续变化。例如，一段 60 秒的音频在 16 kHz 采样率下包含：$N = 60 \times 16000$，$N = 960000$ 个采样点。如果进一步使用 10 ms 帧移提取声学特征，则大约会形成每秒 100 个时间帧。因此，60 秒音频可能产生约 6000 个时频特征位置，具体数量取决于窗口及边界处理方式。

如果将这些特征直接输入使用标准全连接注意力的 Transformer，那么注意力交互规模可能随序列长度平方增长。因此，长音频通常需要考虑时间降采样、片段划分、局部建模或其他高效时序处理方法。

时频帧的时间戳 对于 STFT 或 Log-Mel 特征，相邻时间帧之间的间隔通常由帧移决定。例如：$H = 160,\quad f_s = 16000$ 那么每帧的标称时间间隔为：

$$\Delta t = \frac{H}{f_s} = 0.01 \text{ s} \tag{3.37}$$

因此，相邻特征位置相差约 10 ms。如果不使用中心填充，并且第 m 帧从采样位置 (mH) 开始，其分析窗口的起始时间为：

$$t_m^{start} = \frac{mH}{f_s} \tag{3.38}$$

如果希望使用窗口中心作为特征时间，可以进一步写为：

$$\frac{mH + (L-1)/2}{f_s} \tag{3.39}$$

其中，L 为窗口长度。如果 STFT 使用中心填充，具体时间索引约定可能不同，因此需要根据实际实现定义时间戳。

编码器降采样后的时间对应关系 经过卷积或其他时间降采样后，编码器输出的时间帧数可能减少。例如，原始 Log-Mel 特征每秒约有 100 帧。如果编码器总时间降采样倍数为 4，那么输出特征可能约为每秒 25 帧。此时，输出序列的标称时间步长约为：$\Delta t' = 4 \times 0.01$，$\Delta t' = 0.04$ s 即约 40 ms。但输出帧之间的时间间隔，不等于每个输出特征实际利用的输入时间范围。

由于卷积核、注意力或其他上下文聚合机制的存在，每个输出特征可能依赖更长的音频片段。因此，音频编码器的时间对齐需要同时考虑输出步长、理论感受范围以及具体的时间位置定义。

时间戳与语音识别 在语音识别任务中，模型通常需要将音频内容转换为文本。如果只输出完整转写文本，可能无法直接知道每个词语出现在录音中的哪个时间位置。因此，一些系统还会提供句子级、片段级或词级时间戳。例如：0.5–1.2 秒：“你好”。1.2–2.8 秒：“欢迎使用语音助手”。这类时间信息可以用于字幕生成、音视频编辑和语音内容检索。但模型输出的文本 Token 位置不一定直接对应真实声音时间。

例如，一个汉字或词语可能跨越多个音频帧，而不同语言的文本 Tokenizer 也会采用不同的切分粒度。因此，精确词级时间对齐可能需要额外的对齐算法或专门训练的模型，而不能仅根据文本 Token 顺序进行简单换算。

缓存与跨片段上下文 在长音频和流式处理过程中，模型可能需要保留此前片段的部分信息。例如，可以缓存卷积状态、注意力中的历史 Key/Value，或者已经提取的片段级语义表示。当新的音频片段到达时，模型可以利用已有缓存减少重复计算，并维持一定的上下文连续性。但缓存并不是无限免费的。随着历史长度增加，存储和计算成本可能上升。此外，如果缓存包含与当前任务不再相关的信息，也可能对后续计算造成负担。

因此，长音频模型需要决定保留哪些历史信息、保留多久，以及如何在新信息到达时更新表示。这一问题与长上下文语言模型具有一定相似性，但音频还具有持续时间、实时到达和声学事件边界等特殊要求。

长音频的计算与信息权衡 长音频处理始终面临信息范围与计算成本之间的权衡。较短片段通常具有较低的局部计算成本，但可能缺乏完整上下文。较长片段可以保留更加连续的语音或声学事件信息，但可能产生更多时间 Token 和更高计算负担。更高时间分辨率能够提供更细致的事件定位能力，但也会增加后续序列长度。较强的时间降采样可以减少 Token 数量，但可能削弱短暂事件或精细时间边界信息。因此，不同任务需要采用不同的时间粒度。

例如，整段声音分类可能允许较强的时间聚合，而词级时间对齐和短时声学事件检测通常需要更加细致的时间表示。对于多模态对话，模型还可能需要结合长期语义上下文与近期高频声学信息。因此，长音频编码的核心问题，不只是让模型能够处理更长的输入，而是**如何在有限计算预算下保留足够的时间结构、语义上下文和任务相**

关声学信息。

3.3 语音预训练与非语言声学信息

3.3.1 语音自监督学习

语音自监督学习的机制 传统监督式语音识别需要大量音频及对应的文本转写。例如，一段语音：“请打开会议室的灯光。”需要配有相应文本标签，模型才能直接学习从声音到文字的映射。但自然界中大量录音并不具备人工转写。为这些录音逐一提供准确标注，通常需要较高成本。语音自监督学习（Self-Supervised Speech Learning）尝试利用音频自身的结构构造训练信号，使模型能够先从未转写的语音中学习表示，再通过少量或适量标注数据适配具体任务。

其基本过程可以表示为：$X \xrightarrow{f_\theta} Z$，其中，$X$ 为原始波形或声学特征，f_θ 为语音编码器，Z 为学习得到的表示。预训练阶段并不直接要求输出最终的文本转写，而是通过掩码预测、对比学习、聚类标签预测或教师—学生特征预测等任务优化编码器。因此，自监督学习不是没有训练目标，而是训练目标主要来自音频数据本身及相应的自监督机制。

为什么语音适合自监督预训练 语音具有丰富的时间结构。在连续讲话过程中，相邻音频片段通常存在声学连续性，音素、音节和词语之间也存在一定的组合规律。例如，一个元音的持续发声会形成相对稳定的声学结构，而一句话中的不同发音单位又按照语言规则组织。因此，模型可以利用已观察到的音频片段，对其他时间位置的信息进行预测。如果模型能够完成这样的预测，就需要学习一定的局部声学模式和较长范围的上下文关系。

但是，语音自监督模型学到的信息并不只包含语言内容。原始语音中还可能包含说话人、音色、语速、韵律和录音环境等因素。不同预训练目标可能强化其中某些信息，也可能弱化另一些信息。因此，预训练表示能否迁移到具体任务，需要结合实际的训练目标判断。

掩码预测 掩码预测（Masked Prediction）是语音自监督学习中的重要方法。其基本思路是选择音频特征序列中的部分时间位置，将这些位置遮挡，再要求模型根据其余上下文预测被遮挡的信息。假设音频经过特征提取后得到：$Z = (z_1, z_2, \ldots, z_T)$ 选择一个掩码位置集合：

$$\mathcal{M} \subseteq 1, \ldots, T \tag{3.40}$$

模型在无法直接观察被遮挡特征内容的条件下，预测相应目标。例如，一段语音中间的某些声学帧被遮挡后，模型可以利用前后的语音上下文推测被遮挡位置应当具有怎样的声学表示。这种训练方式与前面介绍的 BERT 掩码语言建模、视觉掩码图像建模具有相似思想。但三种模态中的预测对象并不相同。文本掩码预测通常针对离散

文本 Token；视觉掩码任务可能预测像素或视觉特征；语音掩码任务则可以预测连续声学表示或离散学习目标。

因此，掩码是一种通用的自监督训练策略，具体表示能力仍然取决于预测目标。

wav2vec 2.0：**掩码与对比学习** wav2vec 2.0[7] 是语音自监督学习的重要代表。其主要思路是先利用卷积网络从原始波形提取连续潜在特征，再通过 Transformer 建立上下文表示。假设原始音频为：

$$x \in \mathbb{R}^N \tag{3.41}$$

卷积特征编码器输出：$Z = (z_1, z_2, \ldots, z_T)$，随后，模型对部分潜在时间位置执行掩码处理，并利用上下文网络产生对应表示：$C = (c_1, c_2, \ldots, c_T)$ 预训练中，模型还利用量化模块从潜在声学特征构造目标表示。对于被遮挡的时间位置，模型需要从候选量化目标中识别相应的正确表示。一种简化的对比损失可以写为：

$$-\log \frac{\exp(\mathrm{sim}(c_t, q_t)/\tau)}{\sum_{\tilde{q}\in\mathcal{Q}_t} \exp(\mathrm{sim}(c_t, \tilde{q})/\tau)} \tag{3.42}$$

其中，c_t 为上下文表示，q_t 为对应目标量化表示，$\mathcal{Q}_t$ 包含正确目标及候选负样本，τ 为温度参数。实际 wav2vec 2.0 还包含促进码本利用等训练约束。通过这种方式，模型能够利用大量未转写语音学习特征，随后再通过 CTC 等下游训练目标适配自动语音识别。

wav2vec 2.0 在预训练中使用量化目标，并不意味着其下游编码器必须输出离散音频 Token。用于任务适配的上下文隐藏状态通常仍然是连续向量。

HuBERT：**隐藏单元预测** HuBERT（Hidden-Unit BERT）采用另一种具有代表性的自监督训练方式。它首先通过无监督聚类等方法，为语音时间位置生成离散的伪标签。例如，假设某种声学表示经过聚类后得到 K 个簇，那么不同时间位置可以对应：$u_t \in 1, 2, \ldots, K$ 的离散目标。随后，模型对部分音频时间位置进行掩码，并根据其余上下文预测被遮挡位置的聚类标签。相应的预训练损失可以概括为：

$$-\frac{1}{|\mathcal{M}|} \sum_{t\in\mathcal{M}} \log P_\theta(u_t \mid X_{\text{masked}}) \tag{3.43}$$

其中，$\mathcal{M}$ 表示被遮挡的位置集合。HuBERT 的特点是通过离散隐藏单元提供学习目标，而不需要事先为每段语音标注准确的音素或文字。但这些聚类标签本身并不天然等于真实音素，也不意味着编号相同的所有语音片段具有完全相同的语言学意义。它们主要用于构造具有一定一致性的自监督学习信号。

data2vec：**预测连续上下文表示** 自监督学习并不一定需要将目标转换为离散聚类标签或量化索引。data2vec 提出了一种基于教师—学生结构的表示预测思路。其基本方法是让学生网络根据经过掩码处理的输入，预测教师网络从完整输入中提取的上下文特征。可以用简化形式表示为：$Z_T = f_{\theta_T}(X)$，

$$\hat{Z}_S = g_{\theta_S}(X_{\text{masked}}) \tag{3.44}$$

然后在指定时间位置约束：

$$\frac{1}{|\mathcal{M}|}\sum_{t\in\mathcal{M}} |\hat{z}_{S,t} - z_{T,t}|_2^2 \tag{3.45}$$

这里采用平方误差说明特征预测思想，并不代表所有 data2vec 实现都必须采用这一完全相同的损失形式。这种方法的重要特点，是可以直接预测连续的上下文目标，而不必首先将全部目标离散化。此外，data2vec 还探索了将相近的自监督学习思想应用于语音、视觉和语言等不同模态。但跨模态通用的训练思想，并不意味着不同模态已经具有统一的语义表示空间。它主要说明某些学习机制可以跨数据类型复用。

3.3.2 音频文本对齐

为什么需要音频与文本对齐 语音自监督学习可以从音频中获得有用特征，但不一定直接建立声音与自然语言描述之间的对应关系。例如，一段录音可能包含雨声、脚步声和远处的交通噪声。一个音频编码器可以提取相应声学特征，但如果希望用户使用自然语言检索该录音，模型还需要理解：“雨天街道上有人行走。”这样的文字与音频之间具有怎样的语义联系。因此，音频—文本对齐（Audio-Text Alignment）的核心目标，是使音频内容与相应的语言表达建立有效的关联。

这类能力可以支持文本检索音频、音频检索文本、开放词汇声音分类，以及后续的音频问答和音频语言模型。

语音转写监督与一般音频文本对齐的区别 音频与文本之间可以存在不同类型的配对关系。第一种是**语音与转写文本**。例如：音频：某人说出“火车即将进站”。文本：“火车即将进站”。这里的文本主要记录语音中的语言内容。第二种是**音频与自然语言描述**。例如：音频：火车运行、刹车和站台广播的混合声音。文本：“一列火车正在进站，伴随着制动声和广播声。”这里的文本不仅可能涉及语音，还包含环境声音、事件及其关系。

两种数据形式虽然都属于音频—文本配对，但对应的监督目标有所不同。语音转写强调恢复说话内容，而一般音频描述更强调将整体声学事件映射到自然语言概念。因此，面向 ASR 的监督数据，不一定能够直接替代一般音频理解所需的描述性数据。

音频—文本对比学习目标 假设一个训练批次包含 B 组配对样本：

$$\{(A_i, T_i)\}_{i=1}^{B} \tag{3.46}$$

音频编码器和文本编码器分别生成归一化表示。对于第 i 个音频，可以将对应文本作为正样本，将批次中的其他文本作为候选负样本。一种音频到文本的对比损失为：

$$-\frac{1}{B}\sum_{i=1}^{B}\log\frac{\exp(s(A_i,T_i)/\tau)}{\sum_{j=1}^{B}\exp(s(A_i,T_j)/\tau)} \tag{3.47}$$

类似地，还可以计算文本到音频的对比损失：

$$\mathcal{L}_{T\to A} \tag{3.48}$$

最终采用双向目标：

$$\frac{1}{2}\left(\mathcal{L}_{A\to T}+\mathcal{L}_{T\to A}\right) \tag{3.49}$$

其中，τ 表示温度参数。这种训练方式与前面介绍的 CLIP[8] 图像—文本对比学习具有相似思想。但音频具有持续时间、多声源混合和事件先后顺序等特点，因此音频—文本对齐还需要考虑声音描述的时间粒度和覆盖范围。

全局对齐与时间级对齐 音频—文本对齐可以发生在不同粒度。对于图文或音文检索，常见做法是使用一个全局向量表示整段输入。例如：

$$z_A\in\mathbb{R}^D \tag{3.50}$$

与对应的文本向量进行比较。但如果用户询问：“玻璃破碎声是在狗叫之前还是之后出现？”仅使用全局相似度可能不够。模型还需要区分不同声音事件对应的时间区间。因此，可以进一步考虑时间片段级表示：$Z_A=(z_1,z_2,\ldots,z_L)$ 使模型能够建立局部音频片段与文本对象、动作或事件描述之间的联系。整段音频与一句描述具有较高相似度，并不能证明描述中每个事件都已经获得正确的时间定位。

因此，**音频—文本的全局语义对齐，与局部事件定位和时间关系理解属于不同层次的能力。**

配对数据与语义覆盖 音频—文本对齐的质量受到训练数据影响。例如，如果训练文本主要关注声音类别：“Dog Barking”“Rain”“Traffic Noise”。模型可能比较容易学习这些声音概念之间的区别。但对于复杂描述：“远处先出现一声尖锐鸣笛，随后传来持续的发动机声。”模型还需要理解事件顺序、距离相关声学线索和多个声源之间的关系。如果训练数据缺乏这类细粒度描述，模型未必能够可靠学习相应能力。

此外，一段音频可能具有多个合理描述。某条配对文本通常只覆盖其中一部分信息。因此，音频—文本训练中的所谓负样本，也可能包含与当前音频部分相关的内容。这说明，对比学习可以提供有效的语义监督，但简单的批内正负样本设置并不能完整描述所有音频—语言关系。

音频—文本对齐与语言模型连接 CLAP 等模型能够建立音频与文本嵌入之间的关联，但它们通常不是直接生成复杂自然语言回答的完整对话模型。例如，CLAP 可以判断某段音频与“下雨”这一描述是否相关，但要回答：“雨声在什么时候出现？是否有其他声音干扰?”通常还需要具备时间分析与语言生成能力的任务模块。在多模态大模型

中，可以使用预训练音频编码器提取连续表示，再通过连接模块提供给语言模型。例如：$Z_A = f_A(A)$，$Z'_A = P(Z_A)$，

其中，P 可以是线性层、多层感知机、查询式聚合模块或其他跨模态连接结构。但是，音频特征维度与语言模型隐藏维度相同，并不意味着语言模型已经能够正确利用其中的信息。还需要通过相应的联合训练，使模型学会基于音频证据生成回答。因此，音频—文本对齐为音频语言联合建模提供了重要基础，但它并不能直接替代完整的多模态理解训练。

3.3.3 语音内容与非语言声学信息

声音不只有文本内容 语音是音频的重要组成部分，但一段语音包含的信息远不止文字转写。例如，某人说：“我马上回来。”除了这句话的语言内容，声音还可能包含说话人的音色、语速、韵律、停顿和发声方式。如果录音中同时存在脚步声、关门声或背景音乐，这些非语言声音也可能对场景理解具有重要作用。因此，在多模态模型中，需要区分：

语言内容信息：语音表达的词语和句子。

副语言声学信息：语调、节奏、重音及其他说话方式相关特征。

说话人相关信息：与不同说话人有关的声学特征和时间归属。

非语音环境信息：交通、动物、机械、音乐及其他声学事件。

这些信息可能同时出现在同一段波形中，但通常不能通过单一文本转写完整表达。

从音频到文本可能产生信息损失 在语音识别系统中，模型通常将音频转换为文字。例如：$A \xrightarrow{\text{ASR}} T$，这种处理方式能够使语言模型方便地使用语音内容。但音频到文本的转换本质上是一种有选择的信息提取。例如，一段录音中某人以急促语气说：“快走!”普通 ASR 可能只输出：“快走”。这份文本通常不包含完整的发声速度、音量变化、紧张程度相关声学线索或背景声音。

因此，文本转写虽然能够保留语言内容，却不等于完整保留原始声学信息。在多模态问答中，如果用户只需要总结讲话内容，那么文本转写可能已经足够。但如果用户询问：“这句话是用什么语气说出的?”“讲话时背景中出现了什么声音?”那么仅使用转写文本就可能无法提供充分证据。

语音内容与声学表现可以相互补充 语言内容和声学表现有时提供互补信息。例如，同一句：“这真是个好消息。”可以用高兴、平静或讽刺等不同方式表达。文字能够体现明确的语言内容，但语音中的韵律和语调还可能影响具体交流含义。因此，在某些情感或交互理解任务中，可以联合使用文本语义特征与音频声学特征。例如：$Z_T = f_T(T)$，$Z_A = f_A(A)$ 再通过融合模块：$H = F(Z_T, Z_A)$ 获得联合表示。

但某种声学特征与情绪表达之间通常不存在固定的一一对应关系。例如，说话音量较大可能与多种状态或录音条件相关，不能仅根据音量就可靠判断个人真实情绪。因此，模型应当将声学信息作为任务相关证据，而不是把有限的声学模式直接等同于确定的心理状态。

多声源混合 现实录音往往包含多个声源的叠加。在一个简化的线性混合模型中，可以将单通道录音表示为：

$$x(t) = \sum_{i=1}^{K} s_i(t) + n(t) \tag{3.51}$$

其中：

- $s_i(t)$ 表示不同声源的贡献。
- $n(t)$ 表示其他噪声或干扰。
- K 表示声源数量。

例如，一段会议录音可能包含两位说话人的重叠语音、空调噪声和键盘敲击声。这种情况下，单一的全局音频向量可能可以表达整体场景，但未必能够区分每个声源的独立信息。因此，复杂音频理解可能需要结合语音活动检测、说话人分离标注、声源分离或声音事件检测等机制。上述加性模型只是教学近似。真实录音还会受到房间声学、混响、设备响应和其他因素影响。

联合表示与信息分工 当一个多模态系统需要同时利用语音和环境声音时，可以采用不同的信息组织方式。一种方式是首先对语音执行 ASR，再利用文本提供语言内容。与此同时，使用音频编码器保留声学事件或说话方式等信息。例如：

$$Z_{\text{speech}} = f_{\text{ASR}}(A) \tag{3.52}$$

$$Z_{\text{acoustic}} = f_{\text{audio}}(A) \tag{3.53}$$

随后，根据具体任务组合两类信息。另一种方式是使用端到端音频语言模型，直接从连续音频特征中学习语言内容和非语言信息。这两类路线并不存在简单的绝对优劣。前者具有信息接口相对清晰的特点，但可能依赖多个模块，也可能受到转写错误影响。后者有机会联合利用更丰富的音频信息，但仍然需要足够的数据和训练目标证明其能力。因此，设计联合表示时，应根据任务确定哪些信息必须保留，而不是默认所有音频都应首先转换为文本。

3.4 视频采样与时空编码

同一段视频可以由逐帧图像编码，也可以由时空块编码。前者便于复用图像模型，后者在编码阶段引入跨帧交互；两者都受到实际采样的限制。若关键动作发生在两个采样时刻之间，后续增加语言推理步骤也不能恢复没有进入输入的证据。

3.4.1 视频采样与关键帧提取

为什么需要视频采样 实际视频可能包含大量帧。例如，一段 5 分钟、30 FPS 的视频，理论上包含约：$T = 300 \times 30 = 9000$ 帧。如果每帧经过视觉编码器得到 196 个视觉 Token，且不进行任何压缩，那么对应的视觉 Token 总数可达到：$9000 \times 196 = 1764000$ 个。如此长的视觉序列通常会带来很高的计算、显存和上下文管理成本。此外，视频相邻帧之间往往存在较多重复信息。

例如，人物静止站立时，连续多帧可能具有高度相似的场景内容。因此，没有必要在所有任务中对全部帧进行同等精度的处理。视频采样（Video Sampling）的基本目标，是从原始帧序列中选择部分具有价值的视觉观测，在控制计算成本的同时保留任务相关的时间与语义信息。

均匀时间采样 均匀时间采样（Uniform Temporal Sampling）是视频处理中较为常见的基础方法。其基本思想是在视频持续时间内，按照相同时间间隔选择视频帧。例如，对于一段 10 秒视频，希望每秒采样 1 帧，可以选择：$t = 0, 1, 2, \ldots, 9$ s 附近的帧。如果希望使用总共 K 个采样位置，也可以在给定的时间区间内均匀分布 K 个目标时间点。例如，在：

$$[t_{\text{start}}, t_{\text{end}}] \tag{3.54}$$

之间选择目标时间：

$$t_k = t_{\text{start}} + \frac{k}{K-1}(t_{\text{end}} - t_{\text{start}}) \tag{3.55}$$

其中：$k = 0, 1, \ldots, K-1$ 且：$K \geq 2$ 然后在原始视频中选择与目标时间最接近的有效帧。目标时间点不一定与真实视频帧时间戳完全一致，因此实际采样可能存在一定时间误差。

固定步长采样 固定步长采样（Fixed-Stride Sampling）是另一种常用方法。例如，从第 0 帧开始，每隔 4 帧提取一帧：$0, 4, 8, 12, \ldots$，如果原始视频为 30 FPS，采样步长为 4，那么采样后的名义帧率约为：

$$f_{\text{sample}} = \frac{30}{4} = 7.5 \text{ FPS} \tag{3.56}$$

这种方法实现简单，并且能够控制每段视频的帧数。但固定帧序号步长不一定对应固定真实时间间隔，特别是在可变帧率视频中。此外，固定步长采样可能周期性地错过某些快速变化的事件。因此，采样步长需要根据视频运动速度和任务要求进行选择。

视频片段采样 在视频动作识别和时序建模中，经常使用视频片段（Video Clip）作为基本输入单位。例如，某个模型每次输入 16 帧。可以先选择一个起始位置，再按照特定时间步长选取连续的帧序列：

$$C = (I_s, I_{s+\delta}, \ldots, I_{s+(K-1)\delta}) \tag{3.57}$$

其中：

- s 表示起始帧序号。
- δ 表示帧采样步长。
- K 表示采样帧数。

例如，对于 30 FPS 视频，选择 16 帧、步长为 2，则首末采样帧的时间跨度为：

$$\frac{(16-1)\times 2}{30} \tag{3.58}$$

即：$D_{\text{span}} = 1$ s，这里计算的是首末采样帧时间戳之间的跨度，而不是包含末帧显示持续时间的完整播放区间。视频片段采样能够让模型重点分析一个局部时间范围内的运动过程，但也可能无法覆盖完整长视频中的全部事件。

运动感知与任务相关采样 均匀采样和固定步长采样通常不直接考虑视频内容。因此，如果视频包含大量静止片段和少量快速动作，均匀采样可能将计算资源分配给许多重复画面，却遗漏短暂事件。一种改进思路是根据运动变化或任务需求动态选择视频帧。例如，可以优先考虑：发生明显动作变化的时间位置。出现新对象或场景切换的时间位置。与用户问题相关的对象或区域发生变化的时间位置。具有较高不确定性或需要额外观察的时间片段。不过，运动变化与任务重要性并不完全相同。

例如，镜头快速移动可能导致很大的视觉变化，却不一定包含用户关心的事件；而短暂且局部的细小动作可能非常重要。因此，动态采样策略仍然需要通过实际任务表现验证。

视频采样的根本权衡 视频采样能够有效降低后续模型的计算量，但也会减少可观察的信息。例如，原始视频每秒包含 30 帧，如果模型只保留每秒 1 帧，那么大多数中间画面不会直接进入模型。对于整体场景摘要，这种采样可能仍然足够。但对于快速手势、短暂碰撞、细微动作和事件先后顺序判断，则可能导致信息不足。因此，视频采样并不存在适用于所有任务的固定最佳帧率。合理的采样方案需要结合：视频时长、原始帧率、运动速度、事件持续时间、任务需求以及计算预算。

采样过程中没有被保留的原始视觉信息，通常不能仅靠后续模型计算完全恢复。

3.4.2 时空 Patch 与视频 Token

为什么需要时空 Token 在 Vision Transformer 中，图像通常被划分为多个 Patch，再将其转换为视觉 Token。这一思想也可以扩展到视频。由于视频包含时间和空间三个主要坐标维度，因此视频 Token 不仅需要考虑图像中的水平与垂直位置，还需要考虑对应的时间位置。例如，一个视频可以表示为：

$$V \in \mathbb{R}^{T\times H\times W\times C} \tag{3.59}$$

其中：

- T 表示视频时间帧数。
- H、W 表示图像空间尺寸。
- C 表示颜色通道数。

模型可以根据时间和空间划分，将输入转换为一系列时空局部单元。这些单元经过特征映射后，便可以形成用于 Transformer 的视频 Token。

逐帧 Patch Token 最直接的方法，是分别对每一帧执行二维 Patch Embedding。例如，视频包含 16 帧，每帧分辨率为 224×224。如果 Patch 大小为：$P = 16$ 那么每帧产生：$N_s = 14 \times 14 = 196$ 个 Patch Token。全部视频的 Token 数量为：$N_V = T \times N_s$ 即：$N_V = 16 \times 196 = 3136$，

因此，可以将视频表示为：

$$Z_V \in \mathbb{R}^{3136\times D} \tag{3.60}$$

的 Token 序列。这里的 Token 数量对应 16 个采样视频帧，而不一定是完整原始视频中的全部帧数。此外，还需要通过位置编码等机制保留 Token 对应的时间和空间坐标。

Tubelet Embedding Tubelet Embedding 是将局部时空数据块映射为特征向量的一类方法。假设时间 Patch 长度为：P_t 空间 Patch 大小为：$P_h \times P_w$ 那么一个时空 Patch 可以表示为：

$$X_i \in \mathbb{R}^{P_t\times P_h\times P_w\times C} \tag{3.61}$$

将其展平后，可以使用线性层进行投影：

$$z_i = W\,\mathrm{Flatten}(X_i) + b \tag{3.62}$$

其中：

$$z_i \in \mathbb{R}^D \tag{3.63}$$

表示一个 D 维视频 Token。在工程实现中，也可以通过三维卷积实现相应的局部投影。例如，使用卷积核：(P_t, P_h, P_w) 并设置对应步幅，就可以将非重叠时空块映射为特征。Tubelet Embedding 的本质是可学习的时空投影，并不只是对多个视频帧执行简单平均。

时空 Patch 的数量 对于使用非重叠时空 Patch，且各维度能够整除的输入，可以表示为：

$$\frac{T}{P_t}\frac{H}{P_h}\frac{W}{P_w} \tag{3.64}$$

其中：

- P_t 表示时间 Patch 长度。
- P_h、P_w 表示空间 Patch 大小。

例如，输入视频为：$T = 16, \quad H = W = 224$ 设置：$P_t = 2, \quad P_h = P_w = 16$，则：

$$\frac{16}{2} \times \frac{224}{16} \times \frac{224}{16} \tag{3.65}$$

得到：$N_{\text{tube}} = 1568$ 相比逐帧 Patch Embedding 得到的 3136 个 Token，数量减少一半。这一公式针对非重叠、规则划分的理想情况。实际模型还可能使用重叠窗口、填充或不同的时间步幅，因此 Token 数量应根据具体网络参数计算。

视频 Token 数量与计算成本 随着视频帧数和空间分辨率增加，视频 Token 数量也会增长。例如，对于逐帧 Patch Embedding：

$$N_V = T\frac{HW}{P^2} \tag{3.66}$$

其中，P 为正方形 Patch 边长。当空间分辨率保持不变、帧数增加一倍时，Token 数量也增加一倍。如果帧数保持不变，图像高宽均增加一倍，则 Token 数量增加为原来的四倍。因此，如果同时提高时间采样密度和空间分辨率，视频 Token 数量可能迅速增长。例如：16 帧、224×224、16×16 Patch：$N_V = 3136$

32 帧、224×224、16×16 Patch：$N_V = 627216$ 帧、448×448、16×16 Patch：$N_V = 12544$，这些数字说明，视频模型需要同时管理时间维度和空间维度上的 Token 预算。

但 Token 数量更多并不直接等于模型一定具有更好的理解能力。其实际表现还取决于时序编码方式、预训练目标及下游任务。

3.4.3 时间位置编码与帧间关系

为什么视频需要时间位置编码 Transformer 的注意力机制能够在不同 Token 之间建立信息联系。但如果不提供任何位置相关信息，标准自注意力本身并不会直接知道视频帧的实际先后顺序。例如，人物举起手臂和放下手臂可能包含相似的视觉姿态，只是时间顺序相反。如果模型无法区分这些姿态出现的顺序，就可能难以正确判断动作。因此，视频 Transformer 通常需要引入时间位置编码，使模型能够获得 Token 对应的时间信息。

时间位置编码提供的是时间位置线索，并不意味着模型已经自动学会正确的运动关系。真正的时间理解还需要通过网络的信息交互和训练目标形成。

时间位置编码的基本形式 假设逐帧视觉编码器输出：

$$Z \in \mathbb{R}^{T\times N\times D} \tag{3.67}$$

其中：

- T 表示时间位置数量。
- N 表示每帧 Patch 数量。
- D 表示隐藏维度。

可以为第 t 个时间位置引入一个时间嵌入：

$$e_t^{time} \in \mathbb{R}^D \tag{3.68}$$

然后将其添加到对应帧的所有 Patch 特征中：

$$z_{t,i} + e_t^{time} \tag{3.69}$$

其中，i 表示帧内空间 Patch 位置。这样，即使两个时间位置具有相似的视觉内容，模型也可以通过时间嵌入区分它们。

空间位置与时间位置的结合 对于视频 Token，模型通常同时需要空间和时间位置信息。例如，可以分别定义：

$$e_i^{space} \in \mathbb{R}^D \tag{3.70}$$

和：

$$e_t^{time} \in \mathbb{R}^D \tag{3.71}$$

那么视频 Token 可以表示为：

$$z_{t,i} + e_i^{space} + e_t^{time} \tag{3.72}$$

其中：

- $z_{t,i}$ 表示原始视频特征。
- e_i^{space} 表示空间位置。
- e_t^{time} 表示时间位置。

这种设计可以使模型同时知道当前 Token 来自哪个时间位置，以及该时间位置中的哪个空间区域。空间和时间嵌入相加只是常见设计之一。实际视频 Transformer 还可能采用联合时空位置编码、相对位置偏置、旋转位置编码或其他方法。

真实时间戳与物理时间间隔 在上一节中，我们介绍了恒定帧率和可变帧率视频。对于具有真实时间戳的帧序列，可以使用：$t_1, t_2, \ldots, t_T$ 描述每帧的显示时间。不同帧之间的实际时间间隔为：$\Delta t_{ij} = t_j - t_i$，

例如，两帧的真实时间戳为：$t_i = 1.0$ s，$t_j = 1.5$ s 那么：$\Delta t_{ij} = 0.5$ s 即使它们在输入序列中相邻，也不应当被解释为相隔一个固定的视频原始帧间隔。对于需要分析运动速度和时间持续关系的任务，真实时间差可能具有重要作用。

因此，一些视频模型会进一步使用时间戳或时间间隔构造时间特征，例如：$e_t = \phi(t_t)$ 或：$b_{ij} = g(t_j - t_i)$，其中，ϕ 和 g 可以是固定映射或可学习函数。这类方法能够为模型提供更明确的物理时间信息。但时间戳输入本身并不保证模型会准确利用时间尺度，仍然需要通过训练与评测验证。

时间位置与运动关系的区别 时间位置编码能够告诉模型不同特征来自何时，但不能直接回答某个对象如何运动。例如，某个时刻的汽车位于：(x_1, y_1) 另一时刻的汽车位于：(x_2, y_2) 即使模型知道两个时间位置的时间戳，也还需要判断它们是否对应同一辆汽车。如果无法建立对象对应关系，模型可能将不同汽车之间的位置变化错误解释为运动。因此，运动建模至少涉及两个方面：

时间关系：不同观测发生在什么时刻。

内容对应：不同时间位置的哪些视觉区域或对象相互关联。

时间位置编码主要提供第一类信息，而第二类关系需要进一步通过时空注意力、运动估计、对象跟踪或其他机制建立。

3.4.4 空间建模与时间建模的结合

视频模型需要同时学习什么 视频中的信息通常可以从空间与时间两个方面分析。空间建模主要关注某个时间位置中的对象、属性、形状和场景结构。时间建模则主要关注这些视觉内容如何随时间变化。例如，对于一段人物拿起杯子的视频：空间建模可以帮助模型识别人物、手部、杯子和桌面。时间建模则需要分析手部运动、接触关系变化以及杯子离开桌面的过程。因此，视频模型需要在一定程度上同时表示静态外观与动态变化。根据空间与时间信息的组织方式，可以形成多种不同架构。

空间编码与时间编码分离 最直观的方式，是先利用图像编码器获得逐帧空间特征，再通过时间模块进行聚合。例如：$Z_t = f_S(I_t)$，$H = f_T(Z_1, Z_2, \ldots, Z_T)$，其中：

- f_S 表示空间编码器。
- f_T 表示时间编码器。

这种方式有利于复用成熟的图像 Backbone。例如，可以先冻结一个预训练 ViT，提取每帧全局特征，然后训练轻量 Transformer 分析帧间关系。但如果任务需要精细的局部运动信息，只使用全局图像特征可能不够。因此，也可以将 Patch 级特征提供给时间模块，使其利用更加丰富的空间结构。

分解注意力的计算复杂度 假设每帧包含 N 个空间 Token，视频共有 T 帧。如果采用联合时空注意力，其主要 Token 交互规模为：$O(T^2N^2D)$，如果使用一种理想化的空间与时间分解方式：先在每个时间位置的 N 个 Token 之间执行空间注意力。再在每个空间网格位置的 T 个时间 Token 之间执行时间注意力。那么主要 Token 交互规模可以写为：$O(TN^2D + NT^2D)$，其中：TN^2D，

对应空间注意力。NT^2D，对应时间注意力。例如，假设视频包含：$T = 16, \quad N = 196$ 个时间与空间位置。联合时空注意力中的位置配对数量为：$(16 \times 196)^2$ 即：$9,834,496$，而上述分解形式中的配对数量为：$16 \times 196^2 + 196 \times 16^2$ 即：$664,832$ 可以看到，分解注意力能够明显减少这一示例中的位置配对规模。

但以上只比较注意力中的位置交互数量，并不等于整个 Transformer 的实际 FLOPs、显存消耗或推理时间按相同比例减少。实际模型还包含特征投影、前馈网络、归一化等模块，并且不同注意力结构可能需要不同的数据布局和算子实现。

局部时空窗口与层级结构 除了联合注意力和空间—时间分解注意力，还可以使用局部时空窗口限制计算范围。例如，一个注意力窗口可以覆盖：若干连续时间位置。每个时间位置中的某个空间区域。这样，模型只需要在局部窗口内建立时空联系。随后，可以通过窗口移动、窗口之间的信息交换或层级特征聚合，逐步扩大模型的时空感受范围。Video Swin Transformer 等模型采用了具有层级结构的局部时空注意力设计。

这种结构能够利用局部时空关系，并通过不同阶段组织多尺度特征。但局部窗口处理可能限制单层网络的直接交互范围。如果任务需要跨越较远空间或较长时间的关系，模型仍然需要通过多层传播、跨窗口交互或其他全局模块获得相应信息。

递归状态与增量式时序建模 除了卷积和自注意力，视频还可以通过递归状态机制进行时序建模。例如，设第 t 帧的视觉特征为：Z_t 模型维护一个历史状态：H_{t-1} 当新的视觉特征到达时，执行状态更新：$H_t = U_\theta(H_{t-1}, Z_t)$，其中，$U_\theta$ 表示状态更新函数。这种结构可以通过循环神经网络、门控更新模块或其他状态空间机制实现。

其基本思想，是通过不断更新内部状态，逐步积累视频中的历史信息。与一次性处理全部视频 Token 的方式相比，递归状态更新比较适合考虑流式输入和增量计算的场景。但状态更新机制是否真正保留长期信息，取决于状态容量、更新规则和训练目标。仅仅维护一个历史张量，并不意味着模型能够可靠记住所有过去发生的事件。因此，这类方法仍然需要通过长时序和跨片段任务验证其表示能力。

双向视频理解与因果视频处理 在离线视频理解中，模型通常可以提前访问完整视频。例如，给定一段已经录制完成的视频，模型可以同时使用某个时间位置之前和之后的帧。这种方式适合视频分类、离线动作识别和完整视频摘要等任务。但在实时视频分析中，未来帧尚未到达。例如，实时监控系统在当前时刻不能依赖几秒钟之后才会出现的图像。因此，需要区分：

双向时序建模： 允许使用当前时间之前和之后的信息。

因果时序建模： 当前表示只能使用当前及过去已经可获得的信息。

两类模型的输入条件不同，因此不能直接在不说明协议的情况下比较时序预测能

力。因果处理通常更符合在线输入的要求，但并不意味着所有使用因果注意力的模型都能够低延迟运行。实际响应时间还受到视频帧采集、特征提取、缓存和输出解码等环节影响。

3.5 长视频的信息组织

3.5.1 均匀采样与自适应采样

长视频采样的基本问题 视频采样的目标，是从原始视频中选择有限数量的视觉观测，以供后续视觉编码器和多模态模型处理。假设一段视频时长为 D 秒，帧率为 f，那么在理想恒定帧率条件下，原始帧数近似为：$N_{\text{raw}} \approx Df$，例如，一段 60 秒、30 FPS 的视频包含约：

$$N_{\text{raw}} = 60 \times 30 = 1800 \tag{3.73}$$

帧。如果每帧经过视觉编码器生成 196 个 Patch Token，且全部保留，那么视频可以产生：

$$N_{\text{token}} = 1800 \times 196 \tag{3.74}$$

即：$N_{\text{token}} = 352800$ 个视觉 Token。对于许多多模态大模型，这样的输入规模会带来较高的计算成本，并可能超出模型允许的上下文长度。因此，在进入复杂视觉编码和语言建模之前，通常需要控制视频帧数。但减少帧数也意味着舍弃部分视觉观测。因此，采样策略实际上决定了模型有机会看到哪些视频内容。

均匀采样的机制 均匀采样（Uniform Sampling）是一种常见且简单的视频处理方式。其基本思想，是沿视频时间轴按照固定间隔选择帧，使采样位置尽可能覆盖整个视频。例如，一段 60 秒视频，每秒采样 1 帧，可以得到约 60 个采样位置。如果每帧产生 196 个视觉 Token，那么最终为：

$$N_{\text{uniform}} = 60 \times 196 \tag{3.75}$$

即：$N_{\text{uniform}} = 11760$ 个视觉 Token。相比全部保留 1800 帧，采样帧数和对应的原始 Patch Token 总数减少为原来的三十分之一。均匀采样的主要优势是实现简单、计算预算容易控制，并且能够提供相对稳定的时间覆盖。尤其对于视频整体场景理解、一般内容概括和变化较为缓慢的事件，均匀采样可以成为合理的基础方案。

基于视觉变化的采样 一种简单的自适应采样方法，是根据相邻帧之间的变化程度进行判断。例如，可以定义：

$$d_t = \frac{1}{CHW}\|I_t - I_{t-1}\|_1 \tag{3.76}$$

其中，I_t 和 I_{t-1} 为相邻视频帧。当差异分数较大时，说明画面发生了较明显的像素变化，可以增加附近的采样密度。除了像素差，还可以利用颜色直方图、视觉编码器特征或轻量运动估计方法构造变化分数。例如：

$$d_t = 1 - \mathrm{sim}(z_t, z_{t-1}) \tag{3.77}$$

其中，z_t 表示第 t 帧的视觉特征。像素变化和语义重要性并不相同。例如，摄像机抖动可能引起较大像素变化，但不一定包含重要事件。相反，远处交通灯变化或某个人按下按钮，可能只影响少量像素，却对任务有意义。因此，基于变化的采样能够改善部分时间覆盖问题，但不能直接等同于语义关键事件选择。

任务驱动的采样 对于多模态大模型，用户问题可以进一步参与视频帧选择。例如，同一段视频可能支持不同问题："视频中主要出现了哪些人物？""人物什么时候拿起红色杯子？""屏幕上的错误代码是什么？" 对于第一个问题，模型可能更加关注人物出现的代表性时间位置。对于第二个问题，需要关注杯子及其相关动作发生的时间区间。对于第三个问题，则可能需要保留某些分辨率较高、包含文字细节的画面。因此，可以根据查询 Q 和视频内容 V 构造采样策略：

$$\mathcal{S} = F_\theta(V, Q, B) \tag{3.78}$$

其中：

- $\mathcal{S}$ 表示选择的帧或时间片段集合。
- Q 表示当前任务或用户问题。
- B 表示可用的计算或采样预算。

这种方式可以称为查询驱动或任务驱动的视频采样。但查询驱动采样也存在一个实际问题：如果在采样前完全不知道视频内容，就很难直接确定哪些帧与问题相关。因此，系统通常需要某种低成本的初步浏览、特征提取或索引机制，再根据问题决定是否进一步读取特定片段。

采样预算与时间覆盖 自适应采样还需要避免一个问题：全部采样帧都集中在少数剧烈变化的时间区域。例如，一段 10 分钟视频中，只有第 2 分钟发生了明显动作。如果完全依据运动分数选择帧，可能过度关注这一分钟，而忽略其他时间出现的重要静态信息。因此，可以将视频划分为若干时间区间，并为不同区间分配基本采样预算。例如：

$$\sum_{i=1}^{M} n_i \le B \tag{3.79}$$

其中：

- M 表示时间区间数量。
- n_i 表示分配给第 i 个区间的采样帧数。

- B 表示总帧预算。

一种简单策略是保证每个区间至少具有一定的基础覆盖，再将剩余预算分配给更值得关注的区域。这种方式能够在一定程度上平衡时间覆盖和内容重要性。

3.5.2 分段处理与层级摘要

为什么需要视频分段 当视频时长进一步增加时，仅通过采样减少帧数可能仍然不够。例如，一段一小时的视频，即使每秒只采样一帧，也会产生约：3600 个采样帧。如果每帧生成 196 个视觉 Token，那么会形成：$3600 \times 196 = 705600$ 个视觉 Token。对于许多多模态模型，这仍然是较大的输入规模。因此，可以先将长视频划分为多个较短片段，对每个片段分别处理，再逐步整合其内容。

例如，将一小时视频按照每段一分钟划分为若干片段。这样，每个局部计算单元所需要直接处理的时间范围可以得到控制。

固定时长分段 假设视频总时长为：D 每段时长为：L，如果不设置重叠，并且按固定时长划分，那么片段数量近似为：

$$M \approx \left\lceil \frac{D}{L} \right\rceil \tag{3.80}$$

例如，一段 600 秒的视频，每个片段长度为 30 秒，则可以划分为：$M = 20$ 个片段。随后，每个片段可以分别执行视频采样、视觉编码和局部内容理解。这种方式实现简单，便于控制单次模型输入大小。但视频中的动作和事件并不一定恰好发生在分段边界之内。例如，一个重要动作可能从第 29 秒开始，持续到第 33 秒。如果视频按照 0-30 秒和 30-60 秒分段，该动作便被切分到了两个片段中。

因此，固定分段需要进一步处理边界连续性。

片段级表示 对于每个视频片段，可以使用视频编码器提取局部特征：$Z_i = f_\theta(C_i)$，其中：

- C_i 表示第 i 个视频片段。
- Z_i 表示该片段对应的特征。

这些特征可以具有不同粒度。例如，可以使用单个全局向量：

$$z_i \in \mathbb{R}^D \tag{3.81}$$

概括片段整体内容。也可以保留多个具有时间与空间位置的 Token：

$$Z_i \in \mathbb{R}^{N_i \times D} \tag{3.82}$$

以支持后续的细粒度分析。不同表示方式会影响信息保留程度和计算成本。例如，单个全局向量比较紧凑，但通常难以直接定位片段内部的具体事件；保留局部 Token 则能够提供更丰富的信息，但也会增加存储和后续计算开销。

层级视频表示 层级表示（Hierarchical Representation）是一种适合长视频的信息组织思路。例如，可以将长视频组织为三个层次。第一层是帧级表示，记录局部图像特征。第二层是片段级表示，概括局部动作和事件。第三层是视频级表示，整合全局主题与长距离事件关系。可以概括为：

$$\text{Frames} \rightarrow \text{Segments} \rightarrow \text{Video} \tag{3.83}$$

在特征层面，可以表示为：$f_{\text{local}}(C_i)$ 然后：

$$f_{\text{global}}(Z_1^{segment}, \ldots, Z_M^{segment}) \tag{3.84}$$

这种结构使模型不必始终在同一个计算模块中处理全部原始时空 Token。但层级聚合过程中可能逐步丢失信息，因此需要根据下游任务决定保留哪些细节。

摘要中的误差传播 层级摘要还可能存在误差逐层传播的问题。例如，第一个片段的真实内容是：“讲解者将学习率从 0.001 调整为 0.0001。”但局部摘要错误写为：“讲解者提高了学习率。”如果后续全局摘要直接依赖这条错误内容，就可能继续生成：“模型通过提高学习率改善了训练效果。”此时，局部错误被传递到更高层，并可能进一步影响事件解释。因此，层级摘要不能只考虑信息压缩率，也需要控制事实错误。

一种基本思路是保留摘要与原始时间片段之间的对应关系，使系统在回答具体问题时，能够返回原始视频核验。

3.5.3 长上下文与视觉 Token 压缩

长视频为什么带来上下文压力 在多模态大模型中，视频通常会被转换为一系列视觉特征，再通过连接模块提供给语言模型。例如，假设一段视频包含：T 个采样帧，每帧产生：N_p 个视觉 Token。如果全部保留，那么视频 Token 数量为：$N_V = TN_p$，如果再加入文本 Token，则统一序列长度可以表示为：$N = N_V + N_T$，其中，N_T 表示文本 Token 数量。

当视频很长时，视觉 Token 可能占据整个输入序列的主要部分。这不仅会受到模型上下文长度限制，还会增加语言模型预填充阶段的计算、注意力处理以及缓存等成本。因此，如何在不显著损害任务所需信息的条件下减少视频 Token，成为长视频多模态模型的重要研究问题。

上下文窗口不等于实际理解能力 大语言模型的上下文窗口（Context Window）描述其能够处理的输入 Token 长度范围。但更长的上下文窗口，并不自动意味着模型能够有效利用其中的全部信息。例如，模型可能在结构上允许输入数十万个 Token，却仍然难以准确找到视频较早位置的一个短暂事件。类似地，当大量视频帧包含重复内容时，虽然它们占用了上下文预算，但未必为当前问题提供相同比例的新增信息。因此，需要区分：

输入容量：模型能够接收多少 Token。

信息覆盖：输入 Token 是否包含目标任务所需的证据。

信息利用：模型是否能够从这些 Token 中正确检索、关联和推理。

三者之间存在联系，但不能互相替代。

视觉 Token 压缩的机制 视觉 Token 压缩（Visual Token Compression）旨在减少进入后续模型的视觉特征数量。例如，一帧图像经过 ViT 后可能产生：196 个 Patch Token。通过聚合或选择，可以将其减少为：32 个 Token。如果视频包含 60 帧，那么原始视觉 Token 为：

$$N_{\text{before}} = 60 \times 196 = 11760 \tag{3.85}$$

压缩后为：

$$N_{\text{after}} = 60 \times 32 = 1920 \tag{3.86}$$

因此，进入后续语言模型的视觉 Token 数量明显减少。但压缩会改变表示形式，并可能损失部分局部空间信息。例如，如果某帧中只有一块较小区域包含用户关心的文字，那么过度压缩可能使相关细节难以读取。因此，视觉 Token 压缩需要根据任务决定信息保留方式。

时间维度的压缩 视频具有较强的时间冗余，因此可以首先在时间维度减少输入。例如，可以删除相邻时间位置中高度相似的帧，或者对相邻帧的视觉特征进行聚合。假设原始采样得到：

$$Z_V \in \mathbb{R}^{T \times N \times D} \tag{3.87}$$

经过时间压缩后：

$$Z'_V \in \mathbb{R}^{T' \times N \times D} \tag{3.88}$$

其中：$T' < T$，这种方式减少了时间位置数量。但删除时间帧可能损失动作变化和事件顺序信息。例如，人物拿起杯子与放下杯子的过程中，某些关键时间位置可能决定动作方向。如果这些帧被删除，模型可能只观察到相似的起止画面。因此，时间压缩通常需要兼顾时间覆盖与变化检测。

空间维度的压缩 除了减少视频帧数，还可以减少每帧中的空间 Token。例如，一帧具有：$14 \times 14 = 196$ 个 Patch Token。可以通过空间池化等方式，将其转换为：$7 \times 7 = 49$ 个聚合特征。这样，空间 Token 数量减少为原来的四分之一。但空间池化可能削弱对象边界、小目标和文字细节。

例如，一张视频帧中，远处交通标志可能只占据较少的像素区域。如果压缩时将其与大量背景特征聚合，模型可能难以保留足够的辨识信息。因此，空间压缩不仅涉及数量减少，还涉及细粒度视觉信息如何保持。

压缩发生在哪个计算阶段 理解视频高效表示时，一个特别重要的问题是：压缩操作发生在计算链路的什么位置？可以概括为三个主要层次。

第一层：视频解码与帧选择阶段。

在读取原始视频时，只解码或保留部分帧，能够减少后续需要处理的视觉输入。

第二层：视觉编码器内部。

通过局部注意力、层级特征或 Token 选择，尝试降低视觉编码过程中的部分计算成本。

第三层：视觉编码器输出之后。

将完整视觉编码器输出压缩为较短特征序列，再输入多模态连接模块或语言模型。第三种方法能够减少语言模型接收的视觉 Token，但通常不能节省已经完成的视觉编码计算。因此，不同方法虽然都可以被称为视频 Token 压缩，但其实际节省的计算阶段并不相同。

3.6 音视频同步与对应关系

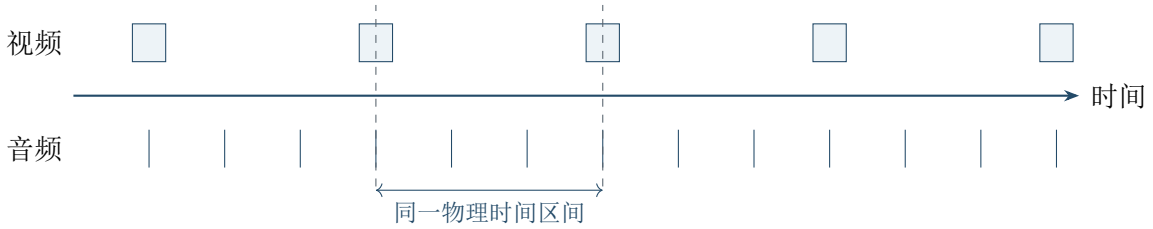


图 3.1 音视频的不同采样粒度。序列索引不同仍可落在同一时间区间；时间偏移和可见性需另行建模。

3.6.1 音频与视频的时间同步

音视频为什么需要时间同步 视频与音频通常同时记录现实世界中的事件。例如，视频画面中有人讲话，与此同时麦克风记录到相应的语音；画面中的车辆撞击障碍物时，音频中可能出现碰撞声。对于这些任务，模型不仅需要识别画面中的对象和音频中的声音，还需要判断它们是否发生在相应时间。例如：“画面中的人说话时，背景中是否有汽车鸣笛?”这个问题涉及两个模态中的时间事件。如果音频和视频存在明显错位，模型可能将某个时刻的声音错误关联到另一个时刻的视觉对象。

因此，音视频同步（Audio-Visual Synchronization）是多模态时间理解的重要基础。

音频与视频具有不同采样频率 音频和视频的一个显著区别，是它们通常具有不同的数据采样率。例如，音频可以按照：$f_s = 16000$ Hz 采样，即每秒产生 16000 个波形采样点。而视频可能采用：$f_v = 25$ FPS 即每秒包含 25 帧画面。在理想固定帧率、同一起始时间的情况下，每个视频帧间隔为：

$$\Delta t_v = \frac{1}{25} = 0.04 \text{ s} \tag{3.89}$$

对应的音频采样跨度为：$N_a = f_s \Delta t_v$ 即：$N_a = 640$ 也就是说，每经过一个视频帧间隔，音频大约产生 640 个新采样点。如果音频进一步采用 10 ms 帧移的 Log-Mel 特征，那么相邻视频帧间隔约对应 4 个声学特征步长。但声学特征本身可能使用重叠窗口，其实际分析范围通常大于帧移。

此外，这种整数比例仅适用于相应的固定采样率组合。对于其他帧率，特别是非整数帧率或可变帧率视频，应当使用真实时间戳建立对应关系。

时间戳与统一时间轴 为了将音频与视频放在同一时间尺度上，通常需要为不同模态建立时间戳。例如，第 i 个视频帧对应的时间可以表示为：t_i^v 某个音频特征位置对应的时间为：t_j^a，如果两个位置在时间上足够接近，就可以根据具体任务建立候选关联。例如：

$$j^* = \underset{j}{\operatorname{argmin}} \left| t_i^v - t_j^a \right| \tag{3.90}$$

表示选择与第 i 个视频帧时间最接近的音频特征位置。但这里最近邻时间匹配只是一种简单对齐方法。实际音频事件具有持续时间，而视觉帧也对应有限的曝光或采样区间。因此，对于语音、动作和其他持续事件，更合理的方式可能是考虑一个时间窗口或区间，而不是只选择单个最近的音频位置。

基于时间窗口的音视频对应 假设某个视频帧的时间为：t_i^v 可以在其附近选择一个音频时间窗口：

$$[t_i^v - \delta_1,\ t_i^v + \delta_2] \tag{3.91}$$

其中，δ_1 和 δ_2 为窗口前后范围。这样，模型可以利用相应时间范围内的音频特征，与当前视觉帧建立联系。例如，在唇语辅助语音识别中，某个视频帧上的嘴部形态可能与邻近时间的多个音频声学帧相关。窗口长度并不是越大越好。如果窗口过短，可能无法覆盖完整声学事件。如果窗口过长，又可能引入其他无关声音，削弱时间对应的准确性。

此外，严格流式系统通常无法访问尚未到达的未来音频，因此还需要根据允许的前视时间设置窗口。

音视频联合语音识别 音视频联合语音识别（Audio-Visual Speech Recognition，AVSR）同时利用声音和嘴部运动等视觉信息，预测说话内容。其基本过程可以表示为：$Z_A = f_A(A)$，$Z_V = f_V(V)$，随后，通过融合模块获得：$H = F(Z_A, Z_V)$ 再进一步生成文本：$Y = g(H)$，其中，F 可以通过特征拼接、注意力或其他方式组合视听信息。

AV-HuBERT 是具有代表性的音视频自监督表示学习工作。它通过多模态掩码预测与隐藏单元学习，探索如何从语音声音和嘴部视频中获得联合表示。音视频联合识

别的主要动机之一，是利用视觉信息弥补部分声学条件下的识别困难。例如，当音频受到较强背景噪声干扰时，嘴部运动可能为某些发音提供额外线索。

但视觉信息并不一定总是有效。例如，当画面中没有正确的说话人、存在配音错位或嘴部区域不可见时，视觉信息可能难以提供帮助，甚至引入错误关联。因此，AVSR需要关注不同模态的可靠性和时间一致性。

3.6.2 视频文本对齐

为什么需要视频与文本对齐 视频自监督学习能够获得一定的时空特征，但这些特征不一定直接对应人类语言中的概念。例如，一段视频可能记录：“人物拿起杯子，然后倒水。”一个视频编码器可以学习相关视觉变化，但如果希望用户通过自然语言检索或询问该视频，就需要建立动态视觉内容与语言表达之间的关联。视频—文本对齐（Video-Text Alignment）的主要目标，是让视频表示与相应的自然语言描述具有可学习的语义联系。例如，模型应当能够判断：

“一个人正在骑自行车。”与相应视频具有较强关联，而与“一个人正在游泳”的描述关联较弱。这种能力可以支持文本检索视频、视频检索文本、零样本动作识别，以及后续的视频语言模型。

视频—文本配对数据 视频与文本之间可以建立不同形式的配对关系。例如：

视频标题：对完整视频的简短概括。

视频描述：对视频内容进行自然语言说明。

片段描述：对某个具体时间范围内的事件进行描述。

字幕与转写：记录视频中人物实际说出的语言内容。

这些文本来源能够提供不同类型的训练信号。例如，标题可能包含整体主题，但不一定描述视频中的全部事件。片段描述可能更加接近局部视觉动作。字幕虽然具有时间信息，却不一定直接描述画面中发生的事情。例如，一段纪录片的旁白可能正在介绍历史背景，而画面只是展示相应地点的风景。因此，字幕时间戳与视频时间戳对应，并不意味着字幕文字能够作为当前帧全部视觉内容的准确标注。这一点对于视频—文本预训练的数据构建非常重要。

视频—文本双编码器 一种常见的视频—文本对齐架构，是采用视频编码器与文本编码器分别提取特征。例如：$z_V = f_V(V)$，$z_T = f_T(T)$，其中：

- V 表示输入视频。
- T 表示对应文本。
- z_V, z_T 表示经过必要投影后的特征向量。

如果两种向量具有相同维度，可以通过余弦相似度比较其语义关系：

$$s(V,T) = \frac{z_V^\top z_T}{\|z_V\|_2 \|z_T\|_2} \tag{3.92}$$

这种方式与图像—文本对比学习具有相似结构。但视频编码器需要处理时间信息，因此视频表示的获得过程通常比单张图像更加复杂。例如，可以先编码多个视频帧，再使用时序模块或聚合机制得到视频级表示。

视频—文本对比学习 假设训练批次中具有 B 个视频与文本配对样本：

$$\{(V_i, T_i)\}_{i=1}^{B} \tag{3.93}$$

模型希望正确配对的样本具有较高相似度，而错误配对的样本相似度相对较低。一种视频到文本的对比损失可以表示为：

$$-\frac{1}{B}\sum_{i=1}^{B}\log\frac{\exp(s(V_i, T_i)/\tau)}{\sum_{j=1}^{B}\exp(s(V_i, T_j)/\tau)} \tag{3.94}$$

其中，τ 为温度参数。同样可以定义文本到视频的损失：

$$\mathcal{L}_{T\to V} \tag{3.95}$$

将两者结合，得到双向对比目标：

$$\frac{1}{2}\left(\mathcal{L}_{V\to T} + \mathcal{L}_{T\to V}\right) \tag{3.96}$$

与 CLIP 的图像—文本对比学习相似，这种训练方式能够帮助视频与文本建立共享语义空间。但这里的目标主要描述全局或样本级语义关联。如果需要细粒度时间定位，还需要设计更加适合片段级或时间级对应关系的学习机制。

视频—文本对齐的根本限制 视频与文本之间的语义对应关系通常具有不完整性。例如，一段视频展示：人物先打开柜门。然后从柜中拿出杯子。最后将杯子放在桌面。训练文本可能只写：“一个人在厨房整理物品。”这段描述整体上与视频相关，但没有完整表达动作顺序和具体对象变化。因此，对比学习可能使视频与这段文本具有较高相似度，却不一定迫使模型保留全部事件细节。此外，相同视频可以拥有多种合理描述，而语言描述也可能包含视频没有直接呈现的信息。

因此，视频—文本对齐需要结合不同任务与数据粒度评价，不能将整体相似度直接等同于完整的视频事件理解能力。

习题与讨论

1. 在 16 kHz 采样率、400 点帧长、160 点帧移、不居中且无补零的条件下，写出长度为 $L \geq 400$ 的音频能形成多少完整窗口，并计算各窗口起始时间。
2. 比较连续声学特征、单码本离散 Token 和多码本表示。它们的序列长度与解码目标分别受什么影响？

3. 一段短时事件完全落在两个均匀采样的视频帧之间。区分改进采样、保留高频运动表示与增加推理 Token 三种操作的作用。
4. 设计一个音视频相对偏移实验。保持内容不变，只改变偏移量，并提出能检测模型是否真正利用同步关系的评价方式。

文献导读 wav2vec 2.0 和 HuBERT 用于比较语音自监督目标，Whisper 用于比较监督转录路线。不要把转录准确率直接当作保留全部声学信息的证据。[7, 6, 9]

第四章　空间、动作与异步传感器表示

本章为选读。三维、机器人与事件数据要求比普通图文更明确的坐标、单位和时间约定。位置可以用连续值或离散 Token 表示，但编码形式不能替代物理含义；一个数值序列只有在参考系、采样与控制接口确定后才具有可解释性。

学习目标　追踪坐标变换方向与单位；区分状态、观测和动作；为异步传感器建立时间与空间对应。

4.1　三维观测与坐标变换

采用齐次变换时，用 $T_{A\leftarrow B}$ 表示把 B 坐标转换为 A 坐标。列向量约定下，$p_A = T_{A\leftarrow B}p_B$，组合为 $T_{A\leftarrow C} = T_{A\leftarrow B}T_{B\leftarrow C}$。读到只写 T_{AB} 的论文时，应先核对作者的方向和左右乘约定，再代入公式。

4.1.1　深度图、点云与三维网格

从二维图像到三维空间　普通 RGB 图像通常采用二维像素网格表示：

$$I \in \mathbb{R}^{H\times W\times 3} \tag{4.1}$$

其中，H 和 W 分别表示图像高度和宽度，3 表示颜色通道数量。每个像素主要记录对应图像位置的颜色信息。虽然图像中包含透视、遮挡、纹理和阴影等深度线索，但像素坐标本身并不能直接确定物体在三维空间中的真实位置。例如，两辆实际尺寸不同的汽车，如果位于距离相机不同的位置，可能在图像中呈现相近的像素宽度。因此，从二维图像恢复三维结构，需要引入额外的几何约束、观测信息或模型先验。

常见的三维信息来源包括双目相机、深度相机、激光雷达、多视角图像以及学习式深度估计等。不同来源提供的几何信息精度、范围和稠密程度可能存在明显差异。

深度图　深度图（Depth Map）是一种按照二维图像网格组织的深度表示。例如：

$$D \in \mathbb{R}^{H\times W} \tag{4.2}$$

其中，(D(u,v)) 表示像素位置 ((u,v)) 对应的深度值。在常见的针孔相机模型中，深度图的值可以定义为对应三维点沿相机光轴方向的坐标：$D(u,v) = Z_c$，其中，Z_c 是相机坐标系中的深度坐标。

沿光轴方向的深度并不等于相机光心到三维点的欧氏距离。

后者通常称为距离或量程，其值为：$r = \sqrt{X_c^2 + Y_c^2 + Z_c^2}$ 只有位于相机主光轴上的点，两者才在理想模型下相等。不同传感器和数据集可能采用不同的深度定义，因此在处理深度数据时需要首先确认其几何含义。

点云 点云（Point Cloud）是三维空间中一组点的集合。可以表示为：

$$P = \{p_i\}_{i=1}^{N} \tag{4.3}$$

其中：$p_i = (x_i, y_i, z_i)$ 表示第 i 个三维点。在实际应用中，每个点还可能具有额外属性，例如颜色、反射强度、时间戳和语义标签。例如：$p_i = (x_i, y_i, z_i, r_i, g_i, b_i)$ 表示具有 RGB 颜色的三维点。与深度图不同，点云不要求各个点排列在规则的二维像素网格中。因此，点云可以表达更加灵活的空间结构，例如来自激光雷达的非均匀三维采样。

点云的优势与局限 点云能够直接表达空间位置，因此适合三维对象识别、三维检测、点云分割和空间测量等任务。例如，自动驾驶系统可以利用激光雷达点云分析道路、车辆与周围障碍物的位置关系。但点云也具有明显局限。点云的空间分布通常不规则，不同区域的采样密度可能存在差异。点云一般只记录被观测到的表面位置，不一定包含物体内部和被遮挡区域的信息。点云中的点本身通常不包含明确的表面连接关系。

因此，仅根据点云坐标不一定能够直接确定连续物体表面，也不能简单假设空间中没有采样点的位置一定是空闲区域。这一区别对于三维重建与空间占据推断十分重要。

三维网格 三维网格（3D Mesh）是一种使用顶点、边和面描述三维表面结构的数据表示。例如，一个三角网格通常由顶点集合与三角面片集合构成：

$$M = (\mathcal{V}, \mathcal{F}) \tag{4.4}$$

其中：

$$\mathcal{V} = \{v_i\} \tag{4.5}$$

表示三维顶点，而：

$$\mathcal{F} = (i, j, k) \tag{4.6}$$

表示由三个顶点索引组成的三角形面片。与点云只提供离散点位置不同，三维网格还显式记录了顶点之间的表面连接关系。例如，一个立方体可以通过若干顶点和三角面片近似描述其表面。因此，网格适合进行表面渲染、几何编辑、碰撞计算及部分物理仿真。

从离散表示到连续空间表示 深度图、点云、网格和体素都可以看作三维几何的不同离散表示。此外，也可以通过连续函数描述空间结构。例如，符号距离函数（Signed Distance Function，SDF）可以描述空间位置与物体表面之间的有符号距离：

$$f(\mathbf{x}) = d_{\text{signed}}(\mathbf{x}, \mathcal{S}) \tag{4.7}$$

其中，$\mathcal{S}$ 表示物体表面。通常可以通过：$f(\mathbf{x}) = 0$ 定义表面位置。距离函数的正负号方向需要事先约定，不同实现可能采用不同符号规范。在三维重建中，截断符号距离函数（Truncated Signed Distance Function，TSDF）可以将距离信息限制在一定范围，并通过融合多次深度观测逐步建立表面表示。这类方法能够支持多视角深度融合和三维表面重建。

但距离场本身与对象语义并不是同一类信息，仍需进一步建立语义关联。

4.1.2 坐标系、位姿与空间变换

为什么需要坐标系 三维点的位置通常不是孤立存在的，而是相对于某个参考坐标系进行描述。例如，一个物体相对于相机可能位于：$(1, 0, 3)$ 的位置。但如果将参考系换成房间的世界坐标系，同一个物体的坐标可能变为另一组数值。因此，三维坐标并不具有脱离参考系的唯一数值形式。在多视角视觉和多传感器融合中，必须明确每组坐标对应哪个参考系。例如，常见坐标系包括：

世界坐标系（World Coordinate System）：用于描述场景中的统一空间位置。

相机坐标系（Camera Coordinate System）：通常以相机光学中心为参考原点。

机器人或设备坐标系（Body Coordinate System）：用于描述机器人或设备自身的空间位置与姿态。

传感器坐标系（Sensor Coordinate System）：例如激光雷达、深度相机或其他传感器的局部参考系。

这些坐标系之间需要通过明确的几何变换建立联系。

三维点的坐标表示 设三维空间中的一个点为：

$$\mathbf{p} = \begin{bmatrix} X \\ Y \\ Z \end{bmatrix} \tag{4.8}$$

其中，X、Y、Z 分别表示三个坐标轴上的位置分量。例如，在某个相机坐标系中，假设 Z 轴沿光轴正方向，则：

$$\mathbf{p}_c = \begin{bmatrix} 0 \\ 0 \\ 2 \end{bmatrix} \tag{4.9}$$

表示该点位于相机正前方、沿光轴方向距离为 2 个坐标长度单位的位置。如果坐标采用米作为单位，则可以解释为 Z 轴方向 2 米。不同计算机视觉、机器人和图形学系统采用的坐标轴方向不一定相同。例如，某些相机坐标系使用 Z 轴指向前方，而某

些设备坐标系可能使用 X 轴作为前进方向。因此，三维几何计算必须首先统一坐标轴方向、单位和左右手系约定。

齐次坐标与变换矩阵 为了将旋转和平移统一为矩阵运算，通常使用齐次坐标。三维点可以写为：

$$\tilde{\mathbf{p}} = \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \tag{4.10}$$

对应的刚体变换矩阵为：

$$T = \begin{bmatrix} R & \mathbf{t} \\ 0\ 0\ 0 & 1 \end{bmatrix} \tag{4.11}$$

其中：

$$T \in \mathbb{R}^{4\times 4} \tag{4.12}$$

于是，空间变换可以写为：

$$\tilde{\mathbf{p}}' = T\tilde{\mathbf{p}} \tag{4.13}$$

这种表示能够将旋转和平移统一为一次矩阵乘法，便于组合多个坐标变换。合法的三维刚体变换属于特殊欧氏群 (SE(3))。本节只需要掌握其基本几何含义，关于旋转参数化和李群优化等更加深入的内容，可以在专门的三维几何与机器人课程中展开。

变换矩阵的方向约定 空间变换中最容易出现的问题之一，是没有明确矩阵的方向。为了避免混淆，可以采用如下约定：

$$\tilde{\mathbf{p}}_A = T_{AB}\tilde{\mathbf{p}}_B \tag{4.14}$$

其中，T_{AB} 表示将点的坐标**从 B 坐标系转换到 A 坐标系**。例如：

$$\tilde{\mathbf{p}}_w = T_{wc}\tilde{\mathbf{p}}_c \tag{4.15}$$

表示将相机坐标转换为世界坐标。反过来：

$$\tilde{\mathbf{p}}_c = T_{cw}\tilde{\mathbf{p}}_w \tag{4.16}$$

表示将世界坐标转换为相机坐标。两者满足：

$$T_{cw} = T_{wc}^{-1} \tag{4.17}$$

部分软件或论文采用其他下标命名习惯，因此实际使用时必须检查变换矩阵的具体定义。尤其要区分相机在世界坐标系中的位姿，与将世界坐标变换到相机坐标系的外参矩阵。

坐标变换的组合 如果需要在多个参考系之间转换，可以组合相应的变换矩阵。例如，某个点首先从 C 坐标系转换到 B 坐标系：

$$\mathbf{p}_B = T_{BC}\mathbf{p}_C \tag{4.18}$$

然后从 B 坐标系转换到 A 坐标系：

$$\mathbf{p}_A = T_{AB}\mathbf{p}_B \tag{4.19}$$

因此：$T_{AC} = T_{AB}T_{BC}$，这说明，多个坐标变换可以通过矩阵乘法组合。但矩阵乘法通常不满足交换律，因此：$T_{AB}T_{BC}$ 不能随意交换顺序。在实际系统中，可以通过明确的坐标系标记、变换方向和单位约定，减少矩阵组合错误。

多传感器标定 当系统同时使用相机、激光雷达和其他传感器时，各个传感器通常具有自己的局部坐标系。例如，激光雷达点可以表示为：$\mathbf{p}_L$，而相机中的对应点表示为：$\mathbf{p}_C$，如果已知激光雷达到相机的外参：T_{CL}，则：

$$\tilde{\mathbf{p}}_C = T_{CL}\tilde{\mathbf{p}}_L \tag{4.20}$$

随后，可以使用相机内参 K 将三维点投影到图像中。这一过程是将激光雷达几何信息与二维视觉特征建立对应关系的基础。多传感器融合还需要考虑时间同步。即使空间外参准确，如果相机与激光雷达在不同时间采集动态场景，直接投影仍然可能产生错位。因此，可靠的多传感器对应关系需要同时考虑空间标定、时间同步和传感器测量误差。

4.1.3 三维几何与二维视觉特征

为什么要结合二维视觉与三维几何 二维图像与三维几何信息具有不同优势。RGB 图像包含丰富的颜色、纹理和视觉语义信息，适合对象识别、场景分类和细粒度外观分析。三维点云和深度信息则能够提供更加明确的空间位置、距离和形状线索。例如，在室内环境中，RGB 图像能够帮助模型识别桌子、椅子和杯子。而三维几何表示能够进一步帮助模型分析这些物体之间的高度、距离和空间位置。因此，二维视觉与三维几何信息具有互补性。

在多模态感知中，可以通过投影、反投影和特征关联，将二维语义表示与三维空间结构联系起来。

二维语义特征 前面第二章介绍了 CNN 和 Vision Transformer[1] 等视觉编码器。对于输入图像：

$$I \in \mathbb{R}^{H\times W\times 3} \tag{4.21}$$

视觉编码器可以提取特征图或 Patch Token：$F_I = f_\theta(I)$，例如：

$$F_I \in \mathbb{R}^{H'\times W'\times D} \tag{4.22}$$

其中，D 表示特征维度。这些特征可能包含对象类别、局部外观和部分场景语义信息。但单独的二维视觉特征通常与相应图像坐标关联，并不直接给出三维世界坐标。因此，如果希望把二维语义特征放入三维空间，就需要进一步建立二维像素与三维点之间的几何对应关系。

从三维点投影到二维特征图 假设一个三维点表示在世界坐标系中：$\mathbf{p}_w$ 首先通过相机外参变换到相机坐标系：

$$\tilde{\mathbf{p}}_c = T_{cw}\tilde{\mathbf{p}}_w \tag{4.23}$$

随后，利用相机内参投影到图像坐标：$(u, v) = \pi(K, \mathbf{p}_c)$，其中，$\pi$ 表示针孔相机投影。如果图像已经经过视觉编码器，便可以根据投影位置从二维特征图中提取相应特征。例如：

$$f_i = \text{Sample}(F_I, u_i, v_i) \tag{4.24}$$

其中，Sample 可以采用适当的双线性插值等方法。视觉特征图的分辨率可能小于原图，因此还需要根据图像缩放、裁剪和特征步幅，正确转换采样坐标。

三维点云与图像融合 在同时具有 RGB 图像和激光雷达点云的场景中，可以将点云投影到图像，并为三维点分配视觉特征。例如：

$$P = \{\mathbf{p}_i\}_{i=1}^{N} \tag{4.25}$$

经过相机投影后，得到对应图像位置：(u_i, v_i)，随后，从 RGB 图像或视觉特征图中提取对应内容。这样，三维点便可以同时具有空间坐标和视觉语义特征。例如：$\tilde{p}_i = (x_i, y_i, z_i, f_i)$，这类表示能够支持三维目标检测、语义分割及其他跨模态任务。但一个重要问题是：并不是所有三维点都能与当前图像建立有效对应关系。例如，某些点可能位于相机视野之外，或被其他对象遮挡。

因此，在执行特征关联时，需要考虑相机视野、深度一致性和可见性。

多视角几何的一致性 对于同一个静态三维点，来自不同相机视角的观测应当能够通过相机位姿和投影关系建立几何一致性。例如，设同一三维点为：$\mathbf{p}_w$，在第 i 个相机视角中的投影为：

$$\mathbf{u}_i = \pi(K_i, T_{c_i w}\mathbf{p}_w) \tag{4.26}$$

其中，这里将三维坐标变换和投影简写在同一表达式中。如果三维点、相机位姿和内参估计正确，那么投影结果应当与对应图像观测一致。这构成多视角三维重建和相机位姿估计的重要几何约束。但实际场景中还可能存在动态对象。例如，同一辆汽车在不同时间的位置可能发生变化。如果仍然将其当作静态点进行多视角融合，就可能导致几何不一致。因此，多视角几何融合需要区分静态场景结构与动态对象状态。

4.2 姿态、运动与动作接口

动作是策略输出的控制变量，状态是描述系统的变量，传感器读数是受测量过程影响的观测。末端位姿、关节位置、速度或力矩对应不同控制接口；即使都写成同维实向量，也不能直接互换。

4.2.1 关键点、骨架与人体姿态

人体姿态 人体姿态（Human Pose）通常指人体在某个时刻的空间构型，包括不同身体部位的位置、方向及相互关系。例如，一个人可能处于站立、坐下、弯腰或抬手的姿态。在计算机视觉中，人体姿态经常通过一组预定义关键点进行近似描述。例如，常见关键点可以包括：头部或鼻子、肩膀、肘部、手腕、髋部、膝盖和脚踝等。这些关键点的位置可以用于描述人体的主要空间结构。

与完整的人体三维网格相比，关键点表示更加紧凑，适合姿态估计、动作识别和人体运动分析等任务。但关键点通常不能完整表达人体表面、衣物、手指细节以及其他复杂几何信息。因此，人体关键点是一种结构化的简化表示，而不是完整的人体几何模型。

二维人体关键点 二维人体姿态估计（2D Human Pose Estimation）主要预测人体关键点在图像平面上的位置。例如，对于一张 RGB 图像，模型可以预测 J 个关键点：

$$P^{2D} = \{p_j\}_{j=1}^{J} \tag{4.27}$$

其中：$p_j = (u_j, v_j)$ 表示第 j 个关键点的二维像素坐标。如果使用 17 个关键点，则可以将姿态表示为：

$$P^{2D} \in \mathbb{R}^{17\times 2} \tag{4.28}$$

在实际系统中，还可以为关键点附加置信度：$p_j = (u_j, v_j, c_j)$，其中，c_j 表示模型对该位置预测的置信程度或相关评分。不同数据集和模型可能使用不同的人体关键点定义。例如，有的关键点体系更加关注人体主要关节，有的则包含更加细致的手部、面部或足部关键点。因此，比较或融合不同人体姿态数据时，需要明确关键点数量、名称与索引约定。

二维姿态到三维姿态 如果输入只有一张普通 RGB 图像，那么从二维关键点恢复三维姿态通常存在几何歧义。在理想针孔相机模型下，像素坐标与三维点之间的关系为：

$$u = f_x \frac{X}{Z} + c_x \tag{4.29}$$

$$v = f_y \frac{Y}{Z} + c_y \tag{4.30}$$

仅根据：(u, v) 通常无法唯一确定：(X, Y, Z) 因为不同深度位置上的三维点可能投影到同一个像素。因此，单目三维姿态估计需要利用人体骨架结构、关节运动约束、训练数据先验和时间信息等辅助推断。例如，模型可以根据二维关节点分布，学习预测相应三维姿态。但是，预测得到的三维姿态并不必然具有完全准确的米制尺度。对于需要绝对距离和真实运动轨迹的任务，还可能需要相机标定、深度信息或其他尺度约束。

根节点相对姿态与全局姿态 为了减少人物整体位置变化对人体姿态分析的影响，可以选择骨盆等位置作为人体根节点。例如，设第 j 个关节在世界坐标系中的位置为：p_j 根节点位置为：r 可以定义根节点相对坐标：

$$p_j^{rel} = p_j - r \tag{4.31}$$

这种表示突出人体各关节之间的相对位置关系。例如，一个人在房间左侧抬手，与其在房间右侧执行相同抬手动作，可能具有相似的根节点相对姿态。因此，相对姿态适合部分动作识别与姿态分析任务。但是，根节点归一化会移除人体整体平移信息。例如，一个人在原地踏步与向前行走，可能存在相似的局部腿部动作，却具有不同的全局位移。如果模型只保留根节点相对姿态，而完全舍弃根节点轨迹，就可能难以区分这些运动。

因此，对于完整的人体运动表示，通常需要同时考虑局部关节运动与整体位移。

人体姿态表示的局限 关键点和骨架能够显著减少人体几何表示的复杂度，但也会舍弃一部分信息。例如，只有主要关节坐标时，模型可能无法准确分析细粒度手指操作、面部表情或衣物变化。另一方面，某些动作还与周围对象密切相关。例如，两个人可能具有非常相似的身体姿态，但一个人手中拿着杯子，另一个人正在使用工具。如果只使用人体骨架，不包含物体信息，就可能难以区分动作的具体语义。

因此，人体姿态是动作分析的重要基础，但对于复杂的人—物交互任务，还需要结合图像外观、对象状态与场景几何信息。

4.2.2 运动轨迹与状态变化

运动轨迹 运动轨迹（Trajectory）描述某个对象、人体根节点或其他参考位置随时间变化的空间路径。例如，一个人在房间中从门口走向桌子，可以通过其位置序列表示运动轨迹。假设某个对象在时间 t 的位置为：

$$r_t \in \mathbb{R}^3 \tag{4.32}$$

那么轨迹可以写为：

$$\mathcal{T} = \{(t_1, r_1), \ldots, (t_T, r_T)\} \tag{4.33}$$

这里同时记录时间与空间位置。这样，轨迹不仅可以描述对象经过哪些位置，还能够进一步估计运动速度和时间关系。对于二维视频跟踪，也可以使用：

$$r_t \in \mathbb{R}^2 \tag{4.34}$$

表示图像平面中的轨迹。但二维像素轨迹与真实三维空间轨迹并不完全相同。

路径与轨迹的区别 路径（Path）主要描述对象经过哪些空间位置。轨迹（Trajectory）通常进一步包含时间信息。例如，两个人可能沿同一条道路从 A 点走到 B 点，但一个人用时 10 秒，另一个人用时 30 秒。两者可能具有相近的空间路径，却具有不同的运动轨迹。因此，轨迹能够进一步表达运动的速度、停顿和节奏。对于机器人运动规划，路径规划和轨迹规划也具有相关但不同的目标。

路径规划主要寻找空间上可行的运动路线，而轨迹规划还需要考虑时间、速度、加速度及其他动力学限制。

位置、朝向与速度状态 对于一个在三维空间中运动的对象，仅使用中心点位置有时不足以描述其完整状态。例如，一辆汽车的空间位置相同，但车头朝向不同，其后续运动可能具有明显差异。因此，可以使用更加完整的运动状态：$S_t = (r_t, R_t, v_t, \omega_t)$，其中：

- r_t 表示位置。
- R_t 表示空间朝向。
- v_t 表示线速度。
- ω_t 表示角速度。

对于人体，还可以进一步加入局部关节姿态与其他信息。这种状态表示能够在一定程度上描述对象当前的运动情况。但在真实感知系统中，上述变量往往不是直接完整观测得到的，而是通过图像、深度、惯性传感器或历史信息估计。因此，需要区分真实物理状态与模型估计状态。

坐标系对轨迹的影响 运动轨迹必须在明确参考系下解释。例如，一个人站在原地不动，但相机正在向左移动。此时，人物在图像中的位置可能不断向右变化。如果直接将像素运动理解为人物在世界坐标系中的运动，就会产生错误。因此，对于需要估计真实空间运动的任务，需要进一步考虑相机位姿。假设对象位置在相机坐标系中为：$p_{c,t}$ 相机到世界坐标系的变换为：$T_{wc,t}$ 那么可以表示为：

$$\tilde{p}_{w,t} = T_{wc,t}\tilde{p}_{c,t} \tag{4.35}$$

通过这一过程，可以将不同时间的观测转换到统一参考系。但实际结果仍然受到相机位姿估计和三维位置估计误差影响。

状态变化与动作语义 运动轨迹描述对象的位置变化，而动作语义还可能涉及对象状态的变化。例如，“打开抽屉”不仅是手部发生位移，还要求抽屉从关闭状态变为打开状态。因此，可以将动态场景中的对象状态表示为：

$$S_t^{obj} \tag{4.36}$$

并描述其状态转移：

$$S_{t+1}^{obj} = F(S_t^{obj}, a_t, \xi_t) \tag{4.37}$$

其中：

- a_t 表示动作或作用。
- ξ_t 表示其他未显式建模的影响因素。

这里的表达用于说明动作与状态变化之间的关系，并不意味着实际场景能够由这个简单函数完整描述。例如，机器人执行“推动杯子”动作，可能导致杯子移动，也可能因接触失败而没有发生预期变化。因此，动作指令、执行过程和最终状态需要分别观察和验证。

4.2.3 动作表示与语言指令的关联

为什么要将动作与语言联系起来 人体动作和自然语言都可以表达行为与事件信息，但它们的表示形式不同。例如，一段人体运动序列可以描述某个人伸出右手、拿起杯子并将杯子移到嘴边的过程。而自然语言可以用一句话概括：“人物拿起杯子喝水。”前者强调具体的空间位置、关节变化和时间过程，后者则强调行为的语义内容。因此，动作与语言之间可以建立跨模态关联。这类关联能够支持动作识别与描述、文本驱动的动作生成，以及机器人或虚拟人物的语言交互等任务。

动作到语言 动作到语言（Motion-to-Language）主要根据人体运动或场景动作生成自然语言描述。例如，输入一段人体骨架运动：$M = (P_1, P_2, \ldots, P_T)$ 模型输出：“一个人向前行走，然后转身。”可以表示为：

$$P(Y \mid M) \tag{4.38}$$

其中：

- M 表示输入运动序列。
- Y 表示动作描述文本。

这种任务与视频描述生成具有联系，但两者的输入信息可能不同。例如，视频描述模型可以利用人物外观、物体和背景场景，而仅使用人体骨架的模型主要依赖身体

运动。因此，对于涉及具体物体类别的动作描述，骨架表示可能存在信息不足的问题。例如，模型可以根据动作推测“人物正在举起某个物体”，却未必能够仅凭人体骨架确定物体是一只杯子还是一部手机。

动作与语言的语义对齐 动作语言模型可以分别构建动作编码器和文本编码器：$z_M = f_M(M)$，$z_T = f_T(Y)$ 然后通过对比学习等方式，使语义对应的动作与文本表示相互接近。例如：“人物坐下。”应当与坐下动作的运动表示具有较高相似度，而与跑步动作相对较弱。这种学习方式与图像—文本、音频—文本对齐具有共同思想。但动作语言对齐具有特殊问题。例如，动作语义往往受到身体姿态变化、运动方向、持续时间和对象交互影响。

如果模型只对整个运动序列进行全局平均，可能难以区分：“先举手再转身。”与：“先转身再举手。”因此，动作—语言对齐不仅需要关注动作类别，还需要根据任务保留相应的时间结构。

连续运动与离散 Motion Token 与图像、音频类似，人体运动既可以使用连续数值表示，也可以进一步转换为离散 Token。例如，一个运动序列可以表示为：

$$M \in \mathbb{R}^{T \times D} \tag{4.39}$$

其中，D 表示人体姿态和其他运动特征的维度。通过运动编码器和向量量化等方法，可以将连续运动转换为离散索引：$C = (c_1, c_2, \ldots, c_L)$，其中：$c_i \in \{1, \ldots, K\}$ 表示某个运动码本中的索引。随后，模型可以将这些 Motion Token 与文本 Token 共同用于序列学习。

MotionGPT 是这一思路的代表性工作之一。它通过离散运动表示构建运动词表，探索将人体动作与自然语言纳入相对统一的序列建模框架。例如，模型可以根据文字生成运动 Token，也可以根据运动 Token 输出动作描述。但 Motion Token 不是自然语言单词，也不一定与某个独立动作一一对应。一个 Token 可能编码某段局部运动模式，而一项完整动作可能需要多个 Token 表示。

动作目标与机器人控制信号 人体运动生成模型通常输出人体关节姿态或运动序列。但真实机器人的动作指令可能是另一种形式。例如，对于机械臂，控制目标可能包括：关节角度。末端执行器目标位置、末端执行器姿态、夹爪开合状态。或其他设备支持的控制量。因此，可以将某个简化动作表示为：

$$a_t = (\Delta p_t, \Delta R_t, g_t) \tag{4.40}$$

其中：

- Δp_t 表示末端位置变化。
- ΔR_t 表示末端姿态变化。
- g_t 表示夹爪控制量。

但是，具体动作空间由机器人硬件和控制接口决定。对于不同机器人，同一语言

目标可能需要完全不同的动作序列。例如，人形机器人拿起杯子的关节运动，与机械臂拿起杯子的运动通常不能直接使用相同的关节参数。因此，文本驱动人体动作生成与机器人真实动作控制是相关但不同的研究任务。

闭环动作执行 在实际机器人系统中，动作执行通常不能只依赖一次性生成的完整运动序列。例如，机器人准备抓取杯子时，杯子可能被其他人移动。如果机器人仍然按照原先生成的动作执行，就可能抓取失败。因此，机器人通常需要根据实时观测更新动作。可以将其概括为：

$$a_t = \pi_\theta(O_{\leq t}, Y, S_t) \tag{4.41}$$

其中：

- $O_{\leq t}$ 表示当前及过去已经获得的观测。
- Y 表示语言指令。
- S_t 表示当前估计的相关状态。
- a_t 表示下一步动作。

这种形式体现了语言目标、环境观测和动作控制之间的联系。但使用这一公式并不意味着模型已经具有完整的闭环控制能力。实际执行还需要考虑控制频率、设备约束、安全机制和状态估计误差等问题。

4.3 惯性与机器人状态

4.3.1 IMU、加速度与角速度

惯性测量单元 惯性测量单元（Inertial Measurement Unit，IMU）是一类能够测量物体惯性运动相关信息的传感器组件。典型的六轴 IMU 包含三轴加速度计和三轴陀螺仪。其中，加速度计测量沿不同坐标轴方向的比力，陀螺仪测量物体绕不同坐标轴的角速度。某些设备还会组合磁力计，提供与地磁场相关的测量信息。这类组合有时被称为九轴惯性传感器模块。磁力计并不是所有 IMU 都必须具备的组成部分。

IMU 广泛应用于移动机器人、无人机、自动驾驶、手机、运动捕捉和其他动态系统。它的主要特点是能够以相对较高的采样频率提供局部运动信息。但 IMU 并不能直接提供完整、无漂移的绝对位置和姿态。要根据惯性测量估计机器人运动状态，还需要结合运动模型、初始化条件以及其他观测。

IMU 的局部坐标系 IMU 测量值通常定义在传感器自身的坐标系中。例如，可以记 IMU 坐标系为：

$$\mathcal{F}_I \tag{4.42}$$

世界坐标系为：

$$\mathcal{F}_W \tag{4.43}$$

IMU 测得的三轴加速度和角速度，通常首先表示在：

$$\mathcal{F}_I \tag{4.44}$$

之下。因此，如果机器人安装 IMU 时采用了不同的方向，同一真实运动可能产生不同的轴向数值。例如，机器人向前加速时，某个 IMU 的 X 轴可能出现明显测量变化，而另一种安装方式下可能主要体现在 Y 轴。因此，使用惯性数据前，需要明确传感器坐标轴、安装方向和单位。如果 IMU 与机器人机体坐标系不一致，还需要利用相应外参建立变换关系。

从角速度估计姿态 机器人在三维空间中的朝向可以使用旋转矩阵、四元数或其他方式表示。例如，设：$R_{WI,t}$ 表示时刻 t 的 IMU 到世界坐标系的旋转。如果角速度在 IMU 坐标系中表示，那么可以在较小时间步长下根据角速度更新姿态：

$$R_{WI,t+1} \approx R_{WI,t} \operatorname{Exp}\left([\boldsymbol{\omega}_t^I]_\times \Delta t\right) \tag{4.45}$$

其中：

- Δt 表示时间间隔。
- $[\boldsymbol{\omega}]_\times$ 表示对应角速度向量的反对称矩阵。
- Exp 表示从旋转代数到旋转矩阵的指数映射。

这里采用右乘形式，是因为角速度被定义在 IMU 自身坐标系中。实际计算还需要根据陀螺仪偏置对角速度进行修正。对于初学者，可以先通过二维平面旋转理解这一过程。例如：

$$\theta_{t+1} \approx \theta_t + \omega_t \Delta t \tag{4.46}$$

表示根据角速度和时间间隔更新旋转角度。但在完整三维旋转中，不同旋转轴之间的关系更复杂，不能始终将三个欧拉角独立累加视为严格的姿态积分。

IMU 积分为什么会产生漂移 理论上，如果准确知道初始姿态、速度、位置以及所有惯性测量，可以通过积分估计后续运动状态。例如，在忽略部分离散化误差的情况下，可以根据补偿重力后的加速度更新速度：

$$\mathbf{v}_{t+1} \approx \mathbf{v}_t + \mathbf{a}_t^W \Delta t \tag{4.47}$$

再通过速度更新位置：

$$\mathbf{p}_{t+1} \approx \mathbf{p}_t + \mathbf{v}_t \Delta t + \frac{1}{2}\mathbf{a}_t^W \Delta t^2 \tag{4.48}$$

然而，真实 IMU 测量存在偏置和噪声。例如，如果加速度计某一轴存在恒定偏置：$b_a = 0.05\ \mathrm{m/s^2}$ 即使机器人实际静止，错误地把该偏置当成真实线加速度，也可能产生不断增长的位置误差。在理想一维、初始位置和速度均为零的简化条件下：

$$\Delta p(t) = \frac{1}{2} b_a t^2 \tag{4.49}$$

当：$t = 10\ \mathrm{s}$ 时：$\Delta p = 2.5\ \mathrm{m}$，这说明，即使加速度偏置看似很小，经过较长时间积分后也可能产生明显的位置漂移。在真实系统中，陀螺仪偏置还可能造成姿态误差，进一步影响重力补偿，因此误差传播会更加复杂。

采样频率、时间戳与同步 IMU 通常可以使用较高采样频率。例如，一个示例系统可以设置：

- 相机：20 Hz。
- 机器人关节状态：100 Hz。
- IMU：200 Hz。

这些数值仅用于说明不同传感器可能具有不同采样率，并非所有设备的固定配置。在这种情况下，IMU 在相邻相机帧之间可以提供多次惯性观测。但传感器数据到达计算机的时间，不一定等于实际测量发生的时间。例如，通信、驱动程序和数据缓存可能引入不同程度的延迟。因此，在多传感器融合中，应当尽可能使用可靠的测量时间戳，并明确不同设备是否共享同一时间基准。如果直接按照消息到达顺序进行数据配对，就可能把不同时间的机器人运动与图像错误关联。

4.3.2 关节状态、机器人位姿与控制指令

机器人内部状态 机器人不仅需要感知外部环境，还需要感知自身的运动与构型。例如，一台机械臂需要知道各关节当前角度、转动速度以及末端执行器所在的位置。移动机器人则可能需要知道基座位姿、运动速度以及自身与周围障碍物之间的关系。这种关于机器人自身的信息，通常与本体感知（Proprioception）有关。在机器人系统中，本体感知可以来自关节编码器、IMU、电机传感器、力矩传感器和其他内部测量设备。

本体感知并不一定能够完整描述机器人周围的环境，因此仍然需要视觉、深度等外部感知信息。

关节位置 对于具有 n 个关节自由度的机器人，可以使用关节位置向量：

$$\mathbf{q} = [q_1, q_2, \ldots, q_n]^\top \tag{4.50}$$

描述其关节构型。对于旋转关节，关节位置通常是角度，常用单位为弧度。对于移动关节，关节位置通常是沿相应轴向的位移，常用单位为米。例如，一台六关节机械臂可以使用：

$$\mathbf{q} \in \mathbb{R}^6 \tag{4.51}$$

表示六个关节的位置。但关节向量的长度不等于机器人末端任务空间的自由度定义。例如，一台具有七个关节自由度的机械臂，也可以控制末端执行器在三维空间中的位置和方向。因此，机器人关节自由度与末端位姿的自由度需要分别理解。

关节状态向量 可以将关节位置、速度与其他内部信息组织为机器人状态向量。例如：

$$s_t = (\mathbf{q}_t, \dot{\mathbf{q}}_t, \boldsymbol{\tau}_t) \tag{4.52}$$

对于 n 个关节，如果三类变量均可用，那么可以形成长度约为 3n 的数值表示。但实际机器人并不一定提供所有这些量。例如，某些系统可能只读取关节角度和速度，或者只能得到估计的关节作用力矩。因此，实际模型需要明确哪些状态是直接测得的、哪些是估计的，以及哪些数据可能缺失。

关节状态与机器人末端位姿 关节状态描述机器人内部构型，而机器人末端位姿描述工具或末端执行器在某个参考系中的位置和方向。例如，机械臂的末端执行器可以是夹爪、吸盘或其他工具。设机械臂关节位置为：$\mathbf{q}$，则可以通过正向运动学计算末端相对于基座的位姿：$T_{BE} = FK(\mathbf{q})$，其中：

- B 表示机器人基座坐标系。
- E 表示末端执行器坐标系。
- FK 表示正向运动学。

如果机器人基座相对于世界坐标系的位姿为：T_{WB} 那么末端在世界坐标系中的位姿为：$T_{WE} = T_{WB}T_{BE}$，这一关系与 6.1 节介绍的坐标变换具有直接联系。例如，当机器人基座移动时，即使机械臂关节位置没有变化，末端在世界坐标系中的位置也可能发生变化。

状态测量与控制指令 机器人系统中一个特别重要的区别，是实际测量状态与期望控制状态并不相同。例如，关节当前角度为：$q_t = 0.5$ rad 控制器接收到的目标角度为：

$$q_t^{target} = 1.0 \text{ rad} \tag{4.53}$$

并不意味着该关节已经运动到了：1.0 rad 实际关节角度可能需要经过一定时间才能逐渐接近目标。如果受到负载、关节限位或其他因素影响，甚至可能无法达到目标。因此，需要区分：

Measured State：传感器测得的实际状态。

Estimated State：根据多种观测估计的状态。

Commanded State：控制系统希望达到的目标状态。

三者可能具有不同数值，也可能对应不同时间位置。

机器人状态记录的基本要求 机器人关节和控制数据通常具有明确的物理单位。例如，旋转关节位置与速度可以分别使用 rad 和 rad/s，移动关节可以分别使用 m 和 m/s。在数据处理中，还需要记录每个状态值对应哪个关节。例如，不同机器人或驱动接口可能采用不同的关节排列顺序。因此，不能只根据向量的第一个元素就假设它始终对应某个固定关节。此外，实际数据记录还应当区分：关节名称。实际测量值和命令值。采样时间戳。机器人基座和末端参考坐标系。

控制模式及相关单位。这些元信息直接影响不同机器人状态能否正确解释和跨平台使用。

4.3.3 连续数值信号与离散 Token 表示

机器人状态为何主要是连续数值 机器人内部状态通常具有明确的数值和物理含义。例如：某个关节角度为：$q = 0.75$ rad 机械臂末端位置为：$p = (0.4, 0.2, 0.5)$ mIMU 某轴角速度为：$\omega_z = 0.15$ rad/s，这些数值不是普通的自然语言概念，而是对应实际物理量。它们之间还存在几何、运动学和动力学关系。

因此，对于机器人状态表示，模型需要尽可能保持必要的数值精度、时间顺序与物理单位。如果直接将所有连续数值转换为普通文本字符串，再使用语言 Tokenizer 处理，可能引入较高序列成本，并不一定适合精确数值控制。

连续状态向量 一种直接的方法是将机器人各类数值状态组织为连续向量。例如：

$$s_t = [\mathbf{q}_t, \dot{\mathbf{q}}_t, \mathbf{p}_t, \boldsymbol{\omega}_t, \ldots] \tag{4.54}$$

然后使用相应的数值编码器映射到隐藏空间：$z_t = f_\theta(s_t)$，其中：

$$z_t \in \mathbb{R}^D \tag{4.55}$$

可以供后续 Transformer、时序网络或其他模型使用。例如，使用多层感知机（MLP），就可以将低维机器人状态映射为与语言模型或视觉模型兼容的隐藏维度。但特征维度一致并不意味着机器人状态已经获得正确的语义解释。例如，模型必须经过适当训练，才能理解某个数值代表关节角度、末端位置还是传感器加速度。

连续状态的时序编码 机器人状态通常以较高频率持续产生。例如，关节状态和 IMU 可能在一段时间内形成大量时间采样。可以将其组织为：$S = (s_1, s_2, \ldots, s_T)$ 然后使用时序卷积、Transformer、循环模型或其他连续状态编码方法提取特征。例如：$Z = f_{\text{temporal}}(S)$，这样，模型不仅能够读取某个时刻的状态，还能够分析机器人运动趋势、关节变化和不同传感器之间的时间联系。

但模型输入序列的时间位置不一定对应固定的真实时间间隔。如果设备存在丢包或非均匀采样，应当保留相应时间戳或时间差，避免把所有相邻样本简单视为等间隔观测。

均匀量化 假设一个连续动作变量的范围为：$x \in [x_{\min}, x_{\max}]$ 将其均匀划分为 K 个区间，则每个区间宽度为：

$$\Delta = \frac{x_{\max} - x_{\min}}{K} \tag{4.56}$$

可以用区间编号：$c \in \{0, 1, \ldots, K-1\}$ 表示量化结果。例如，可以使用：

$$c = \text{clip}\left(\left\lfloor \frac{x - x_{\min}}{\Delta} \right\rfloor, 0, K-1\right) \tag{4.57}$$

将连续值转换为编号。如果使用区间中心作为反量化值：

$$\hat{x} = x_{\min} + \left(c + \frac{1}{2}\right)\Delta \tag{4.58}$$

那么在输入位于合法范围内、且采用理想均匀量化的情况下，最大绝对量化误差不超过：

$$\frac{\Delta}{2} \tag{4.59}$$

超出量化范围的输入如果被裁剪，其误差可能大于这一上界。

基于码本的动作表示 除了对每个数值维度执行均匀量化，也可以使用可学习码本压缩动作特征。例如，先将一段连续动作序列：

$$A \in \mathbb{R}^{H \times d_a} \tag{4.60}$$

通过编码器转换为潜在表示：$Z = f_\theta(A)$，随后，使用向量量化模块将潜在向量映射到某个离散码本。例如：

$$c_i = \underset{k}{\text{argmin}} \|z_i - e_k\|_2^2 \tag{4.61}$$

其中，e_k 表示码本中的第 k 个向量。之后，可以利用动作解码器将离散码本表示恢复为连续动作。这种方式有机会利用多个时间步之间的运动相关性，减少需要生成的离散符号数量。但学习式码本的压缩效果与重建质量受到训练数据、码本容量和动作空间定义影响。因此，较短的动作 Token 序列不一定意味着控制更加准确。

状态 Token 与动作 Token 的区别 在多模态机器人模型中，还需要区分状态表示与动作表示。

状态 Token 主要编码机器人当前已经观测到或估计到的信息。

例如，当前关节角度、机器人位姿和传感器特征。

动作 Token 主要编码希望机器人执行的动作或控制目标。

例如，下一时间步的末端位置增量或未来动作片段。这两类 Token 即使具有相同隐藏维度或采用相似的量化方式，其语义也不相同。例如，一个关节角度状态值：$q_t = 0.5$ rad，与一个目标指令：

$$q_t^{target} = 0.5 \text{ rad} \tag{4.62}$$

虽然数值相同，但前者描述实际状态，后者描述期望控制目标。因此，训练机器人多模态模型时，需要明确每个数值和 Token 的角色，防止模型将观测值和动作命令混为一谈。

4.4 事件流的生成与编码

4.4.1 事件相机的工作原理

传统相机如何记录视觉信息 传统帧相机（Frame-based Camera）通常在特定曝光和读出机制下，按照设定频率输出图像。例如，假设一台相机的帧率为：$f = 30$ FPS 那么理想情况下，相邻帧的时间间隔约为：

$$\Delta t = \frac{1}{30} \approx 33.3 \text{ ms} \tag{4.63}$$

图像可以表示为：

$$I_t \in \mathbb{R}^{H \times W \times 3} \tag{4.64}$$

其中，每个像素保存相应的颜色信息。这种方式的优势是能够提供较完整的空间外观，适合对象识别、语义分割和其他视觉理解任务。但对于快速运动，成像过程中仍然可能面临一些困难。例如，当物体在单次曝光期间发生较大位移时，图像可能出现运动模糊。如果某个短暂事件完全发生在两帧采集之间，也可能缺乏足够的直接视觉观测。

传统相机并非都采用相同的曝光与读出方式。全局快门、滚动快门、短曝光与高速帧相机等技术，均会影响其运动成像能力和实际时间精度。因此，事件相机与传统相机的区别首先是视觉信号的采集机制不同，而不只是帧率数值不同。

事件相机的机制 事件相机通常由大量能够独立检测局部亮度变化的像素构成。与传统相机在统一时间产生完整图像不同，典型事件相机中的每个像素持续监测本地的亮度变化。当某个像素检测到的亮度变化达到预设的对比度阈值时，便产生一个事件。如果该像素的亮度没有达到新的变化阈值，则通常不会因为经过了一个固定时间间隔而自动产生事件。例如，一幅静止的室内场景中，桌面、墙壁和椅子均保持不动，照明也基本稳定。

在理想无噪声条件下，事件相机可能只产生很少的事件，甚至完全不产生事件。当人物从画面中走过时，人体边缘附近的像素亮度可能发生变化，从而触发一系列事件。

因此，事件相机关注的是局部亮度何时发生足够变化，而不是周期性记录整个画面的绝对亮度。

对数亮度表示 事件相机的理想触发模型通常建立在对数亮度或对数光强之上。设某个像素位置为：$\mathbf{x} = (x, y)$，其光强为：$I(\mathbf{x}, t) > 0$ 可以定义对数亮度：

$$L(\mathbf{x}, t) = \log I(\mathbf{x}, t) \tag{4.65}$$

在数值实现中，为避免对零取对数，也可能采用带有适当正数下限的处理方式。使用对数亮度的一个重要特点，是光强的相对变化可以表示为对数差。例如：

$$\log I_2 - \log I_1 = \log \frac{I_2}{I_1} \tag{4.66}$$

因此，事件相机的理想触发条件与相对亮度变化有关，而不是仅由某个固定的绝对光强差决定。

事件触发阈值 设一个像素上次事件对应的参考对数亮度为：L_{ref} 当前对数亮度为：$L(\mathbf{x}, t)$ 那么，可以定义变化量：

$$\Delta L(\mathbf{x}, t) = L(\mathbf{x}, t) - L_{\text{ref}} \tag{4.67}$$

对于一种理想的对称阈值模型，当：$\Delta L(\mathbf{x}, t) = pC$ 时产生事件。其中：

- (C>0) 表示对数亮度变化阈值。
- $p \in \{-1, +1\}$ 表示事件极性。

当：$p = +1$ 时，表示该像素检测到足够大的亮度增加。当：$p = -1$ 时，表示该像素检测到足够大的亮度降低。上述公式描述的是理想阈值到达模型。实际像素可能具有不同的正负触发阈值，并受到噪声、延迟、器件不一致性及其他电路因素影响。因此，并不能认为真实相机中的每个事件都严格对应同一个固定大小的亮度变化。

亮度变化阈值的直观例子 假设某像素的理想事件阈值为：$C = 0.2$ 当对数亮度相对于参考值增加：0.2 时，触发一次正极性事件。由于：

$$\Delta L = \log \frac{I_2}{I_1} \tag{4.68}$$

因此，当：$\Delta L = 0.2$ 时：

$$\frac{I_2}{I_1} = e^{0.2} \approx 1.221 \tag{4.69}$$

也就是说，在这一理想设置中，光强相对于参考值增加约 22.1%，就可以触发一次正极性事件。如果亮度持续增加，像素在更新自身参考状态后，可能再次达到阈值并产生后续事件。类似地，当对数亮度降低到负阈值时，可以产生负极性事件。这一例

子说明，事件相机通常以亮度变化是否达到阈值作为事件触发依据，而不是记录某一时刻的完整亮度数值。

事件相机为何对运动敏感 物体或相机发生运动时，场景中的亮度结构可能在图像平面上移动。例如，一个明暗边缘经过某个像素时，该像素的亮度可能随时间发生变化。在满足一定亮度恒常假设的条件下，可以使用光流约束关系：

$$\frac{\partial L}{\partial t} + \nabla L^{\top} \mathbf{v} \approx 0 \tag{4.70}$$

其中：

- (L) 表示对数亮度。
- ∇L 表示图像空间亮度梯度。
- $\mathbf{v}$ 表示图像平面上的运动速度或光流。
- $\partial L/\partial t$ 表示固定像素位置处的时间亮度变化率。

该关系说明，局部运动与图像亮度变化之间存在联系。因此，物体或相机的运动可以通过局部亮度变化触发事件。但是，亮度变化不只可能由运动产生。例如，照明闪烁、屏幕亮度变化或其他光照变化，也可能导致事件产生。同时，如果运动方向与局部亮度梯度的关系使亮度变化较弱，也可能产生较少事件。因此，**事件相机对运动较为敏感，并不意味着每个事件都是直接测量得到的物体运动信息。**

4.4.2 事件流、时间戳与极性

单个事件的数据结构 典型事件相机输出的基本单位是一个事件：$e_i = (x_i, y_i, t_i, p_i)$，其中：

- x_i 表示事件发生的水平像素坐标。
- y_i 表示事件发生的垂直像素坐标。
- t_i 表示事件时间戳。
- p_i 表示极性。

例如：$e_i = (120, 80, 1.2534, +1)$ 可以表示在第 1.2534 秒附近，像素位置 ((120,80)) 产生了一个正极性事件。时间戳的数值单位必须由具体数据接口定义。上述例子以秒表示，但真实事件数据可能使用微秒、纳秒或其他时间计数单位。此外，实际数据集可能采用：$p \in \{0, 1\}$ 等方式编码极性，因此需要根据数据格式转换为所使用的数学约定。

事件流 在一段时间内，事件相机可以产生连续的事件序列：

$$\mathcal{E} = \{e_1, e_2, \ldots, e_N\} \tag{4.71}$$

其中，N 表示这段时间内实际产生的事件数量。与普通视频不同，N 并不由固定

帧率直接决定。例如，在场景基本静止且照明稳定时，可能只产生少量事件。而当画面中出现大量运动边缘或明显亮度变化时，事件数量可能迅速增加。因此，事件流是一种事件数量随场景变化而变化的数据形式。这也说明，不能直接将事件相机的数据率简单描述为固定的每秒帧数。

固定时间窗口 实际深度学习系统通常需要从事件流中选取某个时间范围的数据。例如，可以定义：

$$\mathcal{E}_{[t_0,t_1)} = \{e_i \mid t_0 \le t_i < t_1\} \tag{4.72}$$

表示时间区间：$[t_0, t_1)$ 内的全部事件。如果窗口宽度为：$\Delta T = t_1 - t_0$，则可以按照固定的时间长度组织事件输入。例如，每 10 ms 提取一个事件窗口。固定时间窗口的优点是每次输入具有明确的时间跨度，便于与视频帧和机器人状态进行同步。但窗口内事件数量可能差异很大。例如，低变化场景中的 10 ms 可能只有少量事件，而高速运动场景中的相同时间范围可能包含大量事件。

因此，固定时间窗口能够控制时间跨度，但不能保证每个窗口具有相同的计算负载。

固定事件数量窗口 另一种方式是每次提取固定数量的事件。例如，每次读取：$N = 10000$ 个事件。这样可以使每个事件包包含相同的事件数量。但它对应的实际时间跨度可能不断变化。例如，事件产生频繁时，10000 个事件可能集中在很短时间内。而在低活动场景中，积累相同数量的事件可能需要较长时间。因此，固定事件数量窗口能够控制数据规模，但不一定适合要求稳定时间延迟的任务。特别是在实时系统中，不能仅根据事件数量判断信息的时间新鲜程度。

事件流与 RGB 时间对齐 在 RGB 与 Event 联合感知中，常见情况是 RGB 相机按一定帧率输出图像，而事件相机持续产生异步事件。例如，RGB 帧的时间戳为：t_k 可以根据任务选择其对应的事件窗口：$[t_k - \Delta T, t_k]$，这样的窗口只包含当前时刻及此前已经发生的事件，适合某些因果在线处理设置。但如果是离线分析，也可能使用围绕图像参考时间的双侧窗口：

$$[t_k - \Delta T_1, t_k + \Delta T_2] \tag{4.73}$$

这种窗口可能包含 RGB 参考时间之后的事件。因此，在比较算法时必须说明，事件窗口究竟采用前向、后向还是双侧时间范围。对于严格在线或因果评测，不能无意中使用当前预测时刻之后才会产生的未来事件。此外，RGB 图像本身可能具有非零曝光时间。如果需要精确配准，还应明确图像时间戳表示曝光开始、结束还是其他参考时刻。

4.4.3 事件帧、体素网格与时间表面

为什么需要事件表示转换 事件流是一种非规则、可变长度的数据序列。例如，某个时间窗口可能包含：$N_1 = 5000$ 个事件。另一个时间窗口可能包含：$N_2 = 50000$ 个事件。但传统 CNN 和许多视觉 Transformer 更习惯接收形状固定的张量输入。因此，研究者提出了多种将事件流转换为规则张量的方式。

其中，较为常见的表示包括事件帧（Event Frame）、事件体素网格（Event Voxel Grid）和时间表面（Time Surface）。这些表示都可以从同一段原始事件流构造，但它们保留的信息不同。

表示转换不是无损操作。

如果将高精度事件时间戳累积成固定数量的时间通道，原始事件顺序和时间精度可能受到不同程度的压缩。因此，事件表示方法会直接影响模型能够利用的时间信息。

事件帧 事件帧是将某个时间窗口内的事件按照空间像素位置进行累积，转换为二维图像形式。例如，在时间窗口：$[t_0, t_1)$ 内，可以统计每个像素的事件数量：

$$C(x, y) = \sum_{i=1}^{N} \mathbf{1}[x_i = x, y_i = y] \tag{4.74}$$

其中，$\mathbf{1}[\cdot]$ 表示指示函数。这样，就可以得到：

$$C \in \mathbb{R}^{H \times W} \tag{4.75}$$

的事件计数图。图中较大的数值表示该像素在对应时间窗口内产生了较多事件。这种表示能够把非规则事件流转换为普通二维数组，方便使用 CNN 等视觉编码器处理。但如果只统计事件总数，正负极性和精细时间顺序就会被忽略。

事件帧的局限 事件帧的主要局限是时间聚合。例如，假设某个窗口持续：50 ms，如果只记录窗口内每个像素的正负事件总数，那么模型无法从这一张事件帧中直接知道哪些事件发生在第 5 ms、哪些发生在第 45 ms。因此，事件帧能够表示一定时间内的空间变化分布，但不一定保留精细事件时间顺序。如果需要分析高速运动方向或不同事件之间的微小时间差，仅使用一个较长窗口的事件帧可能不够。

此外，事件帧中的像素值通常表示事件计数或极性累积，而不是实际 RGB 强度。因此，尽管事件帧与普通图像具有类似张量形状，它们的物理意义却完全不同。

事件时间到 Bin 的映射 设某个事件的时间为：t_i 当前时间窗口为：$[t_0, t_1]$ 为了映射到 B 个离散时间位置，可以先定义归一化位置：

$$b_i = (B - 1)\frac{t_i - t_0}{t_1 - t_0} \tag{4.76}$$

其中：$b_i \in [0, B - 1]$，这里采用包含窗口两端的归一化约定，便于说明线性时间插值。实际使用半开窗口时，需要相应处理时间边界。一种简单方法，是把事件分配

给最近的时间 Bin。但这种硬分配会使时间信息受到 Bin 边界影响。例如，两个发生时间非常接近的事件，如果位于边界两侧，可能被分配到不同 Bin。因此，还可以使用连续插值方式，使一个事件向邻近时间 Bin 分配权重。

带时间插值的体素表示 对于事件的归一化时间位置：b_i 可以使用三角形核进行时间插值。例如，对于第 b 个时间 Bin，可以定义：$w(b, b_i) = \max(0, 1 - |b - b_i|)$，这样，一个事件可以根据其时间位置，被分配到一个或两个相邻的 Bin。对于采用正负极性分离的表示，可以写为：

$$V(p, b, x, y) = \sum_{i=1}^{N} \mathbf{1}[x_i = x, y_i = y, p_i = p] w(b, b_i) \tag{4.77}$$

其中：$p \in \{-1, +1\}$，这种方式能够比简单的最近邻时间分配保留更加细致的相对时间位置信息。Zhu 等人的事件视觉研究中使用了类似的插值式时空体积表示，使事件能够输入深度网络进行光流、深度和自运动估计。但插值式体素仍然是对原始连续时间信息进行离散化，不能认为其完全保留了所有原始事件时间戳。

指数衰减时间表面 为了突出最近发生的事件，还可以根据当前参考时间：t_{ref} 构造指数衰减的时间表面：

$$S_p(x, y; t_{\text{ref}}) = \exp\left(-\frac{t_{\text{ref}} - T_p(x, y)}{\tau}\right) \tag{4.78}$$

其中：

- $\tau > 0$ 表示时间衰减常数。
- $T_p(x, y)$ 表示最近一次同极性事件时间。

对于没有有效历史事件的位置，可以将对应值定义为零，或者使用单独的有效性掩码。这种表示使较新的事件具有较高响应，而较早的事件随时间逐渐衰减。例如，若：$\tau = 20$ ms 某个像素最近一次事件发生在 20 ms 之前，则：

$$S = e^{-1} \approx 0.368 \tag{4.79}$$

如果最近一次事件发生在 40 ms 之前，则：

$$S = e^{-2} \approx 0.135 \tag{4.80}$$

因此，时间表面能够较紧凑地描述近期事件活动。HOTS 等研究利用时间表面构建层次化事件特征，用于事件模式识别。

时间表面的特点与局限 时间表面主要保留每个像素最近一次事件的时间，因此具有一定的存储与更新优势。例如，当某个像素产生新事件时，可以直接更新该位置的最新时间戳。但如果同一个像素在短时间内产生大量事件，时间表面通常只保留其中最近的时间信息。因此，它可能丢失事件数量、较早事件的时间分布，以及部分复杂变化

过程。例如，一个像素连续触发 20 次同极性事件，与只在最近时刻触发 1 次事件，在某些时间表面定义下可能得到相同的最新时间值。

因此，时间表面适合强调事件的新近程度，却不一定适合所有需要完整事件历史的任务。

原始事件流与直接异步处理 事件帧、体素网格和时间表面都是常见的事件数据表示，但它们并不是唯一选择。另一类方法直接处理原始事件序列。例如，对于：$e_i = (x_i, y_i, t_i, p_i)$ 可以通过事件编码器获得特征：$z_i = f_\theta(e_i)$，随后，模型可以按事件时间顺序更新内部状态：$H_i = U_\theta(H_{i-1}, z_i)$，这种方式能够在形式上保留逐事件更新能力。

例如，某个事件到达时，可以立即更新相关空间位置或局部状态，而不必始终等待完整的固定时间窗口结束。除此之外，还存在事件点集处理、事件图结构和脉冲神经网络等技术路线。但直接逐事件处理并不一定具有更低计算成本。如果高变化场景在短时间内产生大量事件，逐个运行复杂神经网络同样可能导致较高处理负载。因此，应根据具体任务、事件率和硬件条件选择事件表示与计算机制。

4.5 时间对齐与空间标定

算例：变换链的方向检查

将点云点投影到相机时，应依次完成传感器坐标到相机坐标的外参变换，以及相机三维坐标到像素的投影。检查链路至少需要一个已知位置点、深度符号、单位及图像边界；仅检查矩阵维度相容，不能排除逆变换或单位错误。

4.5.1 时间同步与不同采样频率

为什么需要时间同步 在动态场景中，不同时间的观测可能对应不同的物理状态。例如，一辆汽车正在道路上行驶。RGB 相机在某一时刻拍摄到汽车的位置，激光雷达却在稍后的时间测得汽车的空间坐标。如果车辆在两次观测之间已经发生位移，那么直接把激光雷达点云投影到较早的 RGB 图像上，就可能产生空间错位。这种问题并不一定来自相机标定错误，而可能来自传感器观测时间不一致。因此，多传感器融合首先需要明确每个观测实际发生在什么时刻。

对于某个传感器 s，可以将一条观测表示为：

$$o_i^{(s)} = (t_i^{(s)}, z_i^{(s)}) \tag{4.81}$$

其中：

- $t_i^{(s)}$ 表示观测时间戳。
- $z_i^{(s)}$ 表示传感器观测值。

不同传感器的时间戳需要在可比较的时间参考系下解释。只有明确这些时间信息，才能进一步建立不同观测之间的对应关系。

不同传感器的采样频率 异构传感器通常具有不同的数据产生频率。例如，假设某个机器人系统采用以下配置：以上只是教学示例，并不代表不同类型设备必须采用这些频率。例如，在 100 ms 内，30 Hz 相机通常产生约 3 帧图像，200 Hz IMU 可以提供约 20 次测量，而事件相机可能产生数量不固定的事件。因此，不能简单要求所有传感器在每个时间位置上都具有一条完全对应的观测。实际系统需要根据任务选择统一的参考时刻、时间窗口或异步处理方式。

连续状态的时间插值 对于 IMU、关节编码器等高频连续数值信号，可以根据相邻测量估计某个参考时刻的状态。例如，已知关节位置：$q(t_0),\quad q(t_1)$ 希望估计：$t_0 \le t \le t_1$ 处的关节位置。在适当条件下，可以使用线性插值：

$$\hat{q}(t) = (1-\alpha)q(t_0) + \alpha q(t_1) \tag{4.82}$$

其中：

$$\alpha = \frac{t - t_0}{t_1 - t_0} \tag{4.83}$$

例如：$q(1.00) = 0.4$ rad，$q(1.02) = 0.6$ rad 那么在：$t = 1.01$ s 时，线性插值得到：

$$\hat{q}(1.01) = 0.5 \text{ rad} \tag{4.84}$$

但这一结果依赖局部运动近似线性的假设。如果机器人在短时间内发生明显加减速，插值结果可能存在误差。此外，三维旋转与角度周期变量需要使用适当的表示和插值方法。例如，对于四元数姿态，可以考虑球面线性插值，而不是简单逐元素线性插值后直接视为合法旋转。

插值与严格因果处理 时间插值还需要考虑是否允许使用未来观测。例如，在离线数据处理中，可以使用：$t_0 < t < t_1$ 两侧的观测估计时间 t 的状态。但在严格实时系统中，如果当前时刻为 t，那么：$t_1 > t$ 处的观测尚未到达。因此，直接使用未来采样点完成插值，会引入额外等待时间，或者违反严格因果计算条件。对于实时系统，可以根据任务选择使用最近已获得的观测、历史状态预测，或者允许有限的等待延迟。因此，需要区分：

离线时间对齐：可以在完整记录中使用前后观测。

在线因果对齐：只能使用当前实际已经可获得的信息。

这一差异对于高频事件视觉、实时机器人控制和在线多模态感知尤其重要。

事件流如何与帧数据对齐 事件相机与普通相机的数据产生方式不同。例如，RGB 图像在某个参考时刻：t_k 产生，而事件相机在相邻 RGB 帧之间持续输出事件。因此，通

常不能为每张 RGB 图像简单寻找一个时间完全相同的事件。一种常见方式是构造事件时间窗口：

$$\mathcal{E}_k = e_i \mid t_k - \Delta T < t_i \le t_k \tag{4.85}$$

其中，ΔT 表示窗口长度。然后将窗口内的事件转换为事件帧、体素网格或其他表示，再与 RGB 图像结合。这种方式能够在固定 RGB 参考时刻组织事件数据。但窗口长度会影响运动信息和计算成本。较短窗口具有较细的时间局部性，但可能包含较少事件。较长窗口能够积累更多变化信息，却可能混合不同时间位置的运动过程。

此外，事件窗口也可以与激光雷达、IMU 或机器人状态时间范围关联，不必始终以 RGB 帧作为唯一参考。

4.5.2 空间标定与坐标变换

为什么时间同步后仍然需要空间对齐 即使不同传感器在同一时刻完成观测，它们也可能从不同位置和方向观察场景。例如，一台机器人在机体前部安装 RGB 相机，在顶部安装激光雷达。由于安装位置不同，两种传感器对同一个物体的观测坐标可能不同。例如，激光雷达测得某个三维点：$p_L = (x_L, y_L, z_L)$，而相机使用的是：$p_C = (x_C, y_C, z_C)$，这两组数值通常不能直接视为相同坐标。

因此，需要通过空间标定建立不同传感器坐标系之间的几何关系。

相机内参与传感器外参 空间标定通常涉及内参和外参。

内参（Intrinsic Parameters）描述传感器自身的观测或成像模型。

例如，针孔相机的内参矩阵可以写为：

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \tag{4.86}$$

其中，f_x, f_y 表示焦距参数，c_x, c_y 表示主点坐标。真实相机还可能需要畸变参数。

外参（Extrinsic Parameters）描述不同坐标系之间的相对旋转和平移。

例如，激光雷达与相机之间的外参可以表示为：T_{CL} 外参不仅包含三个平移量，还包含空间方向关系。如果只知道两个传感器相距多远，却不知道它们各自的朝向，通常不足以完成三维点投影。

统一空间变换约定 与 6.1 节保持一致，本节采用：$\tilde{p}_A = T_{AB}\tilde{p}_B$ 的记号。其中，T_{AB} 将 B 坐标系中的点转换到 A 坐标系。对于三维刚体变换：

$$T_{AB} = \begin{bmatrix} & R_{AB} & & t_{AB} \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{4.87}$$

这里：$R_{AB} \in SO(3)$ 表示旋转矩阵，而：

$$t_{AB} \in \mathbb{R}^3 \tag{4.88}$$

表示平移向量。因此：$p_A = R_{AB}p_B + t_{AB}$，例如，激光雷达点转换到相机坐标系：$\tilde{p}_C = T_{CL}\tilde{p}_L$，$T_{CL}$ 和 T_{LC} 的方向相反：

$$T_{LC} = T_{CL}^{-1} \tag{4.89}$$

实际使用标定文件时，应当阅读其变换定义，不能仅凭字段名称猜测矩阵方向。

空间标定误差的影响 即使外参只有较小误差，投影位置也可能发生明显变化。例如，假设一个三维点位于相机前方：$Z_C = 5$ m 相机焦距参数为：$f_x = 400$，如果横向外参误差为：$\Delta X = 0.1$ m 那么在理想针孔模型和其他条件不变的情况下，水平像素误差近似为：

$$\Delta u \approx f_x \frac{\Delta X}{Z_C} \tag{4.90}$$

代入得到：$\Delta u \approx 8$ 像素，这一例子说明，看似较小的空间标定误差，也可能影响图像中的精细空间对应。例如，一个小目标只占据十几个像素宽度时，8 像素的错位可能使点云特征被关联到错误的对象区域。上述近似只考虑了特定横向平移误差。旋转误差及更加一般的几何条件会产生不同形式的投影误差。

图像缩放、裁剪与特征坐标 在深度学习中，图像通常会经过缩放、裁剪或填充。例如，原始图像分辨率为：1280×720 模型输入则可能被调整为：640×360，如果水平和垂直方向均缩小一半，那么像素坐标和相机内参也需要进行相应转换。例如，在忽略坐标约定细节的简化情况下：$f'_x = s_x f_x$，$c'_x = s_x c_x$ 类似地：$f'_y = s_y f_y$，$c'_y = s_y c_y$，

如果还存在裁剪，则主点坐标需要根据裁剪偏移进一步调整。因此，不能直接使用原始分辨率下的内参，将三维点投影到已经经过任意预处理的模型输入图像上。对于视觉特征图，还需要考虑编码器的空间步幅和 Patch 网格定义。这说明，空间对齐不仅发生在原始传感器层面，也需要在模型的数据预处理与特征采样阶段保持一致。

4.5.3 数据缺失、时间错位与噪声

真实传感器数据为什么不完整 理想情况下，多传感器系统可以持续提供准确、同步且完整的观测。但真实系统通常无法完全满足这一条件。例如：RGB 图像可能受到运动模糊或强光影响。深度图可能在透明、反光或超出量程的区域缺少有效值。激光雷达点云可能由于遮挡和采样密度限制而缺少某些表面。IMU 可能受到偏置和振动影响。事件相机可能产生噪声事件，也可能在静态场景中几乎没有有效事件。此外，数据传输过程中还可能出现丢包、延迟或时间戳异常。

因此，多模态系统需要处理的不只是不同模态的数据格式，还包括不同数据源的可靠性。

数据缺失的不同类型 传感器数据缺失可以发生在不同层次。

模态级缺失，指某个传感器暂时无法提供数据。

例如，深度相机发生故障，导致整个时间段没有有效深度输入。

样本级缺失，指某些时间位置的观测不存在。

例如，一段 RGB 视频中丢失了某一帧图像。

区域级缺失，指一条观测内部只有部分空间位置有效。

例如，深度图中的某些像素没有可用深度。

属性级缺失，指某些状态量无法测量或不可用。

例如，机器人能够提供关节位置，但没有有效的关节力矩反馈。这些缺失具有不同原因，也需要不同处理方式。例如，模态级故障可能需要暂时依赖其他传感器，而区域级深度缺失可以根据相应有效性掩码限制几何融合。

时间错位与空间误差 时间错位可能转化为空间对应误差。例如，假设一个物体在世界坐标系中沿某个方向以近似恒定速度：v 运动。如果两种传感器的观测时间相差：Δt 那么在简化的一维条件下，物体对应的位置差可能近似为：$\Delta x \approx v\Delta t$，例如，物体运动速度为：$v = 2$ m/s 时间错位为：

$$\Delta t = 20 \text{ ms} = 0.02 \text{ s} \tag{4.91}$$

则：

$$\Delta x \approx 0.04 \text{ m} \tag{4.92}$$

即约 4 cm。如果物体速度提高到：15 m/s 且时间错位达到：40 ms 那么对应位移可能达到：0.6 m，这说明，在高速动态场景中，几十毫秒的时间误差就可能产生不可忽略的几何错位。这一估计假设近似恒定运动，并没有考虑相机运动、旋转和三维透视关系。实际像素错位需要根据具体成像几何计算。

随机噪声与系统性误差 不同传感器的误差大致可以从随机误差和系统性误差两个方面理解。随机误差可能表现为测量值围绕真实值波动。例如：$z = x + n$，其中：

- x 表示真实物理量。
- n 表示测量噪声。

系统性误差则可能表现为持续偏差。例如：$z = x + b + n$，其中，b 表示偏置。对于 IMU，偏置可能随着时间变化。对于深度相机，误差可能与距离、材质和成像条件有关。对于相机与激光雷达融合，错误的外参可能使大量点产生具有相似方向的系统性投影偏差。因此，随机噪声和系统性偏差通常需要采用不同方式处理。例如，简单平均可能降低部分随机噪声，但通常不能自动消除稳定的标定偏差。

置信度与不确定性 为了描述不同传感器的可靠性，可以为观测附加置信度或不确定性信息。例如，设两种传感器对某个标量状态提供估计：$z_1,\ \ z_2$，如果它们具有已知的

误差方差：$\sigma_1^2,\quad \sigma_2^2$ 并且在相应假设下误差独立、无偏，那么可以使用逆方差加权：

$$\hat{z} = \frac{z_1/\sigma_1^2 + z_2/\sigma_2^2}{1/\sigma_1^2 + 1/\sigma_2^2} \tag{4.93}$$

在这种简化设置中，不确定性较小的观测获得较大权重。例如，如果：$\sigma_1^2 = 1$，而：$\sigma_2^2 = 4$ 那么第一种观测在融合中具有更大的权重。但这种公式依赖明确的误差假设。如果两种传感器存在相关误差、系统性偏置或异常值，直接使用逆方差加权不一定能够获得可靠结果。对于深度学习模型中的置信度分数，也不能简单假设其已经等同于经过校准的测量方差。

时间新鲜度与观测可靠性 在异步传感器系统中，不同模态的最新观测可能具有不同时间新鲜度。例如，当前时刻为：t 某个传感器最近一次有效观测发生在：t_s 可以定义该观测的年龄：$A_s = t - t_s$，其中，假设：$t \geq t_s$，例如，RGB 相机最近一帧距今 25 ms，而事件相机最近有效事件距今 1 ms。这表明两类信息对应的观测时间不同。但观测更加新鲜并不必然代表信息更加可靠。

例如，最新事件可能来自光照闪烁，而稍早的 RGB 图像仍然提供较为稳定的对象语义。因此，时间新鲜度只是判断观测价值的一个维度，不能直接等同于语义正确性或测量可信度。

习题与讨论

1. 证明 $T_{A\leftarrow C} = T_{A\leftarrow B}T_{B\leftarrow C}$，并写出将同一点从 A 转换回 C 的表达。
2. 机器人状态和动作都可以包含关节角。给出一个例子，说明两者在时间、监督和可执行性上的区别。
3. 比较事件帧、体素网格和时间表面分别保留了哪些时间信息。哪些变化无法通过后续语言建模补回？
4. 在同一物理过程由两个不同频率传感器记录的情况下，设计一个无需同频重采样也能描述对应关系的数据结构。

第二部分

对齐、架构与生成

第五章　跨模态对齐与学习目标

对齐学习先确定哪些观测应当关联，再规定模型如何为这种关联打分。监督可以来自配对、区域、时间或生成目标，各自强调不同信息。本章集中建立学习目标，后续训练章只讨论这些目标如何被组织进实际训练。

学习目标　写出双向对比目标并推导相似度梯度；辨认批内负例和假负样本；区分独立检索、融合匹配与条件生成。

5.1　对应关系与表示空间

5.1.1　模态差异与语义鸿沟

跨模态差异包括信号结构、统计分布和语义粒度。图像可包含颜色、纹理和空间关系，配对文本往往只描述其中一部分；文本也可能包含画面无法验证的意图。因而“配对”规定了一种监督关系，没有规定两种观测携带完全相同的信息。

同一幅图可以有多个合理描述，一段描述也可适用于多幅图。Patch 与词语之间通常没有一一映射：一个对象可能跨越多个块，属性可能只对应局部区域。实际目标应明确是整图—整句、区域—短语，还是事件—时间片段的对应。

语义对齐与物理标定解决不同问题。点云投影依赖几何参数和参考系；“汽车”一词与对象的关联依赖语义监督。标定正确不能证明类别判断正确，表示相近也不能证明标定正确。多传感器系统往往同时需要两者。

5.1.2　共享语义空间的基本思想

双编码器分别计算 $h_I = f_I(I)$ 和 $h_T = f_T(T)$，再经投影与归一化得到 $u, v \in \mathbb{R}^d$，满足 $\|u\|_2 = \|v\|_2 = 1$。余弦相似度为

$$s(I, T) = u^\top v \in [-1, 1]. \tag{5.1}$$

图像与文本可独立编码，候选库也可预先建立表示，因此这种结构适合检索。相同维度使点积可计算；相似度的语义仍由训练数据与目标决定。它不是天然校准的事实正确概率。

共享嵌入空间约束跨模态可比较性，融合表示则根据多个输入共同计算状态。前者并不要求模型在每次编码时同时看到另一模态，也不要求丢弃全部模态专属信息。二者应从计算图而非模型命名区分。

5.1.3 全局语义对齐与局部语义对齐

局部语义对齐 局部语义对齐（Local Semantic Alignment）关注图像中的局部区域与文本中的词语、短语或其他语义单元之间的对应关系。例如，在图片中：“一个女孩拿着黄色雨伞站在路边。”可以建立：“女孩”对应人物区域。“黄色雨伞”对应雨伞区域。“路边”对应相应场景区域或关系背景。如果进一步包含：“女孩拿着雨伞。”则还需要判断人物与雨伞之间的交互关系。因此，局部对齐不仅涉及对象名称，还可能涉及属性、空间关系与动作关系。

不是所有语言成分都具有明确的单一区域。例如，“旁边”“正在等待”等表达，可能需要通过多个对象、时间上下文或更广泛的场景信息来解释。

视觉局部表示 局部视觉表示可以采用不同形式。例如，Vision Transformer[1] 可以将图像划分为 Patch Token：$Z_I = (v_1, v_2, \ldots, v_M)$，其中，M 表示视觉 Token 数量。另一种方式是通过对象检测或区域提议等机制获得局部区域特征：$R_I = (r_1, r_2, \ldots, r_K)$，其中，$r_i$ 可以对应图像中的某个候选对象区域。两种表示的语义粒度不同。

一个 Patch Token 不一定完整对应一个对象。而对象区域也可能覆盖多个 Patch，并且区域提取结果本身可能不准确。因此，局部语义对齐既可以在 Patch 级别开展，也可以在候选区域、实例或其他视觉结构上进行。

文本局部表示 文本同样可以构建局部表示。例如，一句话：“一名男孩正在追逐一只白色小狗。”可以通过文本编码器获得 Token 序列：$Z_T = (w_1, w_2, \ldots, w_L)$，其中，每个 w_j 表示文本编码器输出的上下文特征。但模型的文本 Token 不一定等于自然语言中的完整词语。例如，一个较长的中文或英文词组可能被分解为多个子词 Token。

因此，如果需要将“白色小狗”作为完整的对象短语进行对齐，可能需要进一步聚合相关 Token，或者使用特定的短语表示机制。这种语义单位与分词 Token 之间的区别，对细粒度图文对齐十分重要。

局部相似度矩阵 假设图像包含 M 个视觉 Token，文本包含 L 个文本 Token。经过必要投影后，两者具有相同隐藏维度 D：

$$Z_I \in \mathbb{R}^{M \times D} \tag{5.2}$$

$$Z_T \in \mathbb{R}^{L \times D} \tag{5.3}$$

如果特征已经归一化，可以计算：

$$S_{\text{local}} = Z_I Z_T^{\top} \tag{5.4}$$

其中：

$$S_{\text{local}} \in \mathbb{R}^{M \times L} \tag{5.5}$$

矩阵元素：S_{ij} 表示第 i 个视觉 Token 与第 j 个文本 Token 的相似度。例如，与“杯子”有关的文本特征，可能与图像中覆盖杯子区域的若干视觉 Token 具有较高相似度。这种方式提供了构建局部对应关系的基础。但相似度矩阵本身并不能自动保证每个 Token 的语义解释正确。其有效性仍然依赖特征学习和训练目标。

基于局部最大相似度的对齐 一种局部对齐思路，是对每个文本 Token 寻找相似度最高的视觉 Token。例如：$m_j = \max_i S_{ij}$，其中，m_j 表示第 j 个文本 Token 在图像中的最大匹配相似度。随后，可以对文本 Token 的匹配分数进行聚合：

$$s(I,T) = \frac{1}{L}\sum_{j=1}^{L} m_j \tag{5.6}$$

这种方式使整体图文相似度受到局部 Token 匹配情况的影响。FILIP 是采用细粒度 Token 交互进行图文预训练的代表性研究。它使用图像 Patch 与文本 Token 之间的相似度构建后期交互机制，使图文整体匹配能够利用局部语义对应信息。但最大相似度匹配不一定意味着已经找到了准确的对象区域，也不自动保证模型能够理解对象之间的复杂关系。

例如，一个文本 Token 可能与多个相似对象区域相关，而不同文本 Token 也可能匹配到同一视觉区域。因此，局部相似度只是实现细粒度对齐的一类方法。

短语与图像区域对齐 除了 Token 级匹配，还可以使用文本短语与图像区域之间的对应。例如，文本为：“一个穿蓝色上衣的人拿着红色杯子。”可以分解出：“穿蓝色上衣的人”。“红色杯子”。然后分别在图像中寻找对应区域。例如，模型可以为某个短语预测一个包围盒：$b = (x_1, y_1, x_2, y_2)$ 表示其在图像中的位置。这种任务通常与短语定位（Phrase Grounding）或视觉定位（Visual Grounding）有关。

Flickr30k Entities 提供了文本实体提及与图像区域之间的人工对应标注，是相关研究的重要资源。例如，同一张图像可以拥有多段自然语言描述，而部分实体短语能够与具体图像区域建立联系。这种数据有助于研究模型是否真正理解局部词语与视觉区域之间的关系。

局部对齐与视觉定位 局部对齐和视觉定位密切相关，但仍然需要区分训练信号与最终任务。例如，某个模型通过全局图文配对学习到了 Patch 与文本 Token 的局部相似度。这种模型可能在一定程度上显示出局部响应。但如果希望准确输出目标物体的边界框，就需要根据相应的定位任务进行验证。例如，模型可能根据文本“红色杯子”在杯子所在区域产生较高响应，但这不一定意味着它已经能够预测准确的杯子边界。

Grounded Language-Image Pre-training（GLIP）等工作，则进一步将语言条件下的视觉定位与目标检测联系起来，学习对象级语言感知表示。因此，局部语义匹配与精确空间定位之间存在联系，但不能直接视为同一能力。

局部对齐与注意力机制 在多模态 Transformer 中，可以使用交叉注意力（Cross-Attention），使文本 Token 根据视觉特征获取相关信息。例如，文本特征作为查询：

$Q = Z_T W_Q$，视觉特征作为键和值：$K = Z_I W_K$，$V = Z_I W_V$ 那么可以计算：

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \tag{5.7}$$

这种机制允许文本表示根据当前查询，与多个视觉 Token 进行交互。例如，在回答："图像中的红色杯子位于哪里？"时，语言查询可能与杯子相关的视觉特征发生较强交互。但是，**注意力权重并不必然等于真实语义对应关系，也不能直接视为因果解释或精确对象定位结果**。因为注意力分布受到模型结构、特征投影和训练目标影响，某些较高权重可能并不对应人类标注的目标区域。

因此，局部对齐能力仍然需要通过独立的对象定位和关系理解任务验证。

全局与局部对齐的评价差异 对于全局对齐，可以采用图文检索或零样本分类等任务。例如，评价模型是否能够从大量图像中检索出与某段文本相关的图像。对于局部对齐，可以进一步评价文本短语所指向的图像区域是否正确。例如，使用预测区域和参考区域之间的交并比：

$$\text{IoU}(B_p, B_g) = \frac{|B_p \cap B_g|}{|B_p \cup B_g|} \tag{5.8}$$

其中：

- B_p 表示预测区域。
- B_g 表示真实标注区域。

此外，也可以根据具体任务使用指代表达定位准确率、区域召回率等指标。图文检索成绩不能直接替代局部定位成绩。例如，一个模型能够从候选图像中选出含有红色杯子的图片，不意味着它一定能够准确找到图片中红色杯子的位置。因此，评价跨模态对齐时，应当明确对齐目标对应的语义粒度。

5.2 双向对比目标与梯度

5.2.1 正负样本与相似度学习

设一个批次含 B 组配对 (I_i, T_i)。标准一一配对设置把 (I_i, T_i) 作为正对，$(I_i, T_j), j \neq i$ 作为批内负对。这是训练标签，不是对全部未配对组合的语义判断。若多个描述都适用于同一图像，直接将其余描述视作负例就会引入假负样本。

困难负样本与正样本接近，能提供更细的区分信号，也更容易与有效的多正样本混淆。研究负样本策略时，应同时报告候选集合、去重规则和语义冲突处理。

5.2.2 对比损失与双向对比目标

将归一化图像与文本向量堆为 $U, V \in \mathbb{R}^{B\times d}$，定义温度 $\tau > 0$ 和 Logit 矩阵 $A = UV^\top/\tau$。以行作图像查询、以列作文本候选，有

$$\begin{aligned} p_{ij}^{I\to T} &= \frac{\exp(A_{ij})}{\sum_{k=1}^{B}\exp(A_{ik})}, \qquad \mathcal{L}_{I\to T} = -\frac{1}{B}\sum_{i=1}^{B}\log p_{ii}^{I\to T}, \\ p_{ij}^{T\to I} &= \frac{\exp(A_{ij})}{\sum_{k=1}^{B}\exp(A_{kj})}, \qquad \mathcal{L}_{T\to I} = -\frac{1}{B}\sum_{j=1}^{B}\log p_{jj}^{T\to I}, \\ \mathcal{L} &= \tfrac{1}{2}(\mathcal{L}_{I\to T} + \mathcal{L}_{T\to I}). \end{aligned} \tag{5.9}$$

第二个概率按列归一化。忽略其中一个方向，优化的检索条件就不同；矩阵同为 $B \times B$ 并不表示两项损失相同。

对图像到文本方向，令 $s_{ij} = u_i^\top v_j$，交叉熵的导数为

$$\frac{\partial \mathcal{L}_{I\to T}}{\partial s_{ij}} = \frac{p_{ij}^{I\to T} - \mathbf{1}\{i=j\}}{B\tau}. \tag{5.10}$$

正对项推动相似度提高，负对项推动其降低；权重随当前概率而变。温度同时改变概率的集中程度和梯度尺度，不能只描述为“让分类更自信”。双向目标的总梯度还需加上另一方向的贡献。上述导数针对相似度，反传到原始未归一化向量时还要经过归一化的雅可比。

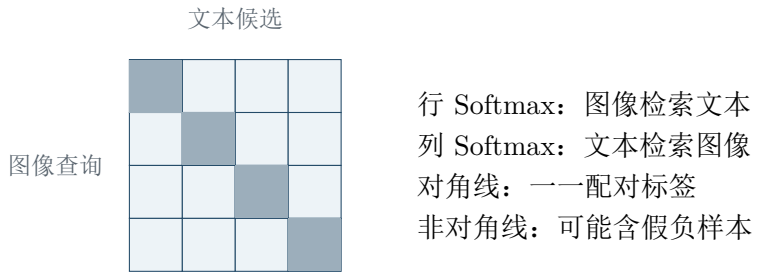


图 5.1 双向对比目标的归一化方向。对角线为监督指定的正对，颜色不表示真实测得的相似度。

SigLIP 使用成对的 Sigmoid 损失，改变了候选间归一化方式；它不等同于把同一 Softmax 公式更名。比较两种目标时，批量、负例、偏置和温度等设置也应明确。

5.3 批内负样本与监督噪声

5.3.1 批量训练与负样本构造

在一一配对批次中，每个查询有一个监督正例和 $B-1$ 个批内负例。$B = 256$ 时，每个方向均有 255 个批内负例；整批需计算一个 $B \times B$ 的相似度矩阵。嵌入计算、相似度计算和通信是不同成本。

多卡训练可以汇集不同设备的嵌入以扩大候选集合，但必须说明汇集操作是否传播梯度，以及本地损失如何归一化。仅累积多个微批次的梯度，不会自动形成跨微批次的相似度矩阵。第十章进一步区分名义有效批量与实际损失的统计单位。

多正样本目标应显式定义正例集合；否则图文重复、近义描述或共同对象会受到错误排斥。困难负例挖掘也需避免将未标注正例当作更有价值的负例。比较策略时，应控制编码器、数据量和训练预算，单独检查细粒度区分与语义召回。

5.4 检索与零样本分类

5.4.1 跨模态检索与零样本分类

给定查询嵌入，按候选相似度排序即可进行跨模态检索。候选库、重复项和每个查询的正例集合属于评价协议；不同候选集合上的 Recall 不能直接横比。

零样本分类把类别名称写入提示模板，编码为类别向量 v_c，然后计算

$$\widehat{c} = \arg\max_{c} u(I)^{\top} v_c, \qquad p(c \mid I) = \frac{\exp(u(I)^{\top} v_c/\tau)}{\sum_k \exp(u(I)^{\top} v_k/\tau)}. \tag{5.11}$$

有多个模板时，应说明在归一化前还是之后进行平均，并在最终相似度前采用一致的归一化约定。零样本指没有使用该下游训练集的类别监督进行适配，不表示预训练从未遇到这些概念。

上述概率依赖当前候选类别和温度。类别集合变化可以改变概率，却不改变已编码图像；因此不能将其直接解释为开放环境下“对象存在”的概率。开放词汇分类、开放集拒识与输出对象边界框是不同任务。

5.5 匹配、掩码与条件生成

对比学习在共享空间中比较候选；匹配学习通常进一步交互两个输入；生成目标预测缺失内容或目标序列。选择目标时，应从监督粒度与推理成本出发。多个目标可以共存，但其损失值不能因都称作“交叉熵”就直接相加而不检查归一化。

5.5.1 图文匹配目标

图文匹配的任务定义 ITM 通常将一组图像与文本作为联合输入，并预测二者是否具有合理的配对关系。设训练样本为：

$$\mathcal{D} = \{(I_i, T_i, y_i)\}_{i=1}^{N} \tag{5.12}$$

其中：

- I_i 表示图像；
- T_i 表示文本；

- y_i 表示图文配对标签。

对于二分类匹配任务：$y_i \in \{0, 1\}$，其中：$y_i = 1$ 表示当前训练协议将图文组合视为匹配样本；$y_i = 0$ 表示当前训练协议将其视为不匹配样本。例如，图像显示一名儿童正在骑自行车，文本“一个孩子骑着自行车”可以作为正样本，而文本“一名儿童正在游泳”可以作为负样本。ITM 中的配对标签通常来自数据标注和负样本构造规则。它不等于对图像与文本全部事实内容进行严格验证。

例如，一段文本只描述图像中的一个主要对象，而没有描述其他背景信息，也可能是合理的正样本。

联合编码与跨模态交互 在典型的双编码器对比学习中，图像和文本可以先独立编码，再通过相似度函数进行比较。ITM 则常采用能够进行跨模态交互的联合编码结构。假设图像编码器输出多个视觉特征：$Z_I = (v_1, v_2, \ldots, v_M)$，文本编码器输出语言特征：$Z_T = (w_1, w_2, \ldots, w_L)$，随后，可以利用多模态编码器形成联合表示：

$$H_{IT} = F_{\text{multi}}(Z_I, Z_T) \tag{5.13}$$

其中，F_{multi} 表示跨模态特征交互模块。这一模块可以采用联合自注意力、交叉注意力或其他结构，使视觉信息与语言信息相互作用。例如，当文本包含“红色雨伞”时，模型可以利用相关视觉区域的信息判断该属性是否正确。当文本包含“男孩站在自行车左侧”时，模型则需要进一步结合人物与自行车的空间关系。因此，联合编码相比简单的全局向量比较，能够为细粒度语义判断提供更加充分的信息交互条件。

但采用跨模态注意力结构并不意味着模型必然掌握精确的对象关系。实际能力还取决于训练数据、任务目标和模型是否真正利用相应视觉证据。

图文匹配分类头 假设多模态编码器输出一个用于匹配判断的联合特征：

$$h_{IT} \in \mathbb{R}^D \tag{5.14}$$

可以通过一个分类头预测图文匹配分数：$a_{\text{ITM}} = W h_{IT} + b$，其中，W 和 b 为可学习参数。使用 Sigmoid 函数后，可以得到：

$$p_{match} = \sigma(a_{\text{ITM}}) \tag{5.15}$$

其中：

$$\sigma(x) = \frac{1}{1 + e^{-x}} \tag{5.16}$$

模型可以据此判断当前图文组合是否属于训练定义下的匹配样本。实际模型也可以使用双类别 Softmax 等方式实现匹配分类。对于二元匹配任务，可以采用二元交叉熵损失：

$$\mathcal{L}_{ITM} = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \log p_i + (1-y_i)\log(1-p_i)\right] \tag{5.17}$$

其中，p_i 表示模型预测的匹配分数。这一损失会鼓励模型对正样本给出较高分数，对负样本给出较低分数。但这里的匹配概率仍然受到训练数据分布和负样本构造方式影响，不能未经校准就直接解释为图文事实正确性的绝对概率。

图文匹配中的负样本 训练 ITM 时，需要同时提供匹配与不匹配的图文组合。最简单的方式是随机替换文本或图像。例如，正样本为：图像：一辆汽车停在路边。文本：“一辆汽车停在道路旁。”随机负样本可以是：图像：一辆汽车停在路边。文本：“一只鸟停在树枝上。”这种负样本具有明显的对象类别差异，通常比较容易区分。但是，如果训练中大量使用这类简单负样本，模型可能主要依据对象类别判断是否匹配，而没有充分学习颜色、数量和关系等细粒度信息。例如，对于正样本：

“一个人骑着自行车。”更加困难的负样本是：“一个人推着自行车。”这两段描述包含相同的人物和物体，但动作关系不同。类似地：“红色杯子在蓝色书本左侧。”与：“红色杯子在蓝色书本右侧。”也只在空间关系上存在差异。这类困难负样本（Hard Negative）能够为更加细粒度的图文匹配提供训练信号。

基于相似度的困难负样本选择 一种常见方式，是先利用已有图文表示计算相似度，再从相似度较高但不属于标注正配对的候选中选择负样本。例如，对当前图像：I_i 可以使用对比式编码器计算：$s(I_i, T_j)$ 然后从其他文本中选择相似度较高的候选，提供给 ITM 模块。ALBEF[10] 等模型采用了利用图文对比相似度进行困难负样本构造的思路。这种方式有助于把训练重点放在较难区分的图文组合上。

但具有较高相似度的未配对文本，也可能实际上能够正确描述图像。因此，困难负样本选择仍然可能引入假负样本问题。例如，一张图像可以同时被描述为“一个人在街道上行走”和“一名行人经过城市道路”。即使数据集中只保留其中一个正配对，另一段文本也可能具有合理的语义对应。因此，ITM 的训练质量不仅取决于模型结构，还取决于负样本构造是否符合真实语义关系。

图文匹配与图文对比学习的区别 虽然 ITM 和图文对比学习都使用正负图文关系进行监督，但它们的计算方式和训练目标不同。典型图文对比学习采用：

$$s_{ITC}(I, T) = \text{sim}\left(f_I(I), f_T(T)\right) \tag{5.18}$$

图像与文本可以独立编码，并在共同嵌入空间中进行比较。而 ITM 通常采用：

$$p_{ITM}(I, T) = g(F_{\text{multi}}(I, T)) \tag{5.19}$$

在联合输入条件下预测匹配关系。因此，两者具有不同特点。对比学习更加适合建立可独立存储和比较的图文表示，便于大规模跨模态检索。联合 ITM 则可以对具体图文组合进行更充分的特征交互，适合候选重排序或精细匹配判断。不过，这一区分并

不是绝对的。例如，局部对比学习也可以利用 Patch 与词语之间的细粒度相似度；ITM 也未必能自动判断所有复杂关系。

图文检索中的两阶段结构 ITC 和 ITM 可以结合，形成一种两阶段图文检索方法。第一阶段，使用双编码器对全部候选图像进行向量检索：

$$\mathcal{C}_K(T) = \mathrm{TopK}_I \; s_{ITC}(I, T) \tag{5.20}$$

得到前 K 个相关候选。第二阶段，再将这些候选图像与查询文本共同输入 ITM 模型，计算匹配分数：$p_{\mathrm{ITM}}(I, T)$ 并据此重新排序。这种方式兼顾了全局向量检索的效率和联合匹配的细粒度交互能力。但第二阶段只能重新排序已经进入候选集合的样本。如果正确图像未被第一阶段召回，后续重排序通常无法恢复。此外，联合编码需要针对每个候选图文对进行计算，因此其成本通常高于直接比较已缓存的全局向量。

5.5.2 掩码建模与条件生成

掩码语言建模 掩码语言建模（Masked Language Modeling，MLM）源自语言模型预训练中的重要思想。设文本 Token 序列为：$T = (w_1, w_2, \ldots, w_L)$ 选择一组待预测位置：

$$\mathcal{M} \tag{5.21}$$

并将这些位置的输入替换为掩码或采用相应的遮挡方式，得到：T_{mask} 模型根据可见上下文预测原始 Token。在视觉语言场景中，可以进一步加入图像条件：I，因此，MLM 目标可以表示为：

$$\mathcal{L}_{MLM} = -\sum_{i \in \mathcal{M}} \log P_\theta \left(w_i \mid T_{\mathrm{mask}}, I \right) \tag{5.22}$$

其中，模型根据图像和未遮挡文本预测指定位置的词语。例如：图像：桌面上有一个蓝色杯子。文本：“桌面上有一个 [MASK] 杯子。”如果模型能够利用图像中的颜色信息恢复“蓝色”，就说明这种训练任务可以为图文属性对应提供学习信号。

语言捷径问题 考虑下面的文本：“太阳从东方 [MASK]。”即使没有图像，模型也可能根据语言常识预测“升起”。如果这类样本在训练中占据较大比例，模型可能主要依赖文本上下文完成预测，而没有充分利用视觉输入。类似地：“一个人在厨房里做 [MASK]。”模型可能根据场景和语言先验预测“饭”，但实际图像可能并未提供明确依据。因此，MLM 损失较低并不意味着跨模态对齐一定充分。

为了检验模型是否真正依赖图像，可以保持文本不变，分别提供正确图像、错误图像和无图像输入，分析预测结果是否合理变化。如果模型在替换图像后几乎没有变化，说明某些预测可能主要来自语言上下文，而不是视觉证据。

掩码视觉建模 除了掩码文本，还可以遮挡图像中的部分内容。例如，先将图像划分为 M 个视觉 Patch：$X_I = (x_1, x_2, \ldots, x_M)$ 再选择其中一部分 Patch 作为预测目标。模型

利用可见视觉内容以及可选的语言信息，恢复被遮挡的视觉表示。这种方法通常称为掩码图像建模（Masked Image Modeling，MIM）。被预测的内容不一定是原始像素。不同方法可能采用：

原始像素值、离散视觉码本、视觉编码器输出特征，或其他经过处理的目标表示。这些预测目标所强调的信息不同。例如，像素重建通常包含大量外观细节，而特征重建可能更关注由目标编码器所表达的语义和结构。因此，MIM 的实际效果与掩码方式、目标表示和网络结构密切相关。

掩码区域建模 在一些早期视觉语言预训练模型中，图像通常先经过目标检测器，得到多个候选区域特征：$R = (r_1, r_2, \ldots, r_K)$，随后，模型遮挡部分区域特征，并根据剩余视觉区域及文本信息进行预测。这种方式称为掩码区域建模（Masked Region Modeling，MRM）。例如，一张图像包含一个人和一辆自行车，文本为：“一个人在骑自行车。”

如果自行车区域被遮挡，模型可以利用其他视觉区域和文本上下文预测该区域的相关特征或语义类别。但 MRM 和基于规则 Patch 的 MIM 不完全相同。MRM 的视觉单元可以来自对象检测器，具有一定的候选对象区域含义；MIM 则可能直接基于图像 Patch 或其他视觉编码单元展开。因此，二者在视觉表示来源、预测目标和监督机制上存在差异。

条件掩码学习 在视觉语言预训练中，可以在遮挡一种模态时，尽可能保留另一种模态的信息。例如：预测被遮挡文本时，保留完整的图像区域特征。预测被遮挡图像区域时，保留完整文本。这种方式通常称为条件掩码建模（Conditional Masking）。其目的之一，是鼓励模型利用另一模态的完整上下文恢复当前模态的缺失内容。UNITER 是相关研究中的代表性模型。

UNITER 将 MLM、MRM、ITM 和词语—区域对齐等目标结合，并采用条件掩码方式构建跨模态预测任务。例如，进行 MLM 时，完整图像区域可以为被遮挡语言提供条件；进行 MRM 时，完整文本可以为被遮挡视觉区域提供补充语义。这一设计体现了掩码建模从单模态上下文预测向跨模态条件预测的扩展。

图像条件语言建模 设图像为：I 目标文本为：$T = (w_1, w_2, \ldots, w_L)$ 根据自回归分解，可以写为：

$$P_\theta(W \mid I) = \prod_{t=1}^{L} P_\theta(w_t \mid w_{<t}, I) \tag{5.23}$$

其中：$w_{<t} = (w_1, \ldots, w_{t-1})$ 表示前面已经给定或生成的 Token。模型在每个位置根据图像和此前文本，预测下一个 Token。对应的负对数似然损失为：

$$\mathcal{L}_{LM} = -\sum_{t=1}^{L} \log P_\theta(w_t \mid w_{<t}, I) \tag{5.24}$$

实际训练通常还会根据有效 Token 数量对损失进行适当归一化。通过这一目标，视觉语言模型可以学习根据图像组织文本描述，而不再局限于候选文本匹配判断。

自回归生成与因果注意力 为了使模型能够逐步生成语言，文本解码器通常需要遵守因果注意力约束。例如，在预测：w_t 时，模型可以访问图像特征和已经给定的文本前缀：$w_{<t}$，但不能直接读取目标位置之后的正确文本。这种方式与允许同时利用前后语言上下文的典型 MLM 不同。例如，MLM 可以根据：“一个儿童正在 [MASK] 自行车。”中的前后文本恢复被遮挡词语。而自回归语言模型在生成“自行车”时，主要依靠已生成的前缀及视觉条件。

训练时通常使用参考文本前缀进行 Teacher Forcing；推理时则需要使用模型自身生成的前文继续预测。因此，训练阶段的文本预测条件与实际生成过程之间可能存在差异。

MLM **与条件生成的区别** MLM 和条件语言生成都涉及文本预测，但其基本学习任务不同。MLM 通常在部分已知的文本中恢复缺失内容。例如：“一个女孩拿着 [MASK] 雨伞。”模型可以利用两侧语言上下文与图像，预测被遮挡词语。条件语言生成则从图像和可选提示出发，逐步生成完整描述。例如：“请描述这张图片。”模型输出：“一个女孩拿着蓝色雨伞站在街道上。”因此，MLM 更强调基于可见上下文的条件恢复，而自回归生成更强调从给定前缀连续产生语言。

这一区别描述的是训练目标，并不意味着采用 MLM 的模型只能理解而不能生成，也不意味着生成模型不具备视觉理解能力。实际能力仍然取决于整体模型结构与训练方式。

Prefix Language Modeling 除典型的自回归条件语言建模之外，还可以采用前缀语言建模（Prefix Language Modeling，PrefixLM）。其基本思想是将输入分为前缀与待预测部分。模型可以充分利用前缀中的信息，并以自回归方式预测后续内容。例如，前缀可以包含图像特征和部分文本：图像 + “一个男孩正在……”模型随后生成：“公园里骑自行车。”SimVLM 是采用 PrefixLM 进行视觉语言预训练的代表性工作。

其特点之一，是利用较为统一的语言生成目标学习图像与文本之间的关系，而不必须依赖 ITC、ITM 和 MLM 的组合。这说明，视觉语言预训练既可以采用多种独立任务联合优化，也可以通过单一的条件语言预测目标实现跨模态学习。

5.5.3 生成式学习与对比式学习的差异

训练目标的区别 图文对比学习通常使用：

$$\mathcal{L}_{ITC} \tag{5.25}$$

通过当前候选集合中的正负关系，使正确图文配对具有较高的相对相似度。图文匹配可以采用：

$$\mathcal{L}_{ITM} = \mathrm{BCE}(p_{\mathrm{match}}, y) \tag{5.26}$$

对联合图文输入进行二分类判断。生成式预训练则可以采用：

$$\mathcal{L}_{LM} = -\sum_{t} \log P_\theta(w_t \mid w_{<t}, I) \tag{5.27}$$

学习视觉条件下的文本预测。这三种目标约束了不同类型的模型行为。ITC 更侧重塑造可比较的跨模态表示空间。ITM 更侧重联合条件下的图文配对判别。LM 则直接训练连续语言生成。但它们并不是彼此互斥的类别，可以在同一框架中共同使用。

对模型结构的要求 典型图文对比学习可以使用两个相对独立的编码器。图像编码器提取视觉特征，文本编码器提取语言特征，两者通过相似度函数连接。这种结构便于提前计算和缓存图像或文本向量。相比之下，条件语言生成需要能够逐 Token 预测文本的解码模块，并使解码过程能够获取相应视觉信息。例如，可以使用视觉编码器和具有跨模态注意力的语言解码器。此外，ITM 通常采用联合编码结构，使图像特征与文本特征进行更充分的交互。

因此，不同目标常常对应不同的信息交互路径。但实际模型可以通过共享参数、任务特定注意力掩码和模块化设计，同时支持多种训练方式。

信息监督粒度不同 对比学习通过整组图文之间的相对关系进行监督。例如，一张图像应当与正确文本比其他候选文本更加接近。这种目标通常不直接规定图像中的每个对象必须对应哪个词语。条件语言生成则需要预测目标文本中的多个 Token。例如：“一个穿红色外套的人站在自行车旁边。”模型需要逐步预测其中的对象、属性和关系描述。因此，条件生成提供了序列级的文本预测监督。但不能因此简单认为生成式目标一定具有更准确的细粒度视觉监督。

例如，文本中的常见连接词可以主要依靠语言上下文预测，而不需要视觉信息。相反，对比学习也可以通过局部 Token 相似度或困难负样本等机制，引入更加细粒度的语义区分。因此，训练信号是否能够促进细粒度对齐，取决于目标设计和数据内容，而不只是目标名称。

检索效率与生成开销 双编码器结构的一项重要优势，是图像和文本能够独立编码。例如，对于包含大量图像的数据库，可以提前保存所有图像嵌入。当新的文本查询到达时，只需编码查询文本并进行向量相似度搜索。因此，典型对比式双编码器适合大规模图文检索。生成式视觉语言模型则通常需要按照序列生成过程逐步预测文本 Token。例如，生成一段包含 L 个 Token 的回答，需要执行相应的解码计算。

因此，生成式模型更加适合开放式语言输出，但不一定是大规模向量检索的最经济实现方式。实际推理效率还与模型规模、视觉 Token 数量、缓存机制和硬件实现有关，不能简单认为所有对比式模型都必然快于所有生成式模型。

对比式模型是否只能学习粗粒度语义 需要避免把对比式学习与全局粗粒度匹配完全等同。例如，FILIP 等方法探索利用图像 Patch 和文本 Token 的局部相似度进行对比学习。这说明，对比式目标也可以结合细粒度视觉语言表示。类似地，生成式目标也不保证模型能够准确理解数量、属性和空间关系。例如，一个能够生成流畅图像描述的模型，仍然可能混淆：“男孩追逐狗。”与：“狗追逐男孩。”因此，应根据模型结构、

训练数据和具体评测结果判断其细粒度理解能力。

5.6 联合学习与知识迁移

5.6.1 多种对齐目标的联合训练

多目标损失函数 假设模型采用图文对比学习、图文匹配和图像条件语言生成三种目标。可以构造：

$$\mathcal{L} = \lambda_c \mathcal{L}_{ITC} + \lambda_m \mathcal{L}_{ITM} + \lambda_g \mathcal{L}_{LM} \tag{5.28}$$

其中：

- $\mathcal{L}_{ITC}$ 表示图文对比损失；
- $\mathcal{L}_{ITM}$ 表示图文匹配损失；
- $\mathcal{L}_{LM}$ 表示条件语言生成损失；
- $\lambda_c, \lambda_m, \lambda_g$ 表示对应目标的权重。

如果还包含掩码语言建模或区域建模，可以进一步扩展为：

$$\mathcal{L} = \sum_{k=1}^{K} \lambda_k \mathcal{L}_k \tag{5.29}$$

这种形式说明，不同预训练目标可以通过加权求和构造统一优化目标。但这只是多任务训练的一种常见实现形式。实际模型也可以通过不同批次交替优化多个任务，或者采用分阶段训练，而不一定在每次参数更新时计算全部损失。

联合训练中的模型共享 多目标联合训练通常需要明确哪些参数在不同任务之间共享。例如，一个模型可以使用统一的视觉编码器提取图像特征：$Z_I = f_I(I)$，随后构建不同任务路径。对比学习路径使用图像与文本的独立嵌入计算 ITC。图文匹配路径使用联合编码器计算 ITM。语言生成路径则使用视觉条件文本解码器计算 LM。这样，三种目标可以利用相同的视觉特征来源，但通过不同模块完成各自任务。

另一类方法则进一步共享文本编码与解码模块的部分参数，并利用不同注意力掩码实现相应功能。因此，联合训练的关键不仅是增加损失函数，还在于设计共享参数和不同任务路径之间的关系。

ALBEF：**先对齐，再融合** ALBEF（Align Before Fuse）是将图文对比学习与跨模态交互结合的代表性模型。它的核心思想，是先通过独立编码器和对比目标建立图文表示之间的语义关联，再利用跨模态编码器进行更充分的信息融合。其主要预训练目标包括：图文对比学习（ITC）。图文匹配（ITM）。掩码语言建模（MLM）。概念上可以写为：

$$\mathcal{L} = \mathcal{L}_{ITC} + \mathcal{L}_{ITM} + \mathcal{L}_{MLM} \tag{5.30}$$

这里省略了具体实现中的任务权重、动量监督及其他细节，主要用于说明训练目标之间的关系。ITC 作用于独立模态表示，帮助模型建立共享语义空间。ITM 通过联合编码判断图文配对是否成立。MLM 则利用图像和语言上下文预测缺失文本。此外，ALBEF 还提出了动量蒸馏（Momentum Distillation），利用动量模型生成软监督目标，尝试缓解网络图文数据中的噪声和不完整配对问题。

因此，ALBEF 的主要启示是：先建立一定的跨模态语义对齐，可以为后续跨模态特征交互提供更合适的表示基础。

BLIP：兼顾理解与生成 BLIP[11]（Bootstrapping Language-Image Pre-training）进一步探索了能够同时服务于视觉语言理解和生成任务的预训练框架。BLIP 的主要训练目标包括：图文对比学习（ITC）。图文匹配（ITM）。语言生成（LM）。可以概括为：

$$\mathcal{L} = \mathcal{L}_{ITC} + \mathcal{L}_{ITM} + \mathcal{L}_{LM} \tag{5.31}$$

其中，ITC 学习共享图文表示。ITM 训练联合条件下的配对判断。LM 则学习根据图像生成语言描述。与 ALBEF 使用 MLM 不同，BLIP 将图像条件语言建模作为其主要生成式训练目标之一。在结构上，BLIP 采用可以支持不同任务的多模态编码—解码设计，通过不同路径或注意力机制执行图文编码、联合匹配和文本生成。这一设计使模型能够根据下游任务选择合适的处理路径。

例如，图文检索可以利用对比式表示与匹配机制，而图像描述则可以利用语言解码路径。

BLIP 的数据自举机制 BLIP 不仅关注模型结构，还关注大规模网络图文数据的质量问题。例如，网络收集的图像替代文本可能过短、不准确，或者包含与图像无关的信息。为了利用这类数据，BLIP 提出了 Captioning and Filtering 的数据自举思路。其中，描述生成模块可以根据图像生成新的文本描述。筛选模块则用于判断候选描述是否与图像具有较好的语义对应关系。通过生成与筛选相结合，可以尝试改善训练文本的质量。

但是，自动生成的描述并不必然正确。例如，图像描述模型可能根据语言先验生成不存在的对象或动作，而筛选模型也可能存在判断错误。因此，数据自举的有效性仍然依赖模型质量和相应实验验证。BLIP 说明，视觉语言预训练的性能不仅由网络结构和损失函数决定，训练数据的构建与清洗同样重要。

CoCa：对比学习与描述生成的联合建模 CoCa（Contrastive Captioners）是另一种同时结合对比目标与图像描述生成目标的代表性工作。其核心设计是，在统一的图文编码—解码结构中，同时优化：

$$\mathcal{L}_{ITC} \tag{5.32}$$

和：

$$\mathcal{L}_{Caption} \tag{5.33}$$

概念上可以表示为：

$$\mathcal{L} = \lambda_c \mathcal{L}_{ITC} + \lambda_g \mathcal{L}_{Caption} \tag{5.34}$$

其中，Caption 表示图像条件文本生成目标。CoCa 的文本解码器采用分层设计。前面的部分主要建立文本的单模态表示，用于与视觉表示进行对比学习。后续部分则通过跨模态注意力利用图像特征，进行条件语言生成。这种设计使对比式与生成式训练能够在一个计算框架中共享部分参数和特征。CoCa 的重要意义在于，它表明模型可以同时学习适合全局语义比较的图文表示，以及适合连续语言输出的跨模态表示。

CoCa 的代表性训练组合是对比学习与图像描述生成，并不要求所有联合模型都同时加入 ITM 和 MLM。

多目标训练中的梯度关系 假设模型采用两种损失：

$$\mathcal{L} = \lambda_1 \mathcal{L}_1 + \lambda_2 \mathcal{L}_2 \tag{5.35}$$

对于共享参数：θ 可以得到：

$$\nabla_\theta \mathcal{L} = \lambda_1 \nabla_\theta \mathcal{L}_1 + \lambda_2 \nabla_\theta \mathcal{L}_2 \tag{5.36}$$

这说明，不同目标的梯度会共同作用于共享参数。如果两种目标推动参数更新的方向具有一定一致性，联合训练可能产生互补效果。但如果两种目标存在明显冲突，也可能使训练更加困难。例如，某些对比目标强调不同图文配对之间的区分，而生成目标强调对特定文本序列的条件预测。两种目标不一定始终要求模型强化相同的信息。因此，不能仅根据损失项数量判断模型是否更强。

损失权重与归一化 不同训练目标的损失数值可能存在明显差异。例如，ITM 是图文二分类任务，而 LM 可能涉及数十个文本 Token 的预测。如果 ITM 按样本数取平均，而 LM 直接对全部 Token 求和，那么两种损失的尺度可能不同。因此，在联合训练中，应首先明确每种损失如何归一化。例如，可以按照有效 Token 数量对语言生成损失取平均。

同时，还需要合理设置：$\lambda_c, \lambda_m, \lambda_g$ 等权重。损失数值较大不意味着该目标在实际任务中一定更重要，损失较小也不意味着该能力已经充分学习。因此，目标权重应当结合训练稳定性、梯度尺度和下游评价进行选择。

联合训练与分阶段训练 不同目标可以在同一个训练阶段联合优化，也可以采用分阶段方式。例如，某个模型可以先通过图文对比学习建立共享语义空间，再加入联合匹配与语言生成目标。也可以在统一训练过程中交替优化 ITC、ITM 和 LM。前一种方式

有利于先形成某类基础表示，再逐步增加新的训练任务。后一种方式则能够在训练早期就使多种监督共同影响共享参数。但两种方式没有无条件的优劣关系。

例如，如果视觉编码器已经具备较好的视觉语义表示，冻结部分参数并训练跨模态连接模块，可能更符合计算预算要求。而如果希望模型进一步学习细粒度视觉语言对应，端到端调整部分视觉与语言参数也可能带来不同效果。因此，训练方式应当结合已有模型能力、数据条件和目标任务进行选择。

5.6.2 多模态蒸馏与知识迁移

教师模型与学生模型 假设教师模型为：f_T 学生模型为：f_S 两者接收相同或相关输入，分别输出：$z_T = f_T(x)$，$z_S = f_S(x)$，如果希望学生学习教师的表示，可以定义：

$$\mathcal{L}_{feat} = \|P(z_S) - z_T\|_2^2 \tag{5.37}$$

其中，P 表示必要的特征投影。这种方式称为特征蒸馏（Feature Distillation）的一种简单形式。如果教师与学生的特征维度、空间分辨率或语义粒度不同，就需要先建立合理的对应关系。不能只因为两个特征经过插值或投影后具有相同形状，就认为它们可以直接进行有意义的蒸馏。

预测分布蒸馏 除了特征蒸馏，还可以利用教师输出的预测分布进行训练。例如，教师对 K 个候选类别输出 Logits：

$$a_T \in \mathbb{R}^K \tag{5.38}$$

学生输出：

$$a_S \in \mathbb{R}^K \tag{5.39}$$

可以使用温度参数 τ_d 构造软分布：

$$p_T^{(\tau_d)} = \text{Softmax}\left(\frac{a_T}{\tau_d}\right) \tag{5.40}$$

$$p_S^{(\tau_d)} = \text{Softmax}\left(\frac{a_S}{\tau_d}\right) \tag{5.41}$$

然后定义：

$$\mathcal{L}_{KD} = \tau_d^2 D_{\text{KL}}\left(p_T^{(\tau_d)} \parallel p_S^{(\tau_d)}\right) \tag{5.42}$$

其中，KL 散度衡量两个预测分布之间的差异。这里的温度平方项是经典蒸馏中的常见缩放方式，具体实现可以根据目标定义和权重设置有所调整。这种方法不仅可以利用教师预测的最高类别，还可以利用不同候选类别之间的相对分数。例如，教师可能认为某张图像最接近“狗”，同时与“狼”也具有一定相关性。软目标能够向学生提

供这种类别间的相对关系。但要注意，教师预测分布本身可能存在偏差，学生并不能通过简单复制自动获得比教师更加准确的语义理解。

关系蒸馏 在跨模态对齐中，教师模型还可能学习到样本之间的相似度关系。例如，一个批次中具有多个图像与文本：$I_1, \ldots, I_B$，$T_1, \ldots, T_B$ 教师模型可以构建图文相似度矩阵：S_T 学生模型也可以构建：S_S 然后训练学生保留教师提供的部分相对关系。例如：

$$\mathcal{L}_{rel} = |S_S - S_T|_F^2 \tag{5.43}$$

其中，$|\cdot|_F$ 表示 Frobenius 范数。这一公式用于说明关系蒸馏的基本思想，实际方法也可能对相似度进行归一化，并采用 KL 散度或其他损失。相比直接复制单个特征向量，关系蒸馏更加关注多个样本之间的相对结构。但教师相似度矩阵未必能够完整表达真实语义关系，错误的图文相似度也可能被传递给学生。

动量蒸馏与软监督 多模态蒸馏不一定始终使用一个完全独立、预先训练完毕的教师模型。例如，在 ALBEF 中，可以使用通过动量更新得到的教师式模型，为训练提供软目标。其基本思想是：学生模型根据当前参数产生预测。动量模型则通过参数的指数移动平均进行更新，为训练提供相对平滑的监督参考。可以概念性地表示为：

$$\theta_T \leftarrow m\theta_T + (1-m)\theta_S \tag{5.44}$$

其中：

- θ_T 表示动量教师参数。
- θ_S 表示学生参数。
- m 表示动量系数。

实际训练中，动量教师可以根据输入产生软监督目标，用于辅助图文表示学习。这种思路具有一定意义，因为网络图文数据可能存在噪声和不完整配对。但动量教师不等于真实标签，也不保证其产生的所有软监督都正确。因此，动量蒸馏主要提供一种利用模型预测辅助训练的机制，而不是对错误图文数据进行绝对可靠修正的方法。

跨模态知识迁移 知识蒸馏还可以发生在不同输入模态之间。例如，一个 RGB 视觉模型已经学习到丰富的对象语义。研究者希望让深度、点云或事件视觉模型获得一定的对象识别能力。可以利用具有配对关系的数据，让学生模型学习与教师相关的语义表示。设教师使用 RGB 输入：$z_T = f_T(I_{\text{RGB}})$ 学生使用另一模态：$z_S = f_S(X_{\text{other}})$，

则可以通过适当的投影与对齐目标进行训练。例如：

$$\mathcal{L}_{distill} = |P_S(z_S) - P_T(z_T)|_2^2 \tag{5.45}$$

其中，X_{other} 可以是其他传感器的有效观测表示。但跨模态蒸馏与同模态模型压缩具有不同特点。RGB 与深度、点云或事件相机所测量的信息并不相同。因此，学生未必能够仅根据自身模态恢复教师包含的全部语义与细节。例如，一张 RGB 图像能够

提供颜色信息，而某些深度或事件表示可能缺少直接颜色观测。如果强制学生完整复制教师的所有特征，可能要求学生学习自身输入中并不存在的信息。

因此，跨模态蒸馏需要明确哪些知识可以合理迁移，哪些内容属于教师模态特有的信息。

蒸馏需要可靠的时空对应 如果教师和学生使用不同传感器输入，就需要确认它们是否观察到相同对象与事件。例如，RGB 相机和事件相机可能具有不同采集时间和视角。如果将某个 RGB 时刻的对象语义，强制对应到明显错位的事件窗口，就可能产生错误监督。类似地，RGB 图像中的某个对象区域与三维点云中的对象区域，也需要通过适当的几何关系建立联系。因此，跨模态蒸馏常常依赖合理的时间同步、空间标定或对象级关联。

这一问题与第四章介绍的异构传感器数据对齐具有直接联系。教师模型预测正确并不能弥补输入配对错误。只有教师目标和学生观测之间具有合理对应关系，蒸馏损失才具有相应物理或语义含义。

知识迁移的能力边界 蒸馏并不是一种能够无条件提高学生性能的方法。例如，教师可能具有较强的通用图像语义能力，但学生需要执行的是高频运动检测任务。如果教师主要关注静态外观，那么其特征未必完整覆盖学生任务需要的时间信息。此外，学生模型的容量、输入质量和训练数据也可能限制蒸馏效果。因此，应当根据具体任务判断是否需要：完整特征蒸馏、局部区域蒸馏、预测分布蒸馏。样本相似度关系蒸馏。或其他形式的选择性知识迁移。

最终，应通过学生在目标任务上的独立评价，判断蒸馏是否真正带来收益。

习题与讨论

1. 推导本章图像到文本损失对 s_{ij} 的导数，并解释双向目标为什么还需要列方向的梯度。
2. 将四个配对拆成两个微批次，分别列出真正大批次与梯度累积所含的负对。两者遗漏了哪些相似度？
3. 同一图像有两个有效描述。设计一个多正样本目标，并说明它如何避免把另一个描述错误排斥。
4. 给零样本分类加入一个语义近似的新类别。分析原类别 Softmax 概率和原相似度分别可能发生什么变化。

文献导读 CLIP 是双编码器目标与零样本迁移的主阅读；ALBEF、BLIP 用于观察对比、融合和生成怎样组合。模型结构与损失应对照阅读。[8, 10, 11]

第六章　视觉语言模型的连接与条件注入

连接器既规定张量如何接入语言模型，也规定信息在哪里被压缩。逐位置投影、查询式压缩和交叉注意力的区别，应在计算图、梯度路径与成本上说明。本章用 BLIP-2[12]、Flamingo[13] 和 LLaVA 作为三种机制的实例，避免把架构学习变成模型名称目录。

学习目标　追踪编码器、连接器和语言模型的张量形状；解释冻结参数时的反向传播；按查询位置和可见性比较三类连接机制。

6.1　统一的数据流与接口

6.1.1　从视觉输入到语言输出的数据流

第二步：视觉特征提取　预处理后的图像进入视觉编码器：$Z_V = f_V(I')$，例如，假设采用一种 ViT[1] 编码器：图像大小为：336×336Patch 大小为：14×14，在不计特殊 Token 的简化设置下，可得到：$N_V = 576$ 个视觉 Token。如果视觉特征维度为：$D_V = 1024$ 那么：

$$Z_V \in \mathbb{R}^{576 \times 1024} \tag{6.1}$$

这一张量包含了图像不同空间位置的特征。但它仍然是视觉编码器输出的连续表示，并不是语言模型能够直接解释为文本单词的序列。

第三步：模态连接　假设语言模型隐藏维度为：$D_L = 4096$，如果使用逐 Token MLP 投影，可以将视觉特征转换为：

$$H_V \in \mathbb{R}^{576 \times 4096} \tag{6.2}$$

如果使用具有压缩功能的连接模块，也可以得到：

$$H_V \in \mathbb{R}^{N_Q \times 4096} \tag{6.3}$$

例如，在一个假设设置中：$N_Q = 32$，则连接模块可以输出 32 个视觉条件 Token。上述维度和 Token 数量只是用于说明不同架构的张量变化，并非指定模型的统一配置。此外，压缩后的 32 个 Token 不代表能够完整无损地恢复原始 576 个视觉 Token 中的信息。

第四步：文本指令编码 用户问题为：“桌面上的杯子是什么颜色?” 模型首先通过 Tokenizer 将文本转换为离散序列：$Q = (q_1, \ldots, q_{L_Q})$ 再转换为语言嵌入：$E_Q = \mathrm{Embed}(Q)$，其中：

$$E_Q \in \mathbb{R}^{L_Q \times D_L} \tag{6.4}$$

在实际对话模型中，输入还可能包含角色标记、系统指令、回答起始标记以及其他控制信息。这些内容同样会占用相应的文本 Token 或计算位置。因此，模型实际处理的文本上下文长度通常不等于用户问题所包含的自然语言词语数量。

第五步：组织视觉与文本输入 对于视觉前缀拼接结构，可以将视觉条件嵌入与文本嵌入组织为联合序列。例如：$X = [H_V; E_Q]$ 假设连接模块保留 576 个视觉 Token，而文本及必要提示包含 40 个 Token。那么联合输入序列长度约为：$N = 576 + 40 = 616$，这里没有计入可能存在的额外特殊 Token。如果连接模块将视觉特征压缩为 32 个 Token，那么相应输入长度可变为：$N = 32 + 40 = 72$，

但这种比较只说明视觉前缀长度的变化。不同连接模块自身的计算开销，以及压缩造成的信息损失，仍然需要另外评价。

第六步：条件语言生成 完成视觉和文本条件组织后，语言模型开始生成回答。设回答 Token 序列为：$Y = (y_1, \ldots, y_{L_Y})$，则：

$$P(Y \mid I, Q) = \prod_{t=1}^{L_Y} P(y_t \mid y_{<t}, I, Q) \tag{6.5}$$

在每一步，模型根据当前上下文计算下一个 Token 的预测分布。例如，对于问题：“杯子是什么颜色?” 模型可以生成：“红” $\rightarrow$ “色” $\rightarrow$ “。” 这里的过程只是自然语言层面的示意。实际输出可能采用不同粒度的 Token。生成过程通常持续到遇到结束标记，或达到相应的停止条件。

最终，生成的 Token 经过解码，形成用户看到的自然语言回答。

训练阶段与推理阶段的区别 训练时，通常具有图像、问题和参考回答：

$$(I, Q, Y^{gt}) \tag{6.6}$$

模型可以使用 Teacher Forcing，根据参考回答的已知前缀预测后续 Token。例如，训练目标为：

$$\mathcal{L} = -\sum_{t=1}^{L_Y} \log P_\theta \left(y_t^{gt} \mid y_{<t}^{gt}, I, Q \right) \tag{6.7}$$

这一损失主要约束模型对目标回答的条件预测。如果采用回答监督训练，通常可以只在回答 Token 对应的位置计算语言生成损失，而不要求模型预测视觉前缀或用户

问题本身。具体实现还需要根据对话模板、Token 位移与注意力掩码正确设置标签。推理时则没有参考回答：

$$Y^{gt} \tag{6.8}$$

模型需要使用自己已经生成的 Token，逐步产生后续内容。因此，训练阶段与推理阶段虽然使用相关模型结构，但可获得的答案信息不同。

为什么要区分输入与监督标签 在视觉指令训练中，模型可能接收：图像、用户问题、参考回答。但是，这三部分并不具有相同的训练角色。图像与问题主要提供生成条件。参考回答主要提供训练目标。例如：输入：一张包含红色汽车的图片。问题：“汽车是什么颜色?”回答：“红色。”模型需要利用图像和问题预测“红色”。不能在预测回答开始之前，就将完整正确答案作为模型可读取的条件，否则会产生训练信息泄漏。

同样，如果采用只监督助手回答的训练协议，应当明确哪些 Token 参与语言生成损失计算。因此，构建视觉语言训练数据时，不仅需要正确组织图像与文本，还需要区分条件信息和预测目标。

多轮对话中的视觉信息使用 对于多轮视觉对话，模型可能需要在不同轮次继续利用同一张图像。例如：用户：“图像中有几辆汽车?”模型：“两辆。”用户：“左边那辆是什么颜色?”模型：“红色。”第二个问题中的“左边那辆”依赖前面已经提及的汽车对象，同时需要利用原始图像中的空间信息。因此，多轮视觉语言模型需要正确组织历史文本与视觉条件。如果模型在后续轮次中只保留文本回答，而无法有效利用视觉特征，就可能丢失必要的空间与外观证据。

此外，多轮上下文还可能发生错误累积。例如，模型第一轮错误地识别了对象数量，第二轮可能继续根据错误结果回答。因此，多轮视觉对话不仅需要语言上下文连续性，也需要保持对视觉证据的适当访问。

多图像输入与视觉上下文扩展 当模型接收多张图像时，每张图像可能产生一组视觉特征。例如：

$$Z_V^{(1)}, Z_V^{(2)}, \ldots, Z_V^{(K)} \tag{6.9}$$

随后，连接模块可以分别处理这些视觉特征，再根据模型架构将其组织为统一输入。例如，用户提出：“比较这两张图像中汽车的位置有什么变化。”模型需要识别两张图像中的对应对象，并进一步比较空间关系。因此，多图像任务不仅需要处理更多视觉 Token，还需要保留图像之间的身份与顺序信息。如果直接将全部视觉特征混合，而没有明确图像来源，模型就可能难以区分哪些内容属于第一张图像，哪些属于第二张图像。

这一问题在视频和长时序视觉模型中会进一步扩展为帧间关系和时间位置建模。

6.2 逐位置映射与梯度传播

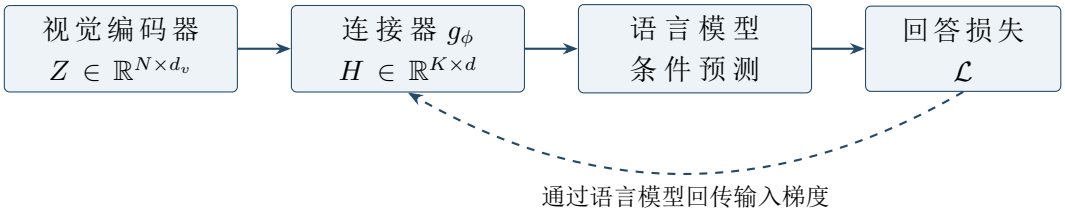


图 6.1 冻结主干时仍需保留连接器到损失的计算图。冻结参数和对整个前向使用无梯度执行方式有不同后果。

6.2.1 线性投影与维度匹配

设视觉输出 $Z \in \mathbb{R}^{N\times d_v}$，语言模型要求隐藏维度 d。逐位置线性连接器为

$$H = ZW + \mathbf{1}b^\top, \qquad W \in \mathbb{R}^{d_v\times d}, \quad b \in \mathbb{R}^d. \tag{6.10}$$

输出 $H \in \mathbb{R}^{N\times d}$，含偏置时参数量为 $d_v d + d$。映射改变通道维，没有改变 N；它完成了形状接口，但语义适配来自训练。

线性连接器便于建立简单基线。若视觉表示本身没有保留某项细节，逐位置投影也不能恢复该项真实观测。评价连接器时应固定编码器输出、训练数据和语言模型，避免把主干或数据变化的收益归因于映射形式。

6.2.2 多层感知机与非线性映射

两层逐位置 MLP 可写为

$$H = \sigma(ZW_1 + \mathbf{1}b_1^\top)W_2 + \mathbf{1}b_2^\top, \tag{6.11}$$

其中 $W_1 \in \mathbb{R}^{d_v\times h}$、$W_2 \in \mathbb{R}^{h\times d}$。含两组偏置的参数量为 $d_v h + h + hd + d$。激活函数使映射不再局限于线性变换，但逐位置 MLP 仍输出 N 个位置，并不自动形成空间汇聚。

将 MLP 与查询压缩比较时，应把表达能力和位置数量分开控制。若一种方案同时减少视觉 Token 又增加训练参数，单个结果不能区分两种因素。LLaVA 的线性及后续 MLP 连接路线可用来理解这一接口变化，完整训练流程仍需对照各自版本。

6.2.3 视觉特征与语言嵌入空间的对接

视觉特征可以投影到语言嵌入维度后，与问题的词嵌入共同组成输入。实现通常先识别图像占位符，再插入视觉状态，随后更新位置、注意力和监督掩码。插入后的位置数量未必等于原始文本 Token 数量，标签不能沿用未展开序列的下标。

连续视觉状态不要求对应词表中的某个真实词 ID，也不需要为每块图像指定词语标签。接口适配还包括特征数值分布、位置语义和输入边界。维度相同仅保证张量运算相容，是否可用仍需受控证据实验。

6.2.4 连接模块的训练与参数更新

令 $Z = f_{\theta_v}(I)$、$H = g_\phi(Z)$，语言模型参数为 ψ。基于目标回答 $Y = (y_1, \ldots, y_L)$ 的条件语言损失为

$$\mathcal{L} = -\frac{1}{L}\sum_{t=1}^{L} \log p_\psi(y_t \mid y_{<t}, Q, H), \tag{6.12}$$

此处 L 指有效目标位置数；批内变长目标需重新明确整体平均方式。训练图像描述或指令回答时，用户提示与视觉输入通常提供条件，目标位置提供监督。

冻结 θ_v, ψ，只更新 ϕ 时，梯度仍可经过语言模型回到连接器：

$$\nabla_\phi \mathcal{L} = \left(\frac{\partial H}{\partial \phi}\right)^\top \nabla_H \mathcal{L}. \tag{6.13}$$

冻结是参数更新规则，不是删除语言模型的输入导数。若实现把语言模型整个前向放入切断自动求导的区域，连接器就可能得不到需要的梯度。

初始化、学习率和解冻策略应与主干的预训练状态相容。训练后用原图、无图、错配及相关局部变化检查回答是否使用视觉条件；仅观察损失下降或回答更长，不能判定连接正确。

6.3 查询式视觉压缩

6.3.1 Q-Former 的基本原理

可学习查询向量 Q-Former 引入一组可学习的查询嵌入（Learnable Query Embeddings）。可以表示为：$Q_0 = (q_1, q_2, \ldots, q_K)$，其中：

$$Q_0 \in \mathbb{R}^{K \times D_Q} \tag{6.14}$$

K 表示查询数量，D_Q 表示查询特征维度。在 BLIP-2 的经典实现中，采用 32 个可学习查询，每个查询具有相应的隐藏表示。因此：$K = 32$

这里的 Query 并不是用户输入的自然语言问题。

例如，用户可能提出：“图像中有几个人?” 但 Q-Former 的可学习查询，并不是直接将这句话的 Token 当作查询向量。它们是模型训练过程中学习得到的参数，用于从视觉编码器输出中提取信息。这一区别对于理解 Q-Former 的工作机制十分重要。

查询与视觉特征的交叉注意力 假设视觉编码器输出：

$$Z_V \in \mathbb{R}^{N_V \times D_V} \tag{6.15}$$

Q-Former 可以使用查询向量作为注意力中的 Query，并利用视觉特征构造 Key 和 Value。例如：$Q = Q_0 W_Q$，$K = Z_V W_K$，$V = Z_V W_V$，通过交叉注意力得到：

$$H_Q = \text{Softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \tag{6.16}$$

其中，d_k 为注意力查询与键的维度。这一表达式用于说明查询式视觉信息提取的基本原理。实际 Q-Former 还包含多层自注意力、交叉注意力、前馈网络和残差连接等结构。交叉注意力使每个查询能够根据学习到的匹配关系，从多个视觉 Token 中聚合信息。例如，一个查询可能对某些对象区域具有较强响应，另一个查询可能更多地聚合场景上下文。

但这些查询并不一定天然对应固定的对象类别，也不能未经验证就将某个查询解释为“专门识别人物”或“专门识别颜色”。它们具体学习到什么，取决于训练目标和数据分布。

查询之间的交互 如果每个查询只独立读取视觉特征，那么不同查询可能提取到重复信息。因此，Q-Former 不只是执行一次简单的交叉注意力。它还通过 Transformer 自注意力，使查询表示能够相互交互。例如，查询状态可以概念性地写为：

$$\tilde{Q} = \text{SelfAttention}(Q_0) \tag{6.17}$$

再根据视觉特征更新：

$$H_Q = \text{CrossAttention}(\tilde{Q}, Z_V) \tag{6.18}$$

在实际网络中，自注意力、交叉注意力和前馈网络可以按照具体结构多次组合。这种机制使查询能够在多层处理中逐步聚合视觉信息，并形成具有一定互补性的输出特征。但查询之间存在自注意力交互，并不保证它们自动学习到完全不同的语义内容。例如，如果训练任务主要强调整体场景分类，多个查询可能都关注较为显著的对象或背景。因此，还需要通过实际任务验证查询表示是否保留了不同视觉信息。

Q-Former 的输出 假设输入视觉编码器具有：$N_V = 576$ 个视觉 Token。而 Q-Former 使用：$K = 32$ 个查询。那么输出可以表示为：

$$H_Q \in \mathbb{R}^{32\times D_Q} \tag{6.19}$$

这样，Q-Former 将较长的视觉特征序列转换为固定数量的查询表示。虽然输出 Token 数量较少，但每个查询可以通过交叉注意力访问多个视觉 Token。因此，Q-Former 并不是简单地从 576 个视觉 Token 中挑选 32 个原始 Token。它通常产生经过学习式聚合的新表示。这一特点区别于只保留部分原始 Patch 的硬 Token 选择方式。

Q-Former 的完整结构 BLIP-2 中的 Q-Former 不应被简单理解为一个只有 32 个查询的交叉注意力层。其结构还包括能够处理文本的 Transformer 部分。按照原论文，Q-Former 包含两个具有部分共享自注意力参数的子模块：一个用于通过可学习查询提取视觉信息。另一个用于处理文本，并根据训练目标承担文本编码或生成相关功能。图像查询通过交叉注意力访问冻结视觉编码器的输出。

文本与查询之间是否能够交互，则由不同训练任务所采用的注意力掩码决定。因此，BLIP-2 的 Q-Former 同时承担视觉信息提取与视觉语言表示学习的任务，而不只是一个固定长度的视觉池化器。

Q-Former 的预训练目标 在 BLIP-2 的第一阶段，视觉编码器保持冻结，主要训练 Q-Former。该阶段采用三个具有代表性的目标：图文对比学习（Image-Text Contrastive Learning，ITC）。图文匹配（Image-Text Matching，ITM）。图像条件文本生成（Image-grounded Text Generation，ITG）。这些目标已经在第五章介绍过基本原理。

它们在 Q-Former 中的作用可以概括如下。ITC 促进视觉查询表示与文本语义之间的对齐。ITM 使模型能够利用图像和文本之间的联合信息判断配对关系。ITG 则通过文本预测，要求视觉查询表示提供与语言生成相关的信息。在原始 BLIP-2 中，不同目标使用不同的注意力掩码，控制查询 Token 和文本 Token 之间的信息交互。例如，在图文对比任务中，需要避免视觉查询与文本提前交换信息，从而保持对比表示的意义。

而在图文匹配任务中，则允许查询与文本进行更加充分的交互。因此，Q-Former 的训练不仅学习如何压缩视觉特征，还学习哪些视觉信息与语言任务具有联系。

从 Q-Former 到语言模型 BLIP-2 的第二阶段，进一步将 Q-Former 与冻结的预训练语言模型连接。设 Q-Former 输出：

$$H_Q \in \mathbb{R}^{32\times D_Q} \tag{6.20}$$

如果语言模型的输入嵌入维度为：D_L 可以通过线性映射得到：$H_L = H_Q W_P$，其中：

$$W_P \in \mathbb{R}^{D_Q\times D_L} \tag{6.21}$$

最终：

$$H_L \in \mathbb{R}^{32\times D_L} \tag{6.22}$$

这些视觉条件表示再按照所使用语言模型的接口进入生成过程。例如，对于自回归语言模型，可以将其组织为连续视觉前缀。对于不同类型的预训练语言模型，条件输入方式可能有所差异。在这一阶段，BLIP-2 主要训练 Q-Former 及其必要的连接参数，而预训练视觉编码器和语言模型保持冻结。这种分阶段训练设计，使 Q-Former 先学习语言相关的视觉表示，再进一步适应语言模型的生成接口。

6.3.2 Resampler 与查询式特征提取

Resampler 的机制 Resampler 通常指通过一定的信息聚合机制，将输入特征序列重新组织为较少或指定数量的输出表示。在视觉语言模型中，其重要用途之一，是将数量

较多、甚至随输入分辨率或视频长度变化的视觉特征，转换为规模更加可控的视觉条件序列。例如：

$$Z_V \in \mathbb{R}^{N_V \times D_V} \tag{6.23}$$

经过 Resampler：

$$H_R = \text{Resampler}(Z_V) \tag{6.24}$$

得到：

$$H_R \in \mathbb{R}^{K \times D_R} \tag{6.25}$$

其中，K 表示输出的 Token 数量。如果 K 是预先设置的固定值，那么即使不同图像具有不同数量的输入视觉 Token，也可以获得数量相同的输出表示。Resampler 是较为宽泛的概念，具体实现可以采用池化、注意力或学习式查询等不同机制。在经典视觉语言模型中，Flamingo 使用的 Perceiver Resampler 是查询式压缩的代表性实现。

Flamingo 的 Perceiver Resampler Flamingo 是能够处理图文交错输入，并支持图像和视频条件语言生成的代表性视觉语言模型。其视觉处理过程包含两个重要步骤。使用预训练视觉编码器提取视觉特征。随后，通过 Perceiver Resampler 将视觉特征转换为固定数量的视觉条件表示。在原始 Flamingo 的经典配置中，Perceiver Resampler 产生 64 个视觉输出 Token。因此，可以表示为：

$$Z_V \in \mathbb{R}^{N_V \times D_V} \longrightarrow H_R \in \mathbb{R}^{64 \times D_R} \tag{6.26}$$

这里，输入视觉 Token 数量可以随图像或视频特征组织方式变化，而输出数量由潜在查询数量决定。这种设计有利于控制后续视觉—语言交叉注意力所需要访问的视觉条件规模。

可学习潜在查询 Perceiver Resampler 同样使用一组可学习潜在向量：$L_0 = (l_1, l_2, \ldots, l_K)$，其中：

$$L_0 \in \mathbb{R}^{K \times D_R} \tag{6.27}$$

这些潜在向量作为注意力查询，从视觉特征中提取信息。最简单的查询式聚合可以写为：

$$H_R = \text{Attention}\left(L_0, Z_V, Z_V\right) \tag{6.28}$$

这一表达式说明潜在查询读取视觉特征的基本思想。但 Flamingo 的实际 Perceiver Resampler 比这一简单形式更加复杂。其模块通过多层注意力和前馈计算逐步更新潜

在向量。在原论文给出的结构中，潜在向量作为查询，而键和值由视觉输入特征与当前潜在表示共同构成。因此，某一层可以概念性地写为：

$$L' = L + \text{Attention}\left(LW_Q, [Z_V; L]W_K, [Z_V; L]W_V\right) \tag{6.29}$$

其中，$[Z_V; L]$ 表示在序列维度上连接视觉特征与潜在特征，省略了必要的维度变换与归一化细节。随后，模块继续通过前馈网络和残差结构更新潜在表示。经过多层处理，最终得到固定数量的视觉输出 Token。这一设计既允许潜在查询读取外部视觉信息，也允许潜在表示在更新过程中发生相互作用。

Flamingo **如何使用压缩后的视觉特征** Flamingo 并不是简单将所有视觉 Token 作为普通文本前缀拼接到语言模型输入端。其主要思路，是保留预训练语言模型的原有结构，并在部分语言模型层之间插入可训练的门控交叉注意力模块。Perceiver Resampler 产生的视觉特征，为这些交叉注意力模块提供视觉条件。例如，设语言模型当前隐藏状态为：H_L 压缩后的视觉表示为：H_R 那么语言模型可以通过：

$$\text{CrossAttention}(H_L, H_R) \tag{6.30}$$

获取视觉信息。这样，视觉特征不必全部变成语言模型文本自注意力序列中的普通前缀位置。Flamingo 采用的 Perceiver Resampler 和门控交叉注意力分别承担不同作用。前者负责视觉特征重采样和压缩。后者负责将视觉信息提供给语言模型。因此，不能将 Resampler 本身等同于完整的视觉语言融合结构。

图像与视频输入 Flamingo 的一个特点是可以处理图像和视频等视觉输入。对于视频，视觉编码器可以分别处理采样得到的视频帧，并形成具有空间和时间组织关系的视觉特征。随后，Perceiver Resampler 可以对这些特征进行聚合。例如，设视频包含 T 个采样帧，每帧提取 S 个空间特征：

$$Z_{\text{video}} \in \mathbb{R}^{T \times S \times D_V} \tag{6.31}$$

可以在加入必要的时间信息后，将其整理为：

$$Z_{\text{flat}} \in \mathbb{R}^{TS \times D_V} \tag{6.32}$$

再通过 Resampler 得到：

$$H_R \in \mathbb{R}^{K \times D_R} \tag{6.33}$$

这样，输入特征数量可以随视频采样帧数变化，而输出规模保持相对可控。但将视频压缩为固定数量的视觉 Token，并不保证其中全部的时间顺序、短时运动和事件变化都被保留。因此，对于需要精细时序理解的任务，Resampler 的训练方式和时间信息组织同样重要。

Q-Former 与 Perceiver Resampler 的异同 Q-Former 和 Perceiver Resampler 都采用可学习查询，对较多视觉特征进行信息聚合。但它们并不是完全相同的模块。Q-Former 在 BLIP-2 中承担了视觉信息提取与视觉语言表示学习的双重作用。其训练涉及图文对比、匹配和图像条件文本生成等目标，并具有专门的文本处理和注意力掩码机制。

Perceiver Resampler 在 Flamingo 中主要负责将图像或视频视觉特征重新组织为固定数量的表示。其输出进一步提供给语言模型内部的门控交叉注意力模块。因此，二者的主要区别不仅在于查询数量，还在于各自的网络结构、训练方式和后续使用接口。这些配置体现了对应原始论文的典型设计，不代表所有后续 Q-Former 或 Resampler 实现都必须采用相同结构。

6.3.3 Token 压缩的信息保留与损失

信息瓶颈 可以将视觉 Token 压缩理解为一种信息瓶颈（Information Bottleneck）。设原始视觉表示为：Z_V 压缩后的表示为：$H_V = g(Z_V)$ 下游任务目标为：Y，则模型希望在缩短表示长度的同时，使：H_V 仍然包含足以完成任务 Y 的信息。从概念上，可以表示为：

$$Z_V \rightarrow H_V \rightarrow \hat{Y} \tag{6.34}$$

其中，H_V 是连接视觉信息与任务输出的中间表示。对于只依赖压缩后特征的下游模型，压缩过程无法凭空恢复原始表示中已经缺失的真实观测信息。但一个经过任务训练的压缩器，可以优先保留与任务相关的特征，减少无关或冗余信息所占用的表示容量。因此，压缩的目标并不是保留全部原始特征数值，而是保留完成任务所需的充分信息。

信息保留具有任务依赖性 例如，假设一张图片展示：“一名男子站在公交车旁，公交车上显示线路编号 18。”对于问题：“图像中的主要交通工具是什么？”模型只需要识别公交车。对于问题：“公交车的线路编号是多少？”模型还需要准确读取数字 18。同一张图像的不同任务，对压缩表示提出了不同要求。如果压缩器主要根据常规图像描述目标训练，可能优先保留“男子”和“公交车”等主要对象语义。但是，公交车上较小的线路数字，可能没有被充分保留。

因此，一种压缩表示可以在图像描述任务上表现较好，却在细粒度 OCR 任务上出现明显下降。这并不矛盾，而是因为两种任务需要的信息粒度不同。

小目标与局部细节损失 当视觉信息被聚合为较少 Token 时，局部细节可能被弱化。例如，原始视觉 Token 中只有少量位置对应远处的一名行人。如果压缩模块主要关注图像中更显著的汽车和建筑，那么与小行人有关的信息可能被覆盖或稀释。类似地，图像中的细小文字、交通标志、表盘刻度和工业缺陷，也可能受到压缩影响。因此，Token 压缩对小目标和局部细节任务具有潜在风险。这并不意味着查询式压缩必然忽略小目标。

如果训练数据和任务目标充分强调小目标，学习式查询也可能主动提取相关信息。

关键问题是：压缩后的表示是否能够保留这些任务所需的局部证据。

对象数量与多个实例 视觉 Token 压缩还可能影响对象数量判断。例如，一张图像中存在五个外观相似的杯子。如果压缩模块将这些杯子的区域特征聚合为一个表示，语言模型可能能够判断图像中存在杯子，却难以准确区分五个独立实例。这是因为对象存在性与对象实例数量是不同的信息。对于图像分类任务，“存在杯子”可能已经足够。但对于计数任务，需要进一步保留多个对象实例之间的区别。因此，压缩机制是否保留实例级信息，是评价视觉语言模型细粒度能力的重要方面。

空间位置与关系信息 空间关系同样可能受到 Token 压缩影响。例如，图像 A 中：红色杯子位于蓝色书本左侧。图像 B 中：红色杯子位于蓝色书本右侧。两幅图像具有相同的对象和颜色信息，但空间布局不同。如果压缩器产生的表示主要概括对象类别和属性，而没有充分保留位置关系，那么两幅图像可能形成高度相似的压缩特征。此时，语言模型即使知道问题要求判断“左边还是右边”，也可能缺少足够的空间证据。

因此，视觉 Token 压缩不只是决定保留多少特征，还涉及是否保留空间位置、对象身份和相互关系。对于语言条件定位和空间推理任务，保留这类结构信息尤其重要。

时间顺序与动态信息 在视频视觉语言模型中，还需要考虑时间关系是否受到压缩影响。例如，一段视频显示：人物先拿起杯子，再将杯子放到桌面。另一段视频显示：人物先从桌面拿起杯子，再将其递给其他人。这些视频可能包含相似的视觉对象和部分相同动作，但事件顺序与行为结果不同。如果压缩器主要聚合所有帧中的视觉内容，而不充分保留事件发生的时间位置，就可能造成时序混淆。因此，视频特征压缩不仅要保留空间语义，还需要根据任务保留必要的时间结构。

例如，可以利用时间位置编码、分层时间聚合或对关键变化片段的选择性处理等方法。但不同方法是否能够真正保持因果顺序，仍然需要通过专门的时序任务验证。

查询数量与表达能力 对于查询式压缩模块，一个重要超参数是查询数量：K，例如，可以比较：$K = 8,\ 16,\ 32,\ 64,\ 128$ 不同设置。较少查询通常可以降低后续视觉条件规模。但查询数量过少，可能限制模型同时表达多个对象、属性和关系的能力。增加查询数量，则可能为复杂场景提供更大的表示容量，但也会提高连接模块和后续语言模型的计算成本。因此，需要通过实际任务评价确定合适的查询数量。

例如，可以观察图像描述、视觉问答、OCR、计数和空间关系任务的性能随 K 变化的情况。但查询数量只是影响信息保留的因素之一。查询向量的训练目标、视觉特征质量、注意力结构和数据分布，也可能对最终效果产生重要影响。

任务条件查询与固定查询 在 Q-Former 和典型 Perceiver Resampler 中，可学习查询通常来自模型参数，其初始化查询并不直接等于用户的自然语言问题。因此，压缩模块提取的信息可能需要对不同下游问题具有一定通用性。例如，同一张图像可能在不同轮次中被询问：“有几个人？”“汽车是什么颜色？”“路牌上写了什么？”如果压缩后的视觉表示已经丢失某些细节，语言模型仅仅改变文本问题，也未必能够恢复这些信息。

一种值得研究的方向，是使视觉信息提取过程根据当前任务或文本查询进行调整。

例如，可以使用由文本条件产生的查询表示：$Q = Q(T)$ 再通过交叉注意力从视觉特征中提取相关信息：

$$H_V(T) = \text{CrossAttention}\left(Q(T), Z_V\right) \tag{6.35}$$

这样，视觉压缩过程可以根据具体问题选择不同的信息。但这种思路也带来新的权衡。例如，如果每次文本问题变化都需要重新提取视觉特征，就可能增加多轮交互成本。同时，如果查询条件本身存在错误，也可能引导模型忽略其他重要视觉证据。因此，固定查询与任务条件查询具有不同的适用场景，需要根据任务与计算约束选择。

压缩后的表示能否恢复原始信息 一个重要认识是：压缩后的视觉 Token 不一定能够重建原始全部视觉特征。例如，多个原始视觉 Token 经过加权聚合后形成一个新的表示：

$$h = \sum_{i=1}^{N} a_i v_i \tag{6.36}$$

其中：

$$\sum_i a_i = 1 \tag{6.37}$$

如果不同的视觉特征组合能够产生相同的聚合结果，那么仅根据 h 通常无法唯一恢复原始全部 v_i。因此，Token 压缩可能具有不可逆性。但视觉语言模型的目标通常不是恢复原始特征，而是完成语言相关任务。所以，不可逆并不意味着压缩一定不合理。关键在于，丢失的信息是否属于目标任务所需要的内容。

6.4 交叉注意力与条件访问

6.4.1 交叉注意力的计算机制

交叉注意力的输入 设语言模型当前层的隐藏状态为：

$$H_T \in \mathbb{R}^{L \times D_L} \tag{6.38}$$

视觉编码器及连接模块产生的视觉条件表示为：

$$H_V \in \mathbb{R}^{N_V \times D_V} \tag{6.39}$$

其中：

- L 表示文本序列长度；
- N_V 表示视觉 Token 数量；

- D_L 表示语言隐藏维度；
- D_V 表示视觉特征维度。

为了执行交叉注意力，需要将两种特征投影到适当的注意力维度。例如：$Q = H_T W_Q$，$K = H_V W_K$，$V = H_V W_V$，这里的 W_Q、W_K 和 W_V 是可学习的特征投影矩阵。对于一个注意力头，通常可以设：

$$Q \in \mathbb{R}^{L \times d_k} \tag{6.40}$$

$$K \in \mathbb{R}^{N_V \times d_k} \tag{6.41}$$

$$V \in \mathbb{R}^{N_V \times d_v} \tag{6.42}$$

因此，语言与视觉特征原本的隐藏维度不必完全相同。交叉注意力模块可以通过独立的投影矩阵，将它们转换到符合运算要求的维度。这也是交叉注意力与简单视觉 Token 拼接之间的一项重要区别。

缩放点积注意力 给定 Q、K 和 V，可以计算：

$$A = \text{Softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) \tag{6.43}$$

其中：

- $QK^\top$ 表示语言查询与视觉键之间的相似度；
- d_k 表示 Key 的特征维度；
- M 表示可选的注意力掩码；
- Softmax 通常沿视觉候选位置维度进行归一化。

由于：

$$Q \in \mathbb{R}^{L \times d_k} \tag{6.44}$$

$$K \in \mathbb{R}^{N_V \times d_k} \tag{6.45}$$

因此：

$$A \in \mathbb{R}^{L \times N_V} \tag{6.46}$$

这里的 A 可以理解为文本查询位置与视觉特征位置之间的注意力权重矩阵。矩阵中的元素：A_{ij} 表示第 i 个文本隐藏状态在当前注意力计算中分配给第 j 个视觉位置的权重。随后，利用这些权重聚合视觉 Value：$O = AV$ 得到：

$$O \in \mathbb{R}^{L \times d_v} \tag{6.47}$$

经过必要的输出投影后，视觉条件信息可以转换为语言模型隐藏状态所需的维度。这一过程就是典型的语言查询视觉信息的交叉注意力计算。

从单个文本查询理解视觉信息读取 考虑一张图像，画面中包含一个红色杯子和一本蓝色书。用户提出："杯子是什么颜色？"在语言模型的某个隐藏状态中，模型可能形成与"杯子颜色"相关的查询表示。假设视觉条件包含三个特征：v_1：杯子区域，v_2：书本区域，v_3：背景区域，

经过 Query 和 Key 的相似度计算，可能得到一组示意分数：$s = (2.0, 0.5, 0.1)$ 对其进行 Softmax 后，杯子区域将获得相对较高的注意力权重。随后，模型根据这些权重聚合视觉 Value，使与杯子相关的视觉信息参与语言隐藏状态更新。这样，在后续预测回答时，模型就有机会利用相关视觉特征生成："红色。"这一例子说明，交叉注意力提供了一种根据语言上下文读取视觉特征的计算机制。

但注意力权重由学习得到的特征投影决定，而不是由自然语言词语直接指定。模型是否真正关注正确的杯子区域，仍然需要通过训练和评价验证。

多头交叉注意力 实际 Transformer 通常采用多头注意力（Multi-Head Attention）。其基本思想是，将输入特征分别投影到多个注意力子空间，使不同注意力头能够学习不同的信息交互方式。对于第 h 个注意力头：

$$O_h = \text{Attention}\left(H_T W_Q^{(h)}, H_V W_K^{(h)}, H_V W_V^{(h)}\right) \tag{6.48}$$

将多个注意力头的结果拼接后，再通过输出投影得到：

$$O = \text{Concat}(O_1, \ldots, O_H) W_O \tag{6.49}$$

其中，H 表示注意力头数量。多头机制为语言模型提供了不同的视觉信息读取通道。例如，不同注意力头可能学习到与对象属性、局部结构或场景上下文有关的交互模式。但不能未经分析就认为某个特定注意力头必然对应颜色，另一个头必然对应空间关系。这些功能是通过训练形成的，可能分布在多个头与不同网络层之间。

残差连接与隐藏状态更新 交叉注意力的输出通常不会完全替换语言模型的原有隐藏状态，而是通过残差连接等方式进行更新。可以概念性地表示为：

$$H_T' = H_T + \text{CrossAttn}(H_T, H_V) \tag{6.50}$$

这一公式省略了实际模型中可能包含的 LayerNorm、输出投影、门控和其他操作。通过残差连接，模型可以在保留原有语言隐藏状态的基础上，引入视觉条件信息。例如，语言模型已经知道当前问题要求回答颜色，交叉注意力则为其补充与图像中杯子颜色相关的证据。因此，可以将交叉注意力理解为：根据语言模型当前的表示需求，从视觉条件中获取增量信息，并影响后续语言计算。

交叉注意力与因果约束 在自回归语言生成中，语言模型不能直接访问未来尚未生成的目标 Token。因此，文本自注意力需要采用因果掩码。但交叉注意力访问的是已经提供的外部视觉条件，情况有所不同。例如，一张图像在生成回答之前已经完成编码，那么当前文本位置通常可以访问这张图像的全部可用视觉特征。此时，交叉注意力不需要像文本自注意力一样，对视觉 Token 按空间索引施加未来文本遮挡。

不过，在处理交错排列的图像、视频和文本时，仍然可能需要限制模型访问尚未出现在当前上下文中的视觉信息。例如，如果某段文本之前只有第一张图像，模型就不应在遵守因果输入协议的情况下提前访问后面的第二张图像。因此，交叉注意力是否需要掩码，取决于视觉信息的组织方式与任务的因果约束。

注意力权重的解释限制 交叉注意力可以产生语言位置与视觉位置之间的权重矩阵。这为观察模型的信息交互提供了一个窗口。但需要注意：

注意力权重较高，并不直接证明模型已经正确理解了该区域的语义，也不意味着该区域就是输出结果的唯一因果来源。

例如，模型可能在杯子周围的桌面区域产生较高注意力，或者多个注意力头通过不同方式共同影响答案。此外，视觉 Value 的内容、输出投影以及后续网络层都会影响最终生成结果。因此，分析交叉注意力时，应当结合视觉条件替换、局部区域遮挡和任务性能等证据，而不是仅根据一张注意力热力图判断模型是否正确完成了视觉定位。

6.4.2 视觉特征作为外部条件

门控交叉注意力 为了控制新增视觉模块对语言模型原有表示的影响，Flamingo 使用了门控机制。设语言隐藏状态为：H_T，视觉特征为：H_V 一个简化的门控交叉注意力残差更新可以写为：

$$H'_T = H_T + \tanh(\alpha)\,\mathrm{CrossAttn}(H_T, H_V) \tag{6.51}$$

其中，α 为可学习的门控参数。在 Flamingo 的设计中，新增交叉注意力输出与后续前馈模块输出分别受到相应的可学习门控控制。例如，可以概念性地表示为：

$$H'_T = H_T + \tanh(\alpha_c)\,\mathrm{CrossAttn}(H_T, H_V) \tag{6.52}$$

$$H''_T = H'_T + \tanh(\alpha_f)\,\mathrm{FFN}(H'_T) \tag{6.53}$$

这里省略了模块内部的归一化和其他实现细节。两个门控分别控制新增的交叉注意力和前馈变换对残差流的贡献。

零初始化门控 Flamingo 中一个重要设计是，将新增模块的门控参数初始化为零：$\alpha_c = 0, \quad \alpha_f = 0$ 由于：$\tanh(0) = 0$，因此，新增模块在初始状态下对语言隐藏状态的残差贡献为零。这使预训练语言模型在训练初始时，能够保持与原有文本模型相一致的

计算行为，减少新加入模块立即扰动已有语言表示的风险。随着训练进行，门控参数可以发生变化，使视觉条件逐渐参与语言模型计算。

门控初始为零并不表示新增模块永远无法学习。门控参数仍然可以根据损失获得梯度，并随着优化过程逐渐允许视觉信息进入残差流。这种设计体现了模块化多模态训练中的一个重要思路：在尽量保留已有语言模型能力的基础上，逐步学习视觉条件对语言生成的影响方式。

冻结语言模型与视觉交互训练 在 Flamingo 的典型训练设计中，预训练视觉编码器和原有语言模型参数保持冻结，而 Perceiver Resampler 及新增交叉注意力相关模块参与训练。这种方式可以将训练重点放在视觉条件如何接入语言模型上。例如，模型根据图像和文本上下文预测下一个 Token。如果预测错误，语言生成损失可以为新增视觉交互模块提供监督。因此，即使语言模型原有参数不更新，新增的跨模态模块仍然能够学习怎样影响语言隐藏状态。

冻结语言模型参数并不意味着可以切断训练损失向新增交叉注意力模块传播所需的计算路径。这一问题与上一节介绍的 Projector 参数训练具有相似的梯度原理。

外部条件的重复访问 对于视觉前缀结构，视觉 Token 通常作为语言模型输入序列的一部分。而采用独立交叉注意力的结构，可以在语言模型的多个网络深度位置重复访问视觉条件。例如，语言模型在较浅的层中形成初步文本表示。进入中间层后，可以根据当前隐藏状态读取视觉信息。随后，在更深的层中，又可以根据更新后的语义表示再次访问视觉特征。因此，视觉条件不必只在语言模型输入端参与一次计算，而可以通过多个交叉注意力模块持续影响隐藏状态。

但这种重复访问并不自动意味着模型会执行多轮正确的视觉推理。实际效果仍然取决于交互模块的训练目标、视觉特征质量及语言模型自身的表示能力。

多图像条件与图文交错输入 Flamingo 的一个重要特点，是能够处理图像与文本交错排列的输入。例如：图像 A。文本：“这是一只狗。”图像 B。文本：“这是一只猫。”随后给出图像 C，并要求模型完成相应描述。这种输入形式要求系统明确不同视觉条件与文本片段之间的对应关系。在 Flamingo 中，语言位置可以根据相应的媒体位置，通过经过掩码约束的交叉注意力访问有关视觉内容。

其设计使用了媒体相关的注意力约束，使文本位置主要访问在输入序列中与其对应的前序视觉媒体，而不是无条件访问所有图像。这有助于保持图文交错序列中的条件关系及其因果顺序。因此，当视觉语言模型处理多张图像或视频时，交叉注意力不只是解决特征维度问题，还需要正确处理不同视觉观测的身份、顺序与可见范围。

6.4.3 交叉注意力与 Token 拼接的区别

Token 拼接如何完成跨模态交互 设视觉条件为：

$$H_V \in \mathbb{R}^{N_V \times D_L} \tag{6.54}$$

文本嵌入为：

$$E_T \in \mathbb{R}^{L \times D_L} \tag{6.55}$$

将二者拼接：$X = [H_V; E_T]$ 然后输入语言模型：

$$H = \text{Transformer}(X) \tag{6.56}$$

在适当的因果掩码设置下，后面的文本 Token 可以通过自注意力访问前面的视觉 Token。因此，视觉信息可以通过语言模型原有的注意力计算参与文本隐藏状态更新。这种方式的主要特点，是将不同模态组织到同一个连续序列中，并复用语言模型的自注意力结构。但视觉 Token 需要满足语言模型的输入隐藏维度要求，其位置和掩码设置也必须符合所采用的生成协议。

Cross-Attention **如何完成跨模态交互** 对于独立交叉注意力，文本隐藏状态和视觉条件分别保留：

$$H_T \in \mathbb{R}^{L \times D_L} \tag{6.57}$$

$$H_V \in \mathbb{R}^{N_V \times D_V} \tag{6.58}$$

然后在指定网络层进行：

$$O = \text{CrossAttn}(H_T, H_V) \tag{6.59}$$

其核心是：语言提供 Query。视觉提供 Key 和 Value。这样，视觉信息不必以普通文本序列位置的形式参与语言模型全部自注意力层。同时，交叉注意力可以使用独立的视觉特征投影和任务相关的注意力掩码。但这通常需要新增计算模块，并涉及这些模块的训练与维护。

两者的信息交互方式不同 对于视觉前缀拼接，视觉和文本在同一自注意力序列中发生交互。如果图像 Token 位于文本 Token 之前，那么后续文本位置可以读取视觉位置的信息。但在典型因果自注意力中，前面的视觉位置不能读取位于其后的问题文本。因此，视觉前缀的早期隐藏状态通常不是根据后面的用户问题直接更新的。不过，后续文本隐藏状态可以根据问题内容，对已经存在的视觉 Token 进行不同的注意力选择。

因此，视觉前缀方法仍然能够产生一定的任务条件视觉信息利用能力。对于交叉注意力，语言模型在某个网络层形成当前文本隐藏状态之后，再以这些隐藏状态作为 Query 访问视觉条件。这使视觉信息读取发生在指定层，并根据该层的文本上下文形成查询。因此，两者的信息交互机制存在区别，但不能简单认为视觉前缀完全不具有动态信息选择能力。

计算复杂度比较 假设文本序列长度为：L，视觉条件长度为：N_V 隐藏维度为 D。对于将视觉与文本直接拼接后执行标准全自注意力的结构，注意力分数计算的序列规模与：$O((L+N_V)^2D)$ 相关。这一表达式描述联合序列中注意力操作的一般复杂度。对于使用独立视觉条件的结构，语言自注意力主要处理文本序列：$O(L^2D)$，同时，语言到视觉的交叉注意力主要包含：$O(LN_VD)$ 的相关计算。

但这两种复杂度不能在脱离模型结构的情况下直接作为端到端效率比较。例如，交叉注意力可能插入多个语言模型层，产生额外投影和前馈计算。而视觉前缀方式可以复用已有语言模型的计算结构，并在自回归生成中使用相应的 Key-Value 缓存。此外，如果交叉注意力之前已经使用 Resampler 压缩视觉特征，其输入长度也可能小于原始视觉 Token 数量。

因此，比较效率时应使用实际的连接器输出长度、交互层数和生成长度，而不是仅根据是否采用 Cross-Attention 判断。

缓存与多轮生成 在自回归生成过程中，前面已经处理过的视觉条件和文本前缀通常不需要在每一步都从头计算。视觉前缀结构可以利用语言模型的 Key-Value 缓存，复用已处理的视觉及历史文本信息。独立 Cross-Attention 结构也可以在条件不变时，复用已经计算好的视觉特征，以及适当的视觉 Key、Value 表示。因此，两种结构都有机会减少重复计算。但缓存内容不同。视觉前缀方法主要复用联合自注意力序列中的历史键值。

Cross-Attention 方法则可以分别管理语言自注意力缓存与视觉条件缓存。在多图像、多轮对话和长视频条件下，这些缓存方式可能具有不同的存储和更新成本。例如，如果新图像进入上下文，需要决定是否重新编码视觉输入、是否扩展视觉缓存，以及哪些文本位置能够访问新的视觉条件。因此，真实系统中的计算效率还与缓存管理和多模态输入组织方式有关。

特征维度和参数结构 视觉前缀方式通常要求 Projector 输出的视觉 Token 与语言模型输入嵌入具有相同维度。例如：$D_{\text{proj}} = D_L$，这样才能按照模型接口进行序列拼接。而 Cross-Attention 可以通过独立的 Query、Key 和 Value 投影矩阵处理两种特征。因此，视觉条件原始维度与语言隐藏维度不必完全相同。

不过，这并不意味着 Cross-Attention 不需要学习模态之间的特征对应。相反，独立投影矩阵和注意力模块本身也需要通过训练，学习视觉特征怎样影响语言隐藏状态。因此，两种方法都需要跨模态参数适配，只是参数所在位置和作用方式不同。

对预训练语言模型的改动程度 使用 Token 拼接时，通常可以保留语言模型原有的 Transformer 层结构，只在输入侧新增 Projector。这是一种较直接的模块化连接方式。但如果语言模型原本只使用文本进行预训练，仍然需要通过连接训练或视觉指令微调，使其学会利用连续视觉嵌入。使用 Flamingo 式 Cross-Attention 时，则需要在语言模型内部插入新的注意力模块。

其优点之一，是可以保持原有语言模型权重冻结，并通过新增模块学习视觉条件注入。但这种方法也需要修改或包装语言模型的中间层计算结构。因此，简单 Projector

与独立 Cross-Attention 在模型工程、参数管理和推理实现方面具有不同的复杂度。不能只根据新增参数数量判断哪种方法更容易训练或部署。

6.5 三种经典架构

三种实例不要求采用相同训练阶段。比较其贡献时，应分别记录连接器结构、主干冻结范围、数据和目标；将一个系统的完整后训练收益全部归因于连接器，会混淆变量。

6.5.1 BLIP-2：冻结编码器与连接模块

BLIP-2 的整体架构 BLIP-2 主要包含三个部分：预训练视觉编码器。查询式 Transformer，即 Q-Former。预训练语言模型。其基本数据流可以表示为：

$$I \xrightarrow{f_V} Z_V \xrightarrow{\text{Q-Former}} H_Q \xrightarrow{P} H_L \xrightarrow{\text{LLM}} Y \tag{6.60}$$

其中：

- (I) 为输入图像；
- f_V 为冻结的视觉编码器；
- Z_V 为视觉特征；
- H_Q 为 Q-Former 提取的查询表示；
- (P) 为必要的视觉—语言投影；
- H_L 为提供给语言模型的视觉条件；
- (Y) 为语言生成结果。

与普通逐 Token Projector 不同，Q-Former 使用有限数量的可学习查询，从完整视觉特征中提取与语言有关的信息。因此，Q-Former 不仅承担特征转换作用，还承担视觉信息聚合和跨模态表示学习任务。

第一阶段：视觉语言表示学习 BLIP-2 采用分阶段预训练。第一阶段的主要目标，是在冻结视觉编码器的条件下，训练 Q-Former 学习与语言相关的视觉表示。这一阶段使用三种主要目标：图文对比学习（Image-Text Contrastive Learning，ITC）。图文匹配（Image-Text Matching，ITM）。图像条件文本生成（Image-grounded Text Generation，ITG）。可以概括为：

$$\mathcal{L} = \mathcal{L}_{ITC} + \mathcal{L}_{ITM} + \mathcal{L}_{ITG} \tag{6.61}$$

这里省略了具体实现中的损失归一化和训练细节，用于说明主要目标的组合。ITC 促使查询提取的视觉表示与文本语义建立对应关系。ITM 通过图文配对判断，进一步训练视觉与语言之间的联合理解。ITG 则要求模型根据视觉条件进行文本预测，使查

询表示能够为语言生成提供有用信息。

BLIP-2 的 Q-Former 并不只是视觉查询网络，还包含与文本处理相关的 Transformer 结构。不同预训练目标通过不同注意力掩码，控制查询表示与文本表示之间的信息交互。例如，在图文对比学习中，需要保证分别构建的图像与文本表示不会提前泄漏彼此的信息。而在图文匹配任务中，可以允许图文表示进行更充分的联合交互。因此，第一阶段实际上是在训练一种语言相关的视觉信息提取接口。

第二阶段：连接冻结语言模型 完成视觉语言表示学习后，BLIP-2 进一步将 Q-Former 与预训练语言模型连接。这一阶段的主要目标是：使冻结语言模型能够利用 Q-Former 输出的视觉特征进行条件文本生成。设 Q-Former 输出为：H_Q，经过必要的线性投影得到：$H_L = H_Q W_P$，其中，H_L 的特征维度与语言模型输入接口相匹配。对于自回归语言模型，可以将这些特征作为连续视觉前缀，与文本输入共同组织。

语言生成过程可以概念性地表示为：

$$P(Y \mid I, Q) = P_{\text{LLM}}\left(Y \mid H_L, Q\right) \tag{6.62}$$

BLIP-2 也研究了与 Encoder–Decoder 类型语言模型的连接，因此具体的视觉条件组织和生成损失会随语言模型结构而变化。在这一阶段，视觉编码器和预训练语言模型仍然保持冻结，主要训练 Q-Former 以及相应的连接参数。这种设计使视觉语言生成能力的建立，不必依赖对整个语言模型进行大规模参数更新。

冻结策略与训练成本 BLIP-2 的一个重要特点，是通过冻结两端较大的预训练模型，控制需要更新的参数规模。例如，可以将参数划分为：θ_V ：视觉编码器参数，θ_Q ：Q-Former 参数，θ_L : 语言模型参数 典型训练设置中：

$$\theta_V, \theta_L \text{ 冻结} \tag{6.63}$$

而：θ_Q 参与训练 连接投影等必要参数也参与相应阶段的优化。这样，优化器不必为冻结的全部视觉编码器和语言模型权重维护完整的训练状态。但是，冻结参数并不意味着这些模块没有计算开销。视觉编码器仍然需要执行图像特征提取。语言模型仍然需要完成文本条件预测。如果需要通过冻结语言模型向 Q-Former 传递梯度，也必须保留必要的求导路径。

因此，BLIP-2 的参数效率主要体现为减少可训练参数及相关优化开销，不能简单理解为整个模型的训练或推理成本与参数减少比例相同。

6.5.2 Flamingo：跨注意力与多模态序列

Flamingo 的基本结构 Flamingo 主要包含以下模块：预训练视觉编码器。Perceiver Resampler。预训练自回归语言模型。插入语言模型内部的门控交叉注意力与前馈模块。其主要数据流可以概括为：

$$I \xrightarrow{f_V} Z_V \xrightarrow{\text{Resampler}} H_V \tag{6.64}$$

随后，语言模型根据文本隐藏状态，通过交叉注意力访问视觉表示：

$$H'_T = F_{\text{gated}}(H_T, H_V) \tag{6.65}$$

这里，F_{gated} 表示新增的视觉条件交互模块。与 BLIP-2 使用 Q-Former 提取特征、再将其作为语言模型条件输入不同，Flamingo 在语言模型内部设置专门的视觉交互路径。因此，语言模型不仅在输入端接收视觉条件，还可以在指定网络层通过交叉注意力读取视觉特征。

多模态交错序列 Flamingo 的重要特点，是能够处理图像或视频与文本任意交错排列的输入。例如：图像 A → 文本 A → 图像 B → 文本 B → 图像 C → 问题。

对于这种输入，模型需要知道不同文本位置对应哪些视觉媒体。如果所有文本位置都能够不受限制地访问后续图像，就可能破坏输入序列的因果顺序。因此，Flamingo 对视觉交叉注意力使用相应的媒体可见性约束。在原始设计中，某个文本位置的交叉注意力主要访问其前面最近出现的视觉媒体所对应的特征，而不是直接访问此前所有媒体的视觉 Token。

同时，语言模型仍然可以通过文本自注意力保留此前上下文的信息，因此模型仍然能够在一定程度上利用较早视觉媒体产生的上下文影响。这种机制有助于将多模态输入组织为符合序列顺序的条件生成过程。这种设计并不等同于把所有视觉媒体直接拼接为一个无区别的巨大特征序列。Flamingo 特别关注多模态输入中不同媒体的位置、顺序及其与文本之间的关系。

上下文少样本学习 Flamingo 的研究重点之一是多模态上下文少样本学习（Multimodal In-context Few-shot Learning）。其基本思想是，在推理时向模型提供少量示例，使模型根据这些示例完成当前任务，而不必为每个新任务专门更新模型参数。例如，可以提供：示例一：图像：一只猫。问题：“这是什么动物?” 回答：“猫。” 示例二：图像：一只狗。问题：“这是什么动物?” 回答：“狗。”

随后提供新的图像和问题，让模型根据上下文形式继续回答。这种少样本学习并不等同于使用少量标注数据进行梯度微调。在上下文少样本学习中，示例通常作为模型输入上下文，模型参数在推理期间保持不变。而少样本微调则需要利用少量训练样本更新模型参数。因此，二者虽然都使用少量示例，但适配机制并不相同。

训练数据与目标 为了学习多模态序列建模能力，Flamingo 使用包含图像、视频和文本的多种训练数据。其中，具有交错图文结构的网络数据，对于学习多模态上下文的组织方式具有重要作用。模型的主要训练目标是根据已有多模态上下文预测后续文本。可以概括为：

$$\mathcal{L} = -\sum_{t} \log P(y_t \mid y_{<t}, V_{\leq t}) \tag{6.66}$$

其中，$V_{\leq t}$ 表示按照输入顺序对当前文本预测可见的视觉条件。这里的符号用于概括媒体因果约束，并不要求视觉输入一定采用逐时刻流式格式。通过这种训练，模型可以学习根据不同图像或视频内容生成描述、回答问题，并使用上下文中的示例。与 BLIP-2 的 Q-Former 第一阶段同时采用 ITC、ITM 和 ITG 不同，Flamingo 更突出多模态交错序列的条件语言建模。

因此，两者的训练设计是与各自的目标能力相联系的。

参数冻结与训练方式 Flamingo 同样强调复用大型预训练模型。在原始设计中，预训练视觉编码器和语言模型的原有模块保持冻结，而新增的 Perceiver Resampler、门控交叉注意力及相关模块参与训练。这种方式能够在已有语言模型基础上引入新的视觉条件接口。但与 BLIP-2 只需训练中间 Q-Former 和相关投影不同，Flamingo 需要在语言模型内部插入多个视觉交互模块。

因此，其新增可训练参数与计算成本会受到语言模型规模、交叉注意力层数及其他结构选择影响。这说明，即使两种模型都采用冻结预训练主干的策略，实际可训练模块和系统成本也可能存在明显差异。

6.5.3 LLaVA：视觉指令微调的基本路线

LLaVA 的基本结构 原始 LLaVA[14] 的基本架构包含：预训练 CLIP[8] 视觉编码器。可训练线性 Projector。预训练自回归语言模型。其基本过程可以写为：$Z_V = f_V(I)$，$H_V = P_\phi(Z_V)$，视觉特征映射完成后，与文本指令嵌入共同组织为语言模型的输入。例如：$X = [H_V; E_Q]$，随后，语言模型根据图像与用户指令生成回答。这一结构与 Flamingo 存在明显区别。

LLaVA 不需要在语言模型内部另外插入门控交叉注意力层，而主要通过输入侧的视觉 Token 组织，让语言模型原有的自注意力机制处理视觉与文本信息。因此，它是一种较为直接的模块化视觉语言连接方案。

视觉指令数据的构建 视觉指令微调需要具有图像、指令和回答的数据。例如：图像：一张包含汽车和行人的街道照片。用户指令：“图像中有哪些交通参与者？”参考回答：“图像中有汽车和行人。”与普通图像描述数据相比，视觉指令数据具有更加明确的任务要求。同一张图像可以用于构造多种任务。例如，用户可以询问对象类别、颜色、数量或整体场景，也可以要求模型比较多个对象之间的关系。因此，视觉指令微调不仅要求模型识别图像内容，还要求它根据文本指令选择和表达信息。

原始 LLaVA 的一个重要贡献，是探索利用已有语言模型生成多模态指令数据。研究者使用图像描述、对象边界框等图像相关的文本或结构化信息，为语言模型提供场景依据，再利用语言模型生成不同类型的指令与回答。原始 LLaVA 的相关数据构建使用的是语言版 GPT-4，主要通过图像的文本化描述及结构化场景信息提供条件，而不能简单描述为 GPT-4 直接观察全部训练图像并完成标注。

三类视觉指令数据 在原始 LLaVA 的数据构建中，具有代表性的训练样本包括三种类型。第一类是多轮对话。例如，围绕一张街道图像，用户连续询问：“图像中有什么？”“汽

车是什么颜色？”“它停在什么地方？”这类数据帮助模型学习在对话上下文中理解后续问题。第二类是详细描述。例如，要求模型较完整地描述图像中的主要对象、场景和相关视觉细节。这类数据能够训练模型产生比简短图像标题更丰富的自然语言输出。第三类是复杂推理型问答。

例如，给定图像和问题，要求模型结合场景内容进行解释或判断。但这类数据中的解释并不总能代表经过严格验证的视觉推理过程。由语言模型生成的数据仍然可能包含不准确的事实或推断。因此，视觉指令数据的类型越丰富，越需要关注其视觉依据是否充分。

第一阶段：视觉语言特征对齐 原始 LLaVA 采用分阶段训练。第一阶段主要训练视觉连接模块，使预训练视觉编码器的特征能够被语言模型有效利用。在这一阶段：视觉编码器冻结。语言模型冻结。主要更新 Projector 参数。可以概括为：

$$\theta_V, \theta_L \text{ 冻结} \tag{6.67}$$

θ_P 训练 训练数据主要采用图像与描述文本构成的配对样本。原始论文使用从 CC3M 筛选得到的约 59.5 万组图文数据，构建相应的特征对齐训练。通过图像条件语言生成损失，模型训练 Projector，将视觉特征转换为语言模型能够利用的连续条件表示。这里的“特征对齐”不表示视觉 Token 必须与词表中的某个文本 Token 一一对应。

它更接近于使冻结语言模型能够根据视觉条件正确预测相关文本。

第二阶段：视觉指令微调 完成初始视觉连接训练后，LLaVA 进一步使用视觉指令数据训练模型。在原始 LLaVA 的这一阶段：视觉编码器仍然冻结。Projector 继续训练。语言模型参数参与更新。因此，可以表示为：θ_V 冻结，

$$\theta_P, \theta_L \text{ 训练} \tag{6.68}$$

模型根据图像和用户指令，学习生成相应回答。训练目标通常采用自回归条件语言建模：

$$\mathcal{L} = -\sum_{t \in \mathcal{A}} \log P_\theta(y_t \mid y_{<t}, I, Q) \tag{6.69}$$

其中，$\mathcal{A}$ 表示参与回答监督的目标 Token 位置。这一阶段与初始图文特征对齐的主要区别，不在于完全更换模型架构，而在于训练数据包含更加丰富的用户指令与回答形式，语言模型也能够参与适配。原始论文使用“端到端微调”等表述时，仍应结合具体冻结策略理解：其视觉编码器在这一阶段保持冻结，并非全部视觉语言模型参数均参与更新。

LLaVA-1.5 的改进 LLaVA-1.5 是在经典 LLaVA 基础上形成的重要改进版本。它保留了视觉编码器、连接模块和语言模型的模块化思路，但对部分结构和训练配置进行调

整。其中，具有代表性的变化包括：采用 CLIP ViT-L/14 的 336 像素输入配置。使用两层 MLP 代替原始的简单线性连接模块。进一步利用公开的视觉问答等任务数据，并通过相应的回答格式提示改进训练。

这些改动表明，视觉语言模型性能不仅受到连接模块影响，还与图像分辨率、训练数据类型和指令格式密切相关。尤其不能将 LLaVA-1.5 相比原始 LLaVA 的性能差异全部归因于 MLP。因为视觉输入配置与数据组织等因素也发生了变化。因此，LLaVA-1.5 的研究启示，是相对简单的模型连接结构结合适当的数据和训练策略，也可以形成具有竞争力的视觉语言基线。

6.6 架构选择的诊断方法

6.6.1 架构与任务需求的匹配

OCR 与文档理解 OCR 和文档理解往往需要较高的视觉细节保留能力。例如，一张文档图像包含多列文字、表格、数字和图形。用户提出：“请读取表格第三行的数值。”模型不仅需要识别文本，还需要理解版面结构、行列关系和相应位置。如果图像在预处理阶段被过度缩小，某些字符可能难以辨认。如果连接模块对大量视觉 Token 进行强压缩，某些局部文字信息也可能丢失。因此，对于文档理解任务，应重点考虑高分辨率输入、多区域处理和布局信息保留。

同时，模型还需要通过相关数据训练，学习文字内容与空间结构之间的联系。更高分辨率并不保证表格推理一定正确。例如，模型可能准确识别表格中的所有数字，却仍然选择错误的行或列。因此，视觉细节识别与文档结构理解需要分别评价。

对象计数与空间关系 对象计数需要区分多个相似实例。例如，图像中存在五个杯子。模型需要判断：Count = 5，而不是仅仅识别“图像中存在杯子”。因此，如果视觉 Token 压缩过度聚合多个对象，就可能损失实例级信息。空间关系任务则需要保留对象位置与相互关系。例如：“红色杯子位于蓝色书本左侧。”模型必须识别杯子、书本及其位置关系。因此，这类任务通常需要更加充分的局部特征与空间结构表示。

但保留更多 Patch 并不必然带来准确计数和空间推理。模型还需要经过适当的训练，使其能够利用实例边界和对象关系，而不只是学习对象语义的整体共现。

语言条件目标定位 对于语言条件定位任务，用户可能提出：“找到戴黄色安全帽、站在红色工具箱旁边的工作人员。”模型不仅需要理解文本，还需要输出对象的具体位置。因此，单纯依赖生成式语言回答可能不足以满足任务要求。例如，如果系统最终需要输出边界框：$b = (x_1, y_1, x_2, y_2)$ 就需要明确这些坐标如何从视觉表示中产生，以及怎样与原始图像坐标关联。一种选择是利用具有区域特征的视觉编码器，结合语言条件定位模块。

另一种方式是使用经过相应空间监督的多模态生成模型，输出结构化坐标或其他空间表示。但无论采用哪种方式，都需要评价预测区域与真实对象位置之间的一致性。因此，架构选择不应只依据语言模型是否能够描述目标，而应当取决于目标任务是否

需要明确、可验证的空间输出。

多图像比较 在多图像任务中，模型需要同时保留多个视觉观测及其身份信息。例如：“比较第一张和第二张图像中汽车的位置变化。”如果两个图像的特征被无区别地合并，就可能难以判断哪些对象属于第一张图像，哪些属于第二张图像。因此，模型需要合理组织图像顺序、视觉位置和跨图像对应关系。可以采用为不同图像分别编码，再通过统一序列或独立条件表示进行交互的方式。但能够接收两张图像，并不意味着模型已经学会判断同一对象在不同图像中的对应关系。

例如，两个场景中可能出现外观相似的汽车，模型仍然可能错误匹配对象身份。因此，多图像模型还需要相应的比较任务监督与评价。

视频与时序任务 对于视频理解，模型需要在空间语义之外处理时间信息。例如：“人物先打开门，还是先进入房间?”这一问题需要识别动作及其发生顺序。如果模型只对若干独立视频帧提取语义，然后进行简单平均，可能丢失事件顺序。因此，视频任务需要考虑帧采样、时间位置编码、跨帧信息聚合和长时间上下文管理。Flamingo 的图文交错序列设计，为多媒体条件语言生成提供了代表性架构思路。

但对于需要精确运动轨迹或高频状态更新的任务，仅采用稀疏视频帧和通用图文表示可能不足。因此，视频视觉语言架构应当根据任务需要的时间分辨率与时序关系精度进行设计。

领域专用视觉语言任务 对于专业领域任务，还需要考虑通用视觉预训练与目标数据之间的差异。例如，工业巡检可能需要识别细小缺陷。遥感图像可能包含大范围空间区域和密集目标。医学影像可能需要结合专业语义和精细结构。这些任务中的视觉特征分布，与常见互联网自然图像可能存在明显差异。因此，直接使用通用视觉语言模型可能不足以满足专业要求。可以优先评估预训练视觉编码器是否能够保留目标特征，再决定是否采用领域适配或视觉编码器微调。

但在专业领域中，语言模型已有的常识也可能与目标任务的严格定义不同。因此，专业视觉语言系统的训练和评价需要明确领域标准，而不能只依赖通用图文语义相关性。

架构选择的多目标优化 视觉语言模型架构可以被理解为一个受到性能和资源约束的优化问题。设候选架构为：

$$a \in \mathcal{A} \tag{6.70}$$

其任务性能为：$Q(a)$ 推理延迟为：$T(a)$ 显存占用为：$M(a)$ 训练成本为：$C(a)$ 可以概念性地写为：

$$\max_{a \in \mathcal{A}} Q(a) \tag{6.71}$$

满足：$T(a) \le T_{\max}$，$M(a) \le M_{\max}$，$C(a) \le C_{\max}$，其中，$T_{\max}$、$M_{\max}$ 和 $C_{\max}$ 分别表示实际任务允许的推理延迟、显存占用和训练成本上限。这一形式说明，模型选

择并不一定追求某个公开基准上最高的绝对准确率，而是需要在资源约束下获得满足任务需求的性能。

如果还需要考虑可靠性，可以进一步引入鲁棒性或风险相关约束。例如，某个模型在普通图像描述中取得较高成绩，但在目标领域中经常错误识别关键对象，那么其总体性能优势不一定足以满足实际应用要求。

选择简单架构作为基线 对于新的视觉语言任务，一种具有较强可操作性的研究方式，是首先采用成熟、结构清晰的基础架构。例如，可以使用预训练视觉编码器、MLP Projector 和预训练语言模型，构建一个能够运行的基线系统。随后，对其进行任务评价。如果模型主要在小目标或 OCR 上表现不足，应首先分析图像预处理和视觉特征是否保留了足够细节。如果模型存在明显的视觉 Token 计算瓶颈，可以考虑查询式压缩。

如果模型需要处理复杂交错多模态输入，可以进一步研究独立视觉条件管理和跨注意力机制。如果模型已经具备足够视觉信息，但回答不符合任务指令，则可以优先分析训练数据与语言侧适配。这种从基线出发、逐步验证的方式，有助于明确不同改动的实际作用，避免同时增加大量模块而难以解释性能变化。简单架构优先并不意味着复杂架构没有价值。它强调的是：应当让架构复杂度由明确的任务需求和实验依据驱动，而不是单纯追求更多网络模块。

习题与讨论

1. 对 $Z \in \mathbb{R}^{N \times d_v}$，给出线性与两层 MLP 连接器的输出形状和参数量。说明它们是否必然减少 Token 数。
2. 冻结语言模型时，推导 $\nabla_\phi \mathcal{L}$ 的链式表达。解释为什么关闭整个语言模型的自动求导可能使训练失败。
3. 比较文本查询读取视觉键值与视觉查询先压缩再拼接的计算位置。两者的压缩结果是否同样依赖当前问题？
4. 固定视觉编码器与语言模型，设计一个连接机制消融。分别控制视觉预算、可训练参数和训练数据。

文献导读 BLIP-2、Flamingo 和 LLaVA 分别用于追踪查询桥接、门控交叉注意力与视觉指令路线。先画计算图，再读取其冻结策略。[12, 13, 14]

第七章　多模态融合与统一架构

融合需要回答信息在哪里相遇、哪些参数共享、哪些输出由同一状态产生。把所有方案都称作“统一”，会掩盖输入映射、注意力可见性、预测头和损失之间的差别。本章沿计算路径比较这些选择，并把信息保留与任务竞争作为设计条件。

学习目标　按输入、交互、参数和目标四个维度描述架构；计算序列交互代价；辨认共享语义、专属信息和专家路由的作用。

7.1　融合发生在哪里

输入级、特征级与输出级描述融合位置；连续与离散描述表示形式；共享与专属描述参数关系。这些分类轴可以交叉组合。读到一个融合名称时，应先画出观测到输出的路径，再判断它落在哪些分类轴上。

7.1.1　数据级与输入级融合

基本思想　数据级融合（Data-level Fusion）通常指在较接近原始观测的阶段，对不同模态的信息进行组合。输入级融合（Input-level Fusion）则强调在模型的主要特征提取过程之前，将不同来源的输入组织为一个联合表示。两者具有较强联系，但并不完全等价。数据级融合更侧重原始测量值及其物理或统计关系；输入级融合则可能包含必要的预处理、特征标准化和统一输入组织。

例如，在 RGB-D 感知中，RGB 图像提供颜色信息，深度图提供像素对应的距离信息。设 RGB 图像为：

$$I_{\mathrm{RGB}} \in \mathbb{R}^{H\times W\times 3} \tag{7.1}$$

深度图为：

$$D \in \mathbb{R}^{H\times W\times 1} \tag{7.2}$$

如果两者具有合理的空间对应关系，可以沿通道维度拼接：

$$X = \mathrm{Concat}(I_{\mathrm{RGB}}, D) \tag{7.3}$$

得到：

$$X \in \mathbb{R}^{H\times W\times 4} \tag{7.4}$$

随后，将四通道输入提供给相应的视觉网络。在这一结构中，两种模态在进入主要视觉特征提取网络之前就已经组合，因此属于典型的输入级融合。但这种融合方式的有效性，依赖 RGB 像素与深度数据之间具有合理的空间对应关系。

空间配准的作用 在涉及多个空间传感器的任务中，数据级融合通常需要合理的空间配准。例如，一台机器人同时具有 RGB 相机和深度相机。如果两者存在不同视角，那么深度图中的某个位置未必对应 RGB 图像中的相同像素坐标。因此，可能需要利用传感器标定参数和几何投影，将深度信息转换到 RGB 相机坐标系统中。只有建立合理的空间对应后，才能进行具有物理意义的逐像素融合。

例如，可以利用相机内参和外参，将三维点：$P = (X, Y, Z)$ 投影到图像平面：

$$u = f_x \frac{X}{Z} + c_x \tag{7.5}$$

$$v = f_y \frac{Y}{Z} + c_y \tag{7.6}$$

其中，f_x, f_y, c_x, c_y 为相应相机参数。该公式表达的是经过适当坐标变换后、在针孔相机模型下的简化投影关系，实际系统还可能需要考虑畸变、遮挡和深度有效性。因此，空间配准是部分数据级融合的重要前提，但配准正确本身并不能保证语义融合有效。

时间同步与不同采样频率 对于具有时间序列性质的模态，早期融合还需要考虑时间对应。例如：RGB 相机以 30 Hz 采样。IMU 以 200 Hz 采样。事件相机则异步记录局部亮度变化事件。如果希望在某个时刻联合使用这些观测，就需要确定哪些数据属于同一时间范围。一种简单方式是选择目标时间：t 然后构造相应输入：

$$X_t = \Phi\left(I_t, U_{[t-\Delta,t]}, E_{[t-\Delta,t]}\right) \tag{7.7}$$

其中：

- I_t 表示相应时刻的图像；
- $U_{[t-\Delta,t]}$ 表示时间窗口内的惯性观测；
- $E_{[t-\Delta,t]}$ 表示对应窗口内的事件流；
- Φ 表示输入组织方式。

例如，可以将一段时间内的事件流转换为事件帧、事件体素或其他时间窗口表示，再与 RGB 图像联合处理。但这样做也会改变事件流原有的时间表达方式。如果为了与低频 RGB 图像进行输入级融合，而将事件全部累积到较长时间窗口，可能弱化事件数据的高时间分辨率优势。因此，早期融合虽然便于建立统一输入，但也可能要求不同模态采用共同的数据采样或表示粒度。

输入级融合的局限 输入级融合的主要挑战之一，是不同模态的原始数值和统计特性可能存在较大差异。例如，RGB 像素描述光学颜色信息，而深度值表示空间距离。即使二

者能够拼接为四通道张量，也不意味着四个通道应当采用完全相同的归一化方式。此外，部分模态可能存在大量无效值。例如，深度相机可能在透明或反光表面产生测量缺失。如果直接将无效深度作为普通数值输入网络，模型可能学习到错误的统计关联。对于异步传感器，输入级融合还可能依赖时间窗口划分或插值。

如果这些操作与实际观测时间不一致，就可能引入额外误差。因此，早期融合通常较适合具有明确对应关系、数据质量较稳定，并且能够建立合理联合输入表示的模态组合。

7.1.2 特征级与中间层融合

从原始数据组合到特征交互 当不同模态在数据形式、采样频率或统计特征上存在较大差异时，直接组合原始输入可能不够合理。例如，RGB 图像、点云和音频波形具有不同的数据结构。图像通常采用规则像素网格。点云由三维空间中的离散点组成。音频则主要表现为随时间变化的连续信号。如果要求这些模态全部采用同一种原始数据表示，可能需要大量预处理，并造成部分模态特有信息损失。

因此，特征级融合（Feature-level Fusion）通常先使用相应编码器提取不同模态的特征，再在特征空间中进行组合。设两种模态输入为：$X_1,\ \ X_2$，对应编码器为：$f_1,\ \ f_2$ 首先得到：$Z_1 = f_1(X_1)$，$Z_2 = f_2(X_2)$，随后执行：

$$H_F = \Phi_{\text{feature}}(Z_1, Z_2) \tag{7.8}$$

最终：$Y = G(H_F)$，其中，Φ_{feature} 表示特征融合模块。与数据级融合相比，这种方式允许不同模态在融合之前使用各自适合的表示学习方法。

特征拼接 最简单的特征级融合方式之一，是特征拼接。例如，两个编码器分别产生：

$$z_1 \in \mathbb{R}^{D_1} \tag{7.9}$$

$$z_2 \in \mathbb{R}^{D_2} \tag{7.10}$$

可以构造：$h_F = [z_1; z_2]$，其中：

$$h_F \in \mathbb{R}^{D_1+D_2} \tag{7.11}$$

然后通过 MLP 或其他网络进行预测：$Y = G_\theta(h_F)$，这种方式能够保留来自两个模态的特征入口，使后续模型学习它们之间的组合关系。但拼接本身并不代表已经充分利用了模态之间的互补信息。例如，如果后续网络主要依赖其中一种模态，那么另一种模态虽然被拼接进入系统，却可能对最终预测几乎没有贡献。因此，评价特征拼接的有效性，需要观察联合模型是否真正利用了各模态的信息。

特征加权融合 另一类方法，是在不同模态特征之间执行加权组合。设两个模态已经映射到相同维度：

$$Z_1, Z_2 \in \mathbb{R}^D \tag{7.12}$$

可以定义：

$$H_F = \alpha Z_1 + (1-\alpha) Z_2 \tag{7.13}$$

其中：$0 \leq \alpha \leq 1$ 表示融合权重。当：$\alpha = 0.5$ 时，得到简单均值融合。当 α 由模型学习或根据输入条件动态计算时，则可以形成自适应加权融合。例如：

$$\alpha = \sigma\left(g_\theta(Z_1, Z_2)\right) \tag{7.14}$$

随后利用该权重决定不同模态的贡献。但这种加权方式具有必要前提：两个特征必须已经具有适合组合的语义及数值尺度。例如，将没有经过适配的 RGB 特征和音频特征直接求平均，通常缺乏明确的表示含义。此外，学习得到的融合权重并不必然等价于传感器可靠性。例如，较大的权重可能只是模型在当前数据分布下形成的任务偏好，而不意味着对应模态的测量一定更加准确。

基于注意力的特征融合 随着 Transformer 的发展，注意力成为多模态特征交互的重要机制。例如，可以使模态 1 的特征作为 Query，模态 2 的特征作为 Key 和 Value：

$$H_{1\leftarrow 2} = \text{Attention}\left(Z_1 W_Q, Z_2 W_K, Z_2 W_V\right) \tag{7.15}$$

这种机制允许一种模态根据自身当前表示，从另一模态中读取相关信息。例如，在视听语音识别中，音频特征可以根据当前声音片段，从视觉特征中检索相关的唇部运动信息。类似地，视觉特征也可以根据图像内容，从音频中提取相关的声音信息。注意力并不要求两个模态具有完全相同的序列长度。例如，音频和视频通常具有不同的采样频率，交叉注意力可以在两个不同长度的序列之间建立学习式关联。

MulT（Multimodal Transformer for Unaligned Multimodal Language Sequences）便探索了利用定向跨模态注意力，学习语言、视觉和音频等不同时间序列之间的关系。但是，注意力能够处理不同长度的输入，并不意味着时间同步和因果约束可以被无条件忽略。如果实际任务要求严格的实时预测，模型仍然需要防止读取尚未发生的未来观测。

多层次特征融合 不同网络层所表达的信息通常具有不同粒度。例如，视觉编码器较浅层可能更关注局部纹理和边缘，较深层则可能形成更加抽象的对象和场景表示。因此，可以在不同深度建立跨模态信息交互。例如：

$$H^{(l+1)} = F^{(l)}\left(H^{(l)}, Z_1^{(l)}, Z_2^{(l)}\right) \tag{7.16}$$

这种方式不要求所有信息仅在某个固定层级融合一次。例如，较早阶段可以融合局部结构信息，较晚阶段再结合更加抽象的语义表示。但不同层级的功能并不总具有

明确固定的语义划分。因此，不能简单认为浅层融合一定负责细节，中层融合一定负责关系，深层融合一定负责推理。其实际作用仍然需要通过特征分析和任务消融验证。

受限的信息交换与融合瓶颈 如果两个模态均包含较长的 Token 序列，直接执行完整的跨模态两两注意力可能具有较高的计算成本。一种研究思路，是设置少量专门的融合 Token，使不同模态之间的信息经过受限的中间表示进行交换。Attention Bottlenecks for Multimodal Fusion 是这一技术方向的代表性工作。

该方法通过少量融合瓶颈 Token，在视觉和音频等模态之间建立信息交换通道，避免要求全部跨模态 Token 始终进行完整交互。这种设计体现了一个重要思想：

多模态融合并不一定要求交换每个模态的全部特征，而可以通过任务相关的中间表示进行有选择的信息传递。

但融合瓶颈的存在也可能限制信息带宽。对于需要精细局部对应或高时间分辨率的任务，过少的中间 Token 可能无法保留充分的细节。因此，融合瓶颈数量与信息交换方式需要结合任务要求确定。

7.1.3 决策级与输出级融合

基本思想 数据级和特征级融合通常要求不同模态在模型内部较早发生交互。决策级融合（Decision-level Fusion）则采用另一种思路：先分别利用不同模态完成相应预测，再在较靠后的阶段组合预测结果。例如，一个视听情感识别系统具有：音频分类模型。视频分类模型。前者根据声音特征预测情感类别，后者根据面部表情预测情感类别。然后，系统根据两种预测结果形成最终判断。这种方式属于典型的晚期融合（Late Fusion）。

设两种模态分别得到：

$$p_1(y \mid X_1) \tag{7.17}$$

$$p_2(y \mid X_2) \tag{7.18}$$

融合模块进一步计算：

$$p(y \mid X_1, X_2) = \Phi_{\text{decision}}\left(p_1(y \mid X_1), p_2(y \mid X_2)\right) \tag{7.19}$$

其中，Φ_{decision} 表示预测结果的组合方式。这里假设两种模态输出能够映射到相同的目标变量 y。若不同模型的预测空间不同，则需要先建立合理的任务对应关系。

概率加权融合 一种常见的决策级融合方式，是对不同模态的类别概率进行加权平均。设：

$$p_m(y \mid X_m) \tag{7.20}$$

表示第 m 个模态模型预测的类别概率分布。可以构造：

$$p_F(y) = \sum_{m=1}^{M} w_m p_m(y \mid X_m) \tag{7.21}$$

其中：$w_m \geq 0$ 并满足：

$$\sum_{m=1}^{M} w_m = 1 \tag{7.22}$$

在各模型输出具有相同类别定义且概率有效的情况下，这种加权平均仍然是一个归一化概率分布。例如，某个场景中存在两个分类模型。RGB 模型认为目标属于“行人”的概率为 0.8。另一传感器模型给出 0.6。如果使用相同权重：$w_1 = w_2 = 0.5$ 可以得到：$p_F(\text{pedestrian}) = 0.7$，这种方式简单、容易实现。

但它能否提高性能，取决于各模型预测质量及错误之间的关系，而不是取决于是否使用了多个模态。

决策权重的含义 决策融合可以使用固定权重，也可以根据输入条件动态调整权重。例如：

$$w_m = \frac{\exp(a_m)}{\sum_j \exp(a_j)} \tag{7.23}$$

其中，a_m 可以由模型根据当前输入特征计算得到。这种方式可以使模型在不同条件下改变模态贡献。例如，在具有强烈背景噪声的场景中，视听识别系统可以尝试更多利用视觉唇部信息。但动态权重并不天然代表经过验证的模态可靠性。如果权重网络在训练中主要学习到数据集中的常见共现模式，那么它可能在分布变化时产生错误选择。此外，一个模态模型给出较高置信分数，也不意味着其预测必然正确。

因此，决策权重应当结合预测校准、质量信息和目标任务进行评价。

概率融合与 Logit 融合 在分类任务中，模型可能输出 Logit：

$$\ell_m \in \mathbb{R}^K \tag{7.24}$$

随后通过 Softmax 转换为类别概率：

$$p_m = \text{Softmax}(\ell_m) \tag{7.25}$$

可以在概率层面进行融合，也可以先组合 Logit。例如：

$$\ell_F = \sum_m w_m \ell_m \tag{7.26}$$

随后计算：

$$p_F = \text{Softmax}(\ell_F) \tag{7.27}$$

但 **Logit 加权与概率加权不是同一种运算，一般不会得到相同的结果**。此外，不同模型的 Logit 尺度可能明显不同。例如，一个模型输出绝对值较大的 Logit，并不必然说明其预测质量高于另一个模型。如果未经校准直接组合，可能使某个模型在数值上占据过大权重。因此，决策融合需要明确使用的是类别分数、Logit 还是经过校准的概率，以及不同输出之间是否可比较。

决策级融合的能力边界 决策级融合的主要限制，是不同模态通常已经分别完成了主要特征提取和任务预测。因此，原始信息中的某些跨模态关系可能无法被充分利用。例如，一段语音由于噪声被音频模型识别为错误词语，而对应视频中存在明显的唇部运动。如果音频模型只输出最终词语标签，而没有保留其他候选与中间表示，那么晚期融合模块能够恢复的信息可能十分有限。同样，在自动驾驶任务中，一个传感器可能能够帮助另一个传感器消除对象歧义。

如果两个检测器已经分别遗漏了目标，那么仅融合最终检测框，通常无法重新获得没有被提出的候选对象。因此，晚期融合更适合整合已有预测证据，而不一定能够充分利用跨模态特征之间的细粒度互补关系。

预测一致不代表证据独立 当多个模态模型给出相同判断时，系统可能倾向于认为该判断更加可信。但多个预测一致，并不意味着它们提供了相互独立的证据。例如，两个分类器可能都严重依赖相似的场景背景，因而产生相同错误。或者两个模态在训练中使用了相同的错误标签，导致预测偏差具有相关性。因此，重复出现的相同预测不能简单地被当作多个独立事实证据。对于需要可靠概率解释的系统，模型之间的误差相关性和概率校准均可能影响融合结果。

这也是为什么多数投票或预测平均并不能无条件保证融合模型优于最好的单模态模型。

7.2 序列组织与交互代价

7.2.1 多模态 Token 拼接

从模态专用表示到统一计算接口 不同模态具有不同的原始数据形式。例如，文本由符号序列构成，图像表现为二维像素阵列，音频是连续变化的时间信号，而点云则由三维空间中的离散采样点组成。为了使这些信息共同参与 Transformer 计算，可以首先通过相应的编码器或 Tokenizer，将它们转换为连续特征序列。设系统具有 M 种模态，第 m 种模态的输入为：

$$X^{(m)} \tag{7.28}$$

经过模态编码器：

$$Z^{(m)} = f_m(X^{(m)}) \tag{7.29}$$

得到：

$$Z^{(m)} \in \mathbb{R}^{N_m \times D_m} \tag{7.30}$$

其中，N_m 为该模态的 Token 数量，D_m 为其特征维度。如果希望这些特征进入一个使用统一隐藏维度 D 的 Transformer，可以通过适当的特征投影得到：

$$H^{(m)} = P_m(Z^{(m)}) \tag{7.31}$$

其中：

$$H^{(m)} \in \mathbb{R}^{N_m \times D} \tag{7.32}$$

随后，将不同模态的特征按序列维度组织：

$$H = [H^{(1)}; H^{(2)}; \cdots ; H^{(M)}] \tag{7.33}$$

得到：

$$H \in \mathbb{R}^{N \times D} \tag{7.34}$$

其中：

$$N = \sum_{m=1}^{M} N_m \tag{7.35}$$

这一过程构成多模态 Token 拼接的一种基本形式。统一隐藏维度仅表示不同模态的特征满足后续计算接口要求，并不意味着它们已经具有完全一致的语义分布。

连续 Token 与离散 Token 在多模态架构中，还需要区分连续 Token 表示与离散 Token 编码。例如，在 LLaVA[14] 一类模块化视觉语言模型中，图像通常经过视觉编码器与 Projector，形成连续的视觉特征：

$$H_V \in \mathbb{R}^{N_V \times D} \tag{7.36}$$

文本则首先经过离散 Tokenization：$T = (t_1, t_2, \ldots, t_L)$ 再通过词嵌入形成：

$$H_T \in \mathbb{R}^{L \times D} \tag{7.37}$$

随后，二者可以在嵌入层面进行拼接：$H = [H_V; H_T]$，在这种结构中，视觉 Token 并不一定具有词表中的离散 Token ID。另一类方法则使用离散视觉 Tokenizer，将图像编码为离散索引，使视觉内容能够以某种离散符号序列的形式进行建模。例如，Chameleon[15] 研究了基于离散视觉 Token 的图文混合序列建模，使图像和文本能够在统一的自回归框架中参与理解与生成。

因此，“多模态 Token 拼接”既可能指连续嵌入的序列拼接，也可能涉及经过离散 Tokenizer 处理后的混合模态序列。两者在输入表示、训练目标和输出能力上并不完全相同。

模态标识与位置编码 将不同模态拼接成一个序列后，还需要让模型区分 Token 的来源和结构。例如，假设图像与文本输入为：$H = [H_V; H_T]$ 模型不仅需要区分哪些 Token 来自图像、哪些来自文本，还需要理解视觉 Token 在图像中的空间位置，以及文本 Token 的语言顺序。一种方式是为输入特征加入不同类型的位置信息和模态标识：

$$\tilde{h}_i = h_i + e_{\text{pos}(i)} + e_{\text{mod}(i)} \tag{7.38}$$

其中：

- h_i 为原始 Token 嵌入；
- $e_{\text{pos}(i)}$ 为相应的位置表示；
- $e_{\text{mod}(i)}$ 为模态身份表示。

这一公式用于说明可能的组织方式，并不是所有多模态模型的固定实现。例如，有些模型通过模态专用投影或特殊边界标记提供模态信息，并不显式使用可相加的模态嵌入。对于视频和音频，还需要考虑时间戳、时间顺序或相对时间位置。因此，统一 Token 序列不等于抹除模态之间的结构差异。相反，有效的多模态建模通常需要在共享计算接口中保留必要的模态结构。

共享自注意力与跨模态信息交换 将不同模态 Token 拼接后，可以使用共享 Transformer 对联合序列进行处理：

$$H' = F_{\text{Transformer}}(H) \tag{7.39}$$

如果采用允许跨模态位置相互访问的注意力掩码，不同模态之间就可以通过自注意力交换信息。例如，给定文本 Token 与图像 Token，注意力矩阵可以概念性地划分为：

$$A = \begin{bmatrix} A_{TT} & A_{TV} \\ A_{VT} & A_{VV} \end{bmatrix} \tag{7.40}$$

其中：

- A_{TT} 表示文本内部的注意力；
- A_{VV} 表示视觉内部的注意力；
- A_{TV} 表示文本查询访问视觉特征；
- A_{VT} 表示视觉查询访问文本特征。

实际允许哪些注意力连接，取决于模型采用的掩码和训练目标。对于典型的自回归视觉前缀结构，视觉 Token 位于问题文本之前。在因果注意力约束下，后续文本 Token

可以访问前面的视觉 Token，而前面的视觉 Token 通常不能访问后续问题文本。因此，虽然不同模态被组织为统一序列，其信息交换也未必在两个方向上完全对称。

7.2.2 交错式多模态序列

从模态分组到交错排列 简单 Token 拼接可以将各模态按固定顺序组织：$H = [H_V; H_T]$，例如，先放置全部图像特征，再放置文本问题。但真实多模态交互并不总是采用这种输入形式。例如，一段对话可能依次包含：用户提出问题。用户提供一张图像。用户进一步补充说明。用户提供第二张图像。用户要求比较两张图像。这种输入不是一次性给出全部视觉内容和全部文本，而是视觉媒体与文本在语义上具有明确顺序的交错结构。

可以概念性地表示为：$S = [T_1, V_1, T_2, V_2, T_3]$，其中：

- T_i 表示相应文本片段；
- V_i 表示图像、视频或其他视觉媒体的表示。

这种形式称为交错式多模态序列（Interleaved Multimodal Sequence）。交错式序列的关键，不只是不同模态都出现在输入中，而是不同片段的相对顺序和对应关系具有实际意义。

交错序列中的结构信息 例如，一段图文资料可以包含：第一张图像：汽车整体外观。第一段文字：介绍车型和外观。第二张图像：汽车内部仪表盘。第二段文字：介绍仪表参数。如果模型忽略图文片段的相对顺序，就可能将仪表盘的描述错误关联到汽车外部图像。因此，交错序列需要记录媒体边界和内容位置。在实际实现中，可以采用特殊标记表示视觉媒体的插入位置。例如：`<image>` 或者使用其他模型特定的媒体标记。

但这些标记可能只是序列中的边界信息。实际图像内容仍然需要通过视觉编码器或视觉 Tokenizer 转换为模型可用的表示。因此，应当区分媒体占位符、媒体本身的 Token 序列，以及相应的视觉特征缓存。

视频与音频的时间结构 当交错序列包含视频和音频时，模态顺序问题会进一步扩展到时间维度。例如，一段视频可以表示为：

$$V = (I_{t_1}, I_{t_2}, \ldots, I_{t_K}) \tag{7.41}$$

其中：$t_1 < t_2 < \cdots < t_K$ 每一帧还可能被转换为多个空间视觉 Token。对应的音频则可以划分为：$A = (a_1, a_2, \ldots, a_M)$ 由于音视频采样频率不同，通常不能简单假设第 i 个视频 Token 与第 i 个音频 Token 描述同一时刻。因此，模型需要利用时间戳、时间位置编码、采样窗口或学习式对齐机制，建立合理的跨模态时间关系。

例如，当人物开始说话时，唇部动作和声音变化可能具有相关性，但并不意味着音频与视频的每个特征位置都必须一一对应。因此，交错式多模态建模既要保留序列顺序，也要正确理解不同模态的时间粒度。

因果可见性 交错序列还涉及一个重要问题：当前预测位置允许访问哪些模态信息？例如，某个在线系统在时刻 t 生成回答或状态估计。如果输入中包含视频帧：

$$I_{t_1}, I_{t_2}, \ldots, I_{t_K} \tag{7.42}$$

那么对于严格在线任务，模型在时刻 t 不应读取满足：$t_k > t$ 的未来视频帧。类似地，当前文本生成位置不应直接读取未来尚未生成的目标文本。因此，多模态注意力需要根据任务协议设置相应的可见性约束。一种一般性的注意力掩码可以表示为：

$$M_{ij} = \begin{cases} 0, & \text{允许位置}i \text{ 访问位置}j \\ -\infty, & \text{禁止访问} \end{cases} \tag{7.43}$$

随后计算：

$$A = \text{Softmax}\left(\frac{QK^\top}{\sqrt{d}} + M\right) \tag{7.44}$$

对于离线视频理解任务，模型可能被允许访问整段视频；但对于在线感知或实时交互，不能将离线可访问的未来信息直接用于当前预测。因此，是否需要严格因果约束，应由任务定义决定。

交错式序列的训练与监督 在具有生成能力的多模态模型中，可以将预测目标概括为：

$$P(Y \mid S_{\text{context}}) \tag{7.45}$$

其中，S_{context} 包含按照输入协议组织的多模态上下文。但不同模型的训练目标并不完全相同。例如，一些视觉语言模型主要预测文本 Token，而视觉信息只作为外部条件。另一些模型则可能同时预测文本和离散视觉 Token，实现混合模态的自回归生成。如果模型需要生成图像或音频，还必须具有相应的输出表示与解码机制。仅能接收图像 Token，并不意味着模型天然具备图像生成能力。

因此，交错序列建模需要区分输入模态与输出模态，以及不同模态是否都参与生成损失。

7.2.3 模态交互深度与计算开销

多模态共享自注意力的计算成本 假设一个联合序列包含 M 种模态：$N_1, N_2, \ldots, N_M$ 个 Token。总长度为：

$$N = \sum_{m=1}^{M} N_m \tag{7.46}$$

对于标准密集自注意力，其注意力分数计算规模与：$O(N^2 d)$ 相关，其中 d 表示每个注意力头相应的特征维度。将 N 展开可以得到：

$$N^2 = \sum_{m=1}^{M} N_m^2 + 2\sum_{i<j} N_i N_j \tag{7.47}$$

第一项对应各模态内部位置关系的数量级。第二项则反映不同模态之间的成对位置关系规模。例如，对于文本、图像和音频三种模态：$N = N_T + N_V + N_A$，其二次项包含：$N_T^2 + N_V^2 + N_A^2$ 以及：$2N_TN_V + 2N_TN_A + 2N_VN_A$，因此，增加模态或增加某一模态的 Token 数量，可能显著扩大共享注意力的计算规模。

但这一分析主要针对标准密集注意力的数学运算量。实际显存占用还取决于注意力实现，部分高效注意力算法并不需要显式存储完整的 $N \times N$ 注意力矩阵。

定向 Cross-Attention 的计算成本 如果不采用完整联合自注意力，而让文本特征定向读取视觉特征，则跨注意力分数矩阵具有：$L_T \times N_V$ 个元素。其主要计算规模与：$O(L_T N_V d)$ 相关。例如：$L_T = 128, \quad N_V = 576$，则单个注意力头的文本到视觉相关性矩阵包含：$128 \times 576 = 73,728$ 个元素。这可以减少不必要的全序列跨模态位置连接，但并不意味着系统总计算成本一定更低。

原因在于，模型可能仍然需要对视觉和文本分别执行自注意力。此外，如果 Cross-Attention 插入多个语言模型层，相关计算还会重复发生。因此，完整成本需要结合模态内部计算、跨模态模块数量和连接器处理方式共同分析。

多层交互的累计成本 假设每个跨模态交互模块的主要成本为：C_{cross}，如果在：L_C 个网络层中使用相应模块，那么跨模态计算的总成本可以近似写为：

$$C_{\text{interaction}} \approx L_C C_{\text{cross}} \tag{7.48}$$

这说明，增加交互层数可能提高计算和参数开销。例如，在每层都插入视觉交叉注意力，与每隔若干层插入一次相比，通常需要执行更多跨模态运算。但计算成本增加是否带来能力提升，需要通过实验判断。例如，对于需要复杂对象关系的任务，多层交互可能帮助模型逐步整合相关信息。然而，如果原始视觉特征已经缺少关键细节，增加交互层数也未必能够解决感知信息缺失的问题。因此，多模态交互的深度应当与任务难度和信息质量共同考虑。

融合瓶颈与稀疏交互 为了控制成本，可以限制跨模态信息交换的范围。例如，使用 B 个融合瓶颈 Token，使不同模态通过这组 Token 进行信息交换。如果：$B \ll N_A, N_B$，则与直接建立全部：N_AN_B 个跨模态连接相比，可以采用规模与：$O(B(N_A + N_B)d)$ 相关的瓶颈交互形式。这一公式只描述相应瓶颈信息交换部分的量级，不包含各模态自身的编码和自注意力计算，也不包括不同实现可能产生的额外运算。

类似地，还可以通过局部窗口注意力、稀疏连接或 Token 筛选，减少参与跨模态交互的位置数量。这些机制的共同目标，是避免在所有层、所有位置之间无条件执行完整信息交换。但限制交互范围可能造成信息损失。例如，如果与某个问题有关的视觉细节没有进入被选中的 Token 或融合瓶颈，后续模块可能无法利用它。因此，效率提升需要与任务表现一起评价。

交互频率与模态更新频率 对于音频、视频和其他时间序列，交互深度还需要与观测更新频率区分。例如，RGB 相机每隔一定时间输出一帧图像，而音频或事件流可以提供更高频率的变化信息。如果系统要求每接收一个高频观测就重新执行全部多模态 Transformer，计算成本可能较高。一种可能的方式，是根据模态变化和任务需求，选择何时进行跨模态交互。例如，某些中间状态可以持续保留，而新到达的观测只更新相关部分。

但这种异步更新方式需要明确状态保留、时间因果与更新机制，不能只通过减少注意力层数解决。因此，网络中的交互层数和系统中的交互触发频率，是两个不同的设计维度。前者属于网络结构选择，后者更多涉及流式处理和时序状态管理。

7.3 参数共享与信息保留

算例：确定性压缩的边界

设 $H = g(Z)$ 且 g 在给定条件下为确定性映射，没有额外观测进入。数据处理不等式给出 $I(Y;H) \le I(Y;Z)$。压缩可以改变可学习性、正则化与计算成本，但不能凭空增加来自 Z 的信息。若任务条件 q 进入压缩器，则应比较同一条件下的量，不能把无条件表达直接用于任意问题。该不等式不等于“所有未压缩模型的实测分数都更高”。

7.3.1 统一编码与共享参数

统一编码的机制 统一编码（Unified Encoding）通常是指使用相对一致的模型接口或计算结构，对不同模态的数据进行表示学习。其中，一种具有代表性的方式是参数共享（Parameter Sharing），即让不同模态的输入在某些网络层中使用同一组可训练参数。

例如，假设模型接收图像、文本和音频三种模态：$X_I,\ \ X_T,\ \ X_A$ 首先利用不同的输入映射模块，将它们转换为统一隐藏维度的特征：$Z_I = P_I(X_I)$，$Z_T = P_T(X_T)$，$Z_A = P_A(X_A)$，

随后，可以使用同一个编码器：$H_I = f_\theta(Z_I)$，$H_T = f_\theta(Z_T)$，$H_A = f_\theta(Z_A)$，其中，f_θ 表示参数共享的编码网络。这一结构的核心特点是：虽然不同模态仍然采用不同输入映射，但后续编码过程可以复用相同的参数 θ。

统一编码并不意味着所有模态必须采用相同的原始 Tokenizer。由于数据形式不同，模态专用的输入适配通常仍然是必要的。

结构相同与参数共享的区别 讨论共享编码器时，需要区分两种情况。第一种情况是，不同模态采用结构相同的网络，但参数彼此独立。例如，图像编码器和文本编码器都使用 Transformer：$H_I = f_{\theta_I}(Z_I)$，$H_T = f_{\theta_T}(Z_T)$ 虽然二者的网络层数和隐藏维度可能相同，但如果：$\theta_I \neq \theta_T$ 那么它们并没有真正共享编码参数。

第二种情况是，不同模态使用同一组网络权重：$H_I = f_\theta(Z_I)$，$H_T = f_\theta(Z_T)$ 此时才属于参数共享。这一差异十分重要。例如，FLAVA 使用图像编码器、文本编码器和多模态编码器。其中，图像编码器与文本编码器采用相同类型的 Transformer 架构，但

使用不同的参数；随后由专门的多模态编码器进行联合建模。

因此，网络结构一致、隐藏维度相同和参数共享是三个不同的概念，不能直接等同。

共享参数的主要挑战 不同模态的数据分布和结构可能存在明显差异。例如，文本 Token 的关系主要受语言顺序和语义结构影响，而图像 Patch 具有二维空间邻接关系。音频信号具有连续的时间变化，点云则涉及不规则三维几何结构。如果不同模态被强制使用完全相同的编码参数，模型就需要通过统一计算处理这些差异。当模型容量有限或训练数据不均衡时，可能出现某些模态性能下降的问题。一种可能的情况是，不同模态对共享参数提出不一致的优化要求。例如：

$$g_I = \nabla_\theta \mathcal{L}_I \tag{7.49}$$

$$g_A = \nabla_\theta \mathcal{L}_A \tag{7.50}$$

如果某次优化中：$g_I^\top g_A < 0$ 说明两个梯度在当前参数空间中具有相反的局部分量。这可能反映不同任务或模态之间存在优化冲突。但单次梯度内积为负并不能直接证明共享编码器已经发生有害的负迁移，也不能仅凭梯度一致就证明多模态共享一定有效。仍然需要结合训练过程、独立验证数据和实际任务性能判断。因此，参数共享的核心挑战，是如何让不同模态利用共同计算规律，同时避免过度牺牲模态专用表示能力。

7.3.2 独立编码与异构预训练

模态专属编码器的机制 与统一共享编码器不同，独立编码（Independent Encoding）采用相对独立的网络处理不同模态。例如：$Z_I = f_{\theta_I}(X_I)$，$Z_A = f_{\theta_A}(X_A)$，$Z_T = f_{\theta_T}(X_T)$，其中，不同编码器分别具有参数：

$$\theta_I, \quad \theta_A, \quad \theta_T \tag{7.51}$$

并不要求它们使用相同的网络结构。例如，图像可以采用 ViT[1]，音频可以采用适合声学特征的 Transformer，三维点云则可以采用针对不规则空间数据设计的编码器。不同模态经过专用编码后，再通过投影、共享语义空间或联合交互模块建立联系。这一设计的主要优势，是允许每个编码器根据自身模态特点选择适当的输入表示与预训练方法。

为什么需要异构预训练 不同模态中可获得的数据规模、标注形式和自监督任务往往不同。例如，在 RGB 图像领域，可以使用图像分类、图文对比或视觉自监督等方法预训练编码器。对于音频，可以利用掩码声学建模、语音识别或其他音频学习目标。对于三维点云，可以采用点云重建、掩码建模或三维几何相关任务。因此，不同模态并不必然适合使用同一种预训练目标。

所谓异构预训练（Heterogeneous Pretraining），可以理解为根据不同模态的数据条件和表示特点，选择相应的预训练模型及其训练方法。例如：

$$\theta_I^* = \arg\min_{\theta_I} \mathcal{L}_{\text{image}} \tag{7.52}$$

$$\theta_A^* = \arg\min_{\theta_A} \mathcal{L}_{\text{audio}} \tag{7.53}$$

随后将两种编码器整合到同一个多模态系统中：

$$H_F = \Phi\left(f_{\theta_I^*}(X_I), f_{\theta_A^*}(X_A)\right) \tag{7.54}$$

这里，Φ 为相应的跨模态对齐或融合模块。这种技术路线能够充分利用各模态已经取得的预训练成果。但不同编码器的特征维度、语义粒度和数值分布可能不同，因此通常还需要额外适配。

ImageBind：**独立编码与统一嵌入空间** ImageBind[16] 是研究多模态共享语义表示的代表性工作。它关注图像、文本、音频、深度、热成像和惯性测量等六种模态。其基本思路，是为不同模态建立相应编码路径，并利用图像作为共同的对齐参照，将这些模态的输出映射到联合嵌入空间。设图像模态为 I，其他模态为 M。分别得到：$z_I = f_I(I)$，$z_M = f_M(M)$，随后利用成对数据训练两种表示之间的语义关联。例如，可以定义：

$$s(I, M) = \frac{z_I^\top z_M}{\|z_I\|_2 \|z_M\|_2} \tag{7.55}$$

并通过对比学习，使真实对应的跨模态观测具有较高相似度。ImageBind 的一个重要研究特点，是不要求为所有模态两两收集配对数据，而是利用与图像配对的数据建立共享语义空间。例如，可以使用图像—文本配对，以及图像—音频、图像—深度等配对数据。这使不同模态能够通过共同的图像参照建立一定联系。但 ImageBind 学习到的共享嵌入主要服务于跨模态关联和语义迁移，不意味着不同模态的全部物理信息都被完整保留。

例如，图像与 IMU 数据可以通过运动场景形成一定的语义对应，但不能由此认为共享嵌入已经完整表示了惯性测量中的全部高频运动细节。因此，ImageBind 说明：即使使用模态专用编码路径，也可以通过适当的对齐目标建立多模态共享表示。

FLAVA：**独立编码与联合编码的结合** FLAVA（A Foundational Language and Vision Alignment Model）采用了另一种具有代表性的结构。其模型包含三个主要部分：图像编码器、文本编码器、多模态编码器。图像编码器和文本编码器分别提取各自模态特征：$H_I = f_I(I)$，$H_T = f_T(T)$ 然后将经过相应投影的特征提供给多模态编码器：

$$H_{IT} = F_{\text{multi}}(H_I, H_T) \tag{7.56}$$

这种结构保留了单模态编码能力，也提供了专门的跨模态联合表示计算路径。例如，在某些视觉任务中，可以直接使用图像编码器特征。对于文本任务，可以使用文本

编码器。对于需要联合推理的视觉语言任务，则可以使用多模态编码器。因此，FLAVA 体现了一种将单模态表示与联合多模态表示同时纳入模型设计的思路。但 FLAVA 的图像编码器和文本编码器即使具有相同的网络架构，也采用不同参数，不应将其误写为完全共享编码器。

7.3.3 共享表示与模态专属信息

共享表示不等于模态信息完全相同 多模态学习通常希望不同模态能够建立某种共享表示，使模型识别它们之间的对应关系。例如，图像显示一辆汽车，文本描述“一辆汽车停在道路旁”，两者可以在对象语义层面建立联系。但图像和文本并不包含完全相同的信息。图像可能提供汽车颜色、外观和空间位置。文本可能只描述车辆类别和活动状态。因此，二者的共享语义范围通常小于各自包含的全部信息。

类似地，RGB 图像与深度图可以共同观察一个物体，但 RGB 更直接地提供颜色和纹理，深度则提供距离与几何结构。因此，多模态共享表示应当强调任务相关的共同信息，而不是要求不同模态的全部内容一致。

共享信息与专属信息的概念划分 从表示学习角度，可以将一种模态的特征概念性地划分为共享部分和专属部分：

$$Z^{(m)} = \left(Z^{(m)}_{\text{shared}}, Z^{(m)}_{\text{specific}}\right) \tag{7.57}$$

其中：

- $Z^{(m)}_{\text{shared}}$ 表示能够与其他模态建立联系的任务相关信息；
- $Z^{(m)}_{\text{specific}}$ 表示该模态特有或暂时无法在其他模态中直接对应的信息。

例如，在视听语音识别中，音频与视频可能共享部分语音内容和说话时序信息。但音频还包含声调、音色等声学特征，视觉则包含人物外观和唇部运动等信息。上述形式是一种概念性划分，不表示所有多模态网络都自然产生两组明确、相互独立的特征。实际研究如果希望显式分离共享与专属信息，还需要设计相应网络结构、训练目标和验证方法。

共享语义空间与共享内部状态 共享表示还存在不同层次。一种是共享语义嵌入空间。例如，通过对比学习，使来自不同模态的语义向量可以通过余弦相似度比较。这一类表示适合跨模态检索、零样本分类和语义关联任务。另一种是跨模态联合特征。例如，通过 Cross-Attention 或共享 Transformer，使多个模态的 Token 在计算过程中交换信息。还有一种可能是共享的动态状态或记忆表示。

例如，在时序多模态系统中，可以根据不同模态连续到达的观测更新某种内部表示，并在需要时从中查询任务信息。这三者并不是同一概念。例如：$\text{sim}(z_I, z_T)$ 能够计算图像与文本的相关程度，并不意味着系统已经形成可以持续更新的场景状态。同样，经过一次跨模态注意力得到联合特征，也不意味着模型已经具备长期时序记忆。因此，应当根据任务明确共享表示究竟用于语义比较、联合推理，还是动态信息维护。

过度共享可能造成的信息损失 如果要求不同模态输出完全相同的特征，可能无意中削弱模态专属信息。例如，RGB 图像包含静态纹理和颜色信息，而事件相机主要反映局部亮度变化及其时间结构。当二者观察同一个运动对象时，可以建立对象级语义对应。但是，事件流中的高时间分辨率变化，并不一定能够从单帧 RGB 图像中恢复。如果通过强约束要求：

$$Z_{\text{RGB}} \approx Z_{\text{Event}} \tag{7.58}$$

在全部特征维度上成立，就可能使 Event 表示过度追随 RGB 的静态外观结构，从而削弱事件数据本身具有价值的时间信息。因此，跨模态对齐目标应该明确约束哪些信息需要对应，而不是不加区分地压缩全部表示差异。一种可能的设计是：

$$\mathcal{L} = \lambda_s \mathcal{L}_{\text{shared}} + \lambda_t \mathcal{L}_{\text{task}} + \lambda_p \mathcal{L}_{\text{preserve}} \tag{7.59}$$

其中：

- $\mathcal{L}_{\text{shared}}$ 约束需要共享的语义信息；
- $\mathcal{L}_{\text{task}}$ 确保表示能够完成目标任务；
- $\mathcal{L}_{\text{preserve}}$ 表示用于保留必要模态信息的约束。

这一公式仅说明一种设计思路，具体是否需要独立的信息保留损失，应由任务和实验结果决定。

共享与专属特征的分离并非天然可辨识 在实际模型中，如何证明一部分特征属于共享语义，另一部分属于模态专属信息，并不是简单的问题。例如，某个网络可以将特征划分为：Z_s, Z_p 并使用不同损失训练。但即使两个特征子空间在数值上具有较低相关性，也不一定说明它们分别完整表示了共享信息和专属信息。例如，正交约束可以降低特征之间的某些线性相关性，却不保证消除全部统计依赖。同样，重建损失较低，也不能直接证明所有模态特有的任务信息都得到了合理保留。

因此，共享与专属表示的研究需要避免只根据网络结构命名就作出结论。例如，将一个分支命名为 Shared Encoder，并不意味着其输出一定是经过验证的共享语义。更加可靠的做法，是通过实际任务评价、跨模态迁移和信息扰动实验判断其表示作用。

如何评价共享表示 共享表示的评价需要与其设计目的相对应。如果共享表示主要用于跨模态检索，那么可以评价图像检索文本、音频检索图像等任务。如果共享表示用于模态互补感知，那么应当比较单模态与融合模型在相同任务下的性能。如果希望保留模态专属能力，还应评价专属信息是否在联合训练后发生下降。例如，可以比较：仅使用 RGB 特征完成颜色或对象识别。仅使用深度特征完成几何判断。融合表示完成联合任务。

如果使用共享编码器后，联合任务性能略有提高，但某个关键单模态任务显著下降，就需要判断这种性能交换是否符合应用要求。此外，还可以通过模态替换、错误配对、时间错位和局部信息遮挡等方式，分析模型是否真正利用共享与专属信息。因此，

理想的多模态表示不一定追求所有模态输出完全相同，而是应当在共享能力与模态特有能力之间形成合理平衡。

7.3.4 混合架构及其适用场景

共享主干与模态专用输入映射 一种较直接的混合架构，是保留模态专用输入映射，再使用共享编码主干。例如：$Z_m = P_m(X_m)$，

$$H_m = F_{\text{shared}}(Z_m) \tag{7.60}$$

其中，P_m 处理不同模态的输入差异，而 F_{shared} 执行共享表示学习。例如，图像输入可以先转换为 Patch Token，文本输入可以先通过词嵌入，而音频则可以通过声学特征提取和投影形成 Token。随后，它们进入统一 Transformer 主干。这种设计适合探索不同模态是否可以复用相似的上下文计算机制。

但如果不同模态的主要信息结构存在明显差异，仅有输入投影可能不足以完成有效适配。例如，不规则三维点云和文本序列的邻接关系完全不同。为了保留几何结构，可能还需要专门的位置编码、局部聚合或模态相关处理模块。

共享注意力与模态专用前馈网络 另一种具有代表性的混合方式，是共享注意力计算，同时保留模态专用的前馈变换。例如，在 Transformer 层中，自注意力负责不同 Token 之间的信息交互，前馈网络负责对每个 Token 的特征进行非线性变换。可以采用：

$$\tilde{H} = H + \text{SelfAttn}_{\theta_s}(H) \tag{7.61}$$

随后，对于来自不同模态的 Token，使用不同前馈专家：

$$H^{(m)} = \tilde{H}^{(m)} + \text{FFN}_{\theta_m}\left(\tilde{H}^{(m)}\right) \tag{7.62}$$

其中：

- θ_s 表示共享注意力参数；
- θ_m 表示模态相关的前馈网络参数。

这一结构允许不同模态共享部分上下文信息处理机制，同时保留专门的非线性特征变换路径。但它并不意味着所有模态之间都必须在每一层无条件交换全部信息。实际模型仍然可以通过注意力掩码、输入任务类型和特征路由规则，控制不同 Token 的交互。

BEiT-3：**共享注意力与模态专家** BEiT-3 是采用混合参数共享结构的代表性模型。其核心架构是 Multiway Transformer。与简单地把图像和文本输入完全相同的网络不同，BEiT-3 使用共享自注意力模块，并根据输入模态选择相应的前馈专家。例如，模型包含面向视觉和语言的专家路径，并在相关层中设置用于视觉语言融合的专家模块。其基本思想可以概括为：

$$H' = \text{SharedAttention}(H) \tag{7.63}$$

随后：

$$H'' = \text{Expert}_m(H') \tag{7.64}$$

其中，m 表示对应的模态或任务路径。通过这种方式，模型能够共享部分 Transformer 计算，同时为不同模态保留专门的特征变换。BEiT-3 还采用基于掩码数据建模的统一预训练思想，对图像、文本以及图文配对开展训练。这项工作的价值在于，展示了参数共享与模态专用计算可以在同一网络中结合，而不是必须在完全共享和完全独立之间二选一。

BEiT-3 的专家结构主要由模态和输入类型决定，不能将其直接等同于所有动态混合专家模型中根据每个 Token 学习选择任意专家的路由机制。因此，应当根据其实际 Multiway Transformer 结构理解专家选择方式。

模态专用适配器 除了使用独立前馈专家，还可以在共享编码器的基础上添加模态专用适配器（Adapter）。例如：

$$H_m = F_{\text{shared}}\left(P_m(X_m); \theta_s, \phi_m\right) \tag{7.65}$$

其中：

- θ_s 表示共享主干参数；
- ϕ_m 表示第 m 个模态的适配参数。

这些适配参数可以通过轻量网络、附加投影或其他方式影响共享主干的计算。这种设计可以减少为每种模态重新保存完整大模型的需求，并允许新模态通过相对少量的参数完成适配。但适配器参数少不意味着它能够解决全部模态差异。例如，对于特殊的三维几何任务，仅靠浅层适配可能不足以替代专门设计的空间特征提取网络。因此，模态专用适配器更适合在共享主干已经具有较强的通用表示能力，且新模态可以通过合理接口与其连接的情况下使用。

独立编码器与共享融合主干 另一类混合架构保留完整的模态专用编码器，但在后续使用共享融合网络。例如：$Z_I = f_I(I)$，$Z_A = f_A(A)$，$Z_T = f_T(T)$，随后：

$$H_F = F_{\text{shared}}\left(P_I(Z_I), P_A(Z_A), P_T(Z_T)\right) \tag{7.66}$$

这种设计允许每种模态继续使用成熟的预训练编码器，再利用共享 Transformer 建立跨模态关联。例如，FLAVA 的独立图像与文本编码器，加上专门的多模态编码器，就体现了模态独立处理与联合融合相结合的思想。与前一种共享编码主干的方案不同，这里的单模态特征提取仍然主要由独立网络完成。因此，不能仅因为模型最后采用共享融合模块，就将其描述为所有模态共享同一个编码器。这一点对于比较不同研究的参数效率和结构差异十分重要。

混合架构中的参数效率 混合架构可以根据任务需要共享部分参数，减少重复网络结构。例如，如果每个模态都使用独立编码器，那么总参数量可以概念性地表示为：

$$P_{total} = \sum_{m=1}^{M} P_m + P_F \tag{7.67}$$

其中，P_F 为融合模块参数。如果采用共享主干与模态专用输入映射，则可以写为：

$$P_{total} = P_{\text{shared}} + \sum_{m=1}^{M} P_{\text{adapter},m} + P_{\text{heads}} \tag{7.68}$$

这种方式可能减少部分重复参数。但是否真正减少参数量，取决于共享主干、模态专家和适配器的实际规模。例如，如果在共享主干之外为每种模态增加较大的专家网络，那么混合架构的参数量也可能很高。此外，模型参数量与实际计算成本并不完全相同。共享参数可以减少重复存储，但若多个模态分别经过同一主干，仍可能需要执行多次计算。因此，混合架构的效率应通过可训练参数、总模型参数、计算量、显存与延迟等指标共同评价。

7.4 条件路由与专家机制

7.4.1 模态条件化与动态信息选择

模态条件化的机制 在多模态系统中，不同任务对各模态信息的需求可能不同。例如，对于一个包含视频和音频的输入：当用户询问“说话者说了什么”时，音频信息通常具有直接作用。当用户询问“人物穿着什么颜色的衣服”时，视觉信息更为关键。当用户询问“正在说话的人穿着什么衣服”时，则需要音视频共同参与。因此，模型可以根据任务、输入内容和当前上下文，调整对不同模态信息的使用方式。

这种思路可以称为模态条件化（Modality Conditioning），即让某些计算或信息选择受到模态类型、任务条件或输入状态的影响。其一般形式可以表示为：

$$H_F = F\left(Z_1, \ldots, Z_M \mid C\right) \tag{7.69}$$

其中，C 表示条件信息，可以包括文本问题、任务标识、模态可用性或其他相关上下文。条件化并不必然意味着模型采用动态路由或显式门控。输入侧条件嵌入、交叉注意力和条件归一化等机制，也可以实现不同形式的条件化计算。

基于文本问题的条件选择 在视觉问答中，文本问题可以决定模型需要关注哪些视觉信息。例如，图像包含：一辆红色汽车。一辆蓝色自行车。一名穿黄色衣服的行人。对于问题：“汽车是什么颜色?” 模型需要更多使用汽车区域的视觉特征。对于问题：“行人旁边有什么?” 模型则需要关注人物和周围对象之间的空间关系。可以使用文本特征作为查询：$Q = Z_T W_Q$ 使用视觉特征作为键和值：$K = Z_V W_K, \quad V = Z_V W_V$ 然后计算：

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \tag{7.70}$$

这一机制使视觉信息的读取方式能够随文本上下文变化。但文本查询决定了模型关注的方向，不保证注意力必然落在正确对象上。例如，模型可能由于数据偏差而关注背景区域。因此，仍然需要通过细粒度任务和视觉扰动测试验证其信息选择能力。

基于门控的信息融合 另一种条件化方式，是使用门控机制调整不同模态的信息贡献。设有两个模态特征：$Z_1,\quad Z_2$，经过必要的维度适配后，可以计算：

$$g = \sigma\left(W[Z_1; Z_2] + b\right) \tag{7.71}$$

其中，σ 为 Sigmoid 函数。随后形成：

$$H_F = g \odot Z_1 + (1 - g) \odot Z_2 \tag{7.72}$$

其中，$\odot$ 表示逐元素乘法。这种方式允许模型根据输入特征动态调整两种模态的贡献。这里使用的是两模态门控的简化形式。在多模态系统中，可以使用多个权重，并通过 Softmax 等方式构建相应组合。此外，门控可以作用于整个模态、某个空间区域、某段时间或某组特征通道。这些选择对应不同的信息选择粒度。

选择粒度与表示能力 模态条件化可以发生在多个层次。在模态级，模型为整种模态分配一个权重。例如，在当前任务中更重视视觉还是音频。在 Token 级，模型可以根据局部内容选择不同视觉或音频位置。在通道级，模型可以调整某些特征维度的贡献。在模块级，模型还可以根据输入选择不同网络专家或计算路径。这些机制的灵活性与计算成本有所不同。例如，模态级标量门控结构较简单，但它可能无法处理同一模态内部不同区域具有不同质量的情况。

Token 级选择具有更细的表示粒度，但通常需要计算更多位置之间的相关性。因此，信息选择的粒度应根据任务需要决定，而不是无条件越细越好。

动态权重不等于真实可靠性 在多模态研究中，门控或注意力权重经常被解释为模型对不同模态的信任程度。但这种解释需要谨慎。例如，模型可能给予 RGB 特征较大的门控值：$g = 0.9$，这只能说明在当前参数和输入下，模型对 RGB 分支赋予了较大计算权重。它并不直接证明 RGB 观测具有 90% 的可靠性，也不意味着事件或深度信息只有 10% 的可信度。模型权重可能受到训练数据分布、特征尺度和任务损失的共同影响。

因此，如果希望将门控解释为可靠性，需要进一步定义可靠性目标，并使用独立数据验证其预测是否与实际错误风险相对应。否则，更准确的称呼应当是信息融合权重或条件选择系数。

模态缺失与条件化计算 当某些模态暂时不可用时，动态信息选择具有另一项作用。设：$a_m \in \{0, 1\}$ 表示第 m 个模态是否可用。可以在融合时只对可用模态进行归一化：

$$w_m = \frac{a_m \exp(s_m)}{\sum_j a_j \exp(s_j)} \tag{7.73}$$

其中，假设至少存在一个可用模态。然后得到：

$$H_F = \sum_m w_m Z_m \tag{7.74}$$

这一形式能够避免将缺失模态当作正常观测参与简单加权。但这只是具有缺失模态掩码的融合机制示例。它不能保证模型已经学会处理所有真实的传感器故障。例如，传感器可能并非完全缺失，而是产生质量下降或局部错误的数据。因此，模态条件化还需要结合训练数据、缺失模式和实际鲁棒性评价。

7.4.2 混合专家模型的基本思想

从共享网络到专家网络 在传统的密集神经网络中，输入通常经过相同的主要网络参数进行处理。例如，对于一个 Transformer 前馈层：

$$y = \mathrm{FFN}_\theta(x) \tag{7.75}$$

不同输入 Token 使用相同的一组 FFN 参数。当模型需要处理多种模态和多种任务时，一种可能的设计是扩大共享网络容量，使其同时学习所有类型的信息。但这也可能增加计算成本，并使不同任务或模态在相同参数中发生优化竞争。混合专家模型（Mixture of Experts，MoE）提供了另一种方式。其基本思想是设置多个专家网络（Experts），再通过路由器（Router）决定当前输入由哪些专家处理。

例如：$E_1, E_2, \ldots, E_K$ 表示 K 个专家网络。路由器根据输入 x 计算不同专家的选择分数：

$$p(x) = \mathrm{Softmax}(W_r x + b_r) \tag{7.76}$$

其中：

$$p(x) \in \mathbb{R}^K \tag{7.77}$$

表示输入与各专家之间的路由权重。随后，系统可以选择部分专家参与计算。因此，MoE 的核心区别，不只是存在多个子网络，而是能够通过路由机制控制专家的激活。

密集专家组合与稀疏专家组合 在一般的专家混合模型中，可以使用全部专家的加权输出：

$$y = \sum_{k=1}^{K} p_k(x) E_k(x) \tag{7.78}$$

这种方式允许不同专家共同参与预测。但如果所有专家都需要执行完整前向计算，那么专家数量增加也会提高计算成本。为此，稀疏 MoE 通常只选择部分专家。设：

$$\mathcal{S}(x) = \mathrm{TopK}(p(x), k) \tag{7.79}$$

表示根据路由分数选出的 k 个专家。输出可以表示为：

$$y = \sum_{j \in \mathcal{S}(x)} \tilde{p}_j(x) E_j(x) \tag{7.80}$$

其中，$\tilde{p}_j(x)$ 为选中专家的组合权重。不同实现可以保留原始门控分数，也可以对选中专家重新归一化。这样，一个 Token 不必经过全部 K 个专家，而只需要执行相应的 k 个专家。当：$k \ll K$ 时，模型可以在增加总参数容量的同时，限制每个 Token 实际激活的专家数量。但稀疏激活不能直接等同于端到端推理更快。路由、专家分配和跨设备通信也会产生开销。

专家网络不一定具有预先定义的语义 在多模态 MoE 中，研究者可能希望不同专家擅长处理不同类型的信息。例如：一个专家更多处理视觉输入。一个专家更多处理文本输入。另一个专家更多处理两种模态的共同语义。但这些划分可能采用不同的实现方式。一种方式是预先定义模态专用专家，并根据输入类型选择对应专家。另一种方式是使用可学习路由器，让模型通过训练决定不同 Token 使用哪些专家。两者具有不同含义。在预定义专家结构中，专家的模态分工来自模型设计。

在学习式专家路由中，专家的分工可能通过数据与训练目标逐步形成。因此，不能仅因为模型中存在多个 FFN 分支，就认为它已经学会了动态、输入条件化的专家选择。

BEiT-3 的模态专家与稀疏 MoE 的区别 BEiT-3 采用 Multiway Transformer，其中包含共享自注意力和模态相关的前馈专家。它可以根据图像、文本或图文联合输入所对应的计算路径，使用相应专家进行表示处理。这种方式能够在共享注意力计算的基础上，保留模态相关的特征变换。但 BEiT-3 的模态专家主要根据输入类型和模型设计选择，并不等同于 Switch Transformer 中每个 Token 由可学习路由器进行动态 Top-1 专家选择。

因此，可以将二者区分为：

模态预设式专家机制：根据输入模态或计算路径选择专家。

学习式稀疏专家机制：根据 Token 表示和路由器输出选择专家。

两者都可以属于广义的专家化网络设计，但具体的信息路由和训练机制不同。

LIMoE：面向视觉语言学习的混合专家 LIMoE（Language-Image Mixture of Experts）是将稀疏专家机制应用于视觉语言预训练的代表性工作。它采用能够同时处理图像和文本的多模态架构，并使用对比学习建立图文表示之间的联系。在这一模型中，视觉 Token 和文本 Token 可以通过共享的专家体系进行处理，而路由器根据 Token 表示选

择相应的专家计算路径。这种设计使模型不必预先为每一种模态完全指定固定的专家组合。

例如，某些专家可能在训练过程中更多处理视觉 Token，而另一些专家可能更多处理文本 Token。LIMoE 的研究观察到，不同模态的专家使用模式可以在训练过程中形成一定差异。但专家出现模态使用偏好，并不等于专家自动获得了可以独立解释的固定语义功能。例如，某个专家被视觉 Token 频繁选择，不意味着它只负责颜色、空间或对象识别。这些功能需要通过进一步的输入分析和任务实验验证。

LIMoE 的训练问题 将稀疏 MoE 用于多模态数据，会带来新的训练挑战。例如，图像与文本 Token 的数量可能存在明显差异。如果一批训练数据包含大量视觉 Token 和较少的文本 Token，路由器可能更多受到视觉输入影响。这可能导致部分专家主要接收某一模态的数据，而其他专家利用不足。此外，专家使用可能过度集中，造成部分专家负载过高，而其他专家缺少训练样本。

LIMoE 针对多模态专家使用与训练稳定性问题，研究了基于熵的路由正则化方法，促进模型合理利用专家容量。这表明，多模态 MoE 不只是将语言模型中的专家层直接用于视觉和文本，还需要考虑不同模态 Token 分布带来的路由偏差。

负载均衡与专家塌缩 在稀疏 MoE 中，如果多数 Token 长期只选择少数几个专家，就可能出现专家利用不均衡的问题。例如，模型具有八个专家：$E_1, \ldots, E_8$，但绝大部分输入都选择：E_1, E_2 那么其他专家可能很少获得训练机会。这种现象可能导致专家容量利用不足，并影响训练稳定性。因此，实际 MoE 通常需要考虑负载均衡。例如，可以定义每个专家接收的 Token 比例：

$$f_j = \frac{n_j}{N} \tag{7.81}$$

其中：

- n_j 为分配给第 j 个专家的 Token 数；
- N 为参与统计的 Token 总数。

可以进一步观察各专家的使用分布。但专家负载均衡不一定意味着所有专家在每个批次中必须接收完全相同数量的 Token。尤其在多模态任务中，某些专家可能具有合理的模态或任务偏好。因此，均衡机制需要避免两种极端：既要防止专家长期闲置，也不能无条件强迫不同模态使用完全相同的专家分布。

专家容量与 Token 分配 实际稀疏 MoE 通常还需要控制每个专家能够处理的 Token 数量。例如，当大量 Token 同时选择同一个专家时，可能超出该专家在当前批次中的计算容量。此时，系统需要采用相应的 Token 分配、溢出处理或重新路由机制。不同 MoE 实现对这些问题的处理方式不同。例如，可以限制每个专家的 Token 容量，或者通过计算调度使输入尽量均衡地分配到不同专家。这些机制会影响训练效率、模型稳定性和最终性能。

因此，MoE 的计算成本不能只用每个 Token 激活多少专家来描述，还需要考虑实

际专家负载和计算资源分配。

MoE 的参数容量与激活计算 假设一个 MoE 层包含 K 个相同规模的专家，每个专家参数量为：P_E 路由器参数量为：P_R，则该层总参数量近似为：$P_{total} \approx KP_E + P_R$，如果每个 Token 只激活 k 个专家，那么专家网络实际参与的参数量约为：

$$P_{\text{active}} \approx kP_E \tag{7.82}$$

其中，$k \ll K$。这意味着，MoE 可以具有较大的总参数容量，但每个 Token 只执行部分专家。然而，这不代表总模型权重显存会按激活专家数量减少。如果全部专家常驻显存，模型仍然需要存储所有专家参数。此外，分布式部署中可能需要将不同 Token 发送到不同设备上的专家，引入额外的通信开销。因此，评价 MoE 时应明确区分：总参数量。每个 Token 激活的参数量。实际执行的浮点运算。权重存储与显存。

路由与跨设备通信。这些指标反映的是不同的资源维度。

7.5 统一输入输出与训练目标

比较维度	应说明的内容	不能据此直接推断的结论
表示	连续特征、离散码及其位置	相同维度即语义相同
参数	输入层、主干、专家和预测头的共享范围	共享参数即所有任务相互促进
序列	拼接、交错和可见性规则	交错输入即统一词表
目标	自回归、去噪及任务损失的位置	同一主干即同一种生成机制

7.5.1 原生多模态输入与统一序列建模

原生多模态建模的机制 相较于将其他模态作为语言模型的外部条件，一种更加统一的研究思路，是从模型表示、计算架构和训练任务设计阶段，就考虑多种模态共同参与建模。例如，模型可以将文本、图像和视频组织为混合多模态序列，并通过共享 Transformer 学习序列中的统计关系。更进一步，模型不仅可以读取不同模态的 Token，还可以根据上下文预测属于不同模态的输出。

因此，统一多模态序列建模可以概括为：$S = (s_1, s_2, \ldots, s_N)$，其中，$s_i$ 可以根据相应表示方案，来自文本、图像、音频或其他模态。这一表达式描述的是序列组织方式，不意味着所有模态的 Token 都具有相同的物理含义。实际模型仍然可能使用不同的模态 Tokenizer、码本、位置编码和输出解码器。

统一序列与统一词表 当多种模态使用离散 Token 表示时，可以将它们组织为统一的符号序列。例如，文本经过文本 Tokenizer 产生：$T = (t_1, \ldots, t_L)$，图像经过离散视觉 Tokenizer 产生：$V = (v_1, \ldots, v_K)$ 模型可以进一步构造：$S = [T; V]$ 也可以使用文本与图像交错排列的形式：$S = [T_1; V_1; T_2; V_2]$，这里的分号表示序列连接。

如果模型为不同模态分配可识别的 Token 类型，并建立相应的嵌入和输出映射，就可以在一个自回归序列模型中对混合 Token 进行预测。但统一序列不意味着图像和

文本必须在原始 Tokenizer 层面采用相同编码算法。例如，文本 Tokenizer 负责将语言符号转换为离散索引，视觉 Tokenizer 则可能通过向量量化将图像转换为视觉码本索引。

它们可以在模型内部共享嵌入宽度和 Transformer 主干，但其离散符号所代表的内容仍然不同。

Emu3：以下一 Token 预测统一多模态建模 Emu3 进一步探索了将下一 Token 预测扩展到文本、图像和视频的技术路线。其核心思想是，将不同模态转换为离散 Token 序列，并从随机初始化开始训练共享的自回归 Transformer。例如：$S = (s_1, s_2, \ldots, s_N)$ 模型学习：

$$P_\theta(S) = \prod_{i=1}^{N} P_\theta(s_i \mid s_{<i}) \tag{7.83}$$

其中，不同位置的 Token 可以来自文本、图像或视频。对于视觉理解任务，模型可以在图像 Token 和文本问题之后继续生成回答 Token。对于图像生成任务，模型可以根据文本条件生成视觉 Token，再经过视觉解码器还原图像。对于视频生成，也可以使用相应的视频 Token 序列进行条件预测。

Emu3 的研究价值，在于展示了多种模态的理解与生成可以在统一的下一 Token 预测目标下进行学习，而不一定要求每种生成任务都配置完全独立的生成模型。但这并不意味着所有多模态生成方式都应当采用离散自回归预测。不同模态的生成质量、序列长度和效率，仍然可能使其他方法具有优势。

连续特征与离散 Token 的结合 离散 Token 序列有利于采用统一的分类式下一 Token 预测目标，但也存在量化精度和序列长度方面的限制。因此，另一类统一多模态模型保留连续视觉特征，并使用共享计算主干开展多模态理解与生成。例如，可以使用：$H_V = f_V(I)$ 获取连续视觉表示，再将其投影到共享 Transformer 的输入空间。对于文本生成，模型可以预测离散语言 Token。

对于图像生成，模型则可以通过专门的生成目标或解码器输出视觉内容。这说明，统一多模态架构并不要求全部模态都被离散化。更准确的区别在于：不同模态能否通过协调设计的表示、计算和训练目标形成整体系统。因此，应当将统一 Token 序列视为一种重要技术路线，而不是原生多模态架构的唯一实现形式。

从图文输入扩展到音频、视频和动作 随着应用范围扩大，多模态模型不再局限于图像和文本。例如，在语音交互中，输入可能同时包含音频和视频，输出则可能包括文字与语音。在具身智能系统中，输入还可能涉及机器人状态、三维环境信息，输出则可能包含动作或其他控制相关表示。不同任务需要的输入输出形式可以概括为：

$$Y^{(b)} = F_\theta\left(X^{(a_1)}, X^{(a_2)}, \ldots\right) \tag{7.84}$$

其中，输入和输出对应不同的模态类型。例如：图像和问题输入，输出文本回答。文本输入，输出图像。音频输入，输出转写文本。视频和问题输入，输出事件描述。视

觉及状态输入，输出相应的动作表示。能否支持这些任务，取决于模型是否具有相应的输入表示、输出机制和训练监督。统一的 Transformer 主干本身并不能自动提供所有模态的生成能力。

7.5.2 多模态统一训练目标

统一自回归预测目标 对于已经转换为离散 Token 的多模态序列，可以采用统一的下一 Token 预测目标：

$$\mathcal{L}_{AR} = -\sum_{i=1}^{N} \log P_\theta(s_i \mid s_{<i}) \tag{7.85}$$

其中，s_i 可以表示文本、图像或其他模态的离散 Token。这一形式能够将多种模态的序列学习转化为类似的条件分类问题。例如：对于文本任务，模型预测下一个文本 Token。对于图像任务，模型预测下一个离散视觉 Token。对于视频任务，模型根据此前内容预测后续视频 Token。Chameleon 和 Emu3 体现了这种统一自回归建模思想。

其中，Emu3 特别强调将多模态理解与生成统一到下一 Token 预测目标之下。但统一损失公式并不意味着不同模态的训练难度相同。例如，一张图像可能需要大量视觉 Token 表示，而一个简短回答可能只有几十个文本 Token。如果直接对所有位置累加预测损失，较长的模态序列可能在训练中形成更大的损失贡献。因此，还需要考虑不同模态的 Token 数量、采样比例与损失归一化方式。

不同模态的损失平衡 对于多模态联合训练，一项重要问题是平衡不同模态的训练信号。例如，文本数据规模可能远大于高质量图文配对数据。视频 Token 数量也可能明显多于单张图像或简短文本。因此，可以根据模态类型定义不同损失权重：

$$\mathcal{L} = \lambda_T \mathcal{L}_T + \lambda_I \mathcal{L}_I + \lambda_A \mathcal{L}_A + \lambda_V \mathcal{L}_V \tag{7.86}$$

其中：

- $\mathcal{L}_T$ 为文本相关训练损失；
- $\mathcal{L}_I$ 为图像相关训练损失；
- $\mathcal{L}_A$ 为音频相关训练损失；
- $\mathcal{L}_V$ 为视频相关训练损失。

这里的符号只用于说明多模态损失平衡的基本思想，不表示每个模型都必须设置四个独立损失。例如，在统一离散 Token 自回归模型中，这些项可以只是按 Token 类型划分的同一交叉熵损失。而在使用不同输出解码器的模型中，它们可能采用不同数学形式。因此，实际训练需要根据模型的输入输出表示决定损失设置。

统一训练中的梯度竞争 当不同任务共享模型参数时，其优化方向可能存在差异。设共享参数为：θ 两个任务的损失分别为：

$$\mathcal{L}_1, \quad \mathcal{L}_2 \tag{7.87}$$

则：

$$\nabla_\theta \mathcal{L} = \lambda_1 \nabla_\theta \mathcal{L}_1 + \lambda_2 \nabla_\theta \mathcal{L}_2 \tag{7.88}$$

不同任务产生的梯度可能在部分参数上相互促进，也可能存在冲突。例如，视觉理解任务希望模型提取相对抽象的对象语义，而图像生成任务可能更加重视局部纹理和精细外观。如果两类任务完全依赖同一组表示参数，就可能发生优化竞争。Janus[17]对视觉理解与生成编码路径的解耦，便体现了对这类任务差异的架构回应。任务目标不同并不意味着它们必然产生负迁移。共享训练也可能改善表示泛化。

因此，需要通过联合训练与单任务训练的对照，判断共享参数究竟带来正向迁移还是性能损失。

Unified-IO 2：**统一输入输出与多任务训练** Unified-IO 2 是探索更广泛多模态统一建模的代表性工作。该模型尝试将文本、图像、音频、视频、动作和其他结构化输出纳入相对统一的输入输出体系。其基本思路，是通过相应 Tokenizer 和输入输出表示，将不同任务组织为统一的编码器—解码器计算形式。例如：

$$X_{\text{multi}} \xrightarrow{\text{Encoder}} H \xrightarrow{\text{Decoder}} Y_{\text{multi}} \tag{7.89}$$

其中，输入和输出可以涉及不同模态。Unified-IO 2 并非简单地把所有任务压缩为未经区分的单一下一 Token 分类任务。其原始研究采用多模态混合去噪目标（Multi-modal Mixture of Denoisers），并通过多种任务数据与指令格式开展联合训练。这一设计说明，统一多模态架构不仅可以通过统一自回归 Token 预测实现，也可以通过共享编码器—解码器框架和协调设计的多任务去噪与生成目标实现。

因此，统一多模态训练的关键，是使不同任务可以在共同的模型结构中被有效组织和优化，而不是要求所有任务的原始数据与目标函数完全相同。

理解与生成目标的结合 在视觉语言模型中，理解任务与生成任务通常具有不同的监督形式。例如，视觉问答可以使用文本生成损失：

$$\mathcal{L}_{VQA} = -\sum_t \log P_\theta(y_t \mid y_{<t}, I, Q) \tag{7.90}$$

而图像生成可以预测离散视觉 Token：

$$\mathcal{L}_{image} = -\sum_j \log P_\theta(v_j \mid v_{<j}, T) \tag{7.91}$$

其中，T 表示文本条件。如果图像生成采用连续潜在表示与扩散式训练，则可能使用不同的噪声预测或流匹配目标，而不是上述离散视觉 Token 交叉熵。因此，在统

一模型中同时支持理解与生成，至少需要解决两个问题。一方面，模型必须具有能够表达不同任务结果的输出空间。另一方面，不同训练目标需要在共享参数中形成合理的优化关系。这说明，多模态统一训练既是架构问题，也是训练目标与数据组织问题。

7.5.3 统一模型中的输入输出设计

统一模型首先需要明确输入输出接口 理解与生成一体化的基础，是使模型能够根据任务区分不同的输入和输出。例如，一个统一模型可能需要完成：根据图像回答文本问题。根据文本生成图像。根据参考图像和文字指令编辑图像。根据音频生成文本描述。根据文本生成语音。这些任务可以概念性地表示为：

$$Y^{(b)} = F_\theta(X^{(a)}, C) \tag{7.92}$$

其中：

- $X^{(a)}$ 表示一种或多种输入模态；
- $Y^{(b)}$ 表示目标输出模态；
- C 表示任务指令或其他条件。

统一模型需要明确当前任务使用哪些输入、目标输出属于哪种模态，以及相应输出应当如何解码。因此，输入输出统一不是简单地将所有数据拼接到一起，还需要设计模态标识、任务边界、位置编码和相应的输出接口。

离散 Token 的统一输入输出 一种重要方式是将不同模态转换为离散 Token，并通过共享的序列模型进行理解与生成。例如，文本经过文本 Tokenizer 得到：$T = (t_1, \ldots, t_L)$，图像经过视觉 Tokenizer 得到：$V = (v_1, \ldots, v_K)$，随后构造混合序列：$S = [T_1; V_1; T_2; V_2]$，其中，不同片段可以代表文本说明、输入图像、后续问题或生成图像。模型通过自回归预测学习：

$$p_\theta(S) = \prod_{i=1}^{N} p_\theta(s_i \mid s_{<i}) \tag{7.93}$$

Chameleon 是这一技术路线的代表性研究。其通过离散图像 Token 和文本 Token 的早期融合，使同一个自回归模型能够处理图文混合序列，并支持相关的理解与生成任务。这种设计在形式上具有较高的统一性。但是，离散视觉 Tokenizer 的性能会影响图像细节保留，而文本和图像的 Token 数量也可能存在较大差异。因此，统一序列预测目标虽然简洁，却仍然需要解决不同模态的表示粒度和生成效率问题。

理解与生成采用不同视觉编码路径 另一种方式是保持共享计算主干，但为理解与生成保留不同的视觉输入表示。Janus 是这一思路的重要代表。该模型认为视觉理解和图像生成对视觉信息的要求存在差异，因此为两类任务设置不同的视觉编码路径。例如，对于理解任务，模型使用适合提取语义信息的视觉编码器：

$$H_I^{(U)} = E_U(I) \tag{7.94}$$

对于图像生成任务，则使用适合重建和生成的离散视觉表示：$V_I = E_G(I)$，其中，E_U 和 E_G 分别表示两类视觉编码过程。二者通过适当的输入映射和训练设计，与共享自回归 Transformer 连接。这一方式允许理解任务使用连续语义特征，而生成任务使用离散视觉 Token。因此，Janus 所体现的统一性主要在于共享计算主干和多任务训练，而不是强制两种任务使用完全相同的视觉编码器。

其研究说明，保留任务专用的输入表示，可能是实现理解生成一体化的一种合理设计选择。

联合编码器—解码器结构 除统一自回归 Decoder-only 模型外，还可以采用共享编码器—解码器结构组织多模态输入输出。例如：$H = E_\theta(X, C)$，$Y = D_\theta(H)$，其中，输入和输出可以根据任务使用不同模态表示。Unified-IO 2 是这一方向的重要代表。

该模型通过相应 Tokenizer 将图像、文本、音频、动作和其他结构化数据组织为统一的模型输入输出，并使用编码器—解码器 Transformer 进行处理。例如，在视觉问答中，模型根据图像和文本问题输出回答。在图像生成任务中，模型根据相应文本条件生成视觉表示。在动作或结构化预测任务中，模型则可以输出相应的任务表示。统一编码器—解码器结构并不意味着所有模态使用相同的原始 Tokenizer。

不同模态仍然需要适合自身信息结构的输入输出处理方式。因此，该路线主要强调多种任务共享模型组织形式与训练体系。

同一模型中结合自回归与扩散生成 统一理解生成模型不一定要求所有任务采用相同的生成损失。例如，文本回答通常适合离散 Token 自回归预测，而图像生成可以采用连续扩散模型。Transfusion[18] 是将两种生成目标结合到同一模型中的代表性研究。其主要思路是，在一个共享 Transformer 中同时处理离散文本和连续图像表示。对于文本 Token，模型采用下一 Token 预测目标。对于连续图像表示，模型采用扩散式训练目标。可以概念性地表示为：

$$\mathcal{L} = \lambda_T \mathcal{L}_{\text{AR}} + \lambda_I \mathcal{L}_{\text{diff}} \tag{7.95}$$

其中：

- $\mathcal{L}_{\text{AR}}$ 表示文本自回归训练损失；
- $\mathcal{L}_{\text{diff}}$ 表示图像扩散训练损失。

这种方式将不同模态的生成目标组织到共享模型中，而不必强制将全部图像内容转换为离散视觉码本索引。Transfusion 的统一性体现在共享 Transformer 与混合训练体系中，文本和图像仍然具有不同的预测目标。因此，统一参数与统一损失函数是两个不同概念。

自回归与流匹配的结合 JanusFlow 是将自回归语言模型与 Rectified Flow 结合的重要研究。其目标是在相对统一的模型中同时支持视觉理解和图像生成。对于语言相关任

务，模型利用自回归预测完成文本输出。对于图像生成，则使用连续流匹配目标学习视觉潜在表示的生成过程。其一般形式可以写为：

$$\mathcal{L} = \lambda_U \mathcal{L}_{\text{understanding}} + \lambda_F \mathcal{L}_{\text{flow}} \tag{7.96}$$

其中，$\mathcal{L}_{\text{flow}}$ 用于学习连续视觉生成过程中的速度场。JanusFlow 还采用理解与生成编码器解耦，以及两类表示对齐等设计，以缓解不同任务在表示需求上的差异。这说明，统一理解生成模型可以在共享计算体系中结合离散语言预测与连续图像生成，而不必将全部任务转换为相同类型的 Token 分类问题。但采用不同训练目标并不保证模型能够同时达到所有专用模型的最佳表现。

实际结果还受到共享参数容量、训练数据和任务优化方式的影响。

输出模态与专用解码器 对于生成式多模态系统，模型内部表示与最终输出数据通常并不相同。例如，模型可能先生成离散视觉 Token，再通过图像 Tokenizer 对应的解码器恢复像素图像。也可能先生成连续图像潜在特征，再通过图像自编码器解码器恢复图像。对于音频生成，模型可以输出音频编解码 Token 或连续声学潜在特征，再由音频解码器产生波形。因此，可以将统一系统表示为：

$$X \xrightarrow{\text{输入编码}} H \xrightarrow{\text{共享建模}} Z_Y \xrightarrow{\text{模态解码}} Y \tag{7.97}$$

其中，Z_Y 为目标输出模态的内部表示。这一结构说明，统一多模态模型并不需要由同一输出头直接生成全部类型的原始数据。例如，图像、文本和音频可以共享某些高层语义计算，但仍然使用不同的输出恢复机制。因此，**统一模型的核心不一定是取消模态专用模块，而是让不同模态任务能够在共同的表示和计算体系中有效协同。**

7.5.4 多模态联合训练的关键挑战

多模态联合训练的基本目标 多模态联合训练希望利用不同来源的数据，使模型在多个任务之间学习可共享的表示与能力。例如，模型可以同时使用：纯文本。图像—文本配对。视频—文本配对。音频—文本配对、音视频同步数据、交错多模态文档。机器人观测与动作。这些数据可能具有不同训练目标。例如，文本使用下一 Token 预测。图像理解使用监督问答。图像生成使用扩散损失。跨模态检索使用对比学习损失。因此，可以将联合训练目标表示为：

$$\mathcal{L} = \sum_{k=1}^{K} \lambda_k \mathcal{L}_k \tag{7.98}$$

其中：

- $\mathcal{L}_k$ 表示不同任务或模态对应的损失；
- λ_k 表示训练权重。

但联合优化目标能够写成加权和，并不意味着不同任务之间天然具有一致的优化方向。

这也是多模态联合训练需要解决的重要问题。

不同模态的数据统计差异 文本、图像、视频和音频具有不同的统计结构。例如，文本主要由离散符号序列构成。图像具有二维空间相关性。视频同时包含空间和时间变化。音频具有高频采样和连续声学结构。因此，不同模态通常需要不同的编码方式。例如，图像可以使用 Patch 表示。音频可以使用频谱或神经音频 Token。文本则使用子词 Token。但不同表示的序列长度、信息密度和损失尺度可能差异很大。例如，一段音频可能需要远多于一句文本的 Token。

因此，在联合训练中，需要考虑不同模态的数据规模和计算占比。

训练梯度冲突 不同任务可能对共享参数提出不同要求。例如，图像生成需要保留细粒度视觉信息。图像理解可能更加强调语义抽象。如果两种任务同时更新共享参数，就可能产生相互冲突的梯度。设两个任务的梯度为：

$$g_1 = \nabla_\theta \mathcal{L}_1 \tag{7.99}$$

$$g_2 = \nabla_\theta \mathcal{L}_2 \tag{7.100}$$

可以通过余弦相似度分析其方向关系：

$$\cos(g_1, g_2) = \frac{g_1^\top g_2}{\|g_1\|\|g_2\|} \tag{7.101}$$

例如，负余弦值表示两个梯度方向之间存在冲突趋势。但局部梯度冲突并不自动意味着最终任务性能一定下降。训练过程还受到优化器、学习率和模型容量影响。因此，梯度分析应当与实际任务表现和收敛情况结合。

空间和时间对齐 多模态数据不仅需要语义对应，还可能需要空间和时间对应。例如，一段视频中的动作需要与相应音频事件关联。机器人动作需要对应正确的摄像头观测。多传感器数据则需要根据坐标系和时间戳进行对齐。可以将多模态观测表示为：$O = (x_i, m_i, t_i, r_i)$，其中，t_i 和 r_i 表示时间与空间或来源关系。但统一序列中的相邻 Token 不一定来自相同物理时刻。

例如，视频帧和音频片段可能具有不同采样率。因此，模型需要合理保留时间信息。Qwen2.5-Omni 的时间对齐位置编码，就是针对音视频时间关系提出的一种方法。但更复杂的异步传感器融合仍然需要考虑独立事件时间、数据延迟和历史状态更新等问题。

序列长度与计算成本 多模态输入可能具有很大的 Token 数量。例如，一张高分辨率图片可以被划分为大量视觉 Patch。一个长视频又包含许多图像帧。如果全部输入共享 Transformer，则注意力计算可能成为瓶颈。对于具有 N 个 Token 的标准全注意力结构，其主要注意力交互计算和存储复杂度随 N 增长，通常具有近似二次关系。例如：$O(N^2)$，

但实际 Transformer 的总计算还受到隐藏维度、层数和实现方式影响。此外，FlashAttention[19] 等实现可以改善显存访问和运行效率，并不改变所有情况下的理论计算量级。因此，多模态统一建模需要考虑视觉 Token 压缩、局部注意力、稀疏注意力和分层时序表示等方法。但压缩也可能造成信息损失。例如，过度压缩视频可能遗漏短暂发生的重要事件。因此，需要在计算效率和信息保留之间进行权衡。

模态交互的因果验证 在多模态联合训练中，模型可能表面上使用多个模态，但实际预测主要依赖其中一个。例如，某个视频问答任务可以仅根据字幕回答。即使模型结构包含视频编码器，也不能因此证明模型充分利用了视觉信息。因此，需要通过干预实验分析模态使用情况。例如：移除视觉输入。移除音频输入。交换不同视频的字幕。改变某个对象的视觉属性。比较原始输入与错配输入的结果。但干预实验应当控制任务语义和输入分布。

例如，完全随机替换视频可能产生明显不自然的输入，导致模型下降不能完全归因于原始模态信息的重要性。因此，应当设计具有明确物理或语义含义的干预条件。

7.6 理解与生成的结构取舍

7.6.1 理解任务与生成任务的联合训练

联合训练的基本目标 理解生成一体化不仅是模型结构设计问题，还涉及训练数据与优化目标的协调。例如，视觉理解任务可以采用图像问答、图像描述和目标识别等监督。图像生成任务则可以使用文本—图像配对数据，通过自回归、扩散或流匹配等方式进行训练。如果两类任务共享部分参数，则可以构造联合损失：

$$\mathcal{L} = \lambda_U \mathcal{L}_U + \lambda_G \mathcal{L}_G \tag{7.102}$$

其中：

- $\mathcal{L}_U$ 表示理解任务损失；
- $\mathcal{L}_G$ 表示生成任务损失；
- λ_U, λ_G 为相应权重。

这一形式允许两类任务共同影响共享参数。但不同损失的数值尺度、数据数量和优化难度可能不同，因此不能仅根据损失项数量判断训练是否均衡。

理解任务的监督形式 多模态理解任务可以采用多种训练目标。例如，视觉问答可以通过文本自回归损失进行监督：

$$\mathcal{L}_U = -\sum_{i=1}^{L} \log p_\theta(y_i \mid y_{<i}, I, Q) \tag{7.103}$$

其中，I 为输入图像，Q 为文本问题，y_i 为答案 Token。图像描述任务也可以使用

类似的条件文本生成目标。对于目标检测、空间定位或其他结构化输出任务，还可以根据输出表示选择分类、回归或序列预测等监督方式。因此，多模态理解不一定对应单一固定的损失函数。对于以语言模型为输出主干的理解生成一体化架构，许多理解任务可以组织为条件文本生成，但这并不意味着所有感知任务都应该强制转换为自然语言描述。

例如，需要精确输出边界框、几何位姿和连续控制量的任务，仍然可能需要相应的结构化表示和训练目标。

生成任务的监督形式 对于图像生成，如果使用离散视觉 Token，可以采用自回归损失：

$$\mathcal{L}_G = -\sum_{j=1}^{K} \log p_\theta(v_j \mid v_{<j}, T) \tag{7.104}$$

其中，T 表示文本条件。如果采用连续扩散模型，则可以使用相应的噪声预测目标：

$$\mathcal{L}_{diff} = \mathbb{E}\left[|\epsilon - \epsilon_\theta(z_t, t, T)|_2^2\right] \tag{7.105}$$

对于使用直线插值的流匹配模型，可以采用速度场回归目标：

$$\mathcal{L}_{flow} = \mathbb{E}_{z_0, z_1, t, T}\left[\|v_\theta(z_t, t, T) - (z_1 - z_0)\|_2^2\right] \tag{7.106}$$

其中，z_0 为基础分布样本，z_1 为目标潜在样本，并采用相应的时间插值约定。这些目标对应不同的生成表示与学习机制。因此，联合训练既可以采用相同的离散 Token 交叉熵，也可以组合离散序列预测与连续生成损失。这也是理解生成一体化模型在技术路线上的重要区别。

数据混合与任务采样 联合训练需要考虑不同数据类型之间的比例。例如，视觉问答数据通常具有明确的问题和答案，而文本生成图像数据主要包含图像及其文字描述。两类数据在样本数量、文本长度和监督粒度上可能存在明显差异。如果直接将全部数据按数量混合训练，数据规模较大的任务可能主导参数更新。因此，可以通过任务采样分布控制不同数据的训练频率。设任务集合为：

$$\mathcal{T} = \{1, 2, \ldots, K\} \tag{7.107}$$

可以为不同任务设置采样概率：$P(k) = \pi_k$，其中：

$$\sum_{k=1}^{K} \pi_k = 1 \tag{7.108}$$

训练过程中，根据相应采样策略选择任务和样本。这种方式使不同任务的训练贡献不完全由原始数据规模决定。但采样比例并不存在适合全部模型和数据条件的固定

最优设置。过度增加某一种任务的训练频率，可能改善该任务性能，同时影响其他能力。因此，任务采样和损失权重通常需要联合考虑。

共享参数与梯度竞争 当理解任务和生成任务共同更新一组参数时，它们的优化方向可能存在差异。设共享参数为：θ_s 理解任务和生成任务的梯度分别为：

$$g_U = \nabla_{\theta_s} \mathcal{L}_U \tag{7.109}$$

$$g_G = \nabla_{\theta_s} \mathcal{L}_G \tag{7.110}$$

那么联合梯度为：

$$g = \lambda_U g_U + \lambda_G g_G \tag{7.111}$$

当两类任务的梯度方向存在较大差异时，共享参数可能面临优化竞争。例如，可以观察：

$$\cos(g_U, g_G) = \frac{g_U^\top g_G}{\|g_U\|_2 \|g_G\|_2} \tag{7.112}$$

当余弦相似度为负时，说明当前梯度方向存在一定冲突。但某次梯度夹角为负并不自动证明模型已经发生有害的负迁移。实际影响还取决于参数空间、优化过程和长期训练表现。因此，更可靠的评价方式，是在相同任务和数据条件下，比较独立训练、联合训练以及不同参数共享方式的结果。

表示对齐与功能协同 当模型使用不同编码器处理理解和生成任务时，还需要考虑两类表示之间如何建立联系。例如，理解编码器可以提取图像中的对象类别、属性和场景语义。生成编码器则需要保留重建图像所需的视觉信息。为了使共享模型能够利用两类表示中的相关知识，可以引入适当的表示对齐或关联目标。例如，概念性地写为：

$$\mathcal{L}_{cons} = d\left(P_U(H_U), P_G(H_G)\right) \tag{7.113}$$

其中：

- H_U 为理解表示；
- H_G 为生成表示；
- P_U, P_G 为相应映射；
- d 表示某种表示差异度量。

这一公式只表达了候选设计思路，并不代表所有统一模型都需要对两种表示执行逐元素距离约束。如果过度强制语义特征与重建特征保持完全一致，反而可能压制各自有价值的信息。因此，对齐目标应当明确共享的究竟是对象语义、空间关系还是其他任务相关信息，并通过实验验证其必要性。

7.6.2 统一建模的优势、代价与局限

统一模型的潜在优势 理解与生成一体化具有几个方面的潜在优势。模型可以复用不同任务之间的语义知识。例如，视觉问答学习到的对象和属性关系，可以在一定条件下为文本生成图像提供辅助。共享部分计算主干可以减少多个模型之间的结构重复。例如，理解与生成任务可以使用相同的语言表示或 Transformer 参数，而不必为每个任务独立建立完整的语言计算模块。统一模型可以在同一个交互过程中执行不同任务。

例如，用户可以先要求描述图像，再要求编辑某个对象，并继续询问编辑结果。这种任务切换不一定需要重新建立完全独立的对话系统。但这些都是潜在优势。其实际效果取决于模型结构、训练数据、参数容量和系统实现，而不是统一模型这一名称本身。

理解与生成的相互迁移 在联合训练中，理解和生成可能相互促进。例如，理解任务可以使模型更加关注对象语义、属性和空间关系。生成任务则可能促使模型保留更加丰富的视觉细节和场景表示。如果两类任务的训练目标具有一定互补性，共享表示可能得到改善。但也可能出现相反情况。例如，图像生成目标要求模型学习大量局部纹理，而某些视觉理解任务更需要抽象语义。如果模型容量有限或训练权重不合适，就可能发生负迁移。

因此，任务之间是否具有协同关系，需要通过联合训练与独立训练的比较验证。不能仅因为模型同时完成两类任务，就推断两类能力必然互相增强。

参数共享不一定带来推理加速 统一模型的一项常见动机，是减少多个独立模型之间的参数重复。例如，两个任务可以共享某个 Transformer 主干：

$$P_{total} = P_{\text{shared}} + P_U + P_G \tag{7.114}$$

其中：

- P_{shared} 表示共享参数量；
- P_U 表示理解专用参数量；
- P_G 表示生成专用参数量。

相比两个完全独立的模型，统一设计可能减少部分重复存储。但参数共享不等于推理计算量一定减少。例如，理解一张图像可能只需要执行视觉编码与语言解码，而生成一张图像还可能需要执行多步扩散或 ODE 积分。因此，即使二者共享同一 Transformer，图像生成的采样过程仍然需要相应计算。此外，统一模型可能同时保存理解编码器、生成编码器和图像解码器，导致整个系统仍具有较大的参数规模。

因此，应区分模型权重总量、任务激活参数量、实际计算量和端到端延迟。

训练数据与优化成本 理解生成一体化通常需要覆盖多种任务的数据。例如，视觉理解需要图像问答、描述和定位等数据。图像生成需要文本—图像配对及其他相应监督。如果进一步扩展到音频和视频，还需要更多类型的多模态数据。因此，统一模型的训练

数据组织可能更加复杂。不同数据集的样本质量、任务数量和监督形式存在差异，需要相应的数据筛选和混合策略。此外，模型可能需要同时优化交叉熵、扩散或流匹配等不同目标。

这些目标在数值尺度和训练行为上并不相同，增加了联合优化的难度。因此，统一模型虽然具有参数复用潜力，但其训练过程不一定比独立模型更简单或更节省计算。特别是在从头训练大规模多模态模型时，数据准备和计算资源可能构成较高门槛。

循环一致性及其局限 一种可能的评价方式，是让模型根据文本生成图像，再根据生成图像产生描述，检查前后内容是否一致。例如：

$$T \xrightarrow{\text{图像生成}} \hat{I} \xrightarrow{\text{图像理解}} \hat{T} \tag{7.115}$$

可以比较：T，与：$\hat{T}$ 之间的语义关系。类似地，也可以根据原始图像生成描述，再尝试依据描述重建图像。但循环一致性不能简单理解为最终结果必须与输入完全相同。例如，同一段文字可以对应多张合理图像。一张图像也可以有多种正确描述。因此，精确像素重建或逐字文本一致并不一定是合理目标。更适合评价的是对象、属性、空间关系和其他任务约束是否得到保持。

此外，如果理解模块与生成模块具有相似偏差，也可能在循环中产生彼此一致但偏离真实内容的结果。因此，循环一致性可以提供辅助证据，但不能单独证明模型实现了可靠的理解生成协同。

习题与讨论

1. 为输入级、特征级和决策级融合各画一个计算图，指出同一系统能否同时包含三者。
2. 视觉、音频和文本长度分别为 N_v, N_a, N_t。比较共享全注意力与文本单向读取视听键值的主要配对数量。
3. 解释为什么 MoE 的路由权重不能直接当作某一模态的可靠概率，并给出一种检验方案。
4. 为一个同时理解与生成图像的模型列出输入编码、输出解码、共享模块和目标位置。说明“统一”具体体现在哪些维度。

文献导读 Chameleon 与 Transfusion 适合比较统一序列和混合生成目标；阅读时重点核对表示、遮罩和损失位置。[15, 18]

第八章　条件生成与理解生成协同

生成模型学习条件分布，而不是寻找唯一的正确输出。自回归、扩散和流匹配对分布的参数化不同，训练样本、预测量和采样过程也不同。本章先把这些对象写清，再讨论图像、视频和音频的条件控制。

学习目标　区分序列概率、噪声预测与速度场；追踪条件进入网络的位置；把生成质量、条件一致性和任务有效性分开评价。

8.1　自回归、扩散与流匹配

统一比较生成范式时，至少记录五项：数据表示、训练时随机变量、网络预测量、损失和推理更新。扩散或流模型即使使用 Transformer，也不因此变成语言模型式的下一离散 Token 预测。

8.1.1　自回归生成

自回归建模的机制　自回归生成（Autoregressive Generation）的核心思想，是利用已经出现的序列元素，预测下一个元素的条件分布。设目标序列为：$X = (x_1, x_2, \ldots, x_N)$ 根据概率链式法则，其联合分布可以表示为：

$$p_\theta(x_1, \ldots, x_N) = \prod_{i=1}^{N} p_\theta(x_i \mid x_1, \ldots, x_{i-1}) \tag{8.1}$$

也可以简写为：

$$p_\theta(X) = \prod_{i=1}^{N} p_\theta(x_i \mid x_{<i}) \tag{8.2}$$

这一分解不要求序列元素之间相互独立，而是通过前面所有可见位置的条件信息，建立后续元素的概率分布。例如，对于句子：“今天的天气非常晴朗。”模型可以根据“今天的天气”，预测下一个词语或文本 Token；随后，再根据已经形成的文本前缀继续预测。因此，自回归模型能够通过逐步生成的方式，构造完整的语言序列。

Transformer Decoder 是这一技术路线的重要实现形式。通过因果自注意力，模型可以利用已有 Token 形成上下文表示，同时避免训练时当前位置直接访问未来目标 Token。

自回归模型的训练目标　在基于离散 Token 的生成模型中，训练通常采用最大似然估计。给定训练序列：$X = (x_1, \ldots, x_N)$ 可以最小化负对数似然：

$$\mathcal{L}_{AR} = -\sum_{i=1}^{N} \log p_\theta(x_i \mid x_{<i}) \tag{8.3}$$

在实际训练中，模型通常利用教师强制（Teacher Forcing），以真实历史 Token 作为条件，预测相应的下一个 Token。例如，模型输入一段已知文本，在每个位置预测其后的目标 Token。由于 Transformer 可以通过因果注意力同时处理多个训练位置，因此虽然模型的概率分解具有自回归结构，训练时也不一定需要像推理一样逐个 Token 串行执行完整网络。这一区别非常重要。

自回归模型在训练阶段可以并行计算多个预测位置的损失；在常规逐 Token 推理阶段，则需要利用前一步生成的结果继续预测。

自回归采样过程 完成训练后，模型可以从初始条件出发逐步生成序列。例如，给定文本提示 c，首先预测：

$$x_1 \sim p_\theta(x_1 \mid c) \tag{8.4}$$

随后：

$$x_2 \sim p_\theta(x_2 \mid x_1, c) \tag{8.5}$$

依次得到：

$$x_i \sim p_\theta(x_i \mid x_{<i}, c) \tag{8.6}$$

直到生成满足停止条件的完整序列。在每一步预测中，模型通常输出候选 Token 的概率分布，再根据相应的解码策略选择下一个 Token。例如，可以选择概率最高的 Token，也可以采用温度采样、Top-k 或 Top-p 等方式控制生成结果的多样性。不同解码策略可能产生不同的文本或多模态输出，其差异不一定来自模型参数本身。因此，评价自回归生成时，通常需要说明所采用的解码策略。

自回归生成与多模态 Token 自回归生成最容易理解的应用是文本生成，但其建模方式并不只适用于自然语言。如果图像经过离散视觉 Tokenizer 转换为 Token 序列，也可以采用自回归模型逐步预测视觉 Token。例如：

$$I \xrightarrow{\text{Tokenizer}} V = (v_1, \ldots, v_K) \tag{8.7}$$

随后学习：

$$p_\theta(V) = \prod_{i=1}^{K} p_\theta(v_i \mid v_{<i}) \tag{8.8}$$

对于文本条件图像生成，可以进一步写为：

$$p_\theta(V \mid T) = \prod_{i=1}^{K} p_\theta(v_i \mid v_{<i}, T) \tag{8.9}$$

其中，T 表示文本条件。模型生成完整视觉 Token 序列后，再利用相应图像解码器恢复视觉内容。类似地，音频可以通过离散音频编解码器转换为声学 Token，视频也可以通过相应 Tokenizer 组织为空间和时间序列。因此，只要建立适当的离散表示，自回归模型就可以从文本扩展到更多模态。

统一多模态自回归建模 自回归生成的一项重要特点，是能够在形式上使用统一的下一 Token 预测目标。例如，模型可以将文本、图像和视频转换为离散 Token，并组织为混合序列：$S = (s_1, s_2, \ldots, s_N)$，随后学习：

$$p_\theta(S) = \prod_{i=1}^{N} p_\theta(s_i \mid s_{<i}) \tag{8.10}$$

在这种结构中，不同位置的 Token 可以属于不同模态，但模型仍然使用共享的自回归预测框架。Chameleon[15] 和 Emu3 是相关技术路线的代表。其中，Emu3 通过将文本、图像和视频离散化为 Token 序列，探索以统一下一 Token 预测完成多模态理解与生成。但统一预测目标并不意味着不同模态具有相同的信息结构。

例如，图像 Token 可能来自二维空间量化，音频 Token 具有时间和声学结构，文本 Token 则主要表示语言符号。模型仍然需要通过 Tokenizer、位置编码和输出解码器，处理这些差异。

自回归生成的优势与局限 自回归生成的主要优势，是具有明确的概率分解形式和相对统一的训练目标。对于文本和其他离散 Token 序列，可以使用交叉熵损失开展训练，并通过共享 Transformer 进行序列建模。此外，历史 Token 能够作为后续生成的条件，使模型适合处理语言上下文、事件顺序和交错多模态序列。但其主要限制之一，是常规生成过程通常具有序列依赖。

例如，如果一张图像需要通过数千个视觉 Token 表示，那么逐 Token 生成可能需要较多解码步骤。对于视频和高质量音频，输出 Token 数量可能进一步增加。因此，自回归生成的计算成本受到输出表示长度和解码策略的明显影响。此外，如果图像或音频经过离散量化，生成质量还可能受到 Tokenizer 的重建能力影响。

较长输出序列并不必然意味着实际推理速度一定低于其他生成范式。不同方法的单步计算量、并行程度和推理优化方式并不相同。因此，应通过实际的端到端生成延迟和质量评价比较不同方法，而不只是比较生成步骤数量。

8.1.2 扩散模型的基本原理

扩散模型引入逐步加噪的前向过程，再学习反向生成。以离散时间、各向同性高斯噪声的常见设置为例，定义 $\alpha_t = 1 - \beta_t$、$\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$，可直接采样

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t}\, \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I). \tag{8.11}$$

网络读取 x_t, t 及条件 c，一种常见的简化训练目标是噪声均方误差

$$\mathcal{L}_\epsilon = \mathbb{E}_{x_0,t,\epsilon}\left[\|\epsilon - \epsilon_\theta(x_t, t, c)\|_2^2\right]. \tag{8.12}$$

它对应噪声参数化的代表性训练设置，不涵盖所有扩散目标。反向过程根据所选参数化与采样器更新噪声状态，反复调用网络得到样本；训练目标与推理更新不能混为同一公式。[20]

条件可通过交叉注意力或调制进入网络。若文本状态作为键和值，去噪网络的隐藏状态负责查询相关条件；这种接口与理解模型的注意力有联系，但输出与监督量不同。

潜在扩散先用自编码器 $z_0 = E(x_0)$ 压缩数据，再在潜在空间生成，最后经 D 解码。计算规模降低的同时，最终细节受到自编码器重建与生成模型共同限制。采样步数减少还会改变计算与质量，实际收益需要在相同条件、分辨率和采样设置下评价。[21]

8.1.3 流匹配的基本思想

流模型用时变速度场把基础分布的样本变换为目标分布。给定 x_0 来自基础分布，求解

$$\frac{dx_t}{dt} = v_\theta(x_t, t, c), \qquad t \in [0, 1]. \tag{8.13}$$

时间方向与下标必须先约定；本节 $t = 0$ 为基础端，$t = 1$ 为数据端。流匹配通过选定概率路径及其速度监督训练该场，不要求在每次训练中完整求解生成 ODE。[22]

用原稿中的线性插值作教学实例：对独立抽取的端点 x_0, x_1，令 $x_t = (1-t)x_0 + tx_1$，条件路径速度为 $x_1 - x_0$。可训练

$$\mathcal{L}_{\mathrm{CFM}} = \mathbb{E}_{x_0,x_1,t,c}\left[\|v_\theta(x_t, t, c) - (x_1 - x_0)\|_2^2\right]. \tag{8.14}$$

同一个 x_t 可以对应不同端点，最小二乘学习的是给定当前状态与条件的速度平均关系，不能把网络视为知道每个样本未输入的真实端点。不同概率路径或耦合会改变速度目标，线性插值不是全部流匹配的定义。

推理时从基础样本出发，按数值求解器积分。实际调用次数、步长和误差由求解设置决定；更直的路径或更少步数也不自动保证在任意模型与数据上质量更好。

8.2 生成表示与条件机制

条件信息可以进入输入序列、交叉注意力或特征调制。可控性既依赖条件接口，也依赖训练中是否出现足够的对象、属性和关系组合。只检查画面是否流畅，无法确认模型遵守了条件。

8.2.1 连续特征与离散 Token

生成对象的表示问题 生成模型的目标是产生符合任务要求的数据，但原始数据未必适合直接进入生成网络。例如，一张 RGB 图像可以表示为：

$$I \in \mathbb{R}^{H\times W\times 3} \tag{8.15}$$

一段单通道音频可以表示为：

$$A \in \mathbb{R}^{T} \tag{8.16}$$

一段视频则可以表示为：

$$V \in \mathbb{R}^{F\times H\times W\times 3} \tag{8.17}$$

其中，F 为视频帧数量。这些数据在维度、结构和取值范围上存在明显差异。如果直接在原始数据空间进行生成，模型可能需要处理大量连续变量，尤其对于高分辨率图像、长音频和视频，计算成本较高。因此，许多生成模型首先通过编码器或 Tokenizer，将原始数据转换为更适合学习的表示。这一表示可以是连续潜在特征，也可以是离散 Token 序列。

生成模型中的 Token 不一定表示离散符号。Transformer 处理的连续特征向量也经常被称为 Token。因此，在讨论多模态生成时，应当明确区分连续 Token 表示与离散 Token 索引。

连续潜在表示 连续潜在表示通常通过编码网络，将原始数据映射到维度较低或结构更加紧凑的连续空间。例如，图像编码器可以实现：$z = E(I)$，其中：

$$z \in \mathbb{R}^{h\times w\times d} \tag{8.18}$$

表示图像的潜在特征。随后，解码器根据潜在特征恢复图像：$\hat{I} = D(z)$，这种编码与解码结构，使生成模型不必直接处理原始图像的全部像素，而可以在潜在空间中学习数据分布。例如，在潜在扩散 [21] 模型（Latent Diffusion Model，LDM）中，图像首先经过预训练自编码器转换为连续潜在特征，再通过扩散模型学习潜在特征的生成过程。生成结束后，模型将潜在特征交给图像解码器，得到最终图像。

这种方法的重要价值，是通过适当压缩降低生成网络面对的数据规模。但潜在特征并不保证无损保留原始图像的全部信息。编码器与解码器之间的重建质量，直接影响最终可以恢复的图像细节。

连续表示的信息保留 连续潜在空间的设计，需要在压缩程度和信息保留之间取得平衡。例如，如果编码器将一张高分辨率图像压缩为非常小的特征图，生成模型需要处理的变量数量可能明显减少。但过度压缩也可能损失小目标、细小文字和局部纹理等信息。可以用重建损失描述编码器与解码器的基本训练要求：

$$\mathcal{L}_{rec} = \mathbb{E}_I \left[d\big(I, D(E(I))\big) \right] \tag{8.19}$$

其中，d 表示图像重建误差度量。例如，d 可以是像素误差，也可以包含感知特征距离等其他评价形式。实际自编码器还可能包含对抗训练、潜在分布正则化等目标。因

此，连续潜在表示的质量不能只通过潜在特征维度判断，还应当考虑重建图像的视觉质量及关键内容是否得到保留。对于多模态条件生成，这一点尤其重要。例如，即使文本条件准确描述了某个交通标志，如果视觉解码器无法恢复相应的细小字符，最终图像仍然可能无法正确表达指定内容。

离散 Token 表示 另一种方法是将连续数据转换为离散 Token。例如，图像编码器首先输出连续特征：$z = E(I)$，随后，通过向量量化（Vector Quantization），将每个连续特征映射到码本中的某个向量。设码本为：

$$\mathcal{C} = \{e_1, e_2, \ldots, e_K\} \tag{8.20}$$

对于一个连续特征向量 z，可以选择：

$$k^* = \arg\min_k \|z - e_k\|_2^2 \tag{8.21}$$

其中，k^* 为离散码本索引。通过量化过程，一张图像可以表示为离散索引序列：$V = (v_1, v_2, \ldots, v_N)$，其中，每个 v_i 表示一个码本索引。这一思想在 VQ-VAE（Vector Quantised Variational AutoEncoder）中得到了系统研究。

VQ-VAE 使用离散潜在变量表达输入，并通过相应解码器恢复数据。其离散表示可以进一步由自回归模型或其他生成模型学习。因此，离散 Tokenizer 为连续视觉和音频数据使用离散符号生成模型提供了基础。

离散表示与生成模型的连接 经过离散化后，可以将图像 Token 作为序列建模对象。例如：$V = (v_1, \ldots, v_N)$，通过自回归模型学习：

$$p_\theta(V) = \prod_{i=1}^{N} p_\theta(v_i \mid v_{<i}) \tag{8.22}$$

对于文本条件图像生成，则可以写为：

$$p_\theta(V \mid T) = \prod_{i=1}^{N} p_\theta(v_i \mid v_{<i}, T) \tag{8.23}$$

其中，T 为文本条件。模型首先生成视觉 Token 序列，再使用视觉 Tokenizer 对应的解码器，将离散索引转换为图像。Chameleon 和 Emu3 等工作利用离散视觉 Token，将图像与文本组织为混合模态序列，并通过共享自回归模型学习相应的理解与生成任务。离散化只是建立序列生成接口的一种方式，并不要求全部图像生成都采用逐 Token 自回归预测。

例如，MaskGIT 使用掩码 Token 预测与迭代并行解码，在离散视觉 Token 空间中实现图像生成。因此，离散表示与自回归生成具有较强联系，但并不是一一对应关系。

音频中的连续与离散表示 音频生成同样可以采用连续潜在特征或离散 Token。连续表示可以是声学频谱特征、神经网络潜在特征或其他与波形相关的表示。例如：$Z_A =$

$E_A(A)$，随后，模型可以通过连续生成过程预测 Z_A，再利用声码器或音频解码器恢复声音。另一种方式是使用神经音频编解码器，将音频转换为离散 Token。例如：$U_A = Q(E_A(A))$，随后通过序列模型预测：

$$p_\theta(U_A \mid T) \tag{8.24}$$

其中，T 可以是对应文本内容。音频编解码器可能在不同时间位置使用多个量化码本，因此音频 Token 不一定是简单的一维单码本序列。此外，离散 Token 需要同时表达语言内容、说话者特征和声学细节，不同音频 Tokenizer 的信息保留能力可能存在明显差异。因此，音频生成的质量不仅取决于语言模型或生成主干，也受到声学表示与解码器质量影响。

8.2.2 条件输入与条件生成

条件生成的基本定义 生成模型可以在不提供特定外部条件的情况下，从学习到的数据分布中采样。例如，无条件图像生成可以表示为：$x \sim p_\theta(x)$，而在多模态任务中，模型通常需要根据其他信息控制生成结果。例如：根据文本描述生成图像。根据参考图像修改局部内容。根据文本生成语音。根据图像和动作描述生成视频。这些任务可以统一表示为：

$$x \sim p_\theta(x \mid c) \tag{8.25}$$

其中，x 为目标生成对象，c 为外部条件。条件 c 不一定是文本。它可以是离散标签、连续特征、空间结构、另一张图像，或多个模态信息的组合。因此，多模态条件生成首先需要解决两个问题：如何表示条件信息，以及如何让条件影响生成网络的计算过程。

条件编码与条件注入 设输入条件为：c 通常可以通过条件编码器得到：$H_c = f_c(c)$，随后，将 H_c 提供给生成网络。例如，对于扩散模型：

$$\hat{\epsilon} = \epsilon_\theta(x_t, t, H_c) \tag{8.26}$$

对于流匹配模型：$\hat{v} = v_\theta(x_t, t, H_c)$，对于自回归模型：

$$p_\theta(y_i \mid y_{<i}, H_c) \tag{8.27}$$

这些形式说明，虽然生成目标不同，但模型都可以利用条件表示影响生成过程。条件编码器可以是独立的预训练模型，也可以与生成主干共同训练。例如，文本条件通常可以由预训练文本编码器提取；参考图像条件可以由视觉编码器提取；某些空间控制信息则可以通过专门的结构编码模块处理。因此，条件编码与生成主干是两个可以独立分析的结构部分。

通过 Cross-Attention 注入条件 另一种方法是将条件信息保留为独立特征序列，通过交叉注意力提供给生成网络。例如，当前图像生成特征为：H_X 文本条件表示为：H_T 可以计算：

$$H'_X = \text{Attention}\left(H_X W_Q, H_T W_K, H_T W_V\right) \tag{8.28}$$

这样，图像生成网络可以根据当前隐藏状态，从文本中读取相关语义信息。在潜在扩散模型中，Cross-Attention 是一种具有代表性的条件注入机制。例如，文本描述包含多个对象和属性时，生成网络可以通过跨注意力访问相应文本特征，使不同空间位置的生成过程受到文本条件影响。但注意力权重并不等于严格的对象—文字对应证明。例如，模型可能对某个词语给予较高注意力，却仍然没有正确生成相应对象。

因此，Cross-Attention 提供了条件信息交互的计算机制，而条件遵循能力仍然需要通过生成结果验证。

通过特征调制注入条件 除序列拼接和 Cross-Attention 外，还可以通过特征调制使条件影响生成过程。例如，给定隐藏特征：H 条件网络根据 c 产生缩放和偏移参数：$\gamma(c),\quad \beta(c)$，随后执行：

$$H' = \gamma(c) \odot H + \beta(c) \tag{8.29}$$

其中，$\odot$ 表示逐元素乘法。这种机制可以在不同层调整隐藏特征的数值分布，使网络计算受到条件影响。例如，FiLM（Feature-wise Linear Modulation）使用条件相关的特征缩放与偏移，实现对中间表示的调制。

在扩散 Transformer 中，DiT 进一步研究了基于条件的自适应层归一化（Adaptive Layer Normalization，adaLN）以及相应的 adaLN-Zero 设计。这些方法通常适合将扩散时间、类别标签或经过聚合的条件向量注入网络。但向量级特征调制与 Token 级 Cross-Attention 的信息交互粒度不同。

如果条件是一段较长文本，并且生成过程需要区分不同词语之间的关系，那么仅使用单个全局条件向量，可能难以提供与 Token 级交互完全相同的细节。因此，实际架构也可以结合多种条件注入机制。

条件引导与 Classifier-Free Guidance 在扩散模型中，除了将条件信息输入去噪网络，还可以在采样阶段调整条件对生成过程的影响。Classifier-Free Guidance（CFG）是具有代表性的条件引导方法。其基本思想是，训练模型既能够处理给定条件，也能够处理条件被移除或替换为空条件的情况。在采样时，可以组合无条件预测和条件预测。例如，对于噪声预测参数化：

$$\hat{\epsilon} = \epsilon_\theta(x_t, t, \varnothing) + s\left[\epsilon_\theta(x_t, t, c) - \epsilon_\theta(x_t, t, \varnothing)\right] \tag{8.30}$$

其中：

- $\varnothing$ 表示空条件；

- c 表示目标条件；
- s 为引导尺度。

在这一公式约定下：$s = 1$，对应原始条件预测。当：$s > 1$ 时，进一步增强条件预测相对于无条件预测的差异。不同实现对引导尺度的定义可能采用不同记号或重参数化方式，因此比较具体模型时应以对应实现为准。此外，较强的条件引导通常可能改善部分条件遵循表现，但也可能降低样本多样性，并在某些情况下产生过度饱和或其他生成伪影。因此，CFG 是采样阶段的条件控制方法，不等同于提高条件编码器的语义理解能力。

8.2.3 生成目标与训练损失

训练目标与生成范式的关系 多模态生成模型可以采用不同的训练目标。例如，自回归模型通常学习下一个 Token 的条件概率；扩散模型可以学习噪声预测；流匹配模型学习连续速度场。因此，模型的训练损失与其生成表示和生成过程具有直接联系。但生成任务不一定只有一种损失。例如，图像 Tokenizer 可能首先需要重建损失训练；随后，自回归生成主干再使用交叉熵损失学习离散视觉 Token。

类似地，潜在扩散模型可以先训练图像自编码器，再训练潜在空间中的去噪网络。因此，完整生成系统可能包含表示学习阶段与生成学习阶段，两者的训练目标需要分别分析。

离散 Token 的自回归损失 对于离散 Token 序列：$Y = (y_1, y_2, \ldots, y_N)$ 可以使用负对数似然：

$$\mathcal{L}_{AR} = -\sum_{i=1}^{N} \log p_\theta(y_i \mid y_{<i}, C) \tag{8.31}$$

其中，C 为条件信息。在文本生成中，y_i 通常为文本 Token。在离散图像生成中，y_i 可以是视觉码本索引。对于音频生成，则可以使用相应音频 Token。因此，自回归损失具有较强的跨模态适用性。但不同模态的 Token 数量可能存在明显差异。例如，输出图像可能需要大量视觉 Token，而文本回答可能只有几十个 Token。

如果直接使用未归一化的 Token 损失总和，较长的生成序列可能对总体损失贡献更大。因此，多任务训练需要明确 Token 数量归一化和任务采样策略。

掩码 Token 预测损失 并非所有离散 Token 生成都采用严格自回归训练。例如，可以随机遮挡部分离散视觉 Token，让模型根据可见内容预测被遮挡的位置。设输入 Token 序列为：$V = (v_1, \ldots, v_N)$ 掩码位置集合为：

$$\mathcal{M} \tag{8.32}$$

则可以构造：

$$\mathcal{L}_{mask} = -\sum_{i \in \mathcal{M}} \log p_\theta(v_i \mid V_{\setminus \mathcal{M}}, C) \tag{8.33}$$

其中，$V_{\setminus \mathcal{M}}$ 表示模型能够观察到的未遮挡 Token。MaskGIT 使用掩码视觉 Token 预测进行训练，并在推理阶段通过多轮并行预测和逐步减少掩码的方式生成图像。这一方法说明，离散 Token 生成不仅可以通过逐位置自回归预测，也可以采用非自回归的迭代填充方式。但掩码预测与扩散模型虽然都包含逐步完善样本的思想，其训练变量和生成机制并不相同。

因此，不应将掩码 Token 迭代生成直接视为连续扩散模型。

连续潜在表示的重建损失 如果模型通过自编码器得到连续潜在表示，则还需要训练编码器和解码器。例如：$z = E(I)$，$\hat{I} = D(z)$ 可以使用：

$$\mathcal{L}_{rec} = d(I, \hat{I}) \tag{8.34}$$

其中，d 为适当的重建误差。对于图像，可以使用像素距离、感知损失或其他相应目标。对于音频，则可以采用波形、频谱或多分辨率声学重建损失。自编码器的重建目标与生成主干的训练目标不同。例如，潜在扩散模型的图像自编码器负责提供可恢复的潜在空间，而去噪网络负责学习潜在变量的生成分布。因此，较低的重建误差不意味着生成模型已经能够从噪声中产生高质量样本。

反过来，如果生成网络学习充分，但解码器本身存在明显信息损失，最终生成质量也会受到限制。

扩散模型的去噪损失 对于扩散模型，可以从真实样本：x_0 构造带噪状态：

$$x_t = \alpha_t x_0 + \sigma_t \epsilon \tag{8.35}$$

其中：

$$\epsilon \sim \mathcal{N}(0, I) \tag{8.36}$$

α_t 和 σ_t 为相应噪声日程参数。对于条件噪声预测模型，可以使用：

$$\mathcal{L}_{diff} = \mathbb{E}_{x_0, t, \epsilon, C}\left[|\epsilon - \epsilon_\theta(x_t, t, C)|_2^2\right] \tag{8.37}$$

这一损失使模型学习在给定条件和噪声时间的情况下，预测噪声分量。在潜在扩散中，x_0 可以进一步替换为图像编码器产生的潜在特征。因此，扩散损失可以在连续潜在空间中计算，而不必直接作用于原始像素。不同扩散模型还可以采用预测干净数据、速度参数化或其他形式，并且可能设置不同时间步权重。因此，上式代表常见噪声预测训练形式，而非全部扩散模型的唯一训练损失。

流匹配的速度回归损失 对于流匹配，可以首先设定基础分布样本：$x_0 \sim p_0$ 以及目标数据样本：$x_1 \sim p_{\text{data}}$ 采用简单直线插值：$x_t = (1-t)x_0 + tx_1$，对应的目标速度为：$u_t = x_1 - x_0$ 于是，可以通过平方误差训练速度场：

$$\mathcal{L}_{FM} = \mathbb{E}\left[|v_\theta(x_t, t, C) - u_t|_2^2\right] \tag{8.38}$$

其中，C 为相应条件信息。这种方式使模型学习在当前中间状态和条件下，如何沿相应概率路径更新样本。但这里展示的是直线插值路径下的简化流匹配目标，并不代表所有流匹配方法都使用相同路径。此外，流匹配中的速度回归与扩散模型中的噪声预测虽然使用不同的监督目标，但二者在连续生成动力学中可能存在数学联系。因此，不能仅根据训练损失公式不同，就认为两类模型完全没有关联。

多模态任务的联合损失 对于同时学习多种输入输出任务的模型，可以组合不同训练目标。例如，模型同时学习：文本生成、图像生成、音频生成、跨模态条件理解。其联合损失可以写为：

$$\mathcal{L} = \lambda_T \mathcal{L}_T + \lambda_I \mathcal{L}_I + \lambda_A \mathcal{L}_A + \lambda_C \mathcal{L}_C \tag{8.39}$$

其中，不同损失可以分别对应文本、图像、音频和条件相关任务。这些损失不必采用同一数学形式。例如，文本生成可以使用交叉熵，图像生成可以使用扩散或流匹配损失，音频生成可以使用离散 Token 损失或连续声学生成目标。因此，多模态理解生成一体化不意味着所有模态必须使用相同的损失函数。更加重要的是，不同训练目标能否在共享计算体系中形成有效协同。

目标 Token 掩码与监督范围 在多模态条件生成中，并不是所有输入 Token 都需要作为预测目标。例如，用户提供一张图像和一段问题，模型需要生成文字回答。如果训练目标只要求预测回答，那么输入图像和用户问题通常作为条件，而回答 Token 作为监督位置。可以定义：$m_i \in \{0, 1\}$ 表示第 i 个位置是否参与训练损失。则：

$$\mathcal{L} = -\frac{\sum_i m_i \log p_\theta(y_i \mid y_{<i}, C)}{\sum_i m_i} \tag{8.40}$$

这里假设至少存在一个有效监督位置。这种形式能够明确区分条件输入和预测目标。但如果采用混合模态联合预训练，模型也可能需要同时预测文本和视觉 Token。因此，哪些位置参与损失，应当由训练任务决定。监督掩码并不等同于注意力掩码。监督掩码决定哪些位置计算训练损失，注意力掩码则决定某个位置能够读取哪些上下文信息。两者作用不同，不能混用。

任务采样与损失平衡 多模态训练不仅需要考虑不同损失的数学形式，还需要考虑训练数据的组成。例如，文本数据规模可能明显大于高质量图像—音频配对数据。图像生成数据可能包含大量视觉 Token，而文本问答数据的输出较短。如果简单按原始数据量混合训练，某些模态可能主导共享参数更新。因此，可以通过任务采样概率和损失权重，调整不同任务的训练贡献。例如：

$$\mathcal{L} = \sum_{k=1}^{K} \lambda_k \mathbb{E}_{(X,Y)\sim\mathcal{D}_k}[\mathcal{L}_k(X,Y)] \tag{8.41}$$

其中：

- $\mathcal{D}_k$ 为第 k 类任务数据；
- λ_k 为相应损失权重；
- $\mathcal{L}_k$ 为该任务的训练损失。

Unified-IO 2 通过统一输入输出表示及多模态混合去噪目标，将不同类型的任务组织到共同训练体系中。这一类工作表明，多模态统一训练不仅涉及生成主干，还涉及任务数据、输出表示与监督目标之间的整体协调。

8.3 潜空间图像生成与控制

8.3.1 潜空间与图像解码

为什么需要潜空间生成 图像通常包含大量连续像素变量。例如，一张分辨率为：1024×1024 的 RGB 图像包含：$1024 \times 1024 \times 3$ 个像素通道数值。如果直接在像素空间中执行复杂生成计算，模型可能需要较高的显存和计算资源。因此，一种重要技术路线是先将图像压缩到潜在空间（Latent Space），再在潜在表示上开展生成。设图像编码器为 E，解码器为 D。

可以得到：$z = E(I)$ 并通过：$\hat{I} = D(z)$ 恢复图像。其中，z 为连续潜在表示。通过适当的空间压缩，潜在表示的变量数量可以明显少于原始图像。但潜空间生成的目标并不是简单让特征越小越好，而是寻找计算成本与视觉信息保留之间的合理平衡。

潜在扩散模型的基本架构 潜在扩散模型（Latent Diffusion Model，LDM）将扩散过程从原始像素空间转移到预训练自编码器的潜在空间。其主要过程可以分为两个阶段。第一阶段训练或获得图像自编码器，使其能够将图像压缩为潜在表示，并从潜在表示恢复图像。第二阶段使用潜在特征作为数据分布建模对象，训练条件扩散生成网络。例如，对于真实图像 I：$z_0 = E(I)$，通过向潜在特征添加噪声，构造：

$$z_t = \alpha_t z_0 + \sigma_t \epsilon \tag{8.42}$$

其中：

$$\epsilon \sim \mathcal{N}(0, I) \tag{8.43}$$

生成网络根据当前潜在状态、噪声时间和文本条件，预测相应的去噪信息：

$$\hat{\epsilon} = \epsilon_\theta(z_t, t, H_T) \tag{8.44}$$

推理时，模型从潜在噪声出发，执行相应的反向采样，得到生成的潜在表示：$\hat{z}_0$ 最后经过图像解码器：$\hat{I} = D(\hat{z}_0)$ 获得生成图像。这一结构使生成网络可以在压缩空间中学习视觉分布，同时利用自编码器恢复最终图像。

图像解码器与去噪网络的区别 在介绍潜在扩散模型时，需要区分两个经常被混淆的模块。一个是扩散模型中的去噪网络，它负责根据带噪潜在表示预测相应的去噪信息。另一个是图像自编码器的解码器，它负责将生成完成的潜在表示转换为像素图像。例如：

$$z_t \xrightarrow{\text{去噪网络与采样器}} \hat{z}_0 \xrightarrow{\text{图像解码器}} \hat{I} \tag{8.45}$$

二者承担不同功能。去噪网络主要学习潜在变量的生成分布。图像解码器主要学习潜在表示到图像空间的恢复映射。因此，生成网络即使具有较强的分布建模能力，也仍然受到图像解码器的重建能力限制。反过来，一个具有较高重建质量的图像自编码器，也不意味着系统已经能够根据文本从随机噪声中生成高质量图像。这一区分对于理解潜在扩散模型的完整训练与推理流程十分重要。

Stable Diffusion 的代表性技术路线 Stable Diffusion 系列早期具有代表性的模型建立在潜在扩散技术基础上。其典型系统包含文本编码器、潜在去噪网络和图像自编码器。文本编码器负责将自然语言描述转换为条件特征。生成网络在潜在空间中，根据文本条件预测噪声或其他相应目标。完成采样后，图像解码器将潜在结果恢复为最终图像。其中，文本条件可以通过 Cross-Attention 注入生成网络，使视觉潜在特征能够读取语言信息。

这种结构的一个重要特点，是将语义条件提取、视觉分布建模和图像重建划分为相互联系的模块。因此，研究者可以分别分析文本编码质量、生成网络容量和潜在空间压缩对最终结果的影响。但 Stable Diffusion 系列不同版本的网络主干、文本编码器和训练目标可能不同，不应将早期基于 U-Net 的潜在扩散结构直接视为全部后续版本的统一实现。

从 U-Net 到 Diffusion Transformer 经典潜在扩散模型通常采用 U-Net 形式的去噪网络。U-Net 具有多尺度编码和解码结构，能够在不同空间分辨率上处理图像特征，并通过相应跳跃连接保留局部信息。随着 Transformer 在视觉生成中的发展，研究者进一步探索使用 Transformer 作为连续生成模型的主要计算网络。

例如，Diffusion Transformer（DiT）将带噪潜在图像划分为 Patch Token，通过 Transformer 进行特征处理。在文本条件生成中，模型还可以进一步利用联合注意力，使文本特征与图像潜在 Token 在多个网络层交换信息。

Stable Diffusion 3 对应的研究《Scaling Rectified Flow Transformers for High-Resolution Image Synthesis》便探索了采用多模态 Diffusion Transformer（MMDiT）的高分辨率文本生成图像架构。该架构为图像与文本使用相应的参数处理路径，并在注意力计算中建立双向信息交互。

同时，其生成训练采用与 Rectified Flow 相关的连续速度场建模方法。这一发展说明，潜空间与图像解码的思想可以和不同生成网络及训练目标结合，而不局限于某一

种 U-Net 去噪模型。

潜空间压缩的能力边界 潜空间压缩降低了生成模型需要处理的数据维度，但也可能限制能够恢复的视觉细节。例如，对于包含大面积自然纹理的图像，自编码器可能在较高压缩程度下仍然获得较好的感知重建结果。但对于细小字符、规则几何线条或密集对象，压缩造成的局部误差可能更加明显。因此，评价潜在图像生成系统时，可以分别检查：原始图像经过自编码器重建后的质量。生成网络在潜在空间中的建模能力。最终图像是否满足文本条件。

如果模型始终无法准确生成某类细节，就需要判断问题主要来自潜在表示的重建限制，还是生成网络没有正确利用文本语义。例如，若自编码器对真实清晰文字的重建已经存在明显失真，那么单独扩大生成网络规模通常不能完全消除这一表示瓶颈。因此，潜空间设计不仅关系到计算效率，也直接影响最终图像的细节上限。

8.3.2 图像生成的可控性

从语义控制到结构控制 文本生成图像不仅要求模型生成视觉合理的内容，还需要按照用户要求控制图像的具体属性。例如，文本可以要求生成特定对象、颜色和场景。但在实际应用中，用户可能进一步希望控制对象位置、人物姿态、图像布局、拍摄视角或特定区域的内容。因此，可以将图像生成的条件控制区分为不同层次。语义控制主要决定图像包含什么对象及其属性。结构控制进一步涉及对象的空间布局、轮廓、深度或姿态。局部编辑控制则要求仅修改指定内容，同时保留其他视觉信息。

这些控制要求具有不同的信息粒度，因此不一定适合完全采用同一种条件表示方式。

文本提示词与语义控制 最直接的可控生成方式，是通过文本提示词（Prompt）表达目标要求。例如：“一辆红色汽车停在道路左侧，后方有一栋白色建筑。”这一文本同时给出对象类别、颜色和空间布局。文本编码器将其转换为语义特征，并通过条件注入机制影响生成过程。但自然语言通常不能完整规定一张图像的所有视觉细节。例如，文本可能没有指定汽车的具体型号、建筑物的精确形状和道路的透视结构。

因此，生成模型仍然需要根据训练数据中的视觉先验，对这些未指定部分进行补充。此外，较长的提示词不一定带来更准确的控制。例如，当文本同时包含多个对象、颜色和复杂空间关系时，模型可能出现属性错误绑定或关系混淆。因此，提示词表达能力和模型条件遵循能力应当分别评价。

条件引导与生成强度 在扩散和部分流匹配生成系统中，可以通过条件引导调整文本信息对生成结果的影响。例如，Classifier-Free Guidance（CFG）通过组合条件预测和无条件预测，改变采样过程中条件信号的作用。对于常见的噪声预测参数化，可以写为：

$$\hat{\epsilon} = \epsilon_\theta(z_t, t, \varnothing) + s\left[\epsilon_\theta(z_t, t, T) - \epsilon_\theta(z_t, t, \varnothing)\right] \tag{8.46}$$

其中，s 为引导尺度。当引导尺度增加时，模型可以进一步强化条件预测相对于无

条件预测的差异。但引导强度并不是越大越好。较大的引导尺度可能改善部分语义遵循表现，同时也可能降低生成多样性，或者产生颜色过饱和、局部结构异常等问题。此外，CFG 主要改变采样阶段的生成轨迹，并不能凭空补充文本编码器未能正确表达的信息。因此，条件引导应当视为一种采样控制机制，而不是直接等同于语义理解能力的提升。

ControlNet：**向预训练模型加入结构条件** ControlNet 是可控图像生成的重要代表性工作。其核心思想是，在已有预训练文本条件扩散模型的基础上，增加用于处理空间条件的可训练控制分支。例如，可以使用：边缘图、深度图、语义分割图、人体姿态图。这些条件经过控制分支处理后，将相应的结构信息注入生成网络。ControlNet 的一个重要设计，是冻结原有预训练扩散模型的相关主干权重，并通过可训练分支学习额外的控制能力。

该方法使用零初始化卷积（Zero Convolution）连接相应模块，使新增分支在训练开始时不会立即对原有网络产生任意幅度的干扰。随着训练推进，控制分支逐步学习将空间条件转换为有效的生成特征。这种设计能够在复用预训练模型生成能力的同时，增加对空间结构的控制。但 ControlNet 不能保证生成结果严格满足每一个几何约束。例如，输入边缘可能存在噪声，或者文本描述与结构条件存在冲突。

因此，结构控制能力仍然需要通过生成结果的空间一致性进行验证。

多条件生成中的冲突问题 实际生成任务可能同时具有多个控制条件。例如，用户提供一张人体姿态图，同时输入文本：“一名坐在椅子上的人物正在奔跑。”如果姿态图和文本描述明显矛盾，模型就需要决定如何综合不同约束。这说明，条件数量增加不一定提高生成效果。不同条件可能属于不同信息层次，也可能在局部发生冲突。例如，文本希望生成俯视视角，但参考深度图对应明显的平视布局。因此，多条件生成需要考虑条件优先级、约束兼容性和信息有效性。

不过，模型学习得到的条件权重并不必然代表输入条件的真实可靠性。因此，如何处理条件冲突，仍然需要在训练任务和评价标准中作出明确设计。

8.4 视频的运动与长时一致性

视频评价还应区分局部连续、对象身份保持、相机运动和长时状态。单帧质量好可以与跨帧结构错误同时出现。长时生成若依靠分段续写，应报告片段之间如何共享条件、状态和随机信息。

8.4.1 时空一致性与运动建模

时空一致性的基本含义 时空一致性（Spatiotemporal Consistency）是视频生成中的核心问题之一。它要求生成视频在空间外观和时间变化两个方面保持合理联系。空间一致性主要关注单帧中的对象形状、几何结构和场景布局是否合理。时间一致性则关注同一对象及其所在环境，是否能够在连续帧中表现出合理变化。例如，一辆汽车从道

路左侧行驶到右侧时，其位置应当随着时间改变，但汽车的主要外观特征和对象身份需要保持稳定。

因此，时间一致性并不意味着不同帧应该尽可能相似。如果整个视频完全静止，虽然帧间差异较小，却未必满足用户对运动场景的要求。更合理的目标是：应当保持的信息得到保留，需要变化的信息按照合理的时间规律发生变化。

对象身份与外观一致性 在连续视频中，同一个对象可能经历位置变化、姿态变化、视角变化和局部遮挡。模型需要维持对象身份的连续性。例如，生成一段人物行走的视频，人物的服装颜色、主要面部特征和身体外观应当在不同时间保持相对稳定。但是，当人物转身时，其可见外观又必须随着视角变化。因此，对象一致性不能简单理解为连续帧中的像素保持相同。可以概念性地表示某个对象的状态为：$s_t = (a_t, p_t, r_t)$，其中：

- a_t 表示对象外观特征；
- p_t 表示空间位置；
- r_t 表示姿态或其他动态属性。

对于同一个对象，其部分身份相关属性应当具有连续性，而位置和姿态则可以发生合理变化。这种状态表达主要用于说明对象一致性与运动变化之间的关系，并不是所有视频生成模型都显式维护这一形式的对象状态。实际模型可以通过时空注意力、潜在特征交互和其他机制隐式学习对象关系。

运动信息的表示 视频中的运动可以表现为对象位移、姿态变化、物体形变或相机视角变化。一种常见的局部运动表示是光流（Optical Flow）。设相邻两帧为：$I_t,\quad I_{t+1}$ 光流可以表示为：$u_t(p)$，其中，p 表示图像中的某个像素位置，$u_t(p)$ 表示其在两帧之间的对应位移估计。在某些条件下，可以利用光流描述帧间关系：

$$I_{t+1}(p + u_t(p)) \approx I_t(p) \tag{8.47}$$

这一关系建立在局部亮度近似保持、对应位置可见等条件之上。对于遮挡、新出现区域、反光表面和复杂光照变化，该近似可能不再成立。因此，光流可以辅助描述局部运动，但不能完整表达所有物体的三维运动或场景演化。在视频生成中，光流和其他运动特征可以用于训练监督、条件控制或结果评价，但并不意味着所有生成模型都必须显式预测光流。

时空 Patch 与 Transformer 随着 Transformer 架构的发展，视频生成开始探索将时空潜在特征转换为 Token，通过共享注意力学习空间和时间关系。例如，首先使用视频潜在编码器，将原始视频压缩为：

$$Z_V \in \mathbb{R}^{F' \times H' \times W' \times D} \tag{8.48}$$

随后，将其划分为时空 Patch，并通过相应投影形成 Token 序列。在这一结构中，每个 Token 可以对应一定范围内的时间和空间信息。Sora 的早期技术报告展示了通过

时空潜在 Patch 和扩散 Transformer 进行视频生成的技术方向，并讨论了在不同视频时长、空间分辨率和宽高比下组织训练数据的方法。

这一思想的重要意义，是将视频表示为统一的时空计算单元，使生成模型能够在更加一般的序列计算框架中处理视频内容。但时空 Patch 统一了输入表示接口，并不意味着模型自动具备正确的三维几何、物体恒存性或物理动力学知识。这些能力仍然取决于训练数据、模型结构和相应的学习目标。

时间连续性与物理合理性的区别 视频生成中还需要区分运动平滑性与物理合理性。例如，一只杯子从桌面滑落，模型可以生成连续、平滑的下落画面。但如果杯子穿过桌面，或者在没有外力的情况下突然改变运动方向，即使局部运动平滑，也不符合通常的物理约束。类似地，在人物拿取物体的过程中，模型可能生成看似自然的手部运动，却无法保持物体与手部之间正确的接触关系。因此，视频生成的物理合理性不仅涉及相邻帧是否平滑，还涉及对象状态、接触关系、运动约束和事件因果。

对于某些任务，可以利用三维几何、物体状态或物理先验增强生成约束。但生成模型的视觉效果逼真，并不能单独证明其已经学习到通用、可泛化的物理世界模型。例如，一个模型可能在常见的行走、奔跑和镜头运动场景中表现良好，却在复杂碰撞、物体形变或长时间交互中产生错误。因此，应当通过专门的动态场景和物理事件测试评价其能力，而不是仅根据展示视频的视觉效果作出判断。

8.4.2 长视频生成与计算效率

长视频生成的主要困难 当生成目标从几秒钟的视频扩展到更长时间时，模型面临的问题不仅是需要生成更多帧。视频时长增加会带来更大的数据规模。模型需要在更长的时间范围内保持对象身份与场景连续性。较长视频通常包含多个事件阶段，而不是一个简单的连续动作。例如，文本要求生成：“一名行人进入咖啡馆，在柜台点餐，随后坐到窗边阅读。”这一过程涉及人物位置变化、场景内部结构、动作顺序和物体交互。

如果模型只能够生成局部自然动作，却无法保持整个事件顺序，就可能出现人物突然回到入口、餐桌位置改变或动作重复等问题。因此，长视频生成不仅涉及局部运动建模，还需要考虑更长时间尺度上的场景和事件组织。

时空 Token 数量增长 对于采用时空 Token 的视频模型，计算成本受到视频长度和空间分辨率影响。假设每帧经过相应压缩和 Patch 划分后，得到 S 个视觉 Token。如果视频包含 F 帧，则联合 Token 数量为：$N = FS$，对于标准密集自注意力，注意力分数计算的主要规模与：$O(N^2 d) = O(F^2 S^2 d)$ 相关。

例如，若每帧保留 256 个视觉 Token，视频包含 24 帧，则总长度为：$N = 6144$，如果时间长度增加为 48 帧，而每帧 Token 数保持不变，则：$N = 12288$，在其他条件相同、使用完整密集注意力的理想化情况下，注意力位置关系数量会增加至原来的四倍。这只是注意力主要运算项的量级分析，并不代表完整视频生成模型的运行时间一定增加四倍。

实际成本还受到潜在空间压缩、局部注意力、网络深度、采样步数和硬件实现等

因素影响。

分解式时空计算 除了压缩输入，还可以调整时空注意力的计算结构。例如，对每一帧单独进行空间注意力，再在对应的时间位置之间进行时间注意力。在具有 F 个时间位置、每帧 S 个 Token 的理想化分解结构中，主要注意力计算规模可以表示为：$O(FS^2d + SF^2d)$，与全部时空位置之间直接执行完整注意力相比，这种方式可以减少部分成对交互。但分解计算可能限制信息直接交换的方式。

例如，某个对象在不同帧中发生明显位移，其对应特征不一定始终处于相同空间位置。因此，时间注意力如何建立跨帧对象对应，仍然是重要问题。实际模型还可以使用局部窗口、稀疏注意力或其他机制，在信息交换范围与计算成本之间进行权衡。

分段生成与视频延长 对于较长视频，一种直观策略是将目标时长划分为多个片段，逐段生成。例如：$V = [V_1, V_2, \ldots, V_K]$，其中，$V_k$ 表示第 k 个视频片段。一种条件分解可以表示为：

$$p_\theta(V \mid C) = \prod_{k=1}^{K} p_\theta(V_k \mid V_{<k}, C) \tag{8.49}$$

其中，C 为全局文本或其他生成条件。这种方式不要求一次性处理整个目标视频的全部时空 Token。例如，可以先生成第一段，再使用已生成内容作为下一段的条件。但是，分段生成需要解决片段之间的连续性问题。如果模型在生成后续片段时无法正确保留此前人物、场景和对象信息，就可能发生内容漂移。此外，如果前面某段生成了不合理的动作，后续片段还可能继续依赖这一错误状态，使偏差逐渐累积。

因此，分段生成降低了一次性生成的计算需求，却增加了跨片段状态保持和错误控制的要求。

长时间一致性与历史信息保留 长视频中的信息具有不同的时间尺度。例如，人物的身份和服装可能需要在整个视频中保持一致。动作姿态可能在短时间内不断变化。镜头位置和场景布局则可能随着拍摄过程逐步改变。因此，长视频生成不能仅依赖相邻帧之间的局部一致性。它还需要根据不同属性的变化规律，在适当时间范围内保留历史信息。例如，可以将生成条件概念性地分为：C_{global} 和：$C_{\text{local},k}$，

其中，全局条件描述跨片段需要保持的内容，局部条件描述当前片段需要完成的动作或事件。随后：

$$V_k \sim p_\theta\left(V_k \mid C_{\text{global}}, C_{\text{local},k}, H_{k-1}\right) \tag{8.50}$$

其中，H_{k-1} 表示此前片段提供的相关历史信息。这一公式用于说明分层条件与历史信息之间的关系，并不对应某一种具体的视频模型实现。实际系统可以采用历史帧、潜在 Token、显式对象状态或其他形式进行条件组织。因此，长时序一致性的核心问题，不只是如何生成更多帧，还包括如何决定哪些历史信息需要持续影响后续内容。

自回归视频生成与多模态统一建模 除了连续扩散和流匹配模型，离散视频 Token 也可以通过自回归方式生成。例如，视频 Token 序列为：$S_V = (v_1, v_2, \ldots, v_N)$ 模型可以学习：

$$p_\theta(V \mid C) = \prod_{i=1}^{N} p_\theta(v_i \mid v_{<i}, C) \tag{8.51}$$

其中，C 可以包含文本、图像或其他模态信息。VideoPoet 是这一技术方向的代表性工作。其使用 Decoder-only Transformer，在离散多模态 Token 框架中处理文本、图像、视频和音频等输入，并通过多任务预训练支持不同的视频生成和编辑任务。这种方式有利于将视频与其他模态组织到相对统一的序列生成体系中。

但视频离散 Token 可能具有较长的输出序列，因此实际生成成本还受到视频 Tokenizer 的压缩能力和自回归解码长度影响。因此，自回归与连续生成并不是由视频长度简单决定的两种路线，而是分别对应不同的表示和生成计算方式。

8.5 语音与音视频生成

8.5.1 文本转语音与语音合成

文本转语音的基本任务 文本转语音是指根据给定的自然语言文本，生成能够表达相应语言内容的语音波形。设输入文本为：$T = (t_1, t_2, \ldots, t_L)$ 目标语音波形为：$A = (a_1, a_2, \ldots, a_N)$，其中，$t_i$ 表示文本 Token，a_i 表示音频采样值。TTS 的生成目标可以概括为：

$$A \sim p_\theta(A \mid T) \tag{8.52}$$

其中，θ 为模型参数。语音合成不只是将文本符号转换为声音。模型还需要决定词语如何发音、音节持续多长时间、哪些位置需要停顿，以及整句话采用怎样的语调和节奏。例如，对于文本：“你今天真的要走吗？”模型需要正确表达句子中的汉字和词语，同时生成符合疑问语气的韵律变化。如果模型能够准确读出每个字，但声音缺少自然停顿和语调变化，仍然可能给人机械、不自然的听觉感受。

因此，TTS 同时涉及语言内容与声学表现两个层次。

文本前端与发音表示 语音合成系统通常需要对原始文本进行适当处理。例如，在中文 TTS 中，可能需要处理数字、日期、多音字和标点符号等问题。对于“今天是 10 月 27 日”这样的输入，模型需要确定其中数字和日期的实际读法。对于包含多音字的句子，还需要结合上下文判断发音。因此，部分系统在文本输入与声学模型之间设置文本前端，完成文本规范化、发音分析和相关语言处理。一种典型流程可以表示为：

$$T \xrightarrow{\text{文本处理}} P \xrightarrow{\text{声学模型}} M \xrightarrow{\text{声码器}} A \tag{8.53}$$

其中：

- (T) 为原始文本；
- (P) 为音素、拼音或其他发音表示；
- (M) 为相应声学特征；
- (A) 为最终语音波形。

不同 TTS 模型对文本前端的依赖程度不同。一些系统使用显式发音表示，另一些模型可以直接处理字符或文本 Token，并由神经网络学习相应的发音关系。因此，文本规范化和发音建模属于语音合成的重要组成部分，但并不要求所有模型采用相同的中间表示。

非自回归语音合成与时长建模 经典序列到序列 TTS 模型通常需要逐步预测声学帧，其生成速度可能受到自回归计算影响。此外，文本与语音之间的对齐关系也可能影响模型稳定性。例如，某些模型可能发生词语重复、漏读或异常停顿等问题。为此，研究者进一步探索非自回归语音合成。例如，FastSpeech 系列通过显式时长建模，将文本或音素表示扩展到相应的声学时间长度，再并行生成声学特征。

设输入音素序列为：$P = (p_1, \ldots, p_L)$ 每个音素对应时长：$D = (d_1, \ldots, d_L)$ 可以通过时长扩展得到与输出声学帧长度相适配的序列。FastSpeech 2 进一步引入时长、音高和能量等变化因素，使模型能够在非自回归框架中表达不同的韵律特征。非自回归不意味着模型不需要学习时间顺序。它主要改变生成时的计算依赖方式，而语音中的时长和韵律关系仍然需要建模。

韵律、音色与说话者条件 高质量语音合成不仅需要正确的语言内容，还需要表现出自然的韵律和音色。例如，同一句话可以通过改变语速、音高和停顿，呈现不同的情绪与表达重点。因此，可以在文本条件之外引入说话者和风格条件：

$$A \sim p_\theta(A \mid T, S, R) \tag{8.54}$$

其中：

- (T) 为文本；
- (S) 为说话者相关表示；
- (R) 为韵律、情绪或风格条件。

这些条件可以采用说话者嵌入、参考音频特征或其他形式。例如，在多说话者语音合成中，同一模型可以根据不同说话者嵌入，生成具有不同音色的语音。但说话者身份与情绪风格并不是同一个变量。同一说话者可以采用不同情绪和语调发声。因此，如果希望实现精细的语音控制，需要区分语言内容、说话者特征和韵律变化等不同因素。

语音合成的评价要求 语音合成通常需要同时评价内容正确性和听觉质量。一种常用的主观评价指标是平均意见分（Mean Opinion Score，MOS），通过听音者对语音自然程

度等属性进行评分。但 MOS 需要明确具体评价问题、评分尺度和听音实验协议。此外，还可以使用自动语音识别模型转写生成语音，再通过字错误率或词错误率评价内容正确性。例如，词错误率为：

$$\mathrm{WER} = \frac{S + D + I}{N} \tag{8.55}$$

其中，S、D、I 分别为替换、删除和插入错误数，N 为参考文本中的词数。对于中文任务，也常使用字符错误率（CER）等指标。但较低的识别错误率并不代表语音一定自然。声音可能发音清晰，却存在音色不自然或韵律单调等问题。因此，TTS 评价通常需要结合内容准确性、自然度、说话者相似性和生成效率等多个维度。

8.5.2 音频 Token 与自回归生成

从连续波形到离散音频 Token 原始音频通常具有较高的时间采样率。例如，一段音频即使只有几秒钟，也可能包含大量连续采样值。如果直接使用自回归模型逐采样点生成波形，通常需要较长的序列预测过程。因此，一种重要技术路线是使用神经音频编解码器，将连续波形压缩为离散 Token。设原始音频为：$A = (a_1, \ldots, a_N)$，

通过音频编码器得到：$Z_A = E_A(A)$ 再通过量化模块得到离散索引：$C = Q(Z_A)$ 最后通过音频解码器恢复：$\hat{A} = D_A(C)$，其中，(C) 为离散音频表示。SoundStream 和 EnCodec 等神经音频编解码技术，为这一生成路线提供了重要基础。音频编解码器中的 Token 并不是文字意义上的词语，而是用于表示声音潜在特征的离散索引。

因此，这些 Token 可以包含发音、音色、环境声音和其他声学信息。

多级量化与音频 Token 结构 为提高压缩表示的重建质量，一些神经音频编解码器采用残差向量量化（Residual Vector Quantization，RVQ）。在这种结构中，同一个时间位置可能由多个量化层级共同表示。可以概念性地写为：

$$C \in \{1, \ldots, K\}^{F \times Q} \tag{8.56}$$

其中：

- F 为音频编码帧数量；
- Q 为量化层级数；
- K 为单个码本的索引数量，这里为简化假设各层具有相同码本大小。

不同层级的 Token 共同用于恢复相应的声学特征。例如，较前面的码本可以提供部分主要声学信息，后续码本进一步补充量化残差。因此，生成音频 Token 不一定等同于预测单一的一维序列。实际模型可能需要处理时间维度与量化层级两个结构维度。如何组织多级 Token 的预测顺序，会直接影响音频生成模型的训练方式和推理效率。

音频自回归建模 当音频被转换为离散 Token 后，可以使用语言模型的序列预测方式学习相应分布。设音频 Token 序列为：$C = (c_1, \ldots, c_M)$，则：

$$p_\theta(C) = \prod_{i=1}^{M} p_\theta(c_i \mid c_{<i}) \tag{8.57}$$

对于条件音频生成，还可以加入文本、参考音频或其他信息：

$$p_\theta(C \mid X) = \prod_{i=1}^{M} p_\theta(c_i \mid c_{<i}, X) \tag{8.58}$$

其中，X 表示生成条件。例如，在文本条件语音生成中，X 可以包含目标文本和参考说话者音频。在音乐生成中，X 可以是自然语言描述或旋律片段。因此，离散音频 Token 能够将连续声音生成转换为条件序列建模问题。但音频生成对时间连续性和声学细节具有较高要求，因此音频 Tokenizer 的重建质量与生成模型的序列组织方式同样重要。

AudioLM：**语义与声学层次的结合** AudioLM 是基于离散音频 Token 开展生成建模的代表性工作。其基本思路，是将音频中的长期结构与局部声学细节分别组织，再通过语言建模方式进行生成。具体而言，AudioLM 结合了音频预训练模型产生的离散语义表示，以及神经音频编解码器产生的声学 Token。可以概念性地表示为：

$$A \to (C_{\text{semantic}}, C_{\text{acoustic}}) \tag{8.59}$$

其中：

- C_{semantic} 主要用于表达较高层次的内容和长期结构；
- C_{acoustic} 用于恢复更加具体的声学细节。

这种分层方式试图兼顾音频的长期一致性与局部音质。例如，语音不仅需要保持语言结构的连贯性，还需要在连续生成中维持说话者音色与韵律特点。AudioLM 展示了根据短音频提示生成后续语音或音乐的能力。但 AudioLM 原始研究主要围绕音频提示与音频续写等任务展开，并不应将其简单描述为典型的文本输入语音合成模型。其重要意义，是将离散音频表示与语言模型生成方式结合起来，展示了层次化音频 Token 在声音生成中的作用。

VALL-E：**神经编解码语言模型与零样本语音合成** VALL-E 将神经音频编解码器与文本条件语言建模结合，探索零样本说话者适配的语音合成。其基本思路是，将 TTS 视为根据文本和声学提示生成离散音频 Token 的任务。可以表示为：

$$C_{\text{target}} \sim p_\theta(C_{\text{target}} \mid T, C_{\text{prompt}}) \tag{8.60}$$

其中：

- T 为需要合成的文本；
- C_{prompt} 为参考音频的编解码 Token；

- C_{target} 为目标语音 Token。

VALL-E 采用自回归与非自回归相结合的方式，对多级音频编解码 Token 进行生成。其中，自回归模块主要负责相应初始码本层级的序列预测，后续非自回归模块进一步生成其他量化层级。其代表性特点是，可以根据较短的参考语音，生成具有相似说话者特征的新语音，而不必针对每个新说话者单独进行完整模型微调。但这种能力来自训练数据和声学提示条件的共同作用，不意味着模型可以在所有语言、噪声环境和说话风格下稳定保持相同的语音质量。

此外，零样本语音合成需要注意说话者身份授权与音频内容来源，避免未经许可复现可识别个体的声音。

8.5.3 音频生成与扩散式建模

连续音频生成的研究动机 离散 Token 提供了将声音转换为序列建模任务的途径，但另一条重要技术路线是直接对连续声学表示进行生成建模。例如，音频可以表示为波形、频谱或连续潜在特征。对于这些连续变量，可以采用扩散模型或流匹配等生成方法，学习其相应的数据分布。与离散 Token 预测相比，连续生成不需要将全部生成变量先转化为码本索引。但这并不意味着连续生成一定不使用压缩。实际模型同样可能在连续潜在空间中开展生成。因此，音频生成需要区分三个层次：

原始波形空间。Mel 频谱等声学特征空间。由神经编码器学习得到的连续潜在空间。不同生成方法可以作用于其中不同的表示层次。

扩散式波形生成 扩散模型可以通过向真实音频添加噪声，并学习相应的恢复过程，实现波形生成。设真实波形为：a_0 构造带噪音频：

$$a_t = \alpha_t a_0 + \sigma_t \epsilon \tag{8.61}$$

其中：

$$\epsilon \sim \mathcal{N}(0, I) \tag{8.62}$$

模型预测相应噪声：

$$\hat{\epsilon} = \epsilon_\theta(a_t, t, C) \tag{8.63}$$

其中，C 为可选条件。在生成过程中，模型从噪声出发，通过相应的去噪采样过程得到音频波形。DiffWave 是扩散式音频生成的代表性工作。它研究了基于扩散模型的波形生成，并支持无条件音频生成以及根据 Mel 频谱进行条件波形恢复等任务。其中，根据 Mel 频谱生成波形，可以视为一种扩散式神经声码器。这一研究说明，扩散生成可以直接作用于连续声音波形，而不必先将声音转换为离散音频 Token。

潜在音频扩散 原始音频波形通常具有较高维度，因此直接在波形空间进行生成可能带来较大的计算成本。为了降低生成难度，可以先将音频转换到连续潜在空间，再使用

扩散模型进行生成。例如：$z_0 = E_A(A)$，其中，E_A 为音频潜在编码器。模型在潜在空间中学习：

$$p_\theta(z_0 \mid C) \tag{8.64}$$

并通过相应的扩散过程生成潜在音频表示。最终使用解码器恢复声音：$\hat{A} = D_A(\hat{z}_0)$，这种方式与潜在图像扩散具有相似思想，能够在适当压缩数据的基础上降低生成网络的计算负担。但音频潜在表示需要保留时间结构、频谱特征和音色等信息，因此其编码器与图像编码器的设计并不完全相同。

AudioLDM：**文本条件通用音频生成** AudioLDM 是潜在音频扩散的重要代表性工作。其目标不是只生成语音，而是根据自然语言描述合成更加一般的声音。例如：“雨水持续落在窗户上。”“远处传来火车经过的声音。”“室内响起一阵短促的掌声。”AudioLDM 采用潜在扩散框架，并利用音频—文本对比预训练模型 CLAP 建立条件表示。

其研究采用了一种值得注意的训练策略：利用 CLAP 的音频嵌入为潜在扩散模型提供训练条件，并利用 CLAP 学习到的音频—文本对应关系，在推理时使用文本嵌入进行生成控制。因此，AudioLDM 的核心不仅是将扩散模型应用到音频，还包括如何利用预训练跨模态语义空间组织文本与音频之间的生成条件。CLAP 特征主要用于表达跨模态语义关联，并不能唯一指定目标音频的全部声学细节。

例如，同一句“雨声”描述可以对应不同强度、节奏和空间环境的降雨声音。因此，音频生成模型仍然需要根据训练数据中的声学分布，生成文本未明确指定的具体声音特征。

从扩散模型到流匹配 除传统扩散式建模外，流匹配也逐渐被用于连续音频生成。其基本思想是在基础噪声分布与目标音频潜在分布之间建立概率路径，并训练神经网络预测相应的速度场。例如，设初始噪声为：$z_0 \sim p_0$ 目标音频潜在表示为：$z_1 \sim p_{\text{data}}$，

在简单直线插值设定下：$z_\tau = (1-\tau)z_0 + \tau z_1$ 可以训练：$v_\theta(z_\tau, \tau, C)$ 预测相应速度。生成时，从基础分布采样，并通过数值积分得到音频潜在表示，再利用解码器恢复波形。这一方式与前面介绍的流匹配理论一致，但音频生成还需考虑声音的时间结构和声学连续性。

MMAudio 将流匹配用于视频条件音频生成，并通过多模态联合训练学习相应的声音分布，是这一技术路线在音视频生成中的代表性应用。

8.5.4 音视频联合生成

音视频联合生成的基本问题 在真实环境中，许多声音与视觉事件具有密切联系。例如，汽车驶过时可能产生发动机声音；玻璃杯落地时可能出现撞击声；人物说话时，语音内容与嘴部动作具有一定的时间对应关系。因此，音视频联合生成不仅需要产生高质量的声音和画面，还需要保证二者在内容和时间上的合理联系。设文本条件为 T，视频为 V，音频为 A。完整的文本条件音视频生成可以表示为：

$$(V, A) \sim p_\theta(V, A \mid T) \tag{8.65}$$

其中，目标是同时生成具有相互关系的视频与音频。但实际研究中还存在另一类重要任务：已知视频后，为其生成相应声音。这可以写为：

$$A \sim p_\theta(A \mid V, T) \tag{8.66}$$

其中，T 为可选文本条件。后者属于视频条件音频生成，并不等同于同时生成视频与音频。两类任务都涉及音视频对应，但它们的输入、生成范围和模型要求存在区别。

视频条件音频生成 视频条件音频生成（Video-to-Audio Generation，V2A）要求模型根据已有视频内容生成相应声音。例如，输入一段没有声音的玻璃杯落地视频，模型需要生成与碰撞事件相对应的声音。这类任务不仅要求声音类别正确，还需要与视频中的事件时间保持对应。例如，玻璃杯在第 3 秒接触地面，那么主要撞击声也应当出现在相应时间附近。如果模型生成了自然的玻璃碰撞声，但声音提前或延后较长时间，仍然存在明显的视听不一致。

因此，视频条件音频生成的评价至少包含两个不同问题。模型是否生成了语义上合适的声音。生成声音是否在时间上与可见事件对应。二者缺一不可。

音视频时间对齐 音频与视频具有不同的采样方式。视频通常表示为帧序列：

$$V = \{(I_j, t_j^V)\}_{j=1}^{F} \tag{8.67}$$

音频则可以表示为连续波形或较高时间分辨率的声学特征：

$$A = \{(a_i, t_i^A)\}_{i=1}^{N} \tag{8.68}$$

由于两种模态的时间分辨率不同，不能简单地假设一个视频 Token 对应一个音频 Token。模型需要利用时间戳、时间窗口或学习式同步机制建立对应关系。例如，可以先提取视频特征：$H_V = f_V(V)$ 再根据视频时间信息，将其提供给音频生成网络：

$$A \sim p_\theta(A \mid H_V) \tag{8.69}$$

但具有相同时间戳的声音与画面内容，不一定具有正确的事件对应关系。例如，视频中同时存在人物说话和车辆经过，某个声音究竟来自人物还是车辆，还需要根据实际事件内容判断。因此，时间同步必须与语义对应共同考虑。

MMAudio：**视频条件与声音同步生成** MMAudio 是视频条件音频生成的重要代表性工作。其目标是根据视频和可选文本条件，生成具有较高音质并与画面同步的声音。该研究采用多模态联合训练，同时利用视频—音频和文本—音频等数据，使模型能够学习声音内容及其条件关系。在模型机制上，MMAudio 使用流匹配目标开展音频生成，并设计相应的条件同步模块，使视频信息能够以帧级时间关系参与音频潜在表示的生成。

其技术意义在于，将两个原本容易分离的问题结合起来：根据视觉内容判断应当出现什么声音，以及根据视频中的事件时间确定声音何时出现。例如，模型不仅需要知道视频包含鼓掌，还需要使掌声与可见的拍手动作合理对应。但即使模型可以生成同步声音，也不意味着它已经恢复了真实环境中唯一正确的物理波形。例如，相同的碰撞动作可能因为物体材料、录制距离和环境混响不同而产生多种合理声音。

因此，视频条件音频生成通常学习的是与视觉条件相符的声音分布，而不是从视频确定性恢复真实录音。

音视频生成的主要局限 音视频联合生成仍然面临若干基本挑战。视觉条件通常无法唯一确定声音。例如，一辆汽车从远处经过，其实际声音还受到发动机状态、道路材料和录制位置等因素影响。因此，生成模型可能产生合理声音，却未必与某个真实录音完全一致。音视频时间对齐具有较高要求，尤其对于短促碰撞、口型同步和快速连续动作，较小的时间误差也可能影响整体感受。复杂场景可能包含多个声音源。例如，在街道场景中，车辆、行人、风声和远处环境噪声可能同时存在。

模型不仅需要生成相应声音，还需要合理组合这些声音的时间、响度和空间关系。长时间音视频生成还需要维持事件连续性。例如，某辆汽车驶离画面后，其声音应当随场景关系发生合理变化，而不应在后续片段中毫无依据地突然出现。因此，音视频联合生成不仅涉及跨模态语义融合，还涉及声学场景建模、时间同步和连续事件生成。

习题与讨论

1. 为自回归、噪声预测扩散和流匹配各列出网络输入与监督量，说明三者训练目标不能直接数值横比的原因。
2. 潜在扩散的输出错误可能来自自编码器或生成网络。设计一个先测重建、再测生成的诊断流程。
3. 若模型正确生成了所有对象，却把颜色绑定给错误对象，提出两种比整体图文相似度更直接的检查。
4. 视频模型在分段边界出现对象位置跳变。区分增加片段长度、跨段状态和改变采样器可能解决的问题。

文献导读 DDPM 用于核对前向噪声与反向预测，潜在扩散用于核对自编码器接口，Flow Matching 用于核对路径与速度场。[20, 21, 22]

第三部分

数据与训练

第九章　多模态数据与监督设计

数据集是一组带来源、对应规则和监督边界的观测。文件规模不能替代有效信息量，正确的文本也可能与当前图像无关。本章从一条样本如何进入训练出发，检查配对、标注、合成和混合中的具体决策。

学习目标　设计可追溯的样本结构；区分完整性与匹配正确性；按实体、来源和时间隔离数据；解释采样权重与 Token 权重的区别。

9.1　样本结构与监督粒度

一个多模态样本至少应记录媒体标识与版本、时间或空间范围、问题、目标及监督掩码、来源分组、使用许可和质量标记。媒体内容与标注最好分层存储：修改提示模板不应丢失原始媒体的版本，重新编码媒体也不应悄悄改变标注坐标。

9.1.1　图文配对数据

图文配对数据的基本形式　图文配对数据（Image-Text Paired Data）是多模态预训练与视觉语言学习的重要数据类型。其基本形式是将图像与相应的自然语言文本建立联系，例如：$D_i = (I_i, T_i)$，其中：

- (I_i) 表示图像；
- (T_i) 表示与该图像相关的文本。

例如，一张包含草地和奔跑犬只的照片，可以对应文本：

“一只狗正在草地上奔跑。”这种数据可以用于学习图像与语言之间的语义关系。对于图文对比学习，模型可以利用配对关系，使相关图像和文本在共享表示空间中具有较高相似度。对于图像描述任务，模型可以根据图像生成文字描述。对于文本生成图像，图文数据则能够提供从文本条件到视觉内容的训练监督。因此，图文配对是多种视觉语言理解与生成任务共同使用的数据基础。

一张图像与多条文本的关联　实际数据集中的图文关系不一定是一对一。例如，同一张图像可以包含多个对象和事件，不同描述者可能关注不同内容。对于一张街道照片，可以出现以下描述：“几名行人正在穿过马路。”“一辆公交车停在道路旁。”“城市街道上有车辆和行人。”这三条描述都可能正确，但分别关注不同的场景信息。因此，更一般的图文数据组织形式为：

$$D_i = (I_i, T_{i1}, T_{i2}, \ldots, T_{iK_i}) \tag{9.1}$$

其中，K_i 表示第 i 张图像对应的文本描述数量。COCO Captions 是典型代表。它在图像描述任务中，为同一图像提供多条人工撰写的自然语言描述。这种一对多关系使模型能够学习同一视觉内容的不同语言表达。但多条描述之间不一定具有完全相同的语义范围。某些描述可能强调对象，另一些描述则关注动作或场景关系。因此，数据组织应当保留文本与原始图像之间的关联，而不宜简单将所有描述合并为一条未经区分的长文本。

图文配对的不同语义粒度 图文配对可以发生在不同层次。一种是图像级描述，文本概括整幅图像的主要内容。另一种是区域级描述，文本对应图像中的某个对象或局部区域。例如：整图描述：“一名男子站在厨房里。”区域描述：“男子手中的白色杯子。”对于区域级数据，除了文本内容，还需要提供相应空间范围，例如边界框或分割掩码。可以表示为：$D_{ij} = (I_i, B_{ij}, T_{ij})$，

其中，B_{ij} 表示与文本 T_{ij} 对应的图像区域。这种数据可以用于目标定位、区域描述和细粒度视觉语言对齐等任务。因此，在组织图文数据时，需要明确文本描述究竟对应整个图像、某个区域，还是图像中的多个对象及其关系。不同粒度的标注不能在缺少说明的情况下视为等价。

图文关联的强弱差异 从关联程度看，可以将图文数据区分为较强配对和较弱关联两种情况。较强配对通常具有明确的人工标注或经过验证的对应关系。较弱关联则可能仅能说明图像与文本来自同一网页、文章或相关上下文。例如：(I, T, ρ)，其中，ρ 可以用于记录配对来源、可信度或其他相关质量信息。但 ρ 并不是所有图文数据集天然具有的标准字段。如果需要建立这样的质量标识，应当根据明确的标注或评价规则定义。

对于对比学习任务，较弱关联可能引入噪声配对。对于文本生成图像任务，错误描述可能使模型学习到不准确的对象和属性关系。因此，图文配对的数据质量不仅取决于图像与文本是否同时存在，还取决于两者之间的语义联系是否符合目标训练任务。

图文数据的组织边界 图文数据中还可能存在同一图像对应多个版本、裁剪图或重复来源的情况。例如，一张原始照片可能同时具有不同分辨率的副本。如果这些图像进入不同的数据划分，就可能造成训练集与测试集之间的内容泄漏。此外，同一图像具有多条描述时，不能将其中部分描述放入训练集、另一些描述放入测试集，就认为两个集合在图像层面完全独立。因此，图文数据集需要明确数据划分的基本单位究竟是描述、图像还是原始来源。

对于希望评价模型对未见图像泛化能力的任务，通常更应关注图像及其来源之间的独立性。具体的数据去重和划分方法，将在后续数据处理章节进一步讨论。

9.1.2 视频与文本描述数据

视频文本数据的基本形式 视频与文本描述数据（Video-Text Data）在图文配对基础上进一步引入了时间维度。设视频为：$V = (I_1, I_2, \ldots, I_F)$，其中，$I_t$ 表示视频中的第 t 帧。其关联文本为 T。则一个简单的视频文本样本可以表示为：$D_i = (V_i, T_i)$，例如，一段视频显示人物从桌面拿起水杯，可以对应描述：“一名男子拿起桌上的杯子。”

与图像描述不同，视频描述需要表达动作和事件随时间发生的变化。因此，视频文本数据的关联不仅涉及对象和场景，还可能涉及动作顺序、时间区间和事件之间的联系。

完整视频与视频片段 视频数据的一个重要特点是，同一段原始录制可能包含多个不同事件。例如，一段十分钟的视频可能依次包含：人物进入房间、打开冰箱、取出饮料、坐在桌边。如果仅使用一条整体描述，可能无法完整表达各个局部事件。因此，视频语言数据集通常可以采用视频片段（Clip）作为基本样本单位。设原始视频为：V_i，其中某个时间区间为：$[t_s, t_e]$，

则片段可以表示为：$C_{ij} = (V_i, t_s, t_e)$ 相应文本标注为：T_{ij}，因此，一条视频片段描述记录可以写为：

$$D_{ij} = (V_i, t_s, t_e, T_{ij}) \tag{9.2}$$

这种组织方式能够建立较明确的事件与时间关系。时间区间应当具有明确的时间单位及参照，例如以原始视频起点为零的秒数，而不能只保存未经说明的数字。

视频描述的时间粒度 视频文本描述可以具有不同粒度。一种是视频级描述，用一句或几句话概括整段视频。另一种是片段级描述，为某段连续时间区间提供对应文本。还有一种是细粒度事件描述，将长视频划分为多个具有明确时间边界的动作或事件。例如：

$$\mathcal{A}_i = (t_s^{(j)}, t_e^{(j)}, T_j)_{j=1}^{K} \tag{9.3}$$

其中，每个三元组对应一个时间区间及其文本描述。这种形式可以用于时间定位、密集视频描述和事件理解等任务。例如，Ego4D 提供了与第一视角活动相关的叙述及多种时间标注任务，为研究日常行为与长视频理解提供了数据基础。因此，视频文本数据应当保留文本标注的时间粒度，而不宜把所有描述直接附加到整个视频上。

MSR-VTT：**视频片段与多条自然语言描述** MSR-VTT 是视频语言研究中的经典数据集。它收集不同类别的网络视频片段，并为每个片段提供多条自然语言描述。这种设计与图像描述数据中的一对多关系具有一定相似性。例如，一个视频片段可以对应：“一名男子正在打篮球。”“一个人在球场上投篮。”“篮球运动员尝试将球投入篮筐。”这些描述可能指向同一段可见动作，但在措辞和信息侧重点上存在差异。因此，可以采用：

$$D_i = (V_i, T_{i1}, \ldots, T_{iK_i}) \tag{9.4}$$

组织这类数据。MSR-VTT 原始论文报告了约一万个视频片段，以及约二十万个片段—句子关联。这类数据可以支持视频描述、文本检索视频和视频—文本语义对齐等任务。但不同文本描述不一定覆盖片段中的全部动作。对某个片段没有提及的对象或事件，也不能自动认定不存在。

HowTo100M：**旁白与视频的弱对齐** 除了人工视频描述，视频还可以通过已有语音旁白和自动转写获取语言信息。HowTo100M 是这一思路的代表性研究。该数据集从大量具有解说内容的教学视频中提取视频片段，并利用旁白转写文本建立视频与语言之间的联系。例如，在烹饪教学视频中，讲解者可能说：“现在把切好的洋葱放进锅里。”这段文字可以与相应的视频时间区间建立关联。但旁白文本与实际画面并不一定严格同步。

例如，讲解者可能在动作发生之前提前说明下一步，也可能在动作完成后进行总结。因此，由旁白或自动语音识别得到的文本，不应直接等同于人工核验的视觉事件描述。HowTo100M 的重要价值之一，是展示了如何利用现有音视频内容中的语言信号，规模化构建弱监督视频文本学习数据。但这种数据通常需要考虑旁白质量、时间对应误差及非视觉语言内容等因素。

9.1.3 音频、语音与文本数据

音频文本数据的不同任务形式 音频、语音与文本数据是语音识别、音频理解和声音生成等任务的重要基础。但与图文数据相比，音频与文本之间可能存在不同的对应方式。例如，一段语音可以对应其语言内容转写。一段环境声音则可以对应描述声音事件的自然语言。一段音频还可以同时具有说话者身份、情绪标签和事件时间区间等信息。因此，需要区分语音转写数据（Speech Transcription Data）和音频描述数据（Audio Captioning Data）。

前者主要记录说了什么。后者主要描述听到了什么声音或发生了什么声学事件。二者虽然都包含音频与文本，但其语义关系和任务监督不同。

语音转写数据 语音转写数据的基本形式为：$D_i = (A_i, T_i)$，其中：

- (A_i) 为语音片段；
- (T_i) 为对应的语言转写。

例如，音频中说：

“请打开会议室的灯。”其转写文本也应当表达相应语言内容。这类数据主要用于自动语音识别、语音语言建模和相关任务。LibriSpeech、Common Voice 等是具有代表性的语音数据资源。例如，Common Voice 通过社区参与的方式收集不同语言的语音及相关文本信息，具有多语言语音数据建设的特点。

不同版本和子类型的 Common Voice 数据在采集方式、字段和许可要求上可能存在差异。因此，在引用和使用时应当明确版本及具体数据类型。

环境声音与自然语言描述 并非所有音频都包含语音。例如，一段录音可能主要包含：雨声、车辆鸣笛、风吹树叶的声音、玻璃碰撞声。对于这些声音，可以通过自然语言描述记录主要声学事件。例如：“一辆汽车快速驶过，随后传来短促的鸣笛声。”这类数据可以表示为：$D_i = (A_i, C_i)$，其中，C_i 为音频描述文本，而不是语音转写。AudioCaps 和 Clotho 是音频描述研究中的代表性数据集。

其中，Clotho 的设计强调使用多条人工自然语言描述表达同一段声音内容。因此，与图像描述类似，一段音频也可以对应多条不同但合理的文本。声音描述可能只包含主要声学事件，并不完整记录所有背景声音。因此，数据组织需要区分音频事件标注、自然语言描述和实际语言转写。

音频事件的时间标注 对于声音事件检测，模型可能不仅需要判断某种声音是否出现，还需要判断声音在什么时间发生。例如，一段十秒音频中，前半段是背景交通噪声，第七秒附近出现鸣笛声。可以通过时间区间表示：

$$D_i = (t_s^{(j)}, t_e^{(j)}, c_j) \tag{9.5}$$

其中，c_j 为声音事件类别。这类标注与整段音频对应一条文本描述不同。例如，整段描述可能只写“道路交通环境中传来汽车鸣笛”，而事件时间标注则进一步指出鸣笛开始和结束的时间。因此，音频数据的标注粒度需要根据识别、检测、检索或生成等任务确定。

音频生成与文本条件 对于文本转语音，音频和文本可以作为输入与生成目标之间的条件对应。例如：

$$A \sim p_\theta(A \mid T) \tag{9.6}$$

其中，T 为目标语音内容。对于文本条件环境音生成：

$$A \sim p_\theta(A \mid C) \tag{9.7}$$

其中，C 为声音事件描述。两种任务在形式上都属于文本条件音频生成，但文本与音频之间的关系不同。在 TTS 中，文本通常需要被准确表达为语音内容。在环境音生成中，文本描述更多提供声音类别、事件和场景条件，而不唯一确定某一段具体波形。因此，同样是音频—文本配对，实际训练目标可能存在明显差异。这种区别需要在数据字段和任务标识中得到体现。

9.1.4 多轮交互与多模态指令数据

从配对样本到指令样本 传统图文或音频文本数据通常描述不同模态之间的对应关系。例如，一张图像对应一句描述，一段语音对应一条转写。但多模态助手还需要根据用户指令完成不同任务。例如：“描述这张图像。”“这张图像中的汽车是什么颜色？”“根据视频说明人物动作的先后顺序。”“请比较这两张图片之间的不同。”这些任务不能仅通过原始图文配对关系完整表达。

因此，需要引入多模态指令数据（Multimodal Instruction Data），明确记录输入内容、任务要求及期望输出。例如：$D_i = (X_i, Q_i, Y_i)$，其中：

- (X_i) 为多模态输入；
- (Q_i) 为用户指令；

- (Y_i) 为目标回答或输出。

这一形式使模型能够学习根据不同指令使用同一模态输入完成多种任务。

图像指令与图像描述的区别 图像描述数据通常要求模型概括图像内容。例如：图像：道路上的一辆汽车。描述：“一辆汽车正在城市道路上行驶。”但多模态指令数据可以围绕同一图像构造不同任务。例如：指令一：“描述图像中的主要对象。”指令二：“图像中汽车位于什么位置?”指令三：“行人与车辆之间有什么空间关系?”因此，同一张图像可以支持多个不同的指令—回答样本。这些样本不一定对应完全相同的监督目标。

例如，有些问题要求对象识别，有些问题要求空间推理，还有些问题需要整合多个视觉线索。因此，指令数据组织需要保留任务要求，而不能只保存图像和最终回答。

多轮对话的基本数据结构 多轮交互数据与单轮指令数据的一个重要区别，是同一任务可能包含多个连续消息。例如：用户：“这张照片里有什么?”助手：“有一张桌子和两个杯子。”用户：“左边的杯子是什么颜色?”助手：“左边的杯子是白色的。”用户：“请只描述右边的杯子。”助手：“右边的是一个蓝色杯子。”在这一过程中，后续问题可能依赖前面的对话上下文。因此，数据集不能只保存最后一句问题和回答，还需要保留相应的消息顺序。

可以表示为：$C_i = (m_1, m_2, \ldots, m_K)$，其中，每条消息可以进一步包含：

$$m_k = (\text{role}_k, \text{content}_k) \tag{9.8}$$

其中，role 用于标记用户、助手或其他消息角色，content 表示相应内容。如果消息包含图像、音频或视频，还需要进一步关联对应的媒体数据。

多模态消息的组织方式 与纯文本对话不同，多模态消息可能同时包含文本和非文本内容。例如，一条用户消息可能包含文字：“请说明视频中发生了什么。”以及一个视频文件引用。也可能在多个文本片段之间交错包含图像或音频。因此，可以将消息内容表示为有序片段集合：

$$\text{content} = (c_1, c_2, \ldots, c_J) \tag{9.9}$$

其中，每个片段可以具有相应类型。例如：文本片段、图像引用、音频引用、视频引用。时间位置或其他结构化信息。这种组织方式允许模型训练数据保留多模态输入的实际顺序，而不是把全部媒体信息简单放在对话末尾。例如，用户可能先提供一张图像，随后补充另一张图像，并要求比较二者。此时，输入顺序和图像引用关系都可能影响任务含义。

多轮数据的训练与评价边界 在多轮对话数据中，不同消息之间可能存在较强依赖。例如，一个会话的后半部分可能引用前面已经出现的对象、图像和答案。如果在构建训练集与测试集时，将同一会话中的不同回合分别放入两个集合，就可能形成明显的信息重叠。类似地，如果多个会话使用同一张图像或同一个视频片段，也需要考虑原始

媒体来源之间的关系。因此，多轮指令数据的划分应当结合完整会话和原始媒体来源，而不是仅按照单条消息随机划分。

此外，评价模型多轮能力时，还需要明确是否提供完整历史上下文，以及模型是否允许读取此前的图像或视频内容。这些设置会直接影响任务难度和结果可比性。因此，多轮多模态数据的基本单位不仅可能是单条消息，还可能是包含完整上下文的会话记录。

9.2 来源、去重与质量控制

去重应在训练、验证和测试的整体范围内设计。按文件名去重无法阻止近重复图片、同一视频相邻片段或同一文档不同页面跨集合泄漏。隔离单位应与研究主张一致，例如按实体、原视频或文档来源分组。

9.2.1 数据来源与采集策略

数据来源与任务目标 多模态数据采集的首要问题，是明确数据将服务于什么研究任务。不同任务对数据类型、标注精度和内容分布的要求存在明显差异。例如，图文对比预训练可以利用规模较大的弱关联图文数据，通过跨模态对比目标学习语义关系。但如果任务要求精确判断对象数量或空间位置，仅有粗略图像描述的弱配对数据可能不足以提供相应监督。

类似地，通用语音理解需要覆盖不同语言、说话者和声音环境，而实时机器人感知则可能更关注传感器时间连续性、空间标定和动作相关信息。因此，数据采集不宜仅以获得最大样本数量为目标，而应首先确定所需的模态组合、任务监督以及目标应用场景。可以概念性地表示为：

$$\mathcal{D} = {D_i}_{i=1}^{N} \tag{9.10}$$

其中，每个 D_i 表示一个有效的多模态样本。但数据集的价值并不能简单地由 N 决定，还需要考虑不同样本所覆盖的场景、对象、时间变化和模态关系。例如，对于目标检测任务，大量来自同一路段、相近时间和相似天气的图像，未必能够充分代表不同道路环境。因此，数据来源与目标任务之间的适配程度，是制定采集策略的重要依据。

多模态数据的主要来源 多模态数据可以来自互联网内容、人工组织的采集活动、真实设备记录及模拟环境等多种渠道。互联网数据通常具有规模较大、内容丰富和采集成本相对较低的特点。例如，网页图片及其附近文字可以用于构建图文数据，在线视频及其字幕能够形成视频语言数据。人工采集则可以围绕明确任务开展。例如，为研究复杂光照条件下的多传感器感知，可以按照预定路线和天气条件采集 RGB、事件相机及其他传感器数据。

对于机器人和具身智能任务，还可以通过遥操作、人工示教或自主执行，记录观测与动作序列。此外，仿真平台能够生成具有特定场景配置的数据，并在一定条件下

提供相应的状态、深度、分割或动作标注。但仿真数据与真实观测在图像外观、传感器噪声和物理动态方面可能存在分布差异。因此，仿真数据的标注便利性并不意味着其能够无条件替代真实数据。不同来源各有适用条件，数据采集应根据任务目标及可用资源进行选择。

主动采集与覆盖范围设计 对于真实传感器和专业任务数据，采集策略通常可以更加主动地设计。例如，自动驾驶数据可以从道路类型、光照、天气、交通密度和运动速度等方面制定覆盖要求。机器人数据可以围绕物体类别、任务类型、初始位置、抓取方式和操作结果等因素组织采集。这种方式能够使数据集不只是包含大量记录，还能够覆盖研究任务关心的变化条件。

例如，如果研究目标是提升视觉感知模型在夜间或高速运动条件下的鲁棒性，那么主动采集这些困难场景可能比继续增加大量正常日间图像更有价值。因此，可以将采集覆盖问题概念性地理解为：

$$\max_{\mathcal{D}} \text{Coverage}\left(\mathcal{D}, \Omega_{\text{target}}\right) \tag{9.11}$$

其中，Ω_{target} 表示目标任务关注的场景和条件集合。这里的覆盖函数需要根据具体任务定义，不能将这一公式直接视为通用的数据采集优化方法。但它强调了一个重要原则：数据采集应当关注目标条件是否得到覆盖，而不仅是样本总量是否增加。

自然分布与目标分布 数据来源决定了训练数据的分布，而采集条件可能与最终应用环境存在明显差异。例如，网络图像中通常包含较多经过选择、构图较好或视觉内容突出的照片。而机器人摄像头记录的真实工作场景，可能包含大量普通背景、局部遮挡、运动模糊和不完整观测。因此，网络图文模型在通用语义任务中表现良好，不意味着它能够直接适应全部真实传感器条件。类似地，某个驾驶数据集如果主要来自特定地区、道路和天气，其数据分布可能不能完整代表其他城市或气候环境。

因此，数据采集需要考虑训练分布：$p_{\text{train}}(x)$，与目标应用分布：$p_{\text{target}}(x)$ 之间的关系。如果二者存在明显差异：

$$p_{\text{train}}(x) \neq p_{\text{target}}(x) \tag{9.12}$$

模型就可能面临分布偏移。但完全匹配目标应用分布通常并不容易，也不是所有预训练任务的必要条件。通用预训练可能主动覆盖多种数据来源，以获得更加广泛的表示能力。因此，采集策略应结合通用性与目标任务需要进行设计。

数据来源记录与使用条件 多模态数据的来源还涉及可追溯性和使用权限。例如，公开可访问的网页内容并不必然具有可以任意复制、再发布或训练使用的许可条件。类似地，某些音视频数据可能包含个人信息、受保护的作品或其他需要特别处理的内容。因此，数据工程不仅需要记录数据来自什么位置，还应当明确数据的许可信息、预期用途以及必要的使用限制。对于来源复杂的数据集，可以采用结构化记录方式保存来源、采集时间、处理过程与版本信息。

Datasheets for Datasets 和 Data Cards 等研究提供了数据集文档化的系统思路，强调记录数据集动机、组成、采集、用途和限制等信息。这类文档不仅有利于数据治理，也能够帮助研究者判断数据是否适合新的训练或评价任务。因此，数据来源记录应当被视为数据集设计的一部分，而不是训练完成后才补充的说明材料。

9.2.2 去重、过滤与异常样本处理

数据清洗的目标 多模态数据清洗（Data Cleaning）是对原始数据进行检查、筛选与必要修正，使其更加符合目标任务要求的过程。其中，常见问题包括重复样本、损坏文件、错误标注、模态不对应，以及缺少必要元数据等。例如，一张图像可能存在多个完全相同的文件副本。一段视频可能由于下载或解码异常而缺少部分画面。某条图文记录中的文字可能描述了完全不同的图像。这些问题需要采取不同的处理方式。

但数据清洗的目标不应简单理解为删除所有看起来不够理想的样本。例如，夜间低照度图像、运动模糊图像和被遮挡的目标，可能正是鲁棒感知任务需要学习的内容。因此，需要区分数据错误与真实困难场景。

精确重复与近似重复 数据重复可以发生在不同层次。精确重复（Exact Duplicate）是指两个样本在选定的比较对象上具有完全相同的内容。例如，两个图像文件具有相同的字节序列，或者两条文本记录经过统一规范化后完全一致。近似重复（Near Duplicate）则指不同样本具有高度相似的内容，但并不完全相同。例如，同一张照片可能经过缩放、裁剪或重新压缩后再次出现。一条文本也可能仅修改少量标点或词语。

此外，还存在语义近重复（Semantic Near-Duplicate）。例如，不同图像或文本虽然不是相同内容，却表达高度相似的场景与语义。因此，去重需要首先明确比较的是文件内容、视觉外观、文本形式还是语义表示。

基于感知哈希的图像近重复检测 为了识别经过一定处理后仍然具有相似视觉内容的图像，可以使用感知哈希（Perceptual Hash）。感知哈希通常将图像转换为紧凑的视觉摘要，使部分缩放、压缩或轻微视觉变化后的图像仍然具有接近的哈希结果。例如，可以使用 Hamming 距离：

$$d_H(h_i, h_j) = \sum_{k=1}^{B} \mathbf{1}[h_{ik} \neq h_{jk}] \tag{9.13}$$

其中，B 为二进制哈希长度。如果两个图像的感知哈希距离较小，可以将它们识别为潜在的视觉近重复样本。但感知哈希不是严格的语义识别模型。例如，内容结构较相似的不同图像，可能得到较接近的哈希。另一方面，经过较大裁剪或明显视角变化的同一对象图像，也可能不再具有接近的感知哈希。因此，实际去重可以采用多阶段方法，先筛选候选，再进行更加精确的图像内容比较。

文本近重复与 MinHash 文本数据中的重复也不一定表现为完全相同的字符串。例如：“一辆汽车停在路边。”与：“一辆车停在道路旁边。”两条文本语义相近，但字面内容不同。对于具有较多公共词语或片段的文本，可以通过 n-gram 或 Shingle 集

合衡量相似性。设两段文本的 Shingle 集合分别为：S_i, S_j 可以计算 Jaccard 相似度：

$$J(S_i, S_j) = \frac{|S_i \cap S_j|}{|S_i \cup S_j|} \tag{9.14}$$

MinHash 等方法可以用于近似估计集合相似性，并降低大规模文本重复检测的计算成本。但字符串相似与语义相似并不完全相同。例如，两个句子只有一个表示否定的词语不同，也可能具有较高的词面相似度，却表达相反的事实。因此，文本去重不能只根据相似度阈值机械处理全部样本，尤其需要保护对目标任务具有重要差异的文本。

语义近重复与嵌入检索 对于规模更大的多模态数据，可以利用预训练模型提取语义嵌入，检测内容相近的样本。例如，设图像或文本嵌入为：$z_i = f(D_i)$ 可以计算余弦相似度：

$$s_{ij} = \frac{z_i^\top z_j}{\|z_i\|_2 \|z_j\|_2} \tag{9.15}$$

当相似度较高时，可将样本作为潜在语义近重复候选。SemDeDup 是基于预训练嵌入开展大规模语义去重的代表性研究。其通过语义特征和聚类等方式识别具有较高语义相似性的样本，探索减少冗余数据对训练效率和泛化能力的影响。原始研究在特定数据与模型设置下展示了，在保留较少样本的情况下仍可获得接近的训练效果。但这种结果不能直接推广为“任何数据集删除一半样本都没有明显损失”。

例如，某些语义相似样本可能对应不同摄像机视角、光照条件或传感器噪声，这些差异对鲁棒性研究具有价值。因此，语义去重应当结合任务需求和样本多样性进行判断。

清洗规则与数据分布变化 任何过滤规则都可能改变数据集的实际分布。例如，如果删除所有低亮度图像，训练数据中的夜间场景比例就会下降。如果删除所有短文本描述，可能同时减少某些类别中原本较常见的简短标注。如果根据语义相似度大量删除重复场景，也可能影响特定类别或场景的样本权重。因此，数据清洗应当同时记录删除比例及其分布影响。例如，可以比较清洗前后的类别分布：$p_{\text{before}}(c)$，

与：$p_{\text{after}}(c)$，其中，c 表示类别或某种数据属性。如果数据分布发生明显变化，就需要分析这种变化是否符合目标任务。因此，清洗效果不仅应当根据保留样本数量判断，还应关注训练数据的覆盖范围与代表性。

9.2.3 图像质量与文本质量评估

质量评价的多层次含义 图像和文本质量评估是多模态数据筛选中的重要环节，但“质量”并不是单一、普遍适用的概念。例如，一张图像可能具有较高分辨率和良好曝光，但其文字描述与图像内容不一致。另一张图像可能存在运动模糊，却准确记录了目标研究所关心的快速运动场景。因此，需要区分单模态内容质量与跨模态关联质量。可以概念性地表示为：

$$Q(I_i, T_i) = F\left(Q_I(I_i), Q_T(T_i), Q_{IT}(I_i, T_i)\right) \tag{9.16}$$

其中：

- (Q_I) 表示图像质量评价；
- (Q_T) 表示文本质量评价；
- (Q_{IT}) 表示图文关联质量评价。

这里的 F 仅用于说明不同评价维度之间的关系，并不意味着实际数据集都需要将所有分数融合为单一质量值。

在实际应用中，保留多个独立质量指标通常更有利于分析和追溯数据问题。

图像的技术质量 图像技术质量主要关注成像和数据有效性。例如：图像分辨率是否满足任务要求。是否存在明显的文件损坏或不合理解码结果。是否出现严重压缩伪影。图像是否具有足够的亮度和对比度。是否存在异常模糊。其中，部分指标可以通过传统图像处理方法进行分析。例如，可以使用图像梯度或拉普拉斯算子的统计量作为清晰度相关的启发式指标。设图像为 I，可以定义：

$$Q_{\text{blur}} = \text{Var}(\nabla^2 I) \tag{9.17}$$

其中，$\nabla^2 I$ 表示图像的拉普拉斯响应，Var 表示方差。较低的数值在某些图像条件下可能反映较少高频结构，但这一指标并不等同于通用、可靠的图像质量评分。例如，具有大面积平滑背景的清晰图像，也可能具有较低的梯度统计量。因此，传统图像质量指标更适合作为候选异常筛选信号，而不是唯一的样本保留标准。

图文语义相似度 对于图文配对数据，单独检查图像与文本质量仍然不够。例如，一张清晰的犬只图片与一段语法正确的汽车描述，分别都可能具有较高单模态质量，却不是正确的图文配对。因此，可以使用图文对齐模型计算语义相似度。设图像编码器和文本编码器分别为：$z_I = f_I(I)$，$z_T = f_T(T)$，则：

$$s(I, T) = \frac{z_I^\top z_T}{\|z_I\|_2 \|z_T\|_2} \tag{9.18}$$

这一指标可以用于大规模图文候选筛选。LAION-5B 便利用预训练图文模型计算相似度，并依据相应的筛选条件保留部分候选配对。但高相似度并不意味着图文关系的全部细节都正确。例如，图像包含两只狗，文本却写成三只狗，预训练模型仍然可能因为对象类别和场景高度相似而给出较高相似度。因此，相似度更适合进行大规模候选筛选，而精细的属性与关系错误仍然需要其他方法检测。

模型评分与数据选择偏差 使用预训练模型评价训练数据，能够提高大规模清洗效率，但也可能引入新的偏差。例如，如果使用某个视觉语言模型的相似度作为唯一筛选标准，可能优先保留该模型已经能够较好理解的图文关系。对于专业图像、罕见对象和复杂关系，模型可能因为预训练经验不足而给出较低分数。这会使数据筛选进一步减

少困难或长尾内容，形成对原有模型能力的依赖。因此，自动评分模型的输出不应被视为完全客观的数据真值。

可以通过不同评分器、分组抽样和人工复核等方式，检查筛选规则是否对某些类别或领域产生系统性偏差。

多模态关联质量的任务差异 不同类型的数据需要不同的关联质量评价。例如，在图文配对中，重点是语义内容是否对应。在语音转写中，重点是音频中的语言内容与转写文本是否一致。在视频文本数据中，还需要考虑描述对应的事件是否位于正确时间范围。在多传感器数据中，则需要检查时间戳和空间标定关系。因此，可以将多模态关联质量概括为：

$$Q = Q_{\text{rel}}\left(X^{(1)}, X^{(2)}, \mathcal{R}\right) \tag{9.19}$$

其中，$\mathcal{R}$ 表示数据集定义的模态关系。这一表达式强调，关联质量评价应当依据实际关系类型进行设计。例如，视觉语义相似度并不能替代激光雷达与相机之间的几何配准误差。同样，语音识别准确率也不能单独判断一段音频与视频是否正确同步。因此，多模态数据质量评价需要尊重不同模态和不同任务的关联机制。

质量阈值与样本保留 在实际清洗中，可以根据多个指标制定样本保留规则。例如：

$$R(D_i) = \begin{cases} 1, & Q(D_i) \text{ 满足保留条件} \\ 0, & \text{其他情况} \end{cases} \tag{9.20}$$

其中，$R(D_i)$ 表示样本是否被保留。但数据质量往往具有多个维度，不一定适合压缩为单一分数。例如，一个图文样本可能图像质量较高，但配对文本不准确。对于图像预训练，它可能仍然具有使用价值；对于图文对比学习，则可能需要过滤或修正文本。因此，可以采用分层处理方法。首先判断样本是否满足基本文件有效性要求。随后检查单模态内容质量。再根据任务要求评价模态关联关系。最后结合类别、场景和来源分布决定是否保留。

这种方式有利于避免单一评分阈值造成不必要的信息损失。

9.3 配对、时间与区域标注

9.3.1 图文配对与图像描述

图文配对的基本形式 图文配对（Image-Text Pairing）是视觉语言预训练中最常见的数据组织方式之一。其基本形式是将一张图像与一段相关文本关联起来：$D_i = (I_i, T_i)$，其中，I_i 表示第 (i) 张图像，T_i 表示与之关联的文本。文本可以是人工撰写的图像描述，也可以是网页标题、图片说明、产品信息或其他与图像相关的自然语言内容。例如，一张展示公园场景的图像，可以对应描述：

“一名男子正在公园里骑自行车。”这种配对关系能够为模型提供图像内容与语言

表达之间的对应信息，使模型有机会学习视觉特征与语言语义之间的联系。但图文配对并不意味着图像中的所有信息都已经被文本完整描述。上述文本仅覆盖了人物、动作和场景等部分内容，并未说明图像中的全部对象、空间关系或背景细节。因此，图文配对通常是一种具有信息选择性的监督形式，而不是对图像内容的完整编码。

图像描述与图像标签的区别 图像描述（Image Captioning）通常使用自然语言概括图像中的对象、动作、场景或事件。例如：“几名行人在雨后的街道上行走。”相比之下，图像分类标签可能只包含“行人”“街道”或“雨天”等类别信息。两者的区别不仅在于表达形式，还在于所包含的信息结构。图像标签主要提供离散类别监督，而图像描述可以进一步包含对象属性、动作以及对象之间的关系。

例如，“一名儿童正在将球传给另一名儿童”不仅涉及人物和球，还包含动作主体、动作对象及两个人物之间的关系。这类关系信息对于视觉语言模型的细粒度理解和推理具有重要价值。但自然语言描述也存在局限。描述者通常只选择部分显著内容，而不会对所有对象、属性和空间关系进行穷尽式描述。因此，未在描述中出现的对象，不一定意味着图像中不存在该对象。

图文配对不一定是一对一关系 在实际数据工程中，图像与文本之间可以存在多种关联结构。一张图像可以对应多条描述；一段文本也可以描述多张相似图像；同一段文本还可能对应视频中的多个相关片段。因此，图文配对并不总是简单的一对一关系。可以将其表示为一个关联集合：

$$\mathcal{P} = (I_i, T_j) \mid r_{ij} = 1 \tag{9.21}$$

其中，r_{ij} 表示图像 I_i 与文本 T_j 之间是否存在符合数据集定义的关联。这一表达方式允许一张图像对应多个文本，也允许不同图像共享具有相似语义的描述。文本语义相似并不意味着它们必然是同一图像的正确描述。数据集构建时仍然需要明确配对关系的来源和可信程度。

从整图描述到区域级图文配对 整图描述主要建立图像整体与文本句子之间的关联，但无法直接说明文本中的每个对象或短语位于图像的什么位置。例如，文本“一个戴着红色帽子的孩子站在自行车旁边”包含多个可以定位的实体及其关系。如果只将整张图像与整句话配对，模型可以学习整体语义，却不一定能够准确确定“红色帽子”“孩子”和“自行车”分别位于何处。因此，可以进一步建立区域级图文配对。

例如，Flickr30k Entities 将文本中的实体提及与图像区域对应起来，使数据不仅包含图像与句子的关系，还包含短语与具体区域之间的关联。这种标注形式可以表示为：$(T_{i,j}, B_{i,j})$，其中，$T_{i,j}$ 表示文本中的某个实体短语，$B_{i,j}$ 表示与之对应的图像区域。区域通常可以通过边界框表示，也可以采用分割掩码或其他空间标注形式。

这一扩展能够为短语定位、指代表达理解和细粒度视觉语言对齐提供更加直接的监督。

图像描述的质量控制 图像描述的质量不仅取决于语言是否流畅，还取决于内容是否有图像证据支持。例如，一条描述可能语言自然，却错误地将图像中的白色汽车描述为黑色汽车。另一条描述可能没有语法问题，但包含图像中无法确认的人物身份或事件原因。因此，图像描述标注需要明确基本规则：描述应当尽量基于图像中可观察到的信息，避免将猜测当作事实。对于对象数量、颜色、动作和空间关系等信息，应当尽量保证准确。

不同标注者可以关注图像的不同方面，不应强求所有描述完全一致。如果某项信息无法从图像中可靠判断，应当允许不描述该信息，而不是要求标注者补充没有依据的细节。这些原则能够减少错误监督，并使图文配对更加适合模型训练。

9.3.2 视频字幕与时间戳标注

视频标注与图像标注的区别 视频不仅包含空间信息，还包含连续的时间变化。一张图像通常描述某个时刻的场景，而视频可以记录动作的发生、发展和结束。因此，视频标注除了说明画面中出现了什么，还需要描述事件在什么时间发生，以及不同事件之间如何衔接。例如，一段视频可能记录：一个人走向桌子，拿起杯子，然后将杯子放回桌面。如果只对整段视频生成一句“一个人在桌子旁活动”，虽然概括了部分内容，却丢失了动作顺序和具体事件信息。

因此，视频标注通常需要将自然语言描述与时间区间建立对应关系。

字幕、转录与视频描述 视频文本标注并不只有一种形式。字幕可以包含视频中实际出现的对白或旁白。语音转录强调将音频中的语言内容转换为文字。视频描述则侧重于对画面中对象、动作或事件的自然语言概括。例如，某段视频中的人物说“请把杯子递给我”，这句话属于语音内容；而“另一名人物将杯子递给说话者”则是对画面中发生的动作进行描述。两者可以相互补充，但不能混为一谈。

如果训练目标是语音识别，重点是语言内容及其时间位置；如果训练目标是视频理解，则可能还需要标注视觉事件及其与语音之间的关系。

视频事件的时间区间 视频事件描述通常需要记录事件开始和结束的时间。可以将一个事件标注表示为：

$$D_i = (t_i^s, t_i^e, T_i) \tag{9.22}$$

其中：

- (t_i^s) 表示事件的开始时间。
- (t_i^e) 表示事件的结束时间。
- (T_i) 表示该时间区间对应的自然语言描述。

例如，可将视频事件标注需要保存的信息整理如下：

以上按正文整理标注字段，未另设事件时间戳。实际数据中，不同事件之间可能连续发生，也可能具有时间重叠。例如，两个人可以在同一时间交谈并进行不同动作。

因此，不能默认所有事件都必须严格首尾相接。

时间戳标注中的一致性问题 视频标注中的时间戳需要具有明确的参照系。例如，时间戳可以相对于原始视频开始时刻，也可以相对于某个裁剪后片段的开始时刻。如果在数据处理过程中改变视频起始时间，却没有同步更新字幕时间戳，就会导致标注错位。因此，数据集应当明确时间戳的定义，并记录必要的时间转换关系。

此外，帧率和时间戳之间也存在关系。对于恒定帧率视频，可以根据帧索引换算时间，但对于具有可变帧率或不规则时间信息的视频，不能简单地假定每一帧之间的时间间隔完全相同。在多模态视频数据中，如果还包含独立采集的音频，就需要进一步考虑音视频之间的同步关系。因此，视频时间标注不仅是填写开始和结束时间，还涉及原始媒体、时间基准和数据处理流程的一致性。

从事件描述到时空定位 如果只记录某个事件的时间范围，仍然无法确定事件涉及的具体对象位于画面的什么位置。例如，“一个人拿起桌上的杯子”包含人物、杯子和动作之间的关系。如果进一步记录人物与杯子的图像区域，就能够将事件描述与具体视觉证据关联起来。这类标注可以将视频事件的时间区间与图像区域、对象身份和动作关系结合起来。例如：$D_i = (t_i^s, t_i^e, T_i, B_i)$，其中，$B_i$ 可以表示相关对象的空间区域。

对于连续运动的对象，还可以记录不同帧中的位置，并建立对象轨迹。ActivityNet-Entities 等研究进一步将视频描述中的实体短语与视频帧中的对象区域关联起来，说明视频描述可以从事件级语义扩展到实体级空间定位。这种方式有助于减少模型只根据场景先验生成合理句子、却未真正依据视频内容描述事件的风险。

9.3.3 细粒度定位与结构化标注

从整体语义到局部定位 图文配对、视频描述和音频转录可以为模型提供丰富的语义信息，但这些监督形式不一定能够精确指出信息对应的局部对象、空间区域或时间位置。例如，一条图像描述可能指出画面中有一辆汽车，却没有说明汽车的边界在哪里。一段视频描述可能指出人物拿起杯子，却没有说明具体动作发生在哪些帧。一段音频描述可能指出存在鸟鸣，却没有标明鸟鸣发生的时间区间。

因此，细粒度标注（Fine-grained Annotation）进一步将语义信息与明确的空间位置、时间位置或结构化对象关联起来。这类标注能够为对象定位、指代表达理解、动作定位和跨模态检索等任务提供更直接的监督。

图像区域与边界框标注 边界框（Bounding Box）是图像细粒度标注中常见的形式。对于一个目标，可以使用矩形框记录其在图像中的位置。例如：

$$B = (x_{\min}, y_{\min}, x_{\max}, y_{\max}) \tag{9.23}$$

其中，四个坐标分别表示边界框左上角与右下角的位置。但在实际数据集中，需要进一步明确坐标的参考系，例如使用原始图像像素坐标、归一化坐标，还是经过缩放后的图像坐标。此外，还需要明确边界框是否采用整数像素坐标、边界是否包含最

外侧像素，以及图像裁剪后如何变换坐标。这些定义会直接影响模型训练和评价结果。如果标注文件中的坐标与实际图像尺寸不一致，即使标注数值本身看起来合理，也可能导致错误的空间监督。

分割掩码、关键点与其他空间标注 边界框能够提供目标的大致位置，但无法精确表示复杂形状。因此，还可以使用分割掩码（Segmentation Mask）标注目标的像素级区域。对于人体姿态或动作理解任务，则可以使用关键点标注记录头部、肩部、肘部、膝部等特定位置。不同空间标注形式提供的信息粒度不同：这些标注形式不能简单地相互替代。例如，边界框无法完整表示对象的精确轮廓；关键点也不能单独描述对象全部表面形状。

因此，应当根据模型需要学习的目标选择适当的空间监督形式。

视频中的对象轨迹与时空定位 对于视频任务，空间标注通常还需要与时间维度结合。例如，一个行人在视频中从画面左侧移动到右侧，可以通过不同帧中的边界框记录其位置变化，并使用一致的对象标识将这些位置关联起来。这类标注可以表示为：

$$\mathcal{T}_i = (t_k, B_{i,k})_{k=1}^{K} \tag{9.24}$$

其中，t_k 表示第 (k) 个时刻，$B_{i,k}$ 表示对象在该时刻的位置。这种结构可以用于多目标跟踪、动作理解和时空定位。如果任务还要求识别对象在运动过程中的行为，就可以将轨迹与动作类别、事件时间区间或语言描述进一步关联。但连续帧中的相同外观并不必然意味着同一对象，遮挡、出画和重新出现也可能导致身份关联错误。因此，轨迹标注需要明确对象身份的维持规则，以及遮挡和暂时不可见情况下的处理方式。

从自然语言到结构化监督 细粒度标注还可以进一步与自然语言指令和问答结合。例如，对于问题“红色汽车位于什么位置?”，数据不仅可以提供答案，还可以关联到对应汽车的图像区域。对于视频问题“人物什么时候拿起杯子?”，数据可以将答案关联到具体时间区间。这种方式使模型不仅学习输入与输出之间的语义关系，还能够学习输出所依据的具体证据。因此，可以将一个多模态标注样本概念性地表示为：$D_i = (X_i, Y_i, A_i)$，其中：

- (X_i) 表示原始观测，可以包含图像、视频、音频或其他模态。
- (Y_i) 表示语言描述、类别、问答答案或其他任务标签。
- (A_i) 表示与标签相关的空间位置、时间戳、对象关系或其他结构化关联信息。

这里的表示方式是概念性示例，实际数据集可以根据任务使用不同的字段。

但其核心思想是：当任务需要模型给出细粒度、可定位的结果时，标注不应只保留最终文本答案，还应尽可能明确其对应的观测证据。

9.4 指令与合成数据

9.4.1 多模态指令数据构建

从多模态配对到指令学习 传统多模态配对数据主要用于建立不同模态之间的对应关系。例如，一张图像可以对应一条文字描述：“一辆红色汽车停在道路旁。”这种数据能够支持图文对比学习和图像描述任务，但如果用户进一步提出：“图像中汽车的颜色是什么？”模型就需要根据问题选择相关视觉信息，并生成具有针对性的回答。因此，指令数据不仅需要记录图像与文本，还需要明确任务要求。可以将单轮多模态指令样本表示为：$D_i = (X_i, Q_i, Y_i)$，其中：

- (X_i) 表示图像、音频、视频或其他多模态输入；
- (Q_i) 表示任务指令或问题；
- (Y_i) 表示目标输出。

例如：

输入图像：一辆红色汽车停在街道旁。任务指令：“这辆汽车是什么颜色？”目标回答：“红色。”在这一样本中，模型需要根据指令确定应该关注汽车的颜色，而不是简单复述整张图像的所有内容。因此，指令数据可以看作在已有多模态观测和标注基础上，进一步明确任务语义及目标输出的数据组织形式。

指令数据的任务多样性 多模态指令可以覆盖不同类型的任务。例如，对于同一张图像，可以设计：图像描述：“请描述图像中的主要内容。”对象识别：“图像中有哪些交通工具？”属性判断：“汽车是什么颜色？”空间关系：“行人位于汽车的哪一侧？”数量统计：“图像中有几辆汽车？”视觉推理：“根据画面，哪辆汽车更靠近路口？”这些任务关注不同的信息层次。例如，图像描述主要要求概括场景，属性判断需要定位特定对象，而空间关系和数量统计则可能需要更加细粒度的视觉分析。

因此，指令数据的多样性不能仅通过文字表达是否不同来衡量，还应当关注任务所需的视觉能力是否不同。例如，将“描述这张图像”改写为“请介绍这幅图片”，虽然增加了语言形式变化，但并没有明显扩展任务能力范围。相比之下，增加对象计数、细粒度定位和复杂关系判断等任务，可能提供更加不同的学习信号。

指令数据的统一组织形式 为了使不同任务能够进入共同训练流程，可以设计统一的数据表示。例如：$D_i = (X_i, Q_i, Y_i, M_i)$，其中，$M_i$ 表示必要的元数据。元数据可以包括任务类别、输入模态、输出模态、原始数据来源、标注方式和质量审核结果。对于单轮视觉问答，模型可以读取图像与问题，并生成文字回答。对于文本条件图像生成，模型则可以根据文本指令输出图像或相应的图像生成表示。

因此，在统一多模态指令数据中，目标输出不一定始终是自然语言文本。例如，一条图像编辑指令：“将图片中的红色汽车改为蓝色。”如果目标任务是实际图像编辑，那么数据除了包含文字指令，还需要提供目标编辑图像、编辑区域或其他合适的生成监督。仅提供文字回答“已完成修改”，并不能代替实际图像生成任务的监督。因此，指

令数据构建应当明确输出模态及相应的训练目标。

将已有数据集转换为指令格式 一种常见的数据构建方式，是利用已有多模态数据集及其标注，将原本面向特定任务的数据转换为指令样本。例如，一个图像分类数据集包含图像和类别标签：(I_i, y_i) 可以转换为：输入：图像 I_i。指令：“这张图片中的主要动物是什么?” 回答：相应类别名称。类似地，目标检测数据可以根据边界框和类别信息构造对象识别与位置判断问题。视频动作数据可以根据动作标签和时间区间构造视频事件问答。

语音转录数据可以转换为“请将这段语音转写为文字”等任务。InstructBLIP[23]是这一技术路线的代表性研究。该工作汇集了 26 个公开视觉语言数据集，并将其中多种任务组织为指令微调格式。这种方法能够利用已经存在的监督信息，降低重新标注的成本，同时增加模型接触的任务种类。但将已有标签转换为自然语言问题，并不意味着能够自动产生原始标注中不存在的信息。

例如，如果某个图像数据集只有场景类别标签，就不能据此可靠生成复杂的对象数量和空间关系答案。因此，任务转换应当受到原始监督信息的约束。

多模态多轮指令 与单轮问答相比，多轮指令数据需要保留更完整的交互上下文。例如：用户：“这张图像中有什么?” 助手：“有一辆红色汽车和两名行人。” 用户：“汽车位于行人的哪一侧?” 助手：“汽车位于两名行人的右侧。” 后一轮问题需要利用前面的图像和对话上下文。因此，多轮数据可以表示为：$C_i = m_1, m_2, \ldots, m_K$，其中，每条消息包含角色、内容和必要的媒体引用。

MIMIC-IT 是多模态上下文指令数据的重要研究。该工作通过自动化与人工参与相结合的方式，构建了约 280 万组多模态指令—回答数据，并引入相应的多模态上下文信息。这类数据能够使模型学习在更复杂的交互过程中利用视觉和语言信息。但多轮对话中的后续回答可能依赖早期的错误回答。如果直接采用模型自动续写构造长对话，错误可能在多个回合中逐渐积累。因此，多轮指令数据应当特别关注会话上下文与真实多模态证据之间的一致性。

9.4.2 问答、推理与任务型数据

多模态问答数据的基本形式 多模态问答（Multimodal Question Answering）是多模态指令学习的重要任务。设输入观测为 (X)，问题为 (Q)，目标答案为 (A)，则数据可以表示为：$D_i = (X_i, Q_i, A_i)$，例如，一张图像显示一个红色杯子位于蓝色书本左侧。可以构造：问题：“杯子是什么颜色?” 答案：“红色。” 问题：“杯子位于书本的哪一侧?” 答案：“左侧。”

前者主要测试对象属性识别，后者进一步涉及空间关系。因此，问答数据可以通过不同问题形式，对同一模态输入提供不同的监督信号。

基础感知问题与组合推理问题 根据任务所需的信息处理程度，可以将多模态问答大致区分为基础感知和组合推理两类。基础感知问题主要要求识别直接可观察的内容。例如：“图像中有没有汽车?”“交通灯是什么颜色?” 组合推理问题则需要结合多个对象、

属性或关系。例如：“位于红色汽车左侧的行人手里拿着什么？”模型首先需要识别红色汽车，然后确定汽车左侧的行人，再判断行人手中的物体。因此，该问题涉及多个相互关联的信息选择步骤。

问题句子较长不一定代表推理复杂，而问题较短也可能需要较深的视觉分析。例如，“哪个更近？”虽然只有几个字，却可能要求模型结合三维空间关系进行判断。因此，问答数据的难度应当依据任务所需的视觉信息与推理结构分析，而不能只依据文本长度衡量。

推理过程与最终答案 对于复杂问答，数据可以只包含最终答案，也可以进一步包含解释或中间步骤。例如：问题：“根据图中的柱状图，哪个季度的销售额最高？”答案：“第三季度。”解释：“比较四个季度对应柱子的高度，第三季度最高。”可以将这类数据表示为：$D_i = (X_i, Q_i, R_i, A_i)$，其中：

- (R_i) 为解释或解题步骤。
- (A_i) 为目标答案。

加入解释能够为模型提供更加丰富的训练监督，使模型不仅学习最终输出，还可以学习一种可供表达的分析过程。

但解释文字合理，并不意味着它一定真实反映模型内部的推理过程。尤其对于模型自动生成的解释，可能出现答案正确但解释错误，或者解释流畅却无法由输入图像支持的情况。因此，推理数据的质量控制应当分别检查最终答案和解释内容。对于数学和程序化任务，还可以利用计算结果或结构化规则验证中间结论，而不只是评价文本表达是否合理。

任务型数据与结构化输出 除自然语言问答外，多模态指令还可以对应结构化任务。例如：“找出图像中的全部行人，并返回边界框。”“指出视频中人物开始跑步的时间。”“根据当前机器人观测，预测下一步操作。”这些任务需要的输出可能是坐标、时间区间、动作或其他结构化数据，而不是一段普通自然语言描述。例如，对于目标定位，可以表示为：$D_i = (I_i, Q_i, B_i)$，其中，B_i 为目标位置。

对于视频时间定位，可以表示为：$D_i = (V_i, Q_i, [t_s, t_e])$，这些结构化输出应当与原始数据使用的坐标系、时间基准和任务协议相一致。因此，将检测、分割和动作预测等任务转换为自然语言指令形式时，仍然需要保留输出数据的实际物理或几何含义。

答案偏差与视觉证据依赖 多模态问答数据可能存在答案分布偏差。例如，如果大量问题以“图像中有几个人？”开头，并且训练样本的答案主要集中在“一”或“二”，模型可能依靠语言模式猜测答案，而不是充分分析图像。类似地，如果某种对象总是与固定颜色共同出现，模型可能依赖训练数据中的常见共现关系作答。因此，问答数据构建需要关注答案分布和问题模板是否过于集中。可以通过增加不同场景、改变对象属性组合和设计受控对照样本，减轻部分偏差。

例如，对于“红色杯子位于蓝色书本左侧”这一场景，可以进一步构造对象颜色或位置不同的合法样本，避免模型将某个固定答案与问题形式直接绑定。但受控对照

样本必须具有真实或可验证的视觉依据，不能只修改文本答案而不修改对应图像。

9.4.3 模型生成的合成数据

合成数据的基本含义 合成数据（Synthetic Data）通常指由模型、程序、规则系统或仿真环境等人工构造机制产生的数据，而不是直接以原始形式从真实环境中采集的数据。在多模态研究中，合成数据可以包含多种形式。例如：使用语言模型生成任务指令。使用语言模型生成图像问答及解释。使用图像生成模型合成训练图像。使用语音合成模型生成语音及对应文本。使用视频生成模型合成动作片段。使用仿真环境产生传感器观测、状态和动作序列。

这些数据虽然都可以被称为合成数据，但其生成对象和可信依据不同。例如，基于真实图像生成问答，并不意味着图像本身是合成的。反过来，程序生成的图像可能具有精确的结构标签，但其视觉外观未必与真实场景一致。因此，合成数据需要根据来源和生成机制进一步分类。

指令合成与内容合成的区别 多模态合成数据可以大致分为两类。一类是基于已有真实观测生成新的任务监督。例如，给定一张已经标注对象与关系的图像，使用模型或模板生成不同的问题与答案。另一类是直接生成新的观测内容。例如，根据文本生成新的图像、音频或视频，再构建相应训练样本。两类方法的区别在于，前者主要扩展监督形式，后者则进一步改变训练数据中的观测分布。例如：

$$(I, Y_{\text{raw}}) \xrightarrow{\text{指令生成}} (I, Q, A) \tag{9.25}$$

与：

$$C \xrightarrow{\text{内容生成}} (\tilde{I}, \tilde{Y}) \tag{9.26}$$

分别代表两种不同的数据构建过程。其中，$\tilde{I}$ 和 $\tilde{Y}$ 表示合成观测及其相应标注。合成观测的标签可以来自程序或仿真系统，也可能来自另一个模型的预测。两种来源的可靠程度不应被视为相同。

程序化合成与可验证视觉数据 除根据已有真实图像生成指令外，还可以通过程序直接构造图像和相应监督。例如，可以利用绘图程序生成柱状图、折线图、表格或几何图形。由于图像内容由程序控制，生成过程中可以同时保存对象位置、数值、颜色和几何关系等真值。例如，程序生成一个柱状图，其各类别数值为：$(10, 15, 8, 20)$ 就可以直接构造：问题：“哪个类别对应的数值最大?”答案：对应数值 20 的类别。

答案可以通过原始数据表验证，而不需要依赖另一个视觉语言模型猜测。Multimodal Self-Instruct 是这一方向的代表性研究。该工作利用语言模型的代码生成能力，构造图表、表格、模拟地图、流程图等视觉内容及相应的推理指令。这种方法能够生成一些较难通过网络爬取获得、但具有明确结构真值的视觉推理样本。

不过，程序生成图像通常具有相对规则的布局与渲染风格，因此其训练价值还受

到合成样本与真实应用数据之间分布差异的影响。

教师模型与伪标签 另一种合成监督方式，是使用已有较强模型，为大量未标注数据生成伪标签（Pseudo-label）。例如，使用视觉语言模型为图像生成描述，或者使用目标检测模型为未标注图像产生对象框。可以表示为：

$$\tilde{Y}_i = f_{\text{teacher}}(X_i) \tag{9.27}$$

随后使用：$(X_i, \tilde{Y}_i)$ 训练学生模型。这一过程可以与知识蒸馏、自训练和半监督学习等方法结合。但教师模型的输出并不等于真实标签。例如，教师模型可能漏检小目标，或者生成不存在的对象描述。如果学生模型将这些伪标签全部当作确定真值进行学习，就可能继承教师模型的系统性错误。因此，伪标签训练通常需要结合置信度筛选、人工抽样或其他独立证据进行质量控制。

合成数据与真实数据的混合使用 实际训练中，合成数据往往与真实数据共同使用。设真实数据集为：

$$\mathcal{D}_{\text{real}} \tag{9.28}$$

合成数据集为：

$$\mathcal{D}_{\text{syn}} \tag{9.29}$$

可以构造联合训练分布：

$$p_{\text{mix}} = (1-\lambda)p_{\text{real}} + \lambda p_{\text{syn}} \tag{9.30}$$

其中：$0 \le \lambda \le 1$ 表示训练采样中合成数据的权重。这里的表达用于说明数据混合方式，并不意味着真实训练只能采用固定比例随机混合。例如，可以先使用规模较大的合成数据进行预训练，再利用真实数据进行微调；也可以根据任务难度逐步调整合成样本的比例。不同策略在训练效率和模型泛化方面可能具有不同表现。因此，合成数据是否有效，应当通过固定模型和训练预算下的对照实验验证，而不能仅根据数据量增长判断。

9.4.4 合成数据的质量控制与偏差

合成数据质量问题的特殊性 合成数据的一个重要特点是，其内容或标注可能由已有模型自动生成。与人工原始标注相比，合成样本通常能够以较低成本快速扩展，但也可能继承生成模型的能力限制。例如，语言模型可能生成语法正确但事实错误的解释。视觉语言模型可能漏掉图像中的小目标。图像生成模型可能无法准确满足对象数量或空间关系要求。因此，合成数据质量不仅取决于最终文本或图像是否自然，还需要检查其内容是否具有可靠依据。对于多模态指令数据，可以将质量控制分为三个层次：

数据结构是否有效。监督内容是否正确。数据分布是否适合目标任务。这三个层次需要分别评价，不能用一个综合分数完全替代。

结构有效性检查 合成样本首先需要满足预先定义的数据格式。例如，一条视觉问答记录应当包含有效的图像引用、问题和目标答案。多轮对话样本需要具有合理的消息顺序及角色标识。对于视频事件问答，还需要保证时间区间没有超出原始视频时长。对于目标检测和空间定位数据，需要检查边界框坐标是否位于有效图像范围内。这些问题可以通过程序规则进行较明确的验证。但结构完整并不代表监督正确。

例如，一条问答记录即使具有完整的图像路径、问题字段和答案字段，也可能包含错误的视觉判断。因此，结构有效性只是质量控制的基础环节。

答案正确性与解释正确性 对于推理指令数据，除了最终答案，还可能包含解释或推理步骤。例如，模型可以给出正确答案，但在解释中引用不存在的图像内容。也可能生成逻辑连贯的解释，却由于读取图表数值错误而得到错误结论。因此，可以分别定义：Q_{answer} 和：$Q_{\text{explanation}}$ 分别表示答案正确性和解释有效性。二者不必具有相同结果。

对于可执行的数学或逻辑任务，可以利用独立的计算规则验证最终答案及部分中间步骤。对于视觉推理任务，可以利用原始对象、属性和关系标注，检查解释是否引用了真实存在的视觉证据。但自然语言解释并不一定能够完整反映模型内部的实际计算过程。因此，对解释的评价主要应关注其内容是否合理、可验证，而不是直接把解释当作模型内部推理机制的透明记录。

教师模型偏差与错误传播 合成数据的质量通常受到生成模型自身能力影响。例如，如果教师模型经常遗漏小目标，那么它为未标注图像生成的问答可能较少涉及这些对象。如果教师模型倾向于选择常见答案，那么合成问答中的答案分布也可能发生偏移。此外，当模型生成的样本再次被其他模型用于合成数据时，某些错误或偏差可能不断传播。因此，合成数据生成需要记录教师模型及其版本，并尽量避免在缺少独立监督的情况下，将模型生成结果当作无误差的真实标签。

例如，某个教师模型对图像中的对象数量给出了答案“三”，但如果缺少原始检测标注或其他可靠证据，就不能仅凭这个回答确定图像中确实有三个对象。因此，教师模型输出更适合作为候选监督，而不是无条件的最终真值。

任务分布与表达多样性 合成数据规模较大，并不意味着任务覆盖一定充分。例如，使用固定提示词批量生成视觉问答时，模型可能反复产生：“这是什么？”“图像里有什么？”“请描述这张图片。”这些问题虽然表面不同，但实际任务类型较为接近。如果训练数据中过度集中于某些问题形式，模型可能学习到固定的回答模式，而难以适应新的任务表达。因此，需要从任务类型、问题结构、答案形式和输入场景等多个角度评价数据多样性。

例如，对于图像推理任务，可以检查数据是否覆盖对象属性、数量、位置关系、图表读取和复杂组合等不同能力。同时，还应当检查不同任务之间是否存在严重的不平衡。但多样性并不要求每个类别都具有完全相同的样本数量。实际训练数据分布还需要与目标任务和预训练目标相协调。

评分模型与自动筛选的局限 为了控制大规模合成数据质量，可以使用另一个模型对生成样本进行评分。例如，让评价模型判断图文是否一致、回答是否准确，以及指令是否清晰。这种方式能够降低部分人工检查成本。但自动评分也可能存在局限。生成模型和评分模型可能具有相似的训练数据与偏差。评分模型可能偏好语言流畅、格式规范的回答，却无法准确判断复杂视觉事实。如果反复使用同一个评分器筛选数据，最终训练集可能逐渐偏向评分器容易接受的内容。

因此，自动筛选最好与程序验证、抽样人工复核和独立评价结合使用。尤其对于复杂视觉推理，不能仅因为模型给出较高的自信度或评分，就认定答案具有充分的事实依据。

9.5 数据混合与采样

算例：样本配比与计算配比

设任务 k 以概率 p_k 被抽取，样本的有效目标长度为随机变量 L_k。即使样本比例相等，各任务对 Token 总数和计算的贡献仍可不同。报告混合方案时，应同时列出抽样概率、实际处理样本数、有效目标 Token 数和视觉/音频长度分布。优化器看到的权重还取决于损失按样本还是按 Token 归一化。

9.5.1 数据集配比与采样权重

多数据集混合的基本问题 设多模态模型需要使用 K 个训练数据集：

$$\mathcal{D} = \mathcal{D}_1, \mathcal{D}_2, \ldots, \mathcal{D}_K \tag{9.31}$$

其中，第 k 个数据集包含：

$$N_k = |\mathcal{D}_k| \tag{9.32}$$

个样本。这些数据集可以对应不同的数据来源、模态组合或训练任务。例如：

$$\mathcal{D}_1 : \text{图文配对} \tag{9.33}$$

$$\mathcal{D}_2 : \text{视觉问答} \tag{9.34}$$

$$\mathcal{D}_3 : \text{视频文本} \tag{9.35}$$

$$\mathcal{D}_4 : \text{音频文本} \tag{9.36}$$

如果直接将全部数据拼接，并在所有样本之间均匀采样，那么第 k 个数据集被选中的概率为：

$$p_k = \frac{N_k}{\sum_{j=1}^{K} N_j} \tag{9.37}$$

这种方式可以称为按样本规模比例采样。其特点是，数据规模越大的数据集，在训练中出现的概率越高。例如，某个图文数据集可能包含大量样本，而视频数据集相对较小。如果采用按规模比例采样，模型的大部分训练更新可能来自图文数据。这种分配并不一定错误，但需要判断它是否符合预期的能力目标。

按规模采样与均衡采样 除按样本规模比例采样外，还可以采用数据集均衡采样。例如，当共有 K 个数据集时，设置：

$$p_k = \frac{1}{K} \tag{9.38}$$

使每个数据集具有相同的抽样概率。这种方式可以提高小规模数据集的训练参与度。例如，当视频数据远少于图像数据时，均衡采样能够增加视频任务进入训练批次的频率。但均衡采样也可能带来新的问题。如果某个数据集只有少量样本，却获得与大型数据集相同的采样概率，其样本可能被反复使用。因此，均衡采样并不等于每个样本具有相同的训练机会，也不意味着不同任务能够获得相同的泛化收益。实际训练需要在数据集覆盖和样本重复利用之间取得平衡。

温度式数据采样 为了在按规模采样和数据集均衡采样之间进行调整，可以采用幂指数形式的采样分布。例如：

$$p_k = \frac{N_k^{\alpha}}{\sum_{j=1}^{K} N_j^{\alpha}} \tag{9.39}$$

其中，α 为控制数据规模影响程度的参数。在这一定义下：当 $\alpha = 1$ 时，对应按数据集样本数量比例采样。当 $\alpha = 0$ 时，若各数据集均非空，则对应数据集均衡采样。当 $0 < \alpha < 1$ 时，大数据集仍然获得更多采样机会，但其规模优势被适当减弱。这种方法能够在保留数据规模差异的同时，提高相对小规模数据集的训练参与度。

但温度式采样并不存在适用于全部任务的固定最优指数。例如，过度提升小规模数据集的采样频率，可能使其中的噪声标注或特定场景被反复学习。因此，采样参数应根据数据质量、任务表现和训练预算确定，而不能仅由数据集大小决定。

引入任务重要性权重 除样本规模外，还可以根据任务重要性进一步设置采样权重。例如：

$$p_k = \frac{w_k N_k^{\alpha}}{\sum_{j=1}^{K} w_j N_j^{\alpha}} \tag{9.40}$$

其中：

- (N_k) 表示数据集规模；
- (α) 控制数据规模对采样概率的影响；
- (w_k) 表示额外设置的数据集权重。

例如，如果模型主要面向视觉问答与图表理解，那么可以适当提高相关数据的训练频率。

如果模型需要处理实时音视频交互，也可能需要增加时序数据的参与比例。但任务重要性权重属于训练设计决策，而不是数据集天然具有的属性。例如，一个数据集规模较小，不意味着它一定应该具有较高权重；一个任务较复杂，也不意味着提高采样比例必然能够改善其性能。因此，任务权重应通过明确的目标和实验对照确定。

数据集采样与样本采样的区别 在多模态训练中，需要区分数据集级采样和数据集内部采样。例如，可以先根据概率 p_k 选择数据集，再从该数据集中选择具体样本。设数据集内部的样本采样分布为：

$$q(i \mid k) \tag{9.41}$$

则选择第 k 个数据集中的第 i 个样本的概率为：

$$P(k, i) = p_k q(i \mid k) \tag{9.42}$$

如果数据集内部均匀采样：

$$q(i \mid k) = \frac{1}{N_k} \tag{9.43}$$

那么：

$$P(k, i) = \frac{p_k}{N_k} \tag{9.44}$$

这说明，数据集级配比和样本级重复程度之间具有直接关系。例如，提高一个小型数据集的采样权重，会增加其中每个样本被选中的频率。因此，在记录数据混合策略时，仅报告每个数据集的采样概率还不够，最好同时明确数据集内部的抽样方式、是否允许重复采样，以及训练过程中样本的实际使用次数。

采样概率与损失权重的区别 多模态训练中另一个容易混淆的问题，是数据采样权重与损失函数权重。采样概率决定某个任务或数据集多长时间参与一次训练。损失权重则决定该任务被采样后，其损失对参数更新的影响程度。设第 k 个任务的损失为：

$$\mathcal{L}_k(\theta) \tag{9.45}$$

其损失权重为：λ_k，若每次更新先按 p_k 采样任务，再计算相应加权损失，则理想化的期望训练目标为：

$$\mathcal{L}(\theta) = \sum_{k=1}^{K} p_k \lambda_k \mathbb{E}_{x \sim q_k}[\ell_k(\theta; x)] \tag{9.46}$$

其中，q_k 为第 k 个数据集内部的样本分布。因此，任务对训练目标的贡献可能同时受到采样概率和损失权重影响。但二者在实际优化中并不完全等价。采样比例会改变任务进入训练批次的频率与梯度噪声，而损失权重直接改变相应梯度的数值尺度。此外，优化器状态、批次组织和梯度累积方式也可能影响实际训练过程。因此，不应仅因为两种配置具有相同的 $p_k\lambda_k$，就认定它们一定产生完全相同的训练结果。

训练样本占比与计算预算 除 Token 数量外，不同模态的计算成本也可能不同。例如，高分辨率图像需要较多视觉特征计算。长视频输入可能增加时空注意力的计算量。音频编码器的计算成本可能与音频时长及表示方式有关。因此，如果第 k 类任务平均需要计算成本 c_k，可以将其预期计算预算占比近似表示为：

$$b_k = \frac{p_k c_k}{\sum_{j=1}^{K} p_j c_j} \tag{9.47}$$

其中，c_k 可以根据具体研究目标定义为计算量、训练耗时或其他资源度量。上式只是用于说明样本采样与资源消耗之间的关系。实际训练中的任务混合批次、并行执行和硬件利用率，可能使计算成本不再具有简单的线性关系。因此，在多模态数据混合实验中，应尽可能区分：样本级占比。Token 级占比。任务更新次数。实际训练计算与时间预算。这有助于避免将不同资源条件下的模型性能直接归因于某个数据集的作用。

9.5.2 不同任务之间的数据平衡

数据集平衡与任务平衡 在多模态训练中，一个数据集不一定只对应一种任务，同一任务也可能使用多个数据集。例如，一张图像可以参与图文对比学习、图像描述和视觉问答。不同数据集也可能都包含图像描述任务，但数据质量、语言风格和场景分布不同。因此，需要区分数据集平衡（Dataset Balancing）和任务平衡（Task Balancing）。数据集平衡主要决定不同数据来源进入训练的比例。任务平衡则决定不同学习目标获得多少训练机会。

例如，模型可以从两个图文数据集中抽样，但这些样本共同服务于同一个图文对齐目标。反过来，同一批图文数据可以根据不同任务配置参与对比学习和自回归描述生成。因此，数据集数量和任务数量不能直接等同。

多模态任务的不同监督目标 多模态训练可能同时包含不同类型的监督。例如：图文对齐主要学习视觉与语言的语义对应。图像描述主要根据图像生成自然语言。视觉问答主要学习根据问题选择相关视觉信息。视频事件理解进一步关注时间关系。语音识别需要建立声学信号与文字内容之间的联系。图像或音频生成则可能使用扩散、流匹配或其他生成损失。因此，不同任务的训练目标可能采用不同的数学形式。可以写为：

$$\mathcal{L} = \sum_{k=1}^{K} \lambda_k \mathcal{L}_k \tag{9.48}$$

但不同损失的数值大小不能直接作为任务重要程度的统一衡量。例如，一个交叉熵损失与一个连续特征均方误差损失，其数值尺度可能天然不同。因此，任务平衡不能简单依赖损失数值大小，还应当结合梯度影响、任务表现和训练目标判断。

多任务训练中的负迁移 不同任务共享模型参数时，并不一定总能相互促进。例如，图像描述要求模型生成自然语言，而精细目标定位要求模型保留准确的空间信息。对于视频任务，模型需要关注动作顺序和时间关系；对于静态图像任务，则可能更加关注对象外观与语义。如果共享参数容量不足，或者训练数据分配不合理，不同任务可能出现优化竞争。这种情况通常与负迁移（Negative Transfer）有关。例如，可以比较两个任务的共享参数梯度：

$$g_i = \nabla_\theta \mathcal{L}_i \tag{9.49}$$

$$g_j = \nabla_\theta \mathcal{L}_j \tag{9.50}$$

并计算：

$$\cos(g_i, g_j) = \frac{g_i^\top g_j}{\|g_i\|_2 \|g_j\|_2} \tag{9.51}$$

这一指标可以辅助分析任务更新方向是否相似。但梯度方向不同并不必然意味着某个任务已经受到损害。更加可靠的判断仍然需要比较联合训练与独立训练的实际结果。因此，任务平衡应当结合下游表现进行调整，而不是仅依据某一次梯度统计作出决定。

基于模型表现调整任务比例 除预先固定任务配比外，还可以根据训练过程中的表现调整采样策略。例如，如果某个任务的能力长期明显不足，可以适当增加其训练频率，进一步观察结果是否改善。但直接使用训练损失大小决定采样比例可能产生问题。例如，损失较高可能来自任务较难，也可能来自标注噪声、损失尺度差异或模型无法有效学习的样本。因此，不能简单地认为损失越高的数据就一定越值得增加采样。

一种更加稳妥的做法是，根据明确的独立评价指标、样本质量以及训练稳定性，判断某种任务是否需要增加预算。DoReMi 进一步提出通过较小代理模型和分布鲁棒优化学习数据领域权重，再利用这些权重构建大模型的训练数据混合。DoReMi 的原始研究主要针对语言模型预训练领域配比，其具体方法和效果不能不经验证直接推广到所有多模态任务。但它表明，数据配比可以从人工经验进一步发展为具有明确优化目标的数据选择问题。

能力保持与任务遗忘 多模态模型可能采用分阶段训练。例如，先进行大规模图文预训练，再开展视觉指令微调，最后增加视频或音频任务。在后续训练阶段，如果某些早

期任务数据不再出现，模型可能发生能力退化。例如，继续训练大量视觉问答数据后，模型的原有语言生成或某些图像语义能力可能发生变化。因此，任务数据组织不仅需要考虑新增能力，还需要考虑已有能力的保持。一种常见思路是在后续训练中保留适量的早期任务数据，使模型持续接触部分原有监督。

但保留比例同样需要根据任务表现决定。例如，过多重复旧任务可能限制新任务训练预算，而完全删除旧任务又可能增加遗忘风险。因此，多任务数据平衡应当贯穿模型的整个训练过程，而不只在初始数据混合时确定一次。

9.5.3 课程学习与渐进式数据组织

课程学习的机制 课程学习（Curriculum Learning）是一种根据样本或任务的难度、结构和训练目标，安排数据呈现顺序的训练策略。其基本思想是，不一定从训练开始就让模型以相同概率接触所有样本，而可以按照某种预先定义或动态调整的顺序组织训练数据。例如，模型可以先学习较简单的对象识别，再逐步增加复杂空间关系和多轮视觉推理任务。

Bengio 等人在 2009 年提出的 Curriculum Learning 系统讨论了这种数据组织方式，并通过实验研究了训练样本呈现顺序对优化和泛化的影响。课程学习并不意味着任何任务都应该严格按照从易到难的顺序训练。对于不同模型和任务，训练顺序可能具有不同影响，甚至可能引入不必要的偏差。因此，课程学习应当被视为一种需要验证的数据组织策略，而不是普遍有效的固定规则。

样本难度与数据质量的区别 课程学习需要首先确定“难度”的含义。例如，对于视觉问答：“图像中有没有汽车？”可能比：“位于红色汽车后方、靠近路口的行人手中拿着什么?”需要更少的信息选择步骤。但难度与数据质量不是同一个概念。例如，一个复杂但具有准确标注的视觉推理样本，可能属于高质量难样本。而一个问题十分简单、但答案错误的样本，则属于低质量样本。因此，不能将高质量样本直接定义为简单样本，也不能将低质量样本一律视为困难样本。

课程学习中的难度设计，应当尽可能与任务所需的计算或推理能力有关，而不应仅依据标注是否干净。

课程分布的动态变化 从概率分布角度看，课程学习可以表示为训练过程中数据采样分布随时间发生变化。设训练进度为 τ，则：$p_k = p_k(\tau)$ 表示第 k 类数据在不同训练阶段具有不同采样概率。例如，初始阶段可以适当提高基础任务比例，后续逐渐增加复杂任务权重。也可以构造：

$$p_\tau = (1 - \beta_\tau)p_{\text{base}}(x) + \beta_\tau p_{\text{target}}(x) \tag{9.52}$$

其中：

- (p_{base}) 表示初始训练分布；
- (p_{target}) 表示目标训练分布；

- (β_τ) 表示随训练进度变化的混合系数。

当 β_τ 增加时，训练分布逐渐向目标分布转移。

这种表达能够说明渐进式训练的基本思想，但实际模型的训练策略可以更加复杂，例如分阶段固定配比、动态任务采样或根据能力表现调整训练数据。

课程学习中的能力保持 随着训练阶段变化，模型可能更多接触新任务，而减少对早期任务的训练。例如，持续预训练阶段大量使用视频与 OCR 数据，后续指令微调则集中于复杂问答。如果早期能力所对应的数据长期不再参与训练，模型可能出现一定程度的能力退化。因此，渐进式训练需要考虑能力保持。例如，可以在后续阶段保留少量基础任务样本，使模型持续接触原有监督。也可以通过定期评价决定是否需要调整某些任务的采样比例。但保留旧任务数据并不保证完全避免遗忘。

实际效果还受到模型容量、优化器设置和新旧任务之间关系影响。因此，课程学习和分阶段训练需要同时评价新增能力与已有能力的变化。

课程策略的公平验证 为了判断课程学习是否具有实际作用，需要比较不同训练顺序。例如：所有任务从初始阶段均匀混合训练。先基础任务、后复杂任务。先短视频、后长视频。固定配比与动态配比。但这些实验需要尽可能控制总训练数据、训练计算预算和模型结构。否则，如果课程学习方案使用了更多训练步骤或更多高质量数据，就难以判断性能变化究竟来自训练顺序，还是额外资源。此外，还应当分别评价不同任务能力，而不只报告最终平均分数。

例如，某个课程可能提高复杂图表推理能力，却降低基础图像描述表现。因此，课程策略是否有效，应根据目标任务和多能力评价结果综合判断。

9.6 许可、隐私与可追溯性

9.6.1 训练数据的版权与许可

数据可访问性与使用权的区别 在互联网环境中，大量图像、文字、音频和视频可以通过网页、开放数据平台或公共存储服务获得。但是，能够访问或下载某项数据，并不意味着使用者自动获得了对该数据进行所有形式处理和再分发的权利。例如，研究者可能在公开网页中找到一张照片，并能够通过浏览器下载该图片。但这并不意味着该图片必然可以被重新发布、纳入公开数据集，或用于任何商业训练用途。

类似地，某个公开数据集可能允许学术研究使用，却限制商业用途，或者要求使用者遵守特定的署名和再分发条件。因此，数据工程需要区分几个不同概念。公开可访问描述数据的获取状态。拥有复制或使用权限涉及具体法律依据或授权范围。允许再分发涉及是否可以将数据文件或其衍生内容提供给其他使用者。允许用于模型训练则需要结合实际利用方式、适用法律和数据许可条件判断。这些概念之间具有联系，但不能简单等同。

著作权保护与多模态训练 文字、摄影作品、插图、音乐和视听作品等可能受到著作权及相关权利保护。在多模态模型训练中，数据往往需要经过下载、复制、格式转换、预处理和模型计算等步骤。因此，训练活动可能涉及受保护作品的复制或其他利用行为。例如，文本生成图像模型可能使用大量图文配对数据，其中的图片可能包含摄影作品、插画或其他具有独创性表达的内容。音频生成模型可能使用音乐作品及录音，涉及作品著作权、录音制作者权利或其他相关权益。

视频生成模型还可能同时涉及画面、音乐、对白和其他多种受保护内容。因此，多模态数据的权利关系可能比单一文本数据更加复杂。模型训练是否构成需要授权的特定使用行为，以及是否适用法定例外或限制，应当依据相关司法辖区的法律和具体事实判断。不能仅根据数据是否进入神经网络，就作出适用于所有地区的统一法律结论。

开放许可与具体使用条件 为了促进数据共享，部分数据集及其内容采用开放许可协议。例如，Creative Commons（CC）提供了多种作品共享许可。其中，不同许可元素可能具有不同要求：

- BY 表示署名要求。
- NC 表示限制商业性使用。
- SA 表示在相关条件下要求改编作品采用相同或兼容的许可。
- ND 表示对改编作品的分享设置限制。

例如，CC BY 与 CC BY-NC 对允许的使用范围具有不同规定。

但不能简单地认为所有 CC 许可条款都会以相同方式自动适用于模型训练。相关条款是否被触发，可能取决于训练活动涉及的版权权利、法定例外和具体利用方式。此外，一个数据集标注为开放许可，并不意味着其中每一张图像、每段音频或每条文本都必然拥有相同的授权状态。例如，数据集整理者可能有权许可自己编写的标注文件，却未必有权重新许可所有第三方原始图像。因此，训练数据的许可审查应当尽可能区分数据集整体的发布许可与底层内容的权利来源。

合成数据与衍生内容的许可问题 使用模型生成新的图片、音频或文本，并不意味着相关内容一定不再受到原始素材许可条件的影响。例如，图像编辑模型可能根据一张具有版权保护的原始摄影作品生成修改后的图像。如果生成结果仍然包含原始作品中受到保护的表达，就可能涉及相关权利问题。类似地，使用特定音乐录音或人物声音作为生成条件，也可能涉及录音、表演或其他权益。因此，合成数据的许可审查不仅需要考虑生成模型本身，还需要考虑其输入素材及生成结果的实际内容。

此外，由程序自动生成的图表或合成场景，也不意味着其数据来源、模板或其他组成部分必然不受许可约束。因此，应当为合成数据记录必要的生成来源，并根据实际使用方式确认其可用范围。

许可信息的记录与传播 在大规模数据工程中，数据经常经过多次整理、转换和再分发。例如，一个原始图文数据集可能被重新筛选，生成新的子集，再与其他数据集合并用于多模态模型训练。如果在这些过程中只保留图像和文本，而丢弃原始来源与许可信息，

就可能使后续使用者难以判断数据的合法使用范围。因此，可以为每个数据来源保存：原始数据集或内容来源。来源版本及获取时间。发布者或权利主体信息（如可获得）。许可协议及其链接。已知使用限制。相关授权或审查状态。

派生数据集与原始数据之间的关系。The Data Provenance Initiative 对大量公开训练数据集的来源、授权和再分发信息进行了系统研究，发现部分数据集存在许可信息缺失、分类错误或与原始来源不一致等问题。这一研究说明，数据治理需要关注完整的来源链，而不应只依赖最后一个下载平台展示的许可标签。因此，训练数据许可管理应当与数据版本和来源追踪共同设计。

9.6.2 隐私保护与敏感信息

多模态数据中的个人信息 多模态数据可能包含与特定个人相关的信息。例如，图像和视频可能记录人物面部、车辆牌照、住宅环境和活动地点。音频可能包含人物声音、姓名、联系方式或其他私人谈话内容。文本可能包含电子邮箱、电话号码、地址、证件信息和个人经历。传感器数据还可能包含连续位置轨迹、健康测量或其他行为记录。这些信息可能直接识别个人，也可能在与其他信息结合后实现间接识别。因此，多模态数据的隐私保护不能仅关注文本中的姓名和电话号码。

例如，即使删除语音转录文本中的姓名，原始声音仍然可能包含可用于识别说话者的特征。同样，即使删除图像中的文字信息，人物面部及背景环境也可能保留可识别线索。因此，应当根据实际数据内容和可识别风险判断是否涉及个人信息。

个人信息与敏感个人信息 个人信息与敏感个人信息需要区分。一般个人信息可以包含与已识别或可识别自然人有关的多种信息。敏感个人信息则通常涉及一旦泄露或被不当使用，可能对个人权益造成较严重影响的信息类型。例如，中国《个人信息保护法》第二十八条列举了生物识别、医疗健康、金融账户、行踪轨迹等敏感个人信息，以及不满十四周岁未成年人的个人信息。

欧盟 GDPR 也对特定类别的个人数据设置了更加严格的处理要求，例如为唯一识别自然人而处理的生物识别数据，以及健康相关数据等。不能仅因为数据包含人物照片或一般语音，就在所有法律场景下自动认定其属于同一种敏感个人信息类别。具体分类还取决于数据处理目的、识别方式及适用法律。因此，实际数据治理需要依据相应法律和使用场景进行分类，而不能只依赖文件类型作出判断。

数据最小化原则 数据最小化强调，根据实际任务收集和处理必要的数据，而不是因为技术上可以获取就保留全部信息。例如，一个研究项目如果只需要判断视频中是否存在行人，那么通常不需要额外建立行人的真实姓名和个人身份档案。类似地，如果语音识别任务主要研究文字转写，也不一定需要长期保存与任务无关的个人联系方式。因此，可以概念性地将数据保留决策表示为：

$$D_{\text{required}} \subseteq D_{\text{collected}} \tag{9.53}$$

其中，D_{required} 表示完成既定任务所需的信息集合。这一表达并不对应具体法律中

的计算公式，而是用于说明数据最小化的基本思想。最小化不意味着删除所有对当前模型暂时没有使用的信息。例如，对于需要可复现的多传感器研究，原始时间戳和必要标定信息可能具有长期价值。因此，需要结合研究目标、合法依据、保存期限和再利用条件，确定合理的数据保留范围。

隐私保护技术及其局限 在数据与模型训练过程中，可以采用多种隐私保护技术。例如，通过访问控制限制能够接触原始数据的人员和程序。通过加密保护数据传输和存储过程。通过删除或替换不必要的直接身份标识，降低部分泄露风险。通过适当的隐私增强训练技术，控制训练过程对个体样本的依赖。例如，差分隐私（Differential Privacy）提供了一类具有数学定义的隐私保护框架。在满足相应机制与参数设定时，可以限制单个样本对发布结果分布的影响。

但差分隐私不是简单地对数据添加随机噪声就能实现。有效的差分隐私保证需要明确邻接关系、隐私预算以及整个机制的隐私分析。此外，隐私保护技术还可能影响模型训练成本和任务表现。例如，过度删除语音特征可能影响说话人相关研究，强隐私约束也可能改变模型的学习效率。因此，隐私保护需要在合法性、风险程度、数据效用和实际计算预算之间进行合理设计。

个人信息删除与模型更新 数据治理还需要考虑个人信息的更正、删除和使用限制等要求。例如，当某项数据不再符合合法使用条件，或者相关个人依法提出有效请求时，数据管理者可能需要采取相应措施。但删除训练数据文件与消除模型已经学习到的相关信息，是两个不同问题。例如，某条数据已经参与模型预训练，即使从本地数据存储中删除原始文件，也不能直接推断模型参数中不再包含相应信息影响。

因此，对于已经完成训练的模型，可能需要结合适用法律、技术可行性和具体风险，评估是否需要重新训练、模型更新、机器遗忘或其他适当措施。相关技术目前仍存在有效性和可验证性方面的挑战，不应将某一种处理方法描述为能够普遍、彻底地消除所有训练影响。因此，个人信息治理需要同时考虑原始数据、派生数据、模型权重和下游应用等不同层次。

9.6.3 数据安全与可追溯性

数据安全的基本要求 数据安全主要关注数据在存储、处理、传输和使用过程中，能否防止未经授权的访问、泄露、篡改、破坏或不可用。对于多模态训练数据，这些风险可能发生在不同阶段。例如，网络下载的文件可能被替换，存储服务器可能发生权限配置错误，数据预处理脚本可能意外覆盖原始标注，训练日志也可能包含不应公开的个人信息。因此，多模态数据安全不仅是防止外部攻击，还包括防止内部误操作、处理错误和不受控制的数据传播。

从一般信息安全角度，可以围绕保密性、完整性和可用性三个方面设计数据保护措施。保密性强调限制未经授权的信息访问。完整性强调数据未被非预期方式修改。可用性则强调数据在需要时能够被可靠使用。这三个方面共同构成数据安全的重要基础。

数据分类分级与访问控制 不同数据来源和模态可能具有不同的安全风险。例如，一般公开且允许再分发的图像，与包含私人对话和精确位置轨迹的原始录制，不应采用完全相同的访问权限。因此，可以根据数据内容、敏感程度、使用条件和相关法律要求进行分类分级。例如，某些数据可以面向整个研究团队使用，而涉及个人敏感信息的原始数据可能只允许经过授权的人员访问。访问控制可以遵循最小权限原则，使用户和程序仅获得完成任务所需的数据访问能力。

例如，负责训练的程序可能只需要读取经过审批的训练数据，而不需要直接访问包含身份映射关系的原始标识表。但数据分类分级不能只根据文件后缀或数据集名称进行判断。相同的图像格式可能包含完全不同的敏感内容。因此，应当结合具体数据内容与使用场景制定访问策略。

数据完整性与文件校验 在大型多模态数据集中，文件损坏和非预期修改可能难以及时发现。例如，同一个图像文件可能经过重复下载，某个视频压缩包可能传输不完整，或某个标注文件在后续处理过程中被覆盖。因此，可以使用内容哈希对文件完整性进行校验。设原始文件字节序列为：B_i，其内容哈希为：$h_i = H(B_i)$，其中，H 为适当的密码学哈希函数。后续可以重新计算文件哈希，并检查是否与已保存的值一致。

但内容哈希只能证明文件内容是否与某个已记录版本一致，不能自动证明该文件本身来自可信来源。例如，如果初始下载文件已经被恶意替换，仅仅保存其哈希并不能发现其语义上的恶意内容。因此，文件完整性校验需要与来源验证和访问控制结合使用。

基于来源图的数据追踪 对于复杂的数据处理过程，可以使用有向图描述来源与派生关系。设：$G = (V, E)$，其中，节点 V 表示数据实体、处理活动或相关责任主体，边 E 表示使用、生成或派生等关系。例如，一条最终视觉问答数据可能来自：原始图像、人工对象标注、模型生成的问题。人工复核后的答案。这些内容具有不同来源与处理历史。W3C 的 PROV 数据模型及 PROV-O 本体，为记录实体、活动和主体之间的来源关系提供了标准化表达方式。

这一思路能够帮助数据工程区分原始来源、处理操作和最终产物。但建立了来源图，并不意味着数据自动具有合法授权或高质量标注。来源追踪主要提供核查和审计的基础，仍然需要结合版权、隐私和质量规则进行判断。

训练数据版本与模型检查点关联 多模态模型可能经过多个训练阶段。例如，首先使用大规模图文数据预训练，再通过视频和音频数据持续训练，最后使用多模态指令数据微调。如果每个阶段使用的数据版本不同，就需要记录相应模型检查点与训练数据之间的关联。例如，可以概念性地表示为：

$$M_j = F(M_{j-1}, \mathcal{D}_j, C_j) \tag{9.54}$$

其中：

- (M_{j-1}) 表示上一阶段模型；

- $(\mathcal{D}_j)$ 表示当前阶段数据版本；
- (C_j) 表示训练配置；
- (M_j) 表示当前阶段产生的模型检查点。

这一表达不是具体的优化算法公式，而是强调模型版本依赖数据版本和训练配置。

因此，记录模型参数文件本身并不足以完整复现实验。还需要能够确定所使用的数据集、清洗规则、训练划分和主要训练设置。同时，由于某些数据可能受到隐私或版权限制，数据可追溯不一定要求向所有外部使用者公开全部原始内容。可以在相应权限范围内保存必要的审计记录，并根据法律和数据使用条件提供适当透明度。

习题与讨论

1. 设计一条包含图像、区域和多轮问答的样本记录，说明媒体版本、坐标系及监督范围如何追溯。
2. 同一原视频截成多个片段后随机划分训练与测试，会支持怎样的泛化结论，又不能支持什么结论？
3. 两项任务的样本抽样率相同，目标长度却不同。比较按样本平均与按有效 Token 平均时的训练权重。
4. 为合成问答建立独立质检。至少覆盖视觉可回答性、答案正确性、重复与生成模型偏差。

第十章　预训练流程与优化

训练协议把目标函数变成可执行的参数更新。数据顺序、遮罩、损失分母、可训练参数与恢复状态共同决定实际优化过程。许多“同一模型、同一损失”的实验，在这些环节上并不相同。

学习目标　说明每阶段的数据、参数与损失位置；正确处理变长样本和梯度累积；估算资源并检验恢复；用监控信号定位数据或数值故障。

10.1　训练阶段与参数范围

阶段名称不能替代更新范围。每个阶段都应列出视觉编码器、连接器、语言主干和输出头是否更新，以及训练目标和数据的变化。冻结参数时可以不保存其参数梯度，但通向可训练输入的计算图仍可能需要保留。

10.1.1　预训练、对齐训练与指令微调的关系

多模态模型的阶段化训练　现代多模态基础模型通常不是通过一次完全相同的数据混合训练，直接获得全部能力。例如，一个视觉语言模型可能首先利用图像数据训练视觉编码器，利用文本数据训练语言模型，再通过图文配对数据学习跨模态连接，最后通过视觉问答和对话数据学习指令遵循。因此，可以将一个常见的训练过程概括为：

$$\text{基础预训练} \rightarrow \text{跨模态连接或对齐} \rightarrow \text{指令微调} \rightarrow \text{可选的后训练} \tag{10.1}$$

但这并不是全部多模态模型都必须遵循的固定流程。例如，CLIP[8] 直接通过大规模图文对比学习进行跨模态预训练，并不需要以大语言模型指令微调作为必要组成部分。Unified-IO 2 等统一模型还可以从头开展多模态训练，并通过大规模多任务微调学习不同技能。因此，不同训练阶段的划分，应当根据模型实际的参数初始化和训练目标确定。

跨模态对齐的不同含义　在多模态研究中，“对齐”（Alignment）具有多种含义，不能简单视为同一个训练阶段。第一种是表示空间对齐。例如，CLIP 通过图文对比目标，使相关图像和文本在嵌入空间中具有较高相似度。第二种是跨模态特征连接。例如，通过 Projector 或 Q-Former，将预训练视觉编码器的输出转换为语言模型能够利用的条件表示。第三种是时间或空间对应。

例如，在视频和音频联合预训练中，使模型能够利用具有相应时间关系的视听观测；对于多传感器任务，还可能需要结合时空标定信息。第四种是模型行为对齐。例

如，通过指令微调、偏好优化或其他后训练方法，使模型的输出更符合预定的交互和行为要求。这些过程虽然都可以使用“对齐”一词，但其目标、监督信号和评价指标不同。因此，在讨论多模态训练时，应当明确指的是跨模态表示对齐，还是模型输出行为的后训练对齐。

视觉特征连接训练 当语言模型已经具有较强的文本能力，而视觉编码器独立完成预训练后，模型需要建立视觉表示到语言条件之间的连接。例如：$Z_I = P_\psi(f_I(I))$，其中，P_ψ 表示需要训练的连接模块。如果语言模型参数保持冻结，可以通过图像描述等条件文本生成任务，训练连接模块，使视觉信息能够影响语言模型的输出。这一阶段的关键不只是使两组向量维度相同，而是建立能够被语言模型有效利用的视觉信息表示。

例如，将视觉特征经过线性变换映射到语言模型隐藏维度，只解决了张量维度兼容问题，并不保证模型已经正确理解输入图像。因此，连接训练需要具有实际语义监督，并通过下游任务评价其效果。

BLIP-2 与 LLaVA 的阶段差异 BLIP-2[12] 和 LLaVA[14] 都利用预训练视觉与语言模块，但其训练组织具有不同特点。BLIP-2 在冻结图像编码器和语言模型的条件下，通过 Q-Former 建立两阶段预训练。其第一阶段主要学习语言相关的视觉查询表示，第二阶段进一步建立视觉条件到语言生成之间的联系。

LLaVA 原始研究则首先通过图文数据训练视觉连接模块，使视觉特征能够进入语言模型；随后通过视觉指令数据进行微调，使模型学习更加丰富的问答、描述和对话行为。两者说明了一个共同问题：已有视觉编码能力和语言生成能力，不一定能够通过简单连接就形成有效的视觉语言交互。但它们对训练阶段的命名和参数更新策略并不完全相同。

例如，BLIP-2 的两阶段设计属于其特定的预训练方法，而 LLaVA 的后续视觉指令微调主要关注实际指令响应能力。因此，在分析模型训练流程时，应当依据具体论文中的训练目标和可训练模块，而不应仅根据“第一阶段”“第二阶段”等名称判断其作用。

持续预训练与领域适配 多模态模型完成基础预训练后，还可能需要适应特定领域。例如，通用视觉语言模型可能主要学习日常图片和常见文本，但医学影像、遥感图像、工业监测和专业视频等任务具有不同的数据特点。因此，可以通过持续预训练（Continued Pre-training），进一步利用特定领域的数据改善相应表示能力。例如，在已有视觉语言模型基础上，使用专业图文数据继续学习相关术语和视觉概念。

对于特殊传感器，还可以进一步引入新的模态编码器和跨模态训练目标。但持续预训练并不必然需要重新训练全部模型参数。例如，可以冻结部分已有编码器，仅更新新增模态模块或连接层。也可以根据数据规模和目标能力，选择适当的参数高效训练方式。持续预训练可能引入领域偏移和能力遗忘。因此，应当同时评价目标领域表现与原有通用能力，而不能只报告新增领域任务的改善。

不同训练阶段的参数更新 现代多模态模型的不同阶段，可能采用不同的参数冻结与更新策略。例如，视觉编码器可以保持冻结，语言模型可以保持冻结，也可以仅更新连接

模块。另一种方式是冻结视觉编码器，但更新语言模型和连接层。还可以对全部模块进行端到端联合训练。因此，训练阶段不仅需要记录数据和损失目标，还需要明确实际参与优化的参数集合。设完整模型参数为：

$$\theta = (\theta_V, \theta_P, \theta_L) \tag{10.2}$$

其中：

- (θ_V) 表示视觉编码器参数；
- (θ_P) 表示跨模态连接模块参数；
- (θ_L) 表示语言模型参数。

例如，在一种连接训练设置中：

$$\theta_V, \theta_L \text{ 冻结}, \qquad \theta_P \text{ 可训练} \tag{10.3}$$

而在某些指令微调设置中，则可能进一步更新：(θ_P, θ_L) 甚至全部参数。这些配置属于不同的工程和研究选择，不存在适用于全部多模态模型的固定最优方案。例如，冻结参数能够减少部分训练计算与存储开销，但可能限制模型适应新的任务表示；端到端训练提供更大的调整空间，却可能增加计算成本和能力遗忘风险。因此，应根据数据规模、训练资源和目标能力决定实际更新范围。

完整预训练流程中的数据与工程组织 从实际训练过程看，多模态预训练不仅包含损失函数，还需要完成数据、模型和训练协议之间的对应。需要确定目标能力与模态组合。例如，模型主要学习图文检索、视觉问答，还是进一步支持图像与音频生成。需要选择适当的数据来源和数据配比。例如，图文对比学习需要图文关联数据，而图像描述生成需要具有相应描述监督的数据。然后，需要确定模态表示和模型初始化方式。

例如，使用已有视觉编码器，还是从头训练视觉表示；使用离散视觉 Token，还是连续视觉特征。接下来，需要明确训练目标和参数更新范围。例如，采用对比损失、掩码建模、条件生成，或者多种目标的组合。随后，应建立独立评价协议，检查训练目标是否真正形成预期能力。例如，对比学习模型可以评价图文检索和零样本分类。图像条件语言模型可以评价图像描述和视觉问答。具有生成能力的模型还需要评价相应生成质量与条件一致性。

需要保留训练配置、数据版本和模型检查点，使实验结果能够被追踪和复现。因此，多模态预训练是一项将数据、表示、架构与优化目标共同组织起来的系统工程，而不是单独选择某个损失函数就能完成的过程。

10.1.2 冻结视觉编码器与语言模型

冻结视觉编码器与语言模型，可复用已有主干并缩小更新范围。需要明确冻结哪些层、归一化或随机模块是否保持训练状态，以及是否缓存视觉特征。缓存只有在预处理、增强与编码器状态相容时才能复用。冻结参数仍不等于切断通往连接器的梯度，

计算路径见第六章。

10.1.3 训练模态连接模块

连接训练使用图文描述或任务条件生成监督，损失位置与第六章一致。阶段协议应记录样本来源、视觉长度、语言模板、有效目标 Token 与参数分组。随机初始化连接器和充分预训练主干的优化状态不同，应分别检查梯度与学习率。

阶段验收包含原图、无图、错配及相关局部变化的对照。先确认模型输出对有效视觉信息敏感，再进入大规模训练；损失下降只说明当前目标被优化，不能替代能力验证。

10.1.4 视觉语言联合训练

联合训练允许损失同时更新连接器与部分或全部主干。具体范围由方案决定，“联合”不自动表示所有参数均可训练。与只训连接器比较时，应固定数据与预算，再把解冻范围作为变量。

输入可包括图文、纯文本和视觉指令等任务，混合权重影响能力保持。应同时检查目标视觉任务、原有语言能力及细粒度证据使用，防止把格式适应当作感知收益。

10.1.5 渐进式解冻与端到端训练

渐进式解冻先稳定接口，再逐步开放主干参数。它可能缓和新连接器对预训练表示的扰动，但并非普遍优于直接联合训练。数据域变化、标注规模与表示缺失决定需要更新哪一部分。

协议中应列明解冻时刻、层范围、分层学习率、优化器状态与调度是否重置。用固定预算消融验证解冻的作用，并检查原有能力是否退化。端到端描述的是损失到参数的优化路径，不能代替可训练模块清单。

10.2 任务混合与损失统计

算例：Token 加权的微批次损失

两个微批次分别有 n_1, n_2 个有效目标 Token，平均损失为 $\bar{\ell}_1, \bar{\ell}_2$。全局 Token 平均为

$$\bar{\ell} = \frac{n_1 \bar{\ell}_1 + n_2 \bar{\ell}_2}{n_1 + n_2}. \tag{10.4}$$

除非有效数量相等，$(\bar{\ell}_1 + \bar{\ell}_2)/2$ 对应另一种权重。跨设备和不完整累积窗口也要核对同一分母。数值正确的平均方式，必须与想优化的统计对象相符。

10.2.1 多任务训练与数据混合

多任务训练与多数据集训练 多任务学习（Multi-task Learning）通常指模型在训练中同时处理多个具有不同监督目标的任务，并通过共享参数学习可迁移的表示。例如，视觉语言模型可以同时学习图像描述、视觉问答和图文匹配。多数据集训练则强调使用多个数据来源，它们不一定对应不同任务：两个图像描述数据集可以共同服务于一个生成目标，同一个图文数据集也可以用于对比学习和条件文本生成。

因此，在设计训练方案时，应分别定义数据来源、任务类型、损失函数和参数更新范围。例如，一条带有图像与文字描述的记录可以进入图文对比任务，也可以经任务模板转换后进入图像描述任务；两者使用相同来源数据，但形成的梯度信号并不相同。

数据混合分布的定义 设共有 K 个任务或数据流，分别记为 $\mathcal{D}_1, \ldots, \mathcal{D}_K$。一种常见的数据混合方式是先按概率 p_k 选择数据流，再从该数据流内部采样：

$$\sum_{k=1}^{K} p_k = 1, \qquad P(k, x) = p_k q_k(x) \tag{10.5}$$

其中，q_k 表示第 k 个数据流内部的采样分布。若不同数据集按原始样本量合并并均匀抽样，则 $p_k = N_k / \sum_j N_j$。这一方式便于实现，但会使规模巨大的数据来源占据较多更新机会。另一种方式是先为不同任务设置固定概率，例如适当提高高质量但规模较小的视觉推理样本的参与频率。对数据规模差异较大的任务，还可以采用幂次平滑：

$$p_k = \frac{w_k N_k^{\alpha}}{\sum_j w_j N_j^{\alpha}} \tag{10.6}$$

其中，w_k 表示任务偏好权重，$\alpha = 1$ 且权重相同时近似按规模分配，$\alpha = 0$ 且权重相同时为数据流均衡采样。此表达仅代表一类采样策略，不存在对所有视觉语言模型都最优的统一指数或权重。对小数据集过度采样可能增加样本重复和过拟合风险，因此应记录样本的实际使用次数。

样本占比、Token 占比与计算占比 多模态样本的“长度”并不一致。一条单图问答可能包含一张图像和较短文本，而长视频问答可能包含数十帧、较长提示和回答；音频样本也会随录音时长增加输入表示长度。因此，某任务的样本抽样比例并不等于其在训练中的 Token 比例，更不等于计算预算比例。设任务 k 每次抽样平均产生 $\bar{n}_k$ 个参与模型计算的 Token，在一定简化条件下，其预期 Token 占比可近似为：

$$r_k \approx \frac{p_k \bar{n}_k}{\sum_j p_j \bar{n}_j} \tag{10.7}$$

若采用可变分辨率、序列打包、动态裁剪或稀疏注意力，上述关系还需要根据实际处理流程修正。视觉 Token 与文本 Token 的计算代价也未必相同。对于包含视频编码器和语言模型的系统，记录每种任务的 GPU 时间、有效 Token 数和优化器更新次数，往往比仅报告样本比例更有解释力。

同批混合与交替任务更新 多任务训练可以在同一个训练批次中混合不同任务，也可以按一定概率在不同优化步骤之间交替切换任务。前者可能便于利用共享计算，但需要处理不同模态的输入长度、填充和损失掩码；后者便于为不同任务使用独立的数据加载器和预处理流程，但单次梯度更新可能主要反映某一种任务。

例如，图文对比学习常需要一个批次内的多个图文对来构造相似度矩阵，而图像描述任务可以对目标文本进行逐 Token 的监督。简单地将两个任务的数据放入同一批次，并不能保证其损失定义和样本权重得到正确处理。因此，在工程实现中，应首先为各任务明确有效监督位置，再决定采用混合批次、交替批次还是梯度累积。

代表性研究与训练数据的互补性 MM1 系统比较了图像描述配对、图文交错文档和纯文本等数据来源，指出在其大规模多模态预训练设置下，不同类型数据的合理组合对少样本任务能力十分重要。VILA 则通过受控实验发现，在其模型中，图文交错预训练数据有助于能力形成，而在视觉语言指令微调阶段重新加入纯文本指令数据，有助于缓解文本任务退化。上述结果说明，数据混合不是将更多样本简单拼接，而是在不同能力目标之间分配学习机会。但这些研究中的具体配比属于特定架构和预算下的实验结果，不能直接作为其他模型的通用最佳比例。

数据混合的验证原则 在检验混合比例时，应预先明确研究问题。例如，要比较“增加视频数据是否改善时间理解”，就需要控制视频预处理、训练预算以及静态视觉数据的其他设置。若更换混合比例的同时增加总 Token 数和训练步数，则性能变化不能单独归因于任务占比。除了总体分数，还应报告各模态任务的变化、常见与长尾子集表现，以及纯文本等已有能力是否保持。

10.2.2 训练阶段与课程设计

阶段化训练的基本逻辑 多模态模型往往需要协调已有单模态能力与新增跨模态任务。一个常见的候选流程是：先利用已有视觉编码器和语言模型初始化；通过图文数据建立连接模块；再进行更广泛的图文联合学习；最后使用高质量问答和指令数据调整输出行为。但这不是所有多模态模型必须依次经历的四个步骤。有些模型从头联合训练多模态参数，有些模型全程冻结部分主干，还有些模型直接在已有基础模型上进行多模态指令训练。

训练阶段真正需要说明的是四项内容：阶段目标、数据分布、可训练参数集合及进入下一阶段的验证条件。例如，连接模块预热阶段主要检查视觉条件是否影响语言生成，而后续指令阶段主要检查模型是否能根据具体问题选择视觉信息。若前一阶段没有建立有效的视觉依赖，仅延长后续对话训练未必能够弥补这一问题。

持续预训练与指令微调 持续预训练通常利用更大范围的多模态数据进一步学习表示或生成能力；指令微调则使用带有明确任务要求和目标输出的数据，使模型学习具体的响应方式。在前者中，可以较多使用图文配对、交错文档和视频—文本样本；在后者中，可以使用对象识别、OCR、图表理解、视频问答和多轮会话等数据。

但“预训练数据一定低质量、微调数据一定高质量”并不准确。高质量数据也可

以用于预训练，弱监督指令数据同样可能出现在微调中。训练阶段的区别，主要来自学习目标及数据监督形式，而不只是数据质量等级。

课程学习的含义 课程学习（Curriculum Learning）根据样本难度、任务结构或输入复杂度安排训练顺序。Bengio 等人的经典研究提出，具有一定组织规则的样本呈现顺序可能改善优化和泛化。多模态场景中，可以考虑先训练基础描述，再增加需要细粒度关系的问答；也可以先使用较短视频片段，再逐渐提高时间长度。

但课程学习不是“先训练简单数据，再训练所有困难数据”的固定公式。较难的真实样本与低质量噪声样本应当区分，训练早期的困难样本也可能提供有价值的监督。如果任务目标是连续事件理解或专业传感器融合，长期只使用静态图文数据甚至可能不利于早期形成必要的时序表示。因此，课程设计必须依据目标能力与任务依赖关系，而非凭经验简单排序。

随时间变化的采样分布 可将阶段化训练与课程学习统一表示为随训练进度 t 变化的数据流采样概率：

$$p_k = p_k(t), \qquad \sum_k p_k(t) = 1 \tag{10.8}$$

例如，在第一阶段提高图文配对的比例，第二阶段逐渐增加图文交错数据，第三阶段提高视觉问答和复杂推理数据的比例。其变化可以采用分段固定配比，也可以连续变化。实际训练应明确阶段的起止更新步数、累计处理 Token 数以及每个阶段的数据版本，避免只用“训练若干 epoch”描述来自不同规模数据池的过程。

训练阶段中的参数与数据协同变化 阶段切换通常还会伴随参数更新范围变化。例如，连接层预热时冻结视觉编码器和语言模型，后续再更新语言模型或视觉主干。但数据分布与参数冻结范围同时改变时，实验结果难以直接归因。因此，在研究阶段设计作用时，最好分别比较数据配比变化与参数解冻的影响。BLIP-2 的两阶段桥接训练和 VILA 的投影初始化、交错数据预训练及联合监督微调提供了有价值的实例，但它们对应的目标和更新参数并不完全相同。

阶段转换与能力保持 切换到某个新任务后，模型可能在目标任务上改善，却在已有能力上退化。例如，集中训练短回答式视觉问答，可能改变模型原有的文本表达能力；引入长视频后，短图像任务的实际训练频率可能下降。因此，每一阶段不仅应监控新增能力，还需要评价基础视觉语义、纯文本能力及此前已达成的任务表现。必要时可以保留一定比例的旧任务数据，或限制部分参数的更新，但这些措施也需要通过公平实验验证。

10.2.3 损失权重与训练稳定性

多目标优化的基本形式 多模态预训练可能同时包含图文对比（ITC）、图文匹配（ITM）、条件文本生成、掩码重建或其他任务损失。在同一批次中计算多个目标时，可以采用：

$$\mathcal{L}_{\text{total}} = \sum_{k=1}^{K} \lambda_k \mathcal{L}_k \tag{10.9}$$

其中，λ_k 为损失权重。与 12.4.1 中定义的采样概率 p_k 不同，λ_k 作用于已经参与训练的任务损失。若按概率 p_k 交替选择任务，期望梯度会受到采样分布与任务权重的共同影响。但由于梯度方差、优化器状态、归一化方式和批次结构不同，相同的 $p_k\lambda_k$ 乘积并不保证训练动态相同。

损失尺度不等于任务重要性 不同训练目标的原始数值并不可直接比较。例如，图文匹配的二元交叉熵、文本 Token 交叉熵和视觉特征均方误差可能具有不同的定义域与平均方式。一个任务损失数值较大，可能只是因为目标的单位或归一化方式不同，并不一定意味着这个任务更困难、监督更可靠或更值得增加权重。

因此，设计损失权重时首先需要统一或明确归一化规则，例如文本损失按有效预测 Token 平均，图像重建损失按有效目标像素或 Patch 平均，跨模态匹配损失按有效配对数量平均。随后再根据目标任务表现调整权重，而不能直接用原始损失大小分配采样概率。

梯度范数与优化竞争 对于共享参数 θ_s，可以计算各任务的梯度：

$$g_k = \nabla_{\theta_s}(\lambda_k \mathcal{L}_k) \tag{10.10}$$

若某个任务的梯度范数长期远大于其他任务，它可能在共享参数更新中占据较大比重；但范数大小并不直接决定对下游任务是否有益。进一步可以计算两个任务梯度的余弦相似度：

$$\cos(g_i, g_j) = \frac{g_i^\top g_j}{\|g_i\|_2 \|g_j\|_2} \tag{10.11}$$

负的余弦相似度表明在当前参数与批次上更新方向存在冲突，但不能仅凭一次负值就断定联合训练已经导致负迁移。更充分的分析还需要结合梯度大小、任务损失曲率、数据质量以及不同任务的实际验证表现。

梯度冲突处理方法 PCGrad（Projected Conflicting Gradients）是针对多任务梯度相互干扰提出的一种方法。其基本思想是，当两个任务梯度方向冲突时，去除一个任务梯度沿另一个梯度的冲突分量，再综合用于更新。其研究指出，梯度冲突本身并不必然有害，但在特定的梯度幅度和优化曲率条件下可能放大训练问题。

这类方法通常要求能够获得不同任务在共享参数上的梯度，因此会增加计算、存储或反向传播复杂度。对于大规模视觉语言模型，并不应该因为存在多个损失就默认采用梯度投影。更稳妥的顺序是：先确保数据标签和损失实现正确，再使用固定权重建立稳定基线，最后在发现可重复的任务竞争时评估是否需要动态权重或梯度干预。

梯度裁剪与异常更新 梯度裁剪（Gradient Clipping）是控制异常大更新的常见方法。以全局范数裁剪为例：

$$\tilde{g} = g \cdot \min\left(1, \frac{c}{\|g\|_2 + \epsilon}\right) \tag{10.12}$$

其中，c 为裁剪阈值。该方法能够限制梯度范数过大带来的单步更新风险，但不能修正错误标签、跨模态错位或不合理的损失定义。若使用 FP16 损失缩放，应在梯度反缩放后再执行相应裁剪；若采用梯度累积，则通常应对累计后的梯度进行符合目标归一化的裁剪。

10.3 优化器与有效批量

10.3.1 学习率、批量大小与优化器

优化参数是训练协议的重要组成部分 训练目标决定模型如何衡量预测误差，而优化器决定如何利用梯度更新参数。对于视觉语言模型，视觉编码器、模态连接模块和语言模型可能具有不同的预训练程度、参数规模和梯度统计特性，因此不能只使用“统一学习率”四个字概括优化设置。除了学习率，还需要明确优化器类型、权重衰减、动量参数、学习率调度、有效批量大小和梯度裁剪等配置。

这些设置会影响模型是否稳定收敛以及训练结果是否可复现。特别是在跨实验比较时，如果数据混合方案改变的同时学习率和有效批量大小也被调整，就难以独立判断数据策略的作用。

学习率与参数更新 以简单的梯度下降为例，参数更新可表示为：

$$\theta_{t+1} = \theta_t - \eta_t \nabla_\theta \mathcal{L}_t \tag{10.13}$$

其中，η_t 为第 t 次优化器更新时的学习率。学习率过大可能使优化过程震荡或不稳定，过小则可能导致训练进展缓慢。对于多模态模型，连接模块可能从随机初始化开始训练，而视觉编码器和语言模型已有预训练权重，因此不同模块不一定适合相同的学习率。

例如，可为视觉编码器、连接模块与语言模型分别设置 η_V、η_P 和 η_L。在某些训练方案中，新初始化的连接模块采用相对较大的学习率，预训练主干采用较小的学习率。但这并非固定规律：模块初始化、是否冻结、数据域变化和优化器类型都可能改变适合的取值。实际研究应依据明确的验证指标及训练稳定性选择，而不宜将某篇论文的数字无条件迁移。

学习率预热与衰减 预热（Warmup）通常用于在训练初期逐步增大学习率，降低模型在初始化或训练阶段切换时发生剧烈参数更新的风险。若总更新步数为 T，预热持续 T_w 步，一种线性预热与余弦衰减的示意形式为：

$$\eta_t = \begin{cases} \eta_{\max}\, t/T_w, & 0 \le t < T_w, \\ \eta_{\min} + \dfrac{\eta_{\max} - \eta_{\min}}{2}\left[1 + \cos\left(\dfrac{\pi(t - T_w)}{T - T_w}\right)\right], & T_w \le t \le T. \end{cases} \tag{10.14}$$

这一公式仅代表常见调度形式之一。实际模型也可能采用常数学习率、分段衰减、逆平方根衰减或重新开始的调度周期。对已经进行充分预训练的模块，还需要判断阶段切换时应继承还是重置学习率调度及优化器状态。所谓“继续训练”并不意味着在所有情形下都应保留之前阶段的相同优化状态。

梯度累积与大批量训练 梯度累积（Gradient Accumulation）允许在显存不足以容纳大批量时，将多个微批量的梯度累积后再执行一次优化器更新。对于样本级可加的损失，在正确归一化、数据顺序一致、模块状态和随机过程适当控制的条件下，梯度累积可以近似模拟较大的批量。但它并不与真正一次性大批量计算在所有情况下严格等价。

例如，使用批归一化时，微批量统计可能与大批量统计不同；对比学习如果依赖批内负样本，分别处理多个小批次并累积梯度，并不会自动产生与单个大批次相同的跨样本相似度矩阵。对于变长文本损失，如果简单地对每个微批量的平均损失再等权累加，还可能导致短序列和长序列的有效 Token 权重不同。因此，在多任务训练中，应明确损失究竟按样本还是按有效 Token 归一化。

优化器与 AdamW 在 Transformer 及其多模态扩展模型中，AdamW 是常用优化器之一。它在 Adam 自适应梯度更新的基础上，将权重衰减与梯度更新解耦。其一种概念性更新形式为：

$$\theta_{t+1} = (1 - \eta_t \lambda_{\text{wd}})\theta_t - \eta_t \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \tag{10.15}$$

其中，$\hat{m}_t$ 和 $\hat{v}_t$ 为偏差修正后的梯度一阶、二阶矩估计，λ_{wd} 为权重衰减系数。Loshchilov 与 Hutter 的 AdamW 研究指出，常见自适应优化器中的 L_2 正则化和解耦权重衰减并不等价。

实际训练还可能采用 SGD、Adafactor 或其他优化器，选择需结合模型规模、内存需求和既有实验协议。此外，偏置和归一化参数是否应用权重衰减，应通过明确的参数分组配置记录，而不是只报告优化器名称。

优化器状态、冻结策略与复现 多模态训练常包含连接层预热、部分参数解冻与完整联合训练。当可训练参数集合发生变化时，优化器参数分组、动量状态、学习率调度和梯度裁剪范围也可能需要改变。例如，原先冻结的视觉编码器在新阶段加入优化器后，其参数是否使用新初始化的优化器状态，应在训练协议中明确。

对于断点恢复，除模型权重外，通常还应保存优化器状态、学习率调度器、梯度缩放状态（如适用）、随机数状态和数据采样进度。仅恢复模型权重但重新开始优化器及采样流程，通常不能被视为严格意义上的完整训练续跑。因此，训练策略的可复现性与数据版本和工程记录直接相关。

10.4 训练资源、规模与分布式优化

10.4.1 GPU 显存与计算量估算

训练显存的主要构成 GPU 显存不仅用于保存模型权重，还需要容纳梯度、优化器状态、前向激活、临时算子工作区及通信缓冲区。可将峰值显存概念性表示为：

$$M_{\text{peak}} \approx M_{\text{param}} + M_{\text{grad}} + M_{\text{optim}} + M_{\text{act}} + M_{\text{temp}} + M_{\text{comm}} \tag{10.16}$$

其中，参数、梯度与优化器状态主要受可训练参数量、精度和分片方式影响；激活存储随网络深度、微批量大小和输入序列长度变化；临时工作区及通信缓冲区则与算子实现、并行方案和内存分配器有关。这是预算分解式，而不是严格可加的峰值恒等式：不同内存对象可能在不同时间存在，真实峰值必须以运行测量为准。

应特别区分总参数量与可训练参数量。冻结视觉编码器能够减少对应参数的梯度与优化器状态，但其前向计算和输出特征仍占用资源。若语言模型冻结而连接模块仍需从其生成损失获得梯度，反向传播仍可能穿过冻结语言主干以计算对视觉条件的梯度，因此不能认为“冻结了参数就完全没有训练激活开销”。

参数与优化器状态的粗略估算 对于具有 P 个可训练参数的模型，采用 BF16 参数与梯度、FP32 Adam 一阶及二阶动量的某种实现时，理论上每个参数可能涉及约 2 字节权重、2 字节梯度和 8 字节动量状态；如果另存一份 FP32 主权重，还可能增加约 4 字节。因此，可用每参数约 12—16 字节作为特定传统配置的初步量级估算，而不能作为所有混合精度训练的固定常数。实际实现可能保留 FP32 原始参数、使用低精度优化器、压缩或分片状态，并产生额外临时缓冲。

例如，假设某一示例模型具有 70 亿可训练参数，仅按照上述 12—16 字节粗略预算，参数、梯度及优化器状态的总量就约为 84—112 GB（十进制），尚未计入激活、临时工作区与通信内存。这并不意味着每张 GPU 必须独立放下所有状态：FSDP、ZeRO 等分片技术可使状态分布在多卡上。该示例也不能直接视为任何特定 7B 模型的实测显存要求。

由此可见，决定显存需求的不只是模型参数规模，还包括哪些模块可训练，以及所用优化器与并行策略如何实际存放状态。

视觉 Token 与序列长度 视觉语言模型的输入成本还受视觉 Token 数量影响。对于一张尺寸为 $H \times W$ 的图像，若使用边长为 P 的不重叠 Patch，且尺寸恰好可整除，Patch 数近似为：

$$N_{\text{patch}} = \frac{H}{P}\frac{W}{P} \tag{10.17}$$

若存在额外类别 Token、多尺度特征、裁剪分块或 Token 压缩，则实际数量需要按模型的具体编码方式计算。对于具有 F 帧的视频，若逐帧采用相同的 Patch 编码而不进行时空压缩，其视觉 Token 数可能随帧数近似线性增长。若模型使用视频专用的

时空 Patch 或抽帧策略，则不能直接采用逐帧乘法。

对传统全注意力而言，注意力分数计算及其相关算术复杂度通常包含随序列长度 L 近似二次增长的项 $O(L^2)$。因此，增加图像分辨率、视频帧数或文本长度，会同时改变前向计算和激活需求。FlashAttention[19] 通过分块及片上存储优化减少高带宽显存读写，并避免以常规方式物化完整注意力矩阵，但它并没有把标准稠密注意力的全部计算自动变为线性复杂度。由此，降低显存占用与降低算术量是两个不同的优化目标。

计算量与实际训练速度 在某些简化的稠密 Transformer 语言模型设置中，可以使用“每训练 Token 约需 $6P$ 次浮点运算”作为参数矩阵计算的粗略量级估算，其中 P 是模型参数量。该估算有特定适用条件，未充分包含长序列注意力、多模态编码器、嵌入层、重计算及复杂并行通信等额外成本，不宜直接套用于所有视觉语言系统。

实际训练效率应使用可复现的测量指标，例如每秒有效训练 Token 数、每秒样本数、每次优化器更新耗时、GPU 峰值显存及通信时间。尤其需要区分数据加载耗时、GPU 计算耗时和模型检查点写入耗时。理论 FLOPs 较低的方案并不必然具有更短的运行时间，因为内存带宽、核函数调用和跨卡通信都可能成为瓶颈。

建议先在真实长度分布的代表性小批量上进行前向、反向和优化器更新测试，再扩大批次与卡数；不要只用短文本、低分辨率图片完成一次运行，就据此推断完整训练预算。

10.4.2 混合精度与梯度累积

混合精度的机制 混合精度（Mixed Precision）是指根据不同运算的数值需求，组合使用不同精度的数据表示和计算。FP32 通常提供较宽的数值精度；FP16 和 BF16 能降低某些张量的存储需求，并在支持的 GPU 上利用低精度矩阵运算单元提高吞吐。

FP16 具有较高的尾数精度但指数范围较窄，某些梯度可能发生下溢或中间值溢出；BF16 指数范围与 FP32 接近，但尾数精度较低。因此，不能简单理解为 BF16 在所有情况下都比 FP16 更准确，或所有计算都适合低精度执行。

自动混合精度（Automatic Mixed Precision，AMP）通常对适合低精度的矩阵运算进行转换，同时在必要的归一化、归约或损失计算中保留更合适的数值精度。实际效果取决于硬件、框架和算子实现。对于含有多种模态的模型，应特别检查自定义注意力、事件累积、几何变换或其他特殊算子在低精度下的数值行为。

损失缩放与数值稳定 在 FP16 训练中，较小梯度可能由于可表示范围限制而下溢。动态损失缩放（Dynamic Loss Scaling）的基本做法是先将损失乘以缩放系数 s：

$$\mathcal{L}' = s\mathcal{L}, \qquad g' = s\nabla_\theta \mathcal{L} \tag{10.18}$$

反向传播后，再在更新前将梯度恢复到原尺度，并根据 Inf、NaN 检测决定是否跳过本次更新。PyTorch 中的 GradScaler 提供了相应机制。对于常见的 BF16 自动混合精度训练，通常不需要为防止梯度下溢而使用与 FP16 相同的 GradScaler 流程，但仍应检测非有限数值和训练稳定性。

梯度裁剪应作用于正确尺度的梯度。例如，当使用 FP16 GradScaler 时，应在完成整个有效批次的累积之后先进行 unscale，再计算梯度范数并裁剪，最后执行优化器更新。过早 unscale 或在同一累积周期内反复改变缩放系数，可能破坏梯度的正确合成。

梯度累积与有效批量大小 梯度累积（Gradient Accumulation）通过连续处理多个微批次、累积其梯度，再执行一次优化器更新，降低单次前向和反向所需的激活峰值。若每卡微批量为 B_μ，累积步数为 A，数据并行副本数为 D，且各副本使用相同样本量，则有效全局批量为：

$$B_{\text{effective}} = B_\mu AD \tag{10.19}$$

例如，每卡微批量为 2，使用 4 张数据并行 GPU，累积 8 次，则有效批量为 64 个样本。这里的“样本”未必具有相同的 Token 长度，因此多模态训练还应记录每次更新实际处理的有效 Token 数。

在所有微批次样本及损失归一化权重一致时，可以将每个微批次损失除以累积次数 A，再执行反向传播。但对于长短不一的图文及视频序列，若每个微批次分别计算 Token 平均损失后再简单平均，结果可能不等于对整个有效批次全部有效 Token 求平均。此时应根据目标定义，对有效 Token 数或相应权重进行正确归一化。

梯度累积的局限 梯度累积主要缓解的是单次微批次激活显存，并不自动减少模型权重、梯度和优化器状态的总量。对于单卡无法容纳模型及其优化器状态的情况，仅提高累积步数不能解决问题，还需要考虑模型分片、优化器分片或其他内存技术。

梯度累积与直接使用同样大的物理批量也不一定完全等价。差异可能来自依赖批内统计的运算、随机数状态、样本长度归一化、优化器更新时机以及浮点数归约顺序。分布式训练时，还应控制不必要的梯度同步，例如 PyTorch DDP 的 no_sync 可以在部分累积步骤暂缓同步；文档要求其上下文同时覆盖相应的前向与反向过程。

激活重计算与内存—速度权衡 激活检查点（Activation Checkpointing，也称梯度检查点）是一种以额外计算换取显存的技术。常规训练会保存部分前向激活以供反向传播使用，激活重计算则在前向阶段保存较少中间结果，并在反向阶段重新执行指定网络段的部分计算。

其重要特点是：它能够降低激活显存，但增加额外前向计算，不应与保存到磁盘的训练检查点混淆。实际实现还需要妥善处理 Dropout 等随机算子的随机状态，以及有状态模块在重计算时的行为，避免前后执行不一致。

10.4.3 分布式训练与并行策略

为什么需要多 GPU 训练 当模型参数、优化器状态及激活需求超出单卡容量，或单卡训练时间无法满足预算时，可以使用多 GPU 并行。常见方案包括数据并行、模型参数分片、张量并行、流水线并行以及它们的组合。应当区分两个目标：一是让单个大模型的训练状态能够放入多卡总内存，二是通过并行缩短训练耗时。某种并行方案可能

更节省显存，却带来额外通信；另一种方案可能易于扩展吞吐，但无法解决单卡模型容量限制。

数据并行与 DDP 数据并行（Data Parallelism）使不同设备处理不同样本，并通过同步梯度维持模型副本一致。在典型的 DistributedDataParallel（DDP）中，每张 GPU 保存一份模型参数，独立执行前向和反向，再通过集合通信对梯度进行归约。假设所有设备初始参数一致，且在同一优化器步使用正确的梯度聚合与更新规则，则模型副本可以持续保持一致。对于参数能够放入单卡的模型，DDP 是相对直接的多卡加速方式。

但 DDP 通常并不分片参数和 Adam 优化器状态；其主要优势在于增加数据并行吞吐。随着设备数增加，数据加载、梯度同步及较小微批次带来的效率问题可能限制加速比。因此，8 张 GPU 并不必然能使训练时间降低到单卡的八分之一。

ZeRO 与 FSDP 的状态分片 为降低数据并行训练中的冗余显存，可以分片存储优化器状态、梯度和模型参数。ZeRO（Zero Redundancy Optimizer）将这类思想系统化，按照所分片的训练状态形成不同阶段的内存优化方法。

PyTorch Fully Sharded Data Parallel（FSDP）同样能够在数据并行设备间分片参数、梯度和优化器状态。以 FSDP2 为例，模型层的参数可以在使用前按需要进行 all-gather，在反向阶段通过 reduce-scatter 等操作完成相应通信，从而降低参数副本长期驻留所需的显存。

但是，分片不代表计算阶段永远只需持有参数分片。模型运行时可能临时收集完整层的参数，峰值显存还受到模块包装粒度、预取、重分片策略和通信缓冲影响。因此，不能将全部训练状态简单除以 GPU 数量并认定所得结果就是实际单卡峰值。

张量并行与流水线并行 张量并行（Tensor Parallelism）将同一网络层的矩阵或其他计算沿特定维度划分到多台设备，使多个 GPU 共同完成单个层的计算。例如，大型线性层可以进行列或行维度分片，但不同分片之间通常需要适当的通信操作。流水线并行（Pipeline Parallelism）则将模型不同层组分配给不同设备，并通过微批次组织前向和反向过程。其主要限制包括跨阶段激活传输、负载不均以及流水线空泡（设备在某些阶段处于等待状态）。

Megatron-LM 系列工作展示了张量并行、流水线并行和数据并行的组合训练思想。但对于规模较小、能够使用 DDP 或 FSDP 高效训练的模型，并不必然需要引入全部并行维度。并行策略越复杂，对通信拓扑、数值一致性和检查点管理的要求通常也越高。

多模态模型的并行划分 视觉语言模型往往具有异构组件，例如视觉编码器、投影模块和语言主干。不同组件的计算形态和参数规模不同，因此可以按实际瓶颈选择策略。例如，视觉编码器若保持冻结，可考虑对确定性图像特征进行离线缓存；语言模型的主要计算可能采用数据并行或参数分片；对于长视频输入，还应重点检查视觉 Token 数量和跨模态注意力开销。

但不同并行策略会影响训练语义。必须明确数据并行组、张量并行组与流水线组

的大小，以及有效批次究竟按哪个并行维度计算。若同时使用数据并行与张量并行，有效全局批量通常只按数据并行副本数计算，而不是将全部 GPU 数无条件相乘。

通信、吞吐与扩展效率 为评价多 GPU 的实际效果，可以记录固定任务和精度下的加速比：

$$\boxed{S_D = \frac{T_1}{T_D}}, \qquad \boxed{E_D = \frac{S_D}{D}} \tag{10.20}$$

其中，T_1 和 T_D 分别为单卡和 D 卡完成相同工作量的耗时。该指标适用于具备可比较单卡基线的场景；如果大模型无法放入单卡，则可以使用能够运行的最小设备规模作为参照，并明确说明比较基准。

训练日志应记录实际 Token/s、每次更新耗时、GPU 峰值显存和通信开销。对多模态批次而言，还应防止各进程数据加载和序列长度差异引起等待。若只有某个进程读取到损坏文件或提前耗尽数据，其他进程可能在集合通信中长时间阻塞。因此，多卡故障排查需要结合所有 rank 的日志，而不能仅观察主进程。

10.4.4 Scaling Laws 的基本思想与适用边界

幂律关系的机制 在某些模型家族和实验条件下，研究者观察到模型预测损失随参数规模或数据规模增加而呈现近似幂律下降趋势。一种简化的表达方式为：

$$L(N) = L_\infty + AN^{-\alpha} \tag{10.21}$$

其中：

- $L(N)$ 表示模型在特定数据分布上的预测损失。
- N 表示模型参数量。
- L_∞ 表示在该拟合形式中的极限损失项。
- A 和 α 是通过实验拟合的参数。

当：$A > 0, \quad \alpha > 0$ 时，随着 N 增大，模型的可降低损失项 $AN^{-\alpha}$ 会逐渐减小。类似地，也可以考虑损失与训练 Token 数的关系：

$$L(D) = L_\infty + BD^{-\beta} \tag{10.22}$$

其中，D 表示训练数据规模，B 和 β 是拟合参数。这类关系说明：在相关假设与有效范围内，增加模型参数或训练数据通常能够降低预测损失，但收益可能逐渐减小。不同论文所使用的 Scaling Law 数学形式、实验条件和拟合参数可能不同，不能直接将某一组拟合系数作为所有模型的共同规律。

参数规模与数据规模的联合关系 如果同时考虑参数规模和数据规模，可以构建一种常见的经验拟合形式：

$$E + \frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}} \tag{10.23}$$

其中：

- E 表示拟合模型中的不可约损失项或极限损失项。
- A, B 是拟合系数。
- α, β 表示参数规模和数据规模对应的幂律指数。

这一公式可以帮助理解两个重要问题。当训练数据规模固定时，不断增加模型参数，可能会逐渐受到数据规模的限制。当模型参数规模固定时，不断增加训练数据，也可能出现收益递减。因此，要获得更好的模型性能，通常需要合理协调参数规模与训练数据规模，而不是无限增加其中一个因素。不过，该公式是对特定实验条件的一种经验模型，不应被理解为适用于所有模型、所有数据分布和所有任务的精确关系。

Chinchilla 与计算最优训练 2022 年，Hoffmann 等人在《Training Compute-Optimal Large Language Models》中进一步研究了固定训练计算预算下，模型参数量与训练 Token 数应该如何分配。这项研究常被称为 Chinchilla Scaling Laws。它的重要结论之一是：在其分析的条件与模型范围内，计算最优训练不仅需要增加模型参数，也需要相应增加训练数据规模。

研究发现，固定计算预算下，模型规模与训练数据规模的平衡会显著影响最终语言建模效果。其中一个具有代表性的结果，是 Chinchilla 模型采用约 70B 参数，并使用约 1.4T Token 进行训练，在相近训练计算预算下取得了优于更大但训练 Token 较少模型的结果。由此可以得到一个直观认识：

一个参数量更大的模型，如果训练数据不足，未必优于参数量较小但训练更加充分的模型。

这一研究推动了业界更加重视训练 Token 数与参数规模之间的合理配比，而不是仅以参数数量衡量模型的发展程度。

为什么经常提到每参数约 20 个 Token 在介绍 Chinchilla 时，经常可以看到“每个参数约对应 20 个训练 Token”的经验比例。例如，对于一个 7B 参数的模型，如果采用这一数量级进行粗略参考，那么对应训练数据规模约为：$D \approx 20N$，$D \approx 20 \times 7 \times 10^9$，$D \approx 140B$ Tokens，这里的 140B 表示约 1400 亿 Token。

但

20 Tokens/Parameter 不是所有模型都必须遵守的固定比例，也不是充分训练的唯一判定标准。

这一数量级来自特定研究背景下的经验分析。真实最优配比可能受到模型架构、数据质量、训练目标、优化策略和计算预算影响。例如，在需要长期服务大量用户的场景下，研究者可能希望通过训练相对较小但更加充分的模型，在保持目标性能的同时降低长期推理成本。因此，训练计算最优和完整生命周期成本最优，可能不是同一个问题。

固定计算预算下如何权衡模型与数据 根据前面介绍的近似公式：$C \approx 6ND$，在训练计算预算 C 固定时，模型参数量 N 与训练 Token 数 D 不能同时无限增加。例如，如果参数量增加一倍，而总训练计算预算保持不变，那么根据这一近似关系，可使用的训练 Token 数需要相应减少。反过来，如果希望使用更多训练 Token，就可能需要减少模型参数量，或者增加总计算预算。

因此，研究者需要寻找适合目标任务和资源条件的平衡配置。例如，在同一训练预算下：

方案 A：较大的模型，使用较少的训练 Token。

方案 B：较小的模型，使用更多的训练 Token。

两种方案的最终效果，需要结合实际的 Scaling Law 拟合或实验评测判断。这也是 Scaling Laws 在模型设计和训练规划中的重要用途之一。

Scaling Laws **的适用边界** 虽然 Scaling Laws 在大模型研究中具有重要价值，但它们也存在明确的适用条件。

第一，经验规律依赖模型家族与实验设置。

不同架构、训练目标和优化策略可能产生不同的规模—性能关系。例如，稠密 Transformer 与 MoE 的计算和参数使用机制不同，不能不加调整地套用同一套模型参数统计与训练计算公式。

第二，数据质量会影响实际收益。

Scaling Laws 中的训练 Token 数通常只是数据规模的量化指标。但不同 Token 对模型训练的贡献并不完全相同。大量重复、低质量或任务无关的数据，未必能产生与高质量数据相同的效果。因此，不能将不同数据来源中的一个 Token 简单视为具有相同的学习价值。

第三，训练损失不能代表所有能力。

很多 Scaling Law 研究以语言建模损失为主要性能指标。然而，模型在复杂推理、事实可靠性、指令遵循和多模态理解等任务中的表现，不一定能够仅凭这一指标准确预测。例如，语言建模损失持续下降，并不意味着模型在所有安全性或复杂推理任务中都会同步改善。

第四，规模外推存在不确定性。

Scaling Laws 通常根据有限规模的实验结果进行拟合。当目标模型远远超出已有实验范围时，拟合误差可能增大，实际训练过程也可能受到新的瓶颈影响。因此，不能将拟合曲线在极大规模下的外推结果视为确定结论。

第五，真实计算成本与理论 FLOPs 不完全一致。

理论计算量并不直接决定实际训练时间和费用。不同模型在硬件利用率、显存需求、并行通信和推理效率方面可能存在明显差异。因此，计算最优配置并不必然是硬件时间最优或经济成本最优配置。

Scaling Laws **在多模态模型中的特殊问题** 当研究对象从语言模型扩展到多模态模型时，规模分析会更加复杂。例如，视觉语言模型同时包含视觉编码器、语言模型以及连

接模块，不同组件的参数规模和训练策略可能不同。此外，图像、音频和视频虽然都可以转换为 Token 或连续特征，但不同模态 Token 所包含的信息量和计算成本并不完全相同。例如，一张图像可能对应多个视觉 Token，一段视频可能对应大量时空 Token，而文本 Token 主要表示离散语言单元。

因此，不能简单地认为："一个视觉 Token 与一个文本 Token 具有完全相同的信息量和训练价值。"同样，对于图文配对训练，实际性能还受到图文对应关系、视觉信息质量和跨模态训练目标的影响。即使增加训练样本数量，也不一定能够保证模型充分利用视觉信息。对于多模态大模型，需要更加关注以下因素：

- 各模态编码器的参数规模及训练状态。
- 不同模态产生的 Token 数量和表示粒度。
- 跨模态对齐与融合机制。
- 数据配对质量与模态分布。
- 不同任务和模态之间的训练预算分配。
- 模型是否真正利用了增加的模态信息。

因此，多模态 Scaling Laws 不应只关注语言模型参数规模与总 Token 数，还需要考虑模态结构和训练目标带来的额外影响。

10.5 恢复、监控与故障定位

检查点的验收是恢复后的等价性检查。除确认文件可加载外，还应在固定数据与随机状态下比较连续训练和中断恢复后的下一次更新。优化器、调度器、精度缩放、采样进度或随机状态遗漏，都可能在权重相同的起点产生不同轨迹。

10.5.1 检查点保存与断点恢复

模型权重保存与训练恢复不同 训练检查点（Training Checkpoint）是为了在中断后延续训练而保存的状态快照；它与前述用于节省显存的激活检查点不是同一个概念。仅保存模型权重，通常足以支持相应结构的推理或作为新训练阶段的初始化，却不足以保证从原始训练轨迹继续优化。

完整恢复通常需要保存模型参数、优化器状态、学习率调度器状态、已完成的优化器步数和其他训练进度信息。采用 FP16 动态损失缩放时，还需要保存 GradScaler 状态；使用 EMA、分布式采样和混合任务调度时，还需要保存相应状态。

训练状态的主要组成 可将检查点的逻辑结构表示为：

$$C_t = (\theta_t, O_t, S_t, R_t, D_t, M_t) \tag{10.24}$$

其中，θ_t 为模型权重，O_t 为优化器状态，S_t 为学习率调度器及其他训练控制状态，R_t 为随机数状态，D_t 为数据采样进度，M_t 为实验元信息。

随机数状态可能涉及 Python、NumPy、CPU 和每个 GPU 的随机数发生器；数据状态可能涉及 sampler 的 epoch、数据流位置、任务采样器和未完成累积周期。若在优化器更新边界保存，通常比保存于未完成的梯度累积中途更容易定义恢复语义。

同时，还应记录训练代码版本、模型配置、数据版本、Tokenizer 和图像预处理参数。因为即使加载同一份权重，如果预处理或任务模板发生变化，恢复后的训练也可能偏离原先的实验协议。

定期检查点与最佳模型 训练工程通常需要区分三类产物：用于故障恢复的最近检查点、根据独立验证指标选出的最佳模型，以及供最终发布使用的推理权重。三者的用途不同。最近检查点应优先具有足够完整的优化状态，保证训练能够继续；最佳模型以明确的验证协议和指标为依据，不一定对应最后一次训练更新；推理模型可只保存必要权重及推理配置，以降低存储和部署成本。

因此，不能简单把 best 和 last 两种文件视为仅在文件名上不同。在某些工程中，最佳模型只保存权重，而最近检查点保存完整优化状态。是否能够恢复训练，需要检查实际文件内容而不是名称。

防止检查点写入中断 大型模型检查点可能需要写入多个文件。如果训练进程、节点或存储系统在保存过程中发生故障，就可能产生不完整文件。工程上可以采用先写入临时路径、完成必要校验后再发布正式检查点的方式；对分布式分片检查点，则需要遵循框架提供的协调保存与提交协议。

对于 FSDP 等分片模型，检查点可能由多个 rank 分别写入。PyTorch Distributed Checkpoint（DCP）支持多进程并行保存和加载，并提供一定的加载时重新分片能力。因此，不能假定所有分布式训练都可以仅由 rank 0 使用普通单文件保存方式完成完整恢复。

采用异步保存时，还应确保保存的数据视图与训练状态具有一致性，防止优化器继续更新期间保存到混合时刻的参数。具体实现应以框架版本的接口和一致性约定为准。

断点恢复的正确性验证 验证恢复功能时，不应仅以“程序能读入文件并继续显示 loss”为标准。更有说服力的方法是构造受控实验：先运行固定若干步并保存检查点，再从该检查点恢复继续运行；同时保留一条不被中断的参考训练轨迹，对比恢复后参数、优化器更新、学习率和数据采样顺序。

在固定算法、硬件和确定性条件下，可以尝试逐元素或严格容差比较。对存在非确定性算子、不同设备拓扑或通信顺序的设置，则应明确可接受的数值容差与评价指标，不能无条件要求逐位一致，也不能因模型能够继续更新便声称实现了严格数值恢复。尤其需要确认恢复后的第一步是否重复使用了之前的数据、学习率调度器是否多前进一次、随机增强是否重新从初始种子开始，以及分布式进程是否采用了正确的数据分片。

10.5.2 训练监控与故障排查

监控训练损失之外的信号 训练损失是判断优化是否进行的重要指标，但不能单独说明训练是否正常。一个模型可能在损失缓慢下降的同时发生视觉输入利用不足、某一任务过拟合、数据采样严重失衡或通信效率下降等问题。因此，应同时记录全局训练损失、分任务损失、验证指标、学习率、梯度范数、优化器更新次数、训练数据量和实际样本分布。对于混合精度，还可以监测是否出现非有限数值、损失缩放变化和被跳过的优化器更新。

对多模态系统，建议额外记录图像分辨率、视觉 Token 数、视频帧数、文本有效 Token 数、任务类别和模态缺失比例，以解释不同批次之间的显存及训练耗时变化。

资源与性能监控 GPU 利用率、峰值显存和实际吞吐是训练基础设施的重要观测量，但它们需要结合具体统计口径解释。

以 PyTorch CUDA 内存管理为例，$memory_allocated$ 表示被张量实际占用的内存，$memory_reserved$ 表示缓存分配器管理的内存，两者可能不同；系统级 GPU 内存占用还会受到上下文和其他进程影响。因此，nvidia-smi 显示的占用值不能直接等同于模型当前活跃张量所需的内存。$empty_cache$ 主要释放未占用的缓存，并不释放仍被活跃张量使用的显存。

此外，GPU 利用率较高不必然表示训练吞吐理想；如果模型频繁执行小规模内核、反复等待同步，仍可能存在明显效率问题。较可靠的性能分析需要结合时间剖析、数据加载记录以及不同算子的执行耗时。

损失异常与数值故障 如果训练突然出现 NaN 或 Inf，应按照数据、损失、梯度、优化器和精度逐层定位。首先检查异常是否来自某个特定批次，例如图像解码失败、全掩码、空标签、非法时间戳或分母为零。随后检查模型前向输出与各分任务损失是否有限，再分析梯度范数是否出现异常增长。

混合精度可能引入额外数值风险，但不能把所有 NaN 都归因于 FP16 或 BF16。例如，错误的目标标签、无效索引和不稳定的归一化也可能在 FP32 下产生异常。对涉及小概率、除法、指数和特殊归约的操作，应结合算法需求选择合适的精度与稳定化实现。如果一个多目标模型的总损失看似正常，还应分别检查各任务损失。否则，某个任务损失可能被其他规模更大的目标掩盖。

从单步验证到长训练验收 正式启动大规模训练之前，宜依次完成数据有效性检查、单卡前向与反向、一次真实参数更新、多卡同步、短周期训练及检查点恢复测试。尤其在更换精度、修改分布式策略或增加新模态后，应重新验证损失和梯度是否合理。短时间的 smoke test 能够证明基本工程路径可执行，但不能代替完整的训练稳定性或复现验收。因此，进入长训练后仍需要通过持续日志、独立验证和异常报警监控模型状态。

10.6 收敛与训练结果诊断

10.6.1 训练损失与收敛情况

训练损失的含义与边界 训练损失是模型参数对当前训练目标拟合程度的一种度量。设训练样本为 x_i，模型参数为 θ，每个样本的损失为 $\ell(\theta; x_i)$，则经验风险可以写为：

$$\mathcal{L}_{\text{train}}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \ell(\theta; x_i). \tag{10.25}$$

对于多模态模型，x_i 可以是一组图文配对、一段视频与字幕，或包含多轮图像问答的指令序列。损失下降通常说明模型正在改善相应的训练目标，但并不直接证明它已经学会准确的跨模态关系。例如，图像条件文本生成损失下降，可能来自更好的视觉理解，也可能部分来自对常见语言模式的预测改善。因此，损失曲线首先是优化状态的证据，而不是下游能力的直接替代指标。

不同预训练目标的损失不能直接比较 对于图文对比学习，损失取决于温度参数、批内负样本数及正样本关系；对于自回归生成，损失与词表、监督位置、文本难度和归一化方式有关；对于掩码重建，其数值还受到重建目标的尺度影响。因此，不能因为模型 A 的对比损失为 2、模型 B 的文本交叉熵为 1，就认定模型 B 更好。多目标训练常采用：

$$\mathcal{L}_{\text{total}}(t) = \sum_{k=1}^{K} \lambda_k(t) \mathcal{L}_k(t). \tag{10.26}$$

其中 $\lambda_k(t)$ 为不同目标在训练时刻 t 的权重。分析时应分别保存原始分项损失、权重及加权总损失。若训练中调整损失权重或数据混合比例，总损失的变化就不再完全反映同一个固定优化目标的改善，必须结合阶段边界解释。

理解收敛曲线与训练阶段 常见的训练过程可能包括初始快速下降、随后减缓及后期相对稳定等表现。但并不存在所有多模态训练都应当遵循的固定曲线形状。学习率预热、余弦衰减、数据难度变化、模态分辨率调整及参数解冻，都可能改变曲线。

例如，模型由只训练连接模块切换为联合更新语言模型时，优化变量集合发生变化；从图像描述数据转向复杂视觉指令数据时，训练样本和目标难度也会变化。此时，即使损失出现暂时升高，也不能直接判定训练失败。相反，如果损失快速下降却伴随视觉问答准确率不变，也需要考虑模型是否主要学习了回答模板而不是视觉证据。因此，训练曲线应与学习率、有效批量大小、数据比例、可训练参数和任务阶段共同绘制或标注，避免脱离上下文解读曲线拐点。

训练损失与验证损失 为了检查模型能否适应未参与参数更新的数据，需要建立与训练过程相互独立的验证集。在与训练目标一致且统计口径固定时，可以比较训练和验证损失的变化。如果训练损失持续下降，而验证损失明显恶化，可能提示过拟合、数据分布变化或监督质量问题。

但跨模态预训练的验证损失同样受到数据来源和目标难度影响。若验证集比训练集具有更复杂的图文关系，其绝对损失可能长期偏高，并不必然意味着训练异常。此外，在训练中反复依据验证集调节配置，会使其逐渐成为模型选择的一部分，因此最终仍需保留独立测试数据进行确认。

优化稳定性与数值异常 训练过程还需要监控梯度范数、参数更新、学习率和数值异常。当损失突然出现 NaN 或无穷大时，应检查混合精度数值范围、异常输入、损失归一化、梯度放大和优化器设置。若训练损失长时间不变，则可能来自参数被冻结、梯度未正确传播、学习率过小或数据重复等原因。

对于多任务模型，还应分别统计任务实际进入训练的次数，防止某类模态或任务因采样器错误而长期没有参与更新。对于大规模分布式训练，吞吐突然降低也可能由数据读取或通信阻塞造成，此时不应仅从损失变化推断模型问题。

检查点选择与“完成训练”的区别 应区分最终训练检查点、验证集最优检查点与专门用于推理的模型权重。“训练到了预定步数”只说明预算已经执行，并不自动说明模型达到预期质量。若选择验证集表现最佳的检查点，应在训练前明确选择规则，并避免反复依据最终测试集挑选参数。

此外，平均训练损失通常会受到样本长度和不同任务混合的影响。为了保证报告可比，需要注明损失是按样本、按监督 Token 还是按任务求平均，以及是否忽略填充和无监督输入位置。缺少这些信息，即使曲线外观相近，也不一定具有相同的物理含义。

10.6.2 训练效果与数据、架构的关系

数据、架构与计算预算共同影响结果 预训练效果来自多个因素的共同作用，而不是某个架构单独决定。可以概念性地表示为：

$$P = F(\mathcal{D}, \mathcal{A}, \mathcal{O}, \mathcal{C}, \mathcal{E}), \tag{10.27}$$

其中 $\mathcal{D}$ 为数据组成与质量，$\mathcal{A}$ 为模型架构，$\mathcal{O}$ 为优化和训练目标，$\mathcal{C}$ 为计算预算，$\mathcal{E}$ 为评价协议。这不是一个已确定的解析性能公式，而是提醒研究者在归因时同时考虑各类条件。

例如，若模型换用更大的视觉编码器，同时提高输入分辨率并增加训练数据，就无法根据最终性能提高判断哪个因素贡献最大。需要进行分层对照，逐步隔离影响来源。

数据质量与数据规模 在图文预训练中，更多数据可以扩大对象、场景和语言表达的覆盖，但也可能增加重复配对、弱关联文本和错误标注。DataComp 为图文数据选择建立了标准化训练框架，通过固定模型训练流程与计算尺度，检验不同数据筛选方法在下游测试中的效果。其意义在于，将数据构建本身变为可以实验比较的研究对象。

因此，分析数据贡献时不应只报告训练样本总量，还应考虑独立来源、配对正确性、覆盖范围与实际使用次数。过度过滤也可能删除夜间、模糊或罕见场景等对目标

任务有价值的困难样本。数据“更干净”与模型“更能泛化”之间并不始终存在简单对应。

数据与架构之间的交互 同一种数据对不同架构的价值可能不同。例如，长图文交错文档可以提供多图像上下文，但模型若只支持单图像输入，就无法直接利用全部关联信息。高分辨率 OCR 数据可能提供更丰富监督，但视觉编码器若在输入阶段丢失小文字，单纯扩大数据量未必解决问题。

反过来，升级视觉编码器或增加视觉 Token 数，如果训练数据缺乏相应细粒度标注，也不一定产生预期收益。因此，研究者应当把“是否具备输入信息”“是否拥有对应监督”“是否存在足够模型容量”和“是否获得充分优化”作为相互关联但可以分别诊断的条件。

性能增长与边际收益 随着模型、数据和训练预算增加，性能可能继续改善，但各能力维度的增长幅度不一定相同。Reproducible Scaling Laws for Contrastive Language-Image Learning 对对比式图文训练的模型、数据和计算规模进行了系统实验，讨论了不同下游任务上的缩放行为。其结果提示，模型规模增长与任务收益之间的关系具有任务依赖性。

因此，不能仅根据训练损失随着计算预算下降，就推断所有下游能力都将按相同比例提高。对于资源受限的研究，分析追加数据、提高分辨率、增加参数或延长训练分别带来的边际收益，通常比只报告最大模型结果更具方法价值。

数据、模型与评测的闭环分析 完整的预训练结果分析可以形成一个闭环：首先利用训练日志确认优化过程正常；随后用分任务评价确定能力增长或不足；再通过分层消融定位数据、架构和训练目标的作用；最后根据证据修改下一轮训练方案。

例如，如果图文检索明显提高，而细粒度对象关系判断没有改善，研究者就需要判断问题是否来自粗粒度配对数据、视觉特征分辨率、缺少融合交互或评价任务与训练目标不匹配，而不能直接认为模型需要更长训练。同样，如果 OCR 任务提升但推理延迟明显增加，则需要报告质量—效率权衡，而不是仅展示最高准确率。这一过程使预训练研究从单纯的性能竞赛转向可解释、可复现的实证分析。

习题与讨论

1. 为仅训练连接器、部分解冻和端到端训练列出参数分组，指出阶段切换时哪些优化器状态需要明确处理。
2. 推导多个变长微批次的全局 Token 平均损失。说明按微批次等权平均对应什么统计目标。
3. 设计一个断点恢复测试，规定比较的参数、下一批数据、随机状态和容许数值误差。
4. 训练损失突然变为 NaN。按数据、前向、损失、反向和更新顺序给出定位方法，避免同时改变多个设置。

第十一章　指令微调与偏好优化

后训练改变模型在给定条件下如何响应任务。监督微调学习示范，偏好优化学习回答之间的选择；二者都受到数据中的视觉依据和输出形式约束。回答更长或更符合偏好，并不能单独证明视觉证据利用更好。

学习目标　构造多模态监督掩码；计算低秩适配参数量；解释奖励与 DPO 目标；用受控输入和能力保持实验检验后训练。

11.1　指令样本与监督位置

媒体占位符、用户消息和助手目标属于同一个序列接口，却不必都计入损失。检查一条样本时，应把 Token、可见性、位置和监督掩码并排打印，而不是只查看原始对话文本。

11.1.1　指令数据与监督目标

指令样本的基本组成　单轮指令数据可以抽象为 $D_i = (X_i, Q_i, Y_i, M_i)$，其中，$X_i$ 为可选的图像、视频、音频等观测；Q_i 为任务指令；Y_i 为参考回答；M_i 为与数据来源、任务、模态及质量有关的元信息。如果当前任务仅有文本输入，则 X_i 可以为空。

例如，图像输入是一张红色汽车停在道路旁的照片，指令为“汽车是什么颜色？只回答颜色”，参考答案为“红色”。如果指令改为“请简要描述整个场景”，则需要使用另一种目标输出。由此可见，同一观测可以产生多条指令样本，但这些样本并不独立于原始图像；进行训练集与测试集划分时，应保留来源关联，避免同一图像的派生问答在划分间泄漏。

从数据来源看，指令样本可以由人工编写、已有标注转换、模型辅助生成或程序化构造。不同来源的监督可信度可能不同，尤其是复杂视觉推理中的模型生成答案，不宜直接当作已核验的事实。第九章已经讨论其数据构建与质量控制；在 SFT 阶段，更需要关注这些数据经过序列化之后，哪些部分真正构成模型的预测目标。

自回归监督微调损失　对于以文本为输出的自回归视觉语言模型，给定上下文 C 和参考答案序列 $Y = (y_1, \ldots, y_L)$，常见的监督目标为

$$\mathcal{L}_{\mathrm{SFT}} = -\frac{1}{L} \sum_{t=1}^{L} \log p_\theta(y_t \mid C, y_{<t}) \tag{11.1}$$

训练过程中通常采用教师强制，即预测第 t 个答案 Token 时，模型可以看到真实参考答案中此前的 Token。上述损失直接约束参考答案的条件概率，而不是先让模型

完整自由生成答案，再对生成结果评分。

这种方法的优点是训练目标明确、易于实现，能够充分利用已有问答和描述标注。但它也存在局限：模型可能学到答案的常见措辞和数据集偏差，训练损失降低不一定意味着回答更忠实于图像；而参考答案并非唯一正确表达时，最大似然目标也会偏向训练集中出现的特定形式。

指令输入与答案位置的损失掩码 实际训练通常将系统消息、用户指令、多模态占位信息和助手回答整理为一个序列。并非序列中的全部位置都应当参与同一训练损失。在常见的“仅监督助手回答”配置下，可以为每个位置设置掩码 $m_t \in \{0,1\}$：目标回答位置取 1，用户输入、无需预测的特殊占位位置及填充位置取 0。其损失可写为

$$\mathcal{L}_{\text{masked}} = -\frac{\sum_{t=1}^{T} m_t \log p_\theta(s_t \mid s_{<t}, X)}{\sum_{t=1}^{T} m_t} \tag{11.2}$$

其中，s_t 为序列中的目标 Token，X 表示由多模态处理器提供的其他条件输入。分母表示有效监督 Token 数；若一个批次没有有效目标位置，应进行数据检查而不是直接用零作为分母。

这种设计使系统消息和用户问题可以作为条件被模型读取，而训练重点集中于助手应该生成的回答。“只监督助手回答”是常见策略，而不是 SFT 的唯一合法定义；某些语言建模式继续训练或特定任务可能采用不同监督区域。多轮对话也可以监督每一轮助手回答，或根据数据设计只监督最后一轮，必须明确实际设置。

由于因果语言模型内部需要将 logits 与下一 Token 标签正确对齐，损失掩码还必须和序列位移保持一致。如果掩码错位，模型可能被要求预测用户文本、媒体占位符，或者漏掉真正的回答。对微调工程而言，这是比单纯检查数据字段是否齐全更关键的正确性问题。

答案长度、格式与损失统计 不同任务的目标输出长度可能差异很大。对象颜色问答可能只需要一个词，图像描述可能需要数十个词，长视频总结则可能需要更长的回答。若直接累加每条样本的全部 Token 损失，长答案可能占据更大的梯度贡献；若对样本分别求平均再在批次上平均，则每条样本的相对权重又不同。

因此，常见的归一化方式包括按有效 Token 平均和按样本平均，它们对长短答案的权重并不完全一致。训练配置应明确采用何种方式，必要时还需要按任务控制抽样比例和答案长度分布，而不是仅比较原始样本条数。

输出格式本身也是重要监督。例如，分类任务可能要求只返回类别名称，文档抽取任务可能要求生成指定字段，视觉定位任务可能要求输出符合协议的坐标。格式示范有助于模型学习输出规范，但格式合法不代表事实正确。尤其对数值、时间戳和空间坐标，应分别检验解析有效性及其与原始观测的对应关系。

多样性、模糊性与拒答样本 单张图像可能对应多种合理描述，同一开放式问题也可能有多种正确回答。SFT 不必要求所有合理答案被规范化成完全相同的句子，但训练集应尽量避免不同参考答案在事实内容上相互矛盾。对于无法从输入确定的问题，适当

加入“信息不足”的示范十分重要。例如，静态图像未必能够判断某人五分钟后会去哪里；模糊视频也未必能辨认远处的标志牌。此时，合理的监督目标是说明证据不足，而不是编造一个确定答案。

另一方面，如果大量样本都以固定套话表达不确定性，模型也可能形成过度拒答。因此，需要区分确实无法回答、需要结合常识推断和可以直接观察的问题，通过独立评价衡量模型是否恰当地使用证据。

11.1.2 多模态输入与答案生成

输入不等于一段普通文本 纯文本指令微调通常主要处理文本 Token，而多模态 SFT 需要同时组织图像、音频、视频等观测。以视觉语言模型为例，图像先经过视觉处理器和编码器，产生一组视觉特征；文本指令则经过分词器转换为文本 Token，随后通过模态连接或交叉注意力机制参与语言模型计算。可以概括为

$$H_V = f_V(I), \qquad Z_V = P_\psi(H_V), \qquad p_\theta(Y \mid Q, Z_V) \tag{11.3}$$

其中，P_ψ 为视觉连接模块。实际实现可以将视觉表示插入文本序列，也可以通过交叉注意力等机制提供视觉条件。不同模型中的视觉占位符、视觉 Token 个数及注意力路径并不相同。

在一份 JSON 指令数据中写入”type”: ”image” 或类似标记，不意味着模型已经得到真实图像。训练数据处理器还必须能够找到媒体文件，完成解码、尺寸变换、视觉编码和对应位置映射。聊天模板中的占位符与实际展开后的视觉 Token 也未必是一一对应关系。

图像问答与视觉依据 对于图像问答，输入通常包括一张或多张图像及文本问题，输出为自然语言或结构化答案。例如，在交通场景图像中询问“最左侧的汽车是什么颜色?”，模型不仅需要识别汽车，还要根据“最左侧”确定对象，再读取相关颜色信息。如果回答来自多个视觉区域之间的关系，训练样本应尽可能保留明确的视觉依据。必要时可以使用已有对象框、区域描述或场景图标注构建问题与答案，以降低语言模型依据常见对象共现关系猜测答案的风险。

然而，简单的 SFT 交叉熵只直接监督最终输出 Token，并不自动保证模型内部采用正确的视觉定位路径。对于对象计数、属性绑定、空间关系或文本识别等任务，仍需要独立的细粒度评价。

多图像与视频输入 当问题需要比较多张图像或理解视频事件时，模型必须能够区分不同观测的顺序与身份。例如，“第二张图中新增了什么物体?”依赖两张图像之间的对应，而“人物在打开车门之后做了什么?”则需要视频中的事件时间关系。因此，视频相关样本除了保存文件位置，还应保存必要的采样帧、时间戳或片段边界，并使模型实际接收到的视觉序列与训练答案一致。若视频被缩短、抽帧过稀或片段时序发生错误，目标答案可能不再得到输入证据支持。

多图像指令微调也不能自动等同于长视频时序理解。模型可以学会比较离散图像，

却不一定能够据此恢复连续运动、动作开始与结束时刻或细粒度事件先后关系。对于这些能力，需要匹配的时序表示、标注和评价。

音频、语音与其他模态 多模态 SFT 的输入范围不局限于图像。对于语音任务，可以根据音频回答“说话者提到了哪个城市?”；对于环境声音任务，可以根据音频描述“是否存在明显的警报声”；对于视听任务，模型可能需要将画面事件与相应声音进行关联。与视觉处理相似，音频输入需要经过采样率、时间分段及编码处理。对于语音转录任务，应保留文本与音频的一致性；对于说话人分离、时间定位等任务，还需要确保相应时间戳及标签使用统一基准。

如果模型的输出不仅是文字，还包括离散音频 Token、图像 Token 或连续媒体表示，那么训练目标可能不再只是文字 Token 上的自回归交叉熵，而需要使用对应模态的解码与生成损失。本节主要讨论以语言或离散结构化答案为输出的典型多模态 SFT，其他生成训练目标应按具体模型分别说明。

多轮对话与历史条件 多轮多模态指令通常由一系列角色消息组成：用户可能先提供图像并询问场景，随后追问某个对象的位置，而下一轮问题可能省略此前已经出现的对象名称。此时，模型的输入条件包括当前问题、历史消息和被引用的多模态观测。设第 r 轮助手回答为 $Y^{(r)}$，历史上下文为 $C^{(r)}$，则可写为

$$\mathcal{L}_{\text{multi-turn}} = -\sum_{r\in\mathcal{R}}\sum_{t}\log p_\theta(y_t^{(r)} \mid C^{(r)}, y_{<t}^{(r)}) \tag{11.4}$$

其中，$\mathcal{R}$ 为参与监督的助手回合集合；实际训练还应使用适当的 Token 归一化和掩码。多轮数据需要避免出现不合理的信息可见性。例如，训练某一轮回答时，不应让模型看到同一轮尚未发生的用户追问；如果答案依赖一张先前上传的图片，则必须在实际输入或可访问上下文中保留该图片及其关联关系。否则，模型可能从错误的对话组织方式中学习到与真实交互不一致的行为。

视觉幻觉与证据约束 多模态模型即使具备流畅的语言生成能力，仍然可能描述输入中不存在的物体，或者错误报告数量与空间关系。监督微调可以通过高质量、具有视觉依据的数据降低部分问题，但不能保证彻底消除视觉幻觉。

例如，如果一张图像显示两辆汽车，而模型生成的训练答案写成三辆，那么使用交叉熵训练会将错误内容当成目标。即使答案文本表达清晰，监督也不再可靠。因此，模型生成的合成问答、复杂推理解释和结构化输出，应依据原始标注、程序验证或必要的人工作业进行审核。特别需要区分“答案语言自然”“答案符合格式”和“答案受观测支持”三个维度。它们可能相关，但不是同一件事。高质量多模态 SFT 需要同时关注这三个方面。

11.1.3 微调数据格式与训练流程

消息格式与媒体引用 在工程实现中，指令数据通常被整理为带角色信息的消息序列。常见角色包括 system、user 和 assistant；对于多模态输入，每条消息还可能包含多个

媒体片段。下面给出一条示意性的视觉问答数据。其字段设计只用于说明结构，不代表某个模型必须使用完全相同的序列化协议：

```
{
  "sample_id": "scene_0001_q1",
  "source_id": "scene_0001",
  "task": "visual_qa",
  "messages": [
    {
      "role": "user",
      "content": [
        {"type": "image", "path": "images/scene_0001.jpg"},
        {"type": "text", "text": " 图中有几辆汽车？只回答数量。"}
      ]
    },
    {
      "role": "assistant",
      "content": [
        {"type": "text", "text": "2"}
      ]
    }
  ],
  "annotation_source": "verified_object_labels",
  "split": "train"
}
```

这里的路径是示例性字段，不是实际可访问的数据文件。正式训练时，应当验证文件存在、媒体能够正确解码，且图像与答案确实来自同一原始样本。

对于多轮样本，messages 可以依次记录用户和助手的多次交互；对于视频或音频，可使用与模型处理器相匹配的媒体字段，并保存必要的时序信息。序列化格式还应明确终止符、角色边界和媒体占位符的具体规则。

聊天模板与多模态处理器 模型通常不能直接读取抽象的消息字典，需要通过聊天模板（Chat Template）将角色信息转换为模型约定的文本或特殊 Token 序列。多模态处理器（Processor）还需要对图像、视频和音频执行相应预处理，并输出模型能够接收的张量。

不同模型可能采用不同的开始、结束、轮次分隔符和模态标记，因此应使用与目标模型及其权重匹配的官方模板。特别是在多模态场景中，一个可见的图像标记可能在处理器内部被展开为大量视觉特征，不能根据文本中占位符的数量推算真实视觉 Token 数量。

对于训练，模板还应明确哪些消息位置属于助手的监督范围。某些通用训练工具支持只计算助手消息或 completion 部分的损失，但具体功能是否生效取决于数据组织方式、模板和软件版本。不能只因为配置字段被设置，就认定训练损失掩码已经正确。

序列构造与损失掩码校验 完成数据清洗后，需要将多模态输入、文本指令与参考答案构造成模型所需的训练批次。通常会得到输入 Token、注意力掩码、媒体张量及目标标签等内容。对不需要计算语言损失的位置，可以使用框架约定的忽略标签或显式损失掩码。

其中，最重要的验收之一是逐样本核对“输入中包含什么”和“模型被监督生成什么”。例如，视觉问答样本应当允许模型读取图像和用户问题，但不应通过错误的标签设置要求它复现整段用户问题。对于多轮样本，需要明确是否监督全部助手回合，并核查不同回合的可见上下文。

对于长文本和长视频，还应合理处理序列截断和批次填充。如果裁剪掉了问题所依赖的图像帧，却保留对应答案，训练就可能出现信息不足的伪监督。如果截断使答案的结束标记缺失，也可能影响模型学习合理的输出终止条件。样本打包还需要确保不同样本之间不会产生不符合预期的注意力或标签泄漏。

参数更新方式与优化过程 SFT 可以采用完整参数微调，也可以使用参数高效微调。对于典型视觉语言模型，参数可以概念性地划分为视觉编码器 θ_V、跨模态连接模块 θ_P 和语言模型 θ_L。在一种常见设置中，视觉编码器保持冻结，而连接模块与语言模型参与优化：

$$\theta_{\text{train}} = \{\theta_P, \theta_L\} \tag{11.5}$$

另一些设置只训练连接模块，或者使用 LoRA[24] 等适配器更新语言模型的部分有效权重；还有一些设置允许视觉编码器、连接模块和语言模型共同更新。参数更新范围应根据任务与训练预算确定，不能把“进行了 SFT”理解为“全部参数必然解冻”。

训练时还需要明确学习率、优化器、有效批量大小、梯度累积、混合精度与最大序列长度。这些工程参数已经在第十章讨论，此处的重点是它们应与指令数据和监督位置共同配置。尤其当视频或多图输入增加视觉 Token 数量时，仅保持文本 Token 上限不变并不能保证总体计算预算不变。

一个可复现的微调实验设计 假设需要构建一个能够识别道路图像中车辆数量、颜色和相对位置的视觉问答模型。首先选择已经预训练的视觉语言基座，整理具有经过核验的对象与属性标注的图像，再将结构化标注转换为不同问题与答案。训练数据中应包括正确可答的问题、不同难度的关系问题，以及观测不足的示例。

随后建立统一的训练数据格式，记录原始场景标识和派生问答关系；通过多模态处理器转换为模型输入，并核对助手回答位置的损失掩码。完成基本训练后，除评价总体问答准确率，还应分别评价计数、颜色、空间关系与信息不足判断，并以原始场景为单位保证测试独立性。

为了分析训练收益，可以比较冻结全部基础模型、只训练连接模块，与进一步更新语言模型或视觉编码器的不同方案。比较时应报告数据规模、有效 Token 数、参数更新范围和训练计算预算；若某种方案取得更好的问答结果，还需要检查其是否引入更多计算或额外训练数据。这一示例说明，SFT 的有效性并不单纯取决于训练是否正

常结束，而取决于高质量监督、正确的输入与标签构造、明确的参数优化范围，以及能够检验真实多模态能力的独立评价。

11.2 多模态指令与证据利用

11.2.1 图像问答与视觉指令

从图像描述到任务条件回答 图像描述（Image Captioning）和视觉问答（Visual Question Answering，VQA）都是视觉语言生成任务，但两者所要求的监督不同。图像描述通常要求模型概括一张图像中的主要内容，而视觉问答不仅需要理解图像，还需要根据问题定位相关信息，并生成具有针对性的答案。

例如，对于一张显示行人站在公交站牌旁的图像，图像描述可能是“几名行人在路边等候公交车”。如果用户进一步提问“站牌上显示的线路编号是什么”，模型就必须利用相应局部区域，而不能仅仅复述图像的整体语义。当图像分辨率不足以辨认编号时，合理回答应说明无法可靠识别，而不是根据常见公交线路随意猜测。可以将单图视觉指令样本表示为：$D_i = (I_i, Q_i, Y_i, M_i)$，

其中，I_i 为图像，Q_i 为自然语言指令，Y_i 为期望输出，M_i 为任务类型、来源、质量及其他必要元数据。模型在训练时学习条件分布 $p_\theta(Y_i \mid I_i, Q_i)$。在典型的文本回答式 SFT 中，损失通常作用于助手需要生成的答案 Token，而图像特征和用户指令作为条件输入。对于存在多个合法表达的任务，目标答案是合理的监督示例，但并不意味着其他措辞必然错误。

图像指令可以覆盖对象识别、属性判断、计数、空间关系、场景概括、图中文字识别、图表理解、问题解释以及多轮对话等能力。它们所需的视觉信息并不相同。例如，普通分类可能依赖整体语义，而计数和文本识别常常要求保留局部细节。因此，指令微调数据的任务覆盖应根据模型希望具备的能力确定，而不应只追求问题措辞的变化。

视觉指令监督的证据粒度 一张图像可以同时支持多种粒度的监督。最粗粒度是整图描述或场景类别，其次是对象属性和关系，更细粒度的任务可能涉及区域定位、文本框、分割掩码或几何坐标。指令越具体，通常越需要相应粒度的证据。例如，对于“图中是否有自行车”，对象类别标注可能提供足够监督；但对于“自行车左侧的行人是否拿着雨伞”，就需要同时识别对象、相对位置和局部属性。如果只有整图描述而缺乏对象级标注，自动生成的复杂问题未必具备可靠答案。

因此，视觉指令的构建应遵循“监督内容不超过可验证证据范围”的原则。在使用真实标注生成训练样本时，应保留原始图像与对象、属性和区域标注的对应关系；在使用语言模型扩展问题时，则需要检查新问题是否引入未经证实的假设。对于开放问题，允许根据任务规范给出解释性回答，但应区分观察事实、一般知识和不确定推断。

LLaVA 与视觉指令生成 LLaVA[14] 是视觉指令微调的代表性研究。其原始工作利用 COCO 图像的已有描述与对象信息，通过语言模型构建视觉对话、详细描述和复杂推理等类型的训练样本，再将视觉编码器与语言模型连接起来进行指令微调。这里需要

明确：原始 LLaVA 的数据生成环节主要将已有视觉标注转化为文本信息供语言模型使用，而不是让当时的语言模型直接观察原始图像。

这一路线的意义，是将图像描述和结构化视觉标注扩展为面向不同任务要求的指令示范。但其限制同样值得注意：自然语言生成模型可以把标注信息组织成流畅的对话，却可能补充原始证据无法支持的细节。因而，合成指令的语言质量与事实正确性应分别评价，不能将教师模型给出的回答直接等同于人工核验的真值。

在后续的 LLaVA-1.5 研究中，研究者使用更高分辨率视觉编码器、MLP 连接模块，并引入面向学术任务的 VQA 数据和响应格式提示，建立了更强的视觉指令训练基线。这说明，指令微调效果不仅取决于问题数量，还与视觉表示粒度、任务数据组成及输出要求有关。

InstructBLIP 与指令感知的视觉信息选择 InstructBLIP[23] 在 BLIP-2[12] 基础上开展系统性的视觉语言指令微调研究，将 26 个公开数据集转换为指令形式，其中 13 个用于训练，13 个保留用于评测，并利用指令感知的查询模块提取与当前问题相关的视觉信息。与对同一图像始终使用完全相同的视觉摘要不同，该设计尝试让视觉特征选择受到问题内容影响。

例如，对于“图中有什么动物”，相关视觉区域可能是动物本身；对于“背景中的标志牌写了什么”，模型则需要关注远处文字。二者来自同一张图像，但有效回答需要不同的视觉信息。InstructBLIP 的代表性启示在于，指令并不只影响最终文本解码，也可以参与视觉表示选择与跨模态交互。

不过，模型具有指令感知机制并不自动保证其关注区域一定正确。训练数据质量、视觉分辨率和特征提取方式仍会影响最终结果。对比不同视觉指令模型时，应区分“能够读取指令”“能够根据指令选择视觉证据”和“能够准确完成任务”这三个层次。

图像问答中的回答形式与不确定性 图像问答的正确答案可能具有不同形式。数量统计可以要求返回整数，OCR 可以要求逐字转录，开放问答则允许自然语言表达。对于结构化输出任务，还可能需要坐标、对象标识或其他机器可解析格式。因而，数据格式应同时规定任务含义和输出规范。

例如，问题“图中有几名戴安全帽的工人”可以使用单个数字作为目标；问题“请说明为什么该区域可能存在安全隐患”则需要引用可见证据，同时区分图像所示事实与一般安全知识。对于图像中看不清的文字、被遮挡的对象或无法观察的未来事件，还应允许模型明确说明信息不足。

这类拒绝无依据猜测的示范，不应与任意的保守回答混为一谈。训练数据应当同时包含可以回答的困难样本与确实缺少必要信息的样本，使模型学习在证据充分时作答、在证据不足时保持适当不确定性。

11.2.2 多图、视频与音频指令

多图输入与跨图像关系 多图指令将单张图像扩展为一组视觉观测。设输入图像序列为 $\mathcal{I} = (I_1, I_2, \ldots, I_K)$，问题为 Q，目标回答为 Y，则可表示为：

$$D = (\mathcal{I}, Q, Y) \tag{11.6}$$

与单图任务相比，多图理解不仅要求模型识别每张图像，还需要处理图像之间的对应、差异和顺序关系。例如，用户可能要求“比较两张照片中设备面板上的变化”，或者“第二张图中的零件是否也出现在第一张图中”。这些问题需要维持清晰的图像引用，不能把所有视觉内容压缩为一个不区分来源的场景描述。

训练时，图像在消息中的排列顺序与图像标识具有重要作用。如果问题提到“第一张”和“第二张”，而媒体文件与模板顺序发生错配，即使每张图像本身正确，答案也可能成为错误监督。因此，数据序列化通常需要保存图像数量、顺序、标识及与文本片段之间的绑定关系。

对于多页文档和连续照片，还需要区分“图像组”与“有明确时间顺序的图像序列”。前者可能用于跨页查找和比较，后者则可能涉及事件变化和先后关系。多图训练能够为这些能力提供基础，但并不必然等价于完整的视频动态理解。

视频指令与时间信息 视频可以视为包含时间信息的视觉观测。对于抽样后的帧序列，可将输入表示为：

$$V = \{(t_k, I_k)\}_{k=1}^{K} \tag{11.7}$$

其中，t_k 是相应帧在视频时间轴上的时间戳，I_k 是该时刻的视觉内容。与静态图像问答不同，视频指令可能要求判断动作顺序、事件起止时间、状态变化、持续时间或跨较长时间的因果线索。

例如，一段视频展示人物走近门口、按响门铃、等待片刻后离开。问题“人物在离开前做了什么”需要利用多帧之间的时间关系，而不能只凭某一张静态图像作答。对于“门铃在什么时间响起”，还可能需要音频或具有足够精度的事件标注；如果只提供无声音的稀疏视频帧，就不应强行生成确定的听觉事件答案。

视频数据的采样策略会直接影响可回答的问题范围。固定间隔抽帧能够控制计算量，但可能漏掉短时事件；增加帧数可以提高时间覆盖，却也增加视觉 Token 和长上下文计算成本。对于需要精确事件定位的任务，还应保存原生时间戳和抽样映射，而不是只记录帧序号。若原始视频存在变帧率或镜头剪辑，简单用“帧号除以固定帧率”估算时间可能不够可靠。

Video-ChatGPT、Video-LLaVA 与跨场景训练 Video-ChatGPT 是视频指令微调的重要早期代表。该研究构建约 10 万组视频指令数据，通过人工与半自动化流程生成视频相关对话监督，使模型学习对视频内容进行详细描述和问答。这一方向说明，视频理解任务可以采用语言指令与回答形式进行训练，但必须保留与视频片段相对应的事实依据。

Video-LLaVA 进一步探索图像和视频的统一视觉表示，并使用混合图像—视频数据进行学习，以期利用两类任务之间的互补信息。其意义不是将视频简单地看作更多图像，而是尝试降低不同视觉输入在表示和连接阶段的差异。

LLaVA-OneVision 则进一步面向单图、多图和视频任务构建统一模型，并研究不同视觉任务之间的迁移。这些研究表明，共用视觉与语言基础组件可以促进跨场景的能力复用。不过，某模型在静态图像任务表现较好，并不能由此推断其一定能够准确理解长视频中的动作时间关系。对于时间敏感任务，仍需使用独立的视频评价协议进行验证。

音频指令与任务区分 音频输入与图像输入具有明显差异。音频包含连续波形、频谱特征及时间变化，其训练任务也不只是将说话内容转写为文字。例如，音频指令可以要求模型：识别语音中的词语、回答说话者提出的问题、总结多人对话、判断环境声音、说明事件发生顺序，或者结合声音与文本完成其他任务。这些任务需要的监督信号不同。自动语音识别（ASR）主要关注语言内容与文本之间的对应；声音事件理解则可能依赖非语音声音；语音对话还需要识别指令语义并产生相应答复。

对于音频波形 A、任务指令 Q 和目标回答 Y，可以写为：

$$p_\theta(Y \mid A, Q) \tag{11.8}$$

但这一表达不规定音频必须直接转换为文本。模型可以使用声学编码器输出的连续特征，也可以使用离散音频 Token 或其他表示。若模型仅依赖预先提供的转录文本，它所完成的任务应与直接处理原始音频的系统加以区分。

Qwen2-Audio 是音频语言指令训练的代表性研究，支持不同音频理解任务，并设计语音聊天和音频分析等交互模式。其研究说明，统一音频语言模型需要组织不同任务的输入提示与监督形式，使模型在处理语音内容的同时，也能够利用更广泛的声音信息。

音视频联合指令与同步关系 更复杂的场景可能同时包含视频、声音和文字。例如，用户提供一段监控录像并询问“报警声响起时，画面中的人正在做什么”。这一问题需要在共同时间轴上关联音频事件与视觉动作。可以将视听输入概念性地表示为：

$$X_{AV} = (\{(t_i^V, I_i)\}, \{(t_j^A, a_j)\}, Q) \tag{11.9}$$

其中，a_j 可表示音频片段或相应声学表示。训练数据应保存必要的同步信息，例如音频与视频是否具有同一起始时间、是否存在偏移、字幕是否来自原生音轨，以及媒体处理过程是否改变时间轴。

Qwen2.5-Omni 是将图像、视频、音频等输入与文本和语音输出纳入共同系统的代表性工作，其设计中特别考虑音视频时间信息的组织和流式交互。这说明，多模态指令训练除了关注语义监督，还可能需要针对流式输入、同步和不同输出模态设计专门机制。

不过，对于一般的文本回答式多模态 SFT，输出目标仍然可以是普通文本；如果任务进一步要求生成语音或视频，则需要额外的模态生成表示和相应训练目标，不能把“输出一段描述音频的文字”视为真正完成了音频生成。

模态缺失与任务可回答性 在真实应用中，某些输入模态可能缺失或不可靠。例如，视频只有画面而没有声音，音频存在严重噪声，或者多图任务只上传了其中一张图片。训练数据需要使模型区分“没有提供该信息”“信息已经提供但无法辨认”和“可以依据其他模态直接回答”三种情况。

例如，对于没有音轨的视频，问题“说话者说了什么”通常不能仅凭画面得到可靠答案；而对于“人物是否进入房间”，视频画面可能已经提供充分依据。训练时可以构造合法的模态缺失样本，并给出与证据条件相符的回答，从而避免模型把所有请求都视为必须产生肯定答案。

11.2.3 指令遵循与视觉证据利用

指令遵循不等于证据利用 多模态指令模型可能生成格式正确、语气自然的回答，但这并不能证明回答来自输入图像或视频。模型有时可以根据语言问题、常见场景共现关系或数据集模板猜测答案。例如，对于“桌面上有什么水果”，即使没有正确读取图像，模型也可能给出“苹果”这一常见回答，并偶然与真实场景一致。

因此，需要区分三个层次：第一，模型是否遵守用户指定的任务与输出格式；答案在事实层面是否正确；答案是否真正依赖当前多模态观测。这三个层次可能相关，但并不等价。例如，在“只回答数字”的任务中输出“两个”而不是“2”，可能属于格式不符合要求；在图中只有一人时回答“两人”，属于事实错误；即使回答恰好正确，如果换一张人数不同的图仍给出同一结果，则提示模型可能没有充分利用视觉条件。

视觉幻觉与语言先验 视觉幻觉通常指模型输出了当前图像无法支持、甚至与其直接冲突的信息，例如描述不存在的对象、错误属性或不符合画面的空间关系。它可能来自视觉表征不足、连接模块信息损失、语言模型先验过强，也可能来自指令数据中存在的错误回答。

POPE[25] 研究针对视觉语言模型的对象幻觉进行系统评价，指出视觉指令中对象出现频率及其共现关系可能与幻觉倾向有关，并设计了基于对象存在性的询问式评价。HallusionBench 则利用受控问题考察语言幻觉与视觉错觉相互交织的情形。这些工作说明，仅依赖人工浏览少量流畅回答，难以可靠判断模型是否充分利用视觉信息。需要设计能够区分视觉证据和语言猜测的评价协议。

受控图像变化与成对评价 一种直接的证据利用诊断是构造具有相同问题、但关键视觉事实不同的成对样本。例如，问题始终为“杯子是什么颜色”，但两张图像分别显示红色和蓝色杯子。模型不仅需要分别答对，还需要随着观测变化改变答案。对于第 i 组控制样本 (I_i, Q_i, Y_i) 与 (I_i', Q_i, Y_i')，可以定义成对正确率：

$$A_{\text{pair}} = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}[\hat{Y}_i = Y_i \ \wedge \ \hat{Y}_i' = Y_i'] \tag{11.10}$$

其中，两组样本应保证关键视觉变量被合法改变，且各自答案经过验证。相比只计算单样本平均正确率，成对正确率能够更直接地检查模型是否在同一任务下区分不

同视觉条件。但图像修改必须控制无关变化。如果用生成模型改变杯子颜色时，同时改变背景、角度和其他对象，就可能引入额外混杂因素。因此，受控评价最好使用可靠的场景编辑、真实成对采集或可验证的合成数据。

无图像与错配输入诊断 另一种常见方式是比较完整多模态输入与缺失或替换模态时的模型表现。例如，在相同文本问题和回答规则下，比较完整图像输入与无图像输入的任务准确率：

$$\Delta_V = A_{\text{with image}} - A_{\text{without image}} \tag{11.11}$$

这一差值可以作为视觉信息贡献的辅助诊断，但不是视觉依赖的充分证明。因为删除图像可能使输入格式发生变化，模型的响应策略也可能随之改变；一些问题本身依靠语言常识就能回答，而另一些问题必须观察图像。因而，应按任务类型分别分析，并尽量保持输入格式和其他条件一致。

图像错配或视频帧替换也可以用来检查模型对条件的敏感性，但必须确保相应问题和正确答案在替换后仍被明确重标注。不能把替换后的错误答案率简单解释为性能退化，因为原有目标答案可能已不再适用。

多模态指令类型与监督要求比较 上述类别并非互斥。一段视频可以同时包含 OCR、语音、事件和多轮问答；一项机器人任务也可能同时依赖视觉、声音和动作记录。因此，数据集管理应保存每条样本的实际证据来源，而不只根据最终输出是否为文本进行分类。

11.3 低秩适配与模块选择

算例：低秩更新的参数量

对 $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$，采用 $\Delta W = BA$，其中 $B \in \mathbb{R}^{d_{\text{out}} \times r}$、$A \in \mathbb{R}^{r \times d_{\text{in}}}$。可训练矩阵参数由 $d_{\text{out}} d_{\text{in}}$ 变为 $r(d_{\text{out}} + d_{\text{in}})$。这项计数不包含激活、优化器状态或其他被解冻模块；低秩参数较少不直接给出峰值显存或延迟。

11.3.1 全参数微调与参数高效微调

全参数微调的原理 全参数微调（Full Fine-Tuning）通常以预训练模型参数作为初始化，在下游任务数据上更新模型中的全部可训练参数。设预训练模型表示为：$y = f(x; \theta_0)$，其中，(x) 表示输入，(y) 表示模型输出，θ_0 表示预训练参数。全参数微调根据下游任务数据优化模型参数：

$$\theta^* = \arg\min_{\theta} \mathbb{E}_{(x,y)\sim\mathcal{D}} \left[\mathcal{L}(f(x;\theta), y)\right] \tag{11.12}$$

其中，$\mathcal{D}$ 表示下游训练数据，$\mathcal{L}$ 表示任务对应的损失函数。全参数微调允许模型

在较大范围内调整内部表示和信息处理方式。当目标任务与预训练数据存在明显差异，或者需要对模型内部的多个组成部分进行适配时，这种灵活性可能具有价值。

但其代价也比较明显。训练过程需要为大量参数维护梯度和优化器状态，训练后的模型通常还需要保存完整的任务专属权重。如果需要适配多个任务，分别存储多个完整模型也会增加部署成本。全参数微调并不意味着每个参数都会发生显著变化，也不意味着其性能必然优于参数高效微调。最终效果仍然取决于训练数据、任务差异、模型结构和优化设置。

参数高效微调的原理 参数高效微调通过限制可训练参数集合，减少任务适配所需的资源。设模型参数由冻结参数 θ_f 和可训练参数 ϕ 构成，则模型可以写为：$y = f(x; \theta_f, \phi)$ 训练时保持 θ_f 不变，只更新 ϕ：

$$\phi^* = \arg\min_{\phi} \mathbb{E}_{(x,y)\sim\mathcal{D}} \left[\mathcal{L}(f(x; \theta_f, \phi), y)\right] \tag{11.13}$$

其中，可训练参数可以是新增的低秩矩阵、小型适配器、可学习提示向量，也可以是模型中选定的原有参数。参数高效微调的核心不是某一种固定结构，而是限制需要优化的参数范围，同时尽可能利用预训练模型已有的能力。

参数高效微调的主要优势 参数高效微调通常具有以下优势。减少梯度与优化器状态的存储需求。冻结参数不需要为常规梯度更新保存梯度，也不需要维护对应的优化器状态。降低任务适配参数的存储成本。多个任务可以共享同一基础模型，只为每个任务保存相对较小的适配参数。便于任务切换与模型管理。不同任务可以使用各自的 LoRA 权重或 Adapter，在共享基础模型的情况下切换任务专属参数。

降低部分训练资源门槛。对于大型模型，减少可训练参数有助于控制优化开销，使更多任务适配实验能够在有限硬件条件下进行。但这些优势需要准确理解。冻结参数仍然需要参与前向计算；如果可训练模块位于冻结模块之前，反向传播仍可能需要经过冻结模块的计算路径。因此，减少可训练参数不等于按相同比例减少总计算量，也不意味着激活显存一定很小。

参数量、训练显存与推理成本的区别 评价微调方法时，应至少区分三个维度。例如，LoRA 的权重通常可以合并到原始线性层中，从而避免额外的推理分支；而某些 Adapter 结构会增加前向计算。QLoRA[26] 则通过量化冻结的基础模型降低基础权重的存储开销。因此，不能仅凭可训练参数占比很低，就认定一种方法在所有资源指标上都更高效。

参数高效微调的适用边界 参数高效微调的前提，是预训练模型已经具备一定的目标任务基础能力。它主要调整模型对已有能力的利用方式，不一定能够弥补预训练阶段从未学到的信息。例如，如果视觉编码器无法识别高分辨率文档中的细小字符，仅在语言模型中增加 LoRA 参数未必能够解决问题。如果视觉特征已经足够丰富，而模型主要需要学习新的问答格式、领域表达或任务行为，那么只更新语言模型的一部分参数可能已经足够。

因此，参数高效微调并不是在任何情况下都能替代全参数微调。它是一种参数更新策略，其适用性需要结合任务差异、数据规模和模型各组成部分的能力进行判断。

11.3.2 LoRA 的基本原理

低秩适配的机制 低秩适配（Low-Rank Adaptation，LoRA）是大型语言模型和多模态模型中应用广泛的参数高效微调方法。其基本思路是冻结原有权重矩阵，只学习一个低秩形式的权重增量。设线性层的原始权重矩阵为：

$$W_0 \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}} \tag{11.14}$$

全参数微调直接更新 W_0，而 LoRA 将有效权重表示为：$W = W_0 + \Delta W$，其中，ΔW 是需要学习的权重增量。LoRA 进一步将该增量表示为两个低秩矩阵的乘积：$\Delta W = BA$，其中：

$$A \in \mathbb{R}^{r \times d_{\text{in}}}, \qquad B \in \mathbb{R}^{d_{\text{out}} \times r} \tag{11.15}$$

(r) 为低秩维度，通常远小于输入和输出维度。训练时冻结 W_0，只更新 (A) 和 (B)。这一设计建立在一个基本假设之上：对于某个下游任务，所需的权重变化可能能够由低维参数空间近似表示。模型不必重新学习全部权重，而是通过少量参数描述任务相关的变化。

LoRA 的参数量 原始权重矩阵的参数量为：

$$P_{\text{full}} = d_{\text{out}} d_{\text{in}} \tag{11.16}$$

LoRA 新增的可训练参数量为：

$$P_{\text{LoRA}} = r(d_{\text{in}} + d_{\text{out}}) \tag{11.17}$$

当输入和输出维度均为 (d) 时，两者分别为：

$$P_{\text{full}} = d^2, \qquad P_{\text{LoRA}} = 2rd \tag{11.18}$$

因此，LoRA 参数量与原始矩阵参数量的比值为：

$$\frac{P_{\text{LoRA}}}{P_{\text{full}}} = \frac{2r}{d} \tag{11.19}$$

例如，设某个线性层的输入和输出维度均为 4096，LoRA 秩 (r=8)。原始矩阵有 16,777,216 个参数，而 LoRA 只需训练：$8(4096 + 4096) = 65,536$ 个参数，约为原矩阵参数量的 0.39%。这一比例只针对单个线性层。实际模型的可训练参数总量还取决于 LoRA 被应用到哪些层，以及是否同时训练偏置、连接模块或其他组件。

LoRA 的前向计算 对于输入向量 (x)，普通线性层的输出为：$y = W_0 x$，加入 LoRA 后，输出变为：

$$y = W_0 x + \frac{\alpha}{r} BAx \tag{11.20}$$

其中，α 为缩放超参数，(r) 为低秩维度。不同实现也可能采用不同的缩放约定，因此复现实验时需要明确具体配置。LoRA 在原有线性映射上叠加一个可训练的低秩更新分支。基础权重不需要直接更新，优化器只需维护低秩矩阵的训练状态。常见实现会将矩阵 (A) 随机初始化，将矩阵 (B) 初始化为零，使训练开始时的权重增量为零。这样，模型初始输出可以与原始基础模型保持一致，再逐步学习任务相关的变化。

LoRA 的目标模块 在 Transformer 中，LoRA 常用于注意力模块的线性投影，也可以用于前馈网络中的线性层。常见配置包括：

- Query 与 Value 投影：通过较少参数调整注意力的信息选择与内容映射。
- Query、Key、Value 和输出投影：覆盖更完整的注意力变换，但增加可训练参数。
- 注意力与前馈网络同时适配：允许更广泛的内部表示变化，但训练开销也会上升。
- 只适配部分高层模块：进一步压缩参数规模，但可能限制适应能力。

这些配置不存在普遍适用的最佳答案。不同模型的模块命名、权重组织和结构设计可能不同，不能直接照搬其他模型的配置。

LoRA 秩与缩放系数 LoRA 的秩 (r) 决定低秩更新能够表达的变化范围。较小的秩通常意味着更少的参数，但也可能限制任务适配能力；较大的秩提高了更新容量，同时增加参数与优化开销。缩放系数 α 用于控制低秩增量对输出的影响。在常见实现中，增量乘以 α/r，因此秩和缩放系数需要结合考虑。

此外，LoRA 的效果还受到目标模块、学习率、训练数据质量、训练步数和随机种子等因素影响。不能仅凭增加秩就断言模型性能一定提升。

LoRA 与 QLoRA 在部署时，LoRA 权重通常可以合并到原始权重中：

$$W_{\text{merged}} = W_0 + \frac{\alpha}{r} BA \tag{11.21}$$

合并后可以按普通线性层执行前向计算，避免额外的 LoRA 分支延迟。但合并后的模型会形成任务专属的完整权重副本；如果希望多个任务共享基础模型并动态切换适配器，则通常需要保留独立的 LoRA 权重。QLoRA 则将低秩适配与基础模型量化结合起来。其典型设置是将冻结的预训练模型权重量化为 4 bit，再训练 LoRA 参数，从而降低基础权重的存储需求。原始研究还提出了 NF4、双重量化和分页优化器等技术。

需要区分的是，LoRA 解决的是可训练参数的表示问题，而量化主要改变基础权重的数值存储与计算方式。两者可以组合，但量化误差、内核兼容性和数值稳定性仍需考虑。

11.3.3 适配器与其他轻量化方法

LoRA 是参数高效微调的重要方法，但不是唯一选择。不同 PEFT 方法在新增参数的位置、信息注入方式、训练开销和推理路径方面存在差异。

Adapter：在网络内部增加可训练模块 Adapter 是较早被广泛研究的参数高效迁移方法之一。其基本思路是在预训练网络的层之间或层内部插入小型可训练模块，而原有模型权重保持冻结。一种典型的瓶颈 Adapter 结构先将隐藏维度从 (d) 降至较小维度 (m)，经过非线性变换后再映射回 (d)：

$$\text{Adapter}(h) = W_{\text{up}}\sigma(W_{\text{down}}h) + b \tag{11.22}$$

其中，W_{down} 负责降维，W_{up} 负责恢复维度，通常 $m \ll d$。Adapter 的优势是模块边界清晰，可以为不同任务保存不同的适配器；其局限是新增模块通常处于前向计算路径中，可能增加推理延迟和计算开销。

Prompt Tuning 与 Prefix Tuning Prompt Tuning 和 Prefix Tuning 都通过增加可学习的连续向量，引导冻结模型完成下游任务，但二者的参数注入位置有所区别。Prompt Tuning 通常在输入嵌入序列前添加可学习的软提示向量。它们并不是人工书写的离散词语，而是直接通过训练优化得到的连续向量：$X' = [P; X]$，其中，(P) 是可训练的软提示，(X) 是原始输入嵌入。

Prefix Tuning 则为 Transformer 的注意力层引入可学习前缀，通常将其作为注意力计算中的额外上下文，使模型在主体参数冻结的条件下适应任务。这两类方法的参数量通常较小，但效果会受到提示长度、注入位置、模型规模和任务类型影响。对于视觉语言模型，还需要考虑软提示作用于文本输入、视觉输入还是跨模态交互模块。

BitFit：只更新偏置参数 BitFit 是一种较为简单的参数高效微调方法，其基本思想是冻结大部分预训练参数，只更新模型中的偏置项。与增加 Adapter 或低秩矩阵相比，BitFit 不需要引入复杂的新模块，参数管理也较为简单。但它对可训练参数范围的限制更强，适应能力可能受到影响。当任务需要明显改变注意力模式、特征映射或模态连接方式时，仅调整偏置参数未必足够。因此，BitFit 可以作为参数极少的对照方法，但不应预设其一定比其他方法更有效。

IA^3：对内部激活进行可学习缩放 IA^3 通过学习少量缩放向量，对 Transformer 内部的激活进行调整。例如，可以对注意力中的 Key、Value 或前馈网络的中间激活进行逐通道缩放。其概念性形式可以表示为：$\tilde{h} = s \odot h$，其中，(h) 为原始激活，(s) 为可学习的缩放向量，$\odot$ 表示逐元素乘法。

与 LoRA 通过低秩矩阵表达权重增量不同，IA^3 通过少量缩放参数改变已有网络内部的信息流。它的参数开销可以很小，但表示变化的形式也受到限制。

DoRA 等 LoRA 扩展方法 后续研究也对 LoRA 的参数化方式进行了扩展。例如，DoRA 将权重分解为幅值与方向相关的部分，再利用低秩适配调整其中的变化，以期改善某

些任务中的适配能力。这类方法说明，参数高效微调仍在持续发展。但新增机制不一定在所有模型、数据和任务中都取得更好结果。评价时应明确比较条件，并区分特定实验中观察到的收益与普遍适用的结论。

主要方法的比较 上述方法并非完全互斥。例如，LoRA 可以与量化结合，也可以只作用于语言模型的部分线性层；部分方案也可以与可训练连接模块共同使用。实际方案需要明确哪些参数被冻结、哪些参数可训练，以及不同方法是否改变推理计算路径。

11.3.4 可训练模块的选择与效果比较

多模态模型的主要组成部分 典型视觉语言模型通常包含三个主要部分：

$$\text{视觉编码器} \rightarrow \text{模态连接模块} \rightarrow \text{语言模型} \tag{11.23}$$

视觉编码器负责提取图像或视频帧的特征；连接模块将视觉表示映射为语言模型能够利用的条件表示；语言模型根据视觉条件和文本指令生成回答。采用参数高效微调时，可以选择不同的模块进行更新：

- 只训练连接模块；
- 只在语言模型中加入 LoRA；
- 同时训练连接模块与语言模型 LoRA；
- 在视觉编码器中加入 LoRA，或解冻部分视觉层；
- 同时适配视觉编码器、连接模块和语言模型中的指定部分。

不同选择对应不同的能力假设。只训练连接模块，意味着认为语言模型已经具备足够的任务能力，只需要改善视觉信息的映射；只训练语言模型 LoRA，则意味着现有视觉输入接口基本足够，主要需要调整语言模型的任务响应行为。

依据任务问题选择模块 模块选择不应只根据参数量决定，而应结合任务失败的原因。情况一：模型回答格式不正确，但视觉内容基本准确。问题可能主要涉及语言模型的指令遵循和输出组织。可以优先验证语言模型 LoRA，并保留视觉编码器和连接模块。情况二：模型能识别主要对象，但难以回答颜色、数量或局部关系问题。

问题可能与视觉特征粒度、连接模块的信息保留能力，或语言模型对视觉证据的使用有关。应先检查视觉编码器是否包含必要信息，再决定是否训练连接模块、语言模型或视觉模块。情况三：输入来自明显不同的视觉领域。例如，模型从自然图像迁移到专业仪器图像或复杂文档。如果视觉编码器的表示与目标数据存在较大差距，单独调整语言模型可能不足，此时可以评估视觉编码器的部分解冻或参数高效适配。

情况四：基础模型和连接路径已经具备足够能力，但训练资源受限。可以优先比较 LoRA、QLoRA 或其他轻量化方法，并记录训练吞吐、显存、任务指标和适配器存储成本。以上判断是实验设计的起点，而不是无需验证的结论。同一任务的失败可能由多个模块共同造成，因此需要通过受控实验定位原因。

参数减少不等于性能必然提高 参数高效微调的价值，应从实际任务表现与资源成本综合判断。例如，某个 LoRA 配置可能只训练总参数中的很小一部分，但如果目标模块选择不合理，模型仍可能无法充分适应目标任务。另一方面，全参数微调虽然成本更高，却可能在任务与预训练分布差异较大时提供更充分的适应能力。因此，需要避免以下几种不严谨的结论：

- 可训练参数更少，所以方法一定更高效；
- 显存占用更低，所以训练时间一定更短；
- 使用更高的 LoRA 秩，所以模型性能一定更好；
- PEFT 与全参数微调得到相似结果，因此二者在所有任务上等价。

更严谨的做法是报告具体任务、数据规模、模型结构、训练设置和评价结果，并明确结论适用的范围。

多任务场景中的适配器管理 在多个下游任务中，PEFT 还可以减少模型版本管理的负担。基础模型共享一份权重，不同任务分别保存自己的 LoRA 或 Adapter 参数。部署时根据任务加载对应适配器，避免为每个任务保存完整的独立模型。但如果多个适配器需要同时使用，或者需要将多个任务的适配器合并为一个模型，就需要进一步考虑参数合并后的冲突、任务间干扰以及推理开销。因此，参数高效微调不仅是一种训练方法，也是一种模型部署和任务管理策略。

实验复现与结果报告 最终报告应明确记录：

- 基础模型及其预训练权重版本；
- PEFT 方法、目标模块、秩、缩放系数和其他关键超参数；
- 哪些模块被冻结，哪些模块参与更新；
- 训练数据、数据混合比例和有效训练预算；
- 任务指标、随机种子、计算成本与推理设置；
- 检查点保存方式、适配器合并方式和推理复现条件。

只有这些信息足够明确，其他研究者才能判断实验结果是否来自 PEFT 方法本身，还是来自不同的数据、预算或训练配置。

11.4 奖励建模与策略优化

11.4.1 偏好数据与奖励建模

偏好数据的基本形式 偏好数据通常由同一条件下的多个候选输出及其相对评价构成。以视觉问答为例，设输入条件为 $x = (I, q)$，其中 I 为图像，q 为用户问题。模型生成两个回答 y_a 与 y_b，评价者根据指定标准选择更优的回答。如果评价结果认为 y_w 优于 y_l，可以将一条偏好样本表示为：$D_i = (x_i, y_{w,i}, y_{l,i})$，

其中，下标 w 与 l 分别表示被偏好的回答（winner）和未被偏好的回答（loser）。例如，对于一张包含两辆汽车的道路图像，问题为“图中有几辆汽车？”，候选回答分别为“有两辆汽车”和“有三辆汽车”。在对象数量能够可靠核实的前提下，第一种回答应当获得更高的事实正确性评价。

但并不是所有偏好都具有如此明确的客观依据。对于“请详细描述这张图像”，两段回答可能都准确，只是在信息完整度、篇幅或表达方式上存在差异。因此，偏好数据应当记录比较标准，不能把所有“更喜欢”都等同于“事实更正确”。此外，偏好标签也可能包含平局、难以判断或多个评价维度。强制评价者在质量接近的候选之间作出选择，可能放大标注噪声。实际数据集可以根据任务需要记录平局、分维度评分和评价者之间的不一致性。

偏好与监督示范的区别 监督微调样本通常表示为 (x, y^*)，训练目标是提高参考回答 y^* 的条件概率。偏好样本则表示为 (x, y_w, y_l)，它提供的是相对质量关系，而不是唯一的正确输出。这种区别在开放式生成任务中尤其重要。例如，两段图像描述都可能没有事实错误，但其中一段更精炼，另一段涵盖更多细节。偏好反馈可以用来表达这些软性的质量差异。

不过，偏好数据并不能自动补充缺失的知识。如果两个候选回答都错误，只是在错误程度上不同，选择其中较好的一个仍然不能保证模型最终学到正确结论。因此，高质量偏好数据需要兼顾候选多样性和事实核验。

奖励模型的学习目标 奖励模型（Reward Model，RM）试图将输入与候选输出映射为一个实数分值：$r_\phi(x, y) \in \mathbb{R}$，其中，$\phi$ 为奖励模型参数。分值通常表示在当前偏好评价标准下模型对输出质量的估计，而不应被理解为客观、普遍有效的真实效用。一种常用方式是根据 Bradley–Terry 型成对比较模型，假设候选 y_w 优于 y_l 的概率为：

$$P_\phi(y_w \succ y_l \mid x) = \sigma(r_\phi(x, y_w) - r_\phi(x, y_l)) \tag{11.24}$$

其中，σ 为 Sigmoid 函数。相应的奖励模型训练损失为：

$$\mathcal{L}_{\mathrm{RM}}(\phi) = -\mathbb{E}_{(x, y_w, y_l)} \log \sigma[r_\phi(x, y_w) - r_\phi(x, y_l)] \tag{11.25}$$

这一目标鼓励奖励模型对偏好回答给出较高分数。成对偏好本身通常只确定相对差异，奖励的绝对数值、尺度与校准方式可能依赖模型和训练设置；两个独立训练奖励模型的原始分数并不一定能够直接比较。

偏好数据的采集与标注质量 成对偏好可以来自人工评价，也可以通过具有明确评分规则的自动系统或其他模型辅助产生。人工标注时，应明确评价者是否看到原始图像或视频、是否能够暂停查看关键帧、是否具有所需的领域知识，以及对准确性与表达风格如何排序。

例如，对于医学影像描述，如果评价者无法确认图像中的专业征象，仅根据文字表达是否流畅选择答案，所生成的偏好数据就可能奖励语言风格而非医学事实性。对

于视频问答，缺少时间戳或未提供完整片段，也可能使评价者误判事件顺序。因此，数据构建应关注标注指引、候选输出来源、评价者一致性、分歧样本复核以及来源与版本记录。偏好数据也应划分独立的训练、验证和测试集，避免同一图像及其近重复问题在多个划分之间泄漏。

偏好模型的评价 奖励模型的成对偏好预测准确率，是一种基本评价指标，但并不能完整体现其质量。一个模型可能在已见过的问题类型上正确预测人工选择，却在新领域、复杂视觉关系或较长回答中失效。更重要的是，奖励模型可能偏好更长、更自信或语言更流畅的回答，而忽视其中的视觉事实错误。因此，需要在独立样本上检查奖励与人工偏好的一致程度，比较不同长度和不同任务类别的结果，并通过真实证据核验检验其事实判断能力。

经典的人类反馈研究，如 Christiano 等人的 Deep Reinforcement Learning from Human Preferences 以及 Stiennon 等人的 Learning to Summarize with Human Feedback，为利用人类相对偏好学习奖励函数提供了方法基础。其核心启示不是奖励模型能够完全替代人类判断，而是人类难以直接写成解析公式的部分质量要求，可以通过有限的比较反馈近似建模。

11.4.2 基于人类反馈的强化学习

经典 RLHF 的训练流程 基于人类反馈的强化学习通常在已经完成预训练或监督微调的生成模型基础上进行。以 InstructGPT 为代表的经典路线可以概括为三个主要阶段。利用人工示范构建监督微调模型，使模型能够按照输入指令生成合理回答。随后，针对同一输入生成多个候选回答，由人工评价者进行排序或成对比较，并据此训练奖励模型。以生成模型作为策略（Policy），利用奖励模型提供的分值，通过强化学习进一步更新模型。该流程可以表示为：

$$\text{监督微调模型} \longrightarrow \text{人类偏好与奖励模型} \longrightarrow \text{策略强化学习} \tag{11.26}$$

上述过程是具有代表性的 RLHF 训练流程，并不是所有后训练方法都必须严格遵循的固定结构。例如，有些系统会多次交替收集反馈和更新策略，也有方法直接使用可验证奖励或离线偏好数据，而不单独训练奖励模型。

语言模型作为生成策略 在生成式模型中，可以将输入条件 x 看作任务上下文，将输出序列 y 看作策略选择的结果：

$$y \sim \pi_\theta(\cdot \mid x) \tag{11.27}$$

其中，π_θ 表示当前可训练策略。在视觉语言模型中，x 可包含图像、文本问题及历史对话；对于文本生成图像任务，x 可以是文本条件，但其生成过程并不一定具有普通语言 Token 的相同动作定义。如果奖励模型给出 $r_\phi(x, y)$，最直接的目标是提高模型所生成回答的期望奖励。然而，如果只追求奖励最大化，模型可能产生偏离原有训练分布的输出，或者学会利用奖励模型的缺陷。

因此，经典 RLHF 通常还引入参考策略约束，限制更新后的模型相对于初始模型发生过大的变化。

带 KL 约束的策略优化 设参考策略为 π_{ref}，则一种常见的优化目标为：

$$\max_{\theta} \mathbb{E}_{x,\, y\sim\pi_\theta(\cdot|x)}\left[r_\phi(x,y)\right] - \beta\, \mathbb{E}_x\left[D_{\text{KL}}(\pi_\theta(\cdot \mid x) \,\|\, \pi_{\text{ref}}(\cdot \mid x))\right] \tag{11.28}$$

其中，β 控制奖励提升与参考策略偏移之间的权衡。KL 散度惩罚用于限制策略远离参考模型，但不能直接保证输出事实正确或完全避免奖励投机。实际训练常对采样输出的逐 Token 对数概率差构造 KL 估计或相应惩罚，具体实现与上式的理想化期望形式可能有所不同。

PPO 在 RLHF 中的作用 近端策略优化（Proximal Policy Optimization，PPO[27]）是经典 RLHF 中常用的策略更新方法。它利用当前策略与采样时旧策略之间的概率比，并通过裁剪机制约束单次更新幅度。设策略概率比为：

$$\rho_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)} \tag{11.29}$$

一种典型的 PPO 裁剪目标为：

$$\mathcal{J}_{\text{PPO}}(\theta) = \mathbb{E}_t\left[\min\left(\rho_t(\theta)A_t, \text{clip}(\rho_t(\theta), 1-\epsilon, 1+\epsilon)A_t\right)\right] \tag{11.30}$$

其中，A_t 是优势估计，ϵ 为裁剪范围。对于语言模型，状态可以与已有输入和生成前缀对应，动作通常是下一个 Token 的选择。这一目标只是 PPO 策略更新的核心部分。实际 RLHF 还可能包括价值估计、优势归一化、参考策略约束和其他稳定训练机制。PPO 的裁剪机制也不等于对策略变化提供绝对保证。

在线采样及其成本 经典 PPO 式 RLHF 需要反复使用当前或旧策略生成候选回答，再计算奖励和优势，并进行多次优化更新。相比只使用固定答案数据的监督微调，这通常需要额外的生成计算，以及奖励模型、参考模型和可能的价值网络等组件。因此，RLHF 的资源成本不仅来自模型参数更新，还包括在线采样、奖励计算和多模型协同。对于视觉语言模型，若每次采样都需要编码多张高分辨率图像或长视频，训练吞吐还可能受到多模态输入成本的明显影响。

不过，不能因此认为全部强化学习后训练都必须采用 PPO 或同样的组件。后续研究提出了通过组内相对奖励估计优势的 GRPO 等方法，以减少某些 PPO 实现中的价值模型开销；这些方法仍然需要结合实际的采样方式、奖励来源和优化约束进行分析。

奖励投机与能力退化 奖励投机（Reward Hacking）是指模型通过利用奖励函数的不足，提高自动评分，却没有真正改善目标行为。例如，如果奖励模型过度偏好回答篇幅，策略可能倾向生成更长文本；如果评价器只检查回答是否包含某些关键词，模型可能通过重复关键词获得较高分数；如果视觉奖励模型对图片内容判断不准确，模型可能学会生成更符合评分器偏好的措辞，而非严格依据图像回答。

Stiennon 等人的摘要反馈研究已经指出，对代理奖励进行过强优化，可能使实际人类评价质量下降。因此，奖励提升不能替代独立的人类评价和任务指标。同时，强化学习可能改变模型原有的语言与多模态能力。训练过程中应持续检查事实正确性、任务完成率、输出长度、策略与参考模型的偏移，以及对其他任务的影响。

11.5 直接偏好优化与多模态反馈

标准语言模型 DPO 比较同一条件下两个回答的相对序列概率，并相对于固定参考策略构造目标。将条件扩为图像、问题和历史时，两条候选及两种策略必须读取相同的条件。改变图像来构造偏好对属于另一种设计，需要另行说明其目标。

11.5.1 DPO 等直接偏好优化方法

直接偏好优化的基本动机 经典 RLHF 将“偏好学习”与“策略更新”划分为两个环节：先训练显式奖励模型，再使用强化学习优化生成策略。这种流程能够利用在线生成数据，但实现相对复杂，并且涉及奖励模型误差和多模型计算成本。

直接偏好优化（Direct Preference Optimization，DPO[28]）尝试在特定理论假设下，将带 KL 约束的奖励最大化问题转化为对生成策略的直接优化。其主要特点是使用偏好对训练模型，而不需要单独拟合一个显式奖励模型，也不需要在标准离线 DPO 训练过程中执行 PPO 式在线策略采样。

DPO 的代表性研究为 Rafailov 等人发表于 NeurIPS 2023 的 Direct Preference Optimization: Your Language Model is Secretly a Reward Model 。

DPO 的训练数据与参考模型 DPO 通常使用偏好数据：

$$\mathcal{D}_{\text{pref}} = \{(x_i, y_{w,i}, y_{l,i})\}_{i=1}^{N} \tag{11.31}$$

并同时使用当前策略 π_θ 和固定的参考策略 π_{ref}。参考策略通常来自监督微调后的模型，用于描述优化前的输出分布。在多模态视觉问答中，x 应包含相同的图像及相同的问题；y_w 和 y_l 则是针对这一条件的两个候选回答。若比较过程中图像或指令被改变，就不再是标准的“同一条件下比较两个回答”的设置，需要另行定义训练目标。

DPO 的目标函数 令：

$$\Delta_\theta = \log \pi_\theta(y_w \mid x) - \log \pi_\theta(y_l \mid x) \tag{11.32}$$

$$\Delta_{\text{ref}} = \log \pi_{\text{ref}}(y_w \mid x) - \log \pi_{\text{ref}}(y_l \mid x) \tag{11.33}$$

则标准 DPO 损失可以写为：

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}_{\text{pref}}} \log \sigma[\beta\,(\Delta_\theta - \Delta_{\text{ref}})] \tag{11.34}$$

其中，β 控制相对于参考策略的偏好优化强度。在原始 DPO 推导中，它与带 KL 约束奖励目标的系数有关，实际训练效果还依赖数据分布和优化设置。该损失鼓励当前策略相对于参考策略，更偏向人类选择的回答。它并不是单纯最大化 $\pi_\theta(y_w \mid x)$，也不是简单地把未被偏好回答的概率降到零，而是通过两个回答的相对对数概率及参考策略构造学习信号。

DPO 与 SFT 的区别 监督微调通常最小化：

$$\mathcal{L}_{\text{SFT}} = -\mathbb{E}_{(x,y^*)} \log \pi_\theta(y^* \mid x) \tag{11.35}$$

其主要目标是提高示范答案的概率。DPO 则同时利用被偏好与未被偏好的回答，直接优化模型对二者的相对选择。例如，一张图像中有两辆汽车。SFT 可以仅训练模型生成“图中有两辆汽车”；DPO 则可以同时利用正确回答和错误回答，要求模型相对于参考策略更加偏好前者。

但是，DPO 并不保证能够解决所有事实错误。如果偏好标签本身来自不可靠的视觉判断，DPO 仍然可能强化错误答案。因此，偏好数据的真实性和比较标准，是方法有效性的基础。

多模态条件下的 DPO 对视觉语言模型而言，可以将标准 DPO 中的文本条件 x 扩展为图像、文本问题和历史上下文的组合：$x = (I, q, h)$，其中，I 表示视觉输入，q 表示当前问题，h 表示可选的历史上下文。策略和参考策略都应在相同的多模态条件下计算候选回答概率。

例如，RLHF-V 利用针对视觉幻觉的细粒度人工纠错，构造更具针对性的偏好反馈，并采用针对纠错片段加强监督的直接偏好优化。虽然论文名称包含 RLHF，但其提出的核心优化方法并非经典 PPO 式在线 RLHF，而是基于细粒度反馈的直接偏好优化。

对于图像生成扩散模型，不能直接将语言模型的离散序列 DPO 公式原封不动应用到连续去噪过程。Diffusion-DPO 针对扩散模型的概率建模特征重新推导了偏好优化目标，展示了将直接偏好思想推广到图像生成的可能性。

11.5.2 多模态奖励信号与优化目标

多模态奖励为什么更加复杂 对于纯文本回答，奖励可以关注内容正确性、指令遵循、语言表达和其他行为要求。但对于多模态系统，还必须明确回答与原始观测是否一致。例如，用户上传一张包含两辆汽车的图像，要求模型回答汽车数量。模型输出“有两辆汽车”与“画面中有三辆汽车”之间的区别，不能只由表达是否流畅决定，而必须依据图像中的实际对象进行判断。

类似地，文本条件图像生成不仅要求输出具有较好的视觉质量，还要求生成内容符合文本中指定的对象、数量、属性和空间关系。视频生成则进一步涉及动作连续性、时间顺序和视觉一致性。因此，多模态奖励的核心不只是评价“输出好不好”，而是评价“输出是否满足给定多模态条件下的特定要求”。

视觉语言问答中的事实性奖励 在视觉问答中，可以区分若干评价维度：视觉事实一致性：回答中的对象、颜色、数量和关系是否得到图像支持。任务完成程度：是否实际回答了用户的问题，而不是只生成笼统描述。表达质量：回答是否清晰、完整且符合任务要求。不确定性处理：当图像模糊或证据不足时，模型是否能够避免给出未经证实的确定结论。

这些维度可以作为奖励设计的参考，但不一定适合简单合并为一个固定分数。特别是涉及客观事实时，较高的语言流畅度不应抵消明显的视觉错误。对于部分任务，可以使用具有确定真值的奖励，例如对象计数、数值计算或严格格式匹配。对于开放描述，则可能需要人工或多模态评价模型判断内容是否有视觉依据。

多维奖励的组合与约束 如果某个任务确实需要多个奖励指标，可以采用加权组合的形式：

$$R(x,y)=\sum_{k=1}^{K} w_k R_k(x,y) \tag{11.36}$$

其中，R_k 表示第 k 个质量维度，w_k 为相应权重。这只是一种便于说明的形式，并不意味着任何多模态任务都适合将所有指标线性相加。例如，如果视觉事实错误属于不可接受的失败条件，就可以将事实核验设计为约束、门槛或独立评价维度，而不是允许美学质量或语言流畅度补偿事实错误。同样，不同奖励模型的分数尺度可能差异较大。在组合之前应检查其范围、分布和校准方式，并通过消融实验评估每个指标的实际作用。

奖励优化的风险与评价 多模态偏好优化需要特别警惕三个问题。首先是奖励投机。模型可能学会满足评分器的表面特征，而没有提升视觉事实性或任务能力。其次是偏好分布偏差。数据中的评价者、候选模型及问题类型，可能使某种回答风格被系统性高估。最后是能力退化。针对某一奖励进行持续优化，可能改变模型的输出长度、生成多样性或原有基础任务表现。

因此，评价不应只报告训练奖励或偏好胜率。对于视觉语言模型，还应检查对象幻觉、属性和数量正确性、受控图像替换、指令遵循、未见场景泛化及纯文本能力保持；对于生成图像和视频，还需分别评价条件遵循、感官质量与时空一致性。特别是在多模态领域，单一的综合奖励分数难以覆盖全部能力。高奖励、较高胜率和真实任务性能改善应被视为三个需要分别验证的结论。

典型方法与训练目标比较 方法名称并不自动决定所使用的数据与训练流程。例如，RLHF-V 虽然包含 RLHF 字样，其核心训练方案是细粒度直接偏好优化；Diffusion-DPO 虽然借鉴 DPO 思想，却需要针对扩散生成过程重新推导目标。比较不同研究时，应以实际损失函数、奖励来源和参数更新机制为依据。

11.6 能力保持与分布偏差

11.6.1 视觉能力与语言能力的平衡

视觉语言模型中的能力构成 视觉语言模型（Vision-Language Model，VLM）通常需要同时处理视觉信息和语言信息。以图像问答为例，模型需要根据输入图像和自然语言问题生成回答，其条件生成过程可以表示为：

$$P_\theta(y \mid I, q) \tag{11.37}$$

其中：

- I 表示输入图像；
- q 表示自然语言问题；
- y 表示生成的回答；
- θ 表示模型参数。

从表面上看，模型只需要学习从图像和问题到回答的映射关系。但实际上，这一过程涉及多个能力环节。

模型需要识别图像中的相关内容，例如物体类别、颜色、数量、文字和空间位置。模型需要理解自然语言问题，判断用户希望获取什么信息。模型需要将视觉信息与语言知识结合起来，生成符合问题要求的回答。例如，用户提供一张交通场景图像，并询问：“图中最靠近路口的汽车是什么颜色？”模型需要识别道路中的车辆，理解“最靠近路口”这一空间关系，找到满足条件的目标，然后输出相应颜色。

因此，视觉语言模型的能力并不是单一的，而是由多个相互联系的部分构成：视觉感知能力主要负责识别图像中的对象、属性、文字和空间结构。语言理解与生成能力主要负责理解指令、组织回答、调用知识和进行语言推理。跨模态关联能力负责建立视觉内容与语言表达之间的对应关系。多模态推理能力则需要在上述能力基础上完成信息整合、关系判断和复杂问题求解。在后训练过程中，某一方面的性能提升并不必然意味着其他能力也同步提高。

例如，模型可能经过大量图像描述数据训练后，更擅长生成详细的视觉描述，但在纯文本数学推理任务上出现性能下降。反过来，模型也可能具有很强的语言推理能力，却无法准确识别图像中的细粒度差异。因此，多模态模型的能力发展需要关注不同能力之间的协调，而不是仅仅追求某个综合评价分数。

语言先验对视觉理解的影响 预训练语言模型通常已经从大规模文本语料中学习到丰富的语言规律和世界知识。这些知识能够帮助多模态模型理解视觉场景，但也可能导致模型过度依赖语言先验（Language Prior），忽略输入图像中的真实证据。例如，假设模型看到一张厨房图像，其中桌面上只有两个杯子。当用户询问：“桌面上有几个杯子？”模型可能正确回答“两个”。但如果用户进一步询问：“桌面上的三个杯子分别是什么颜色？”

模型可能受到问题中“三个杯子”这一错误预设的影响，直接描述三个杯子的颜色，而没有首先检查图像中的实际数量。在这种情况下，模型生成的回答可能在语言上十分自然，却与视觉证据不一致。这反映了视觉语言模型中的一个重要问题：语言表达合理，并不等于视觉理解正确。从条件概率角度看，理想的视觉问答模型应当充分利用图像与问题的联合信息：

$$P_\theta(y \mid I, q) \tag{11.38}$$

但当模型过度依赖语言先验时，其输出可能主要受到以下分布影响：

$$P_\theta(y \mid q) \tag{11.39}$$

也就是说，即使缺少图像信息，模型仍然可能根据问题本身和已有知识给出看似合理的答案。在某些视觉问答数据集中，问题的语言形式、答案分布和对象共现关系本身就包含较强的统计规律。例如，“草是什么颜色”通常对应“绿色”，“天空是什么颜色”通常对应“蓝色”。模型利用这些规律并不一定是错误的，但如果它们替代了对实际图像的观察，就会限制模型面对特殊场景的能力。

例如，一幅夕阳下的天空可能呈现红色，一张艺术作品中的草地也可能被绘制成蓝色。对于这类问题，模型需要根据当前视觉证据作答，而不能仅依赖一般常识。相关研究已经发现，一些多模态评价任务即使不提供图像，语言模型仍然能够取得高于随机猜测的成绩。这说明部分评价结果可能同时反映语言知识和视觉理解，而不能直接视为纯粹的视觉能力。

视觉能力与语言能力为什么可能产生冲突 视觉与语言能力之间的冲突，主要来自训练目标、数据分布和参数更新等因素。不同模态的训练数据往往具有不同特点。纯文本训练数据通常覆盖广泛的知识、推理和语言表达形式，而多模态指令数据可能更加集中于图像描述、视觉问答、对象识别或特定场景中的任务。如果后训练阶段大量使用某一类视觉问答数据，模型的优化方向就可能逐渐偏向这些任务。视觉语言模型中的语言模型参数可能同时承担通用语言能力和多模态回答生成任务。

当这些共享参数发生更新时，新任务的梯度可能与原有任务的优化方向不一致。以混合训练为例，假设模型同时使用视觉任务和语言任务，其总损失可以写为：

$$\mathcal{L} = \lambda_v \mathcal{L}_{\text{visual}} + \lambda_l \mathcal{L}_{\text{language}} \tag{11.40}$$

其中，$\mathcal{L}_{\text{visual}}$ 表示视觉或多模态任务损失，$\mathcal{L}_{\text{language}}$ 表示语言任务损失，λ_v 与 λ_l 为相应权重。两个损失的数值大小并不能直接代表其对参数更新的实际影响。不同任务的样本长度、损失归一化方式、梯度规模和数据采样频率都会影响训练结果。

如果视觉任务在优化过程中占据主导地位，就可能使部分参数更加适应视觉问答，而削弱原有语言任务的表现。类似地，如果训练过程中主要依赖语言生成损失，而视觉特征提供的监督较弱，模型也可能逐渐学会依赖语言模板完成任务，而没有充分利用视觉输入。不过，这并不意味着视觉能力与语言能力必然存在零和关系。合理的多

模态训练可能促进不同能力之间的迁移。例如，结合图表理解与数学推理训练，有时可以同时改善视觉信息读取和结构化推理能力。

真正需要研究的是：哪些训练条件能够产生能力互补，哪些条件会导致能力竞争或相互干扰。

保持视觉与语言能力平衡的主要方法 为了在提升多模态能力的同时尽可能保留通用语言能力，研究中通常采用几类方法。合理组织多模态与纯文本训练数据。在多模态后训练过程中，可以保留一定比例的纯文本指令、推理和知识任务。这种混合训练能够使语言模型在学习新任务时，继续接触原有能力对应的数据分布。不过，混合比例并不存在对所有模型和任务都适用的固定最优值。如果文本数据比例过高，模型可能难以充分适应视觉任务；如果视觉数据比例过高，又可能增加原有能力退化的风险。

因此，应当根据训练目标和验证结果调整数据组成。控制不同模型组件的更新范围。视觉语言模型通常包括视觉编码器、连接模块和语言模型。不同组件的训练策略可以有所区别。例如：

- 冻结语言模型，仅训练视觉连接模块，可以降低对原有语言参数的直接干扰。
- 冻结视觉编码器，训练连接模块及部分语言参数，可以增强视觉特征与语言空间之间的适配。
- 采用 LoRA 等参数高效微调方法，可以限制参数更新范围。
- 对视觉编码器和语言模型分别使用不同学习率，可以控制两部分的适配强度。

这些方法各有适用条件。

例如，冻结视觉编码器虽然能够保留其已有表征，但如果原始视觉特征无法表达目标任务所需的细粒度信息，仅调整连接模块可能不足以解决问题。同样，冻结语言模型可以避免直接修改其权重，却不能保证整个多模态系统在所有语言任务上保持完全相同的输出行为。提高训练样本对视觉证据的依赖程度。如果训练数据中的答案可以仅根据问题文本推断，模型就缺乏必须使用视觉信息的动力。

因此，可以构建视觉不可替代的训练样本，例如改变对象颜色、数量、位置或相互关系，并要求模型给出对应的不同回答。对于同一个问题，也可以提供内容不同的图像，让模型学习根据图像变化调整输出。这种数据设计能够减少模型仅依靠语言先验完成任务的机会。对不同能力设置独立的验证指标。训练过程中不应只监控多模态任务损失，而应同时观察视觉感知、视觉推理、纯文本理解和语言生成等方面的表现。

只有在这些指标之间建立合理的评价关系，才能判断后训练是否实现了预期的能力平衡。

如何判断模型真正利用了视觉信息 视觉语言模型能够回答某个视觉问题，并不能直接证明它使用了图像中的信息。一种简单的验证思路是设置视觉输入消融实验（Visual Input Ablation）。对于同一道问题，可以构造以下输入条件：

1. 完整图像输入：提供原始图像与问题。
2. 无图像输入：仅提供问题文本，或采用模型支持的无视觉输入方式。

3. 错误图像输入：将原始图像替换为内容不相关的图像。
4. 视觉反事实输入：保留问题形式，但改变图像中决定答案的关键属性。

例如，对于问题“图中有几辆汽车”，可以分别输入包含两辆汽车和四辆汽车的图像。

如果模型真正利用了视觉内容，其回答应当随实际汽车数量发生变化。可以定义一个简单的视觉贡献指标：

$$S_{\text{image+text}}, \qquad S_{\text{text-only}} \tag{11.41}$$

其中，$S_{\text{image+text}}$ 表示完整视觉语言输入下的任务成绩，$S_{\text{text-only}}$ 表示无图像条件下的成绩。如果完整输入表现明显优于无图像输入，说明视觉信息可能提供了额外的有效证据。但这一差值只是诊断指标，并不能单独证明模型形成了可靠的视觉理解能力。例如，移除图像可能改变模型的输入分布，错误图像可能引入额外干扰，而部分问题本身也可能不需要视觉信息。

因此，更合理的方式是将无图像对照、图像替换、反事实样本以及答案正确性联合分析。特别是对于高质量视觉语言模型，应当要求其不仅在图像变化时改变回答，还需要使改变后的回答与新的视觉证据一致。

11.6.2 灾难性遗忘与能力退化

灾难性遗忘的基本概念 灾难性遗忘（Catastrophic Forgetting）是神经网络在顺序学习多个任务或数据分布时可能出现的一类问题。当模型完成某项任务的训练后，如果直接在新的任务上继续更新参数，可能导致它在原有任务上的性能显著下降。例如，一个语言模型在预训练阶段已经具备文本摘要、知识问答和逻辑推理能力。随后，研究者使用大量视觉问答数据对其进行多模态指令微调。

训练完成后，模型可能在图像问答任务上取得更好的成绩，但在部分原有的纯文本推理任务上表现下降。这种现象可能反映了新任务学习对旧任务能力的干扰。从参数更新角度看，假设模型在旧任务训练后的参数为 θ_{old}，经过新任务训练后得到：

$$\theta_{\text{new}} = \theta_{\text{old}} + \Delta\theta \tag{11.42}$$

如果参数变化 $\Delta\theta$ 使模型原先依赖的表示或决策关系受到破坏，那么旧任务性能就可能下降。可以通过旧任务得分变化进行初步衡量：

$$S_k(\theta_{\text{old}}), \qquad S_k(\theta_{\text{new}}) \tag{11.43}$$

其中，S_k 表示模型在旧任务 k 上的评价得分。当 $F_k > 0$ 时，表示旧任务性能出现下降。这种前后差值只能说明观察到了性能变化。要进一步将其归因于灾难性遗忘，还需要排除推理配置改变、评价数据分布变化和随机波动等其他因素。

灾难性遗忘是持续学习（Continual Learning）研究中的经典问题。Elastic Weight Consolidation（EWC）和 Learning without Forgetting（LwF）等工作分别从参数约束

和知识蒸馏等角度探索了缓解方法。

多模态后训练中的不同能力退化 在多模态模型中，能力退化并不只有一种形式。通用语言能力退化。模型在多模态训练后，可能出现纯文本任务成绩下降的现象，例如知识问答准确率降低、数学推理错误增多，或者对复杂文字指令的遵循能力变差。这种变化与语言模型参数更新、训练任务分布变化及后训练目标有关。原有视觉能力退化。假设视觉语言模型原本具备较好的对象识别和图像描述能力，随后主要接受图表问答或文档理解任务训练。

如果训练数据过于集中，模型可能更加适应图表、文本密集型图像，却在普通自然图像理解任务上表现下降。这说明灾难性遗忘也可能发生在不同视觉任务之间，而不仅仅是语言能力受到视觉任务影响。跨模态关联能力退化。即使视觉编码器和语言模型各自仍然具有较强能力，二者之间的连接模块或对齐关系也可能受到后训练影响。例如，模型仍能识别图像中的物体，但无法准确将对象与语言描述对应起来。

对于空间关系、对象指代和多图关联等任务，这类问题可能尤其明显。指令遵循与输出行为的变化。经过偏好优化后，模型可能变得更加倾向于输出较长的回答，或者在不确定情形下表现得更加谨慎。某些变化可能有利于交互质量，但也可能降低特定任务的答案准确率、简洁性或任务完成率。需要区分的是，输出风格变化并不必然意味着知识或推理能力被遗忘。例如，模型因为回答格式变化而无法通过严格答案匹配，并不等于它已经失去相应知识。

因此，分析能力退化时，需要区分真实能力损失、输出策略变化以及评价方式造成的差异。

能力退化的主要原因 灾难性遗忘与能力退化通常由多个因素共同作用。神经网络中的参数具有共享性。一个参数可能同时参与多个任务的特征表示和预测过程。当新任务优化需要改变这些参数时，就可能对旧任务形成干扰。新旧任务的数据分布可能存在明显差异。例如，语言模型预训练时接触大量通用文本，而多模态微调数据中的回答往往较短、格式固定，并且大量样本集中于“识别对象—回答问题”的模式。长期在这类数据上训练，可能使模型逐渐偏向新的输出分布。

训练过程中的学习率、训练轮数、可训练参数范围等超参数也会影响参数偏移程度。过大的更新步长或者过长的单一任务训练，可能增加旧能力受到破坏的风险。但这些因素之间不存在完全确定的单调关系。例如，降低学习率通常可以减少单次参数更新幅度，却不能保证训练足够长后仍然不会出现遗忘。此外，如果偏好优化奖励过度强调回答形式，而没有充分约束事实正确性和任务表现，也可能使模型优化方向偏离原有目标。

因此，能力退化的原因应当通过具体实验分析，而不能简单归结为“训练太久”或“参数更新太多”。

缓解灾难性遗忘的主要方法 针对灾难性遗忘，已有研究提出了多种方法，其基本思想是使模型学习新能力时尽可能保留已有能力。经验回放（Experience Replay）。经验回放通过在新任务训练过程中混入部分旧任务样本，使模型持续接触原有数据分布。例

如，在多模态指令微调中，可以同时采样纯文本推理、语言指令和图像问答数据。这种方法实现较为直接，但需要保存或重新获取旧任务数据。

如果旧数据规模庞大，或者受到隐私、版权和使用权限限制，经验回放的实施成本可能较高。知识蒸馏（Knowledge Distillation）。可以将后训练前的模型作为教师模型，通过输出分布或中间表示约束学生模型。例如，对于选定的旧任务输入 x，定义：

$$\mathcal{L}_{\mathrm{KD}} = D_{\mathrm{KL}}\left(P_{\mathrm{teacher}}(\cdot \mid x) \,\|\, P_{\mathrm{student}}(\cdot \mid x)\right) \tag{11.44}$$

其中，D_{KL} 表示 KL 散度。该损失鼓励学生模型在指定输入上的预测分布与教师模型保持接近，从而减少新任务训练对旧行为的改变。不过，蒸馏需要额外的教师模型推理成本，而且教师模型本身也可能存在错误。如果蒸馏约束过强，模型又可能难以充分学习新任务。参数约束与正则化。一类持续学习方法通过限制重要参数的变化，减少旧任务能力受到的干扰。

例如，EWC 根据旧任务中参数的重要程度，对某些参数施加更强约束。其基本思想可以概括为：

$$\mathcal{L} = \mathcal{L}_{\mathrm{new}} + \frac{\lambda}{2} \sum_i F_i(\theta_i - \theta_i^*)^2 \tag{11.45}$$

其中：

- $\mathcal{L}_{\mathrm{new}}$ 为新任务损失；
- θ_i^* 为旧任务训练后第 i 个参数的参考值；
- F_i 表示该参数的重要性估计；
- λ 控制旧知识保持与新任务学习之间的权衡。

这里的 F_i 通常可以通过 Fisher 信息相关的近似方法估计。

EWC 是持续学习中的代表性方法，并不意味着它在所有多模态大模型后训练任务中都具有最优效果。参数隔离与模块化适配。参数隔离通过限制不同任务共享参数的更新范围，减少新旧任务之间的直接干扰。例如，可以冻结基础语言模型，为不同任务分别训练 LoRA 或 Adapter 模块。在这种情况下，原始基础模型参数能够得到保留，不同任务也可以分别加载对应的适配模块。

但如果使用同一个共享适配器连续训练多个任务，仍然可能发生任务间遗忘。因此，参数高效微调与抗遗忘并不是同一概念。PEFT 主要限制训练参数数量，而是否能够避免能力退化，还取决于参数共享方式、数据组织和训练协议。分阶段训练与持续评价。可以将后训练划分为多模态指令学习、任务专项优化和偏好对齐等阶段，并在每一阶段结束后进行能力检查。

如果某个阶段显著提升了目标任务，却导致多个重要旧任务性能下降，就需要进一步检查该阶段的数据和训练设置。这种方法不能直接消除灾难性遗忘，但可以帮助研究者及时发现问题并定位退化来源。

如何开展能力保持实验 为了验证某种后训练方法是否造成能力退化，可以建立固定的能力评价集合。例如，将评价任务划分为：

- 纯文本知识与推理任务；
- 基础视觉感知任务；
- 复杂视觉推理任务；
- 跨模态指令遵循任务；
- 目标领域专项任务。

在训练开始前，首先保存基础模型的评价结果。

随后，分别在多模态指令微调、参数高效适配和偏好优化等阶段结束后，重新使用相同协议进行评价。通过比较不同阶段的性能变化，可以建立模型的能力演化曲线。如果模型在目标任务上得到改善，同时多数已有任务保持稳定，可以认为当前训练方案具有一定的能力保持效果。如果模型在特定旧任务上持续下降，则需要进一步开展数据比例、模块冻结、训练轮数和正则化约束等方面的对照实验。对于多模态模型，尤其应当关注一个问题：

新能力的收益是否足以解释或接受原有能力的损失。这一判断需要结合具体应用场景。例如，专门用于工业视觉检测的模型可能允许牺牲部分通用语言能力，以换取特定场景下的识别性能。但对于面向广泛用户的通用多模态助手，语言理解、基础视觉能力和跨任务泛化通常都具有重要价值，不宜只根据单一任务成绩作出判断。

11.6.3 过拟合与数据分布偏差

多模态后训练中的过拟合问题 过拟合（Overfitting）是指模型过度适应训练数据中的具体样本、模式或统计规律，从而在未见过的数据上表现不佳。在多模态后训练中，过拟合不仅表现为训练损失下降而验证损失上升，还可能表现为模型学会了某些特定任务的回答模板，却没有形成稳定的跨模态理解能力。例如，假设某个视觉问答训练数据集包含大量类似问题：“图片中有什么动物?”对应回答通常为：“图片中有一只狗。”

如果训练数据的场景和回答形式较为单一，模型可能逐渐形成固定的问答模式。当面对新的动物类别、复杂背景或者不同问题表达时，模型未必能够准确理解视觉内容。另外，对于使用大规模合成指令数据进行训练的模型，还可能出现对特定生成风格的依赖。例如，如果训练答案普遍采用相同的描述顺序和语言结构，模型可能更擅长模仿这种回答形式，而不是提高对视觉证据的提取能力。因此，训练数据上的表现改善需要进一步区分：

一类是模型真正学习到了可以迁移的能力；另一类则是模型主要记住了训练数据中的特定规律。两者在训练集上可能具有相似表现，但在新场景和新任务上的泛化能力可能明显不同。

数据分布偏差的主要来源 数据分布偏差（Data Distribution Bias）是指训练数据在内容、来源、任务形式或标签分布等方面存在系统性倾向。在多模态后训练中，这类偏

差可能来自多个方面。视觉场景分布不均衡。图像数据可能主要来自互联网照片，包含大量日常物体、人物和自然场景，而对工业环境、特殊天气、医学影像等数据覆盖不足。模型在常见场景中表现良好，并不意味着它能够适应所有视觉条件。任务类型分布不均衡。

如果训练数据主要由图像描述和简单视觉问答构成，模型可能擅长对象识别，却缺乏复杂空间关系理解、细粒度比较和多步视觉推理能力。类似地，文档理解、图表分析和视频时序理解需要不同类型的监督数据。单纯增加某一种任务的数据量，并不能保证其他任务同步改善。语言和表达形式存在偏差。多模态指令数据可能主要采用某种语言或固定表达方式。例如，模型可能更熟悉英语视觉问答，在面对中文指令、混合语言文本或不同领域的专业术语时出现性能下降。

这种情况可能同时涉及语言覆盖不足、视觉文字识别能力差异以及跨模态对齐问题。标注质量与答案分布存在偏差。多模态训练数据的质量与答案生成方式密切相关。人工标注可能存在主观性与不一致性，自动生成的答案则可能包含视觉幻觉、遗漏信息或错误推理。如果这些错误被大量引入后训练数据，模型就可能学习到不可靠的对应关系。例如，一张包含两个物体的图像被错误标注为三个物体，可能向模型提供与真实视觉证据相矛盾的监督信号。

因此，数据规模与数据质量需要共同考虑。

虚假相关性与跨模态捷径学习 除了传统过拟合，多模态模型还可能利用训练数据中的虚假相关性（Spurious Correlation）完成任务。这种现象通常称为捷径学习（Shortcut Learning）。所谓捷径，是指模型使用某种容易获得、但不能可靠代表目标概念的统计线索进行预测。例如，在某个动物识别数据集中，狼的图像经常包含雪地背景，而狗的图像更多出现在草地或室内。模型可能部分依赖背景区分两类动物，而不是充分关注动物自身的外观特征。

当狼出现在草地上，或者狗出现在雪地中时，这种捷径就可能失效。对于视觉语言模型，捷径学习还可能表现为利用问题文本、选项分布和答案出现频率完成视觉问答。例如，在多项选择题中，如果某个选项的语言表达更加常见，模型可能倾向于选择它，即使图像证据并不支持该答案。这类问题尤其值得关注，因为模型最终输出可能形式正确、语言流畅，却缺少真实的视觉依据。

为了研究模型是否依赖虚假相关性，可以使用反事实测试（Counterfactual Testing）。反事实测试通常保持任务的其他条件尽可能不变，只改变某个关键因素。例如，针对颜色识别问题，可以保留物体类别、位置和背景，仅改变目标物体的颜色。如果模型仍然输出原来的颜色，就说明它可能没有充分利用变化后的视觉属性。类似方法还可以用于对象数量、空间关系、图文对应和视频事件顺序等任务。

反事实样本应尽量保持合理的视觉质量，并避免引入额外的分布变化。否则，模型性能下降可能同时受到图像编辑伪影等因素影响。

视觉幻觉与数据偏差之间的关系 视觉幻觉（Visual Hallucination）通常指模型生成的内容与实际视觉输入不一致。例如，模型可能描述图像中不存在的物体，或者错误地判断物体属性与空间关系。视觉幻觉与语言先验、视觉信息利用不足、训练数据偏差

等问题存在联系。假设训练数据中“餐桌”与“盘子”“杯子”等对象经常共同出现。当模型看到一张只有桌子的图像时，可能受到对象共现规律的影响，错误地生成“桌子上放着盘子和杯子”。

这种现象说明模型可能将常见场景知识当作当前图像中的事实。视觉幻觉研究中的 POPE 方法通过构建对象存在性问题，对模型是否错误报告图像中不存在的对象进行评价。例如，针对一张不包含自行车的图像，可以询问：“图中是否有自行车？”如果模型回答“有”，就可能构成对象存在性幻觉。不过，视觉幻觉并不是单一的数据偏差问题。它也可能来自视觉特征表达不足、跨模态连接不充分、推理过程错误或生成阶段的行为偏差。

因此，仅依靠增加训练数据或修改提示词，未必能够全面解决视觉幻觉。更合理的研究方式是通过不同类型的测试，区分视觉感知错误、语言先验干扰和生成策略问题。

训练数据与评价数据的泄漏问题 数据泄漏（Data Leakage）是指模型训练过程直接或间接接触了原本应当用于独立评价的信息。在大规模多模态训练中，这一问题可能更加隐蔽。例如，训练数据可能包含与评价集相同的图像，或者包含经过裁剪、缩放、重新压缩后的重复图像。即使没有完全相同的文件，训练数据也可能包含评价题目的文本描述、标准答案或高度相似的问答内容。在这种情况下，模型取得较高的评价成绩，并不能充分证明其具备相应的泛化能力。

数据泄漏与模型依赖语言先验是两个不同的问题。即使某个模型能够在没有图像的情况下回答视觉问题，也不能立即断定训练中一定存在泄漏。它可能只是利用了常识、题目线索或答案分布。相反，如果模型能够准确回答某些原本必须观察图像、且缺乏一般推断线索的具体题目，则需要进一步检查是否存在样本记忆和数据污染。

MMStar 相关研究专门关注视觉依赖不足与潜在数据泄漏对多模态评价的影响，强调不能把所有视觉问答成绩都直接解释为真实的多模态能力。对于研究者而言，数据泄漏问题不仅影响结果可信度，也可能导致不同模型之间的比较失去意义。

提高多模态训练泛化能力的方法 为了缓解过拟合和数据分布偏差，可以从数据组织、训练设计与评价协议三个方面进行改进。应提高训练数据的任务与场景多样性。例如，视觉问答训练可以同时覆盖对象识别、属性判断、数量统计、空间关系、OCR、图表理解和复杂推理，而不是过度依赖单一题型。需要重视长尾场景和困难样本。对于模型容易出错的视觉关系、特殊对象和不常见场景，可以进行有针对性的数据补充。

但困难样本的选择应当依据可验证的错误分析，而不是简单增加模型回答较长或语言更加复杂的样本。可以采用合理的数据去重和分组划分策略。例如，在视频任务中，如果同一段视频的相邻帧同时进入训练集和测试集，可能导致评价结果过于乐观。因此，对于具有时间或场景连续性的数据，通常需要按照视频、场景或数据来源进行合理划分。对于图像数据，也应尽量检查重复图像、近似重复图像及其不同分辨率版本。

评价集应当覆盖不同的数据分布，并尽可能包含独立于训练数据来源的测试样本。可以分别考察同分布测试（In-Distribution，ID）和分布外测试（Out-of-Distribution，OOD）表现。同分布测试用于衡量模型在预期数据条件下的任务能力，分布外测试则帮助研究者了解模型面对新环境、新对象或新组合时的稳健程度。

OOD 并不是一个完全统一的评价条件。背景变化、成像质量变化、任务形式变化和语义组合变化可能分别考察不同的泛化能力。因此，研究中应明确说明分布变化的类型，而不能将所有分布外测试结果混合解释。

习题与讨论

1. 构造一条含图像与两轮对话的训练序列，标明哪些位置计入 SFT 损失，并说明历史助手回答的处理取决于什么。
2. 推导 LoRA 参数量与全矩阵参数量的比值；列出一个只减少可训练参数却未显著降低推理成本的原因。
3. 写出 DPO 中策略与参考策略对两个回答的对数概率差。它与只提高优选回答概率的 SFT 有何不同？
4. 后训练提升某个 VQA 分数却降低纯文本能力。设计区分遗忘、训练预算差异和数据分布偏差的消融。

文献导读 LoRA 与 QLoRA 分别核对更新参数和量化基座；DPO 应连同参考策略与偏好模型的推导假设阅读。[24, 26, 28]

第四部分

能力与验证

第十二章　感知、文档理解与细粒度定位

感知任务的差异首先体现在输出空间：一句描述、一个类别、文本转录、边界框、区域掩码或时间片段，具有不同监督与错误代价。将它们统称为“理解能力”，会使模型选择和评价失去对象。

学习目标　按输出与监督粒度分解任务；区分识别、关联与定位；在文档和音视频中追踪证据；用结构化结果检验视觉信息的利用。

12.1　任务输出与监督粒度

同一图像问答可以只要求答案，也可以要求答案附对象或区域依据。后一种输出有助于定位错误来源，但模型给出的依据仍需独立检查。自然语言解释的流畅程度，不能替代区域正确率或事实核对。

12.1.1　视觉问答的任务形式

视觉问答的基本概念　视觉问答（Visual Question Answering，VQA）是指模型根据输入图像和自然语言问题，生成与视觉内容相关的答案。与图像描述不同，视觉问答并不要求模型完整描述图像，而是要求模型根据问题选择相关视觉信息。例如，对于同一张道路图像，可以提出：“图中有几辆汽车？”“汽车是什么颜色？”“行人位于汽车的哪一侧？”“交通信号灯当前是什么颜色？”

这些问题关注的视觉信息不同，因此需要模型对输入图像进行有针对性的理解。视觉问答通常可以形式化为：

$$\hat{y} = \arg\max_{y} P_\theta(y \mid I, q) \tag{12.1}$$

其中，I 表示图像，q 表示问题，y 表示候选答案，θ 表示模型参数。这一表达强调，答案应当同时受到图像和问题的约束。对于生成式模型，上式可以理解为选择条件概率较高的完整答案序列；实际系统也可能采用采样等其他解码策略。从任务本质上看，视觉问答包含三个主要环节。模型需要理解自然语言问题，识别问题所要求的信息类型。模型需要从图像中获取与问题相关的视觉证据。

模型需要根据视觉证据生成答案，必要时结合逻辑推理或外部知识。因此，VQA 不只是一个语言生成任务，而是对视觉感知、语言理解和跨模态信息整合能力的联合检验。Antol 等人在 2015 年提出的 VQA 工作，系统建立了以图像和自然语言问题为输入、以自然语言答案为输出的经典任务与评价框架。

视觉问答的常见任务类型 根据问题内容和所需能力，视觉问答可以分为多种形式。对象识别类问题。例如：“图片中主要的交通工具是什么？”模型需要识别图像中的目标类别，并将其转换为语言答案。这类任务通常与图像分类和目标检测关系密切。属性判断类问题。例如：“左边汽车的颜色是什么？”“这个人的上衣是什么颜色？”模型不仅需要识别对象，还需要将属性与正确的对象实例关联起来。当图像中存在多个相似对象时，这类问题可能比简单对象识别更加困难。

数量统计类问题。例如：“图中有几个人？”“桌面上有多少个杯子？”数量统计要求模型区分不同对象实例，并避免重复计数或遗漏目标。对于密集场景、小目标和遮挡条件，数量统计通常具有较大挑战。空间关系类问题。例如：“红色汽车位于白色汽车的哪一侧？”“桌子上的杯子是否靠近电脑？”这类问题要求模型理解对象之间的相对位置。

“左侧”“前方”“后方”等关系可能依赖观察视角。在视觉问答中，应当明确是图像平面中的关系，还是具有三维场景含义的空间关系。动作与交互理解类问题。例如：“图中的人正在做什么？”“人物与自行车之间是什么关系？”模型需要识别人物姿态、对象状态及其相互作用。对于单张静态图像，一些动作可以通过姿态和上下文进行判断，但真实运动方向、动作持续时间和事件先后顺序通常无法仅凭单帧可靠确定。常识与知识推理类问题。例如：

“图中的人物为什么可能需要佩戴头盔？”如果图像展示骑行场景，模型可以结合视觉信息与交通安全知识进行解释。这类任务不仅需要感知图像，还需要调用与图像内容相关的外部知识或常识。OK-VQA 是知识型视觉问答的代表性数据集，其问题设计强调单靠图像内容通常不足以得到答案，还需要额外知识。此外，还有需要读取图像中自然场景文字的 TextVQA，以及需要进行多步关系推理的 GQA 等任务。

这些任务共同说明，VQA 已经从基础对象识别扩展到文本理解、知识推理和组合式视觉分析等多个方向。

封闭式问答与开放式问答 从答案空间和交互形式来看，视觉问答可以分为封闭式问答与开放式问答。

封闭式问答（Closed-ended VQA）通常具有预先规定的答案集合。

例如，判断问题：“图中是否有自行车？”答案可以限制为“是”或“否”。多项选择形式则可能提供：A. 汽车 B. 自行车 C. 摩托车 D. 公交车模型只需要从候选答案中选择一个。这种任务形式便于自动评价，但模型也可能利用答案选项中的语言偏差进行推断。

开放式问答（Open-ended VQA）通常不要求模型从预先给定的有限候选集合中选择答案，而是直接生成自然语言回答。

例如：“请描述图中人物与周围环境之间的关系。”模型可以生成一句话或一段文字。开放式任务更加接近实际人机交互，但评价难度也更高，因为多个不同措辞的答案可能表达相同含义。开放式问题并不意味着答案一定很长。例如，“汽车是什么颜色？”可以是开放式问答，但合理答案可能只有“红色”两个字。因此，应当区分问题是否开放、答案集合是否受限以及输出文本长度这几个不同概念。

从传统 VQA 到多模态大模型问答 早期 VQA 方法通常将问题回答视为分类任务。模型分别提取图像特征和问题文本特征，再将它们融合，预测答案集合中的某个类别。这一方法对于短答案和高频答案较为有效，但难以充分支持自由形式的语言输出。随着视觉语言预训练的发展，模型开始利用跨模态注意力、联合编码等机制学习图像与文本之间的联系。在多模态大语言模型中，VQA 可以进一步统一为条件文本生成任务。用户输入图像和自然语言问题，模型则根据联合上下文生成答案。

这一变化带来了更灵活的交互能力。例如，用户不再只能询问："图中有几个人?"还可以进一步提出："请分别描述这些人的位置和活动。""根据图像信息，解释他们之间可能存在什么关系。"模型能够在不同问题之间切换，并根据需要生成不同粒度的回答。不过，生成式问答也带来了新的问题。由于语言模型具有较强的文本预测能力，它可能在视觉证据不足时仍然生成看似合理的答案。因此，VQA 模型需要避免将一般常识直接当作当前图像中的事实。

尤其对于对象数量、空间关系和细粒度属性等问题，最终答案应当由实际视觉证据支持。

视觉问答中的语言偏差与评价问题 视觉问答模型可能利用训练数据中的语言偏差，而没有充分理解输入图像。例如，问题"香蕉通常是什么颜色?"即使不提供图像，也可能通过常识回答"黄色"。但当实际图像中出现青绿色香蕉时，仅依赖语言先验就可能产生错误。VQA v2.0 针对这一问题引入互补图像设计，使相同问题在不同图像下具有不同答案，从而鼓励模型更加关注视觉证据。GQA 则利用对象、属性和关系构建组合式问答，进一步评价模型的结构化视觉推理能力。

在评价方式上，传统 VQA 数据集通常采用答案准确率或基于多个人工答案一致性的评分方法。对于自由生成式模型，需要考虑答案归一化和语义等价问题。例如："two""2""two people"在某些问题中可能表达相同事实，但直接字符串匹配可能给出不同结果。因此，评价协议需要明确答案规范化规则。对于复杂开放式回答，还可以结合语义评价、视觉事实检查和人工核验。

但自动评价器也可能受到语言表达、提示词和模型偏差影响，不能直接视为完全客观的判断标准。一个重要的验证方式是比较完整图像输入与无图像输入下的模型表现。如果模型在不提供图像时仍然能够取得较高分数，研究者就需要进一步检查任务是否具有较强的语言先验。不过，无图像成绩较高并不直接等于模型没有视觉能力，而是说明当前评价结果可能同时受到语言知识和视觉信息的影响。

12.1.2 物体、属性与关系理解

从物体识别到细粒度语义理解 物体识别（Object Recognition）是场景理解的重要基础。传统图像分类通常预测整张图像的类别，而目标检测进一步识别图像中的多个目标，并输出目标类别及其空间位置。对于多模态模型，物体识别的结果还需要与自然语言表达建立联系。例如，模型不仅需要识别"汽车"，还应能够理解"左边的红色汽车""靠近路口的车辆"等更加具体的语言描述。这意味着模型需要综合利用对象类别、属性、位置和对象之间的关系。假设图像中存在两辆汽车：

一辆红色汽车位于左侧，一辆白色汽车位于右侧。如果用户询问：“左侧的汽车是什么颜色？”模型必须首先根据空间关系确定目标对象，再判断它的颜色。如果模型只识别出“图中有红色和白色两辆汽车”，但无法将颜色与对应对象关联，就可能给出错误答案。因此，细粒度视觉理解不仅要求模型识别特征，还需要完成特征与具体对象实例之间的绑定。

属性理解的主要内容 属性理解（Attribute Understanding）主要关注对象所具有的具体特征。常见属性包括颜色、形状、大小、材质、状态和外观特征等。例如：“一只棕色的狗。”“一张圆形木桌。”“一扇打开的窗户。”这些描述分别涉及颜色、形状、材质和状态属性。在更复杂的场景中，模型还需要理解属性之间的组合关系。例如：“穿着蓝色上衣、背着黑色背包的人。”这一描述包含两个对象，即人物和背包，以及不同对象对应的属性。

如果图像中存在多个穿着相似的人物，模型还需要进一步利用其他视觉线索进行区分。属性理解的难点通常不在于识别单个属性名称，而在于将属性准确地关联到对应对象。对于多模态大语言模型而言，这种对象—属性绑定尤其重要。因为模型可能正确识别图像中存在蓝色衣服和黑色背包，却错误地将它们分配给不同人物。此外，一些属性并不具有完全客观的判断标准。例如，“高大”“年轻”“漂亮”等描述可能受到视角、上下文和主观判断影响。

因此，视觉属性任务需要区分可直接观察的客观属性，以及依赖推断或主观解释的属性。对于证据不足的属性，模型不应将不确定推断表达为确定事实。

对象关系理解 对象关系理解（Object Relation Understanding）关注图像中不同对象之间的联系。常见关系可以分为几类。空间关系。例如：“杯子位于电脑左侧。”“书放在桌子上。”“汽车停在建筑物前方。”第二，动作与交互关系。例如：“人物正在骑自行车。”“小孩正在牵着狗。”“有人拿着雨伞。”第三，组成与归属关系。例如：“车轮是汽车的一部分。”“人物穿着外套。”第四，比较关系。例如：“左侧建筑比右侧建筑更高。”

“红色盒子比蓝色盒子更大。”关系理解不同于简单对象识别，因为它涉及多个实体之间的联合判断。特别是部分关系具有方向性。例如：“人站在汽车左侧”与“汽车站在人左侧”并不等价。同样，“人追赶狗”与“狗追赶人”表达了完全不同的动作关系。因此，关系理解需要同时明确关系主体、关系类型和关系客体。一种常用的表示方式是三元组：(s,p,o) 其中：

- s 表示关系主体（Subject）；
- p 表示关系谓词（Predicate）；
- o 表示关系客体（Object）。

例如：

$$(\text{人}, \text{骑}, \text{自行车}) \tag{12.2}$$

这一表示可以将自然语言中的部分视觉关系转换为更加明确的结构。不过，关系三元组并不自动包含完整的空间位置、时间信息或不确定性。对于复杂场景，还需要进一步扩展其表示形式。

场景图及其在视觉理解中的作用 场景图（Scene Graph）是一种用于表示图像中对象、属性及其关系的结构化形式。通常可以将场景图表示为：$G = (V, E)$，其中：

- V 表示对象节点集合；
- E 表示对象之间的关系边集合。

对象属性可以作为节点特征或额外属性信息保存。

例如，对于一张包含人物、自行车和道路的图像，可以构建：对象节点：

- 人物；
- 自行车；
- 道路。

对象属性：

- 人物：穿黑色外套；
- 自行车：红色；
- 道路：城市道路。

关系：

- 人物—骑行—自行车；
- 自行车—位于—道路。

通过这种表示，模型不仅能够知道图像中有哪些对象，还可以明确对象之间的联系。

Visual Genome 是场景图与细粒度视觉语义研究中的代表性数据集，包含对象、属性、关系、区域描述和视觉问答等多种标注信息。其研究思路强调，真实的图像理解不能仅依靠单独的对象类别识别，而需要理解对象之间的复杂交互。场景图能够支持多种下游任务，例如视觉关系检测、图像检索、图像描述和组合式视觉问答。例如，用户提出：“骑着自行车的人穿着什么颜色的衣服？”

模型可以先通过“骑自行车”关系确定人物节点，再读取该人物对应的衣服属性。这一过程体现了视觉关系信息在复杂问答中的作用。场景图是一种显式表示形式，并不意味着所有现代多模态模型内部都会实际构建场景图。许多端到端视觉语言模型使用隐式特征和注意力机制学习对象关系，并不直接输出图结构。显式场景图的优势在于便于组织、检索和检查对象关系，但其质量也受到对象识别和关系提取误差的影响。

复杂关系与组合式视觉推理 当视觉问题涉及多个对象和连续的关系判断时，模型需要具备组合式视觉推理（Compositional Visual Reasoning）能力。例如：“站在红色汽车旁边、穿着蓝色外套的人手中拿着什么？”这个问题需要模型完成多个步骤。首先识别红色汽车。然后寻找位于汽车旁边的人物。接着根据蓝色外套进一步确定目标人物。最

后识别该人物手中的物体。这类问题通常难以通过单一对象分类结果直接回答。

GQA 数据集正是面向这种真实场景中的组合式视觉问答提出的代表性研究。通过基于场景图构建问题，GQA 可以评价对象识别、属性判断、关系追踪和多步组合推理等能力。对于现代多模态模型，组合推理可以通过隐式表示学习或显式中间步骤实现。但模型能够生成看似合理的推理过程，并不意味着它在内部一定按照这些步骤进行了计算。例如，模型可能输出完整的文字解释，却在最终答案中出现对象指代错误。

因此，评价组合式视觉推理时，不应只检查解释文本是否连贯，还应验证最终答案与实际视觉证据是否一致。

细粒度理解的主要挑战 尽管多模态模型在图像理解方面取得了明显进展，但细粒度对象与关系理解仍然面临多种挑战。小目标和局部细节可能在视觉编码过程中被压缩或丢失。例如，一张高分辨率道路图像中，远处交通标志上的文字可能只占据很小的区域。如果输入图像被过度缩放，模型就可能无法读取相关信息。遮挡和相似对象可能增加实例区分难度。例如，图像中有多名穿着相似的人物，模型可能难以判断哪个人物对应语言描述中的目标。空间关系理解可能受到视角和图像布局影响。

对于“左侧”“右侧”等二维关系，可以根据图像坐标进行判断；但对于“前方”“后方”“靠近”等关系，可能需要结合场景几何信息、深度线索和上下文。此外，对象数量、关系方向、属性绑定和否定关系也是容易产生错误的方面。例如：“图中没有戴头盔的人在哪里？”这一问题同时涉及对象识别、否定条件和目标定位。如果模型忽略否定条件，就可能错误地选择戴头盔的人。

因此，细粒度视觉理解需要超越一般性的场景描述，进一步建立语言表达与具体视觉对象、属性和关系之间的可靠联系。这也自然引出了本章后续将介绍的视觉指代、目标定位和区域级跨模态理解任务。

12.1.3 开放式问答与结构化输出

开放式视觉问答的特点 随着多模态大语言模型的发展，视觉问答逐渐从固定问题和短答案预测，扩展为更加灵活的开放式交互。例如，用户可以针对同一张图像提出：“描述一下这张图像。”“这张图像中最值得关注的内容是什么？”“解释一下图中人物的行为。”“根据图像，列出可能存在的安全隐患。”这些问题不一定存在唯一标准答案。模型需要根据任务要求选择相关视觉信息，并生成符合用户需求的回答。开放式视觉问答通常具有几个特点。

输出长度不固定，可以是一句话，也可以是一段较长的解释。回答内容可能同时包含视觉事实和推理信息。不同用户可能希望获取不同粒度的信息。评价结果不仅取决于答案是否正确，还与信息完整性、解释清晰度和指令遵循程度有关。例如，对于一张显示湿滑道路和车辆的图像，用户询问：“这张图像可能反映什么交通风险？”模型可以根据可见的积水、道路条件和车辆状态，讨论可能存在的风险。

但如果图像没有提供车辆速度、轮胎状态和道路摩擦系数等信息，模型就不应声称已经确认某辆汽车必然会发生侧滑。这说明开放式视觉问答不仅需要解释能力，也

需要合理处理视觉证据与推断之间的边界。

事实描述、推理与不确定性的区分 在开放式视觉问答中，一个重要问题是区分图像中可以直接观察到的内容，以及模型根据视觉内容推断得到的信息。例如，一张图像中有人站在公交站旁边。可以直接观察的事实可能是：“一个人站在公交站附近。”但如果模型回答：“这个人正在等待上班的公交车。”就已经引入了关于人物目的和活动的推断。这一推断可能合理，但未必能够仅凭图像得到确定结论。因此，较为可靠的开放式视觉回答可以区分三个层次。直接视觉事实。

描述图像中能够明确观察到的对象、属性、动作线索和空间关系。基于证据的合理推断。利用视觉信息和常识，对可能的场景含义进行解释。无法由当前图像确定的信息。对于需要额外证据才能判断的问题，应明确表达不确定性。例如：“图中一名人物站在公交站旁边，可能正在等车，但仅凭图像无法确定其具体目的地。”这种回答方式能够减少模型将合理猜测误写为视觉事实的情况。不过，不确定性表达也需要适度。

如果图像中对象颜色清晰可见,模型不应在没有必要的情况下反复使用“可能”“也许”等词语。因此，合理的目标不是让模型始终保守，而是使回答的确定程度与视觉证据的充分程度相匹配。

为什么需要结构化输出 自然语言回答适合人机交流，但在实际应用系统中，模型输出往往还需要被其他程序进一步处理。例如，在自动驾驶场景中，系统可能需要从视觉输入中提取车辆、行人、道路标志和交通关系。如果模型只输出：“图中有一辆汽车和两名行人，其中一名行人靠近斑马线。”这段文字虽然易于理解，但下游程序可能难以直接提取标准化对象类别、目标编号和空间关系。因此，多模态系统有时需要将视觉理解结果转换为结构化输出（Structured Output）。

结构化输出是指按照预先规定的数据结构表达模型结果，例如 JSON、表格、对象列表、关系三元组或场景图。其核心目的，是使模型输出更容易被程序读取、验证和进一步利用。例如，对于一张包含人物和自行车的图像，可以将理解结果组织为：场景类型：“城市道路”对象列表：

- 对象 1：人物
- 对象 2：自行车

属性信息：

- 对象 1：黑色外套
- 对象 2：红色车身

关系信息：

- 对象 1—骑行—对象 2

自然语言描述：

“一名穿黑色外套的人正在骑一辆红色自行车。”这一结构保留了自然语言描述中的主要信息，同时明确区分对象、属性和关系。结构化输出并不等于视觉理解一定正

确。如果模型将自行车颜色识别错误，那么即使输出格式完全符合要求，结构化结果仍然包含事实错误。因此，输出格式正确性和视觉语义正确性需要分别评价。

JSON 与对象—属性—关系表示 JSON 是一种常见的数据交换格式，在多模态理解系统中可以用于组织对象列表、属性信息和关系结果。例如，假设输入图像展示一名穿蓝色外套的人牵着一只黑狗，可以设计如下结构：

```
{
  "scene": " 公园",
  "objects": [
    {
      "id": "obj_1",
      "category": "person",
      "attributes": ["blue_jacket"]
    },
    {
      "id": "obj_2",
      "category": "dog",
      "attributes": ["black"]
    }
  ],
  "relations": [
    {
      "subject": "obj_1",
      "predicate": "walking_with",
      "object": "obj_2"
    }
  ],
  "description": " 一名穿蓝色外套的人与一只黑狗同行。"
}
```

该示例展示了一种简单的结构化组织方式，并不对应某个实际模型的预测结果。

其中，objects 表示对象集合，attributes 表示对象属性，relations 表示对象之间的关系。实际任务中，可以根据需求增加空间位置、置信信息或其他字段。例如，对于需要目标定位的任务，可以进一步为每个对象增加边界框（Bounding Box）。但此时必须明确坐标采用绝对像素还是归一化形式，以及边界框使用左上角与右下角坐标，还是中心坐标与宽高表示。

如果模型没有接受相应的定位训练，或者没有可靠的空间定位能力，就不能仅凭其能够输出坐标字段而认为它具备准确的目标定位能力。因此，结构化输出的字段设计需要与模型实际能力和下游任务需求相匹配。

结构化输出的生成与约束 对于生成式多模态模型，结构化输出通常可以通过指令约束、专门训练或受约束解码等方式实现。一种简单方法是通过提示词明确输出要求。例如：“请识别图像中的主要对象，分别给出类别、颜色和对象之间的关系，并按照指定

JSON 格式输出。”模型可以根据指令生成相应内容。但是，单纯依赖提示词并不保证输出始终符合要求。模型可能遗漏字段、生成额外解释，或者使用不符合约定的数据类型。

因此，对于需要可靠程序解析的应用，可以使用结构化输出约束或受约束解码，使输出满足指定语法或数据模式。例如，可以规定：

- scene 必须是字符串；
- objects 必须是数组；
- 每个对象必须具有唯一标识和类别字段；
- relations 中引用的对象编号必须来自对象列表；
- 不允许使用未定义的关系类型。

其中，部分约束可以通过 JSON Schema 等形式描述，但诸如对象编号引用是否有效、关系是否符合视觉事实等，仍然可能需要额外检查。

因此，结构化输出通常需要考虑不同层次的正确性。语法正确性：输出是否可以被正常解析。模式正确性：字段名称、类型和层级是否符合要求。引用一致性：对象编号、关系引用等是否存在逻辑矛盾。视觉事实正确性：输出内容是否与实际图像一致。任务适用性：输出是否包含下游任务所需的信息。这些方面不能简单相互替代。例如，一个 JSON 结果可能具有完全正确的字段结构，却把图像中的三辆汽车识别为两辆。

相反，模型也可能正确理解图像，却因为输出中存在语法错误而无法被程序解析。因此，多模态结构化输出需要将语言生成、格式约束与视觉事实检查结合起来。

开放式理解与细粒度定位的联系 开放式视觉问答和结构化输出进一步推动了多模态模型从“描述图像内容”向“明确指出视觉证据”发展。例如，用户提出：“图中穿红色衣服的人在哪里?”模型不仅需要识别人物和衣服颜色，还需要将自然语言中的描述对应到图像中的具体目标区域。如果模型只回答“图中有一个穿红色衣服的人”，就没有完整完成定位要求。类似地，当用户询问：“左侧拿着雨伞的人正在做什么?”模型需要识别符合语言条件的目标人物，并进一步分析其动作。

这些任务要求模型建立更加精确的跨模态对应关系。与一般图像描述相比，细粒度定位通常需要输出明确的空间信息，例如边界框、分割掩码或其他区域表示。这种能力也是视觉指代定位（Referring Expression Comprehension）、短语定位（Phrase Grounding）和开放词汇目标定位等任务的重要基础。不同任务对空间精度的要求存在差异。

例如，回答“人物在图像左侧”可能只需要较粗略的空间理解；而输出精确分割掩码则需要像素级的空间预测能力。因此，能够进行自然语言空间描述的模型，并不一定具备精确的区域定位能力。这也说明，视觉语义理解与细粒度空间定位虽然密切相关，但仍然需要通过各自对应的评价指标进行检验。后续章节将进一步讨论如何将自然语言中的对象、属性和关系描述映射到具体图像区域，以及视觉语言模型如何支持更加细粒度的目标定位与区域理解。

12.2 文字、版面与图表

文档中的文字、版面、表格结构和图形编码共同决定答案。把页面先转成一串文本，可能丢失行列、阅读顺序或图例关系；直接读取图像则仍受分辨率和字符识别影响。比较两条路线时，应说明是否使用外部 OCR 以及它贡献了哪些信息。

12.2.1 文本识别与版面结构

光学字符识别的基本概念 光学字符识别（Optical Character Recognition，OCR）是指从图像中识别字符、单词或文本序列，并将其转换为可供计算机处理的文本信息。OCR 是文档理解的重要基础技术。例如，对于一张扫描得到的纸质合同，计算机最初接收到的是像素构成的图像，而不是可以直接检索和编辑的文字。OCR 系统需要从这些像素中识别字符，并重建相应文本。

在传统文档处理系统中，一个常见的 OCR 流程包括图像预处理、文本检测、文本识别和结果整理。图像预处理。文档图像可能存在倾斜、模糊、阴影、透视畸变和分辨率不足等问题。预处理可以根据图像质量进行旋转校正、去噪、对比度调整或几何矫正，以改善后续识别效果。不过，预处理并非越多越好。例如，过度锐化或二值化可能破坏细小笔画，使识别更加困难。文本检测（Text Detection）。

文本检测的目标是确定图像中哪些区域包含文字。检测结果通常采用矩形框、多边形或其他几何表示。例如，一张发票中可能包含标题、商家名称、商品信息和金额等多个文本区域，文本检测需要识别它们的位置。文本识别（Text Recognition）。文本识别进一步将检测区域中的视觉内容转换为字符序列。例如，将图像中的一段文字识别为：“订单总金额：128.50 元”。这一过程需要区分相似字符、数字和标点符号。

结果整理与结构恢复。OCR 输出的多个文本片段通常还需要按照阅读顺序组织，并与原始图像位置保持对应关系。对于简单的单栏文档，可以按照从上到下、从左到右的顺序排列。但对于多栏论文、表格、票据和复杂排版文档，仅依靠这一规则可能产生错误。因此，OCR 通常只是文档理解的基础步骤，而不等于完整的文档语义理解。

文本内容与空间位置的联合表示 在自然语言处理任务中，文本通常被表示为按照一定顺序排列的 token 序列。但在文档理解中，文本的位置同样有意义。例如，一张表单包含：“姓名：张三”“联系电话：138XXXXXXXX”如果将所有内容简单连接为一个字符串，模型虽然可能识别出相关文字，却未必能够准确判断字段之间的对应关系。因此，文档理解通常需要保留文本及其空间坐标。

假设 OCR 系统识别出第 i 个文本片段，可以表示为：$o_i = (t_i, b_i)$，其中：

- t_i 表示识别得到的文本内容；
- b_i 表示文本所在区域的空间坐标。

对于矩形框，可以进一步定义：

$$b_i = (x_{\min}, y_{\min}, x_{\max}, y_{\max}) \tag{12.3}$$

其中，四个变量分别表示文本区域左上角和右下角的坐标。实际系统可以采用绝对像素坐标，也可以将坐标归一化到固定范围。通过这种表示，模型能够同时获得文本内容和空间位置。例如，“总金额”与“128.50 元”虽然在文本序列中可能存在一定距离，但如果它们在文档中位于同一行，且空间距离较近，就可能具有字段与数值之间的关联。空间接近并不意味着两个文本片段一定存在语义关系。

例如，在双栏论文中，左栏末尾与右栏开头可能在空间上接近，却属于不同的阅读顺序。因此，文档理解还需要结合布局结构和上下文语义进行判断。

版面分析与阅读顺序恢复 版面分析（Document Layout Analysis）主要研究文档页面中不同内容区域的类型、位置及其组织关系。常见的版面元素包括标题、正文、页眉、页脚、表格、图片、图注、列表、公式和脚注等。例如，一篇学术论文可能具有双栏正文结构，同时包含跨栏标题、插入式图片和表格。如果直接按照图像坐标从上到下提取文字，就可能把不同栏的内容交错连接，破坏文章原有的语义顺序。因此，版面理解通常涉及两个相互关联的任务。

首先是版面元素识别，即确定页面中有哪些内容区域，以及各区域属于什么类型。其次是阅读顺序恢复（Reading Order Recovery），即判断不同文本块之间合理的阅读顺序。对于简单文档，阅读顺序可能具有较为明确的线性结构。对于复杂文档，则可能需要结合页面分栏、标题层级、脚注引用和图表插入位置等因素进行分析。

DocLayNet 是文档版面分析领域具有代表性的数据集，提供来自多种文档来源的人工标注页面，用于研究不同版面类型的检测与识别。这类研究表明，文档版面本身包含重要的结构信息，不能简单视为文字识别的附属结果。

从 OCR 到版面感知的文档模型 传统 OCR 系统通常将文字识别和后续语义理解分成两个相对独立的阶段。首先识别所有文字，然后将识别结果交给自然语言处理模型进行分类、抽取或问答。这种方法实现较为直接，但可能丢失部分布局和视觉信息。为了更好地利用文档结构，研究者提出了版面感知的预训练模型。LayoutLM 是这一方向的代表性工作。

它将文本内容与二维空间位置共同用于文档表征学习，使模型不仅能够理解词语之间的语义关系，还能够利用词语在页面中的位置关系。其基本思想可以概括为：$h_i = f(t_i, b_i, \text{context})$，其中，$h_i$ 表示文本单元的上下文表示，t_i 表示文本内容，b_i 表示空间位置，context 表示其他文本及相关信息。

这里的公式是对版面感知建模思想的概括，并不代表 LayoutLM 的完整实现细节。在后续研究中，LayoutLMv3 进一步使用文本与图像的联合预训练策略，引入统一的文本和图像掩码学习，以及词语与图像区域之间的对齐目标。这种方法使文档表示不再仅仅依赖 OCR 文本及坐标，还可以利用字体、背景、边框和其他视觉特征。例如，在一份表单中，“申请人姓名”可能位于固定的字段区域，而填写内容位于旁边的空白框中。

单纯的文本语义能够提供一定线索，空间位置和视觉结构则有助于判断字段之间的对应关系。因此，文档模型的发展逐渐体现出一种趋势：从独立的字符识别，走向文本、空间结构和视觉特征的联合表示。

复杂文档中的 OCR 识别难点 尽管 OCR 已经是一项较为成熟的技术，但复杂文档仍然存在大量识别困难。文档图像质量可能受到扫描设备、拍摄角度和压缩方式影响。例如，倾斜拍摄的票据可能产生透视变形，低分辨率图片中的小字体可能无法清晰辨认。不同语言和字体的识别难度存在差异。中文、英文、数字、数学符号和特殊字符可能同时出现在同一页面中，模型需要根据上下文和字形进行区分。表格与公式往往具有特殊的二维结构。例如，数学公式：

$$y = \frac{a+b}{c} \tag{12.4}$$

并不能仅通过将字符从左到右排列来恢复其完整含义。模型还需要理解分子、分母、上下标及其他符号之间的空间关系。此外，文档中的印章、手写文字、下划线和背景图案也可能干扰识别。对于这些情况，OCR 系统可能需要结合专门的公式识别、手写识别和版面恢复方法。因此，文档文字识别不仅是字符分类问题，还涉及复杂视觉结构向符号化表达的转换。

OCR 与版面理解的评价 OCR 系统通常采用字符错误率（Character Error Rate，CER）和词错误率（Word Error Rate，WER）等指标评价识别结果。其中，字符错误率可以表示为：

$$\mathrm{CER} = \frac{S+D+I}{N} \tag{12.5}$$

其中：

- S 表示字符替换错误数量；
- D 表示字符删除错误数量；
- I 表示字符插入错误数量；
- N 表示参考文本中的字符总数。

CER 越低，通常表示字符识别结果越接近参考文本。

对于版面检测，可以使用目标检测任务中的 Precision、Recall、F1 或 mAP 等指标。但这些指标评价的是不同层次的能力。字符识别准确，并不意味着阅读顺序一定正确。版面区域检测准确，也不意味着字段之间的语义关联一定正确。例如，一个模型可能准确识别发票中的全部数字，却将税额与总金额对应错误。因此，文档理解的最终评价还应当根据任务需要检查字段提取、结构恢复和语义理解结果。

12.2.2 表格、图表与图形信息提取

表格理解的基本任务 表格是文档中常见的结构化信息表达方式。与普通自然语言相比，表格通过行、列、单元格和表头之间的对应关系组织信息。例如，一张实验结果表可能包含不同算法在多个数据集上的性能指标。如果模型只识别表格中的文字和数值，却无法理解数值对应的算法名称和评价指标，那么这些信息仍然难以直接使用。因此，表格理解（Table Understanding）通常涉及多个任务。首先是表格检测，即确定文档中

哪些区域包含表格。

其次是表格结构识别，即恢复表格的行列结构、单元格边界和合并关系。然后是单元格内容提取，即识别各单元格包含的文字、数字和符号。最后是表格语义理解，即判断不同字段之间的对应关系，并利用这些信息完成检索、比较或推理。例如，对于一张包含“方法”“准确率”“推理时间”三列的表格，模型需要知道 92.5% 是某个方法的准确率，而不是其推理时间。这种理解依赖的不仅是文本内容，还包括表格的二维结构。

表格结构的恢复 表格结构恢复（Table Structure Recognition）是表格理解的重要环节。一个表格可以包含行、列、单元格和表头信息。对于简单表格，每个单元格可能对应一个固定的行列位置。但在复杂表格中，单元格可能跨越多行或多列。例如，一张实验结果表可能采用两级表头：第一层按照数据集分组；第二层分别列出各数据集对应的准确率和召回率。如果模型无法正确恢复跨列表头，就可能将不同数据集的指标混淆。

因此，表格结构识别通常需要判断：

- 表格包含多少行和多少列；
- 各单元格的边界和位置；
- 是否存在跨行或跨列单元格；
- 哪些单元格属于表头；
- 表头与数据单元格之间存在怎样的对应关系。

PubTabNet 是图像表格识别的重要代表性数据集，其标注使用结构化 HTML 表示表格内容与结构，并提出了用于评价表格结构相似性的 TEDS 指标。

PubTables-1M 则进一步提供大规模表格检测和结构标注，用于研究表格区域、行列、单元格及功能性结构的识别。这些研究说明，表格识别不能只评价文本是否准确，还需要检查表格的组织结构是否得到正确恢复。

表格数据的语义理解 完成表格结构恢复后，模型还需要理解表格中数据的具体含义。例如，可将实验结果表的阅读要求整理如下：如果用户询问：“哪一种方法在两个数据集上的准确率都最高？”模型需要分别比较两列中的数值，并找到共同满足条件的方法。正确答案是“方法 C”。这一任务并不需要复杂的数学运算，但需要模型正确理解表头和数值之间的对应关系。如果用户进一步询问：“方法 C 在数据集 A 上比方法 A 提高了多少个百分点?”

模型还需要执行：$91\% - 85\% = 6$ 个百分点 百分点变化与相对百分比变化并不是同一个概念。前者表示两个百分率之间的直接差值，后者通常需要以原始数值作为分母计算相对变化。因此，表格理解可能同时涉及结构恢复、语义关联和数值计算。对于具有单位的表格，还需要检查单位是否一致。例如，不能直接将以毫秒为单位的推理时间与以秒为单位的数值进行比较，而不进行单位转换。

这说明表格理解虽然以视觉信息为输入，但最终往往需要结构化数据处理与符号

推理共同参与。

统计图表中的视觉信息 除了表格，统计图表也是重要的信息表达形式。常见图表包括柱状图、折线图、饼图、散点图和面积图等。这些图表通常通过视觉形式编码数据。例如，柱状图使用柱子的高度或长度表示数值大小。折线图通过数据点的位置和连接关系展示变量变化。散点图通过点在二维坐标系中的分布表达变量之间的关系。饼图则利用扇形角度或面积比例表达整体中的组成关系。因此，图表理解不仅需要识别文字，还需要理解视觉元素与数据之间的映射。

对于一张折线图，模型通常需要识别标题、横轴、纵轴、刻度、图例和数据曲线。例如，横轴表示年份，纵轴表示销售额，不同颜色的折线表示不同产品。如果模型无法将颜色与图例对应，就可能将不同产品的数据混淆。因此，图表理解通常具有明显的多模态特点：文本提供变量名称、单位和背景说明；视觉图形提供数据位置、变化趋势和相互关系；坐标结构则决定如何将几何位置转换为具体数值。

示意图、流程图与复杂图形理解 除了统计图表，文档中还经常包含流程图、系统结构图、组织结构图和其他示意性图形。这类图形通常不以连续数值为主要内容，而是利用节点、箭头、标签和空间布局表达关系。例如，一个简单的系统架构图可能包含：“图像输入 $\rightarrow$ 特征提取 $\rightarrow$ 多模态融合 $\rightarrow$ 任务输出”。

模型如果只识别每个模块的文字，仍然无法确定模块之间的数据流向。因此，图形理解需要关注对象与连接关系。对于流程图，需要区分处理节点、判断节点和不同分支。对于组织结构图，需要理解不同节点之间的层级与归属关系。对于神经网络结构图，则可能需要识别输入输出、特征分支和模块连接方式。这类任务与前面介绍的场景图理解具有一定相似性，但图形中的关系往往由人工设计的视觉符号表达。

例如，箭头不仅表示两个对象有关联，还可能具有明确的方向和流程含义。因此，图形理解需要联合分析文本标签、图形形状、空间位置和连接关系。Pix2Struct 是面向视觉语言理解的代表性研究之一，通过将网页截图解析为简化 HTML 等预训练任务，学习视觉页面与结构化文本之间的对应关系。这一研究体现了一个重要方向：通过视觉到结构化表示的转换，增强模型对文档、网页和图形界面的理解能力。

不过，复杂流程图和系统结构图仍然存在连接线交叉、嵌套结构和隐含语义等问题，不能仅依靠普通 OCR 完整解决。

12.2.3 文档问答与跨区域信息关联

文档问答的基本概念 文档问答（Document Question Answering）是指模型根据用户提出的问题，从文档中寻找相关信息并生成答案。与普通文本问答相比，视觉文档问答（Document Visual Question Answering，DocVQA）的输入通常包含扫描文档或其他文档图像。因此，模型不仅需要理解自然语言问题，还需要从文档视觉内容中提取证据。例如，对于一张发票图像，用户可以询问：“这张发票的开票日期是什么？”

“购买的商品总金额是多少？”“收款方的名称是什么？”这些问题表面上属于文本信息提取，但实际需要模型识别文档中的文字，并理解不同字段之间的对应关系。例

如，一张发票可能同时出现“订单日期”“开票日期”和“支付日期”。如果模型只根据关键词“日期”搜索，就可能选错目标字段。因此，文档问答要求模型建立问题语义与文档结构之间的联系。

DocVQA 是这一领域的重要基准，包含大量基于真实文档图像构建的问答样本，用于研究文字识别、布局理解和文档信息提取等能力。经典单页 DocVQA 的答案主要来自文档中的文本片段，而更广义的文档问答还可能涉及多步推理、图表计算和跨页面信息整合。

文档问答与普通视觉问答的区别 虽然文档问答和视觉问答都采用图像与问题作为输入，但两者关注的信息类型有所不同。普通视觉问答通常围绕自然场景中的对象、属性、动作和空间关系展开。例如：“图中有几辆汽车？”文档问答则更加关注文字、结构和符号信息。例如：“合同中约定的付款期限是多少天？”对于自然场景，模型可能只需要识别关键对象和整体场景。但对于文档，模型经常需要准确读取小字体、识别数字和符号，并恢复文本之间的逻辑关系。

此外，文档问答对局部信息的准确性要求通常较高。例如，在合同和财务文档中，将“30 天”误读为“90 天”，可能直接改变条款含义。因此，文档问答不能仅追求语言表达合理，而需要强调可核验的事实依据。另外，一些文档问题并不是简单的信息抽取。例如：“根据表格中的数据，哪一项支出增长最快？”这类问题还需要模型完成数值比较和计算。因此，文档问答可以同时包含视觉识别、文本理解、结构关联和推理等多个环节。

文档中的多跳推理 当回答一个问题需要多个中间步骤时，通常称为多跳推理（Multi-hop Reasoning）。例如，用户询问：“在销售额增长最快的产品中，哪一种产品的利润率最高？”模型可能需要首先识别各产品在不同年份的销售额。然后计算或比较增长率。接着确定销售额增长最快的产品。最后查找该产品的利润率。这一问题不能简单通过匹配某个关键词获得答案，而是需要连续执行多个信息处理步骤。对于文档问答，可以将这一过程概括为：

问题理解 $\rightarrow$ 证据定位 $\rightarrow$ 信息提取 $\rightarrow$ 跨区域关联 $\rightarrow$ 逻辑或数值推理 $\rightarrow$ 答案生成。其中，证据定位和信息提取是后续推理的基础。

如果模型在第一个步骤中选择了错误的数据，后续推理即使形式正确，也可能得到错误结果。因此，多跳文档问答不仅需要评价最终答案，还可以根据任务需要检查中间证据是否正确。例如，对于数值推理问题，可以分别检查提取的原始数据、计算过程和最终结果。这有助于判断错误究竟来自视觉识别还是推理过程。

多页文档理解与长上下文问题 现实应用中的文档通常并不局限于单页。例如，合同、研究报告、学术论文和财务文件可能包含数十页甚至更多内容。这种情况下，模型需要在多个页面之间搜索和整合信息。例如，用户询问：“这份报告中第三季度与第四季度的主要经营风险有什么变化？”相关信息可能分别位于不同章节和页面。模型需要先识别与问题相关的页面，再提取相应内容，最后完成比较。多页文档理解面临几个问题。文档包含的信息量较大。

如果将全部页面转换为高分辨率图像 token，模型的输入长度和计算开销可能显著增加。页面之间存在层级和引用关系。例如，目录指向章节，正文引用附录，表格可能跨越多个页面。模型需要避免把不同页面中具有相似标题的内容混淆。例如，多份年度报告可能都包含“营业收入”字段，但对应年份和统计口径不同。MP-DocVQA 是面向多页文档视觉问答的代表性数据集。

相关研究提出的 Hi-VT5 使用分层方式处理页面信息，并探索通过页面级表示完成多页问答。这种思路说明，多页文档理解不仅需要增强单页识别能力，还需要研究跨页面的信息组织和证据选择。

基于检索的文档问答 对于较长的文档，可以先检索与问题相关的内容，再利用生成模型回答。这一过程与检索增强生成（Retrieval-Augmented Generation，RAG）的基本思想相关。在文档场景中，检索对象可以是文本段落、页面、表格、图片或其他文档区域。例如，用户询问：“报告中关于模型推理延迟的实验结果是什么？”系统可以先定位包含“推理延迟”“实验结果”等信息的相关页面或表格。然后将这些内容提供给多模态模型进行理解和回答。

文档检索可以采用不同方式。一种方法是先进行 OCR 和结构恢复，再使用文本检索或语义向量检索。另一种方法是直接对文档页面进行视觉表示学习，根据问题检索相关页面或区域。还可以联合文本与视觉特征进行检索。不同方法的适用条件并不相同。例如，OCR 文本检索适合文字清晰、结构相对简单的文档。但对于图表、复杂布局和大量视觉符号构成的页面，单纯文本检索可能遗漏重要信息。

因此，实际系统可以根据任务需要结合文本检索、版面信息和视觉检索。检索到正确页面并不意味着模型已经正确回答问题。检索阶段主要解决“相关信息在哪里”的问题。文档理解与推理阶段则进一步解决“这些信息表达什么，以及如何利用这些信息回答问题”的问题。

文档问答的证据追踪与可信性 在文档问答中，回答的可验证性有意义。例如，用户询问某份报告的财务数据，系统不仅需要给出数值，还应尽可能说明该数值来自哪一页、哪一张表或哪个字段。这种做法通常称为证据追踪或来源定位。可以将文档问答结果表示为：$A = (y, E)$，其中，y 表示最终答案，E 表示支持答案的证据集合。证据可以包含页面编号、文本区域、表格单元格或其他可定位的信息。例如：

“2024 年营业收入为 120 亿元，数据来自第 15 页的年度经营情况表。”这种回答方式便于用户核验。不过，模型给出了来源位置，并不意味着该来源一定真正支持答案。例如，模型可能引用正确页面，却错误读取表格中的数值。因此，可信文档问答需要分别检查答案正确性、证据定位准确性以及证据与答案之间的支持关系。对于涉及财务、法律和其他高风险决策的文档，还应特别重视数值核验、版本管理和人工复核。

12.3 区域预测与语言条件定位

12.3.1 目标检测与区域级识别

目标检测的基本概念 目标检测（Object Detection）是计算机视觉中的基础任务，其目标是在输入图像中识别目标对象的类别，并确定其空间位置。与图像分类不同，目标检测需要同时回答两个问题：一是图像中存在什么对象。二是这些对象分别位于什么位置。例如，对于一张包含汽车、行人和交通信号灯的图像，图像分类模型可能判断该图像属于道路场景，而目标检测模型则需要分别找出汽车、行人和交通信号灯，并给出它们对应的位置。通常，目标检测结果可以表示为：

$$\mathcal{D} = (b_i, c_i, s_i)_{i=1}^{N} \tag{12.6}$$

其中：

- N 表示预测目标数量；
- b_i 表示第 i 个目标的边界框；
- c_i 表示目标类别；
- s_i 表示预测置信分数。

对于采用矩形框表示的目标，其空间位置通常可以写为：

$$b_i = (x_{\min}, y_{\min}, x_{\max}, y_{\max}) \tag{12.7}$$

其中，四个坐标分别表示边界框左上角和右下角的位置。实际模型也可以使用中心点坐标以及宽度、高度表示边界框。例如，对于一张包含两辆汽车的图像，检测结果可能包含两个汽车实例，分别具有不同的边界框和置信分数。这说明目标检测不仅需要识别类别，还需要区分属于同一类别的不同对象实例。检测模型输出的置信分数通常用于排序和筛选预测结果，并不必然等于经过严格概率校准的正确性概率。

因此，在实际系统中，置信分数的使用还需要结合模型训练方式和评价协议进行判断。

从图像分类到区域级识别 图像分类通常以整张图像作为分析对象，输出一个或多个语义类别。但现实图像往往包含多个对象，并且不同对象可能占据不同的局部区域。因此，仅使用整张图像的全局特征，可能无法充分表示每个对象的具体属性和空间关系。区域级识别（Region-level Recognition）则将分析对象从整张图像进一步细化到局部区域。例如，对于一张办公室图像，可以分别识别桌子、电脑、椅子和人物。对于每个区域，还可以分析其颜色、形状、状态等属性。

区域级识别的一个基本思想是：首先确定候选区域，然后提取该区域的视觉特征，最后进行分类或其他语义分析。假设图像为 I，候选区域为 b_i，可以表示为：

$$z_i = f_{\text{region}}(I, b_i) \tag{12.8}$$

其中，z_i 表示区域特征。随后，可以通过分类模型获得：

$$P(c_i \mid z_i) \tag{12.9}$$

这种形式强调，区域级识别的对象不再是整张图像，而是具有明确空间范围的局部视觉内容。区域级识别与目标检测并不是完全独立的概念。目标检测通常包含区域定位与类别识别两个方面，而区域级识别也可以用于属性判断、图文匹配和视觉关系分析等更广泛的任务。

传统目标检测方法的发展 深度学习目标检测方法通常可以从两阶段检测与单阶段检测两个角度进行理解。两阶段目标检测。两阶段检测通常首先生成可能包含目标的候选区域，再对候选区域进行类别预测和位置优化。Faster R-CNN 是这一方向的代表性工作。该方法通过区域建议网络（Region Proposal Network，RPN）生成候选框，再利用后续检测模块进行分类和边界框回归。

其主要特点是将候选区域生成与目标检测整合到共享特征提取框架中。这一设计使候选区域不再需要完全依赖独立的传统图像处理算法。单阶段目标检测。单阶段检测通常直接根据图像特征预测目标类别和位置，不再采用独立的候选框分类阶段。YOLO 和 SSD 等方法是这一方向的重要代表。单阶段检测在许多应用中具有较高的推理效率，因此被广泛用于实时视觉感知任务。不过，两阶段和单阶段只是目标检测方法的一种经典分类方式。

随着网络结构与训练方法的发展，实际模型在准确率、效率和实现机制方面已经形成更加丰富的设计。不能简单认为两阶段方法一定更准确，或者单阶段方法一定更快。具体性能需要结合模型结构、训练规模、输入分辨率和硬件环境进行比较。

基于 Transformer 的目标检测 随着 Transformer 在视觉任务中的应用，研究者开始探索将目标检测直接表示为集合预测问题。DETR（DEtection TRansformer）是这一方向的代表性工作。DETR 使用 Transformer 编码器和解码器，通过一组可学习的对象查询（Object Queries）预测图像中的目标集合。

与传统依赖候选框和大量手工设计规则的检测方法相比，DETR 通过集合匹配损失建立预测对象与真实目标之间的对应关系。其训练过程使用二分图匹配，为预测结果分配目标监督信息。这种方式使原始 DETR 不再依赖传统的非极大值抑制（Non-Maximum Suppression，NMS）来去除重复检测框。

不过，原始 DETR 也存在训练收敛相对缓慢、对小目标感知较弱等问题，后续研究围绕注意力机制、多尺度特征和查询初始化进行了大量改进。对于多模态感知研究而言，Transformer 检测模型的重要意义不仅是提供另一种目标检测架构，还在于其对象查询机制能够与语言表示和跨模态注意力进一步结合。例如，模型可以根据自然语言描述形成与目标相关的查询，并在视觉特征中寻找相应对象。这为语言驱动的目标检测和指代定位提供了重要技术基础。

从固定类别检测到开放词汇检测 传统目标检测通常依赖预先定义的类别集合。例如，一个针对常见物体训练的检测器可能包含“人”“汽车”“自行车”等类别。如果模型

没有接受某种新类别的训练，就可能无法直接将其识别为独立类别。开放词汇目标检测（Open-Vocabulary Object Detection，OVD）试图缓解这一限制。其基本思路是利用视觉语言预训练获得的语义表示，使检测模型能够根据文本类别描述识别训练阶段未作为目标检测类别充分标注的对象。

例如，用户可以向模型提供：“消防栓”“路边的共享单车”“蓝色旅行箱”模型根据文本语义与图像局部区域之间的关联，预测相应目标的位置。这一方向中的代表性工作包括 OWL-ViT、GLIP 和 Grounding DINO 等。其中，OWL-ViT 研究如何将视觉语言预训练表示迁移到开放词汇目标检测。GLIP 将目标检测与短语定位统一到语言—图像预训练框架中，使模型学习对象级别的视觉语言表示。

Grounding DINO 则结合基于 Transformer 的检测架构与语言引导机制，实现根据类别名称或其他文本表达进行目标定位。与固定类别检测相比，开放词汇检测具有更加灵活的任务形式。不过，“开放词汇”并不意味着模型能够可靠识别现实世界中的任意对象。模型仍然受到预训练数据、视觉编码能力、类别语义相似性和目标尺度等因素限制。

例如，对于训练数据中极少出现的专业设备或细粒度工业零件，即使用户提供了准确的文本名称，模型也未必能够可靠定位。因此，开放词汇检测强调类别表达与迁移范围的扩展，而不是取消目标检测的泛化限制。

边界框、区域与像素级表示 目标检测通常使用边界框描述对象位置。边界框表示简单、计算方便，但只能近似描述目标的空间范围。例如，一名行人的边界框可能同时包含人物、道路和部分背景。如果应用需要精确判断人物轮廓，边界框就无法提供足够细致的信息。因此，在更高空间精度要求下，可以使用像素级分割掩码（Segmentation Mask）表示目标。对于图像中的某个对象，可以定义二值掩码：$M_i(x, y) \in 0, 1$，

其中，当像素 (x, y) 属于目标 i 时，掩码取值为 1，否则为 0。与边界框相比，分割掩码能够更加细致地描述目标形状。不过，目标检测与实例分割的任务目标并不完全相同。目标检测主要预测对象的位置和类别。实例分割进一步要求区分不同对象实例对应的像素区域。此外，语义分割关注像素所属类别，并不一定区分同类别的不同实例。因此，在细粒度视觉感知中，需要根据任务目标选择相应的空间表示形式。

Segment Anything Model（SAM）展示了通过提示驱动进行通用图像分割的一种技术路线。SAM 可以根据点、框等空间提示生成目标区域的分割结果，但原始 SAM 并不是一个能够独立理解任意自然语言指代表达的完整语言定位系统。在实际应用中，可以先使用语言驱动的检测模型得到目标边界框，再利用提示式分割模型生成更加精细的区域掩码。这种组合体现了语义定位与空间分割之间的互补关系。

12.3.2 指代表达与视觉定位

指代表达的基本概念 指代表达（Referring Expression）是指利用自然语言描述特定对象，使接收者能够从场景中的多个对象中确定被指代的目标。例如，在一张包含多辆汽车的图像中，用户可以说：“左侧那辆红色汽车。”“停在公交车后面的白色汽车。”“靠近路口、车门打开的车辆。”这些表达不仅包含对象类别，还可能包含颜色、位置、状

态和对象间关系等信息。

指代表达理解（Referring Expression Comprehension，REC）要求模型根据图像和指代表达，确定其对应的目标区域。可以表示为：$\hat{b} = f_\theta(I, e)$，其中：

- I 表示输入图像；
- e 表示指代表达；
- $\hat{b}$ 表示模型预测的目标边界框。

与普通目标检测相比，指代表达理解具有更强的语言条件性。

目标检测通常回答："图像中有哪些汽车？"而指代表达理解需要回答："用户描述的究竟是哪一辆汽车？"因此，指代表达理解要求模型在多个候选目标之间进行更细粒度的区分。

指代表达中的主要语义线索 自然语言可以通过多种方式指向特定视觉对象。类别线索。例如："图中的自行车。"模型需要根据类别语义识别对应对象。如果图像中只有一辆自行车，这一表达通常已经具有较强的区分能力。属性线索。例如："穿红色外套的人。"模型不仅需要识别人物，还需要判断衣服颜色，并将颜色属性与正确的人物实例关联起来。空间线索。例如："最右边的椅子。""位于桌子下方的箱子。"这些表达要求模型理解对象的空间位置。关系线索。

例如："站在自行车旁边的人。""被小孩牵着的狗。"此时模型需要联合识别多个对象，并判断对象之间的关系。比较与组合线索。例如："两个杯子中较大的那个。""站在左边、穿蓝色衣服并拿着手机的人。"这类表达通常涉及多个属性和条件的组合。因此，指代表达理解并不是简单的关键词匹配。对于复杂指代表达，模型需要将类别、属性、位置和关系信息联合起来，逐步排除不符合描述的候选对象。

指代表达定位的主要方法 早期指代表达理解方法通常采用候选区域匹配的方式。目标检测器生成一组候选对象区域：$B = b_1, b_2, \ldots, b_N$，随后，模型分别提取候选区域的视觉特征，并与语言表达进行匹配。可以将第 i 个候选区域的匹配分数写为：$s_i = g_\theta(v_i, e)$，其中：

- v_i 表示候选区域的视觉特征；
- e 表示指代表达的语言表示；
- g_θ 表示跨模态匹配函数。

最终选择分数最高的候选区域：

$$\hat{i} = \arg\max_i s_i \tag{12.10}$$

这种方法具有较为清晰的处理流程。但其定位能力受到候选区域质量限制。如果真实目标没有被检测器正确生成，后续跨模态匹配就可能无法找到对应区域。因此，后续研究逐渐探索将语言信息直接引入目标定位过程。MDETR 是这一方向的重要代表。该模型根据文本输入调节目标检测过程，通过联合视觉与语言表示，直接预测与文本

语义相关的目标区域。这种方法不必完全依赖一个与语言任务独立训练的固定类别检测器。

语言与视觉定位之间的联系由模型联合学习。Grounding DINO 等工作进一步研究如何将语言引导机制与开放集目标检测结合起来，使模型能够根据类别名称、属性描述和指代表达定位对象。不同方法对语言信息的利用程度和输出形式可能存在差异，不能简单将所有文本条件检测模型视为具有完全相同的指代表达理解能力。

指代定位与指代分割 指代表达定位通常输出目标边界框，而指代图像分割（Referring Image Segmentation，RIS，也称 Referring Expression Segmentation，RES）则要求输出目标对应的像素掩码。例如，用户提出：“请分割图中站在汽车旁边、穿蓝色衣服的人。”如果模型只返回人物的矩形边界框，就完成了粗粒度定位。如果模型进一步输出该人物的像素级区域，就属于指代分割。

两类任务的主要区别在于空间输出形式。指代定位：$(I, e) \to b$ 指代分割：$(I, e) \to M$，其中，b 为边界框，M 为分割掩码。指代分割通常对目标轮廓、遮挡边界和相邻对象的区分提出更高要求。例如，两个人物站得很近时，边界框可能存在明显重叠，而像素级分割需要尽可能区分不同人物的可见区域。

不过，是否能够准确分割目标，还取决于具体数据集对遮挡部分、可见区域和对象边界的标注约定。因此，框级定位与像素级分割既具有联系，也需要分别评价。

指代表达定位的主要挑战 指代表达理解仍然面临多种困难。首先是视觉对象之间的相似性。如果图像中存在多个颜色、形状和位置相近的对象，模型需要利用更加细微的差异完成区分。其次是复杂关系与组合条件。例如：“坐在左侧长椅上、旁边放着红色背包的人。”这一表达涉及人物、长椅和背包三个对象，以及多个空间关系。再次是目标遮挡与小目标问题。当目标只显示少量可见区域时，其属性可能难以直接识别。此外，还存在表达歧义与目标不存在的问题。

例如，用户描述“穿蓝色衣服的人”，但图像中有两个人都穿蓝色衣服。如果没有其他线索，模型可能无法唯一确定目标。类似地，如果图像中不存在符合描述的对象，可靠系统应当能够根据任务协议表达无匹配结果，而不是强行选择一个对象。因此，细粒度定位不仅是寻找匹配分数最高的区域，还涉及如何处理不确定性和语言描述中的歧义。

12.3.3 局部区域与文本片段对齐

从图像级对齐到区域级对齐 视觉语言预训练的重要目标之一，是建立视觉内容与自然语言之间的语义关联。例如，CLIP[8] 等模型通过大规模图像—文本对进行对比学习，使图像和文本能够映射到具有语义关联的表示空间。这种方式为跨模态检索和开放词汇识别提供了重要基础。但图像级对齐并不等于区域级对齐。例如，一张图像的描述为：“一名穿红色衣服的男子站在一辆蓝色汽车旁边。”整张图像与这句话在语义上可能高度匹配。

然而，仅有全局图文匹配结果，并不能直接确定“男子”“红色衣服”“蓝色汽车”

等文本片段分别对应图像中的哪些区域。因此，细粒度图文对齐（Fine-grained Vision-Language Alignment）进一步要求建立文本局部语义与视觉局部区域之间的对应关系。它不仅关注“这张图像是否符合这段文字”，还关注“文字中的哪些内容对应图像中的哪些位置”。

这种能力是短语定位、指代理解、区域描述和基于视觉证据的多模态生成等任务的重要基础。

短语定位的基本任务 短语定位（Phrase Grounding）是指根据给定图像和文本，找到文本中一个或多个短语对应的视觉区域。例如，输入描述为：“一名小孩抱着一只白色小狗，旁边放着一个蓝色书包。”模型需要建立以下对应关系：“一个小孩”对应人物区域。“白色小狗”对应狗的区域。“蓝色书包”对应书包区域。与指代表达理解相比，短语定位通常可以同时处理句子中的多个短语。指代表达理解通常以确定某个特定目标为中心。

短语定位则更加关注一段文本中不同语义片段与视觉区域之间的对应关系。当然，两类任务存在较大交集。例如，当短语只有一个，并且需要定位唯一目标时，两者可能采用相似的输入输出形式。因此，它们的区别主要体现在任务定义、标注单位和评价协议上，而不是完全不同的技术体系。

区域与短语的联合表示 在区域级图文对齐中，可以分别构建视觉区域表示和文本短语表示。假设图像包含 N 个候选视觉区域：$V = v_1, v_2, \ldots, v_N$ 文本包含 M 个需要定位的短语：$T = t_1, t_2, \ldots, t_M$ 模型可以计算每个视觉区域与文本短语之间的相似性：

$$S_{ij} = \mathrm{sim}(v_i, t_j) \tag{12.11}$$

其中，S_{ij} 表示第 i 个视觉区域与第 j 个文本短语之间的匹配分数。通过这些分数，可以建立区域与短语之间的对应关系。例如：

- 区域 1 对应“穿红色衣服的男子”；
- 区域 2 对应“蓝色汽车”；
- 区域 3 对应“道路”。

这种表示形式较为直观，但并不能完整覆盖所有复杂情况。

例如，一个文本短语可能对应多个对象。“一群正在跑步的人”可能涉及多个行人区域。反过来，同一个视觉对象也可能对应文本中的多个表达。例如，“一辆红色汽车”和“这辆车”可能指向同一个对象。因此，区域与短语之间并不总是一对一关系。实际模型需要根据任务定义支持一对多、多对一或更加复杂的关联形式。

区域级对齐与多模态大语言模型 随着多模态大语言模型的发展，研究者开始探索将区域定位能力直接整合到语言生成过程中。传统视觉语言模型主要根据图像生成文本。如果模型回答：“图中一名男子正在骑自行车。”这一回答可以表达对象和动作关系，但未必提供男子或自行车的具体位置。具有视觉定位能力的多模态大模型则可以进一步将文本片段与目标边界框关联起来。例如，系统不仅生成“男子”和“自行车”，还能

够输出它们对应的空间位置。

Kosmos-2 是这一方向的代表性研究。该模型通过带有定位信息的图文数据训练，将文本中的实体或指代表达与视觉目标的位置表示联系起来。这种设计使语言生成与视觉定位不再完全分离，而是能够在统一的多模态框架下进行联合建模。这一思路对于构建具有视觉依据的多模态系统有意义。例如，当模型解释图像中的某个对象时，可以同时提供对象对应的区域。用户不仅能够阅读解释，还可以在图像中检查被解释的目标是否正确。

但模型能够生成坐标信息，并不意味着其坐标一定准确。视觉定位仍然需要通过独立的框级或像素级指标进行验证。

局部对齐与视觉证据的关联 区域级图文对齐不仅服务于目标定位，还可以帮助多模态模型建立更加明确的视觉证据关系。例如，用户询问："为什么图中的行人可能正在等待过马路?" 模型可能回答："因为行人站在斑马线旁边，前方交通信号灯为红色。" 在这一回答中，"行人""斑马线" 和 "交通信号灯" 都对应具体视觉内容。如果模型能够将这些文本片段定位到图像中的相应区域，就可以进一步检查其解释是否具有视觉依据。但局部区域与文本匹配正确，并不意味着整个因果解释一定正确。

例如，行人站在斑马线旁边，并不能直接证明其真实意图一定是等待过马路。因此，需要区分两种不同层次的证据：对象和属性层面的视觉证据，例如红色信号灯是否存在、行人位于何处。语义和推理层面的证据支持，例如视觉事实是否足以支持 "正在等待过马路" 这一判断。区域级对齐主要帮助模型建立前一层次的联系。如果需要进一步验证复杂推理结论，还必须检查视觉事实与推理过程之间的逻辑关系。

12.4 时间片段与音视频事件

12.4.1 时间定位与片段检索

时间定位的基本概念 时间定位（Temporal Localization）是指在视频或音频序列中确定某个动作、事件或语义内容发生的时间范围。与空间定位不同，时间定位关注的主要对象是时间区间。例如，用户提供一段十分钟的视频，并询问："人物第一次拿起红色杯子发生在什么时候?" 模型不仅需要识别拿杯子的动作，还需要确定这一动作的开始和结束时间。

对于视频 V 和查询条件 q，可以将时间定位表示为：$\hat{m} = f_\theta(V, q)$，其中：$\hat{m} = (\hat{t}_s, \hat{t}_e)$，$\hat{t}_s$ 和 $\hat{t}_e$ 分别表示预测事件的开始时间与结束时间。时间定位与视频级分类具有明显区别。视频级分类只需要判断某段视频是否包含某种动作，而时间定位进一步要求指出该动作发生在哪个时间范围。

例如，对于一段包含跑步、休息和喝水的视频，视频级动作识别可能判断其中存在 "跑步" 行为。时间定位则需要确定跑步行为对应的具体时间区间。

时间动作定位与自然语言片段检索 根据输入条件与输出形式，时间定位可以包含不同类型的任务。时间动作定位（Temporal Action Localization）。这类任务通常根据预定

义动作类别，识别视频中不同动作实例的时间范围。例如：“定位视频中所有跳跃动作。”模型可能需要返回多个时间区间，并为每个区间给出动作类别和置信分数。自然语言视频片段定位（Natural Language Video Moment Retrieval）。

这类任务根据自由形式的语言描述，寻找视频中与该描述对应的时间片段。例如：“找到人物将杯子放在桌子上的片段。”相比固定动作类别，自然语言描述可以包含更加丰富的对象、属性和关系信息。视频高光检测（Video Highlight Detection）。高光检测主要识别视频中具有较高重要性、显著性或与特定查询相关的片段。例如，对于一段体育比赛视频，模型可以根据用户需求寻找进球、精彩扑救或其他重要场面。

高光检测与事件定位并不完全相同。某个事件可能持续较长时间，但其中只有部分片段被认为具有较高的重要性。此外，高光片段可能依赖用户偏好和任务目的，而不一定对应某种固定事件类别。因此，三类任务虽然都涉及时间信息，但其监督目标和评价方法存在差异。

时间交并比 tIoU 与图像中的边界框定位类似，时间定位也需要衡量预测区间与真实区间之间的重叠程度。时间交并比（Temporal Intersection over Union，tIoU）是常用指标。假设预测时间区间为：$\hat{m} = [\hat{t}_s, \hat{t}_e]$ 真实时间区间为：$m^* = [t_s^*, t_e^*]$，则：

$$\text{tIoU}(\hat{m}, m^*) = \frac{|\hat{m} \cap m^*|}{|\hat{m} \cup m^*|} \tag{12.12}$$

其中，分子表示两个区间重叠部分的持续时间，分母表示两个区间覆盖时间的并集长度。tIoU 越高，说明预测时间范围与真实事件范围越接近。例如，假设某个事件真实发生在第 20 秒到第 30 秒。如果模型预测为第 21 秒到第 29 秒，其时间区间与真实区间有较大重叠。如果模型预测为第 35 秒到第 45 秒，则两个区间没有交集，tIoU 为 0。

即使模型正确识别了事件内容，如果时间边界明显偏移，仍然可能在严格的 tIoU 阈值下被判定为定位失败。因此，时间定位需要同时关注事件语义和时间边界精度。

基于自然语言的片段检索 自然语言片段检索是视觉语言理解与时间定位相结合的重要任务。例如，用户可以提出：“找到视频中小孩第一次骑上自行车的片段。”“定位车辆从左侧进入画面并停下的过程。”“查找人物打开冰箱之后拿出牛奶的片段。”这些查询不仅包含动作类别，还可能包含对象属性、事件顺序和时间关系。因此，模型需要将语言语义与视频中的时序特征进行匹配。一种常见方法是将视频划分为多个时间片段：$V = v_1, v_2, \ldots, v_K$，

其中，v_k 表示第 k 个片段的特征。随后，将自然语言查询编码为表示 q，并计算文本与视频片段之间的相关性。通过时序特征聚合、注意力或其他匹配机制，模型可以预测目标时间区间。早期方法通常采用候选时间窗口生成与排序的方式。后续研究逐渐探索直接预测时间区间的端到端模型。Moment-DETR 是这一方向的代表性工作之一。

该方法将视频片段检索建模为集合预测问题，通过 Transformer 预测查询相关的时间片段及其显著性分数。对应的 QVHighlights 数据集同时提供自然语言查询、相关

时间片段和片段显著性标注，用于研究视频时刻检索与高光检测。这一研究说明，视频片段检索不仅涉及语言与视频的语义匹配，还需要学习时间边界和事件显著性。

时间定位的评价指标 时间定位通常使用多个指标进行评价。对于单查询片段检索，常见指标为 Recall@K，并结合 tIoU 阈值进行判断。例如：$R@1, tIoU = 0.5$ 表示模型排名第一的预测时间区间与真实区间的 tIoU 达到 0.5 的查询比例。对于包含多个事件实例的时间动作检测任务，通常使用基于不同 tIoU 阈值的平均精度（AP）和平均 AP（mAP）。

对于视频高光检测，则可以使用片段重要性排序相关指标，例如 mAP 或其他与标注形式对应的评分方法。对于文本形式的视频摘要，还可以采用 ROUGE、METEOR、CIDEr 等指标评价生成文本与参考描述之间的相似性。但这些文本指标并不能充分评价摘要是否准确反映了事件顺序和视觉事实。因此，视频摘要和时间定位的综合评价应同时考虑内容正确性、时间范围、事件覆盖和冗余程度。

12.4.2 音频事件与语音内容理解

音频理解的基本概念 音频理解（Audio Understanding）是指从声音信号中识别语音、环境事件、声源属性及其语义内容。与图像不同，音频是随时间变化的信号。对于数字音频，可以使用离散序列表示：

$$A = x[n]_{n=0}^{N-1} \tag{12.13}$$

其中，$x[n]$ 表示第 n 个采样点的信号幅度，N 表示采样点总数。音频信号通常需要以一定采样率进行采集。例如，采样率为 16 kHz 时，每秒包含 16000 个采样点。但音频理解模型通常不会只依赖原始波形的简单统计特征。声音包含丰富的频率结构和时间变化特征，因此常需要进行时频分析。

例如，短时傅里叶变换（Short-Time Fourier Transform，STFT）可以将音频转换为随时间变化的频谱表示。进一步处理后，可以得到功率谱、Mel 频谱或其他声学特征。这些表示能够帮助模型分析声音的频率组成、持续时间和变化规律。例如，车辆发动机声、鸟鸣声和人类语音通常具有不同的时频特征。不过，现代音频模型也可以直接基于波形学习表示，并不一定需要预先计算固定的声学特征。

音频事件识别 音频事件识别（Audio Event Recognition）主要关注声音中发生了什么事件。例如：“狗叫”“汽车鸣笛”“玻璃破碎”“雨声”“脚步声”这些事件并不一定包含语言内容，但可以提供重要的环境信息。在音频分析中，需要区分几个相关任务。

音频事件分类（Audio Event Classification）通常判断某段音频中是否存在指定声音类别。

声音事件检测（Sound Event Detection, SED）进一步要求确定声音事件的发生时间，并可能同时识别多个重叠事件。

声学场景分类（Acoustic Scene Classification）则更加关注声音所反映的整体环境，例如车站、街道、办公室或森林。

例如，一段音频中可能同时包含汽车发动机声、鸣笛声和行人交谈声。音频事件分类可以判断这些声音类别是否存在。声音事件检测还需要确定各事件对应的时间范围。声学场景分类则可能根据整体声音特征判断该音频来自城市道路环境。因此，不同音频任务具有不同的时间粒度和语义目标。

说话人识别与语音时间结构 在多人语音场景中，仅获得文字内容可能还不够。例如，一段会议录音中，系统可能需要知道：“谁在什么时候说了什么？”这一问题涉及说话人分离标注（Speaker Diarization）。其主要任务是将音频划分为不同说话人对应的时间片段，并为不同说话人分配相对一致的匿名身份标签。需要区分说话人分离标注与说话人身份识别。说话人分离标注主要判断不同语音片段是否来自同一说话人，而不一定需要知道说话人的真实身份。

如果系统还能够将说话人与特定个人身份对应，则涉及额外的身份识别问题。在视频会议场景中，系统还可以结合人物口型、面部运动和声音信息，判断当前画面中哪个人正在说话。这类能力与后面介绍的音视频联合理解密切相关。此外，语音还可以进行词级或句级时间对齐。例如，系统可以输出：00:01—00:03：“今天我们讨论研究计划。”00:04—00:06：“首先介绍实验结果。”

这种带有时间信息的转写，为会议摘要、视频字幕和语音片段检索提供了基础。

音频描述与音频语言模型 除了识别固定类别的声音事件，模型还可以根据音频生成自然语言描述。这一任务通常称为音频描述（Audio Captioning）。例如，对于一段包含雨声和汽车经过声的音频，模型可以生成：“持续的雨声中，一辆汽车从附近驶过。”这一任务不仅需要识别声音类别，还需要整合不同声源及其时间关系。AudioCaps 是音频描述研究中的代表性数据集，提供声音片段及其人工编写的自然语言描述。

随着音频语言预训练的发展，研究者开始尝试将声音与文本映射到共享语义空间。例如，CLAP（Contrastive Language-Audio Pretraining）通过音频与文本之间的对比学习，建立声音内容和自然语言描述之间的关联。类似于图像与文本对比学习，这类方法可以支持音频检索和一定条件下的零样本声音分类。音频语言模型开始支持更加开放的交互形式。

Qwen2-Audio 是具有代表性的音频语言模型研究之一，支持音频感知以及基于声音输入的自然语言交互。例如，用户可以提供一段声音并提出：“这段音频中有哪些主要声音？”“是否有人正在说话？”“请概括这段语音的主要内容。”这说明音频理解正在从固定类别预测向开放式音频问答和语音交互扩展。不过，开放式音频回答仍然存在声音幻觉和语义推断过度等问题。

例如，仅根据一段短暂的发动机声音，模型通常无法确定具体车辆品牌或行驶目的地。因此，音频语言模型也需要区分能够直接听到的事实、根据声音合理推测的信息以及无法确定的内容。

音频理解的评价方法 不同音频任务通常采用不同的评价指标。对于音频分类，可以使用 Accuracy、Precision、Recall、F1 和 mAP 等指标。对于多标签声音事件识别，由于一个音频片段可能包含多个声音类别，因此通常需要采用适合多标签任务的评价方法。

对于声音事件检测，还需要评价事件发生的时间范围。常见方法包括基于时间片段的 F1、基于事件边界匹配的 F1，以及其他考虑检测阈值和时间误差的指标。

对于自动语音识别，常用词错误率（Word Error Rate，WER）：

$$\mathrm{WER} = \frac{S + D + I}{N} \tag{12.14}$$

其中，S、D 和 I 分别表示替换、删除和插入错误数，N 表示参考转写中的词数。对于中文语音识别，也常采用字符错误率（Character Error Rate，CER），以避免不同分词方式对评价产生较大影响。对于音频描述，可以使用 CIDEr、SPICE、SPIDEr 等文本生成评价指标。但这些指标并不能完全衡量生成描述是否与真实声音一致。

因此，对于开放式音频理解，还需要结合声音事件正确性、内容完整性和必要的人工核验进行评价。

12.4.3 音视频联合理解

音视频联合理解的基本概念 音视频联合理解（Audio-Visual Understanding）是指同时利用视觉和声音信息，识别事件、理解场景并完成跨模态推理。相比单独处理视频或音频，音视频联合理解能够利用不同模态之间的互补信息。例如，一段视频展示一辆汽车正在驶过路口。视频能够提供汽车的位置、外观和运动方向。音频则可能提供发动机声、鸣笛声和周围交通环境信息。当两种信息具有可靠的时间和语义对应关系时，模型可以获得更加完整的事件描述。

但音视频联合理解并不意味着将两种特征直接拼接就一定能够提升性能。模型需要判断哪些声音与哪些视觉内容相关，以及不同信息之间是否相互支持。例如，画面中存在多辆汽车，而音频中只有一声鸣笛。此时，模型不能仅凭音频与视频同时出现，就确定是哪辆汽车发出了声音。因此，音视频联合理解需要建立空间、时间和语义层面的跨模态关联。

音频与视频的时间同步 音视频联合理解的基础之一，是正确处理两种模态之间的时间关系。视频通常以一定帧率进行采样，而音频采样率远高于视频帧率。例如，视频可能以每秒 25 帧记录图像，而音频以每秒 16000 个采样点记录波形。因此，两种模态具有不同的时间分辨率。如果直接将音频采样点与视频帧按照数组索引进行对应，就会产生错误的时间关系。更合理的方法是利用时间戳建立同步关系。

假设视频帧的时间戳为 τ_t^v，音频特征的时间戳为 τ_j^a。模型需要根据实际时间，确定哪些音频特征对应某个视频帧或时间窗口。例如，可以将视频中的某个时间片段与覆盖同一时间范围的音频片段关联起来。如果两种数据源之间存在时间偏移，还需要进行同步校正。可以使用：

$$\tau_j^{a'} = \tau_j^a + \Delta \tag{12.15}$$

表示对音频时间戳进行偏移修正。其中，Δ 表示估计或已知的时间偏移量。对于真实系统，时间同步可能受到采集延迟、设备时钟误差和数据缓冲等因素影响。因此，

音视频同步不仅是模型内部的特征对齐问题，也涉及数据采集与预处理协议。

音视频融合的主要方式 音视频联合模型通常需要分别提取两种模态的特征，再通过一定机制进行融合。假设视频特征为：$H_v = f_v(V)$ 音频特征为：$H_a = f_a(A)$，其中，f_v 和 f_a 分别表示视觉与音频编码器。模型需要利用某种融合函数：$H_{av} = F(H_v, H_a)$ 获得联合表示。根据融合发生的位置和方式，可以采用不同策略。早期融合（Early Fusion）。

早期融合尝试在较浅的表示阶段整合音频与视觉特征。例如，将经过时间对齐的音频和视频特征拼接，再输入后续网络。这种方法能够让模型较早学习跨模态关联，但可能受到两种特征分布差异和时间分辨率差异的影响。中间层融合（Intermediate Fusion）。中间层融合通常先分别提取音频和视频特征，再通过注意力或其他交互模块建立跨模态联系。例如，模型可以使用音频特征引导视觉注意力，使网络更加关注可能正在发声的对象区域。

这种方式能够保留不同模态的部分独立表示，同时支持跨模态交互。后期融合（Late Fusion）。后期融合通常分别获得音频和视频预测结果，再通过加权、投票或其他决策机制进行整合。例如，一个模型根据视频判断事件类别，另一个模型根据音频判断事件类别，最后综合两者结果。后期融合较容易独立分析不同模态的贡献，但可能难以充分捕获复杂的局部音视频关联。因此，不同融合方式并不存在绝对的优劣关系。

对于需要精确声源定位或口型同步的任务，中间层跨模态交互可能具有重要价值。对于相对独立的环境声音与场景分类，较简单的融合方式也可能有效。实际方法选择仍然需要依据任务需求和公平实验结果进行判断。

音视频中的模态互补与信息冲突 音频与视频之间通常存在互补关系，但也可能出现信息冲突。例如，在道路场景中，视频可能显示一辆汽车正在缓慢行驶，而音频中出现较强的发动机声音。这种情况可能具有多种解释。声音可能来自画面中的汽车，也可能来自附近但不在画面中的其他车辆。因此，模型不应直接将两种信息解释为同一对象的属性。类似地，在经过剪辑的视频中，背景音乐和画面事件可能没有直接的因果联系。因此，音视频联合系统需要区分模态信息的互补性和一致性。

当两种模态提供相互支持的证据时，融合可能提高事件识别能力。当某种模态受到噪声、遮挡或采集错误影响时，另一种模态可能提供额外信息。但如果模型没有正确判断模态之间的对应关系，融合也可能引入错误。例如：视觉信息中不存在鸣笛车辆，但模型根据声音错误地描述“画面中的汽车正在鸣笛”。这种情况说明，融合不仅需要学习跨模态相关性，还需要避免将相关性直接视为确定的对象对应或因果关系。

因此，可靠的音视频理解需要结合时间同步、语义匹配和证据一致性进行判断。

异步观测与连续音视频理解 在实际系统中，音频与视频通常具有不同的采样频率和信息更新特点。音频以较高采样率连续记录声音波形，而普通视频通常以较低帧率提供视觉观测。因此，当视觉信息尚未发生明显变化时，音频中可能已经出现新的事件。例如，一段道路视频中，车辆仍然位于画面远处，但音频中已经出现短暂鸣笛声。如果系统只在每次视频帧到达时处理音频，就可能延迟发现声音事件。对于实时音视频感知，

可以根据不同模态的到达时间更新系统状态。

一种概念性的表示形式为：$S_t = U(S_{t^-}, o_t)$，其中：

- S_t 表示当前系统维护的状态；
- S_{t^-} 表示本次观测到达之前的状态；
- o_t 表示当前到达的音频或视觉观测；
- U 表示状态更新函数。

这一形式强调，系统可以在获得新观测时更新已有信息，而不一定需要等待所有模态同时产生新数据。

对于音频与视频的联合处理，还需要明确时间戳、缓存策略和状态更新的因果约束。例如，如果系统在时刻 t 进行在线预测，就不能利用时间晚于 t 且尚未到达的未来观测。但在离线视频分析中，模型通常可以同时使用事件发生之前和之后的上下文。因此，离线音视频理解与在线连续感知具有不同的计算和评价条件。对于较长时间的视频，还可能需要维护历史信息，例如先前发生的事件、对象状态变化和说话人交互关系。

这种研究方向与长期视频理解、持续状态建模和多模态记忆机制具有一定联系。不过，并非所有音视频理解任务都需要显式维护复杂的持久状态。对于较短、离线处理的音视频片段，直接使用时序编码和跨模态注意力也可能满足任务要求。因此，是否引入长期状态维护，应根据任务中的时间依赖、延迟要求和计算成本进行判断。

音视频联合理解的评价与消融 音视频联合理解的评价不仅需要衡量最终任务成绩，还应分析不同模态的实际贡献。一种常见的实验方式是建立三组输入条件：V-only：仅使用视觉信息。模型只接收视频帧，不使用音频信息。A-only：仅使用音频信息。模型只接收音频，不使用视觉信息。AV：联合使用音频与视频。模型同时接收音频和视觉信息。通过比较三组结果，可以分析单模态与多模态输入之间的性能差异。例如：

$$\Delta_{\mathrm{AV}} = S_{\mathrm{AV}} - S_{\mathrm{V}} \tag{12.16}$$

其中，S_{AV} 表示音视频融合模型的成绩，S_{V} 表示视觉单模态成绩。如果融合模型表现更好，说明联合输入在当前实验条件下带来了性能收益。但这一结果本身并不能直接证明模型正确建立了音视频对应关系。例如，模型可能主要依赖声音类别完成任务，而没有真正利用视觉信息。因此，还可以进一步开展受控实验。音频替换实验。

将原始音频替换为其他视频的音频，观察模型预测是否发生变化。时间错位实验。人为改变音频和视频之间的时间对应关系，检查模型是否对跨模态同步敏感。模态缺失实验。分别移除音频或视频，观察模型是否能够利用剩余信息维持合理表现。模态噪声实验。在音频中加入背景噪声，或者对视频施加遮挡与画质退化，分析模型在不同条件下的稳健性。不同消融实验回答的问题并不相同。

例如，时间错位实验主要用于检查模型是否利用同步关系，而音频替换实验则可能同时改变声音类别、环境背景和语义内容。因此，实验设计应当尽量控制无关因素，避免将多个变化混合解释。此外，评价音视频模型时还需要关注实时性和计算成本。例

如，可以分别报告事件识别准确率、时间定位性能、推理延迟和计算开销。对于在线系统，还应考虑模型预测所需的观测等待时间。

多模态联合理解的可信性，不仅来自融合模型取得较高分数，还来自对单模态贡献、跨模态关联、时间同步和复杂场景泛化能力的系统验证。

12.5 证据缺失与感知结果校验

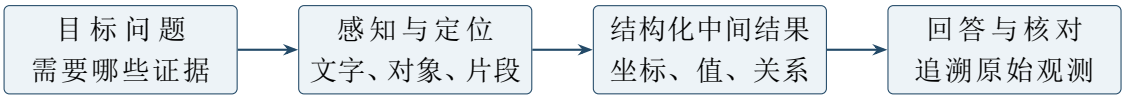


图 12.1 感知结果的可检查接口。中间结果可以帮助区分识别错误、关联错误和后续求解错误，但不能仅凭生成它就认定其正确。

12.5.1 细节遗漏与错误识别

多模态感知错误的基本表现 在多模态感知任务中，模型错误并不总是表现为完全无法识别输入内容。更常见的情况是，模型能够正确理解主要场景，却在局部细节、对象属性或语义关系方面产生偏差。例如，对于一张展示繁忙十字路口的图像，模型可能正确识别出汽车、行人和交通信号灯，但仍然可能出现以下错误：

- 遗漏远处的一名行人；
- 将红色交通信号灯识别为黄色；
- 将两辆相邻汽车错误地识别为同一辆；
- 将“行人位于车辆左侧”判断为“行人位于车辆右侧”；
- 描述图像中实际不存在的自行车。

这些错误在表面上都属于视觉理解不准确，但其形成机制可能存在明显差异。

例如，遗漏目标可能来自视觉分辨率不足，也可能来自注意力分配不合理。对象属性识别错误可能与局部视觉特征质量有关。关系判断错误则可能涉及实例区分、空间结构理解或语言推理。因此，在分析感知能力局限时，需要对错误进行更细致的分类。

细节遗漏与感知信息损失 细节遗漏（Detail Omission）是指输入中存在与任务相关的信息，但模型未能在最终预测或描述中正确保留这些信息。这种现象在高分辨率图像、复杂场景和长视频中较为常见。例如，对于一张包含大量商品的货架图像，模型可能识别出主要商品类别，却忽略角落中的某个小型标签。对于一份具有复杂表格结构的技术报告，模型可能提取出主要指标，却遗漏表格中的单位或脚注。

对于一段较长的视频，模型可能正确总结主要活动，但没有记录持续时间很短的关键事件。细节遗漏可能发生在不同处理阶段。输入预处理可能导致信息损失。例如，原始图像中的小目标在缩放后只占据少量像素，甚至失去可识别的纹理和边缘结构。视觉编码器可能没有充分保留某些局部差异。例如，面向图像级语义学习的编码器可能

更加关注整体对象类别，而对细粒度方向、纹理和局部数量差异不够敏感。

特征压缩或跨模态连接过程可能进一步减少可供语言模型使用的细节。即使细节已经进入模型内部表示，语言生成阶段也可能由于回答长度、任务偏好或注意力分配等因素忽略部分信息。因此，细节遗漏不能简单归因于“模型没有看清楚”。它可能是输入采集、特征提取、信息压缩、跨模态交互和答案生成共同作用的结果。

视觉编码器的表征局限 现代视觉语言模型通常使用预训练视觉编码器提取图像特征。这些编码器可能通过图文对比学习、视觉自监督学习或其他训练目标获得视觉表示。不同训练目标会影响模型所关注的信息。例如，图文对比学习通常要求模型建立整张图像与文字描述之间的语义联系。对于一张包含人物和自行车的图像，模型可能学习到“人物骑自行车”这一整体语义。

但如果任务要求区分自行车车轮的细微结构，或者判断两个对象之间非常具体的空间关系，整体语义表示未必包含足够细致的信息。Tong 等人在《Eyes Wide Shut? Exploring the Visual Shortcomings of Multimodal LLMs》中研究了多模态大语言模型的基础视觉感知缺陷。

该研究通过比较不同视觉编码器的表示，构建了 MMVP（Multimodal Visual Patterns）基准，用于分析模型对方向、数量、视角等基础视觉差异的识别能力。研究表明，部分基于 CLIP 视觉表示构建的多模态模型，可能在一些人类容易区分的视觉模式上出现明显错误。这说明视觉语言模型的性能瓶颈不一定全部来自语言模型或跨模态推理，也可能来自基础视觉表征本身。

为了改善这一问题，该研究还探索了将视觉自监督表示与视觉语言表示结合的方法。不过，不同视觉表征具有不同优势。强调局部视觉细节的表示未必能够直接替代具有较强语义对齐能力的表示。因此，更合理的研究问题是如何根据下游任务需要，利用具有互补性的视觉特征，而不是预设某一种视觉编码器在所有任务中都更优。

动态感知中的细节遗漏 细节遗漏不仅出现在静态图像中，也会影响视频和音频理解。在视频任务中，短暂事件可能因时间采样过于稀疏而被遗漏。例如，一段视频以较低频率采样关键帧，某个持续时间很短的按钮点击动作可能恰好发生在两个采样帧之间。此时，模型无法仅根据剩余静态帧可靠恢复完整动作过程。类似地，在音频理解中，短暂的警报声、玻璃破碎声或其他瞬态事件可能被时间聚合过程削弱。对于音视频联合理解，还可能出现声音与视觉事件错配的问题。

例如，模型识别出某段音频中的鸣笛声，却错误地将声音归因于画面中的某辆汽车。因此，动态感知中的信息损失还需要考虑时间分辨率、时间同步和历史信息保留等因素。对于具有严格实时要求的系统，提高感知精度通常还需要与处理延迟和计算成本共同考虑。

针对细节遗漏与错误识别的改进思路 改善细节遗漏和错误识别，需要根据具体错误来源选择方法。改善输入信息质量。对于小目标、文字和细粒度属性等任务，可以根据需要使用更高分辨率的图像或局部区域。对于视频和音频，可以调整时间采样策略，以尽量保留与任务相关的短时变化。增强局部视觉表示。可以使用多尺度特征、区域级

表示或具有细粒度感知能力的视觉编码器，改善局部信息表达。加强对象与属性的对应关系。

通过区域定位、指代表达监督或其他对象级语义学习方法，提高模型对具体实例及其属性的区分能力。增加困难样本和反事实数据。例如，构造仅在对象颜色、数量或位置上存在差异的图像，使模型需要利用这些细节才能正确回答问题。针对错误类型进行独立评价。分别统计漏检、误检、属性识别错误、关系错误以及视觉幻觉，而不是只观察整体问答准确率。不同改进方法之间可能存在成本与收益的权衡。

例如，增加高分辨率视觉输入可能改善小目标识别，但同时增加计算开销。因此，改进效果仍然需要通过公平的实验进行验证。

12.5.2 视觉证据利用不足

视觉证据利用的基本概念 视觉证据利用（Visual Evidence Utilization）是指模型在完成任务时，能够根据输入图像中的实际信息进行判断，并使最终输出与相关视觉内容保持一致。对于视觉问答任务，理想的模型应当根据图像与问题的联合条件生成答案：

$$P_\theta(y \mid I, q) \tag{12.17}$$

其中：

- I 表示输入图像；
- q 表示自然语言问题；
- y 表示模型回答。

但是，多模态大语言模型通常建立在具有丰富语言知识的预训练语言模型之上。

即使不提供图像，模型仍然可能根据问题文本和已有知识生成合理回答。例如，用户询问：“草地通常是什么颜色?”模型可以根据常识回答“绿色”。但如果实际图像展示的是一块被人工染成蓝色的草地，那么仅依赖语言知识就可能产生错误。因此，在视觉问答中，需要区分一般世界知识与当前图像提供的具体事实。二者可以相互补充，但不能直接相互替代。

语言先验与视觉捷径 语言先验（Language Prior）是指模型根据自然语言数据中学习到的规律，对某些答案形成较高的预测倾向。语言先验本身具有重要价值。例如，当用户询问某种工具的用途时，模型可能需要利用已有知识解释其功能。但如果任务要求判断当前图像中的具体事实，过度依赖语言先验就可能带来问题。例如，一张图像中有四个人，但问题写成：“图中的三个人分别在做什么?”

模型可能受到“三个人”这一错误前提的影响，直接描述三个人的活动，而没有首先检查实际人数。类似地，模型可能根据常见对象共现关系判断某种物品存在，却没有在图像中找到相关证据。这种利用数据统计规律替代实际感知的现象，可以视为多模态任务中的一种捷径学习（Shortcut Learning）。捷径学习不一定表现为模型完全忽略图像。模型可能同时利用部分视觉信息和语言先验，但在某些关键细节上受到语

言偏差影响。

因此，视觉证据利用不足通常需要通过受控实验分析，而不能仅凭某个错误回答直接判断。

利用输入消融分析视觉贡献 为了分析模型是否有效利用视觉输入，可以设置不同的输入条件。例如，对于相同问题，可以分别采用：完整图像输入。提供真实图像和问题。无图像输入。仅提供文本问题，或者在模型支持的条件下使用规范的无视觉输入模式。错误图像输入。使用与问题不相关的图像替换原始图像。视觉反事实输入。保留问题形式，但改变图像中决定答案的关键因素。例如，问题为：“图中有几辆汽车？”原始图像包含两辆汽车，反事实图像包含四辆汽车。

如果模型真正利用视觉信息，其回答应当随着图像中汽车数量的变化而发生相应改变。可以使用一个简单的视觉输入增益指标：

$$S_{\text{full}} S_{\text{text}} \tag{12.18}$$

其中，S_{full} 为完整输入下的任务成绩，S_{text} 为无图像条件下的成绩。当完整输入明显优于无图像输入时，说明视觉信息在当前任务中提供了额外的有效信息。不过，这一指标只是诊断工具。移除图像可能改变模型输入分布，使模型进入与正常多模态推理不同的工作状态。较大的性能差异并不意味着模型必然正确理解全部视觉细节。

部分任务本来就可以利用一般知识回答，不一定适合作为纯视觉能力评价。因此，更合理的做法是将输入消融与视觉反事实测试联合使用。

视觉注意力与视觉证据并不完全等价 在研究模型是否利用视觉信息时，有时会通过注意力权重分析模型关注了哪些区域。例如，模型可能对某个目标对象所在区域分配较高注意力权重。这可以为模型的信息处理方式提供一定线索。但注意力权重并不一定能够直接解释模型输出的因果来源。即使某个视觉 token 获得较高注意力，也不能保证模型最终答案主要由该 token 决定。同样，某些信息可能通过多层计算和间接路径影响预测，而不一定在单层注意力图中表现突出。

因此，不能仅凭注意力可视化就断定模型已经正确利用了某个视觉证据。相比之下，针对实际输入进行受控修改，并观察预测变化，可以提供更加直接的行为层面证据。例如，可以遮挡目标区域、替换局部对象或改变关键属性，检查模型回答是否相应变化。但遮挡本身也可能引入分布变化，因此仍需合理设计对照条件。

增强视觉证据利用的主要方法 改善视觉证据利用不足，可以从模型表示、训练数据和推理机制等方面展开。加强区域级视觉表示。通过对象检测、指代表达定位或其他区域建模方法，使模型能够更加明确地识别与问题相关的视觉对象。Ferret 是这类研究中的代表性方法之一，通过混合区域表示支持不同形状和粒度的视觉区域输入，并增强语言与区域之间的对应能力。引入视觉证据相关的训练数据。

例如，训练模型根据对象区域回答问题，或者要求模型将回答中的实体与图像区域进行关联。这种训练可以使模型更加关注对象、属性和关系的具体视觉依据。采用问题驱动的局部视觉搜索。对于复杂图像，模型可以根据问题选择与任务相关的区域，

进行进一步观察。例如，用户询问远处交通标志的内容时，系统可以优先分析交通标志所在区域，而不是始终依赖整张图像的压缩表示。V* 相关研究强调了视觉搜索机制在处理高分辨率和拥挤场景中的价值。

进行受控的多阶段感知与核验。例如，系统可以先识别目标区域，再提取相关属性，最后生成回答。这种流程有助于将部分感知与推理环节显式区分。但多阶段处理并不天然优于端到端模型。如果第一阶段定位错误，后续步骤可能在错误区域上继续处理，形成误差传播。因此，是否使用多阶段视觉分析，应根据任务复杂度、信息需求与额外计算成本进行判断。

12.5.3 感知结果的校验与结构化表达

为什么需要感知结果校验 多模态模型通常能够直接生成自然语言回答。例如，用户提供一张图像并询问："图中最右侧的汽车是什么颜色？" 模型可能直接输出："最右侧的汽车是黑色的。" 这种交互方式简单、自然，但也存在一个问题：用户无法仅根据回答本身判断模型是否正确识别了目标。如果图像中存在多辆汽车，模型可能选择了错误实例。也可能正确找到目标，却误判其颜色。因此，在需要较高可靠性的场景中，可以通过额外的感知结果校验机制，提高最终输出的可检查性。

感知校验（Perception Verification）主要是指利用输入数据、辅助模型、规则或其他可靠信息，对模型的感知结果进行检查。例如，可以检查：模型所描述的对象是否实际存在。预测的对象属性是否与对应区域一致。对象间的空间关系是否合理。识别出的文字或数字是否与原始图像一致。最终结论是否具有足够的感知证据支持。校验机制并不能保证所有输出一定正确。

它的主要价值在于发现部分错误、提供可追踪证据，并帮助系统在证据不足时采取更加适当的处理方式。

从直接回答到证据支持的回答 传统视觉问答系统通常直接输出最终答案：$(I, q) \to y$，其中，I 表示图像，q 表示问题，y 表示回答。为了提高可检查性，可以进一步要求模型输出与回答相关的证据信息：$(I, q) \to (y, E)$，其中，E 表示支持回答的感知证据。对于图像任务，证据可以是目标边界框、分割掩码或对应的局部图像区域。

对于文档任务，证据可以是页面编号、文本区域或表格单元格。对于视频任务，证据可以是包含相关事件的时间区间。对于音频任务，证据可以是声音事件对应的时间片段。例如，对于问题："图中穿红色衣服的人站在哪里？" 模型可以同时输出自然语言回答和目标人物的边界框。这样，用户不仅可以阅读文字，还能够检查预测区域是否与问题描述一致。但模型提供了证据位置，并不意味着该证据一定正确。例如，边界框可能覆盖了错误的人物。

因此，证据输出和证据正确性仍然需要分别评价。

置信度与不确定性表达 感知系统通常需要判断某个预测是否足够可靠。例如，目标检测器可能为每个预测目标输出置信分数。视觉问答模型也可能利用生成概率、候选答案分数或其他机制估计预测的可靠程度。但模型内部的置信分数不一定对应真实正确

概率。例如，模型对某个答案给出很高的生成概率，并不意味着其视觉判断具有同等程度的正确性。这类问题涉及模型置信度校准（Confidence Calibration）。

理想情况下，如果一组预测的置信度均接近 80%，那么这些预测中实际正确的比例也应当接近 80%。可以用下式概括校准目标：

$$P(Y = \hat{Y} \mid \hat{p} = p) \approx p \tag{12.19}$$

其中：

- $\hat{Y}$ 表示模型预测；
- Y 表示真实标签；
- $\hat{p}$ 表示模型给出的置信度。

这一表达主要适用于具有明确正确性标签的预测任务。

对于开放式自然语言生成，由于答案可能包含多个事实性主张，单一置信度未必能够完整反映整段回答的可靠程度。因此，可以根据应用需求进行更加细粒度的不确定性分析。例如，对对象类别、属性和空间关系分别给出可靠性评估。对于文档数值提取，可以根据 OCR 质量和字段对应关系进行检查。对于证据不足的任务，还可以采用拒答、请求更多输入或交由人工核验等策略。拒答策略本身也需要评价。如果模型过度拒答，可能降低任务完成率。

因此，应同时关注错误回答率、有效回答覆盖率和拒答行为。

利用规则与外部工具校验感知结果 对于部分具有明确规则的任务，可以使用外部工具或确定性计算辅助校验。例如，在文档和图表理解中，模型可以先提取结构化数值，再使用计算程序完成求和、平均值或增长率计算。这可以减少语言模型直接进行数值运算时产生的部分错误。例如，模型从表格中提取：2023 年销售额：120 万元。2024 年销售额：150 万元。随后，可以使用确定性的计算过程得到：

$$\frac{150 - 120}{120} \times 100\% = 25\% \tag{12.20}$$

在这种情况下，如果输入数值正确，计算工具可以避免部分算术错误。但如果模型在视觉读取阶段已经把 150 误识别为 180，那么后续计算即使完全正确，最终结果仍然错误。因此，外部工具主要能够验证或执行其适用范围内的操作，不能自动纠正所有上游感知误差。类似地，在结构化文档理解中，可以通过字段类型检查、数值范围检查和对象引用一致性检查，发现部分格式和逻辑错误。

例如，如果某个字段要求输入日期，却得到无法解析的字符串，系统可以报告数据格式异常。但字段格式正确，并不代表内容一定与原始文档一致。因此，规则校验与感知证据检查应当配合使用。

校验机制的评价方法 如果研究者提出一种新的感知校验方法，仅报告校验后的最终任务准确率通常还不够。还需要考虑校验系统是否能够稳定识别错误，以及其额外成本是否合理。例如，可以设置以下指标：例如，一个校验系统可能显著降低错误回答比

例，但代价是拒绝回答大量正常样本。这种情况下，仅报告错误率下降，可能无法完整反映系统的实际价值。因此，可以绘制回答覆盖率与错误率之间的关系，分析模型在不同可靠性要求下的表现。

对于包含局部视觉证据的系统，还应分别评价答案正确性和证据定位准确率。例如，模型可能输出正确答案，却提供错误的证据框。这种情况下，不能简单认为证据校验已经通过。此外，如果校验系统使用额外的目标检测器、OCR 工具或多次模型推理，还需要明确报告相应的资源成本。

习题与讨论

1. 为类别判断、区域定位和结构化表格抽取分别定义输出、监督与错误类型。说明为何不能共用一个准确率。
2. 一个文档答案数值正确却来自错误行。设计既检查值又检查来源的评价。
3. 语言条件定位中两个对象都满足描述。应如何区分歧义、定位错误和证据不足？
4. 设计原图、局部裁剪、错配图像和纯文本四组输入，诊断模型对局部视觉证据的依赖。

第十三章　多模态推理与证据利用

多模态推理需要把可观察证据转换为约束，再完成关系、数量、时间或空间求解。错误可以发生在读图、绑定变量、计算和验证任一环节。研究推理时，应保留这些环节的可区分性，不能只把最终答案与一段解释作为全部观察。

学习目标　建立问题所需的证据链；区分观测关系与因果结论；设计可检查的中间表示；在固定预算下评价额外推理计算。

13.1　问题分解与证据依赖

将任务拆成证据读取 e 与条件求解 y，可以分别检查 e 是否忠于观测，以及在给定正确 e 后求解是否正确。真实系统不必使用两个独立模型，但这种诊断能避免把感知失败误归因于推理机制。

13.1.1　多步推理与复杂任务求解

从直接回答到多步推理　对于较为简单的视觉问题，模型可能通过一次信息识别直接得到答案。例如：“图中的汽车是什么颜色？”如果图像中只有一辆汽车，模型可能只需要定位汽车并识别颜色。但对于更加复杂的问题，单次对象识别通常不足以得到最终结果。例如，用户询问：“图中所有红色汽车的数量是否多于蓝色汽车？”模型需要完成多个操作。首先识别图像中的汽车。然后判断每辆汽车的颜色。接着分别统计红色和蓝色汽车的数量。最后比较两个数量。

因此，复杂推理往往具有多个相互依赖的中间步骤。这种需要连续执行多个信息处理操作的任务，可以称为多步推理（Multi-step Reasoning）。多步推理的关键并不只是生成更长的文字，而是将不同的感知和逻辑操作按照适当关系组织起来。例如，模型可以先获取视觉事实，再进行数量统计，最后完成比较。如果中间某个步骤出现错误，后续结论就可能受到影响。

多步推理的任务分解　任务分解（Task Decomposition）是复杂问题求解中的常见思路。其基本思想是，将原始问题拆分为多个更容易处理的子问题。例如，对于问题：“图中穿红色外套的人旁边有几个背包？”可以分解为：第一步，识别图像中的人物。第二步，根据服装颜色找到穿红色外套的人。第三步，确定该人物所在区域。第四步，识别该人物附近的背包。第五步，统计满足条件的背包数量。从形式上，可以将原始问题 Q 表示为一组子问题：

$$Q \rightarrow q_1, q_2, \ldots, q_n \tag{13.1}$$

每个子问题可能依赖此前步骤得到的中间结果。例如：$z_i = f_i(z_{<i}, X, Q)$，其中：

- z_i 表示第 i 步的中间结果；
- f_i 表示对应的处理操作；
- $z_{<i}$ 表示此前已经获得的中间信息；
- X 表示原始多模态输入。

最终结果可以表示为：$Y = g(z_1, z_2, \ldots, z_n)$，

这种形式说明，复杂推理可以通过若干相互关联的操作构成。不过，实际模型并不一定显式输出全部子问题。在端到端神经网络中，部分计算可能通过隐式表示完成。因此，任务分解更多是一种分析和设计推理系统的方法，而不是所有多模态模型都必须采用的计算过程。

多模态推理中的数值与符号操作 复杂多模态任务还经常涉及数值计算和符号操作。例如，一张柱状图展示两种产品在不同年份的销售额。用户询问：“哪种产品在 2023 年到 2024 年之间的销售额增长率更高?”模型首先需要从图表中提取对应数值。假设提取的数据为：随后，计算产品 A 的增长率：

$$r_A = \frac{150 - 120}{120} \times 100\% = 25\% \tag{13.2}$$

产品 B 的增长率为：

$$r_B = \frac{140 - 100}{100} \times 100\% = 40\% \tag{13.3}$$

最后进行比较：$r_B > r_A$，因此，正确答案是产品 B。这一过程涉及视觉数据读取、数值提取、公式选择和比较判断。其中任何一个环节出现错误，都可能影响最终结果。MathVista 是多模态数学推理评价中的代表性基准。该基准包含数学相关的视觉问题，涉及图表、几何图形和其他视觉信息，强调模型对视觉内容和数学推理能力的联合使用。

这类任务说明，复杂多模态推理不仅需要正确识别符号，还需要正确理解符号之间的关系，并按照任务要求执行计算。对于精确数值运算，可以在适当情况下结合外部计算工具，减少语言模型直接进行计算时可能产生的错误。但外部工具无法自动纠正错误的视觉输入。如果模型将图表中的 150 错读为 180，那么后续计算即使完全正确，最终答案仍然可能错误。因此，工具使用需要与感知结果校验相结合。

复杂任务求解中的任务规划与信息选择 当任务包含多个步骤时，模型还需要决定以什么顺序执行这些步骤。例如，用户询问：“找出图中距离红色汽车最近、并且没有佩戴头盔的骑行者。”一种处理方式是：先识别所有骑行者。然后筛选未佩戴头盔的人物。最后根据空间位置确定与红色汽车最近的目标。另一种方式则是先找到红色汽车附近的骑行者，再检查其头盔状态。两种方式在逻辑上可能都能够求解问题，但实际计算成本和错误传播路径可能不同。

因此，复杂任务求解不仅涉及正确执行操作，还涉及如何组织操作顺序。当某个

步骤产生的信息不足以得到结论时，模型还可能需要进一步观察相关区域或调用适当的工具。例如，对于高分辨率文档中的细小文字，可以先定位相关表格，再对局部区域进行高分辨率读取。这种按需获取信息的方式，有助于将计算资源集中到与任务相关的部分。不过，复杂任务不一定都需要显式规划。对于较简单的问题，直接端到端生成答案可能更加高效。

因此，是否采用任务分解、工具调用或多阶段推理，需要根据问题难度和系统目标进行选择。

多步推理的评价 多模态推理的评价通常不仅关注最终答案，还需要分析不同能力环节。常见的代表性基准包括：其中，MMMU[29] 关注需要专业知识和推理能力的跨学科多模态任务。MM-Vet 则强调多种视觉语言能力在复杂任务中的综合使用，而不仅仅评价某一种单独能力。不过，这些基准在题目来源、答案格式和评价方式上存在差异，不能简单地将不同基准的准确率直接进行比较。对于研究者而言，还可以进一步从几个方面分析复杂推理能力。

检查模型是否正确读取原始感知信息。检查模型是否能够完成必要的关系判断和计算。检查推理结果是否与问题条件一致。检查模型是否能够在新的对象组合或数据分布下保持性能。对于具有中间标注的任务，可以进一步评价关键步骤。对于没有标准中间过程的开放式任务，则需要避免将某一条参考解释当作唯一正确的推理路径。多步推理评价应当关注最终任务成功率，同时尽可能提供有助于解释错误来源的诊断信息。

13.1.2 语言推理与视觉证据之间的关系

语言推理为什么需要视觉证据 在多模态大语言模型中，语言模型通常负责理解问题、组织推理和生成最终回答。由于语言模型已经在大规模文本数据上学习了丰富的知识和表达规律，因此即使视觉信息不充分，也可能生成看似合理的解释。例如，用户提供一张显示道路和汽车的图像，并询问：“为什么车辆正在减速？”模型可能回答：“因为前方交通信号灯变成了红色。”如果图像中确实能够观察到与车辆通行方向相关的红色信号灯，并且车辆运动信息支持减速判断，那么这一解释可能具有一定依据。

但如果图像中没有交通信号灯，或者无法根据单张图像判断车辆速度变化，那么这一回答就可能属于缺乏充分视觉证据的推测。因此，在多模态推理中，需要明确区分语言上的合理性与感知证据上的充分性。语言模型能够根据常识提出可能的解释，但不能将所有合理解释直接视为当前场景中的真实事实。特别是涉及对象状态、空间关系和事件原因的问题时，推理结论应当受到实际输入信息的约束。

视觉依据与语言先验 视觉语言模型的回答通常同时受到视觉输入和语言先验影响。对于问题 Q 和图像 I，模型预测可以表示为：

$$P_\theta(Y \mid I, Q) \tag{13.4}$$

但是，如果某些问题具有较强的语言统计规律，模型可能在缺少视觉信息的情况

下仍然给出较高概率的答案。例如：“天空通常是什么颜色?”“人们通常用什么工具切水果?”这些问题可以部分依靠常识回答。但如果用户明确询问当前图像中的实际颜色或工具，就需要根据视觉证据进行判断。例如，一幅艺术作品中的天空可能呈现绿色。如果模型仍然根据常识回答“蓝色”，就没有正确利用当前输入。因此，视觉证据与语言先验之间的合理关系应当是：

对于需要当前观测事实的问题，应以观测证据为主要依据。对于需要背景知识的问题，可以在观测事实基础上引入相关知识。对于证据不足的问题，应当区分确定事实与可能解释。这并不意味着语言知识应该被完全排除。相反，很多多模态推理任务必须利用语言知识才能完成。关键在于，背景知识应当帮助解释感知事实，而不是替代或歪曲实际输入。

视觉感知正确不等于推理正确 在多模态任务中，模型可能正确识别视觉内容，但仍然得到错误结论。例如，一张图表中包含以下数据：2023 年销售额为 100 万元。2024 年销售额为 120 万元。模型能够正确读取这两个数值。但如果用户询问：“2024 年比 2023 年增长了多少?”模型可能错误地回答“增长了 120%”。实际上，正确增长率为：

$$\frac{120-100}{100}\times 100\% = 20\% \tag{13.5}$$

这种情况说明，感知结果正确并不保证后续计算正确。反过来，模型也可能使用错误的感知信息，却偶然得到正确的最终答案。例如，在比较两个对象数量时，模型可能错误统计了两个类别的具体数量，但最终仍然碰巧判断出正确的大小关系。因此，最终答案正确并不能直接证明中间感知和推理过程全部正确。这也是为什么复杂多模态推理需要关注过程诊断和证据校验。

视觉推理中的证据链 为了使复杂推理更加可检查，可以将任务相关的视觉事实与推理操作组织为证据链。例如，用户询问：“图中站在自行车旁边的人是否戴着头盔?”可以将任务处理过程概括为：首先识别图像中的自行车。然后确定与自行车具有指定空间关系的人物。接着观察该人物的头部区域。最后判断其是否佩戴头盔。在这一过程中，每个步骤都依赖不同的视觉证据。例如，自行车的位置可以由目标检测结果支持。人物与自行车的关系可以由对象位置和场景关系判断支持。

是否佩戴头盔则需要观察人物头部区域。因此，可以将推理结果表示为：$Y = R(E, Q, K)$，其中：

- E 表示从输入中获得的证据；
- Q 表示问题；
- K 表示所需的背景知识；
- R 表示推理操作。

这一形式强调，推理应当以有效证据为基础。

不过，证据链不一定需要在所有任务中以完整自然语言形式显式生成。某些模型可能通过内部特征计算完成任务。对于需要高可靠性或可追踪性的应用，可以额外输

出相关证据位置和关键判断依据。但模型生成了一条看似合理的证据链，并不意味着其中每个步骤都已经得到验证。因此，证据链与真实证据之间仍然需要建立可检查的对应关系。

面向可靠多模态推理的基本原则 综合上述讨论，可以归纳出若干具有普遍意义的研究原则。推理应当建立在与任务相关的感知信息之上。对于需要观察图像才能回答的问题，模型不应主要依赖语言先验替代实际视觉判断。模型需要区分观测事实、用户假设和背景知识。例如，图像中显示的交通状态属于观测事实，而“假设车辆继续向前行驶”属于任务条件，两者不能混淆。复杂问题可以通过任务分解、局部观察和外部工具等方式求解，但这些机制本身也可能产生错误。

因此，应当根据任务难度和资源约束选择合理方法，而不是预设推理步骤越多越好。模型的最终回答应当尽可能与关键感知证据保持一致。对于证据不足的情况，模型应当区分确定事实和合理推测。对于高可靠性任务，还可以提供证据位置、结构化中间结果和必要的校验机制。多模态推理的目标并不是单纯生成更加复杂的自然语言解释，而是使模型能够在不同模态提供的信息之间建立正确联系，并通过合理的推理操作得到与实际证据一致的结论。

13.2 关系、数量与图表求解

算例：图表求解中的两类错误

若问题要求比较两个柱值之差，先记录柱对应的类别、单位、读数和图例，再做减法。读错单位属于证据解释错误；值正确而减法错误属于计算错误。可分别使用人工核实的结构化读数与原始图像作为输入，比较差异。这个对照衡量求解与感知的作用，不把人工读数条件下的分数冒充完整视觉能力。

13.2.1 物体关系与场景推理

从对象识别到关系推理 对象识别是视觉理解的重要基础，但现实场景的含义往往不只由对象类别决定，还取决于对象之间的联系。例如，两张图像可能同时包含一个人、一辆自行车和一条道路。在第一张图像中，人物正在骑自行车。在第二张图像中，人物正在推自行车。两张图像包含相似的对象，但其动作关系不同，因此表达了不同的场景语义。如果模型只识别出“人物”和“自行车”，就无法完整区分这两种情况。

因此，关系推理（Relational Reasoning）要求模型在对象识别基础上，进一步理解对象之间的空间、动作、组成和语义关系。例如：“人物正在骑自行车。”“杯子放在桌子上。”“汽车位于行人右侧。”“一个人正在牵着狗。”这些关系通常包含两个或多个实体，以及实体之间的联系。对于简单的二元关系，可以采用三元组表示：$r = (s, p, o)$，其中：

- s 表示关系主体（Subject）；

- p 表示关系谓词（Predicate）；
- o 表示关系客体（Object）。

例如：

$$(\text{人物}, \text{骑}, \text{自行车}) \tag{13.6}$$

与单独的对象类别相比，这种表示能够更加明确地表达场景中的交互关系。部分关系具有方向性。例如：“人物追赶狗”与“狗追赶人物”并不等价。如果模型正确识别了两个对象，却错误判断关系方向，仍然会产生语义理解错误。

对象、属性和关系的联合表示 在复杂视觉场景中，仅采用关系三元组可能不足以表达全部信息。例如，一张办公室图像中可能存在多个人物、电脑和桌子。用户询问：“坐在左侧、穿蓝色衣服的人正在使用哪台电脑？”模型需要综合利用人物属性、空间位置和对象交互关系。因此，可以将每个对象表示为包含类别、属性和空间信息的结构：$o_i = (c_i, a_i, b_i)$，其中：

- c_i 表示对象类别；
- a_i 表示对象属性；
- b_i 表示对象的空间位置。

多个对象之间还可能存在关系集合：$R = r_1, r_2, \ldots, r_M$，

在此基础上，可以组织得到具有结构化语义的场景表示。例如：对象 1：人物，穿蓝色衣服，位于图像左侧。对象 2：人物，穿黑色衣服，位于图像右侧。对象 3：电脑，位于左侧桌面。关系：对象 1 正在使用对象 3。通过对象、属性与关系的组合，模型能够根据语言中的多个限制条件确定相关目标。

Visual Genome 是这一方向的重要代表性数据集，提供了密集的对象、属性、关系和区域描述标注，为研究对象级视觉语义和场景结构提供了基础。

场景图与结构化推理 场景图（Scene Graph）是一种表达视觉场景中实体及其关系的结构化方式。通常可以表示为：$G = (V, E)$，其中：

- V 表示对象节点集合；
- E 表示对象之间的关系边集合。

节点可以包含类别、属性和区域位置等信息。

边则表示对象之间的空间、动作、交互或其他语义关系。例如，一张图像中包含人物、自行车和道路，可以构建以下关系：人物—骑行—自行车。自行车—位于—道路。人物—穿着—红色外套。当用户询问：“骑着自行车的人穿什么颜色的外套？”模型可以先根据“骑行”关系找到人物，再读取该人物对应的服装属性。在这种情况下，推理可以表示为对场景图中的节点和关系进行查询。例如：

$$\text{FindSubject}(G, \text{骑行}, \text{自行车}) \tag{13.7}$$

随后：

$$y = \text{GetAttribute}(v_{\text{person}}, \text{外套颜色}) \tag{13.8}$$

这一表示说明，复杂视觉问答可以通过多个结构化操作完成。不过，实际场景图未必能够包含图像中的所有信息。例如，图像中的阴影、姿态细节和复杂遮挡关系，可能难以通过少量节点与关系边完整表达。此外，如果场景图构建阶段已经遗漏对象或错误识别关系，后续推理也可能受到影响。因此，场景图为视觉推理提供了一种可解释的结构表示，但并不意味着它能够自动消除视觉感知错误。

从简单关系到多跳场景推理 当问题涉及多个连续关系时，模型需要开展多跳场景推理（Multi-hop Scene Reasoning）。例如：“站在红色汽车旁边、手里拿着雨伞的人穿什么颜色的鞋子?”这一问题需要依次完成多个操作。识别红色汽车。找到位于汽车附近的人物。然后，根据“拿着雨伞”这一条件确定相关人物。观察该人物的鞋子颜色。这种问题不能仅通过单一对象检测或属性分类直接完成。它需要在多个对象和关系之间进行连续筛选。

GQA 是真实图像组合式视觉推理领域的代表性数据集。该数据集基于场景图构建视觉问答任务，并使用结构化程序表示问题中的语义操作。例如，问题可以包含对象筛选、属性查询、关系判断、数量比较等操作。GQA 不仅提供整体问答准确率，还关注回答一致性、合理性等评价维度。这类研究说明，多模态模型需要在复杂场景中正确组合多个视觉操作，而不能仅依赖对象名称或语言统计规律完成问答。

关系推理中的视觉证据约束 在实际视觉推理中，关系判断需要与原始视觉证据保持一致。例如，模型可能根据图像判断：“人物正在使用电脑。”如果图像中人物坐在电脑前方，这一判断可能具有一定依据。但如果仅能观察到人物与电脑同时出现在同一场景中，就不能直接确认人物正在操作电脑。因此，需要区分空间邻近与实际交互关系。类似地，一张道路图像可能显示汽车位于行人后方，但仅凭单帧图像，通常无法确定汽车是否正在追赶行人。这种情况说明，视觉关系推理可能受到观测条件限制。

对于涉及动作和因果关系的问题，还需要结合时间信息、对象状态变化或其他证据进行判断。因此，关系推理应当区分：直接可观察的关系，例如两个对象在图像平面中的相对位置。需要额外视觉线索的关系，例如人物是否正在操作某个物体。需要时间或背景知识支持的关系，例如某个动作是否导致了后续事件。这种区分有助于减少模型将视觉相关性直接解释为确定因果关系的情况。

关系推理的主要错误类型 物体关系与场景推理可能出现多种错误。首先是对象识别错误。例如，模型将摩托车识别为自行车。其次是属性绑定错误。例如，图像中存在红色汽车和蓝色汽车，但模型将颜色分配给了错误实例。再次是关系方向错误。例如，将“人物位于汽车左侧”判断为“汽车位于人物左侧”，但没有正确处理观察视角。还有关系对象混淆。例如，图像中有两名人物，模型将某个动作错误地归属于另一名人物。最后是关系过度推断。

例如，模型根据两个对象在空间上接近，就断定它们之间具有实际交互或因果关

系。因此，评价场景推理时，需要分别检查对象、属性、关系类型和关系方向是否正确。对于涉及多个关系的复杂问题，还可以进一步分析错误发生在哪个中间步骤。

13.2.2 数量、属性与空间关系推断

数量推断的基本任务 数量推断（Numerical or Counting Reasoning）是视觉推理中常见的任务类型。例如：“图中有几辆汽车？”“有多少个人穿着蓝色衣服？”“左侧桌子上的杯子是否多于右侧？”这些问题涉及对象识别、实例区分和数量计算。对于简单计数任务，模型通常需要识别图像中的所有目标实例。假设某类目标的实例集合为：$O_c = o_1, o_2, \ldots, o_N$，则该类对象的数量为：$n_c = |O_c|$，

其中，$|\cdot|$ 表示集合中元素的数量。例如，如果图像中存在三辆汽车，则：$n_{\text{car}} = 3$，但在复杂场景中，数量统计并不只是简单计算目标框的数量。例如，密集人群可能存在严重遮挡，相邻对象可能被错误合并，也可能由于重复检测而造成计数偏高。因此，数量推断需要建立在合理的实例识别结果之上。

带有条件限制的数量推断 相比简单计数，复杂数量问题通常包含多个筛选条件。例如：“图中有多少名穿黑色衣服、并且站在汽车旁边的人？”这一问题需要同时判断对象类别、服装属性和空间关系。可以定义指示函数：

$$\begin{cases} 1, & P(o_i) \text{ 成立} \\ 0, & P(o_i) \text{ 不成立} \end{cases} \tag{13.9}$$

其中，$P(o_i)$ 表示对象 o_i 是否满足指定条件。则满足条件的对象数量可以表示为：

$$\sum_{i=1}^{N} \mathbb{I}[P(o_i)] \tag{13.10}$$

例如，条件 P 可以定义为：“对象是人物，并且穿黑色衣服，同时位于汽车旁边。”模型首先需要判断各对象是否满足全部条件，然后再进行计数。因此，复杂数量推断通常涉及关系筛选和逻辑运算。TallyQA 是视觉复杂计数研究中的代表性数据集。该研究区分相对简单的数量问题和需要属性、关系等多种条件共同判断的复杂计数问题。其研究说明，数量推断并不总是目标检测的直接延伸。

对于包含多个约束条件的问题，模型还需要具备对象关系理解和组合推理能力。

属性绑定与实例区分 在复杂场景中，属性推断的一个重要问题是正确区分不同对象实例。例如，一张图像包含两个人：人物 A 穿红色外套。人物 B 穿蓝色外套。如果模型能够识别红色和蓝色两种颜色，却无法确定对应人物，就可能发生属性绑定错误。这种错误在存在多个相似对象的场景中较为常见。例如，多个杯子具有相似形状但颜色不同。多个车辆具有相似外观但位置不同。多个行人具有相似服装但动作不同。因此，细粒度属性推断需要建立对象实例与属性之间的明确对应关系。

可以将对象与属性的关联表示为：

$$A(o_i) = a_{i1}, a_{i2}, \ldots, a_{ik} \tag{13.11}$$

其中，$A(o_i)$ 表示对象 o_i 对应的属性集合。例如：

$$A(o_1) = \text{汽车}, \text{红色}, \text{左侧} \tag{13.12}$$

这一表示能够帮助研究者明确：属性并不是独立存在的标签，而是需要与具体对象实例建立关联。对于多模态大语言模型，还可以通过指代表达定位和区域级视觉问答等方法，检查属性是否正确绑定到相应区域。

二维空间关系推断 空间关系推断（Spatial Relation Reasoning）是视觉推理的重要组成部分。在二维图像中，常见关系包括左侧、右侧、上方、下方、相邻、包含和重叠等。例如：“红色汽车是否位于蓝色汽车左侧？”对于具有明确边界框的对象，可以根据对象中心坐标进行部分空间关系判断。假设对象 i 的边界框为：

$$b_i = (x_{\min}, y_{\min}, x_{\max}, y_{\max}) \tag{13.13}$$

其中心点可以表示为：

$$x_i^c = \frac{x_{\min} + x_{\max}}{2} \tag{13.14}$$

$$y_i^c = \frac{y_{\min} + y_{\max}}{2} \tag{13.15}$$

在采用横坐标向右增大的图像坐标系中，如果：$x_A^c < x_B^c$，则可以认为对象 A 的中心位于对象 B 中心的左侧。不过，这只是二维空间关系的一种简单判定方法。当两个对象具有较大空间重叠，或者对象形状不规则时，仅比较中心点可能不足以描述实际关系。例如，一个较大的物体可能同时延伸到另一个物体的左侧和右侧。因此，对于复杂对象关系，可以结合边界框、分割掩码或更加细致的区域几何信息进行判断。

比较、否定与逻辑量词 视觉推理还经常涉及比较、否定和逻辑量词。例如：“是否所有人都戴着帽子？”“是否至少有一辆汽车是红色的？”“有没有人没有穿外套？”这些问题不仅需要识别对象和属性，还需要正确处理逻辑条件。例如：“所有人都戴着帽子。”可以表示为：$\forall x \in P, \quad H(x) = 1$，其中，P 表示图像中被纳入判断的人物集合，H(x) 表示人物 x 是否戴帽子。类似地：“至少有一个人戴着帽子。”

可以表示为：$\exists x \in P, \quad H(x) = 1$“所有”和“至少一个”具有不同的逻辑含义。对于“所有人都戴着帽子”，只要存在一名没有戴帽子的人，就应当判断该命题不成立。对于“至少有一个人戴着帽子”，只需要找到一个满足条件的实例。如果人物头部被严重遮挡，模型可能无法判断是否戴帽子，此时也需要区分“没有戴帽子”与“无法确定是否戴帽子”。因此，否定关系与不确定性不是同一概念。

这类问题能够检验模型是否真正理解逻辑条件，而不只是根据关键词生成答案。

13.2.3 图表、文档与视觉数学问题

视觉数学推理的基本概念 视觉数学推理（Visual Mathematical Reasoning）是指模型根据图像、图表、文档或几何图形中的信息，完成数值计算、数学关系判断或其他数学问题求解。与纯文本数学推理相比，视觉数学推理需要首先从视觉内容中获取有效信息。例如，对于一张统计图表，模型需要识别坐标轴、图例和数据。对于一份财务报表，模型需要理解行列结构和指标含义。对于一道几何题，模型需要识别点、线、角度和图形中的已知条件。

因此，视觉数学问题通常包含两个相互关联的方面：视觉信息提取，即从图形中读取与问题有关的事实和符号。数学推理与计算，即根据这些事实执行公式运算、比较或逻辑判断。可以将这一过程概括为：$D = E(I, Q)$，$Y = R(D, Q, K)$，其中：

- I 表示视觉输入；
- Q 表示问题；
- D 表示提取出的相关数据或符号信息；
- K 表示必要的数学规则；
- R 表示推理与计算过程；
- Y 表示最终答案。

这种表示有助于区分视觉读取错误和数学计算错误。

例如，模型可能正确掌握增长率公式，却错误读取图表中的某个数值，最终仍然得到错误结果。

图表中的数据读取与计算 图表是视觉数学推理中的典型输入形式。例如，一张柱状图可能展示某公司不同年份的销售额。用户询问：“2024 年销售额比 2023 年增长了多少？”模型需要首先确定横轴年份与对应柱形。随后根据纵轴刻度读取数值。假设：2023 年销售额为 120 万元。2024 年销售额为 150 万元。则绝对增长量为：$\Delta S = 150 - 120 = 30$ 即增长 30 万元。如果问题要求增长率，则需要计算：

$$r = \frac{150 - 120}{120} \times 100\% = 25\% \tag{13.16}$$

因此，“增长多少”和“增长百分之多少”对应不同的计算目标。类似地，如果图表展示的是百分率指标，那么两个百分率之间的直接差值通常应以“百分点”表示，而不应与相对增长率混淆。例如，准确率从 80% 提升到 85%，表示提高 5 个百分点，而相对提升比例为：

$$\frac{85 - 80}{80} \times 100\% = 6.25\% \tag{13.17}$$

因此，图表推理不仅需要读取数值，还需要理解单位、指标定义和问题所要求的运算方式。

几何图形与视觉数学推理 几何问题是视觉数学推理中的另一种典型任务。例如，一张图像包含三角形及其角度标注，用户要求计算未知角度。模型需要首先识别图形中的点、线和角度信息。然后根据数学规则完成计算。例如，假设题目明确给出一个三角形的两个内角分别为：

$$\alpha = 50^\circ, \qquad \beta = 60^\circ \tag{13.18}$$

根据三角形内角和：$\alpha + \beta + \gamma = 180^\circ$ 可以得到：

$$\gamma = 180^\circ - 50^\circ - 60^\circ = 70^\circ \tag{13.19}$$

这一计算过程本身并不复杂。但如果图像中包含多个相交线段、辅助线或相似角度标注，模型首先需要正确判断哪些线段组成目标三角形，以及不同数值对应哪个角。因此，几何推理通常同时涉及图形结构识别和数学规则应用。此外，几何题中的图形不一定严格按比例绘制。例如，一条线段在图像中看起来更长，并不意味着题目已经给出其真实长度更大。同样，一个角在示意图中看起来接近直角，也不能据此认为它一定等于 90°。

因此，模型应当优先利用题目中明确给出的条件和合法的几何关系，而不是直接根据视觉外观进行未经验证的精确测量。

视觉数学推理的代表性基准 MathVista 是视觉数学推理研究中的重要基准。该基准覆盖图表、几何、数学相关图形及其他需要视觉信息与数学推理共同参与的问题。它强调模型需要同时具备细粒度视觉理解和数学操作能力。与普通视觉问答不同，数学推理任务对数值、单位和逻辑条件具有较高要求。例如，模型可能需要从几何图中提取角度，或者根据统计图中的数值计算比例。因此，视觉数学推理的评价通常需要关注以下方面。模型是否正确提取输入中的数学符号和数值。

模型是否正确理解图形结构与数据对应关系。模型是否选择了合理的计算规则。模型是否按照要求输出正确答案。在复杂问题中，单纯报告最终准确率可能不足以揭示模型瓶颈。例如，如果模型错误读取某个数字，最终计算错误不能简单归因于数学推理能力不足。因此，视觉数学推理研究需要尽可能将感知误差和推理误差区分开来。

视觉数学推理中的工具辅助 对于复杂数学任务，可以通过结构化数据和外部计算工具提高部分计算环节的可靠性。例如，模型首先从图表中提取数据，再使用计算程序求和、计算平均值或执行比例运算。对于表格分析，可以通过明确的列名和字段类型进行数值处理。对于某些几何问题，还可以利用符号计算工具验证代数表达式或方程求解结果。这种方法的主要优势在于，将部分确定性计算交给专门工具执行。

例如，如果需要计算多个数值的平均值，使用计算工具可以减少人工式逐步生成带来的部分算术错误。但工具的可靠性通常建立在输入数据正确的前提下。如果模型错误读取图表中的数值，那么工具仍可能根据错误输入得到错误结果。此外，模型也可能调用了不适当的计算公式。因此，工具辅助推理仍然需要检查数据来源、公式选择和结果解释。在实际研究中，可以将完整视觉推理与使用人工校验数据的推理结果

进行比较。

如果提供正确数据后模型成绩明显提高，则说明视觉读取或数据结构恢复可能是重要瓶颈。如果提供正确数据后仍然无法完成计算，则需要进一步分析推理或工具使用环节。

13.2.4 证据定位、答案校验与错误分析

为什么视觉推理需要证据验证 视觉推理的最终目标是根据输入信息得到正确结论。但对于复杂多模态模型，最终答案正确并不能充分说明其推理过程可靠。例如，模型可能错误识别了图像中的对象数量，却通过偶然的计算结果得到正确答案。也可能正确读取数值，但根据错误的公式形成结论。此外，生成式模型还可能根据语言先验输出看似合理但缺乏视觉依据的解释。因此，视觉推理的评价不仅需要检查答案是否正确，还应关注答案所依赖的关键事实是否得到视觉证据支持。

证据验证（Evidence Verification）主要涉及两个问题。模型是否找到了与问题相关的视觉信息。这些信息是否能够支持最终答案。例如，用户询问："图中左侧汽车的颜色是什么？"模型可以输出"红色"，同时提供该汽车对应的边界框。如果边界框正确覆盖左侧汽车，并且该区域中的汽车确实为红色，就能为回答提供直接的视觉依据。但如果模型提供的边界框指向另一辆汽车，即使最终答案碰巧正确，也不能认为证据定位已经正确。

因此，答案正确性和证据正确性应当分别评价。

不同视觉任务中的证据形式 不同任务所需的证据具有不同形式。对于自然图像，证据可以是对象边界框、分割掩码或局部区域。对于图表，证据可以是坐标轴、数据点、柱形、图例和相关数值标签。对于文档，证据可以是页码、文本区域、表格单元格或引用段落。对于几何问题，证据可以是相关线段、角度标注和题目中给出的约束条件。因此，可以将视觉推理输出概括为：$A = (Y, E)$，其中：

- Y 表示最终答案；
- E 表示与答案相关的证据集合。

在更复杂的任务中，还可以进一步加入推理操作：$A = (Y, E, R)$，

其中，R 表示用于组织证据和形成结论的操作信息。例如，对于图表中的增长率问题：证据可以包括两个年份的销售额。推理操作可以是增长率计算公式。最终答案则是对应的百分比结果。这种表示有助于将感知事实与计算过程分开检查。但并不是所有任务都具有唯一、完整且易于标注的证据集合。例如，一些开放式场景解释可能涉及多个相关区域，也可能依赖背景知识。因此，证据表示需要根据具体任务设计，不能要求所有问题都对应单一矩形框。

推理结果的一致性检查 除了检查单个答案，还可以通过一组具有逻辑关系的问题验证模型输出是否一致。例如，对于同一张图像，分别询问："红色汽车是否位于蓝色汽车左侧？""蓝色汽车是否位于红色汽车右侧？"在明确采用同一观察视角、且关系成立的

情况下，这两个问题应当得到相互一致的回答。类似地，可以询问：“图中有多少辆红色汽车?”“图中有多少辆蓝色汽车?”“图中一共有多少辆红色或蓝色汽车?”如果两类颜色互斥，并且统计范围一致，则相关数量应满足：

$$N_{\text{red}} + N_{\text{blue}} \tag{13.20}$$

如果模型分别给出：红色汽车 3 辆。蓝色汽车 2 辆。红色或蓝色汽车总计 4 辆。那么在上述条件下，三个结果之间存在数量不一致。这种检查能够发现部分内部逻辑错误。GQA 的评价设计也强调了视觉问答中答案一致性的重要性。不过，逻辑一致性并不等于视觉事实正确。例如，模型可能连续给出相互一致但全部错误的答案。因此，一致性检查应当作为证据验证的补充，而不是唯一判断标准。

如何区分视觉感知失败与推理失败 为了定位视觉推理系统的能力瓶颈，可以设计不同的信息输入条件。例如，研究一个需要从图表中计算增长率的问题。条件一：原始视觉输入。模型直接根据图表图像和问题生成答案。这一条件反映完整视觉推理系统的表现。条件二：人工校验的结构化数据。将图表中的正确数值转换为结构化表格，再要求模型完成相同的计算任务。这一条件可以帮助分析数值读取与后续推理之间的关系。条件三：模型自动提取的数据。

先由模型提取图表数值，再使用相同的推理方法进行计算。这一条件有助于分析自动数据提取带来的误差。如果条件二明显优于条件一和条件三，说明视觉读取或结构恢复可能是重要瓶颈。如果条件二仍然表现较差，则可能说明模型在公式选择、逻辑运算或任务理解方面存在问题。原始图像与结构化数据属于不同输入形式，模型在两种输入下的行为可能受到表示差异影响。因此，这种对照属于诊断性实验，不能简单将其视为对内部推理机制的严格因果分解。

更合理的做法是结合中间结果、独立测试和受控输入变化进行综合判断。

公平验证与消融分析 如果研究者提出一种新的视觉推理方法，例如引入区域定位、结构化中间表示或外部计算工具，就需要通过公平实验判断其实际贡献。例如，可以设置如下实验条件。B0：直接视觉问答。模型根据原始图像和问题直接生成答案。B1：区域证据辅助。模型先获取与问题相关的视觉区域，再完成回答。B2：结构化信息辅助。模型将部分视觉信息转换为对象、属性或数值等结构化表示，再进行推理。B3：结构化信息与工具计算结合。

在 B2 基础上，使用确定性计算工具完成适合程序执行的操作。通过比较这些方法，可以分析不同机制是否产生实际收益。不过，这种比较也需要注意训练和推理条件。例如，B1 可能引入额外的目标检测器。B2 可能使用更多中间 token。B3 可能增加工具调用和计算延迟。如果只比较最终准确率，就可能忽略新增资源带来的成本。因此，应尽量报告额外参数、推理次数、视觉 token 数量和运行时间等信息。

此外，如果不同模型采用不同的训练数据或提示词策略，也需要明确记录。对于某个新增模块，最好通过移除该模块的消融实验，验证其独立贡献。如果某种结构化推理方法在完整系统中有效，但移除关键模块后性能没有明显下降，就需要进一步检

查该模块是否真正发挥了作用。因此，实验应当围绕明确的研究假设展开，而不是单纯增加模型组件数量。

面向可靠视觉推理的研究思路 综合上述内容，可以归纳出视觉推理系统改进的一条基本路线。明确任务所需的视觉证据。例如，数量问题需要对象实例，空间关系问题需要对象位置，图表问题需要正确的数据值和单位。分析基础感知是否能够可靠提取这些信息。例如，目标检测器是否能够区分多个相似对象，OCR 是否能够准确读取数字。然后，检查模型是否能够正确组织对象、属性和关系。对于复杂问题，可以考虑使用场景图、结构化中间表示或其他形式的信息组织方式。

在此基础上，再选择适当的推理方法。对于简单问题，端到端视觉问答可能已经足够。对于需要精确计算的问题，可以考虑利用结构化数据和确定性工具。对于复杂对象关系问题，可以考虑显式的区域定位和多步筛选。通过独立测试、反事实样本、证据定位和错误分析，验证系统是否真正获得预期能力。可靠视觉推理并不意味着所有任务都必须采用复杂的多阶段流程。额外模块和中间步骤可能提高可检查性，但也可能增加错误传播和计算成本。

因此，系统设计应当根据任务要求和实验结果决定，而不是预先假设越复杂的推理架构就一定越有效。视觉推理研究需要同时解决两个问题：一是如何从视觉输入中获得正确且充分的事实，二是如何根据这些事实形成有逻辑依据的结论。只有两者共同得到验证，才能较为可信地说明模型具备相应的视觉推理能力。

13.3 时序关系与因果线索

事件先后是时序关系，动作条件预测是条件分布，因果效应要求额外的识别假设或干预证据。三者可以在同一任务中出现，但证据强度不同。在线系统的因果遮罩只限制未来信息访问，不构成因果效应的识别方法。

13.3.1 时间顺序与动作变化

视频中的时间信息 视频是按照时间顺序记录的视觉观测序列。对于一段包含 T 帧的视频，可以表示为：

$$V = (I_t, \tau_t)_{t=1}^{T} \tag{13.21}$$

其中，I_t 表示第 t 帧图像，τ_t 表示对应的时间戳。与静态图像不同，视频中的相邻帧通常具有一定连续性。例如，一辆汽车在前一帧位于道路左侧，在后一帧移动到道路中央，这种变化提供了关于汽车运动的信息。但不同帧之间的差异不一定全部来自对象运动。相机自身的移动、视角变化、光照变化和遮挡，也可能导致图像内容发生明显变化。因此，视频中的时间变化不能简单等同于对象运动。

模型需要在一定程度上区分相机运动、目标运动和场景状态变化。在实际研究中，时间信息还具有不同粒度。例如，短时间内的手部移动可能只持续几百毫秒，而完整

的烹饪活动可能持续数分钟。因此，视频理解需要根据任务需要选择合适的时间范围和时间分辨率。

动作识别与动作变化理解 动作识别通常关注视频中发生了哪一种动作。例如：“拿起杯子”“打开房门”“推动箱子”“坐下”但动作变化理解不仅需要识别动作类别，还需要理解动作的方向、阶段和状态变化。例如：“将杯子放入盒子。”与：“将杯子从盒子中取出。”两种动作涉及相同的对象，但其状态变化方向相反。如果模型只根据某一帧中的对象外观进行判断，就可能无法区分这两种动作。类似地，“坐下”和“站起”可能包含相似的人体姿态，但动作顺序不同。

因此，可以将一个动作理解为对象状态随时间发生的变化。假设某个对象在时刻 t 的状态为 s_t，则其状态变化可以概括为：$\Delta s_t = s_{t+\Delta t} - s_t$，这一公式适用于能够使用数值向量表示的状态。对于离散状态，例如“门打开”或“门关闭”，通常需要采用状态转移而不是直接数值相减。例如：

$$\text{门关闭} \xrightarrow{\text{开门}} \text{门打开} \tag{13.22}$$

因此，动作理解不仅涉及当前状态，还涉及动作发生前后的状态差异。

动作阶段与事件边界 许多动作并不是瞬间完成的，而是包含多个连续阶段。例如，“拿起杯子”可以包含：人物伸手靠近杯子、手部接触杯子、人物抓住杯子、杯子离开桌面。人物将杯子移动到其他位置。这些阶段共同构成完整动作。在视频分析中，可以将动作或事件表示为时间区间：$e_i = (a_i, t_i^s, t_i^e)$，其中：

- a_i 表示动作或事件类别；
- t_i^s 表示事件开始时间；
- t_i^e 表示事件结束时间。

但事件边界并不总是具有完全明确的定义。

例如，“拿起杯子”的开始时间究竟是人物伸手时，还是手接触杯子时？不同数据集可能采用不同标注规则。因此，时间推理需要考虑事件定义和标注约定。对于具有模糊边界的事件，可以通过时间区间、边界容差或其他方式表达不确定性。不能简单认为视频中的所有动作都存在唯一、精确的起止时刻。

时间顺序推理 时间顺序推理（Temporal Order Reasoning）要求模型判断多个事件之间的先后关系。例如，一段视频中依次发生：事件 A：人物拿起钥匙。事件 B：人物打开房门。事件 C：人物走出房间。用户询问：“人物是在打开房门之前还是之后拿起钥匙?”模型需要找到事件 A 和事件 B 对应的时间范围，并比较它们的先后关系。如果：$t_A^e < t_B^s$，则可以判断事件 A 完全发生在事件 B 之前。

这里，t_A^e 表示事件 A 的结束时间，t_B^s 表示事件 B 的开始时间。但如果两个事件存在时间重叠，就不能简单使用这一条件判断严格先后。例如，人物可能一边打开房门，一边拿着钥匙。因此，时间关系需要根据事件区间进行判断。时间顺序推理不仅要求模型识别每个事件，还需要正确组织事件之间的时间关系。

时间扰动与反事实测试 为了判断模型是否真正利用视频中的时间信息，可以构建不同的测试条件。例如：单帧输入。只提供某个代表性视频帧，观察模型能否回答需要时间信息的问题。帧顺序打乱。保持采样帧集合不变，但改变帧的排列顺序。视频反向播放。在适合的任务中，反转视频时间顺序，检查模型是否能够识别动作方向的变化。时间采样变化。改变视频采样间隔，分析模型对不同时间分辨率的敏感程度。关键事件移除。移除或遮挡某个决定答案的关键时间片段，检查模型预测如何变化。

TempCompass 是专门针对视频模型时间感知能力构建的代表性基准。该研究设计具有相似静态内容、但在时间特征上存在差异的视频，用于减少模型依赖单帧信息或语言先验完成任务的可能性。不过，时间扰动实验需要注意任务适用性。例如，将一段视频反向播放后，某些自然动作可能变得不符合常见物理过程。因此，模型性能下降可能同时受到异常动作分布的影响。更合理的方式是结合具有明确答案变化的成对视频和独立标注进行评价。

13.3.2 事件关系与因果线索

从时间顺序到事件关系 在视频中，不同事件之间可能存在多种时间关系。例如：人物打开房门之后进入房间。人物一边走路一边打电话。水壶加热过程持续包含温度升高的变化。这些事件之间不只是简单的先后顺序，还可能存在同时发生、时间重叠和包含等关系。因此，事件关系推理（Event Relation Reasoning）需要建立更加完整的时间结构。

对于两个事件区间：$e_i = [t_i^s, t_i^e]$，$e_j = [t_j^s, t_j^e]$ 可以根据起止时间分析二者之间的关系。例如：

- 事件 e_i 在事件 e_j 之前结束；
- 两个事件在某个时间范围内发生重叠；
- 事件 e_i 完全包含在事件 e_j 中；
- 两个事件具有相同或接近的开始时间。

这种区间关系能够更加准确地描述现实视频中的活动结构。

同时发生与持续事件 现实视频中的多个动作可能同时进行。例如，一名人物正在走路，同时与他人交谈。如果模型只采用单一线性的事件列表，就可能错误地将这两个动作表示为严格先后关系。因此，事件表示需要允许多个事件在同一时间范围内存在。例如：$e_1 = [5, 15]$，$e_2 = [8, 12]$，其中，e_1 可以表示人物走路，e_2 表示人物交谈。两个事件存在时间重叠。这种关系不同于“人物走完路之后才开始交谈”。

对于复杂视频，事件可能形成相互交叉的时间结构，而不是一条完全线性的动作序列。例如，在多人协作任务中，不同人物可能同时操作不同物体。因此，时序推理模型需要能够表达并行活动及其关系。

时间相关性与因果关系的区别 因果推理（Causal Reasoning）关注一个事件是否对另一个事件的发生或状态变化具有因果作用。例如，一段视频中出现：人物碰倒杯子。杯

子从桌面边缘掉落。杯子撞击地面并破碎。这些连续事件在视觉上具有较为明确的物理联系，可以为“碰撞导致杯子掉落和破碎”提供支持。但并非所有相邻发生的事件都具有因果关系。例如，一段视频中：人物打开窗户。随后室外开始下雨。不能仅根据两个事件的时间顺序，就认为“打开窗户导致下雨”。

因此，需要区分：时间先后关系：一个事件发生在另一个事件之前。统计相关关系：两个事件经常共同出现或具有一定关联。因果关系：一个事件的发生或变化对另一个事件产生影响。时间先后通常可以作为部分因果判断的必要线索，但并不是充分条件。在因果分析中，还需要考虑其他可能因素、背景条件和替代解释。

视频中的因果线索 视频能够提供一些静态图像难以直接观察的因果线索。例如：物体接触之后发生运动。支撑物被移除之后，物体掉落。人物按下开关之后，灯光亮起。这些事件包含连续状态变化和物理交互信息。在一些条件下，这些信息可以帮助模型判断可能的事件原因。但单段观察视频通常不能严格证明因果关系。例如，按下开关之后灯光亮起，可能支持两者具有因果联系，但仍需要考虑是否存在其他控制装置或同步事件。

从更严格的因果推断角度看，干预或反事实分析通常能够提供更强的因果证据。例如，比较在其他条件相同的情况下，按下开关和不按下开关时灯光是否发生变化。但在真实视频中，这种干预通常难以直接实现。因此，对于一般视频理解任务，更适合将部分结论表述为“视频中的证据支持某种可能的因果解释”，而不是将所有事件解释都视为经过严格验证的因果关系。

因果视频问答 视频问答研究已经开始专门评价模型对事件原因和时间关系的理解能力。NExT-QA 是这一方向的重要代表性数据集。该研究将视频问答任务进一步扩展到时间动作解释，包含因果、时间和描述性问题。例如：“人物为什么停止移动?”“某个动作发生之前出现了什么事件?”“人物完成某项操作后发生了什么?”这类问题与简单动作分类不同。模型不仅需要识别动作，还需要分析不同事件之间的联系。

NExT-QA 的研究指出，模型可能较擅长回答场景描述问题，但在因果和时间推理方面仍然存在明显困难。这一结果说明，视频语义理解与事件推理之间存在能力差异。数据集中的因果问题通常是根据视频内容和人工判断构建的，并不等同于严格干预条件下的因果识别任务。因此，模型在此类基准上获得较高成绩，说明其更擅长回答事件原因相关的问题，但不能直接证明其具有普遍可靠的因果推断能力。

13.3.3 跨帧信息整合

为什么需要跨帧信息整合 视频中的关键信息通常分布在多个时间点。例如，一段视频开始时，人物将钥匙放在桌子上。随后，人物离开房间。在视频后半段，另一个人将钥匙放入抽屉。如果用户询问：“视频结束时，钥匙在哪里?”模型不能只依据最初的画面回答“桌子上”，也不能只依据最后一帧进行推断。它需要结合多个时间片段，理解同一对象的位置和状态变化。这种过程可以称为跨帧信息整合（Cross-frame Information Integration）。

其核心是将不同时刻的观测联系起来，形成对对象、动作和场景变化的连续理解。跨帧整合通常需要解决三个问题。判断不同帧中的对象是否属于同一实例。确定对象状态如何随着时间改变。根据问题要求选择与最终结论相关的时间信息。因此，跨帧整合不仅是对多帧特征进行简单聚合，还可能涉及对象身份、时间关系和状态更新。

跨帧对象关联 在连续视频中，同一个对象可能在多个时间点被观察到。例如，一辆汽车从画面左侧进入，随后经过路口并驶出画面。虽然汽车在不同帧中的位置发生变化，但它仍然属于同一个对象实例。因此，模型需要建立跨帧对象关联。假设第 t 帧中检测到对象集合：

$$O_t = o_{t,1}, o_{t,2}, \ldots, o_{t,N_t} \tag{13.23}$$

跨帧关联需要判断：

$$o_{t,i} \leftrightarrow o_{t+\Delta,j} \tag{13.24}$$

是否表示同一个真实对象。这种判断可能利用对象外观、空间位置、运动连续性和其他信息。例如，两帧中的车辆如果具有相似外观，并且位置变化符合合理运动轨迹，就可能属于同一实例。但在复杂场景中，对象可能发生遮挡、形变或暂时离开画面。如果模型无法保持对象身份一致性，就可能将同一对象错误地视为多个对象，或者将不同对象混淆。因此，跨帧对象关联是时间推理的重要基础。

传统多目标跟踪与多模态大模型中的对象关联具有联系，但两者的任务目标不完全相同。多目标跟踪主要关注对象轨迹和身份连续性。多模态推理则可能进一步利用这些轨迹回答动作、关系和历史状态问题。

跨帧状态变化建模 对象状态不仅包括位置，还可能包含姿态、运动状态、与其他对象的关系等信息。例如，一个杯子的状态可能包括：位置：桌子上。状态：直立。是否被人物持有：否。随后，人物拿起杯子。杯子的状态可能变为：位置：人物手中。状态：被持有。因此，可以将对象状态表示为：$s_t^i = (p_t^i, a_t^i, r_t^i)$，其中：

- p_t^i 表示对象 i 的位置或空间状态；
- a_t^i 表示对象属性及动态状态；
- r_t^i 表示对象与其他实体之间的关系。

当新的观测到达时，模型需要根据实际变化更新相关状态。

例如：$s_{t+1}^i = U(s_t^i, o_{t+1})$，其中，$o_{t+1}$ 表示新观测，U 表示状态更新函数。这一公式是一种概念性表达，并不意味着所有视频模型都使用显式状态更新模块。在一些模型中，状态变化可能通过 Transformer 注意力或其他时序编码机制隐式表示。但从任务需求角度看，跨帧状态变化仍然是时间推理的重要内容。

时间信息与特征聚合 在视频模型中，一种常见的处理方式是分别编码不同帧，再对帧级特征进行聚合。假设第 t 帧的视觉特征为：$z_t = f_{\text{visual}}(I_t)$，则视频的联合表示可以

写为：$H = F(z_1, z_2, \ldots, z_T)$，其中，F 表示时间信息整合函数。最简单的方式之一是平均池化：

$$H = \frac{1}{T}\sum_{t=1}^{T} z_t \tag{13.25}$$

这种方法能够汇总多个时间点的视觉信息，但通常不适合需要严格区分事件顺序的任务。例如：“人物将物体放入盒子。”与：“人物将物体从盒子中取出。”如果两段视频包含相似的帧集合，只是排列顺序不同，那么简单的平均池化无法根据顺序差异区分两者。因此，对于依赖时间变化的任务，需要保留或建模时间顺序。可以使用时序卷积、循环网络、时序注意力或其他序列模型整合信息。不同方法具有不同的计算成本与时间依赖建模特点。

时间位置编码与时间戳 Transformer 通过注意力机制建立不同 token 之间的联系，但如果缺少适当的位置或时间信息，模型可能无法完整区分不同观测的先后关系。因此，视频模型通常需要通过时间位置编码、时间戳或其他方式表示帧的位置。例如，一段视频包含：第 2 秒：人物拿起杯子。第 5 秒：人物喝水。第 8 秒：人物放下杯子。如果模型只获得三个视觉片段，却不知道它们的时间顺序，就可能无法准确回答“人物是在喝水之前还是之后放下杯子”。

因此，时间信息不仅可以采用帧序号表示，也可以使用实际时间戳。TimeChat 是面向长视频时间理解的代表性工作。该方法将时间戳信息与视频帧编码结合，并使用滑动的视频 Q-Former 处理不同长度的视频表示。其研究目标包括时间定位、密集视频描述和其他时间敏感的任务。这类研究说明，在视频大语言模型中，显式时间信息能够帮助模型建立视觉内容与时间位置之间的联系。不过，加入时间戳并不自动保证模型能够理解复杂事件关系。

模型仍然需要学习如何利用这些时间信息完成具体推理。

13.4 空间约束与动态状态

13.4.1 空间关系与几何推理

空间关系的基本概念 空间关系（Spatial Relation）描述对象在空间中的相对位置、方向、距离和几何组织方式。在日常语言中，空间关系通常通过“左侧”“右侧”“上方”“下方”“前方”“后方”“靠近”“远离”“内部”和“相邻”等表达。例如：“杯子位于电脑右侧。”“椅子位于桌子前方。”“书本放在抽屉内部。”“汽车停在建筑物旁边。”这些关系看似简单，但其含义通常依赖于观察视角、坐标系和对象的实际几何结构。

例如，“左侧”可以表示对象在图像中的左边，也可以表示相对于人物朝向的左侧。“前方”可能表示相对于相机的前方，也可能表示车辆行驶方向的前方。因此，空间推理首先需要明确空间关系的参考系。从任务形式来看，空间推理可以分为定性与定量两类。

定性空间推理（Qualitative Spatial Reasoning）主要判断对象之间的关系类型。例如："椅子是否位于桌子旁边？""汽车是否在行人前方？"

定量空间推理（Quantitative Spatial Reasoning）则进一步估计距离、角度、尺度或其他数值。

例如："杯子与桌子边缘相距多少厘米？""两个物体之间的距离是多少米？""机器人需要转动多少角度才能面向目标？"定性推理不一定需要恢复完整的三维几何结构，但定量推理通常对深度、尺度和坐标信息提出更高要求。

二维图像坐标与三维世界坐标 图像中的空间关系通常建立在二维像素坐标系中。假设一个像素的位置为：$p = (u, v)$，其中，u 和 v 分别表示图像中的水平和垂直坐标。在常见的图像坐标约定下，水平方向从左向右增大，垂直方向从上向下增大。因此，两个对象在图像中的左右关系，可以通过其水平坐标进行部分判断。但真实世界中的对象存在于三维空间。一个三维点可以表示为：$P = (X, Y, Z)$，其中，X 、Y 和 Z 表示三维坐标。

二维图像是三维场景经过相机成像过程得到的投影。在简化的针孔相机模型中，可以表示为：

$$\tilde{p} = K \left(R \begin{bmatrix} X \\ Y \\ Z \end{bmatrix} + t \right) \tag{13.26}$$

其中：

- K 表示相机内参矩阵；
- R 表示从世界坐标系到相机坐标系的旋转；
- t 表示相应的平移；
- s 表示齐次投影尺度。

对于通常采用的相机坐标定义，在点位于相机前方且采用标准针孔模型时，s 对应相机坐标系中的深度。

这一关系说明，三维空间中的不同点可能投影到相同或非常接近的二维位置。例如，一辆较远的大型汽车和一辆较近的小型汽车，可能在图像中具有相似的大小。因此，不能简单地根据图像中的像素尺寸判断物体真实尺度。类似地，两个对象在图像中的水平位置接近，也不意味着它们在三维空间中的实际距离很小。这也是二维视觉关系与真实三维空间关系之间的重要区别。

坐标参考系与视角依赖 空间推理中的一个关键问题是参考系（Reference Frame）。例如："杯子位于椅子左侧。"这一表达可能依赖观察者的位置和朝向。如果相机从房间另一侧观察同一组物体，图像中的左右位置可能发生变化。但两个物体在房间中的真实位置并没有改变。因此，可以区分几种常见参考系。图像参考系以二维图像坐标为基础，适合描述画面中的上下左右关系。相机参考系以相机位置和朝向为基础，适合

表达对象相对于相机的三维位置。

世界参考系使用相对固定的空间坐标，适合描述场景中的全局位置和运动。对象参考系以某个对象自身的位置和朝向为基础，例如车辆前方、人物左侧或机器人右侧。不同参考系之间可以通过坐标变换建立联系。例如，某个点在相机坐标系中的位置可以写为：$P_c = RP_w + t$，其中：

- P_w 表示世界坐标；
- P_c 表示相机坐标；
- R,t 表示坐标变换参数。

在多模态空间推理中，如果问题没有明确参考系，模型可能产生不同解释。

例如，“汽车左边的人”通常指图像中汽车左侧的人，而“驾驶员左侧的车门”则可能依赖车辆或驾驶员自身的朝向。因此，空间问答需要正确理解语言中隐含的参考视角。当表达具有歧义时，应当结合任务约定或明确说明采用的坐标参考系。

相对距离与几何关系 对于三维空间中的两个点：$P_A = (x_A, y_A, z_A)$，$P_B = (x_B, y_B, z_B)$，其欧氏距离为：

$$d(P_A, P_B) = \sqrt{(x_A - x_B)^2 + (y_A - y_B)^2 + (z_A - z_B)^2} \tag{13.27}$$

这一距离用于衡量两个三维点之间的直线距离。但在实际场景中，任务所需的距离可能具有不同含义。例如，机器人从当前位置移动到目标位置时，两个位置之间的直线距离不一定等于实际行驶距离。如果中间存在桌子、墙壁或其他障碍物，机器人需要绕行。因此，需要区分：欧氏距离：空间中两个点之间的直线距离。路径距离：在满足环境约束的条件下，沿可行路径移动的距离。表面距离：沿物体表面或其他指定几何结构测量的距离。

例如，桌子两侧的两个位置可能在欧氏距离上较近，但由于桌子阻挡，实际可行路径更长。因此，空间推理不能始终将“最近”理解为欧氏距离最小。对于机器人导航和环境交互任务，还需要考虑可通行区域和运动约束。

单目视觉的深度与尺度歧义 单张 RGB 图像可以提供丰富的视觉信息，但通常无法在缺少额外条件时唯一恢复完整的真实三维场景。例如，图像中的透视、阴影、遮挡和纹理可以提供一定深度线索。但这些线索未必足以确定物体的精确距离。一个典型问题是尺度歧义。例如，较小且距离相机较近的物体，可能与较大且距离相机较远的物体具有相似的投影。因此，仅根据单张图像，通常难以唯一确定物体的真实尺寸和绝对深度。

单目深度估计模型可以通过学习得到深度预测，但其数值可靠性仍然受到训练数据、相机成像条件和场景分布影响。对于能够提供真实深度观测的 RGB-D 相机，可以利用深度与相机内参恢复三维点。在标准针孔模型中，若像素 (u,v) 对应的相机轴向深度为 Z ，则：

$$P_c = ZK^{-1}\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} \tag{13.28}$$

其中，P_c 表示相机坐标系中的三维点。这一公式需要满足深度定义与相机标定参数一致等条件。实际深度传感器也可能存在噪声、深度空洞和测量误差。因此，RGB-D 输入能够提供更加明确的几何信息，但不意味着三维重建一定完全准确。

空间几何推理的评价 空间推理的评价需要区分定性关系和定量几何。对于“左侧”“右侧”“前方”等关系判断，可以使用分类准确率。对于距离估计，可以采用绝对误差：

$$\mathrm{MAE} = \frac{1}{N}\sum_{i=1}^{N}|\hat{d}_i - d_i| \tag{13.29}$$

其中，$\hat{d}_i$ 为预测距离，d_i 为真实距离。对于三维位置估计，可以使用欧氏位置误差。对于姿态估计，可以分别评价平移和旋转误差。此外，还需要检查模型是否对视角变化具有合理的响应。例如，观察者从物体另一侧拍摄图像时，图像中的左右关系可能发生变化，但物体在世界坐标系中的真实位置关系应保持一致。

3DSRBench 是面向三维空间推理的代表性评价基准，关注高度、位置、方向和多对象空间关系等能力，并通过不同视角和答案平衡设计进行诊断。这类研究说明，不能仅根据普通视觉问答成绩判断模型的空间推理水平，还需要针对几何关系和视角变化开展专门评价。

13.4.2 三维场景与物体关系

三维场景理解的基本概念 三维场景理解（3D Scene Understanding）要求模型识别并组织真实空间中的对象、几何结构和语义关系。与二维图像理解相比，三维场景理解通常不仅需要知道图像中有什么对象，还需要判断对象在三维空间中的位置、形状、方向和相互关系。例如，一张室内图像可以展示桌子、椅子和柜子。二维视觉模型可能识别出这些对象，并预测它们在图像中的边界框。三维场景理解则进一步关注：桌子位于房间中的什么位置。

椅子相对于桌子的方向是什么。柜子与墙壁之间具有怎样的空间关系。机器人能否从门口移动到桌子旁边。因此，三维场景理解不仅涉及对象识别，还涉及空间布局与环境结构。

三维场景的常见表示形式 三维场景可以采用不同的数据结构进行表示。点云（Point Cloud）。点云由一组具有三维坐标的点构成：

$$P = (x_i, y_i, z_i)_{i=1}^{N} \tag{13.30}$$

每个点还可以包含颜色、法向量或其他特征。点云能够较直接地表示物体表面和场景几何结构，常见于激光雷达和 RGB-D 数据处理。体素（Voxel）。体素将三维空间

划分为离散网格。每个体素可以表示空间占据情况、语义类别或其他属性。体素表示便于进行规则的空间计算，但较高空间分辨率可能带来较大的存储和计算开销。三角网格（Mesh）。网格通过顶点、边和面表示物体或场景表面。

相较于无连接关系的点云，网格可以提供更加明确的表面拓扑结构。鸟瞰图（Bird's-Eye View，BEV）。鸟瞰图将三维空间信息组织到俯视平面中，通常保留目标位置、占据或语义等信息。BEV 在自动驾驶和机器人导航中具有广泛应用，但将三维信息投影到二维平面，也可能损失部分高度结构。隐式空间表示。部分方法使用连续函数或神经网络表示场景中的密度、占据、颜色或其他空间属性。

这类方法可以支持连续位置查询，但其语义和几何能力取决于具体训练目标。不同表示形式具有不同优势，并不存在对所有任务都最优的统一选择。例如，点云适合保留几何采样信息，体素便于占据推理，而对象级场景图更加适合表达语义关系。因此，实际三维系统可以根据任务要求组合多种表示方式。

三维对象的空间属性 对于三维场景中的对象，通常可以定义多个空间属性。例如：$o_i = (c_i, p_i, R_i, d_i)$，其中：

- c_i 表示对象类别；
- p_i 表示对象的三维位置；
- R_i 表示对象朝向；
- d_i 表示对象的尺寸信息。

这里的尺寸可以表示为长、宽、高，也可以采用更加复杂的几何描述。

例如，一辆汽车可以用三维边界框近似表示。其位置表示汽车中心所在的空间坐标。其朝向表示车辆面向的方向。其尺寸则描述汽车在三个方向上的空间范围。通过这些属性，可以进一步判断对象之间的相对位置、距离、朝向和空间重叠情况。例如，一辆汽车位于另一辆汽车前方，或者一个箱子位于某个货架内部。三维边界框只是对象几何形状的近似表示。对于形状不规则的物体，仅依靠边界框可能无法准确描述接触、支撑和碰撞关系。

因此，在需要精细物理交互的任务中，可能还需要表面网格、占据表示或其他几何信息。

具有观察者状态的空间问答 某些空间问题不仅依赖场景几何，还依赖观察者所在的位置和朝向。例如："我面前的桌子在哪里？""从当前位置向左转，是否能够看到沙发？""我能否直线走到前方的茶几？"这些问题中的"我""前方"和"左侧"都具有明显的观察者参考系含义。因此，模型不仅需要理解三维场景，还需要知道观察者在场景中的状态。SQA3D 是这一方向的代表性基准。

该任务要求模型结合三维场景和给定的情境描述，理解观察者的位置与朝向，再回答相应的空间问题。例如，模型需要判断某个对象相对于当前观察者的位置，或者分析从指定位置出发的空间关系。这种任务与普通的三维对象定位有所不同。它强调场景理解与观察者自身状态之间的联系。因此，空间推理不仅需要建立客观的场景几何，还需要根据任务参考系对空间关系进行转换。

三维场景理解的主要局限 三维场景理解面临多种挑战。首先是几何重建误差。例如，深度噪声、低纹理区域和相机姿态误差可能影响三维位置估计。其次是遮挡与观测不完整。例如，某个柜子背后的区域可能从未被相机观察到。此时，系统通常无法仅根据已有数据确定该区域中的全部对象。再次是对象实例关联。例如，同一个物体在不同视角中可能具有不同外观，模型需要判断它们是否对应同一实体。此外，三维场景还可能持续变化。例如，人物移动椅子或打开抽屉之后，场景布局发生变化。

如果系统仍然使用旧的三维状态，就可能给出过时的空间判断。因此，三维场景理解不仅需要空间几何，还可能需要连续观测和状态维护。

三维空间理解的评价 三维空间理解可以采用不同任务指标。对于三维目标检测，可以使用三维边界框交并比（3D IoU）及相应的 AP 指标。对于三维对象定位，可以使用基于三维 IoU 阈值的定位准确率。对于三维语义分割，可以使用类别级 IoU 或 mIoU。对于三维空间问答，可以使用答案准确率等指标，并根据任务需要补充对象定位质量。对于距离和姿态估计，则可以分别采用位置误差和旋转误差。不同任务的指标不能直接替代。

例如，三维检测 AP 较高，不意味着模型能够可靠回答复杂空间语言问题。同样，空间问答准确率较高，也不代表模型具有精确的三维重建能力。因此，评价三维场景理解时，应当根据任务目标选择对应指标，并明确输入模态、坐标系和空间精度要求。

13.4.3 动作、运动与状态变化

从静态场景到动态场景 真实环境中的物体通常并不始终保持静止。车辆可能移动，人物可能改变姿态，机器人可能推动物体，场景中的接触和支撑关系也可能发生变化。因此，空间智能不仅需要理解某一时刻的空间结构，还需要分析对象随时间发生的变化。例如，一张图像可能显示杯子位于桌子上。随后，人物伸手拿起杯子。再经过一段时间，人物将杯子放入柜子。如果模型只识别每一帧中的物体类别，就无法完整描述杯子的状态变化。

因此，需要建立跨时间的对象关联，并对位置、姿态和关系变化进行分析。这类问题可以概括为动态场景理解（Dynamic Scene Understanding）。它既包含视觉运动感知，也包含对象状态推理和事件结果判断。

位置、速度与加速度 对于三维空间中的运动物体，可以使用位置随时间的变化表示其运动状态。假设物体的位置为：

$$p(t) = \begin{bmatrix} x(t) \\ y(t) \\ z(t) \end{bmatrix} \tag{13.31}$$

则速度为：

$$v(t) = \frac{dp(t)}{dt} \tag{13.32}$$

加速度为：

$$a(t) = \frac{dv(t)}{dt} \tag{13.33}$$

在离散时间观测中，可以使用有限差分近似速度：

$$v_t \approx \frac{p_t - p_{t-\Delta t}}{\Delta t} \tag{13.34}$$

其中，p_t 和 $p_{t-\Delta t}$ 表示两个时刻的三维位置。但该计算要求两个位置处于一致的坐标参考系中。如果相机自身正在移动，仅根据对象在图像中的位置变化，通常无法直接得到对象在世界坐标系中的真实速度。因此，动态空间理解需要区分相机运动与物体运动。对于移动相机系统，通常需要利用相机位姿估计或其他运动补偿机制。

对象状态与动态关系 在现实任务中，物体状态并不只包含几何信息。例如，一个杯子的状态可能包含：位置：桌面。姿态：直立。是否被持有：否。是否装有液体：是。当人物拿起杯子时，杯子的状态发生变化。因此，可以将物体状态抽象为：

$$s_t^i = (p_t^i, R_t^i, a_t^i, r_t^i) \tag{13.35}$$

其中：

- p_t^i 表示位置；
- R_t^i 表示姿态；
- a_t^i 表示对象的属性或其他状态；
- r_t^i 表示对象与环境中的关系。

例如，杯子从桌面移动到人物手中时，不仅位置改变，“被桌子支撑”与“被人物持有”等关系也可能发生变化。

因此，动态场景理解需要同时关注对象状态和对象间关系的更新。并非所有状态都能够直接从视觉中识别。例如，杯子内部是否装有液体，可能因杯子不透明而无法确定。因此，系统需要区分直接观测状态与通过历史信息或常识推断的状态。

动作与状态转移 动作可以理解为引起对象或环境状态变化的重要因素。例如：“打开抽屉”使抽屉从关闭状态转变为打开状态。“移动椅子”改变椅子的空间位置。“拿起杯子”改变杯子的位置及其与人物的关系。因此，可以将动作与状态变化表示为：$s_{t+1} = F(s_t, u_t, \eta_t)$，其中：

- s_t 表示当前状态；
- u_t 表示执行的动作；
- η_t 表示未建模扰动或其他影响因素；
- s_{t+1} 表示动作之后的状态。

这一形式与状态转移建模有关，在机器人控制、动态系统和世界模型研究中都有意义。

状态转移函数 F 可以是基于物理规律的模型，也可以是从数据中学习得到的近似模型。此外，动作并不是影响状态变化的唯一因素。例如，即使没有机器人执行动作，物体也可能因重力、外部环境变化或其他主体的行为而运动。因此，动态状态预测需要考虑场景中可能存在的多种影响因素。

动态状态中的遮挡与历史保持 在连续感知中，某些对象可能暂时离开视野或被遮挡。例如，一辆汽车驶入建筑物后方，随后又从另一侧出现。如果模型在汽车被遮挡时立即删除其全部状态，可能导致对象身份丢失。但如果模型始终保留旧状态，又可能在对象实际发生变化时使用过期信息。因此，动态状态维护需要在信息保持与更新之间取得平衡。例如：当前能够观察到对象时，可以根据新观测更新状态。对象暂时不可见时，可以根据历史信息维持有限的状态估计。

当新的可靠观测出现时，应根据实际证据修正此前状态。如果长期没有新观测，则应适当增加状态不确定性。这一问题与上一节讨论的长视频记忆具有联系，但空间智能进一步强调状态的几何含义和运动一致性。例如，对象的位置预测不仅需要在时间上连续，还应符合合理的运动范围。因此，动态空间状态维护需要同时考虑时间信息、几何约束和观测可靠性。

动态空间理解的评价 动态空间理解可以从多个维度进行评价。对于目标轨迹预测，可以使用平均位移误差（Average Displacement Error，ADE）和最终位移误差（Final Displacement Error，FDE）。例如：

$$\mathrm{ADE} = \frac{1}{H}\sum_{k=1}^{H} |\hat{p}_{t+k} - p_{t+k}|_2 \tag{13.36}$$

其中，$\hat{p}_{t+k}$ 和 p_{t+k} 分别表示预测位置和真实位置。最终位移误差可以表示为：$|\hat{p}_{t+H} - p_{t+H}|_2$，这些指标主要用于评价轨迹位置误差。对于动作与状态变化理解，还可以采用动作分类准确率、状态预测准确率和关系变化识别准确率。对于交互事件，可以评价接触时间、碰撞发生与否及事件结果。但轨迹误差较小并不意味着预测一定满足全部物理约束。

例如，一条预测轨迹可能与真实轨迹距离较近，却短暂穿过障碍物。因此，实际评价还需要根据任务补充碰撞、可行性和其他约束指标。

13.5 推理时计算与外部验证

额外候选、搜索、工具和校验都消耗预算。比较推理方法时，至少记录候选数、生成 Token、工具次数、延迟与成功率；在相同总预算下才容易解释机制收益。若裁判与候选模型共享偏差，多候选选择也可能稳定选错。

13.5.1 分步求解与中间结果

为什么复杂任务需要分步求解 对于较简单的多模态任务，模型可以根据输入直接生成答案。例如，用户提供一张清晰的图片并询问：“图中的汽车是什么颜色？”如果图像中只有一辆汽车，模型可能只需要识别对象并判断颜色。但对于复杂任务，直接生成答案可能难以同时满足所有条件。例如：“图中站在红色汽车旁边、没有戴帽子的人手里拿着什么？”这一问题至少涉及以下操作。首先识别红色汽车。然后找到与该汽车具有指定空间关系的人物。接着判断相关人物是否戴帽子。

最后识别满足条件的人物手中的物体。因此，复杂任务往往包含多个相互依赖的子问题。分步求解（Step-by-step Problem Solving）的基本思想，是将复杂问题分解为若干能够分别处理的操作，并利用中间结果逐步形成最终答案。这种方式与前面介绍的组合式视觉推理具有联系，但本节更加关注模型如何组织实际推理计算，而不只是讨论任务本身包含哪些逻辑关系。

分步求解的基本形式 假设原始任务为：Q 模型可以生成一组子任务：

$$\mathcal{Q} = q_1, q_2, \ldots, q_n \tag{13.37}$$

每个子任务产生一个中间结果：$z_i = f_i(X, Q, z_{<i})$，其中：

- X 表示原始多模态输入；
- z_i 表示第 i 步的中间结果；
- $z_{<i}$ 表示此前已获得的结果；
- f_i 表示当前处理操作。

最终答案可以表示为：$Y = g(z_1, z_2, \ldots, z_n)$，

例如，对于图表中的增长率问题：第一步，读取上一年度数值。第二步，读取当前年度数值。第三步，根据问题选择计算公式。第四步，执行计算。第五步，输出结果及其单位。如果上一年度数值为 120，当前年度数值为 150，则：

$$r = \frac{150 - 120}{120} \times 100\% = 25\% \tag{13.38}$$

这种处理方式使数据读取、计算和答案生成具有相对明确的分工。不过，实际模型不一定需要将所有中间结果转换为自然语言。例如，视觉编码器可以在特征空间中完成对象筛选，数学运算可以通过外部程序执行，最终只输出必要的结果。因此，分步求解的核心是合理组织信息处理过程，而不是要求模型始终生成较长的文字说明。

中间结果的可验证性 分步求解的一个潜在优势，是允许系统检查部分中间结果。例如，对于图表问题，可以分别检查数值提取和数学计算。对于指代表达问题，可以检查目标区域是否与语言描述一致。对于视频推理，可以检查事件时间区间与实际画面是否对应。这类检查有助于判断错误发生在哪个处理阶段。但是，并非所有中间步骤都具有容易确定的正确答案。例如：“这段视频为什么让人感到紧张？”这种问题可能涉及

主观感受和多种解释，不容易为每个中间步骤提供唯一标准。

因此，可验证性与任务类型密切相关。对于具有明确数值、对象标注或规则约束的任务，可以使用更加直接的校验方式。对于开放式解释任务，则需要关注证据支持、逻辑一致性和必要的不确定性表达。

过程监督与结果监督 在研究多步推理时，还需要区分结果监督（Outcome Supervision）和过程监督（Process Supervision）。结果监督主要根据最终答案是否正确提供反馈。过程监督则进一步评价中间推理步骤的正确性。例如，对于数学问题：结果监督只判断最终答案是否为 25%。过程监督还可能检查原始数值、公式选择和计算步骤是否正确。

Lightman 等人在《Let's Verify Step by Step》中比较了结果监督和过程监督，并通过数学推理任务研究了步骤级验证的价值。该研究表明，在其研究的数学任务设置中，过程监督可以为训练更可靠的验证模型提供有效信息。这项研究主要围绕语言数学推理展开，其结果不能直接视为所有多模态任务中的普遍结论。对于视觉推理，过程监督还需要解决视觉证据本身是否正确的问题。

例如，模型可能使用正确公式，却错误读取图表数值。因此，视觉推理的步骤级监督不仅需要检查逻辑操作，还需要关注中间事实与原始输入的对应关系。

分步求解的局限 分步推理虽然能够帮助组织复杂任务，但也可能引入新的问题。首先是错误传播。如果第一个步骤识别了错误对象，后续操作可能全部受到影响。其次是步骤冗余。某些简单任务可以直接回答，额外生成多个步骤可能只会增加计算成本。再次是推理过程与真实证据脱节。模型可能生成逻辑上连贯的解释，却没有真正利用关键视觉信息。此外，过长的中间上下文还可能引入重复信息和无关内容。因此，不能简单认为推理步骤越多，模型性能就一定越高。

更合理的研究方向是根据任务复杂度选择适当的分步程度，并通过中间结果校验判断额外步骤是否真正产生收益。

13.5.2 多候选生成与结果选择

为什么要生成多个候选答案 对于复杂任务，同一个模型在不同采样条件下可能产生不同的答案或求解路径。例如，对于一张包含多个对象的图像，模型可能采用不同的对象筛选顺序。对于一道视觉数学题，模型可能选择不同的计算表达方式。对于视频事件问题，模型可能关注不同的时间片段。因此，除了只生成一个答案，还可以在推理阶段生成多个候选结果，再根据一定策略进行选择。

这一思路通常称为多候选生成与选择（Multiple Candidate Generation and Selection）。假设模型根据输入生成 N 个候选答案：

$$\mathcal{Y} = y_1, y_2, \ldots, y_N \tag{13.39}$$

随后，通过选择函数得到最终答案：

$$y^* = \text{Select}(\mathcal{Y}) \tag{13.40}$$

其中，选择过程可以使用多数投票、模型评分、外部验证或其他规则。这种方法的潜在优势是增加探索不同求解路径的机会。但其收益取决于候选结果是否具有有效差异，以及选择机制能否识别更可靠的答案。

采样温度与候选多样性 语言模型通常根据预测分布生成 token。对于一个简化的 softmax 输出分布，可以使用温度参数 T 调整采样概率：

$$p_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)} \tag{13.41}$$

其中：

- z_i 表示候选 token 的 logit；
- T 表示采样温度。

在通常情况下，较低温度使分布更加集中，较高温度则使分布更加平坦。

因此，较高温度可能增加生成结果的多样性。但多样性增加并不意味着正确率一定提高。例如，对于需要精确读取数字的视觉任务，过于随机的采样可能产生更多错误结果。另一方面，如果温度过低，多个候选答案可能高度相似，难以提供有效的搜索收益。因此，多候选推理需要在候选质量和候选差异之间取得平衡。此外，候选多样性还受到模型能力、提示词、采样策略和问题性质等因素影响。

独立候选与相关错误 为了理解多候选生成的潜在收益，可以考虑一个理想化情形。假设每次生成正确答案的概率为 p ，且不同采样结果相互独立。那么，N 次采样中至少出现一次正确答案的概率为：$P_{\geq 1} = 1 - (1-p)^N$，例如，当单次正确率为 0.5、采样次数为 4 时：$1 - (1-0.5)^4 = 0.9375$，这意味着在理想独立假设下，四次采样至少包含一个正确答案的概率为 93.75%。

但这并不是多数投票或最终选择后的正确率。它只表示候选集合中至少包含一个正确答案的概率。要将这一潜在收益转化为实际收益，还必须有能力从候选集合中选择正确答案。此外，多模态模型的候选错误往往并不独立。例如，同一个模型可能在所有候选中都错误识别某个对象。不同候选也可能共享相同的语言先验和视觉编码误差。因此，上述公式主要用于说明理想条件下的候选覆盖概率，不能直接用于预测真实多模态系统的实际正确率。

搜索与候选路径选择 当问题包含多个中间决策时，可以进一步将推理过程视为搜索问题。例如，模型需要决定：先观察哪个图像区域。下一步执行哪种计算。是否继续展开某条推理路径。何时终止推理并给出答案。在这种情况下，候选不一定是完整答案，也可以是部分推理状态。例如：

$$s_0 \to s_1 \to s_2 \to \cdots \to s_T \tag{13.42}$$

其中，s_t 表示当前推理状态。模型可以根据一定的评分策略，选择值得继续展开的状态。常见思想包括束搜索、树搜索和基于验证器的搜索等。但搜索并不一定比简单采样更高效。如果搜索评价函数不准确，系统可能将计算资源集中到错误路径上。此外，搜索分支过多会增加计算和内存开销。因此，搜索策略需要根据问题复杂度和验证器质量进行设计。

多候选生成的适用条件 多候选推理通常更加适合具有以下特点的任务。任务具有多种可能的求解路径。例如，数学问题可以通过不同计算顺序得到相同答案。存在相对可靠的答案验证方式。例如，能够通过程序检查计算结果或约束是否满足。单次生成具有一定正确概率。如果模型几乎无法获取任务所需的关键感知信息，仅增加采样次数通常难以从根本上解决问题。此外，多候选方法还需要考虑答案形式。例如，对于精确数值问题，可以直接比较最终数值。

但对于开放式自然语言描述，多个语义等价的答案可能采用不同表达。因此，简单字符串投票可能低估答案之间的语义一致性。反过来，过于宽松的语义归并也可能将实际含义不同的答案合并。因此，多候选结果选择需要根据任务类型设计适当的聚合与验证方式。

13.5.3 工具辅助推理与外部验证

工具辅助推理的机制 多模态大语言模型可以根据输入生成自然语言、结构化数据或其他形式的输出。但部分任务需要的能力，并不一定适合完全通过语言 token 预测完成。例如：精确的数值计算。高分辨率局部 OCR。复杂对象检测与分割。长视频中的时间片段检索、三维几何测量、外部数据库查询。这些任务可以由专门的算法或工具完成。

因此，工具辅助推理（Tool-augmented Reasoning）的基本思想是，使模型根据任务需要调用外部工具，并利用工具返回的信息继续推理。这种方式并不要求单一模型内部完成所有操作。例如，对于一张图表中的增长率问题，系统可以先使用视觉模型提取数值，再调用计算程序执行公式，最后生成自然语言回答。其处理过程可以概括为：

任务理解 → 工具选择 → 参数构建 → 工具执行 → 结果检查 → 答案生成。工具辅助推理的主要价值，是将适合不同处理机制的操作交给相应工具执行。

但工具调用本身也可能产生额外错误，因此需要对工具选择和输出结果进行验证。

推理与行动交替执行 ReAct[30] 是语言模型工具交互研究中的代表性工作之一。该方法探索将推理步骤与环境操作交替组织起来，使模型能够根据当前信息决定下一步操作，并利用外部观察结果更新后续判断。例如：模型发现某个问题需要额外信息。于是调用外部工具获取数据。工具返回结果。模型根据新信息继续推理。这种方式与单次生成完整答案不同。它允许模型在求解过程中逐步获取外部信息。在多模态任务中，这一思想可以扩展到视觉搜索、局部图像读取和历史片段检索。

例如，系统可能先根据低分辨率图像判断需要观察某个局部区域，再调用高分辨率视觉工具重新读取相关内容。不过，工具交互需要注意信息来源和工具权限。外部结果可能包含错误、噪声或不可信内容。因此，模型不能简单将任何工具返回的文本

都视为已经验证的事实。

外部计算工具与视觉数学推理 对于包含精确数值计算的任务，可以使用计算程序提高运算环节的可靠性。例如，一张图表提供：2023 年销售额：120 万元。2024 年销售额：150 万元。模型需要计算增长率。如果正确提取原始数据，并使用：

$$r = \frac{150 - 120}{120} \times 100\% \tag{13.43}$$

就可以得到：$r = 25\%$ 将这一计算交给确定性程序执行，可以减少部分算术错误。但外部计算工具只能保证其实现和输入条件所覆盖的计算过程。例如，模型如果将 150 误读为 180，那么计算程序仍然会根据错误输入生成错误结果。因此，视觉数学推理需要区分：原始数值提取、计算公式选择、计算过程执行、最终结果解释。工具能够提高某些环节的可靠性，但不能自动替代全部视觉理解能力。

工具输出的可信边界 外部工具可能返回不同类型的信息。例如，计算器返回数值结果。OCR 返回识别文本。检索系统返回文档片段。网页工具返回来自外部页面的内容。这些结果具有不同的可信性和适用范围。例如，确定性计算程序对正确输入的基本算术运算具有较强可靠性。但 OCR 结果仍然可能存在识别误差。外部检索得到的文本可能包含过时信息或错误陈述。因此，模型需要区分工具返回的内容和任务真正要求验证的事实。

此外，外部文本不应被直接视为具有更高执行权限的系统指令。在需要访问文件、网络或执行程序的系统中，还需要控制工具权限，避免不可信内容影响系统行为。因此，工具辅助推理不仅涉及能力扩展，也涉及数据来源、执行边界和系统安全性。

工具辅助推理的适用范围 工具辅助推理并不适合所有任务。对于简单对象识别，直接使用视觉语言模型可能更加高效。对于精确数值计算，外部程序可能具有明显优势。对于需要复杂空间关系的任务，可以考虑使用对象检测和几何工具。对于长视频历史查询，可以考虑先进行片段检索。因此，工具选择需要与任务需求相匹配。还需要考虑工具调用成本。例如，多次调用视觉检测器、OCR 和语言模型，可能显著增加推理时间。如果任务本身较简单，额外工具带来的收益可能不足以抵消成本。

因此，是否使用工具，应当通过与直接推理基线的公平比较决定。

13.5.4 推理成本与结果可靠性的权衡

推理时计算预算的基本概念 推理时计算预算通常指模型在完成单个任务时能够使用的计算资源。它可以通过不同方式衡量。例如：生成 token 数量。候选答案数量。视觉 token 数量。模型前向推理次数。外部工具调用次数。推理延迟。GPU 计算量。显存与其他资源占用。这些指标之间具有联系，但并不完全等价。例如，两个推理策略可能生成相同数量的语言 token，但其中一个还进行了额外的视觉编码和工具调用。

因此，仅使用输出 token 数量，不一定能够完整描述多模态推理的实际成本。更合理的方式是根据任务类型，报告与系统运行相关的主要计算资源。

推理时计算与训练时计算 训练时计算通常用于更新模型参数。例如，通过预训练、监督微调或强化学习，使模型获得更强的知识和推理能力。推理时计算则主要用于在模型已经确定的情况下，通过额外计算寻找更好的当前任务结果。例如：生成更长的推理过程。采样更多候选答案。使用验证器选择结果。重新观察图像局部区域。调用外部程序或检索工具。这些方法不一定需要修改基础模型参数。但某些推理时扩展系统可能需要在训练阶段构建专门的验证器、策略模型或其他辅助组件。

因此，不能简单地将“推理时扩展”理解为完全不需要额外训练。研究者应当分别报告基础模型训练、辅助模块训练和实际推理执行的成本。

自适应计算分配 固定推理预算并不一定是最有效的选择。例如，一个简单对象识别问题可能不需要多次采样。但一个复杂的多对象空间推理问题，可能需要局部视觉搜索和多个验证步骤。因此，可以根据问题难度动态分配推理计算。例如：简单问题使用直接回答。中等复杂问题使用分步求解。复杂问题使用多候选生成与验证。需要精确外部信息的问题使用工具辅助推理。

这种策略可以称为自适应推理预算分配（Adaptive Inference Budget Allocation）。从形式上，可以将计算预算表示为任务的函数：$B = B(X, Q)$，其中，X,Q 表示输入和问题。但实际系统需要判断任务是否真的需要更多计算。例如，可以根据问题类型、模型初始输出、验证器反馈或其他信号估计任务难度。模型自身对难度的判断也可能不准确。例如，它可能对错误答案具有过高置信度，从而过早停止。

因此，自适应策略需要评价预算分配是否合理，以及是否会遗漏需要进一步检查的困难样本。

推理时扩展的多模态局限 近年来，多模态推理时扩展研究开始关注增加计算是否能够稳定改善视觉语言任务。相关研究表明，其效果通常具有任务依赖性。例如，对于复杂的视觉数学问题，额外推理步骤和外部验证可能带来一定收益。但对于基础感知错误，例如对象颜色识别错误、细小文字遗漏或空间关系混淆，仅增加语言推理长度未必能够解决。

Ahmadpour 等人在《Limits and Gains of Test-Time Scaling in Vision-Language Reasoning》中研究了不同推理时策略在视觉语言任务中的效果。该研究发现，推理时扩展的收益与模型类型和任务性质有关；部分开放模型中的迭代自我修正甚至可能降低表现，而外部验证在其测试条件下表现出相对稳定的价值。这说明推理时计算并不是能够普遍解决多模态错误的方法。

对于视觉模型，首先需要判断错误来自感知还是后续推理。如果关键视觉信息已经丢失，额外语言计算通常无法凭空恢复原始证据。因此，推理时扩展应当与感知能力诊断结合使用。

结果可靠性与置信度校准 推理时计算的一个潜在目标，是提高最终答案的可靠性。但模型生成多个一致答案，并不意味着这些答案一定正确。例如，多次采样可能共享相同的视觉错误。因此，候选一致性不能直接替代真实正确率。类似地，验证器给出的高分也不一定对应真实正确概率。因此，需要评价系统的置信度校准。例如，使用不

同置信度区间下的实际准确率，检查预测分数是否与经验正确率相符。对于支持拒答的系统，还可以评价覆盖率与错误率之间的关系。

例如，当系统只回答较高置信度的问题时，错误率可能下降，但有效回答数量也可能减少。因此，需要同时报告：回答准确率、错误回答率、有效回答覆盖率、必要的校准指标、推理计算成本。不能只报告“验证后准确率提高”，却忽略大量样本被拒答或额外计算资源增加的情况。

推理时计算的停止策略 当模型采用多步推理、迭代修正或搜索策略时，还需要决定何时停止。例如，系统可以设置固定推理步数。也可以在验证器认为结果达到一定条件后停止。还可以根据新增计算是否带来明显收益决定是否继续。一种概念性的停止条件为：$\Delta S_k < \epsilon$，其中，ΔS_k 表示第 k 步之后的预期性能收益，ϵ 表示设定阈值。但实际系统通常无法直接观察真实的性能提升。

因此，停止策略可能依赖验证器评分变化、候选一致性或其他代理指标。代理指标可能并不可靠。例如，多个候选高度一致，可能来自共同的错误先验。因此，停止条件需要结合任务类型进行验证。对于实时任务，还可以设置严格的最大延迟或最大计算预算，避免推理过程无限扩展。

不同任务对推理时计算的需求 多模态任务对额外推理计算的需求存在明显差异。例如，对于清晰图像中的单一对象颜色识别，直接推理可能已经足够。对于复杂对象关系问题，可以通过分步筛选和区域定位提高可检查性。对于视觉数学问题，可以结合多步计算和确定性工具。对于长视频问题，可以采用历史片段检索，而不必直接增加所有视频帧的输入。对于物理场景预测，可以根据任务需要调用几何估计或状态预测模型。因此，不同任务应优先使用与其信息瓶颈相匹配的推理策略。

例如，如果问题主要受限于低分辨率视觉输入，可以增加视觉计算。如果问题主要受限于逻辑运算，可以增加推理或验证计算。如果问题主要受限于历史信息缺失，可以增加检索与记忆访问。如果问题已经能够通过直接推理可靠解决，则不一定需要复杂的额外机制。

习题与讨论

1. 为一道图表问题建立证据—变量—计算—答案链，并设计能分别定位四个环节错误的对照。
2. 给出仅凭先后顺序无法确定因果的例子。说明控制未来可见性解决了什么，未解决什么。
3. 对一个候选生成与验证系统，提出固定总生成 Token 预算的比较方法，并处理候选长度不同的情况。
4. 模型给出的长解释与答案不一致。设计一个只检查可验证中间量的评价，避免用文字长度代表推理深度。

第十四章　评测协议与研究实验设计

评价协议定义了一个分数能够支持什么结论。候选集合、数据隔离、提示、解码、工具和预算改变时，即使指标同名，也可能估计不同对象。本章集中处理这些通用规则，其他章节只保留任务特有的控制变量。

学习目标　明确评价总体与估计量；建立公平基线、消融和不确定性报告；识别污染与裁判偏差；按任务选择可解释指标。

14.1　估计什么、如何划分

先把研究主张写成可检验对象：在哪个任务分布、哪种输入可见性、哪项资源约束下，改变哪个因素改善哪个结果。数据划分和指标应服务这个主张。平均分通常只是一个加权汇总，需要说明权重与各组样本数量。

14.1.1　任务定义与评价指标

明确评测目标　开展模型评测之前，首先需要明确究竟希望测量什么能力。例如，研究者可能希望评价模型的图像识别能力、视觉推理能力、视频时间理解能力，或者跨模态信息融合能力。这些目标虽然存在联系，但并不相同。例如，对于一张包含汽车和行人的图像，可以设计不同类型的问题。“图中有几辆汽车？”主要涉及对象识别和计数。“哪辆汽车距离行人最近？”涉及对象定位和空间关系判断。“如果行人继续向前行走，可能遇到什么风险？”则进一步涉及动态状态和条件推理。

因此，同样使用图像输入，不同问题可能对应不同能力要求。如果研究目标是评价空间推理能力，却选择了大量可以通过简单对象识别回答的问题，就可能导致评价结果不能充分反映目标能力。因此，评测设计应当从明确的能力定义出发，而不是仅根据数据集是否流行或使用广泛进行选择。

模型能力与具体任务的区别　在多模态研究中，需要区分能力（Capability）与任务（Task）。能力通常指模型能够完成的某种信息处理要求，例如视觉识别、时间理解、关系推理或数学计算。任务则是将这种能力转化为具体输入输出形式的评价问题。例如，空间推理能力可以通过不同任务进行评价。一种任务是判断两个物体之间的左右关系。另一种任务是估计物体之间的实际距离。还有一种任务是根据三维场景判断机器人是否能够到达目标位置。

这些任务都涉及空间理解，但对模型的要求并不完全相同。因此，单一任务成绩只能反映模型在特定评价条件下的表现，不能自动代表其全部空间智能能力。类似地，视频问答可能包含场景描述、事件定位、时间顺序和因果解释等不同问题。如果只报

告整体视频问答准确率，就可能掩盖不同能力之间的差异。因此，评测体系通常需要建立任务与能力之间的对应关系。

构建能力评价维度 对于通用多模态大模型，可以根据研究目标组织多个评价维度。例如：这些维度构成一种面向能力的评价框架。但它们之间并不是完全独立的。例如，视觉推理通常依赖基础感知，空间推理可能同时涉及几何关系和语言理解。因此，评价时不应将每个指标简单视为某种完全独立的内部能力。更合理的方式是明确任务所需的主要能力，并通过必要的对照实验分析其他能力的影响。

评价指标的基本作用 评价指标（Evaluation Metric）用于将模型预测结果与参考信息进行比较，并转化为可分析的数值。不同任务通常需要不同指标。例如，对于具有唯一类别标签的分类任务，可以采用准确率：

$$\text{Accuracy} = \frac{N_{\text{correct}}}{N_{\text{total}}} \tag{14.1}$$

其中，N_{correct} 表示正确预测数量，N_{total} 表示评价样本总数。准确率简单直观，但并不适合所有任务。例如，在类别严重不平衡的情况下，模型可能始终预测多数类，从而获得较高准确率，却无法有效识别少数类。因此，还可能需要使用精确率、召回率和 F1 等指标。精确率定义为：

$$\frac{TP}{TP + FP} \tag{14.2}$$

召回率定义为：

$$\text{Recall} = \frac{TP}{TP + FN} \tag{14.3}$$

其中：

- TP 为真正例；
- FP 为假正例；
- FN 为假负例。

F1 则为：

$$F_1 = \frac{2PR}{P + R} \tag{14.4}$$

其中，P 和 R 分别表示精确率与召回率。这些指标能够从不同角度反映分类或检测系统的表现。但其适用性仍然取决于任务定义。例如，对于文本生成或几何距离估计，通常需要采用其他指标。

聚合指标与分组指标 对于包含多个子任务的数据集，研究者通常需要将不同结果汇总为整体成绩。但不同聚合方法可能得到不同结论。例如，假设一个基准包含两个类别。

类别 A 有 900 个样本。类别 B 有 100 个样本。模型在类别 A 上的准确率为 90%，在类别 B 上的准确率为 50%。如果按照样本数量加权，整体准确率为：

$$\frac{900 \times 0.9 + 100 \times 0.5}{1000} = 86\% \tag{14.5}$$

如果先分别计算两个类别的准确率，再进行等权平均：

$$\frac{90\% + 50\%}{2} = 70\% \tag{14.6}$$

可以看到，两种结果存在明显差异。前者反映当前样本分布下的整体表现。后者强调不同类别具有相同权重。因此，研究者需要根据任务目标选择适当的聚合方式。对于类别不平衡或不同场景表现差异较大的任务，建议同时报告整体成绩和分组成绩。例如，可以按照图像分辨率、任务类别、视频长度、语言类型或场景条件分别统计。宏平均与微平均的具体定义应以对应指标和任务协议为准，不能将不同形式的平均结果混为一谈。

14.1.2 数据集划分与测试协议

数据集在评测中的作用 数据集是模型评价的基础。一个具有代表性的测试集，应当包含与目标任务相匹配的输入和标注，并能够有效区分不同模型的能力。例如，如果研究目标是细粒度视觉定位，测试数据应当包含需要对象区分和局部关系判断的样本。如果研究目标是视频时间推理，测试数据应当包含真实依赖事件顺序和状态变化的问题。如果研究目标是音视频融合，测试样本应当能够体现不同模态之间的互补关系。因此，数据集选择需要与能力目标保持一致。

不能因为某个数据集规模较大，就默认它能够全面评价模型的全部能力。例如，一个大规模图像分类数据集可能很好地覆盖对象类别，却不能充分评价复杂视觉推理。同样，一个视频问答基准可能包含大量场景描述问题，但并不一定对精细时间顺序具有较强区分能力。因此，数据集的任务内容和能力覆盖范围，与样本数量同样重要。

训练集、验证集与测试集 在常见机器学习实验中，数据通常划分为训练集、验证集和测试集。

训练集（Training Set）主要用于更新模型参数。

验证集（Validation Set）主要用于选择模型、调整超参数或确定其他实验配置。

测试集（Test Set）主要用于在模型和实验方案确定后，评价最终泛化性能。

可以表示为：

$$D = D_{\text{train}} \cup D_{\text{val}} \cup D_{\text{test}} \tag{14.7}$$

对于相互独立的划分，通常要求：$\varnothing$ 并且：$\varnothing$，但仅保证样本文件名不同，并不一定意味着不同集合在信息层面相互独立。例如，同一段视频的相邻帧可能被分别划分

到训练集和测试集。虽然文件不同，但它们可能包含高度相似的场景和对象。因此，数据划分需要进一步考虑样本之间的关联结构。

分组划分与相关样本 假设一个视频数据集包含 20 段独立录制。每段录制提取 500 个时间片段。则总样本量为：$N = 20 \times 500 = 10000$，但这并不意味着 10000 个片段都能够视为相互独立的观测。同一录制中的片段可能共享相机、背景、人物和环境条件。如果模型在某些录制中表现特别好，而在另一些录制中表现较差，那么只报告片段级平均准确率，可能无法完整反映跨录制泛化表现。因此，可以采用录制级的数据划分。

例如，将部分完整录制用于训练，部分用于验证，其余用于测试。对于评测统计，还可以分别报告整体样本成绩和录制级成绩。录制级划分主要用于控制录制之间的信息重叠。如果不同录制来自同一设备或场景，仍然可能存在更高层级的相关性。因此，在需要评价跨设备或跨场景泛化时，可能还需要采用更加严格的分组方式。

分布内测试与分布外测试 测试集不仅需要具有独立性，还需要明确与目标应用场景之间的分布关系。例如，一个模型可能在晴天道路图像上表现较好，但在夜间、雨雾或其他条件下性能下降。类似地，语音模型可能对标准录音具有较高识别准确率，却难以处理强噪声或不同口音。因此，可以区分：

分布内测试（In-distribution Evaluation）主要评价模型在与训练或开发数据相近条件下的表现。

分布外测试（Out-of-distribution Evaluation）主要评价模型在场景、设备、语言、环境或其他条件发生变化时的表现。

两类评价回答的问题不同。分布内测试可以衡量模型在既定任务分布中的性能。分布外测试则用于分析模型在新条件下的泛化能力。但“分布外”并不是一个完全固定的概念。例如，新的拍摄角度、新的城市和新的传感器设备，可能对应不同程度的分布变化。因此，研究者需要明确具体改变了哪些因素，而不能仅使用“泛化测试”这一笼统表述。

视频与音频协议的特殊要求 对于视频评测，输入采样策略可能显著影响结果。例如，一段视频持续十分钟。模型 A 每秒采样一帧。模型 B 每五秒采样一帧。如果任务涉及短暂动作，二者能够观察到的信息可能不同。因此，需要明确视频帧率、最大帧数、采样位置和时间戳处理方式。类似地，音视频联合任务需要明确是否提供音频、字幕或自动转写结果。Video-MME 通过区分不同视频时长和模态输入条件，体现了这类协议控制的重要性。

在音频任务中，还需要记录采样率、声道处理、音频截断和语音活动检测等设置。例如，如果一个模型使用完整音频，另一个模型只处理前 30 秒，那么二者可能获得不同的信息。因此，音视频评测不能只报告使用了同一数据集，还需要明确实际输入内容是否一致。

评价样本与评价单位 多模态基准中的一个重要问题是：什么才是统计和评分的基本单位？例如，一张图像可能对应多个问题。一段视频可能包含几十个事件片段。一个文档可能包含多个字段查询。如果把每个问题都作为独立样本，某些图像或视频可能在

整体成绩中具有更大权重。例如，视频 A 对应 20 个问题，视频 B 对应 2 个问题。按问题级平均时，视频 A 的贡献更大。但按视频级平均时，两段视频可以具有相同权重。因此，研究者需要根据评价目标明确单位和权重。

例如，若希望评价所有问题的整体回答正确率，可以使用问题级平均。若希望评价不同独立视频上的平均表现，可以先计算视频内成绩，再对视频进行平均。这两种方法都可能合理，但回答的问题不同。因此，应当明确指标聚合方式，并在必要时同时报告两类结果。

14.2 公平基线、消融与统计

算例：成对差异

两个系统在同一批 n 个问题上评价，记逐题分数差为 d_i。报告 $\bar{d} = n^{-1} \sum_i d_i$ 时，应保留成对结构；独立重采样两个系统的分数会丢失共享问题难度的信息。若多个问题来自同一文档或视频，重采样单位还应考虑该聚类关系。随机种子差异与测试样本差异是两种不确定性，不能只测其中一种就声称全部稳定。

14.2.1 基线选择与公平对比

基线的基本作用 基线（Baseline）是用于判断新方法是否具有实际价值的比较对象。例如，研究者提出一种新的 RGB-Event 融合方法。如果只报告融合模型的最终准确率，就无法判断新增事件信息和融合机制是否真正带来了收益。因此，需要设置适当的比较方法。例如：RGB-only。Event-only。简单特征融合。已有成熟融合模型。提出的新方法。通过比较这些设置，可以分析新方法相对于不同方案的性能差异。

具体实验不一定必须包含全部类型的基线。应根据研究问题选择具有代表性且可以公平实现的方案。基线的目的不是使新方法显得更好，而是为研究结论提供可信的参照。

基线应当回答明确的问题 不同基线通常具有不同作用。例如：简单基线用于判断复杂方法是否优于直接且低成本的处理方式。单模态基线用于判断新增模态是否提供实际收益。成熟公开方法用于比较新方法与已有研究的差距。消融基线用于判断某个新增模块是否真正有效。系统成本基线用于比较不同方法在相似资源预算下的效率。例如，如果研究者提出一种具有长期记忆机制的视频推理模型，可以设置：不使用历史信息的模型。直接使用最近若干帧的模型。

使用简单历史平均或检索的模型。使用新增记忆机制的模型。这样可以判断，性能提升究竟来自历史信息本身，还是新增的复杂记忆模块。因此，基线应当围绕研究假设进行设计，而不是仅按照模型发布时间或排行榜名次选择。

预训练权重与训练预算 多模态模型的性能通常受到预训练数据和训练预算影响。例如，一个模型可能使用规模较大的图文数据进行预训练。另一个模型可能只使用较小

的监督数据集。即使两者采用相似结构，性能差异也可能受到数据规模和预训练目标影响。因此，在比较模型时，需要明确：是否使用预训练权重。预训练数据的大致范围和来源。是否包含与目标任务相近的数据。是否使用额外的合成数据或蒸馏监督。是否进行了任务相关微调。

如果研究目标是分析某种新架构的有效性，可以尽量在相同预训练条件下开展对照实验。如果研究目标是比较可获得模型的实际性能，则可以允许不同模型使用各自公开权重，但需要明确说明预训练条件并不相同。因此，不能将不同预训练条件下的性能差异全部归因于模型架构。

推理计算预算的公平性 近年来，部分模型通过较长思维链、多候选采样或工具调用改善任务表现。因此，推理阶段的计算预算也成为模型比较的重要因素。例如，模型 A 只生成一次答案。模型 B 生成十个候选，再通过验证器选择结果。如果只比较最终准确率，而不报告候选数量和额外计算成本，就可能难以判断性能提升的实际代价。因此，评测时可以记录：最大输出 token 数量。实际输出 token 数量、采样温度、候选数量、推理轮数、工具调用次数。

视觉重新编码次数。端到端推理时间。必要时，还可以同时报告固定推理预算下的比较，以及模型使用各自推荐设置时的比较。这有助于区分基础模型能力和额外推理计算带来的系统收益。

基线选择的完整性 如果研究者提出某种复杂模型，却只与明显较弱的简单方法进行比较，就可能高估其实际创新价值。例如，提出新的长视频记忆模型，只与不使用历史信息的单帧模型比较。即使获得明显提升，也不能证明新增复杂记忆机制优于简单的历史帧检索或时间特征聚合。因此，合理的基线应当覆盖不同复杂度和相关技术路线。通常至少需要考虑：一个能够说明任务基本难度的简单方法。一个与新方法直接相关的成熟公开方法。一个用于验证核心创新贡献的消融对照。

但这并不意味着每项研究都需要实现大量无关模型。基线选择应当围绕研究假设，以代表性和可解释性为主要原则。同时，需要尽可能避免只选择对新方法有利的比较对象。

公平对比的结论边界 即使实验使用相同测试集，也需要根据协议差异限定结论范围。例如，新方法在某个特定数据集上优于基线，并不意味着它在全部场景中都具有优势。如果实验只覆盖一种语言或某个设备，也不能直接推断模型能够适应所有语言和设备。如果只报告单次运行结果，也不能直接判断性能差异是否稳定。因此，实验结论需要与证据范围保持一致。例如：“在统一测试协议和固定输入预算下，方法 A 在所评测数据集上优于方法 B。”这一表述通常比：

“方法 A 在多模态推理方面全面优于方法 B。”更加符合实验结论的边界。因此，公平对比不仅是实验配置问题，也是科研结论的解释问题。

14.2.2 可复现性与统计可靠性

可复现性的基本含义 可复现性（Reproducibility）是科学研究的重要要求。在机器学习研究中，可复现性通常强调其他研究者能否根据论文和公开材料，重新执行相应实验，并获得与原始研究相一致的结果或结论。学术领域中对 reproducibility、replicability 等术语的定义可能有所差异。在本节中，可以从两个实际层次理解。计算层面的可复现性强调实验程序、模型配置和数据处理流程能够被重新运行，并得到可比较的数值结果。

研究结论层面的可重复验证性强调在独立运行或合理改变实验条件后，主要研究结论是否仍然成立。例如，研究者公开代码、权重和配置后，其他人能够重复运行模型，这属于重要的工程基础。但如果某个性能提升只在单一随机种子或少量特定样本上出现，就仍然需要进一步验证其稳定性。因此，可复现性既涉及工程条件，也涉及实验结果的科学可靠性。

随机种子与实验波动 机器学习实验通常包含随机因素。例如，模型参数初始化、数据打乱、数据增强和部分计算操作都可能具有随机性。因此，即使使用相同模型和数据，不同训练运行也可能得到不同结果。如果只报告一次训练结果，就可能无法判断观察到的性能提升是否稳定。例如，某个新方法在单次运行中比基线高出 0.5 个百分点。但如果不同随机种子下的性能波动达到 1 个百分点，那么仅凭单次实验很难判断这 0.5 个百分点是否代表稳定收益。

因此，可以使用多个随机种子重复训练，并报告均值和标准差。假设在 K 次独立运行中获得指标：$S_1, S_2, \ldots, S_K$，则均值为：

$$\bar{S} = \frac{1}{K} \sum_{k=1}^{K} S_k \tag{14.8}$$

样本标准差为：

$$s = \sqrt{\frac{1}{K-1} \sum_{k=1}^{K} (S_k - \bar{S})^2} \tag{14.9}$$

均值可以反映多次运行的平均表现。标准差可以描述不同运行之间的波动程度。多随机种子实验通常主要评价训练或随机推理机制带来的运行差异。它并不能直接替代对测试样本不确定性的统计分析。

Bootstrap **重采样** Bootstrap 是一种常用的统计重采样方法。其基本思想是，从已有测试样本中进行有放回抽样，构建多个重采样数据集，并在每个数据集上重新计算评价指标。例如，原始测试集包含 N 个样本。通过有放回抽样构建第 b 个重采样数据集：

$$D^{(b)} \tag{14.10}$$

然后计算：

$$S^{(b)} = \mathcal{E}(M, D^{(b)}) \tag{14.11}$$

重复 B 次，可以得到：

$$S^{(1)}, S^{(2)}, \ldots, S^{(B)} \tag{14.12}$$

这些结果形成经验重采样分布。随后，可以根据适当的 Bootstrap 区间方法估计置信区间。例如，在简单的百分位数 Bootstrap 方法中，可以使用经验分布的相应分位数构造区间。但 Bootstrap 并不是在所有情况下都能够直接使用。例如，如果测试样本之间具有较强相关性，简单逐样本重采样可能低估不确定性。因此，需要根据数据的实际结构选择重采样单位。

录制级与场景级重采样 对于视频、音频和连续感知任务，测试样本可能具有明显的分组结构。例如，某个数据集包含 15 段独立视频。每段视频提取数百个时间片段。如果将所有片段当作独立样本进行 Bootstrap，可能低估不同录制之间的性能波动。因此，可以采用分组 Bootstrap 或聚类 Bootstrap。其基本思路是，以独立录制或其他合适的组为重采样单位。例如，每次从 15 段录制中有放回抽取 15 段，并保留所抽中录制中的相关样本。

随后重新计算整体指标。这种方式能够在一定程度上保留组内样本的相关结构。具体重采样方案还应当与指标的聚合方式一致。如果评价目标是所有片段的总体准确率，可以在保留录制结构的基础上计算片段级汇总指标。如果评价目标是独立录制上的平均表现，则可以先计算每段录制的成绩，再对录制级成绩进行统计分析。因此，统计单位应当与研究希望推广的总体保持一致。

多随机种子与测试集置信区间的区别 多随机种子实验和测试集置信区间经常同时出现在机器学习论文中，但它们回答的问题不同。多随机种子主要反映模型训练或随机推理过程的运行波动。测试集置信区间则主要反映基于有限评价数据进行性能估计的不确定性。例如，即使一个模型在三个随机种子下得到完全相同的结果，也不能说明其在未来新数据上的准确率没有不确定性。反过来，即使测试集样本很多，某种训练方法也可能对随机初始化高度敏感。因此，二者不应相互替代。对于重要研究，可以同时报告：

多个随机种子的平均性能和标准差。固定模型在测试集上的置信区间。新方法与基线之间的配对性能差异及其区间估计。但如果多随机种子模型和测试样本同时存在依赖结构，统计方案也需要相应调整。不能简单将所有种子与样本结果直接拼接，并将其视为完全独立的观测。

统计显著性不等于因果归因 即使某个方法相对于基线具有统计上可靠的性能提升，也不能直接证明提升一定来自研究者提出的核心机制。例如，新方法同时增加了参数量、训练数据和输入分辨率。即使最终性能显著提高，也无法仅凭这一结果确定主要原因。因此，统计显著性与机制归因是两个不同问题。统计分析主要帮助判断观察到的性能

差异是否可能由抽样或随机波动解释。机制归因则需要通过受控实验和消融分析，判断某个设计因素是否对结果产生实际贡献。

例如，如果研究者提出新的跨模态融合模块，应当尽可能控制其他变量，再比较有无该模块的结果。如果还希望证明该模块优于简单融合方式，就需要加入相应的强基线。因此，可信的科研结论通常需要同时具备公平实验设计和合理统计分析。

多次比较与选择偏差 在模型研究中，研究者可能同时尝试多个结构、多个超参数和多个随机种子。如果只报告成绩最高的一次实验，就可能产生选择偏差。例如，在 20 个候选方案中选择测试成绩最高的一个，即使这些方案之间真实性能差异很小，也可能因为随机波动而出现明显的最高值。因此，应尽量在验证集上完成方案选择。对于需要同时检验多个研究假设的情况，还应考虑多重比较带来的错误发现风险。

例如，可以根据研究目的采用适当的多重检验校正方法，或者预先明确主要比较与探索性比较。但不应机械地认为所有实验都必须使用同一种校正方法。具体统计方法应根据研究假设、比较数量和评价目标确定。关键是避免在大量实验中选择最有利结果，然后将其描述为事先明确且独立验证的结论。

14.2.3 消融实验与训练预算控制

消融实验的目的 消融实验（Ablation Study）通过移除、替换或单独改变某个设计因素，分析其对结果的影响。例如，研究者可能希望判断新增连接模块是否有效、图文匹配目标是否提供额外收益，或高分辨率图像训练是否确实改善下游能力。

理想的单因素对照要求：除了待比较因素外，其余关键条件尽可能一致。对于多模态预训练，这些条件通常包括模型初始化、训练数据及划分、随机种子、有效批量大小、更新步数、优化器、输入分辨率、视觉 Token 数和评价协议。但固定训练步数并不必然等于固定计算量。例如，更高分辨率或更复杂的连接模块可能显著增加每步计算，因此需要根据研究问题决定等步数、等 Token 还是等计算预算的比较。

基线与实验变量 假设研究者以一个冻结视觉编码器和可训练连接模块的视觉语言模型为基线，并提出增加图文匹配辅助目标的方法。可设计：这一表格仅给出实验设计，不预设任何性能结果。若加入 ITM 同时改变了负样本采样、训练数据或总计算预算，则应明确记录这些变化，避免将所有收益归因于损失函数本身。

交互效应与组合消融 单因素结果有时不足以说明两个设计能否共同发挥作用。例如，增加视觉 Token 数可能在冻结视觉编码器时改善有限，但在视觉编码器同时解冻后更有效。此时可以设计一个二因素实验：其中 S_{ab} 表示预先指定指标的实测结果。交互效应可以用差分中的差分表示：

$$\Delta_{\text{interaction}} = (S_{11} - S_{10}) - (S_{01} - S_{00}). \tag{14.13}$$

若该值明显偏离零，提示两种因素的组合收益不等于其单独收益的简单相加。但是否具有统计可靠性，还需要考虑样本波动、随机种子与评价集组成。

多随机种子与统计不确定性 训练初始化、采样次序和随机增强都可能影响结果。对于计算允许的实验，应使用多个独立随机种子，并报告均值及离散程度。对于固定模型的下游评价，也可以采用与数据组织方式一致的配对 Bootstrap 等方法估计区间。

例如，视频任务如果同一录制中的帧高度相关，就不宜简单将所有帧视为完全独立的抽样单位；应根据研究协议考虑按视频或录制分组进行不确定性估计。报告单次试验中 0.1 个百分点的提升，而不说明波动范围，通常不足以证明某个模块产生了稳定收益。

消融实验的证据边界 在某个数据集与训练预算下观察到“去掉模块后性能下降”，可以支持该模块在当前系统中具有作用，但并不能自动证明其实现机制与研究者预期一致。例如，新增模块可能增加参数量和计算预算，其收益可能来自容量变化，而不是特定归纳偏置。因此，必要时还应构造参数量、计算量相近的替代模块，并对关键中间表示进行诊断。

此外，不应仅报告有利于提出方法的任务。若一个设计改善文本生成但降低细粒度定位，应如实呈现这种权衡。完整的消融结论应同时给出有效范围、代价、失败条件及尚未隔离的混杂因素。

14.3 感知、检索与定位指标

14.3.1 图像理解与视觉问答

图像理解评测的基本任务 图像理解（Image Understanding）是视觉语言模型和多模态大模型的重要基础能力。其评价范围通常包括对象识别、场景描述、属性理解、对象关系、数量判断以及复杂视觉推理等方面。例如，给定一张包含人物、汽车和交通信号灯的图像，可以设计不同类型的问题：“图中是否存在汽车？”主要评价对象存在性识别。“图中有几辆汽车？”主要评价对象计数。“最左侧汽车是什么颜色？”主要评价对象定位和属性绑定。“行人位于哪辆汽车前方？”

主要评价对象关系与空间理解。“根据交通信号灯的状态，车辆应当如何通行？”则进一步涉及视觉信息、任务条件和交通规则之间的联合推理。这些问题虽然都采用图像输入，但所需能力存在明显差异。因此，图像理解评测应当明确能力分类，而不是仅使用一个总体问答准确率描述全部能力。

视觉问答的基本形式 视觉问答（Visual Question Answering，VQA）通常要求模型根据输入图像 I 和自然语言问题 q ，生成或选择答案：$\hat{y} = f_\theta(I, q)$，其中：

- I 表示输入图像；
- q 表示问题；
- f_θ 表示待评价模型；
- $\hat{y}$ 表示预测答案。

根据答案形式，视觉问答可以分为不同类型。

开放式问答要求模型生成自由形式的答案。例如：问题：“图中最右侧的人穿什么颜色的衣服?” 回答：“蓝色。” 多项选择问答要求模型从给定候选中选择正确答案。例如：A. 红色 B. 蓝色 C. 黄色 D. 黑色两种任务的评价方式有所不同。多项选择题通常可以根据标准选项直接计算准确率。开放式问答则可能需要处理大小写、词形变化、同义表达和其他答案形式差异。因此，在比较模型时，需要明确答案生成形式和具体评分规则。

VQA 数据集及其评价方法 VQA 是视觉语言研究中的经典基准之一。该数据集以真实图像和人工编写的问题为基础，覆盖对象、属性、数量和场景理解等内容，并为每个问题提供多个人工参考答案。VQA v2 进一步针对语言偏差问题构建更加平衡的数据，使部分问题不能仅凭常见答案分布轻易解决。例如，对于相同或相近的提问方式，数据集中可能包含具有不同正确答案的图像。这有助于减少模型只依赖语言先验回答问题的倾向。

与普通分类准确率不同，经典 VQA 评价考虑了多名人工标注者之间的答案差异。其常见近似表达为：

$$\mathrm{Acc}(a) \approx \min\left(\frac{n_a}{3}, 1\right) \tag{14.14}$$

其中，n_a 表示与预测答案 a 一致的参考答案数量。这一表达用于说明多名人工答案一致程度对评分的影响。正式 VQA 评价采用官方实现中针对人工答案子集的计算与规范化规则，应以对应版本的评测代码为准。例如，如果某个问题的多个参考答案中，只有少数标注者采用某一表达，那么预测该表达可能获得部分分数。如果较多标注者给出相同答案，则预测该答案通常能够获得较高分数。

VQA 的答案规范化规则可能包括大小写处理、数字转换和部分标点处理等。因此，同一组模型输出，如果使用不同的答案解析程序，可能得到不同成绩。这说明视觉问答的评测必须同时明确数据集版本和评分实现。

视觉依赖性评价 图像问答的一个特殊问题是：模型能否不看图像，仅根据问题文本推测答案？例如：“草地通常是什么颜色?” 模型可以根据常识回答“绿色”。但如果实际图像中草地呈现其他颜色，语言先验就可能导致错误。MMStar 是面向视觉必要性评价的重要基准。该研究强调筛选需要视觉信息才能可靠回答的问题，以减少部分传统多模态基准中语言线索过强的问题。一种常见诊断方法是比较完整输入与无图像输入条件。可以定义：

$$S_{\text{image+text}}, \quad S_{\text{text}} \tag{14.15}$$

其中，两个成绩应采用相同或可比较的任务评分规则。如果完整输入明显优于无图像输入，说明视觉信息在该测试条件下提供了额外价值。但这并不自动证明模型准确利用了所有视觉细节。因此，可以进一步采用视觉反事实测试。例如，保持问题不变，只改变图像中汽车的颜色或数量，检查模型答案是否随之正确变化。

图像理解评测的主要局限 图像理解评测的困难在于，不同任务之间存在能力重叠。例如，数量问题不仅需要计数，还依赖对象识别。空间问题不仅需要理解位置，还可能依赖正确的实例绑定。因此，最终问答准确率往往无法直接分离不同能力。此外，模型可能依赖训练数据中的语言偏差。答案解析方式也可能影响开放式问答成绩。对于具有较高风险的任务，还需要检查模型是否产生对象幻觉、错误属性或缺乏证据的解释。

因此，图像理解评测最好结合综合成绩、分任务结果、视觉必要性测试和典型错误分析。

14.3.2 跨模态检索与匹配

跨模态检索的基本概念 跨模态检索（Cross-modal Retrieval）要求系统根据一种模态的查询，从另一种模态的数据集合中寻找相关内容。例如，用户输入：“一只狗正在草地上奔跑。”系统需要从大量图像中检索出符合描述的图片。反过来，用户也可以输入一张图片，从文本集合中寻找相关描述。因此，图文检索通常包含两个方向。文本检索图像（Text-to-Image Retrieval）：输入文本查询，返回相关图像。

图像检索文本（Image-to-Text Retrieval）：输入图像查询，返回相关文字描述。这两个方向的输入和候选集合不同，因此不能简单视为完全相同的任务。例如，一张图像可能具有多条合理描述，而一条文本描述在某个特定测试协议中可能只对应一张标注图像。因此，评价时应分别报告两个检索方向的成绩。

跨模态表示与相似度 跨模态检索通常需要将不同模态映射到可比较的表示空间。例如：$z_I = f_I(I)$，$z_T = f_T(T)$，其中：

- f_I 表示图像编码器；
- f_T 表示文本编码器；
- z_I 和 z_T 表示对应的特征向量。

随后，系统计算图像与文本之间的相似度。

例如，常用余弦相似度：

$$s(I,T) = \frac{z_I^\top z_T}{\|z_I\|_2 \|z_T\|_2} \tag{14.16}$$

在给定查询后，可以按照相似度从高到低对候选结果进行排序。CLIP[8] 是图文对比预训练的重要代表性研究。其核心思想是利用大量图文配对数据，学习图像与文本之间的语义对应关系。但图文表示相似度较高，并不代表模型已经正确理解图像中的全部细节。例如，包含相同对象但关系不同的图像和文本，也可能获得较高的语义相似度。因此，跨模态检索的评价不仅需要检查语义相关性，还可以进一步分析对象属性和组合关系的正确性。

Recall@K 指标 Recall@K 是图文检索中最常用的指标之一。在典型的单目标图文检索协议中，可以将其理解为：有多少比例的查询，其至少一个标注正确结果出现在前 K

个返回结果中。设共有 N 个查询，第 i 个查询的相关结果集合为 $\mathcal{R}_i$，模型返回前 K 个结果为 $\mathcal{T}_i^K$，则常见的查询成功率形式为：

$$R@K = \frac{1}{N}\sum_{i=1}^{N}\mathbb{I}\left[\mathcal{R}_i \cap \mathcal{T}_i^K \neq \varnothing\right] \tag{14.17}$$

例如：$R@1 = 70\%$ 表示 70% 的查询，其前 1 个检索结果中存在至少一个标注正确的结果。$R@5 = 90\%$ 表示 90% 的查询，其前 5 个结果中至少包含一个标注正确的结果。经典图文检索中的这种 Recall@K 通常属于查询级命中率。对于一个查询对应多个相关文档的一般信息检索任务，也可以定义“前 K 个结果中检回了全部相关结果的多少比例”。这两种定义并不完全相同，不能混用。

因此，使用 Recall@K 时应说明相关性标签和具体计算规则。

双向检索指标 在图文检索中，通常需要分别报告两个方向。例如：以上按正文整理评价方法与记录字段，未另设示意成绩；实际比较应使用原始测试结果。不同方向的结果可能具有明显差异。例如，一张图像可能对应五条描述，图像 $\rightarrow$ 文本的相关候选数量可能多于文本 $\rightarrow$ 图像。因此，两个方向的检索成绩受到相关结果数量和候选集合结构影响。

部分研究还报告 Rsum，即将两个方向的 R@1、R@5、R@10 相加：

$$Rsum = \sum_{K\in 1,5,10}\left(R_{I\rightarrow T}@K + R_{T\rightarrow I}@K\right) \tag{14.18}$$

Rsum 可以用于汇总双向检索表现。但由于它是多个指标的加和，可能掩盖不同方向的差异。因此，最好同时报告各方向的具体 Recall@K，而不是只提供 Rsum。

排序质量与其他检索指标 Recall@K 主要关注相关结果是否出现在前 K 个位置。但它不一定充分反映多个相关结果之间的完整排序质量。例如，两个模型可能都在前 5 个结果中找到一个正确图像，但相关结果的位置分别为第 1 位和第 5 位。在 R@5 下，两者都被计为成功。因此，对于某些任务，还可以考虑平均倒数排名（Mean Reciprocal Rank，MRR）。假设第 i 个查询的第一个相关结果排名为 r_i，则：

$$\text{MRR} = \frac{1}{N}\sum_{i=1}^{N}\frac{1}{r_i} \tag{14.19}$$

MRR 更加强调第一个相关结果的排名位置。对于一个查询具有多个相关结果的任务，还可以使用平均精度均值 mAP。对于具有不同相关程度标签的检索任务，则可以考虑归一化折损累计增益 nDCG。这些指标适用于不同的相关性定义。如果数据集只提供有限的图文配对标注，那么某些未被标注为相关的结果，实际上也可能具有合理语义对应。因此，检索评价还受到相关性标注完整性的限制。

跨模态检索的公平性问题 跨模态检索评测还需要控制若干特殊因素。首先是检索方向和候选集合。不同模型需要使用相同的候选数据及相关性标注。其次是文本预处理。

例如，是否进行截断、大小写转换或其他归一化。再次是特征提取和相似度计算方式。例如，是否进行特征归一化，是否使用额外的重排序器。此外，还需要明确是否允许利用测试集合中的其他信息进行适配。例如，如果模型在测试前使用了候选集合进行任务相关训练，那么这一设置与直接零样本检索不同。

因此，跨模态检索的成绩应当结合数据划分、候选规模、特征处理和检索策略进行解释。检索评价不仅需要关注相关内容是否被找到，还应关注其排序位置、细粒度语义正确性和系统资源成本。

14.3.3 目标定位与细粒度感知

从图像级理解到区域级评价 图像级视觉问答主要评价模型是否能够根据图像生成正确答案。但在某些任务中，仅给出正确对象类别还不够。例如，用户询问：“图中穿红色衣服的人在哪里?”模型不仅需要识别人物和服装颜色，还需要明确目标人物在图像中的位置。类似地，对于自动驾驶场景中的行人检测，系统需要准确定位行人，而不只是回答“图中存在行人”。因此，目标定位与细粒度感知评测更加关注局部空间信息。这类任务通常需要检查：对象类别是否正确。对象实例是否正确。

目标位置是否准确。属性与对象之间的对应是否正确。是否遗漏或重复识别目标。因此，区域级评价能够提供图像级问答无法完整表达的空间精度信息。

目标检测的评价指标 目标检测（Object Detection）通常要求模型输出对象类别、边界框及相关置信分数。例如：

$$\hat{O} = (\hat{c}_i, \hat{b}_i, \hat{s}_i)_{i=1}^{N} \tag{14.20}$$

其中：

- $\hat{c}_i$ 表示预测类别；
- $\hat{b}_i$ 表示预测边界框；
- $\hat{s}_i$ 表示预测置信分数。

目标检测通常使用 IoU 判断预测框与真实框之间的空间重叠程度。

例如：

$$\mathrm{IoU} = \frac{|B_p \cap B_g|}{|B_p \cup B_g|} \tag{14.21}$$

如果 IoU 达到指定阈值，并且类别等条件满足评价协议要求，则预测可能被视为正确检测。但检测评价还涉及预测与真实对象之间的匹配规则、重复检测处理和置信分数排序。因此，检测 AP 并不是单纯计算所有预测框 IoU 的平均值。AP 通常基于不同置信阈值下的精确率—召回率关系计算。COCO 检测评测采用多个 IoU 阈值，并汇总得到常用 AP 指标。在正式报告中，应区分 AP、AP50 和 AP75 等不同指标。

例如，AP50 相对宽松，AP75 对定位精度提出更严格的要求。因此，一个模型即使在 AP50 上具有较高成绩，也不代表其边界框定位足够精确。

指代表达理解 指代表达理解（Referring Expression Comprehension，REC）要求模型根据自然语言描述，在图像中定位指定对象。例如：“站在蓝色汽车旁边、穿红色衣服的人。”模型需要综合对象类别、属性和关系确定目标。与普通目标检测不同，REC 的查询是自然语言表达。例如，两个人物可能属于相同类别，但语言通过衣服颜色和位置关系区分目标实例。因此，REC 不仅需要目标定位，还需要建立语言与区域之间的对应关系。

常见评价方式是判断预测边界框是否与真实目标框达到指定 IoU 阈值。例如，使用：IoU ≥ 0.5 作为正确定位条件。此时，定位准确率可以表示为：

$$\text{Acc@0.5} = \frac{1}{N}\sum_{i=1}^{N}\mathbb{I}\left[\text{IoU}(\hat{B}_i, B_i) \geq 0.5\right] \tag{14.22}$$

这一指标主要评价目标框是否达到指定空间重叠标准。它与 COCO 检测 AP 的计算方式不同。因此，不能将 REC Accuracy@0.5 与检测 AP50 直接视为同一种指标。

指代表达分割 指代表达分割（Referring Expression Segmentation，RES 或 RIS）进一步要求模型输出与语言描述对应的像素级目标区域。例如：“图中穿红色外套的人。”模型不仅要找到目标人物，还需要生成其像素级分割掩码。这类任务通常使用 IoU 等指标衡量预测掩码与真实掩码的重叠程度。例如，对于第 i 个样本：

$$\text{IoU}_i = \frac{|\hat{M}_i \cap M_i|}{|\hat{M}_i \cup M_i|} \tag{14.23}$$

可以根据样本级 IoU 计算平均值：

$$\text{mIoU} = \frac{1}{N}\sum_{i=1}^{N}\text{IoU}_i \tag{14.24}$$

部分指代分割研究还使用累计交并比（cIoU）。它可以表示为：

$$\frac{\sum_i |\hat{M}_i \cap M_i|}{\sum_i |\hat{M}_i \cup M_i|} \tag{14.25}$$

这两种指标的聚合方式不同。样本级 mIoU 对每个样本赋予相同权重。累计 cIoU 则受到不同样本区域大小和并集规模影响。因此，不能将 mIoU 和 cIoU 直接视为完全等价。此外，对于不存在目标的表达，部分任务还需要评价模型是否能够正确拒绝定位。这类评价应当使用相应数据集的专门协议。

短语定位与细粒度图文对齐 短语定位（Phrase Grounding）通常要求模型将文本中的多个名词短语或其他片段，对应到图像中的具体区域。例如，文本为：“一名男子骑着自行车经过街道。”模型可能需要定位：“男子”。“自行车”。“街道”。与只定位单一指定对象的 REC 相比，短语定位可能涉及多个文本片段与多个图像区域之间的对应关系。Flickr30K Entities 是这一领域的重要数据集。

该数据集在 Flickr30K 图像描述基础上增加实体短语与图像区域之间的标注，为研究区域级图文对应提供了基础。在评价时，可以使用 Recall@K 等指标，判断模型是否能够在前 K 个候选区域中找到与短语对应的目标。但某些短语可能对应多个对象实例或多个区域。例如：“两名男子”。可能需要关联两个不同的人物区域。因此，短语定位的评价需要根据标注定义，明确一个短语对应一个区域、多个区域还是区域集合。

不同数据集和实现之间的匹配规则可能存在差异。

目标定位评测中的协议差异 目标定位任务的成绩容易受到空间协议影响。例如，不同模型可能使用不同的图像缩放和裁剪方式。如果预测框坐标没有正确映射回原始图像，就可能出现系统性的定位偏差。此外，预测框可能采用：左上角与右下角坐标、中心点与宽高、归一化坐标、原始像素坐标。不同格式之间需要正确转换。对于多尺度图像，还需要考虑输入缩放和填充造成的坐标变化。因此，定位评测不仅需要正确模型输出，还需要严格检查坐标处理和后处理过程。

如果使用不同的图像空间或坐标定义计算 IoU，就可能得到无效的评价结果。因此，目标定位实验应当明确坐标系、图像尺寸和评价脚本，并在可能的情况下使用官方协议进行核验。

14.3.4 文档与图表理解

文档与图表评测的特殊性 文档与图表是多模态大模型的重要应用场景。与自然图像相比，这类输入通常具有更密集的文字信息、更明确的空间结构和更高的数值精度要求。例如，一份技术报告可能同时包含：标题和正文、多级段落、表格、公式、流程图、脚注和引用。用户可能询问其中某个具体字段，也可能要求模型结合多个区域进行计算和推理。因此，文档与图表理解评测不能只关注模型是否识别主要内容，还需要考虑文字识别准确性、版面结构、字段关联和数值运算等方面。

例如，一个模型能够概括财务报告的主要内容，却可能错误读取表格中的某个金额。对于一般摘要任务，这种局部错误可能不容易被发现。但对于财务计算和专业信息提取，它可能直接影响最终结论。因此，文档与图表评测通常需要更加严格的文本和数值评价。

OCR 评测 OCR（Optical Character Recognition）主要评价模型从图像中识别文字的能力。典型输入包括：扫描文档。自然场景中的文字、手写内容、表格中的数字、数学公式。与普通图像识别不同，OCR 对字符级错误通常更加敏感。例如：“120.5”与：“1205”只相差一个小数点，但数值含义完全不同。因此，OCR 评价常使用字符错误率（Character Error Rate，CER）。可以表示为：

$$\mathrm{CER} = \frac{S + D + I}{N} \tag{14.26}$$

其中：

- S 表示替换的字符数量；

- D 表示删除的字符数量；
- I 表示插入的字符数量；
- N 表示参考文本的字符数量。

对于以单词为单位的任务，可以使用词错误率 WER。

CER 和 WER 衡量的是文本识别差异，并不直接反映文本所表达的语义是否正确。例如，模型可能只错误识别一个数字，却导致整个计算结果错误。因此，OCR 任务还可能需要结合字段级正确率和下游任务准确率进行分析。

ANLS 指标 文档问答通常存在一定的文本识别差异。例如，正确答案可能为：“International Business Machines”模型可能输出：“International Business Machine”两种表达只存在很小的字符差异。因此，文档问答中常使用平均归一化 Levenshtein 相似度（Average Normalized Levenshtein Similarity，ANLS）。

对于预测文本 $\hat{y}$ 和参考文本 y ，可以定义归一化编辑距离：

$$\mathrm{NLD} = \frac{\mathrm{Lev}(\hat{y}, y)}{\max(|\hat{y}|, |y|)} \tag{14.27}$$

其中：

- Lev 表示 Levenshtein 编辑距离；
- $|\cdot|$ 表示字符串长度。

在常见 DocVQA 协议中，单样本相似度可以表示为：

$$s(\hat{y}, y) = \begin{cases} 1 - \mathrm{NED}(\hat{y}, y), & \mathrm{NED}(\hat{y}, y) < 0.5 \\ 0, & \text{其他情况} \end{cases} \tag{14.28}$$

当问题具有多个参考答案时，通常根据协议选择与预测最匹配的参考答案进行评分。最后对样本进行平均。正式评测还涉及大小写、空格和文本规范化等实现细节。因此，应使用对应任务的官方实现。ANLS 能够容忍一定程度的字符差异，但它仍然主要是文本层面的相似性评价。例如，一个多余或遗漏的小数点可能导致严重数值错误，却在字符串层面只对应少量编辑操作。因此，对于数值敏感的文档任务，还需要补充精确数值和单位校验。

文档与图表中的证据验证 文档和图表任务通常具有较明确的证据来源，因此适合开展证据定位与核验。例如，对于文档问题：“合同的签署日期是什么？”系统可以输出日期，同时提供对应页面和文本区域。对于图表问题：“产品 B 的增长率是多少？”系统可以输出计算结果，同时提供两个年份的原始数值和计算公式。这样可以分别评价：原始文本或数值是否正确。证据区域是否准确。计算方式是否符合问题要求。最终答案是否正确。

但模型生成了证据引用，并不意味着该证据一定支持最终答案。例如，模型可能提供错误页面，或者引用了不相关的表格字段。因此，证据定位正确性与答案正确性

应当分别评价。

文档评测中的输入分辨率与 OCR 工具 文档理解对输入分辨率通常较为敏感。例如，一张高分辨率扫描文档中，页脚可能包含细小文字。如果模型输入被压缩到较低分辨率，相关字符可能难以识别。因此，不同模型的原生图像分辨率和视觉 token 预算可能影响 OCR 和文档问答成绩。此外，一些系统可能直接使用图像输入。另一些系统则先使用外部 OCR 工具提取文字，再将结果送入语言模型。这两种设置具有不同的能力来源。

如果研究目标是评价模型原生视觉文字识别能力，应当明确是否允许外部 OCR。如果研究目标是评价完整文档处理系统，则可以比较不同工具组合，但需要报告额外计算和模型资源。因此，文档理解评测应区分模型本身的能力与外部工具辅助形成的系统能力。

文档与图表评测的常见偏差 文档与图表评测可能受到多种因素影响。例如：文本识别错误可能导致数值计算错误。版面结构错误可能导致字段对应关系错误。答案格式差异可能影响文本相似度评分。高分辨率输入可能带来额外计算成本。测试图表的主题或常见数值分布可能形成语言捷径。此外，对于自动生成问题的数据集，还可能存在问题模板相似、答案分布不均衡等问题。因此，评价文档和图表能力时，应当同时关注原始信息提取、结构理解和下游任务结果。

不能仅根据某个文档问答准确率，就判断模型已经具备全面的文档理解能力。

14.4 时序、音频与生成评价

14.4.1 视频理解与时序推理

视频理解评测的基本目标 视频理解（Video Understanding）要求模型从连续视觉观测中提取对象、动作、事件和场景变化等信息，并根据任务要求进行判断。与静态图像理解相比，视频任务通常同时包含空间信息和时间信息。例如，给定一段人物操作杯子的视频，可以设计不同类型的问题。“视频中出现了哪些物体?”主要涉及对象识别。“人物是否拿起过杯子?”主要涉及动作或事件识别。“人物先拿起杯子还是先打开柜子?”主要涉及事件顺序判断。

“人物最后把杯子放在哪里?”则需要结合对象身份、历史状态和事件顺序进行推理。这些问题所需的信息不同。其中，一部分可以通过单帧或少量关键帧完成。另一部分则必须依赖连续时间观测。因此，视频评测需要区分一般语义理解与真正依赖时间信息的推理能力。

视频分类与动作识别指标 视频分类通常要求模型为整段视频预测一个或多个类别标签。例如，预测视频中的主要动作属于跑步、游泳或骑自行车。对于单标签分类，可以使用 Top-1 Accuracy 或 Top-5 Accuracy。Top-1 Accuracy 衡量排名最高的预测类别是否正确。Top-5 Accuracy 则判断真实类别是否出现在前五个预测中。对于多标签视频

分类，一段视频可能同时包含多个动作或事件。例如，人物一边走路，一边打电话。

此时，可以使用 Precision、Recall、F1 或 mAP 等指标。单标签分类和多标签分类具有不同的评价假设。如果一段视频可以同时包含多个有效标签，就不宜简单采用只允许单一正确类别的评分方式。此外，视频级分类通常不直接评价动作发生的具体时间。例如，模型正确判断视频包含“开门”动作，但可能无法说明动作发生在哪几秒。因此，动作识别与时间定位需要分别评价。

时间定位与事件边界 时间定位（Temporal Localization）要求模型预测事件在视频中的发生区间。例如，给定一段 60 秒视频，用户询问：“人物什么时候开始打开房门?”模型可能需要输出：“第 18 秒至第 22 秒。”对于预测区间：$\hat{T} = [\hat{t}_s, \hat{t}_e]$ 以及真实区间：$T = [t_s, t_e]$ 可以计算时间交并比：

$$\mathrm{tIoU} = \frac{|\hat{T} \cap T|}{|\hat{T} \cup T|} \tag{14.29}$$

其中，区间长度应以对应时间单位计算。例如，真实事件发生在第 10 秒至第 20 秒，预测区间为第 12 秒至第 22 秒。交集长度为 8 秒，并集长度为 12 秒，因此：

$$\mathrm{tIoU} = \frac{8}{12} \approx 0.667 \tag{14.30}$$

对于自然语言时刻检索（Natural Language Moment Retrieval），还可以使用 Recall@K 和不同 tIoU 阈值进行评价。例如，R@1, IoU=0.5 表示模型返回的首个候选时间区间，与真实区间达到至少 0.5 的 tIoU 的查询比例。事件边界可能存在一定标注歧义。例如，“人物开始拿起杯子”可以从伸手阶段或实际接触阶段进行定义。

因此，不同数据集需要使用一致的事件边界标注规则和评分协议。

视频输入采样协议 视频模型通常需要对连续视频进行采样或压缩。例如，一段 120 秒的视频，模型可能只输入 32 帧。如果使用均匀采样，其平均采样间隔约为：

$$\Delta t \approx \frac{120}{32} = 3.75 \text{ s} \tag{14.31}$$

但这种平均间隔并不意味着所有重要事件都能够被观察到。例如，一个持续 0.2 秒的动作可能完全落在两个采样帧之间。因此，视频模型表现不仅取决于推理能力，还取决于采样策略是否保留了必要信息。正式评测应记录：视频解码方式、采样帧数、采样时间位置、最大输入时长、图像分辨率、时间戳处理方式。是否提供音频和字幕。对于实时视频，还需要说明是否允许使用未来帧。

例如，离线模型可以访问完整视频，而在线模型通常只能使用当前时刻之前已经到达的观测。因此，不能将不同因果约束下的结果直接视为相同任务条件。

视频理解评测的主要局限 视频理解基准虽然已经覆盖较多任务，但仍然存在一些共同问题。视频问答可能受到语言先验影响。有限帧采样可能遗漏关键动作。自动评分可能无法准确处理某些开放式解释。此外，长视频任务的样本之间可能具有较强时间相

关性。例如，同一段视频中的多个问题通常共享人物、场景和事件。因此，进行统计分析时，应根据研究目标考虑视频级或录制级独立性。视频评测应同时关注空间内容识别、细粒度时间变化、长时间信息利用和输入协议。

14.4.2 语音识别与音频理解

音频理解评测的基本目标 音频是包含时间结构的重要模态。与图像不同，声音信息通常通过波形或时频表示进行处理。例如：语音包含词语、语调和说话人特征。环境声音可能包含汽车鸣笛、雨声、脚步声和机器运行声。音乐则包含节奏、音高、音色及其他结构信息。因此，音频理解评测通常涉及多个能力层次。首先是声音感知，例如判断输入中是否存在语音或某种声音事件。其次是内容识别，例如将语音转换为文本。再次是语义理解，例如根据音频回答问题。

此外，还可能涉及说话人、情绪、韵律和其他副语言信息的识别。因此，不能仅根据自动语音识别成绩判断模型的全部音频理解能力。

自动语音识别与 WER 自动语音识别（Automatic Speech Recognition，ASR）要求模型将语音信号转换为文字。例如，输入一段语音：“请帮我打开会议室的灯。”模型需要生成对应文本。词错误率（Word Error Rate，WER）是语音识别中的常用指标。其定义为：

$$\mathrm{WER} = \frac{S + D + I}{N} \tag{14.32}$$

其中：

- S 表示替换错误的词数；
- D 表示删除错误的词数；
- I 表示插入错误的词数；
- N 表示参考文本的词数。

例如，参考文本包含 10 个词，模型发生 1 次替换、1 次删除，没有插入错误，则：

$$\frac{1 + 1 + 0}{10} = 20\% \tag{14.33}$$

WER 越低，通常表示识别结果越接近参考文本。但 WER 并不一定限制在 0% 至 100% 之间。如果模型产生大量插入内容，WER 可能超过 100%。此外，WER 对词语分割规则较为敏感。例如，中文文本没有天然的空格分词边界，因此需要明确分词方式。

CER 与文本规范化 字符错误率（Character Error Rate，CER）与 WER 类似，但以字符为基本单位。可以表示为：

$$\mathrm{CER} = \frac{S_c + D_c + I_c}{N_c} \tag{14.34}$$

其中，下标 c 表示字符级编辑操作。在中文语音识别任务中，CER 经常用于衡量识别错误。但 CER 和 WER 的数值不能直接比较。例如，同一句中文语音，在不同分词规则下可能得到不同 WER，但字符序列和 CER 可以保持一致。此外，文本规范化可能明显影响结果。例如：“二零二六年”。“2026 年”。两种表达可能具有相同语义，但字符形式不同。类似地，英文语音识别还需要处理大小写、标点和缩写等问题。

因此，语音识别评测应明确：是否保留标点。是否统一数字表达。是否规范化大小写。如何处理非语音事件。是否允许模型输出额外解释。这些规则应当在模型比较之前确定。

音频中的语言与声学证据 音频大语言模型可能利用转写文本回答部分问题。例如：“说话人提出了什么建议？”通常主要依赖文字内容。但对于：“说话人是否在笑？”“背景中是否出现敲门声？”仅靠转写文本通常不足以可靠回答。因此，可以通过不同输入对照分析模型是否真正利用了声音信息。例如：完整音频输入。仅使用语音转写文本。移除背景声音后的语音输入。加入受控噪声的音频输入。通过比较结果，可以分析音频证据在不同任务中的作用。

但噪声或音频删除可能同时改变声学质量与语义信息。因此，实验设计应尽量控制变化因素，并明确测试目标。

音频评测的稳健性与公平性 音频模型性能可能受到多个因素影响。例如：采样率、录音设备、背景噪声、说话人差异、语言和口音、音频时长、音量归一化。是否提供自动转写结果。因此，不同模型之间的比较需要统一输入协议。例如，如果一个模型能够处理完整录音，另一个模型只能处理前 30 秒，那么其信息输入条件可能不同。类似地，如果某个系统使用外部 ASR 工具，再将文本送入语言模型，其成绩不能直接解释为基础语言模型本身具有原生音频理解能力。

因此，应当区分端到端音频模型与工具组合系统，并报告必要的计算与延迟成本。

14.4.3 图像、视频与音频生成

生成评测与理解评测的区别 生成任务（Generative Task）要求模型根据输入条件生成新的内容。例如：文本生成图像、图像生成视频、文本生成视频、文本生成语音。文本生成环境声音。与分类和问答任务不同，生成任务通常没有唯一正确输出。例如，文本提示：“一只白色小狗坐在草地上。”模型可以生成不同品种、不同姿态和不同构图的图像。只要符合提示要求，就可能属于合理结果。因此，生成评价需要考虑多个维度。首先是生成内容本身的质量。

其次是生成结果与条件之间的一致性。再次是不同生成结果的多样性。对于视频，还需要考虑跨帧一致性和运动合理性。对于音频，则需要考虑听觉质量、语音可懂度和声音事件是否符合条件。因此，不能使用单一指标完整描述生成能力。

图像生成的分布质量评价 图像生成模型通常通过大量样本学习真实图像的统计分布。因此，一类常见评价方式是比较生成图像集合与真实图像集合之间的特征分布差异。Fréchet Inception Distance[31]（FID）是经典指标之一。其基本思想是使用预训练 In-

ception 网络提取图像特征，再分别估计真实图像和生成图像特征的均值与协方差。设真实图像特征的均值和协方差为：(μ_r, Σ_r)，

生成图像特征的均值和协方差为：(μ_g, Σ_g)，则 FID 可以表示为：

$$\mathrm{FID} = \|\mu_r - \mu_g\|_2^2 + \mathrm{Tr}\left(\Sigma_r + \Sigma_g - 2(\Sigma_r^{1/2}\Sigma_g\Sigma_r^{1/2})^{1/2}\right) \tag{14.35}$$

其中，Tr 表示矩阵迹。这一公式对应在高斯特征分布假设下构建的 Fréchet 距离表达。通常情况下，FID 越低，说明生成样本集合与参考真实样本集合在所选特征空间中的分布越接近。但 FID 是集合级指标，而不是逐张图像的准确率。它不能直接判断某一张生成图像是否符合用户提示词。此外，FID 也受到样本数量、特征提取器、图像预处理和参考数据分布影响。

因此，只有在这些条件一致时，FID 比较才具有较好的可解释性。

FID 的局限与 KID FID 能够提供生成分布相似性信息，但不能完整评价生成结果。例如，一个模型可能生成具有自然纹理和真实外观的图像，却经常错误理解提示词中的空间关系。这种情况下，FID 可能仍然较低。此外，FID 的有限样本估计可能具有偏差，且不同数据规模下的估计结果可能不完全可比。Kernel Inception Distance（KID）是另一种基于特征分布的生成评价方法。

KID 利用最大均值差异（Maximum Mean Discrepancy，MMD）的思想，比较生成图像与真实图像的特征分布。在使用适当估计形式时，KID 可以提供无偏的有限样本 MMD 平方估计。但 KID 同样依赖所选择的特征提取器和核函数。因此，它也不能直接代表提示词遵循程度或单个样本的语义正确性。综上，FID 和 KID 主要用于分布层面的生成质量评价，不应直接替代条件一致性评价。

FVD：视频分布质量指标 Fréchet Video Distance（FVD）是视频生成领域使用的代表性指标。其基本思想与 FID 类似。不同的是，FVD 使用视频特征提取器获得包含时间信息的特征表示，然后比较真实视频和生成视频的特征分布。因此，它可以在一定程度上反映生成视频与参考视频集合在视觉和时间特征上的差异。通常情况下，FVD 越低，表示在既定特征空间和测试协议下，生成视频分布越接近参考分布。

但 FVD 并不能完整评价视频中的全部动态质量。例如，模型可能生成具有清晰画面但运动不足的视频。在某些特征提取器下，这类结果可能获得相对较好的分布指标。Ge 等人在《On the Content Bias in Fréchet Video Distance》中研究了 FVD 对单帧内容和时间变化的敏感性。该研究指出，常用 FVD 实现可能更偏向图像内容质量，而对部分时间错误不够敏感。

因此，FVD 不应单独作为视频生成质量的完整证明。

VBench：多维度视频生成评价 VBench 是视频生成模型综合评价的重要代表性基准。与主要关注整体分布差异的指标不同，VBench 将视频生成质量拆分为多个评价维度。例如：主体一致性、背景一致性、时间闪烁、运动平滑性、动态程度、单帧视觉质量、对象类别与数量、动作和空间关系。整体文本条件一致性。这些指标分别关注不同问题。例如，主体一致性主要判断同一个对象在不同帧中的外观是否稳定。运动平滑性

则关注连续动作是否自然。

动态程度用于分析视频是否产生了足够的运动变化。完全静止的视频可能具有较好的外观一致性，但未必满足要求动态动作的提示词。因此，不能单纯追求跨帧相似度越高越好。VBench 通过细分评价维度，为分析不同视频生成模型的优势与不足提供了更具体的依据。

语音合成的 MOS 平均意见分（Mean Opinion Score，MOS）是语音和音频主观质量评价中的重要指标。在典型的五级质量评价设置中，受试者根据听觉体验对语音质量进行评分。例如，使用 1 至 5 的评分量表。如果共有 N 个有效评分，则：

$$\text{MeanRating} = \frac{1}{N}\sum_{i=1}^{N} r_i \tag{14.36}$$

其中，r_i 表示某个评分。MOS 越高，通常说明在对应测试条件下，受试者认为语音质量越好。但 MOS 并不是完全统一的固定指标。不同听测协议可能评价自然度、总体质量或其他属性。ITU-T P.800 及其相关建议提供了语音主观评价的规范；P.800.2 进一步强调 MOS 类型和报告信息的明确性。因此，比较不同研究中的 MOS 时，需要确认评价量表、听测环境、评分问题和受试者条件是否可比。

不能直接将不同实验室、不同测试协议下的 MOS 视为严格相同的测量结果。

14.4.4 质量、一致性与任务成功率

为什么生成评测需要多个维度 生成模型的评价通常比分类模型更加复杂。例如，一张生成图像可能具有较高分辨率和自然纹理，却不符合用户指定的对象数量。一段视频可能每帧都清晰，却缺乏合理运动。一段语音可能非常自然，却错误朗读了原始文本。因此，生成能力不能只通过单一质量指标进行描述。可以将生成评测概括为几个主要维度。感知质量（Perceptual Quality）：生成内容在视觉或听觉上是否自然、清晰和完整。

条件一致性（Condition Alignment）：生成结果是否满足文本、图像、音频或其他输入条件。内部一致性（Internal Consistency）：生成内容自身是否保持合理的对象、时间和结构关系。多样性（Diversity）：模型是否能够生成不同但合理的结果，而不是重复少数固定模式。任务成功率（Task Success Rate）：生成结果是否实际达到了用户要求的目标。

这些维度之间具有联系，但不能完全相互替代。例如，较高的视觉质量不保证文本条件正确。较高的语义相似度也不能证明生成内容具有合理的时间结构。

质量与条件一致性的区别 假设用户要求：“生成一张包含三只黑色猫的图像。”模型生成一张清晰、自然的图片，但其中只有两只猫。从感知质量角度看，图像可能非常优秀。但从条件一致性角度看，它没有满足对象数量要求。因此，需要分别评价图像质量和任务要求完成情况。对于复杂文本提示，可以将条件拆分为多个可检查的要求。例如：对象类别是否正确。对象数量是否正确。对象属性是否正确。空间关系是否正确。

场景风格是否正确。

因此，条件一致性可以采用细粒度指标，而不是只使用整体图文相似度。GenEval 和 T2I-CompBench 等基准正体现了这一评价思路。

多样性与模式覆盖 生成模型不仅需要产生高质量样本，还需要能够覆盖合理的输出变化。例如，提示词为：“一只猫坐在房间里。”合理的生成结果可以包含不同品种、不同房间和不同构图。如果模型每次都生成几乎相同的场景，就可能缺乏多样性。但如果模型生成高度多样的结果，却经常违反提示词，也不能认为生成能力较好。因此，需要在质量、条件一致性与多样性之间取得平衡。对于集合级评价，可以使用 FID、KID 或其他分布指标辅助分析。

但这些指标无法完整区分所有类型的模式缺失和质量问题。因此，在生成研究中，还可以针对不同提示词类别进行分组评价，并检查少见对象、复杂关系和长尾条件下的表现。

任务成功率的基本概念 任务成功率（Task Success Rate）强调生成结果是否完成预期目标。对于明确的二元成功条件，可以表示为：

$$\text{SuccessRate} = \frac{1}{N}\sum_{i=1}^{N}\mathbb{I}[\text{Success}(i)] \tag{14.37}$$

其中，$\text{Success}(i)$ 表示第 i 个任务是否成功。例如，对于图像编辑任务：“将图片中汽车的颜色改为蓝色，并保持其他对象不变。”可以定义成功条件：汽车颜色正确修改。目标汽车仍然存在。其他指定对象没有发生不必要改变。对于视频生成任务：“生成人物先打开门、再走出房间的过程。”可以检查关键事件是否出现及其顺序是否正确。对于语音生成任务：“准确朗读给定文本。”

可以检查是否完整且正确地输出目标内容。任务成功率能够更加直接地反映生成模型是否满足应用要求。但成功条件必须事先定义。不能根据生成结果临时修改评价标准，否则可能产生选择偏差。

主观评价与人工偏好 对于图像、视频和音频生成，人工评价仍然具有重要作用。例如，人类能够判断一段合成语音是否自然，或者一段生成视频是否存在明显的不合理动作。常见人工评价方式包括：绝对评分、成对偏好比较、排序评价。针对具体维度的质量判断。例如，在成对比较中，给定相同提示词下的两个生成结果，让评价者选择更符合要求的一个。如果模型 A 相对于模型 B 在 N 个有效比较中获胜 W 次，则可以定义：

$$\text{WinRate} = \frac{W}{N} \tag{14.38}$$

但平局如何处理应由具体协议规定。例如，可以单独报告平局率，或者使用预先规定的加权方式。此外，人工评价容易受到样本顺序、显示设备、音频播放环境和评价者偏好影响。因此，应尽可能采用随机展示顺序、盲测和清晰的评分说明。必要时，还可以进行多评价者一致性分析。

自动评价器的局限 随着多模态大语言模型的发展，研究者开始使用视觉语言模型或音频语言模型作为生成评价器。例如，将文本提示和生成图像提供给评价模型，让其判断是否符合要求。这种方式具有较好的自动化能力，但也存在问题。评价器可能出现感知错误。例如，错误统计生成图像中的对象数量。评价器可能受到语言提示词影响。例如，更长或更详细的解释可能获得更高评分。评价器可能具有训练数据或模型偏差。例如，对某些风格、内容或生成模型具有系统性偏好。

因此，自动评价器的输出应当通过适当的人类标注进行验证。特别是当多个模型的分数差异很小时，应当分析评价器误差是否可能影响最终排序。

生成模型的统计可靠性 生成模型具有较强随机性。同一个提示词在不同随机种子下可能产生不同结果。因此，仅展示少量精选样本，无法充分反映模型整体生成能力。例如，研究者可能从大量结果中选择视觉质量最好的几个样本进行展示。这些结果可以说明模型能够达到某种生成效果，但不一定代表其平均表现。因此，正式评价应当使用预先确定的提示词集合和采样策略。对于统计指标，可以根据提示词、生成样本和相关独立单位构建合理的置信区间或重复实验分析。

对于人工偏好评价，还需要考虑评分者之间的相关性。此外，集合级指标如 FID 和 FVD 可能受到样本数量影响。因此，应当在相同样本规模和一致特征提取条件下进行比较。

14.5 复杂任务与裁判

裁判必须接受单独验证。至少检查答案顺序、长度、风格及模型身份是否影响判断，再用人工核验的子集测量一致性。裁判给出详细理由，仍不能证明其看到了正确视觉证据。

14.5.1 多步推理与问题求解

多步推理的基本概念 多步推理（Multi-step Reasoning）通常指模型需要通过多个相互关联的信息处理操作，才能得到最终答案的任务。例如，用户提供一张包含多名人物的图像，并询问：“站在红色汽车旁边、穿蓝色衣服的人手里拿着什么？”模型需要识别红色汽车，定位汽车附近的人物，根据服装属性筛选目标，再识别人物手中的物体。如果模型直接识别出一个物体，却没有正确完成前面的对象筛选，就可能回答错误。

因此，多步推理的评价不仅需要检查最终答案，还需要考虑任务内部包含的关系组合和逻辑操作。可以将复杂问题抽象为：

$$Q \to (q_1, q_2, \ldots, q_n) \to Y \tag{14.39}$$

其中：

- Q 表示原始问题；
- q_i 表示第 i 个子问题或操作；

- Y 表示最终答案。

这种表示主要用于分析任务的逻辑结构。

端到端多模态模型不一定显式执行完全相同的子问题序列。因此，评价多步推理能力时，需要区分任务本身所需的逻辑操作，以及模型实际生成的中间推理文字。

推理步骤数量与任务复杂度 多步推理任务的难度可能受到多个因素影响。例如：需要关联的对象数量。推理操作的数量。对象属性和关系的复杂程度。中间条件之间的依赖关系。输入信息是否存在遮挡或歧义。是否需要额外专业知识。因此，推理步骤越多，任务不一定严格越难。例如，一个需要连续执行五次简单加法的问题，可能比一个需要识别复杂几何关系的问题更加容易。类似地，某个视觉问题虽然只包含两个逻辑步骤，但其中一个步骤可能需要识别非常细小的图像细节。

因此，评测任务难度不能只根据自然语言描述长度或推理步骤数量确定。更合理的方式是结合任务结构、输入信息和实际模型表现进行分析。例如，可以根据对象数量、关系类型和问题组合深度进行分组统计。

最终答案与中间结果的联合评价 对于多步推理任务，可以分别评价最终答案和部分中间结果。例如，用户询问："根据图表，2024 年产品 A 的销售额比 2023 年增长了百分之多少?" 可以将任务拆分为：读取 2023 年销售额。读取 2024 年销售额、选择增长率公式、执行计算、生成答案。假设图表中对应数据为 100 和 125，则：

$$r = \frac{125 - 100}{100} \times 100\% = 25\% \tag{14.40}$$

如果模型最终回答 25%，可以判定最终结果正确。但如果中间数值读取错误，仍然需要进一步分析。因此，可以为部分中间操作设置独立标注。例如，对数值提取使用精确匹配，对公式选择使用规则检查，对最终结果使用数值容差评价。这种方式有助于定位错误来源。但并非所有复杂任务都具有唯一的中间推理路径。例如，同一道几何题可能采用不同的正确解法。因此，过程评价不应机械地要求模型生成与参考答案完全相同的步骤。

更合理的方式是检查关键事实、运算约束及最终结论是否正确。

多步推理中的错误传播 复杂任务通常包含多个相互依赖的环节。例如：对象检测、属性识别、关系筛选、数值计算、答案生成。如果前面的关键步骤发生错误，后续计算可能建立在错误信息之上。在一个简化假设下，若某项任务包含 n 个必须全部正确的独立步骤，每个步骤成功概率均为 p ，则完整任务成功概率为：$P_{\text{success}} = p^n$，例如，当：$p = 0.9, \quad n = 5$，则：

$$P_{\text{success}} = 0.9^5 \approx 0.5905 \tag{14.41}$$

即完整任务成功率约为 59.05%。这只是说明多步骤错误累积的理想化示例。实际推理步骤之间往往存在相关性，而且某些错误可能被后续验证修正。因此，不能直接使用这一公式预测真实模型的任务准确率。但它说明，复杂任务的整体成功率可能明

显低于单个步骤的平均成功率。因此，评测时需要关注关键步骤的可靠性，而不仅是最终成功率。

多步推理评测的常见误区 多步推理评测容易出现几个问题。将回答长度视为推理深度。例如，模型生成较长解释，并不一定意味着其完成了更加复杂的推理。将最终答案正确视为全部步骤正确。忽略基础感知错误。例如，错误读取图表数值可能导致最终结果错误，但问题不一定来自数学推理。此外，复杂问题的数据集可能存在语言捷径。例如，多项选择题可能通过选项模式排除错误答案。因此，多步推理评测应当结合任务结构、感知证据和最终结果，并根据需要设置无图像或反事实对照。

14.5.2 工具使用与多步骤任务

从问题回答到任务执行 传统多模态问答主要评价模型是否能够生成正确答案。但在复杂应用中，用户往往要求模型完成具体操作。例如：“读取这份报表，计算平均增长率，并将结果写入指定单元格。”“打开网页，找到某个商品的规格，并填写比较表。”“根据图像中的对象位置，选择合适的操作区域。”这些任务不仅需要理解输入，还需要采取行动。因此，工具使用与多步骤任务评测需要进一步检查模型是否能够正确规划、调用工具和完成环境操作。

可以将任务执行过程表示为：$s_{t+1} = F(s_t, a_t)$，其中：

- s_t 表示当前环境状态；
- a_t 表示模型执行的动作；
- F 表示环境状态转移；
- s_{t+1} 表示执行后的环境状态。

对于部分可观测环境，模型通常无法直接访问完整状态，而是通过截图、文本、接口响应或其他观测了解环境。

因此，工具使用评测还涉及模型能否根据有限观测作出正确决策。

工具选择与参数构建 工具使用通常包含两个不同环节。模型需要判断应该调用什么工具。需要为工具提供正确参数。例如，一个系统提供：图像检测工具。OCR 工具、计算工具、文件操作工具。如果任务要求读取图像中的文字，模型需要选择适当的视觉文字识别工具。如果任务要求计算平均值，则应当正确构造计算输入。因此，可以分别评价工具选择和参数正确性。例如：工具选择准确率：模型是否调用了符合任务要求的工具。参数正确率：工具调用参数是否符合预期结构和语义。

执行成功率：调用是否被工具接受并完成执行。但这些指标并不等同于最终任务成功率。例如，工具调用格式正确，可能只是说明参数满足 API 规范。如果工具读取了错误文件，最终任务仍然可能失败。因此，工具调用的形式正确性与语义正确性应当区分。

多步骤任务的执行轨迹 工具型智能体通常通过多个连续动作完成任务。例如，在网页上完成某项信息填写任务，可能需要：打开页面、定位目标区域、输入搜索条件、选择

结果、填写表单、保存修改。可以将执行轨迹表示为：

$$\tau = (o_1, a_1, o_2, a_2, \ldots, o_T, a_T) \tag{14.42}$$

其中：

- o_t 表示第 t 步的环境观测；
- a_t 表示执行动作。

执行轨迹可以帮助分析模型在何处发生错误。

例如：是否观察到正确页面。是否点击了正确按钮。是否输入了正确内容。是否正确处理页面跳转。是否在任务完成后停止。但轨迹长度本身不代表任务质量。例如，一条较短轨迹可能高效完成任务，也可能因过早停止而失败。因此，轨迹评价需要结合实际任务目标。

最终状态验证 对于能够修改环境的任务，最终状态通常比模型生成的完成说明更加重要。例如，任务要求：“将电子表格 A1 单元格中的数值修改为 100。”模型可能回答：“已经完成修改。”但如果实际文件没有改变，就不能认为任务成功。因此，可以设置任务成功条件：

$$\text{Success}(i) = \mathbb{I}[s_T^{(i)} \in G_i \tag{14.43}$$

] 其中：

- $s_T^{(i)}$ 表示第 i 个任务执行结束后的环境状态；
- G_i 表示满足任务目标的状态集合。

例如，文件中的目标单元格具有正确数值，且其他必要约束也得到满足，就可以认为任务成功。

这种评价方法通常称为基于执行结果或环境状态的验证。它能够减少模型仅通过语言声称完成任务所造成的评价偏差。但任务成功条件需要设计得足够完整。例如，如果模型修改了正确单元格，却删除了其他重要数据，仅检查目标数值可能无法发现问题。因此，复杂任务还需要设置必要的副作用和约束检查。

OSWorld：**真实计算机环境评测** OSWorld 是面向多模态计算机操作智能体的重要基准。该基准提供真实操作系统和应用程序环境，使智能体能够根据任务指令进行连续操作。例如：打开指定文件、修改文档内容、调整软件设置、完成跨应用操作。与静态截图问答不同，OSWorld 需要模型观察环境、执行操作并根据环境变化继续决策。其评价采用与任务相关的执行结果检查。因此，模型能否正确描述操作步骤，并不能替代实际任务完成情况。

这类任务能够评价视觉理解、操作规划和环境交互之间的联合能力。但操作系统版本、软件状态和界面变化，都可能影响评测结果。因此，实验应尽量保证环境一致，并提供必要的状态重置机制。

多次执行与 pass^k 对于具有随机性的智能体，同一任务可能在不同运行中得到不同结果。例如，智能体第一次成功完成任务，第二次却在某个页面跳转步骤失败。因此，单次成功率不能完整反映实际可靠性。τ-bench 提出了 pass^k 指标，用于评价智能体在重复执行中的稳定性。对于任务 i ，假设独立运行 k 次，并定义：

$$s_{i,j} = \begin{cases} 1, & \text{第} j \text{ 次成功} \\ 0, & \text{第} j \text{ 次失败} \end{cases} \tag{14.44}$$

则可以用以下形式理解重复成功率：

$$\text{TaskSuccess} = \frac{1}{N} \sum_{i=1}^{N} \prod_{j=1}^{k} s_{i,j} \tag{14.45}$$

其中，N 表示任务数量。这一形式要求某个任务在全部 k 次执行中都成功，才被计入成功任务。在采用有限次数采样估计时，应遵循基准官方的估计规则。pass^k 与 pass@k 不同。pass@k 通常关注多次尝试中是否至少出现一次成功结果。而 pass^k 强调重复执行是否能够持续成功。因此，两者评价的可靠性含义不同。

对于实际交互系统，重复稳定性可能有意义，因为用户通常不希望同一任务有时成功、有时失败。

14.5.3 自动评测与人工评测

复杂任务为什么需要不同评价方式 对于具有明确正确答案的任务，可以通过程序自动判断结果。例如：多项选择题、数值计算、精确字段提取、边界框定位。工具执行后的数据库状态。但部分复杂任务具有开放式输出。例如：“描述这张图像中可能存在的安全风险。”“解释视频中的人物为什么改变了行动。”“根据多个图表总结主要研究结论。”这些任务可能存在多个合理答案。因此，仅通过字符串匹配可能无法完整判断回答质量。这使得复杂任务评测通常需要结合自动评价和人工评价。

自动评价能够提高效率和可重复性。人工评价则有助于处理开放语义、复杂事实关系和难以程序化判断的问题。但两种方式都存在局限，需要根据任务类型选择。

规则与程序自动评分 对于具有明确标准答案的任务，程序化评分通常具有较高的可检查性。例如，多项选择题可以比较标准选项。数值题可以检查计算结果是否位于合理容差内。目标定位可以计算 IoU。工具任务可以检查执行后的环境状态。这类评分方式具有几个优势。评价规则较为明确。可以在大量样本上自动运行。只要测试数据与评分程序一致，通常具有较好的重复性。但自动规则也可能存在问题。例如，数值容差设置不当，可能错误接受不合理答案。

严格文本匹配可能把语义等价表达判为错误。因此，程序评分的可靠性依赖于规则本身是否合理。对于关键任务，应当检查评分代码和边界情况。

自动评价器的常见偏差 使用大语言模型作为评价器时，可能出现多种偏差。首先是位置偏差。例如，在成对比较中，评价器可能更倾向于选择先展示的回答。其次是长度偏差。例如，评价器可能认为较长、较详细的回答更好，即使其中包含无关内容。再次是风格偏差。例如，语言更加正式或逻辑连接词更多的回答可能获得较高评分。还有模型相关偏差。例如，评价器可能偏好某类模型的表达风格。

Zheng 等人在《Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena》中系统分析了语言模型评价器的适用性和相关偏差。该研究表明，模型评价器可以在一定条件下与人工偏好具有较高一致性，但仍然存在位置、冗长性和自我偏好等问题。该研究主要围绕语言模型对话评价展开。在多模态任务中，还需要额外考虑视觉、音频和视频感知错误。

评分标准与评价维度 为了提高自动评价的稳定性，可以制定明确的评分标准。例如，对于复杂视觉问答，可以采用以下维度：事实正确性。关键视觉证据。问题条件满足程度、推理逻辑、答案完整性。可以根据任务设置离散评分等级。例如：这一评分表仅为示例。实际任务需要明确每个等级的具体边界。例如，对于精确数值计算，可能更适合使用正确或错误的二元评分。对于开放式场景解释，则可能需要更加细致的评价维度。因此，评分等级的复杂程度应当根据任务特点设计。

人工评价的局限 人工评价虽然能够处理复杂语义，但并不一定完全客观。例如，评价者可能不具备特定专业背景。也可能因为视频太长或图像太复杂而遗漏信息。对于物理场景和专业知识问题，人工判断本身也可能存在错误。此外，人工评价通常具有较高的时间和经济成本。因此，对于大规模基准，完全依赖人工逐样本评价可能难以实施。更可行的方法是根据任务特点，结合程序评价、模型评价器和人工抽样复核。例如，对于确定性计算问题，优先使用程序评分。

对于开放式解释，使用模型评价器进行初步评分。对于争议较大或高重要性的样本，再进行人工复核。但具体方案需要根据评价资源和研究目标确定。

复杂任务评价中的统计可靠性 复杂任务通常涉及多个样本、多个执行步骤和不同随机运行。因此，评价结果可能受到多种随机因素影响。例如，智能体可能在相同任务的不同运行中选择不同操作路径。评价器也可能因随机采样产生不同分数。人工评价则可能受到标注者差异影响。因此，需要根据任务设计合理的统计分析。例如，对于两个模型在相同任务上的成功率，可以使用配对比较方法。对于同一模型的多次随机执行，可以报告成功率均值和运行波动。

对于人工评分，可以报告评价者一致性和适当的区间估计。此外，如果同一个视频或软件任务生成多个相关问题，就需要注意这些样本之间可能存在相关性。因此，可以根据任务使用视频级、会话级或其他合适的统计单位。不能将大量高度相关的操作步骤简单视为独立任务，以夸大统计样本量。

14.5.4 模型裁判的偏差

为什么使用模型裁判 对于多项选择题和精确数值任务，可以通过程序判断答案是否正确。但在开放式多模态任务中，模型可能生成较长的自然语言解释。例如：“请总结视频中的关键事件。”“请根据图像分析可能存在的安全风险。”“请解释图表中不同指标之间的变化关系。”这些问题可能存在多种合理回答。因此，简单字符串匹配通常不足以完整评价答案质量。为了降低人工评价成本，研究者开始使用大语言模型或多模态大语言模型作为自动评价器。

这种方式通常称为 LLM-as-a-Judge 或 MLLM-as-a-Judge。例如，评价器可以获得问题、参考答案和待评价回答，再根据评分标准判断答案是否正确。G-Eval 是使用大语言模型进行自然语言生成评价的代表性研究。该研究探索了利用结构化评分准则和语言模型评价生成文本的方法。但模型裁判本身也可能存在错误和偏差。因此，自动评价结果不能天然视为真实标准答案。

模型裁判与确定性评分的区别 确定性评分通常具有明确的计算规则。例如：数值是否相等。边界框 IoU 是否达到阈值。数据库是否处于指定状态。这些任务可以通过程序直接检查。而模型裁判通常需要对自然语言内容进行语义判断。例如：回答是否完整。解释是否合理。事实是否得到证据支持。不同评价器可能对相同回答给出不同评分。即使使用同一个评价器，不同提示词或输出顺序也可能影响结果。因此，模型裁判是一种需要验证的评价方法，而不是完全独立的客观标准。

对于能够通过确定性程序验证的任务，应尽量保留程序化评分。对于难以使用明确规则评价的开放任务，可以使用模型裁判辅助评价，并分析其可靠性。

位置偏差 位置偏差（Position Bias）是模型裁判中的常见问题。例如，评价器需要比较两个答案。答案 A 放在前面。答案 B 放在后面。评价器可能对某个位置具有系统性偏好。如果交换两个答案的顺序，评价结果可能发生变化。Zheng 等人在《Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena》中分析了语言模型评价器的位置偏差、冗长性偏差和其他问题。

Wang 等人在《Large Language Models are not Fair Evaluators》中进一步研究了候选回答顺序对评价结果的影响。这些研究说明，评价器的排序判断可能受到展示位置影响。因此，在成对评价中，可以随机交换回答顺序。对于重要比较，还可以分别在 A–B 和 B–A 两种顺序下评价。如果两次结果不一致，需要明确处理规则。

例如，可以将顺序不一致的样本标记为存在评价不确定性，并进行人工复核。但交换顺序只能帮助诊断或缓解部分位置偏差，不能保证消除所有评价错误。

回答长度偏差 模型裁判还可能偏好更长的回答。例如：回答 A：“汽车位于左侧。”回答 B：“通过详细观察图像中的对象位置，可以清楚地判断，汽车位于画面左侧。”对于问题：“汽车位于哪一侧？”两种回答可能表达相同事实。但某些评价器可能认为回答 B 更完整，从而给出更高评分。这会使评价结果受到语言表达长度影响，而不只是任务正确性。类似地，一段包含较多解释的回答，可能被认为更加专业。但解释越长，也可能包含更多无关信息或错误事实。

因此，对于具有明确答案的任务，应优先检查事实正确性。对于需要详细解释的任务，则可以将正确性、完整性和表达质量作为不同维度评价。不能将回答越长直接等同于推理能力越强。

风格与表达形式偏差 模型裁判可能对不同语言风格存在偏好。例如，一段回答使用较多专业术语和逻辑连接词，可能显得更加严谨。另一段回答使用简短语言，但包含完全正确的信息。如果评价器过度重视语言风格，就可能错误提高前者的评分。因此，评价标准需要明确哪些因素属于任务目标。例如，技术报告任务可能确实要求专业表达。但对于目标检测和对象属性判断，回答语言是否华丽通常不应成为主要评价依据。因此，评价器需要尽量将任务正确性与语言风格区分开来。

在必要时，可以采用标准化输出格式，减少无关表达差异。但格式标准化也不能删除具有任务意义的内容。

模型裁判的合理使用边界 综合来看，模型裁判适合辅助处理部分开放式、难以完全程序化评分的任务。例如，复杂自然语言解释、开放式内容总结和语义等价判断。但对于精确数值、目标坐标、时间区间和工具执行结果等任务，通常应优先使用可以复现的确定性评分。在必须使用模型裁判时，应当明确：评价器名称和版本、评价提示词、评分规则。是否提供原始多模态证据。是否进行顺序交换。是否使用多个裁判。是否通过人工样本验证其可靠性。

模型裁判应当被视为评价系统中的一个需要验证的组件，而不是天然可靠的最终裁决者。

14.6 污染、设置与分组结果

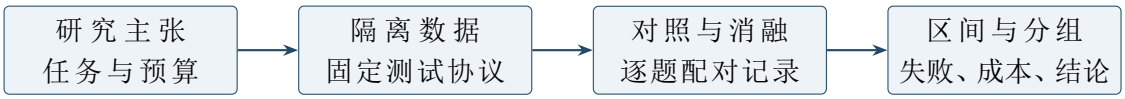


图 14.1 实验设计的依赖顺序。应先固定主张和协议，再选择对照与统计方法。

14.6.1 数据泄漏与基准污染

数据泄漏的基本概念 数据泄漏（Data Leakage）是指模型训练、开发或评价过程中，使用了本不应在相应阶段获得的信息，导致实验成绩无法可靠反映目标泛化能力。例如，在监督学习中，研究者通常将数据划分为训练集、验证集和测试集。训练集用于学习模型参数。验证集用于选择超参数或模型配置。测试集则用于独立评价最终性能。如果测试集中的样本或标签进入训练过程，模型可能在评价时表现得比面对真正未知数据时更好。

因此，数据泄漏会使评价结果产生偏乐观的风险。Kapoor 和 Narayanan 在《Leakage and the Reproducibility Crisis in Machine-Learning-Based Science》中系统讨论了机器学习研究中的数据泄漏问题。该研究指出，数据泄漏可能发生在数据准备、特征

选择、训练和模型评价等不同环节，并可能导致对模型实际能力的错误判断。

数据泄漏不一定来自研究者有意使用测试答案。例如，错误的数据划分方式、共享样本来源或不恰当的预处理，也可能在无意中引入泄漏。因此，数据泄漏应当作为实验设计和数据处理问题进行系统检查。

基准污染与训练数据重叠 对于大规模多模态模型，基准污染（Benchmark Contamination）是一个特别重要的问题。多模态大模型通常使用大规模图文、视频、音频和文本数据进行预训练。这些数据可能来自公开网页、开放数据集、技术文档、论文或其他网络资源。如果某个公开测试集及其答案已经出现在训练数据中，模型就可能在评价时利用已有记忆，而不完全依赖当前输入进行推理。例如，一张公开数学题图片可能同时出现在网页及其文字解析中。

模型在预训练阶段可能接触过：题目图片、题目文字、正确答案、完整解题过程。因此，即使模型在测试时没有正确读取全部视觉内容，也可能凭借记忆生成答案。这会使模型实际的视觉数学推理能力与基准成绩之间产生偏差。

Deng 等人在《Investigating Data Contamination in Modern Benchmarks for Large Language Models》中研究了公开基准的潜在污染问题，并提出基于检索和测试题目片段猜测的检测方法。相关研究说明，在训练语料大规模且来源复杂的情况下，基准污染可能难以完全识别。

无图像测试与视觉依赖性 在多模态评测中，一种重要的诊断方式是移除图像，只保留文本问题和选项。如果模型仍然能够获得较高准确率，说明部分问题可能不需要充分利用当前视觉信息。例如：“香蕉通常是什么颜色？”模型可能仅凭常识回答。但这并不能证明其真正理解了测试图像中的香蕉。MMStar 是针对视觉必要性和评价污染问题提出的重要基准。该研究分析了部分视觉语言基准中存在的语言线索和潜在数据污染问题，并构建了更加强调视觉依赖的测试样本。

可以通过以下对照分析视觉信息的贡献：

$$S_{\text{image+text}}, \quad S_{\text{text}} \tag{14.46}$$

其中：

- $S_{\text{image+text}}$ 表示完整输入下的成绩；
- S_{text} 表示只使用文本条件的成绩。

如果差异较小，需要进一步检查测试问题是否真正要求视觉信息。

但无图像条件下得分较高，并不能单独证明发生了训练数据污染。因为语言先验、选项偏差和问题结构本身，都可能造成这种现象。因此，视觉必要性不足与基准污染应当作为不同问题分别诊断。

基准污染的检测方法与局限 研究者可以通过多种方式检查潜在污染。例如，直接检索训练语料与测试样本之间的重叠。使用文本或视觉相似度发现近似样本。构建内容相似但答案不同的反事实问题。在模型公开之前准备新的独立测试数据。采用时间上较

新的评价样本。对模型在公开基准和新构建基准上的表现进行比较。ConStat 是一种面向基准污染诊断的研究方法。该方法关注模型在主要基准和参考基准之间是否出现异常的性能差异，以识别可能由污染带来的不合理性能提升。

但污染检测本身并不是完全可靠的。例如，模型可能因为真实的任务分布变化而在两个基准上表现不同。也可能已经接触了某些问题的语义变体，却没有记住原始文字。Sun 等人在 2025 年的研究中分析了多种基准污染缓解方法，指出不同策略在保留原始任务性质与抵抗污染之间存在权衡。因此，简单改写题目不一定能够消除污染。如果改写改变了问题难度或正确答案，也可能引入新的评价偏差。

动态基准与持续更新 为了降低公开静态基准被模型记忆的风险，一些研究开始探索持续更新的评价数据。例如，随着时间推移，定期加入新的任务、图像和问题。MMBench-Live 是持续更新多模态基准的代表性研究。该方法尝试通过任务驱动的数据构建和更新策略，为多模态模型提供持续演化的测试样本。动态更新具有一定潜在优势。例如，新加入的样本可能减少对长期公开测试题目的直接记忆依赖。但动态基准也存在挑战。

例如，不同版本的数据分布和难度可能发生变化。如果模型 A 使用旧版本测试，模型 B 使用新版本测试，其成绩未必能够直接比较。此外，新数据也可能随着公开而进入后续模型的训练语料。因此，动态基准并不能自动彻底解决污染问题。研究者仍然需要记录基准版本、发布时间和数据构建条件，并尽可能保持跨版本评价的可比性。

数据污染的基本防范原则 综合来看，降低数据污染风险需要从数据收集、实验开发和结果报告多个环节入手。明确训练、验证和测试数据的来源。根据任务目标选择合理的独立划分单位。对可以访问的训练数据与测试数据进行适当的重复和相似性检查。对于大规模预训练模型，应当说明训练数据透明度的限制。此外，尽量避免根据测试成绩反复修改提示词、模型结构和评价配置。必要时，可以引入新构建的独立测试集，检验主要结论是否成立。

对于无法排除的潜在污染，应当明确说明不确定性，而不是简单宣称数据完全无污染。数据泄漏与基准污染的核心问题，是评测结果是否能够真实反映模型面对未见任务和独立数据时的表现。

14.6.2 提示词、采样与评测设置的影响

模型成绩为什么依赖评测设置 多模态大模型通常通过提示词进行任务交互。例如，同一张图像可以搭配不同的文本指令。“图中有几个人?”“请认真观察图像，并给出人物数量。”“只输出一个数字，不要解释。”这些提示词可能具有相同的基本任务目标，但模型的回答形式和部分预测结果可能不同。因此，模型成绩不仅受到权重和输入数据影响，也受到提示词、采样参数和推理协议影响。可以将这种关系表示为：$S = S(M, D, P)$，其中，P 表示提示词及其他推理设置。

因此，当研究者报告某个模型在特定基准上的成绩时，应当明确其使用的提示词和推理配置。如果不同研究采用不同设置，就不能仅根据成绩差异直接判断模型能力优劣。

提示词格式敏感性 提示词敏感性是语言模型和多模态模型评价中的重要问题。例如，同样的问题可以使用不同格式组织。一种形式是：“问题：……选项：……答案：”另一种形式是：“请根据以下内容选择正确答案，并仅输出选项字母。”即使问题语义基本不变，模型输出也可能发生变化。

Sclar 等人在《Quantifying Language Models' Sensitivity to Spurious Features in Prompt Design》中研究了提示词格式变化对模型表现的影响。该研究发现，在所测试的语言模型与少样本任务设置中，某些看似不影响语义的格式变化也可能导致明显性能差异。虽然研究主要面向语言模型，但对于采用语言模型作为核心的多模态系统，这一问题同样值得关注。

例如，改变图像与文本的排列方式、提示词长度或答案输出格式，都可能影响模型实际表现。因此，评测不能只关注提示词是否在语义上合理，还需要明确具体使用的模板。

多项选择题的选项顺序偏差 在多项选择题中，模型可能受到答案位置影响。例如，问题提供四个候选答案：A、B、C、D。即使候选内容相同，只改变其顺序，也可能影响模型输出。例如，某个模型可能倾向于选择 B。如果测试集正确答案在 B 位置的比例较高，模型可能获得偏高成绩。MMBench[32] 针对这一问题提出了 CircularEval 策略。其基本思想是对同一道选择题循环改变选项顺序，并检查模型是否能够在所有循环排列条件下正确回答。

对于有 K 个选项的问题，可以生成相应的 K 种循环排列。只有模型在全部排列中都正确，才将该题计为通过。因此，CircularEval 相比单次选择题准确率更加严格。但 CircularEval 与普通单次准确率衡量的条件不同。它不仅评价单次答题正确性，也评价模型面对选项顺序变化时的稳定性。因此，不能将两者数值直接视为相同指标。

采样温度与随机生成 多模态大语言模型通常通过概率分布生成输出。采样温度会影响输出的随机性。例如，对于模型生成的 logits：$z_1, z_2, \ldots, z_n$ 可以通过温度 T 调整概率分布：

$$p_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)} \tag{14.47}$$

在常见采样设置中，较低温度使分布更加集中，较高温度通常增加输出多样性。因此，同一个模型在不同采样温度下，可能产生不同答案。例如，对于复杂数学推理，较高温度可能产生更多不同的解题路径。但也可能增加错误答案数量。因此，模型比较时需要记录采样温度、top-p 等设置。如果某个方法使用确定性解码，另一个方法使用多次随机采样，那么二者的输出机制和计算预算可能不同。因此，应根据研究目标采用统一或明确区分的推理协议。

多候选推理与计算预算 部分复杂任务使用多候选生成和答案选择。例如，模型针对一个问题生成五个候选结果，再使用多数投票或验证器选择答案。这种方法可能提高某些任务的准确率。但也增加了推理计算成本。因此，单次推理与多候选推理不应被视为相同预算条件。例如：模型 A：只生成一个答案。模型 B：生成十个答案，并使用额

外验证器选择最优结果。即使模型 B 的最终准确率更高，也需要报告额外的生成和验证成本。此外，应当区分不同类型的成功率。

例如，pass@k 通常关注多次尝试中是否至少有一次成功。而通过验证器选择后的最终成功率，取决于验证器能否从候选中识别正确结果。因此，候选集合包含正确答案，不意味着系统最终一定能够选中正确答案。

视频采样策略的影响 视频评测对采样协议更加敏感。例如，一段 60 秒视频中，关键动作只持续 0.5 秒。如果模型使用稀疏均匀采样，可能完全错过该事件。如果模型使用较高时间分辨率，则可能观察到动作全过程。因此，视频问答成绩受到采样帧率、采样位置和输入帧数影响。例如，两个模型即使处理相同视频，也可能因为采样策略不同而看到不同内容。因此，正式评测需要明确：最大帧数。采样策略。视频时间覆盖范围。帧时间戳。是否提供音频和字幕。

对于在线任务，还需要明确是否允许访问未来视频帧。如果某个模型在当前时刻使用了未来信息，而另一个模型只能根据历史信息进行预测，就不能将二者直接视为相同任务设置。

14.6.3 平均指标与分场景结果的差异

为什么总体平均分可能具有误导性 在模型研究中，研究者通常使用总体准确率、平均交并比或其他汇总指标比较不同方法。例如：模型 A 的总体准确率为 85%。模型 B 的总体准确率为 83%。从总体分数看，模型 A 似乎更好。但如果进一步分析不同场景，可能发现模型 A 在常见简单场景中表现较好，而模型 B 在复杂困难场景中具有明显优势。例如，模型 A 可能擅长普通白天图像，却在夜间、遮挡和长视频任务中表现较差。

因此，整体平均值只能反映特定数据分布和聚合规则下的总体性能。它并不能完整描述模型在所有场景中的表现。这一问题在多模态评测中尤其重要，因为不同模态和任务之间可能具有明显的难度差异。

微平均与宏平均 在多任务或多类别评价中，常见两种聚合方式。

微平均（Micro Average）通常先聚合相关的样本或基础计数，再计算整体指标。

宏平均（Macro Average）通常先计算各组指标，再按照规定权重进行平均。

例如，假设某个视觉问答数据集包含：普通对象识别：900 个问题。复杂空间推理：100 个问题。模型在普通对象识别上的准确率为 90%。在复杂空间推理上的准确率为 50%。则按样本数量计算的整体准确率为：

$$\frac{900 \times 0.9 + 100 \times 0.5}{1000} = 86\% \tag{14.48}$$

如果对两个任务组等权平均：

$$\frac{90\% + 50\%}{2} = 70\% \tag{14.49}$$

两者存在明显差异。86% 主要反映该数据集当前样本分布下的整体表现。70% 则

更强调两类任务具有相同重要性。因此，研究者需要根据评价目标选择适当聚合方式。对于 F1、IoU 等非线性指标，微平均和宏平均还有相应的具体定义，不能简单套用准确率公式。正式评价应遵循指标和数据集对应的聚合规则。

总体排名与场景权重 不同场景的样本权重可能直接改变模型排名。例如，考虑两个模型在简单场景和困难场景中的表现。如果测试数据中简单场景占 90%，困难场景占 10%，那么：

$$S_A = 0.9 \times 95 + 0.1 \times 55 = 91\% \tag{14.50}$$

$$S_B = 0.9 \times 90 + 0.1 \times 75 = 88.5\% \tag{14.51}$$

模型 A 的总体成绩更高。但如果两类场景各占 50%，则：

$$S_A = 0.5 \times 95 + 0.5 \times 55 = 75\% \tag{14.52}$$

$$S_B = 0.5 \times 90 + 0.5 \times 75 = 82.5\% \tag{14.53}$$

此时，模型 B 反而更好。这一例子说明，模型排名可能依赖测试数据的场景组成。这并不必然属于辛普森悖论。因为两个模型在不同场景之间存在性能取舍，权重变化自然可能改变整体排名。它主要说明：当模型在不同场景上的优势不一致时，聚合权重会直接影响最终结论。因此，比较模型时，应明确测试分布与实际目标应用分布之间的关系。

辛普森悖论与不一致的评价权重 辛普森悖论（Simpson's Paradox）通常指分组数据与汇总数据之间出现反直觉的统计关系，例如在各组内观察到某种优势，但汇总后由于组别比例不同而出现相反结论。在模型评价中，如果不同模型实际上使用了不同的测试样本构成，类似现象就可能出现。例如：模型 A 在简单场景中准确率为 95%，在困难场景中为 70%。模型 B 在简单场景中准确率为 90%，在困难场景中为 65%。

因此，模型 A 在两个场景中都优于模型 B。但假设模型 A 主要在困难场景中接受测试，而模型 B 主要在简单场景中接受测试。如果模型 A 的测试样本中简单场景占 10%，困难场景占 90%，则：

$$S_A = 0.1 \times 95 + 0.9 \times 70 = 72.5\% \tag{14.54}$$

如果模型 B 的测试样本中简单场景占 90%，困难场景占 10%，则：

$$S_B = 0.9 \times 90 + 0.1 \times 65 = 87.5\% \tag{14.55}$$

此时，模型 B 的汇总成绩反而高于模型 A。这一示例说明，采用不同的场景权重比较总体成绩，可能产生与各组内结果相反的结论。但如果两个模型使用完全相同的

场景集合和相同的非负权重，并且模型 A 在每个场景都优于模型 B，那么加权平均之后模型 A 不可能反而低于模型 B。因此，在公平的同协议模型比较中，不能随意将任何平均分排名变化都称为辛普森悖论。

真正需要警惕的是不同模型的有效样本集合、场景权重或数据缺失规则不一致，以及总体指标掩盖的真实场景差异。

样本分布与目标应用分布 测试集中的样本比例，不一定与真实应用中的场景比例完全相同。例如，一个公开基准可能包含大量普通白天图像。但实际系统需要在夜间或恶劣天气条件下运行。如果研究者只使用公开基准的原始平均分，可能无法准确反映目标应用性能。因此，可以根据实际部署分布定义目标加权指标。例如：

$$S = \sum_{g=1}^{G} w_g S_g \tag{14.56}$$

其中：

- S_g 表示第 g 个场景的性能；
- w_g 表示目标应用中该场景的权重；
-

$$\sum_{g} w_g = 1 \tag{14.57}$$

这种方法能够使综合评价更加贴近实际需求。

但权重应当根据应用目标或具有依据的数据分布确定，而不是为了获得有利排名而调整。此外，真实部署分布可能随时间变化。因此，某个固定加权平均值仍然不能完整代表所有未来应用条件。

平均表现与稳定性 两个模型可能具有相同平均准确率，但不同场景之间的性能波动不同。例如：模型 A 在三个场景中的准确率为：85%、85%、85%。模型 B 在三个场景中的准确率为：95%、85%、75%。如果三个场景具有相同权重，则两个模型的平均准确率都是 85%。但模型 A 在这些场景中表现更加均衡。模型 B 则在某个场景中具有更高成绩，同时在另一个场景中明显下降。因此，平均分不能完整表达模型的稳定性。

在适当情况下，可以进一步报告场景级性能方差或标准差。例如：

$$\sigma_S = \sqrt{\frac{1}{G-1}\sum_{g=1}^{G}(S_g - \bar{S})^2} \tag{14.58}$$

其中，S_g 表示各场景成绩，$\bar{S}$ 表示场景级平均成绩。但这个指标描述的是指定场景集合中的性能差异，并不是模型训练随机种子波动，也不能直接作为跨场景泛化不确定性的完整统计量。因此，需要区分不同类型的稳定性。

多维度评价与透明报告 HELM 是多维度模型评价的重要代表性研究。其核心思想之一，是在具有代表性的任务场景中，同时考虑准确性、鲁棒性、效率和其他相关维度。

这种方式能够避免单一指标掩盖模型在不同条件下的表现。对于多模态大模型，可以进一步构建包含不同任务和模态的评价矩阵。例如：这种评价方式能够为研究者和实际使用者提供更加完整的信息。但多维度报告并不意味着必须为所有任务使用相同指标。

例如，视频时间定位应使用适当的时间指标，图像检测应使用区域定位指标，工具执行则应检查实际任务状态。因此，多维度评价的重点，是让不同指标准确反映它们所对应的能力，而不是强行建立一个没有清晰依据的综合分数。

14.6.4 研究问题的提出、实验验证与创新评估

从技术想法到研究问题 多模态研究中经常出现新的技术想法。例如：增加一个注意力模块。引入新的模态。设计更加复杂的共享状态。使用动态门控选择不同信息源。增加额外的训练目标。这些想法可以成为研究的起点，但不一定已经构成明确的科学问题。例如，“设计一个更复杂的融合模块”主要描述了拟采用的方法，并没有说明为什么需要这个模块。

相比之下，“在现有融合方法容易受到低质量模态干扰的条件下，是否可以利用可验证的输入质量信息，提高独立场景中的任务稳定性”，则具有更加明确的问题背景和研究目标。因此，研究问题应当从现有方法的具体局限出发，而不是先确定网络结构，再寻找能够支持它的实验结果。

明确研究对象与适用范围 一个合理的研究问题需要明确对象和条件。例如，研究某种多模态状态维护机制时，需要说明：处理哪些模态？面向什么任务？输入具有怎样的时间特征？使用何种数据集和评价协议？现有方法在哪些场景存在不足？例如，“改善多模态长期记忆”过于宽泛。而“在低频 RGB 与高频异步 Event 观测条件下，检验有限状态能否在不重新处理完整历史的情况下，保持遮挡期间的目标定位性能”，则具有更加具体的研究范围。

明确范围能够帮助研究者确定必要数据、评价指标和对照方法。但研究范围较窄不意味着科学价值较低。一个针对关键机制的清晰实验，有时比覆盖多个任务但难以解释的复杂系统更有意义。

提出可以检验的研究假设 研究假设应当描述某种机制与预期结果之间的关系。例如：“当最新观测因遮挡而丢失目标信息时，利用历史状态能够提高目标定位准确率。”这一假设具有明确的条件和预期效果。可以进一步设置零假设：

$$H_0 : \mathbb{E}[M_{\text{state}} - M_{\text{baseline}}] \leq 0 \tag{14.59}$$

以及备择假设：

$$H_1 : \mathbb{E}[M_{\text{state}} - M_{\text{baseline}}] > 0 \tag{14.60}$$

其中，(M) 表示越大越好的任务指标。这种表达仅表示一种可能的统计检验形式。

实际研究还需要明确实验单位、样本相关性和统计假设。例如，同一视频中的连续帧通常具有相关性，不能无条件将每帧视为完全独立样本。因此，研究假设需要与实际数据结构匹配。

统计显著性与效应量 某个模型比基线高出少量指标，并不一定意味着具有稳定收益。例如，不同随机种子、数据划分和训练过程可能产生一定波动。因此，需要结合多次实验或适当的统计方法分析结果。例如，可以计算配对性能差：

$$\Delta_i = M_{\text{new},i} - M_{\text{base},i} \tag{14.61}$$

其中，(i) 表示预先定义的独立实验单位。随后分析平均差异、置信区间和不同单位之间的变化。但实验单位应根据数据结构确定。例如，对于具有较强时间相关性的视频数据，可以优先考虑以录制或场景为单位开展统计分析，而不是将相邻帧都视为独立样本。此外，统计显著性并不等于实际应用价值。例如，一个方法通过大量样本获得统计上显著的微小提升，但额外计算成本可能较高。因此，还需要评价效应量和资源代价。

研究中的计算预算 新方法通常会增加模型参数、训练计算或推理开销。因此，如果只比较任务准确率，可能无法充分判断其实际价值。例如，一个共享状态机制在某个任务上获得提升，却需要明显增加更新和查询延迟。这种结果并不一定说明方法没有研究意义，但需要明确其适用条件。因此，可以采用两种互补的比较方式。第一种是在固定计算预算下比较任务性能。第二种是在固定任务质量要求下比较计算成本。此外，还可以报告完整的性能—成本曲线，而不是只选择单个实验点。

这种方式能够更加全面地分析新方法的效率和适用范围。

值得持续深入的研究方向 综合前面的讨论，多模态大模型仍然具有多个值得持续关注的基础研究方向。是多模态推理机制。研究者需要进一步区分模型对视觉、语言、音频和时序信息的实际利用方式，并通过受控实验分析复杂推理能力的来源。是跨模态表示的合理边界。需要研究共享语义与模态专属信息之间的关系，以及不同对齐方式对任务性能和信息保留的影响。是长期状态的必要性与有效性。

需要检验持续状态是否真正保留了最新观测无法替代的历史信息，以及是否能够在有限计算预算下支持多个任务。此外，是可靠性与泛化。模型需要在模态缺失、环境变化、输入噪声和历史信息过期等条件下维持合理表现，并在证据不足时避免给出过度确定的判断。是研究方法本身。需要建立更加严格的基线复现、评价协议、模块消融和替代解释检验方法，使多模态研究结论具有可重复性和可解释性。

这些问题具有共同特点：它们不一定能够通过简单扩大模型规模解决，而需要结合任务分析、模型机制和实验方法开展系统研究。

习题与讨论

1. 为一个高分辨率模型与低分辨率基线设计公平比较。区分相同训练样本、Token、FLOPs 和墙钟预算。
2. 同一视频产生多个问题。为什么逐题独立 bootstrap 可能过度乐观？提出与目标总体相容的重采样单位。
3. 自动裁判偏好更长回答。设计顺序交换、长度控制和人工子集检查，说明每项检验能排除什么。
4. 总体平均分提升但某个重要子组下降。设计同时报告样本量、区间、错误代价和部署相关性的结果表。

文献导读 MMBench 与 MMMU 用于对照评价任务和协议，FID 用于理解生成分布指标的估计对象。基准名称相同不代表输入和推理预算相同。[32, 29, 31]

第十五章　幻觉、鲁棒性与可靠性

可靠性讨论应落实到错误类型、观测条件与处置规则。模型可能遗漏证据、引用错误来源、在错配条件下自信回答，也可能在拒答后把风险转移到回退模块。需要评价完整流程，而不仅是回答中是否出现一个错误名词。

学习目标　按证据关系分类幻觉；区分缺失、噪声和错配；正确解释置信度、校准与覆盖；检验拒答和外部校验的系统后果。

15.1　证据不一致的类型与成因

先规定什么内容能够由当前观测验证，再定义错误。例如，物体不存在、属性绑定错误和关系倒置可有不同判定；画面无法证实的意图不宜直接作为确定事实。数据中的未标注对象与真正不存在的对象也要区分。

15.1.1　物体、属性与关系幻觉

物体幻觉的基本概念　物体幻觉（Object Hallucination）是多模态幻觉中较早受到系统研究的一种类型。它通常表现为模型在描述图像时，生成了实际上不存在的物体。例如，一张室内图像中包含：桌子、椅子、书本、台灯。模型却回答：“桌子上放着一台电脑，旁边有几本书。”如果图像中没有电脑，那么“存在电脑”这一主张就属于物体幻觉。物体幻觉并不一定表现为完整句子。例如，当用户询问：“图中是否存在电脑?”

模型回答“是”，也可能构成对象存在性幻觉。因此，物体幻觉既可能出现在开放式图像描述中，也可能出现在针对具体对象的问答任务中。

对象存在性错误与对象遗漏　在视觉任务中，需要区分两种不同错误。第一种是模型错误地声称某个不存在的对象存在。例如，图像中没有猫，模型却描述存在猫。第二种是模型遗漏了实际存在的对象。例如，图像中存在猫，但模型回答“没有”。前者通常被视为对象幻觉的典型形式。后者则更直接地表现为漏检或感知遗漏。两者都属于视觉内容理解错误，但错误方向不同。在二元对象判断中，可以使用混淆矩阵分析：这种区分有意义。例如，某种模型可能倾向于对大量对象都回答“存在”。

此时，模型可能获得较高召回率，却具有较高误报率。另一种模型可能过度保守，经常回答“不存在”，从而减少对象幻觉，但也可能遗漏真实对象。因此，幻觉评价不能只关注误报下降，还应同时检查正常感知能力是否受到影响。

CHAIR：图像描述中的对象幻觉　Rohrbach 等人在《Object Hallucination in Image Captioning》中系统研究了图像描述模型的对象幻觉问题，并提出 CHAIR[33] 指标。

CHAIR 的基本思想，是根据图像中的真实对象信息，检查生成描述是否包含不存在的对象。该指标主要包括两个形式。CHAIRi 衡量生成文本中被提及的对象实例中，有多少属于幻觉对象。可以表示为：

$$\text{CHAIR}_i = \frac{N_{\text{hallucinated objects}}}{N_{\text{mentioned objects}}} \tag{15.1}$$

CHAIRs 衡量生成描述中，有多少比例包含至少一个幻觉对象。可以表示为：

$$\text{CHAIR}_s = \frac{N_{\text{captions with hallucination}}}{N_{\text{captions}}} \tag{15.2}$$

两种指标关注不同层面。例如，一段描述可能包含多个不存在的对象。CHAIRi 更加关注对象提及层面的错误比例。CHAIRs 则关注一段描述是否至少存在一次对象幻觉。CHAIR 的原始实现主要基于 MS-COCO 中的对象类别、分割标注和参考描述进行判断。因此，其检测范围受到预定义对象类别和标注完整性的影响。例如，图像中实际存在某个未被相应标注体系覆盖的对象，不能仅因为标注缺失就认定模型描述了不存在的内容。

所以，CHAIR 是评价对象幻觉的重要工具，但并不能完整覆盖所有类型的多模态幻觉。

空间关系中的参考系问题 空间关系幻觉还可能与参考系混淆有关。例如，用户询问：“汽车位于人物的左侧还是右侧？”如果问题没有明确以观察者还是人物自身为参考，可能存在解释歧义。对于具有明确参考系的任务，模型需要根据相应坐标定义判断。例如，图像中的左侧，与人物面向方向的左侧，不一定相同。此外，二维图像中的相邻关系，不一定等于真实三维空间中的近距离关系。因此，空间关系评价需要区分：图像平面位置、相机坐标关系、对象自身参考系。真实三维空间关系。

如果模型将二维投影关系错误解释为真实三维关系，可能产生空间判断错误。但如果问题本身缺少必要参考系信息，则不宜强行将某一种解释视为唯一正确答案。

不同视觉幻觉类型的联系 物体、属性、关系和数量幻觉之间存在联系。例如，一个模型可能首先错误识别了一个不存在的人物。随后又描述该人物穿着红色衣服，并站在汽车旁边。这样，原始对象幻觉会进一步引发属性和关系层面的错误。也可能出现相反情况。例如，模型正确识别全部对象，但错误理解人物与汽车之间的空间关系。因此，错误并不一定都来自对象检测。可以将主要视觉幻觉形式概括如下。这些类型可以共同出现在同一个回答中，因此实际评价需要允许多标签错误分析。

开放式幻觉与问答式幻觉 在多模态大模型中，幻觉既可能发生于针对性问答，也可能发生于自由生成。例如，在问答任务中，用户明确询问某个对象是否存在。模型只需要作出存在性判断。但在开放式描述任务中，模型可以自由决定描述哪些对象和属性。因此，开放式生成具有更大的内容空间，也可能产生更多额外事实性主张。THRONE 是面向视觉语言模型自由形式回答中的对象幻觉提出的基准。

该研究强调，针对性二元问答和开放式生成中的幻觉具有不同特点，某些减少问

答式幻觉的方法未必能够同步改善自由生成中的错误。因此，评价多模态幻觉时，需要明确输出形式。不能仅根据二元问答中较低的误报率，就判断模型在长篇图像描述中同样具有较高可靠性。

15.1.2 图像证据与语言输出不一致

从视觉观察到语言生成 视觉语言大模型通常由视觉编码模块、跨模态连接模块和语言生成模块等部分组成。例如，输入图像首先经过视觉编码器提取特征。然后通过投影层或其他连接机制，将视觉特征转换为语言模型能够处理的表示。语言模型结合用户问题生成自然语言回答。可以将这一过程概括为：$Z_v = f_v(I)$，$H_v = P(Z_v)$，$Y = f_{\mathrm{LM}}(H_v, Q)$，其中：

- I 表示输入图像；
- f_v 表示视觉编码器；
- Z_v 表示视觉特征；
- P 表示跨模态连接或映射；
- H_v 表示语言模型使用的视觉表示；
- Q 表示问题；
- Y 表示生成回答。

这种结构使模型能够利用视觉信息进行语言交互。

但它也引入了多个可能的信息损失环节。例如，视觉编码器可能没有保留某些细节。跨模态映射可能无法完整传递局部对象关系。语言模型则可能根据已有语言知识补充缺失信息。因此，视觉信息进入模型，并不意味着最终生成的全部内容都受到视觉证据约束。

视觉特征与原始图像之间的信息损失 视觉编码器通常需要将高维图像转换为有限数量的特征。例如，一张高分辨率图像中可能包含数百万像素，但模型最终使用的视觉 Token 数量通常有限。因此，在视觉编码和压缩过程中，部分信息可能无法完整保留。例如：远处的小型交通标志。文档页脚中的细小文字。复杂场景中的少数对象。多个相似物体之间的细粒度差异。如果后续问题恰好依赖这些信息，模型就可能无法可靠回答。但语言模型通常仍然需要根据提示生成输出。

因此，当视觉信息不足时，模型可能利用常见场景知识或语言先验填补缺失内容。例如，模型看见一个模糊的厨房场景，可能根据经验描述微波炉或冰箱，即使图像中无法确认这些物体。这种情况说明，幻觉可能与视觉信息压缩和局部细节丢失有关。但并不是所有幻觉都能归因于视觉编码器。即使视觉特征包含足够信息，后续的跨模态推理和语言生成仍然可能发生错误。

视觉证据与语言先验的竞争 语言模型在大规模文本预训练中学习到丰富的对象关系和常识。例如，厨房通常包含冰箱、锅具和餐具。道路可能包含汽车、行人和交通标志。这些知识有助于模型理解场景。但如果模型过度依赖常见共现关系，就可能忽略

当前图像中的实际证据。例如，输入图像中只有一个空桌面。模型可能根据“桌子上通常放着杯子或书本”的经验，错误描述某个不存在的对象。这种现象可以理解为视觉证据与语言先验之间的权衡问题。从条件概率角度，可以表示为：

$$P_\theta(Y \mid I, Q) \tag{15.3}$$

如果模型对图像 I 的有效利用不足，其输出可能更多受到问题 Q 和语言先验影响。这只是一种概念性解释，并不意味着能够将模型的实际概率分布简单、唯一地分解为视觉项和语言项。真正的因果来源需要通过控制输入和模型行为的实验进一步检验。

回答长度与事实主张数量 在开放式描述任务中，模型生成的文字越多，通常包含的事实性主张也可能越多。例如，回答：“图中有一辆汽车。”只包含较少信息。而回答：“图中有一辆红色汽车，停在商店门口，驾驶员正在等待同伴，商店里正在进行促销活动。”则包含更多关于颜色、位置、人物意图和场景事件的主张。如果图像只清楚显示汽车，那么后面部分主张可能缺乏证据。因此，开放式幻觉评价需要考虑回答长度和事实性主张数量。

例如，可以统计所有可检查主张中，不正确或缺乏支持的比例。但这种统计方法需要明确主张分解规则。例如，模型生成的某些句子可能只是一般性解释或合理预测，不应全部按照直接视觉事实评价。因此，应当结合语义类型和证据要求进行判断。

幻觉与拒答之间的权衡 为减少幻觉，模型可能更加倾向于拒绝回答。例如，对于所有难以确认的问题都回答：“无法判断。”这种策略可能降低错误肯定回答的数量。但如果模型对清晰且可以回答的问题也频繁拒答，就会影响任务完成能力。因此，幻觉率下降并不一定意味着模型整体能力提高。例如，一个模型可能通过大量回答“不确定”，使对象存在性误报率降低。但其对象识别召回率也可能明显下降。因此，评价时需要同时考虑：正确回答率、幻觉或误报比例、拒答比例、有效回答覆盖率。

必要时，还可以分析不同置信度阈值下的风险—覆盖率关系。可靠的多模态系统应当根据证据质量合理表达确定性，而不是简单地始终肯定回答或始终拒答。

15.1.3 训练数据、模型偏差与幻觉

多模态幻觉并非单一原因造成 多模态幻觉可能来自不同处理阶段。例如：训练数据中的统计偏差。视觉编码中的信息损失。跨模态连接中的表示不匹配。语言模型对常见表达的偏好。生成过程中的信息偏移。复杂推理中的错误传播。因此，不能将所有幻觉简单归因于语言模型“喜欢编造答案”。同样，也不能认为只要提高视觉编码器准确率，就能够消除全部幻觉。更合理的研究方式是根据具体任务，分析错误在什么条件下出现，以及哪些实验因素与错误具有直接联系。

训练数据中的对象共现偏差 视觉语言模型通常使用大规模图文数据训练。在这些数据中，不同对象之间存在明显的统计共现关系。例如，厨房场景经常与锅、冰箱和餐具共同出现。办公室场景经常与电脑、桌子和键盘共同出现。道路场景经常与汽车、交

通标志和行人共同出现。这些统计关系能够帮助模型理解常见环境。但也可能使模型倾向于在某种场景中补充实际不存在的对象。例如，看到桌子就预测电脑。看到厨房就预测微波炉。

POPE[25] 对象幻觉研究指出，训练或指令数据中的对象频率及对象共现关系，可能影响视觉语言模型的对象幻觉表现。因此，数据中的常见统计关联既可能提高模型的语义理解能力，也可能在当前视觉证据不足时引发错误补全。对象共现与幻觉之间的相关性，并不意味着每次幻觉都由共现偏差直接造成。例如，小目标识别失败也可能产生类似结果。因此，研究者需要通过控制图像内容和语言条件的实验进一步验证原因。

数据覆盖不足与长尾对象 多模态预训练数据通常具有不均衡分布。例如，某些日常对象出现频率较高。罕见工业设备、特殊医学影像或某些地域文化中的对象，可能出现次数较少。因此，模型可能对常见对象具有较好的识别能力，却难以正确理解长尾对象。例如，一张工业场景图像中包含特殊机械部件。模型可能根据外观将其错误识别为常见工具。随后又围绕这一错误类别生成用途和结构描述。这种情况可能同时涉及类别识别错误和后续事实性幻觉。

因此，训练数据覆盖不足可能增加部分场景中的幻觉风险。但罕见对象识别错误不一定全部属于幻觉。如果模型明确表示无法识别，或者仅提供具有不确定性的描述，就不应简单与确定性虚构内容等同。

推理时计算与幻觉风险 复杂多模态任务可能采用分步推理、多候选生成或较长的推理过程。这类方法可以帮助模型完成部分数学和逻辑任务。但额外推理计算并不一定改善视觉证据正确性。例如，模型最初错误读取一个图表数值。随后生成多个推理步骤，可能只是围绕错误数值进行更复杂的计算。因此，最终解释可能更加完整，但其事实基础仍然错误。

Xu 等人在《More Thinking, Less Seeing? Assessing Amplified Hallucination in Multimodal Reasoning Models》中研究了多模态推理过程中推理长度与视觉幻觉之间的关系。该研究发现，在部分多模态推理模型和任务中，较长推理过程可能伴随对视觉信息利用不足和幻觉增加的问题。因此，推理时计算需要同时关注逻辑能力和感知可靠性。

但这一结果不意味着较长思维链在所有任务中都不利。对于具有可靠外部证据验证的任务，额外计算仍然可能带来收益。关键是判断计算是否真正用于检查和利用证据，而不只是增加语言生成长度。

区分相关性与因果归因 幻觉研究中的一个重要原则，是不能将观察到的相关现象直接解释为确定的因果机制。例如，研究者可能发现：回答越长，幻觉率越高。但这一现象可能受到任务难度和事实性主张数量影响。因此，不能只根据相关性就认定回答长度本身是全部错误的根本原因。类似地，模型在某类对象上具有较高幻觉率，可能与对象共现偏差有关，也可能与视觉识别困难或数据标注质量有关。因此，若希望分析某个机制是否导致幻觉，需要设计合理的对照实验。例如：

保持图像不变，改变问题文本。保持问题不变，改变关键对象或属性。比较有无视觉输入的结果。改变输入分辨率或局部区域信息。在控制其他因素的条件下调整推理策略。通过这些实验，可以分析幻觉与特定信息来源之间的关系。但输入干预可能同时改变多个因素。例如，遮挡某个图像区域不仅删除目标信息，也可能改变整体视觉特征。因此，实验解释应与实际控制条件保持一致。

多模态幻觉研究的基本方向 综合前面的讨论，减少幻觉可以从不同层面展开。可以提高训练数据中的模态对应质量，并减少明显错误的监督信息。可以增强视觉和音频编码器对关键细节、对象身份和时间变化的表示能力。可以改善跨模态连接，使对象属性、空间关系和声音事件之间的对应更加稳定。此外，可以在推理阶段加入证据重检、外部工具验证或其他适当机制。对于证据不足的任务，还可以改进模型的不确定性表达和拒答策略。但这些方法的效果需要通过公平实验验证。

例如，减少对象幻觉的方法不一定能够改善关系幻觉。改进静态图像证据利用的方法，也不一定能够解决长视频历史事件混淆。因此，幻觉研究需要根据错误类型和实际证据来源，选择具有针对性的改进方法。多模态幻觉通常不是单一模块的独立问题，而是数据分布、感知表征、跨模态融合、语言生成与推理过程共同作用的结果。

15.2 缺失、扰动、错配与分布变化

15.2.1 模态缺失与信息不完整

模态缺失的基本含义 模态缺失（Missing Modality）是指多模态系统在训练或推理过程中，无法获得原本预期的某种输入信息。例如，一个 RGB-深度融合系统需要同时接收彩色图像和深度图。在正常情况下，模型输入为：

$$X = (X_{\text{RGB}}, X_{\text{Depth}}) \tag{15.4}$$

但如果深度传感器失效，系统可能只能获得：$X' = X_{\text{RGB}}$ 此时，模型需要在信息不完整的条件下完成任务。类似地，音视频交互系统可能只有音频而没有视频，或者只有视频而没有音频。对于部分复杂系统，缺失还可能发生在单个时间片段、局部空间区域或某些特征通道上。因此，模态缺失并不一定意味着某个传感器永久失效，也可能表现为暂时性或局部性的信息不可用。

Wu 等人在《Deep Multimodal Learning with Missing Modality: A Survey》中系统梳理了深度多模态学习中的模态缺失问题，涉及缺失信息重建、鲁棒融合、表示学习和其他相关技术路线。这说明，模态缺失并不是某一种应用系统中的特殊故障，而是多模态学习普遍需要面对的实际问题。

完整缺失、局部缺失与间歇缺失 根据缺失范围，可以区分不同形式。完整模态缺失是指某种输入在整个任务过程中不可用。例如，音视频模型始终无法获得音频。局部信息缺失是指模态仍然存在，但只有部分内容可用。例如，深度图存在大面积无效像素。

间歇性缺失是指模态在某些时刻可用，在其他时刻不可用。例如，摄像头在视频传输过程中发生短暂帧丢失。非连续采集则是某些模态本身采用低频或按需采样机制。例如，某个视觉系统每隔一定时间才获取一帧 RGB 图像。

非连续采集不一定属于故障意义上的模态缺失。如果系统原本就按照异步、多频率方式设计，那么两个 RGB 帧之间没有新 RGB 输入可能是正常状态。因此，评价时需要区分预期采样机制与异常数据缺失。

模态缺失的数学表示 可以使用一个模态可用性向量表示当前输入条件：

$$\mathbf{m} = (m_1, m_2, \ldots, m_M) \tag{15.5}$$

其中：

$$m_i = \begin{cases} 1, & \text{第} i \text{ 种模态可用} \\ 0, & \text{第} i \text{ 种模态不可用} \end{cases} \tag{15.6}$$

例如，对于 RGB、深度和音频三种模态：$\mathbf{m} = (1, 0, 1)$ 表示 RGB 和音频可用，而深度缺失。在工程实现中，可以为缺失模态指定占位输入，并同时提供可用性掩码。例如：$\tilde{X}_m = m_m X_m + (1 - m_m) c_m$，其中，$c_m$ 表示预先定义的占位输入。但占位输入本身并不等同于真实缺失状态。例如，用全零图像表示缺失 RGB，可能与某张实际接近全黑的图像在数值上相似。

对于事件相机，某个时间窗口中没有事件，也可能是完全正常的观测结果，而不一定表示传感器失效。因此，缺失模态的处理最好明确区分数据内容与模态可用性。不能仅根据输入张量是否为零，直接判断模态是否缺失。

模态缺失的评价协议 评价模态缺失鲁棒性时，可以设置不同的可用性条件。例如，对于 RGB、深度和音频三模态系统：在实际实验中，应根据任务信息需求选择合理组合。例如，完全没有有效信息的配置，可能不具备可求解性。此外，还可以设置不同缺失比例。例如，10%、30% 和 50% 的时间片段发生模态缺失。随机缺失比例不能充分代表连续故障。例如，随机丢失 30% 的视频帧，与连续丢失一个较长时间区间，可能对时间推理产生不同影响。

因此，最好分别评价随机缺失、连续缺失和真实故障条件。

完整输入性能与缺失鲁棒性 评价缺失鲁棒性时，应同时报告完整输入性能和缺失条件性能。例如：S_{full} 表示完整输入成绩。S_{missing} 表示指定模态缺失后的成绩。对于越高越好的指标，可以定义性能下降：

$$\Delta S = S_{\text{full}} - S_{\text{missing}} \tag{15.7}$$

例如，完整输入准确率为 85%，缺失深度后的准确率为 75%。则性能下降为 10 个百分点。但性能下降越小，并不一定说明模型越好。例如，模型 A 的完整输入准确率为 60%，缺失后为 58%。模型 B 的完整输入准确率为 90%，缺失后为 80%。模型 A

的下降幅度更小，但模型 B 在两种条件下的绝对成绩都更高。因此，应同时比较绝对性能与相对退化幅度。此外，还应加入单模态基线。

这样才能判断融合模型在模态缺失后，是否仍然优于直接使用剩余模态的简单方法。

15.2.2 噪声、遮挡与图像质量退化

输入扰动与模态缺失的区别 模态缺失通常指某种输入信息完全或局部不可用。输入扰动（Input Perturbation）则强调模态仍然存在，但其观测质量发生变化。例如：图像出现噪声。视频发生运动模糊。音频受到背景噪声干扰。深度图出现测量误差。激光雷达点云发生稀疏化。事件相机产生额外噪声事件。这些情况下，传感器可能仍然正常输出数据，但数据与真实环境之间的对应关系受到影响。

因此，扰动鲁棒性不仅涉及输入是否可用，还涉及模型能否识别并合理利用质量下降的信息。

图像噪声 图像噪声是视觉系统中的常见扰动。例如，在低照度条件下，摄像头可能产生较明显的随机噪声。可以使用一个简化模型表示：$\widetilde{I} = I + \epsilon$，其中：

- I 表示理想图像；
- ϵ 表示噪声；
- $\widetilde{I}$ 表示受噪声影响的图像。

但真实图像噪声不一定符合简单加性模型。

例如，图像传感器噪声可能与光照强度、曝光时间和设备特性有关。因此，人工高斯噪声主要是一种可控测试条件，不能完整代表真实传感器噪声。对于多模态系统，如果某个模态受到噪声影响，而其他模态仍然可靠，融合机制可能利用剩余信息维持性能。但如果模型无法识别受扰模态的质量下降，也可能错误利用噪声特征，导致性能变差。

扰动强度与性能退化曲线 输入扰动通常具有不同强度。例如，图像噪声可以从轻微逐渐增加到严重。因此，仅使用一个扰动强度评价模型，可能无法完整反映其鲁棒性。可以将扰动条件下的性能表示为：$S(\delta)$，其中，δ 表示扰动强度。例如：以上按正文整理评价方法与记录字段，未另设示意成绩；实际比较应使用原始测试结果。通过不同强度的连续测试，可以分析模型何时开始明显退化。还可以比较两个模型在不同扰动区间中的表现。

例如，某个模型可能在轻度噪声下较稳定，但在严重噪声下迅速下降。另一个模型可能正常输入准确率略低，但在较强噪声下表现更好。因此，扰动鲁棒性应当结合正常性能和完整退化曲线评价。

多模态扰动的组合效应 实际环境中的扰动可能同时影响多个模态。例如，恶劣天气可能同时降低 RGB 图像和激光雷达的质量。某些环境噪声可能同时影响语音识别和音视频事件理解。因此，模型不能简单假设不同模态之间的噪声相互独立。例如，在雨

雾场景中，RGB 和某些距离传感器可能同时受到环境影响。如果融合模型始终认为至少有一种模态可靠，就可能错误估计系统能力。因此，可以在实验中分别设置：单模态受扰。多个模态独立受扰。多个模态受到共同环境因素影响。

但组合扰动可能显著增加测试配置数量。因此，应根据实际部署风险选择具有代表性的组合，而不是机械枚举所有不合理情况。

受扰模态是否真的提供帮助 多模态融合系统在正常条件下可能优于单模态模型，但在受扰条件下未必如此。例如，一个 RGB-深度融合模型在完整输入时取得较高准确率。但当深度图出现大量错误测量时，模型可能因为使用了不可靠深度信息而产生错误预测。这种情况下，融合性能甚至可能低于仅使用 RGB 的模型。因此，评价时应当包含同一条件下的单模态基线。例如：正常 RGB-only。正常深度-only。正常 RGB+Depth。受扰 RGB+Depth。

受扰条件下仅使用仍然可靠的模态。通过这些对照，可以分析多模态融合是否真正具有抗扰动能力。但不能仅因为融合结果下降，就认定融合机制本身无效。还需要判断退化来自输入质量、模型对噪声的利用方式，还是任务中信息本身已经不足。

扰动鲁棒性与幻觉 输入退化可能增加多模态幻觉风险。例如，模糊图像中的对象难以识别，模型可能根据场景常识描述不存在的细节。音频受到强噪声干扰时，模型可能将噪声解释为某种具体声音事件。但鲁棒性下降与幻觉并不是完全相同的现象。例如，噪声可能使模型正确率下降，但模型始终明确表示无法判断。这种情况下，系统性能受到影响，却不一定产生无依据的确定性陈述。因此，扰动评价可以分别报告任务性能、幻觉率和拒答比例。

这样有助于分析模型究竟是产生错误内容，还是因为不确定性增加而减少回答。

15.2.3 模态错配与时间不同步

模态错配的基本概念 多模态融合不仅需要获得不同模态的信息，还需要正确建立模态之间的对应关系。例如，在音视频系统中，人物说话时的口型与声音应当具有合理的时间关系。在 RGB-深度系统中，深度值应当对应正确的图像像素或空间区域。在 RGB-Event 系统中，事件流中的时间信息应当与相应的 RGB 观测建立正确联系。如果不同模态的时间或空间关系发生错误，就可能形成模态错配（Modality Misalignment）。

例如，模型将前一时刻的声音与后一时刻的视频对应。或者将深度图中的某个区域与 RGB 图像中的错误对象关联。这种情况下，即使每种模态单独都具有较高质量，融合结果仍然可能不可靠。因此，模态对应关系的正确性，是多模态融合的重要前提。

时间同步与采样频率差异 不同传感器通常具有不同采样频率。例如，摄像头可能以每秒 30 帧采集图像。音频通常以更高采样率记录连续波形。事件相机则异步记录亮度变化事件，而不是以固定帧率输出完整图像。因此，不同模态不能简单按照相同数组索引进行一一对应。例如：RGB 第 10 帧。并不意味着它对应音频中的第 10 个采样点。因此，多模态系统通常需要根据时间戳建立对应关系。可以将不同模态的观测序列表示为：

$$X_m = (x_{m,i}, t_{m,i})_{i=1}^{N_m} \tag{15.8}$$

其中：

- $x_{m,i}$ 表示第 m 种模态的第 i 个观测；
- $t_{m,i}$ 表示对应时间戳。

不同模态的时间戳集合可以不同。

因此，模型需要根据任务选择合适的时间匹配、插值或异步融合方式。

时间偏移与时间漂移 时间不同步可能表现为固定时间偏移，也可能表现为随时间变化的漂移。例如，某个传感器的时间戳与参考时间之间存在固定偏差：$\widetilde{t}_m = t + \beta_m$，其中，$\beta_m$ 表示时间偏移。如果设备时钟速率不同，则还可能出现：

$$\widetilde{t}_m = \alpha_m t + \beta_m \tag{15.9}$$

其中：

- α_m 表示时钟速率比例；
- β_m 表示初始时间偏移。

当：$\alpha_m \neq 1$ 时，时间误差可能随观测时间逐渐累积。

实际系统还可能存在随机延迟和时间抖动。例如，网络传输可能使某些数据包晚于预期到达。因此，时间同步不仅涉及平均时间偏移，也涉及时间戳精度、时钟漂移和数据到达延迟。数据到达时间与传感器实际采集时间并不一定相同。因此，连续感知系统应尽量明确所使用的时间语义。

MulT：**未严格对齐的多模态序列** Tsai 等人在《Multimodal Transformer for Unaligned Multimodal Language Sequences》中提出 MulT（Multimodal Transformer）。该研究关注语言、视觉和声学序列之间的非严格对齐问题。例如，在人类交流中，语音、面部表情和身体动作可能以不同采样频率记录。因此，模型不能总是假设不同模态的每个时间步都具有精确的一一对应关系。

MulT 使用跨模态注意力，在不同时间步之间建立信息交互。这使模型能够在不预先进行严格时间对齐的条件下，学习某些跨模态关系。该研究说明，多模态融合不一定要求所有输入在时间轴上形成完全一致的采样网格。但能够处理未严格对齐的输入，不等于模型能够自动纠正任意传感器时钟错误。例如，严重的时间偏移仍然可能破坏事件之间的真实对应关系。因此，需要根据具体任务区分非严格对齐建模与时间同步故障恢复。

空间错配与几何标定 模态错配还可能发生在空间维度。例如，RGB 图像和深度图可能来自不同传感器。如果两者之间的内参、外参或几何映射存在误差，就可能将深度信息关联到错误像素。类似地，激光雷达点云投影到图像平面时，需要使用正确的坐标变换。例如，三维点可以通过相机投影模型映射到图像坐标：$\tilde{p} = K(R\mathbf{X} + t)$，其中：

- K 表示相机内参；
- R,t 表示坐标系之间的旋转和平移；
- **X** 表示三维点；
- (u,v) 表示图像坐标。

如果标定参数错误，投影位置可能发生偏移。

因此，融合模型可能将某个对象的深度信息关联到邻近对象。对于目标检测、分割和空间推理，这种错误可能影响下游结果。空间标定误差和时间错配可能同时存在。例如，运动场景中的时间偏移也可能使空间投影出现位置不一致。因此，分析时需要根据具体传感器配置和运动状态判断错误来源。

模态错配的受控评价 为了评价时间错配鲁棒性，可以人为引入不同程度的时间偏移。例如：

$$\widetilde{X}_a(t) = X_a(t + \Delta t) \tag{15.10}$$

其中，Δt 表示人为设置的时间偏移。可以设置多个偏移程度。例如：

$$\Delta t \in -200, -100, 0, 100, 200 \text{ ms} \tag{15.11}$$

这些数值仅为说明实验设计的示例，并不是适用于所有任务的统一标准。对于不同传感器，需要结合采样频率和事件时间尺度设置合理偏移。例如，200 毫秒对缓慢变化的场景可能影响较小，但对快速动作和短暂声音事件可能产生明显影响。此外，还可以测试：随机时间抖动、连续时间漂移、空间标定偏差、错误的模态配对。但人工扰动不能直接代替真实同步故障。因此，最好将受控错配与真实传感器记录中的误差分析结合起来。

时间错配评价的基本原则 评价多模态时间错配时，需要明确几个条件。所有输入应当具有可信的原始时间戳或对应关系。应说明人为扰动如何施加，以及是否改变了原始数据内容。需要区分时间偏移和时间采样稀疏。此外，对于在线因果任务，还需要确保测试过程中没有使用未来信息。应当同时报告原始对齐性能和不同错配强度下的性能。通过这些条件，可以分析模型是否真正具有时间关联鲁棒性，以及性能下降来自何种扰动。

15.2.4 分布偏移与跨域泛化

分布偏移的基本概念 机器学习模型通常在有限的数据分布上训练。例如，某个视觉模型主要使用晴天道路图像训练。部署时却需要处理夜间、雨雾或其他城市环境。此时，测试数据与训练数据之间可能存在统计分布差异。这种情况通常称为分布偏移（Distribution Shift）。可以用以下形式表示：

$$P_{\text{train}}(X, Y) \neq P_{\text{test}}(X, Y) \tag{15.12}$$

其中：

- P_{train} 表示训练数据分布；
- P_{test} 表示测试或部署数据分布。

分布偏移可能影响输入特征、标签比例，以及输入与标签之间的关系。

因此，即使模型在训练分布中具有较高准确率，也可能在新分布中表现下降。输入分布发生变化并不必然导致性能下降。如果模型学习到了适当的稳定规律，仍然可能在新环境中保持较好表现。因此，分布偏移与模型泛化失败并不是完全等价的概念。

协变量偏移 协变量偏移（Covariate Shift）通常指输入分布发生变化，而在一定假设下，条件标签关系保持不变。例如：

$$P_{\text{train}}(X) \neq P_{\text{test}}(X) \tag{15.13}$$

同时假设：

$$P_{\text{test}}(Y \mid X) \tag{15.14}$$

例如，同一种对象可能在不同光照和背景条件下出现。输入图像的统计特征发生变化，但对象类别本身没有改变。因此，模型需要在新的输入分布下识别相同类别。但这是一种统计建模假设。例如，对于严重低照度图像，部分关键视觉信息可能已经不可观测，此时简单认为条件标签关系完全不变可能并不合适。因此，实际研究需要根据任务判断是否符合相应假设。

标签分布偏移 标签偏移（Label Shift）通常指不同类别的出现比例发生变化。例如：

$$P_{\text{train}}(Y) \neq P_{\text{test}}(Y) \tag{15.15}$$

在经典标签偏移假设下，还要求类别条件输入分布保持不变：

$$P_{\text{test}}(X \mid Y) \tag{15.16}$$

例如，训练数据中大部分交通场景来自普通城市道路。但部署环境中，某些特殊车辆或交通标志出现得更加频繁。这可能改变模型在实际部署中的总体错误率。真实分布变化可能同时影响类别比例和对象外观，因此未必满足严格的标签偏移假设。因此，研究中应区分纯粹的理论类别偏移与更加复杂的真实场景变化。

概念偏移与条件关系变化 概念偏移（Concept Shift）通常强调输入和目标之间的条件关系发生变化。例如：

$$P_{\text{train}}(Y \mid X) \neq P_{\text{test}}(Y \mid X) \tag{15.17}$$

这意味着相似输入条件下，正确标签或决策规则可能发生变化。例如，在某些业务系统中，任务标签的定义或决策标准发生改变。对于感知任务，类似现象可能来自标注协议或目标概念变化。真实应用中的不同分布偏移类型可能同时存在。因此，上述分类主要用于分析变化来源，并不意味着每个实际问题都能够被唯一划入某一种类型。

多模态分布偏移的特殊性 多模态系统中的分布偏移不仅可能影响单一模态，也可能影响模态之间的关系。例如，在某种训练环境中，汽车画面通常伴随清晰的发动机声音。但在实际环境中，车辆可能距离较远，声音受到遮挡或混杂。因此：

$$P_{\text{train}}(X_a \mid X_v) \tag{15.18}$$

与：

$$P_{\text{test}}(X_a \mid X_v) \tag{15.19}$$

可能不同。也就是说，即使视觉对象类别没有明显变化，视觉与音频之间的统计关联也可能发生变化。类似地，在自动驾驶中，不同天气可能同时影响 RGB 图像、激光雷达和事件相机的观测。因此，多模态分布偏移需要考虑：单模态输入分布。不同模态之间的统计关系。模态可用性。模态时间和空间对应。任务标签及环境条件。这些因素可能共同影响融合模型的表现。

鲁棒性结果的解释边界 对于多模态系统，鲁棒性测试通常能够回答：模型在某种扰动条件下是否仍然有效。性能下降幅度是多少。相对于单模态或简单融合基线是否具有优势。哪些场景最容易出现错误。但鲁棒性测试不一定能够直接证明某个内部机制是主要原因。例如，融合模型在夜间表现较好，可能来自某种模态的信息互补，也可能来自训练数据和模型容量差异。因此，若希望进一步解释机制，应当设置受控消融实验。例如：保持训练和评价协议一致。分别测试单模态与多模态输入。

逐步增加扰动强度。分析不同场景和录制结果。在必要时控制参数量和计算预算。通过这些对照，可以更加合理地判断模型的鲁棒性来源。多模态鲁棒性不仅是模型面对输入变化时仍能得到正确答案的能力，也是系统能够识别信息不足、合理利用剩余证据，并在不同环境条件下维持可信判断的能力。

15.3 不确定性与置信度校准

15.3.1 鲁棒性测试与压力测试

鲁棒性测试的基本目标 鲁棒性测试（Robustness Testing）主要用于评价模型在输入条件发生变化时，是否仍然能够保持适当的任务性能。例如，一个图像分类模型在清晰图像中具有较高准确率，但在噪声或模糊条件下可能明显退化。类似地，一个多模态融合模型可能在 RGB 和深度信息完整时具有较好表现，却在深度传感器失效后出现严重错误。因此，鲁棒性测试关注的不只是模型在某个异常条件下的绝对性能，还

包括性能随异常条件变化的规律。

设正常测试数据集为：

$$D = (x_i, y_i)_{i=1}^{N} \tag{15.20}$$

其中，x_i 表示输入，y_i 表示参考标签。对输入施加扰动：

$$\widetilde{x}_i = \mathcal{T}(x_i; \delta) \tag{15.21}$$

其中，$\mathcal{T}$ 表示扰动方式，δ 表示扰动强度。对于给定模型 f_θ，可以定义扰动条件下的评价成绩：

$$S(\delta) = \mathcal{E}(f_\theta, \widetilde{D}_\delta) \tag{15.22}$$

其中，$\widetilde{D}_\delta$ 表示扰动后的测试集。如果任务指标越高越好，可以进一步定义性能下降：

$$\Delta S(\delta) = S(0) - S(\delta) \tag{15.23}$$

其中，S(0) 表示正常条件下的性能。这种性能下降并不是唯一的鲁棒性指标。例如，对于 WER、CER 和误差率等越低越好的指标，差异方向需要相应调整。因此，评价前应明确指标含义及其优化方向。

压力测试与普通鲁棒性测试的区别 压力测试（Stress Testing）通常用于分析系统在具有挑战性或接近运行边界的条件下，是否仍能保持合理行为。例如，在普通鲁棒性测试中，研究者可能逐步增加图像噪声。而在压力测试中，研究者可能同时设置：低照度、部分模态缺失、较长输入序列、高频任务请求、有限计算资源。这些条件可能共同作用，使系统接近实际运行能力边界。因此，鲁棒性测试更强调对指定扰动的系统性评价，而压力测试通常更关注极端条件、组合异常和实际运行约束。

二者之间存在较大重叠，并不具有完全统一的学术边界。实际研究中，应明确具体测试条件，而不是仅使用“压力测试通过”这一笼统结论。

多模态组合扰动 多模态系统中的异常可能同时影响多个输入来源。例如，自动驾驶系统可能同时遇到夜间低照度和激光雷达测量稀疏。音视频系统可能同时面对背景噪声和视频帧丢失。因此，只评价单一模态的独立扰动，可能不足以反映实际运行风险。可以构建组合扰动条件：

$$\tilde{X} = (\mathcal{T}_1(X_1; \delta_1), \ldots, \mathcal{T}_M(X_M; \delta_M)) \tag{15.24}$$

其中，每个模态可以具有不同扰动方式和强度。例如：RGB 图像轻度模糊。音频存在中度背景噪声。视频出现间歇性帧丢失。通过比较不同组合，可以分析模型是否具备利用剩余可靠模态维持性能的能力。但组合扰动可能形成大量配置。因此，研究者应根据实际任务风险选择代表性场景，而不是机械枚举全部组合。

分布偏移条件下的鲁棒性 压力测试还可以包含训练和测试分布之间的变化。例如：训练数据来自白天，测试数据来自夜间。训练数据来自某个城市，测试数据来自另一城市。训练语音来自安静环境，测试语音包含强背景噪声。模型在这些条件下可能产生性能下降。WILDS 是面向真实世界分布偏移的代表性评价基准。其研究强调，应通过真实环境中的数据分布变化检验模型泛化能力，而不只是使用人工构造的图像扰动。分布偏移不仅可能影响输入外观，也可能改变类别比例和任务难度。

因此，跨域性能变化需要结合具体场景和数据分布解释。

鲁棒性结论的统计可靠性 多模态压力测试通常包含多个场景和连续序列。例如，一段视频中的相邻帧往往具有较强相关性。如果将所有视频帧视为相互独立的样本，就可能低估性能波动。因此，应根据任务选择适当的统计单位。例如，使用完整录制作为重采样单位，或者分别报告录制级性能。此外，不同扰动强度下通常使用同一批原始样本，因此可以采用配对比较分析性能变化。在报告中，建议明确正常与受扰条件下的指标、样本数量、独立录制数量和必要的置信区间。

这样才能避免仅根据少量典型场景得出过于宽泛的鲁棒性结论。

15.3.2 不确定性估计与置信度校准

置信度是模型报告的分数，正确性由任务与标注决定。对分类任务，可取 $c(x) = \max_k p_\theta(k \mid x)$；校准要求报告置信度与相应样本上的实际正确率一致，概念上写为 $P(\widehat{Y} = Y \mid C = c) = c$。有限样本需要估计这种关系，Softmax 本身不提供该性质。

将样本按置信度分入 B 个非空区间 $\mathcal{B}_b$，经验校准误差为

$$\mathrm{ECE} = \sum_{b=1}^{B} \frac{|\mathcal{B}_b|}{N} \left|\mathrm{acc}(\mathcal{B}_b) - \mathrm{conf}(\mathcal{B}_b)\right|. \tag{15.25}$$

分箱定义会改变结果；较低 ECE 也不能代替准确率，且可能掩盖组内差异。校准应与任务性能、可靠性图和重要子组共同报告。温度缩放等后处理须在独立校准集上拟合，不能使用测试标签选择温度。[34]

数据噪声和模型知识不足是不同的不确定性来源，可以同时存在。预测熵、多次采样与模型分歧提供诊断信号，但低熵可能对应自信错误，多个一致输出也可能共享同一偏差。对某种方法的有效性，应在目标分布及相关扰动上验证；正常场景中的校准不能自动转移到模态缺失或分布偏移。

保形预测构造的是预测集合或区间。令独立校准样本及未来样本满足相应交换性条件，利用非一致性分数选择有限样本分位数，可以得到边际覆盖性质

$$P\{Y \in \Gamma_\alpha(X)\} \geq 1 - \alpha. \tag{15.26}$$

该概率对随机校准集及新测试样本联合取平均，不保证每个输入或每个子组都达到相同条件覆盖。时序相关或分布变化会影响经典条件；对开放式文本还需定义集合、评分与可验证目标，不能直接移植分类结论。[35]

15.4 失败检测、拒答与风险控制

选择性预测用接收规则 $a_\tau(x) = \mathbf{1}\{c(x) \geq \tau\}$。覆盖率与接收样本上的风险分别为

$$C(\tau) = \mathbb{E}[a_\tau(X)], \qquad R(\tau) = \frac{\mathbb{E}[\ell(\widehat{Y}, Y) a_\tau(X)]}{C(\tau)}, \quad C(\tau) > 0. \tag{15.27}$$

提高拒答阈值可能降低已回答样本上的错误，也可能只留下少量容易样本。报告时应同时给出覆盖率和风险，并将回退后的结果计入完整系统，避免用低覆盖下的分数冒充总体性能。

15.4.1 失败检测与风险识别

失败检测的基本目标 失败检测（Failure Detection）是指系统在输出结果之前或之后，根据可用信息判断当前任务是否可能出现错误。例如，一个视觉问答模型可能需要判断自己对某张图像的回答是否可靠。一个多模态融合系统可能需要识别当前深度输入是否异常。一个机器人系统则可能需要判断当前视觉定位结果是否足以支持后续操作。失败检测不同于事后使用真实标签判断模型是否错误。在实际部署中，系统通常无法立即获得当前任务的正确答案。

因此，失败检测需要依赖模型输出、输入质量、历史信息或其他可观测信号。例如，可以根据预测置信度、不同模型分歧或外部验证结果，估计当前预测是否可能失败。这些信号通常只是风险代理指标，而不是对真实错误的直接观测。

错误预测检测与分布外检测 失败检测需要区分不同目标。错误预测检测关注模型是否会在某个具体样本上预测错误。

分布外检测（Out-of-Distribution Detection，OOD Detection）关注输入是否与训练或参考数据分布存在显著差异。

这两类问题相关，但不完全相同。例如，一个图像可能属于训练分布中的常见场景，却因为严重遮挡而导致预测错误。另一个图像可能属于新颖场景，但模型仍然能够正确识别其中的主要对象。因此，分布外不等于预测必然错误。同样，预测错误也不意味着输入一定来自分布外。

Hendrycks 和 Gimpel 在《A Baseline for Detecting Misclassified and Out-of-Distribution Examples in Neural Networks》中研究了利用 Softmax 置信度进行错误预测和分布外检测的方法。这类研究说明，简单置信度可以作为风险检测基线，但其可靠性取决于具体模型和数据条件。

最大 Softmax 概率 对于分类任务，最大 Softmax 概率可以作为简单的置信度信号。例如：

$$c(x) = \max_k P(Y = k \mid x) \tag{15.28}$$

相应地，可以将：$u(x) = 1 - c(x)$ 作为不确定性分数。当最大概率较低时，模型可能更加不确定。但最大 Softmax 概率可能在分布外样本上仍然较高。因此，不能仅根

据 Softmax 置信度较高，就认为输入一定属于熟悉场景。例如，一个模型可能在完全不熟悉的图像上仍然自信地输出某个常见类别。因此，最大概率方法适合作为简单基线，但不应被视为可靠失败检测的充分条件。

基于能量的分布外检测 Liu 等人在《Energy-based Out-of-distribution Detection》中提出利用能量分数进行分布外检测。对于分类模型的 logits：$z_1(x), \ldots, z_K(x)$ 可以定义能量分数：

$$E(x) = -T \log \sum_{k=1}^{K} \exp\left(\frac{z_k(x)}{T}\right) \tag{15.29}$$

其中，T 为温度参数。该研究表明，在其测试的分类任务中，能量分数能够改善部分分布外检测结果。与直接使用最大 Softmax 概率相比，能量分数保留了 logits 整体幅度的部分信息。但能量分数的方向、阈值和分布解释需要根据具体实现确定。此外，基于能量的分布外检测不等于直接估计预测正确概率。某个输入被识别为分布外，并不能直接确定其预测一定错误。因此，分布外检测与任务失败检测应当分别评价。

失败检测指标：AUROC 如果研究者能够为测试样本标注是否失败，就可以评价风险评分对正确和错误样本的区分能力。例如，定义：

$$e_i = \begin{cases} 1, & \text{模型预测错误} \\ 0, & \text{模型预测正确} \end{cases} \tag{15.30}$$

同时为每个样本计算风险评分：r_i 可以使用 ROC 曲线和 AUROC 分析风险评分是否能够区分错误与正确预测。AUROC 越高，通常表示风险评分对两类样本具有更好的排序能力。但 AUROC 不直接反映风险分数的概率校准情况。例如，一个风险分数可能能够正确排序样本，却不能准确表示错误发生概率。此外，AUROC 也不直接告诉研究者在某个实际阈值下会产生多少误报和漏报。因此，必要时还需要评价具体工作点的检测表现。

AUPRC 与类别不平衡 实际系统中的错误样本可能只占较小比例。例如，一个模型在正常环境下的准确率为 98%。则错误比例只有 2%。在这种情况下，错误预测检测属于正负样本不平衡问题。因此，可以使用 Precision-Recall 曲线及其面积指标评价。例如，如果将错误预测定义为正类，那么：

$$\text{Precision} = \frac{TP}{TP + FP} \tag{15.31}$$

$$\frac{TP}{TP + FN} \tag{15.32}$$

其中：

- TP 表示正确识别出的错误预测；

- FP 表示被误报为错误的正确预测；
- FN 表示未被发现的错误预测。

AUPRC 可以描述不同检测阈值下 Precision 和 Recall 之间的权衡。

但 AUPRC 对正类比例较为敏感。因此，不同错误率的数据集之间的 AUPRC 不能脱离基础错误比例直接比较。评价时还应明确正类究竟是“错误预测”还是“正确预测”，否则指标含义可能发生变化。

失败检测的主要局限 失败检测可以帮助系统识别部分不可靠结果，但仍然存在局限。例如，模型可能对某些错误预测具有很高置信度。多个模型也可能一致地犯错。输入质量正常时，系统仍然可能出现逻辑推理错误。此外，真实部署环境中的数据分布可能随时间变化，使原先设定的风险阈值失效。因此，失败检测不应被视为万能的正确性验证器。它更适合作为可靠系统中的一个风险识别环节，与证据校验、异常处理和安全回退机制配合使用。

15.4.2 拒答、回退与结果校验

拒答规则应在独立验证或校准数据上选择，测试集只用于最终评价。前文定义的接收比例与选择风险应成对报告，并检查重要子组：阈值提高后，如果只留下容易样本，较低的条件错误率未必满足原任务需求。

回退可以请求更清晰观测、调用工具、采用另一模型或转人工。新增信息与成本必须计入完整流程；调用另一模型还可能带来相关错误。对用户最终收到的结果，应继续核验正确性、时延与失败处置，不能止于主模型拒答。

分类置信度、生成回答自报置信度和保形预测集合具有不同解释，阈值不宜混用。真实标签或未来信息不可用于在线决定是否接收；零接收时的条件风险未定义。

15.4.3 可靠性评估与动态信息选择

为什么需要动态信息选择 在真实场景中，不同模态的质量往往随时间和环境变化。例如，摄像头在白天可能提供清晰的外观信息，但在强烈眩光、运动模糊或遮挡条件下，图像质量可能下降。与此同时，其他传感器可能仍然能够提供有价值的信息。因此，如果模型始终采用固定的融合权重，就可能无法充分利用不同模态在不同条件下的优势。

动态信息选择（Dynamic Information Selection）试图根据当前输入、任务需求和模型状态，调整不同模态对最终结果的影响。这种机制可以表现为动态融合权重、条件计算、模态门控、输入选择或按需调用不同模型。但动态信息选择的目标不是无条件偏向某个模态，而是根据当前任务和可用证据，合理利用不同信息来源。

可靠性不等于模型置信度 在多模态系统中，需要区分几个不同概念。

输入质量描述数据本身是否清晰、完整、有效，以及是否满足预期的采集条件。

预测置信度描述模型对当前预测的确信程度。

预测可靠性则关注模型给出的判断是否能够在真实数据中达到相应的正确率或风

险水平。

跨模态一致性描述不同模态提供的证据是否相互支持。

这些概念存在联系，但不能简单等同。例如，模型可能对一张严重模糊的图像给出高置信度预测，但这一预测仍然可能错误。不同模态也可能在表面上保持一致，却共同受到同一种数据偏差影响。因此，可靠性评估需要结合输入质量、预测结果、模态间关系和实际任务表现，而不能只使用一个置信度分数。

可靠性引导的融合 一种直观思路是为每个模态估计与当前样本相关的质量或可靠性信息，再根据这些信息调整融合权重。设第 (m) 个模态的质量特征为 q_m，不确定性估计为 u_m，与其他模态的一致性信息为 c_m，任务相关性为 r_m。可以使用可学习的门控函数生成权重：

$$\alpha_m = \mathrm{softmax}_m\big(g(q_m, u_m, c_m, r_m)\big) \tag{15.33}$$

随后，对适合直接融合的特征进行加权：

$$z = \sum_{m=1}^{M} \alpha_m h_m \tag{15.34}$$

其中，(g) 表示权重预测函数。这一形式展示了可靠性信息参与动态融合的一种方式。但真实系统不一定适合将所有信息压缩成单一权重。例如，空间几何信息和语义特征可能具有不同的表示形式，直接加权求和未必合理。此外，当某个模态提供独特但罕见的关键信息时，单纯根据平均质量或一致性降低其权重，反而可能丢失重要证据。因此，可靠性引导的融合需要与具体任务和表示结构相匹配。

通过受控干预检验模态贡献 动态融合机制是否有效，还需要通过受控实验验证。例如，可以在保持其他条件不变的情况下，对某个模态进行以下操作：

- 移除该模态。
- 对该模态施加不同程度的噪声。
- 破坏其与其他模态之间的时间对应关系。
- 对其空间位置或语义内容进行受控错配。
- 比较不同输入质量下的融合权重和最终预测。

通过这些实验，可以分析模型是否对真实的模态信息变化作出了合理响应。

例如，如果某个模态被严重退化后，模型仍然始终给予其较高权重，就需要进一步检查可靠性估计机制是否有效。如果移除某个模态后任务表现几乎不变，则说明该模态在当前实验条件下可能没有提供明显的增量收益。但输入干预也可能改变样本分布，因此应当设计与真实故障相符的测试，并在多个样本和场景上进行配对比较。

典型案例：RGB 与事件相机的可靠融合 考虑一个同时使用 RGB 摄像头和事件相机的连续感知系统。RGB 图像能够提供丰富的外观和语义信息，而事件流能够记录像素

亮度变化形成的异步事件。在普通场景中，两者可以提供互补的空间与时间信息。但当环境发生变化时，不同模态的相对价值也可能变化。例如，在快速运动条件下，RGB 图像可能出现运动模糊，而事件流仍然能够提供较细粒度的运动变化信息。

相反，在静止场景中，如果没有明显的亮度变化，事件流可能缺少足够的新观测，而 RGB 图像仍然能够提供场景外观。因此，系统不应当假设事件信息在所有时刻都比 RGB 信息更加可靠，也不应当始终使用固定权重。一种合理的研究方案，是分别分析正常场景、快速运动、静止场景、图像退化和事件信息不足等条件下的表现。随后，通过单模态基线、固定融合、动态融合和受控模态干预，判断不同信息源在什么条件下能够提供增量价值。

但实验结果应当由真实数据和预先定义的评价协议支持，而不能仅根据传感器的理论特点推断融合必然有效。

从可靠融合到可验证的多模态智能 综合来看，多模态可靠性研究需要从单纯的模型结构设计，逐渐转向对信息来源、信息质量和实际任务结果的系统分析。需要明确不同模态分别提供哪些信息，以及这些信息之间存在怎样的关联。需要通过单模态基线和受控消融，分析各模态的实际贡献。需要在缺失、噪声、错位和分布偏移等条件下评价模型的稳定性。需要检验可靠性估计和动态选择机制是否真正降低了错误风险，而不是仅仅改变了内部权重或置信度分数。

因此，未来多模态可靠性研究的重要目标，不是简单地设计更复杂的融合网络，而是建立能够解释不同信息来源的作用、识别关键输入风险，并通过真实任务验证其效果的系统。

15.5 证据引用与外部校验

15.5.1 基于证据的回答与引用

基于证据的回答 基于证据的回答（Evidence-grounded Answering）强调模型生成的事实性内容，应当能够由当前输入或其他允许使用的可靠信息来源支持。例如，用户提供一张图像，并询问：“图中的汽车是什么颜色?”模型回答：“红色。”这一回答应当能够通过图像中的对应汽车区域进行验证。如果图像中的汽车实际上为蓝色，则模型输出与视觉证据不一致。对于更加复杂的问题，模型可能需要组合多个证据。例如：

“报告中哪一种方法在相同计算预算下取得了最高准确率?”模型需要识别相关实验表格、确认不同方法的计算预算，再比较对应指标。因此，最终回答可能依赖多个表格单元格，而不是单一文本片段。这说明，基于证据的回答不仅要求模型获取相关信息，还要求模型正确建立证据与结论之间的联系。

证据与答案之间的关系 可以将多模态任务表示为：$Y = f_\theta(X, Q)$，其中：

- X 表示图像、视频、音频或其他输入；
- Q 表示用户问题；

- Y 表示生成回答。

在基于证据的回答中，可以进一步引入证据集合：$E = e_1, e_2, \ldots, e_K$ 使模型根据问题和证据生成回答：$Y = f_\theta(Q, E)$，

对于直接基于图像或视频的模型，原始多模态输入也可以作为证据的一部分。证据并不一定是自然语言文本。例如：图像中的目标区域。视频中的时间片段。音频中的声学事件区间。文档中的表格单元格。传感器中的位置或测量记录。这些信息都可以在相应任务中构成证据。因此，多模态证据的表示形式应当与任务目标相匹配。

事实性主张的分解 对于较长的回答，可以将其拆分为多个原子事实（Atomic Facts）。例如，模型回答："方法 A 使用双路编码器，在数据集 B 上取得了 82.5% 的准确率，且推理速度比方法 C 快 20%。"其中至少包含三个可以独立检查的主张：方法 A 使用双路编码器。方法 A 在数据集 B 上取得 82.5% 的准确率。方法 A 的推理速度比方法 C 快 20%。这些主张可能来自不同证据。

例如，第一项来自模型结构图，第二项来自实验表格，第三项来自推理效率测试。因此，不能因为回答中存在一条引用，就默认全部主张都得到支持。Min 等人在《FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation》中提出了原子事实级评价方法。

该研究将长篇生成文本拆解为多个事实性主张，并检查这些主张是否能够被可信知识来源支持。其基本思想可以表示为：

$$\text{SupportRate} = \frac{N_{\text{supported}}}{N_{\text{checkable}}} \tag{15.35}$$

其中：

- $N_{\text{supported}}$ 表示有充分证据支持的可检查主张数量；
- $N_{\text{checkable}}$ 表示可检查的事实性主张总数。

这一表达用于说明证据支持比例的基本思想，并不等同于完整的 FActScore 官方评分实现。

对于多模态模型，同样可以借鉴原子事实分解方法，分别检查对象、属性、数值、时间和空间关系。

音频中的证据定位 音频任务同样可以提供时间证据。例如："第几秒出现了汽车鸣笛声？""说话人什么时候提到会议日期？""背景中的敲门声出现了几次？"对于这些问题，可以通过音频事件检测或语音识别获得对应时间区间。例如：$e_i = (t_i^s, t_i^e, a_i)$，其中，$a_i$ 表示声音事件或语音内容。语音转写文本并不等同于完整的音频证据。例如，用户询问说话人是否在笑，单纯的文字转写可能没有保留相关声学特征。

因此，音频证据需要根据任务选择。对于文字内容，可以使用带时间戳的转写结果。对于声音事件和韵律特征，则可能需要检查原始音频片段。

文档与图表中的证据引用 文档理解任务通常具有明确的信息来源。例如，一份研究报告包含多个表格和图表。用户询问：“实验中哪种模型的推理延迟最低？”系统可以引用对应的表格、页码或单元格。对于较长文档，可以将证据表示为：$e_i = (d_i, p_i, r_i)$，其中：

- d_i 表示文档标识；
- p_i 表示页码；
- r_i 表示页面中的相关区域或文本片段。

对于表格，还可以进一步记录行列位置。

例如：“表 3，第 5 行，第 4 列。”这种结构化证据能够使用户更容易核查结论。但引用位置正确，并不意味着模型已经正确理解表格内容。例如，模型可能引用正确单元格，却错误处理单位。因此，需要将证据定位与最终数值解释分别评价。

引用精确率与支持覆盖率 对于具有多个事实性主张的回答，可以从不同角度评价引用质量。例如，定义引用支持精确率：

$$\text{CitationSupport} = \frac{N_{\text{supported cited claims}}}{N_{\text{cited claims}}} \tag{15.36}$$

其中，分子表示已引用且得到引用证据支持的主张数量。这一指标关注已经添加引用的主张中，有多少真正得到支持。还可以定义证据覆盖率：

$$\frac{N_{\text{supported factual claims}}}{N_{\text{factual claims}}} \tag{15.37}$$

其中，分子表示有充分证据支持的事实性主张数量。这两个指标只是概念性定义。正式评测中，应明确主张分解方式、引用匹配规则和证据支持标准。此外，多条引用可能共同支持同一个结论，因此不能简单要求每个主张必须由单一证据独立证明。对于复杂推理，还可能需要多个证据联合才能支持最终答案。

基于证据回答的主要局限 证据引用能够提高输出的可检查性，但并不能自动消除幻觉。例如：检索得到的资料可能本身错误。视觉定位工具可能选择错误区域。OCR 可能读取错误数值。视频时间片段可能与问题无关。模型可能错误组合多个真实证据。因此，证据增强的目标不应只是让回答附带更多引用。更重要的是建立从事实性主张到可信证据的正确对应关系。对于关键应用，还需要进一步进行外部验证和系统级约束。

15.5.2 多模型交叉检查

多模型交叉检查的机制 多模型交叉检查（Cross-model Verification）是指使用多个模型或多个相对独立的处理流程，对同一任务形成预测或检查结果，并根据它们之间的一致性和差异识别潜在错误。例如，用户提供一张复杂图表。模型 A 负责读取数据。模型 B 独立读取同一图表。如果两者得到不同数值，就说明当前任务可能存在识别风险。类似地，一个视觉语言模型判断视频中存在某个动作。另一个独立模型可以检查

对应时间片段，判断动作是否真正出现。

因此，多模型交叉检查的核心不是简单增加模型数量，而是通过具有不同错误特征的处理方式，获得额外的验证信息。

独立预测与结果比较 假设存在 M 个模型：$f_1, f_2, \ldots, f_M$，对于相同输入 X ，分别产生：$\hat{y}_m = f_m(X)$，其中，$m = 1, \ldots, M$。对于分类任务，可以采用多数投票：

$$\hat{y} = \text{mode}(\hat{y}_1, \ldots, \hat{y}_M) \tag{15.38}$$

也可以根据模型验证性能设定权重。例如，若各模型输出同一标签集合上的可比较概率，可以计算：

$$p(y \mid X) = \sum_{m=1}^{M} w_m p_m(y \mid X) \tag{15.39}$$

其中：

$$w_m \geq 0, \qquad \sum_{m=1}^{M} w_m = 1 \tag{15.40}$$

随后选择最大融合概率对应的类别。但这类概率融合要求不同模型的输出具有可比较含义。例如，如果某个模型的置信度严重过高，而另一个模型经过良好校准，直接平均可能产生偏差。因此，必要时需要先进行校准。

同一模型的多次采样 一种低门槛的交叉检查方法，是对同一个模型进行多次随机采样。例如，针对同一问题生成多个答案。如果多个答案之间存在明显矛盾，可能提示当前任务具有不确定性。SelfCheckGPT 是利用随机采样一致性检测语言模型幻觉的代表性研究。其基本思想是：如果模型对某个事实具有相对稳定的知识，不同采样回答可能较为一致；如果某个内容缺乏可靠知识支持，不同回答可能出现更多冲突。

这类方法适合在无法访问模型内部概率的条件下，构建辅助风险信号。但多次采样一致不代表事实正确。例如，模型可能在每次采样中都根据相同语言先验错误描述某个不存在的对象。因此，采样一致性更适合作为不确定性和异常检测信号，而不是完整的事实验证。

自我纠错的局限 另一种常见策略，是让模型重新审查自己的回答。例如：“请检查刚才的答案是否存在错误。”这种方式实现简单。但模型可能继续沿用原先的错误前提。Huang 等人在《Large Language Models Cannot Self-Correct Reasoning Yet》中研究了缺少外部反馈时的模型自我纠错能力。

该研究发现，在其测试的推理任务和模型设置中，单纯依赖模型自身进行反思，可能无法稳定纠正错误，有时还可能使正确答案变差。因此，自我反思不应被视为天然可靠的校验机制。对于多模态问题，如果模型最初就错误识别了图像中的对象，仅要求其“再认真想一遍”，未必能够恢复真实视觉信息。更有效的验证可能需要重新读取图像、调用独立检测工具或检查外部证据。

交叉检查的评价协议 为了判断多模型交叉检查是否具有实际收益，可以设置公平对照。例如：这一表格为一般实验设计示例。不同任务不一定需要包含全部设置。评价时应报告：最终准确率。幻觉或错误率。正确答案被错误修改的比例。错误答案被成功修正的比例。额外计算成本。延迟和工具调用次数。此外，还可以分别报告正常场景和困难场景结果。这样能够判断新增机制是否真正改善了可靠性，而不只是增加系统复杂度。

多模型交叉检查的适用边界 多模型交叉检查具有一定潜力，但也存在明显限制。多个模型可能共享相同偏差。讨论可能产生过早收敛。额外模型和工具会增加计算与维护成本。此外，交叉检查中的验证器本身也可能发生错误。因此，多模型方法不能替代原始证据。更合理的定位，是将其作为提高错误发现率和证据核查能力的辅助机制。对于重要任务，能够独立验证事实的程序或传感器观测，通常比单纯重复询问多个模型更具有直接证据价值。

15.6 攻击、不可信输入与系统边界

不可信的图像文字、网页片段和工具返回都可能包含指令。它们是待解释的任务数据，权限判断应由系统的控制边界执行。防护评价应区分模型是否复述攻击内容、是否产生违规调用，以及实际执行层是否阻止该调用。

15.6.1 对抗攻击与跨模态攻击

对抗攻击的基本概念 对抗攻击（Adversarial Attack）通常指攻击者通过有目的地构造或修改输入，使模型产生非预期行为。例如，一个图像分类模型原本能够正确识别交通标志。攻击者通过修改图像中的部分信息，使模型将其错误分类为其他标志。与普通随机噪声不同，对抗扰动通常具有明确的攻击目标。例如，攻击者可能希望降低模型准确率，也可能希望模型输出某个指定错误答案。

Goodfellow 等人在《Explaining and Harnessing Adversarial Examples》中系统研究了神经网络对抗样本问题，并提出了具有代表性的扰动分析方法。这一研究说明，即使输入变化较小，模型预测也可能出现明显变化。但对抗攻击与自然分布变化具有不同的形成机制。自然扰动可能来自雨雾、模糊或传感器噪声。对抗扰动则通常针对模型的弱点进行设计。

因此，模型对自然扰动具有较好的鲁棒性，并不意味着它能够抵抗针对性对抗攻击。

对抗攻击的形式化表示 设模型为：$f_\theta(X)$，其中，X 表示原始输入。攻击者构造扰动：$\widetilde{X} = X + \delta$，其中，$\delta$ 表示扰动。在一种典型的分类攻击设置中，可以描述为：

$$\max_{\delta} L(f_\theta(X+\delta), Y) \tag{15.41}$$

同时满足：$|\delta|_p \leq \epsilon$，其中：

- L 表示任务损失；
- Y 表示真实标签；
- ϵ 表示允许的扰动范围；
- $|\cdot|_p$ 表示某种扰动范数。

这一形式主要用于说明有约束的对抗扰动问题。

对于图像以外的模态，扰动约束可能具有不同含义。例如，音频扰动需要考虑声学可感知性和信号处理条件。视频扰动还需要考虑时间连续性。对于物理传感器，也需要关注扰动是否能够在真实采集环境中出现。因此，不能简单使用某一种像素范数完整描述所有多模态攻击。

多模态安全对齐的薄弱环节 部分多模态大模型由已经经过指令微调或安全对齐的语言模型扩展而来。例如，在语言模型基础上增加视觉编码器和跨模态连接模块。但语言模型原有的安全约束，并不一定在所有新增模态条件下保持相同效果。

Liu 等人在《MM-SafetyBench: A Benchmark for Safety Evaluation of Multimodal Large Language Models》中系统研究了视觉内容对多模态安全行为的影响。该研究通过图文组合任务分析多模态模型的安全风险，发现部分模型在视觉辅助条件下更容易产生不符合安全要求的回答。这说明，多模态模型的安全能力不能简单从其语言模型基础部分直接推断。

因此，多模态安全评价应当覆盖模型实际支持的输入形式。

对抗鲁棒性与安全对齐的区别 对抗鲁棒性主要关注模型对经过设计的输入扰动是否稳定。安全对齐（Safety Alignment）则主要关注模型是否遵守预设的安全约束和行为边界。两者具有联系，但并不相同。例如，一个图像分类模型可能对像素扰动具有较好的鲁棒性，但它本身并不执行自然语言安全判断。一个多模态聊天模型可能能够正确拒绝不适当的请求，却仍然对某些图像扰动敏感。因此，不能使用单一对抗准确率评价全部安全能力。

对于多模态大模型，还需要结合有害内容生成、工具越权、敏感信息泄露和其他风险开展专项测试。

对抗攻击与数据投毒 除推理阶段的攻击外，安全风险还可能发生在模型训练和数据准备阶段。例如，攻击者可能影响部分训练样本或外部知识库内容，使模型学习到错误关联。这种情况通常涉及数据投毒（Data Poisoning）。例如，错误的图文配对可能使模型将某个视觉对象关联到不正确的语义描述。如果攻击具有特定触发条件，还可能涉及后门（Backdoor）风险。训练数据噪声和恶意投毒具有不同含义。普通标注错误可能来自人工失误。

投毒则通常具有改变模型行为的攻击目的。因此，研究中需要区分无意的数据质量问题与恶意攻击。

安全攻击评价指标 对于安全攻击，可以定义攻击成功率（Attack Success Rate，ASR）。例如：

$$\mathrm{ASR} = \frac{N_{\text{successful attacks}}}{N_{\text{valid attack attempts}}} \tag{15.42}$$

其中，分母为符合预先定义的攻击条件且有效的测试尝试数。何种结果属于攻击成功，必须由具体任务确定。例如：对于分类攻击，成功可能表示模型发生错误分类。对于安全绕过测试，成功可能表示模型生成了不符合相应安全标准的内容。对于工具型智能体，成功可能要求实际发生未经授权的操作。这些成功条件不等价。此外，还应报告攻击前的正常任务性能。例如，一个系统本身无法完成正常任务，即使攻击成功率较低，也不能据此认定它具有良好的实际安全性。

因此，应联合报告正常任务成功率、攻击成功率、防御后的任务可用性和必要的统计信息。

攻击评测的基本原则 科学的对抗安全测试应明确攻击者控制范围、测试目标和成功判断方式。例如，攻击者只能修改图像，还是能够修改外部检索资料。模型是否具有工具访问权限。是否允许多次尝试。是否使用替代模型。评价器如何判断攻击结果。此外，应当检查防御机制是否影响正常任务。例如，某种防御可能通过拒绝处理所有复杂输入，大幅降低攻击成功率。但这种系统可能同时失去正常任务能力。因此，安全防御应当在降低攻击风险和保持合理可用性之间取得平衡。

15.6.2 提示注入与不可信输入

提示注入的基本概念 提示注入（Prompt Injection）是指攻击者通过模型可接收的输入内容，试图影响模型对任务指令的理解和执行，使其偏离原有授权目标。例如，用户要求系统总结某份报告。系统读取的报告中包含一段文字，试图让模型忽略用户的总结任务，并执行其他操作。如果模型将报告正文中的文字当作新的高优先级指令，就可能发生提示注入。因此，提示注入的核心不只是输入中存在某段特殊文字，而是系统错误处理了不同信息来源之间的指令权限。

用户正常提出新的任务要求，并不一定属于提示注入。是否构成攻击，需要结合输入来源、授权边界和预期行为判断。

直接注入与间接注入 提示注入通常可以区分为直接注入和间接注入。直接提示注入通常发生在攻击者直接控制模型输入提示的情况下。例如，攻击者试图通过用户输入改变模型的行为约束。

间接提示注入（Indirect Prompt Injection）则发生在模型读取外部资料或工具结果时。

例如：网页正文。电子邮件。PDF 文档、搜索结果、数据库记录、视频字幕。图像 OCR 文本。这些内容可能本来只是任务数据，但攻击者在其中嵌入具有指令性质的文字。因此，多模态系统可能通过多个输入通道接触间接提示注入。

Greshake 等人在《Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection》中系统研究了间接提示注入风险。相关研究说明，外部内容如果缺乏适当信任边界，就可能影响模型执行原本不应改变的任务。

不可信数据与指令边界 在多模态系统中，需要区分两类信息。第一类是具有授权意义的任务指令。例如，用户要求："请总结这份报告。"第二类是完成任务过程中需要读取的数据。例如，报告正文。报告正文可能包含各种语言表达，但这些表达首先属于被分析对象。例如，报告可能引用某段包含命令形式的文字。模型应当根据用户任务分析其含义，而不是直接执行该文字。因此，系统需要明确：外部数据可以提供事实。外部数据可以被引用和总结。但外部数据不能自动改变系统权限。

同样，图像中的文字、音频转写结果和视频字幕，通常都应被视为数据来源，而不是默认具有更高优先级的指令。

信任边界与最小权限 系统安全设计中的一个重要原则是最小权限。例如，一个模型只需要读取报告，就不应自动获得删除文件的权限。如果任务需要修改一个指定单元格，也不应默认允许修改整个数据库。因此，模型可以提出操作建议，但实际工具执行应受到独立的权限和参数约束。例如：允许读取指定文件。禁止访问无关目录。限制外部网络请求。对重要写操作要求明确授权。记录操作结果。这些措施能够减少提示注入导致的实际后果。最小权限不能保证模型不会产生错误回答。

它主要用于降低错误指令或恶意输入可能造成的系统影响。

输出验证与工具执行隔离 当语言模型输出将被用于下游程序时，需要检查输出是否符合预期格式和权限要求。例如，模型生成一个文件路径或数据库查询参数。系统应当验证该参数是否属于允许范围。类似地，模型生成的网页内容不应未经处理就自动作为可执行程序使用。因此，系统级防护可以包含：结构化参数校验、操作目标白名单、权限检查、隔离执行环境、重要操作确认、执行后状态核验。这些措施应由模型外部的执行系统实际落实。

不能仅依靠模型在自然语言中承诺"不会执行危险操作"，就认为权限边界已经建立。

2026 年 OWASP 大模型应用安全风险 OWASP GenAI Security Project 发布的《OWASP GenAI LLM Top 10 2026》总结了大模型应用中的主要安全风险。2026 年版本将提示注入、敏感信息泄露、过度自主权限、供应链风险、数据与模型投毒等列为重要问题。其中，提示注入仍然处于重要位置。

同时，过度自主权限（Excessive Agency）强调，系统为模型提供的工具、权限和执行能力如果缺少合理限制，就可能放大错误或攻击造成的影响。例如，纯文本问答系统即使受到误导，可能主要表现为错误回答。但具有文件修改权限的智能体，如果被诱导调用不适当工具，可能直接改变实际数据。因此，模型能力越强、工具权限越多，越需要明确外部输入与授权操作之间的边界。

提示注入的防御评价 提示注入防御可以从两个方面评价。检查攻击是否成功。例如，是否出现未经授权的工具调用或数据访问。检查防御是否影响正常任务。例如，系统是否因为过度防御，而无法处理包含普通命令文字的合法文档。因此，可以同时报告：攻击成功率。未经授权操作发生率。正常任务成功率。正确请求被错误拒绝的比例。额外计算和验证成本。某种方法可以通过拒绝全部外部文档实现较低攻击成功率，但这样的系统可能不具备实际使用价值。

因此，防御评价需要同时考虑安全性与可用性。

15.6.3 安全评测、合规与责任边界

安全评测的基本目标 多模态安全评测主要用于判断系统在潜在攻击、恶意输入和高风险任务中，是否能够保持预期的安全行为。与普通准确率评价相比，安全评测更加关注系统是否突破某种约束。例如：是否执行未经授权的操作。是否泄露不应公开的信息。是否被不可信内容改变任务目标。是否产生具有明显风险的输出。是否能够在必要时拒绝不适当请求。因此，安全评测需要明确具体的安全目标。例如，一个模型拒绝了某种不适当请求，并不意味着它对所有形式的输入攻击都具有防护能力。

同样，工具权限控制完善，也不代表模型在普通视觉识别任务中不会发生错误。

安全基准与能力基准的区别 能力基准主要评价模型能否完成任务。例如：视觉问答准确率。视频事件识别准确率。文档信息提取准确率。工具任务成功率。安全基准则进一步关注系统是否在特定风险条件下违反要求。例如，MM-SafetyBench 通过图文组合任务评价多模态模型的安全行为。HarmBench 则为语言模型的自动红队测试和稳健拒答提供了标准化评价框架。HarmBench 主要面向语言模型安全，其结果不能直接代表全部多模态安全能力。

因此，对于图像、音频和视频系统，还需要设置相应的专项测试。

安全攻击成功率 攻击成功率可以表示为：

$$\mathrm{ASR} = \frac{N_{\mathrm{successful}}}{N_{\mathrm{valid}}} \tag{15.43}$$

其中，攻击成功的定义需要与预设威胁模型一致。例如，对于工具越权攻击，只有实际出现未经授权的操作，才符合相应的成功条件。如果模型只是生成某段文字，但没有执行任何操作，就不应自动视为真实系统已经发生越权行为。类似地，如果攻击目标是诱导有害内容生成，则需要根据明确的安全评价规则判断输出是否违反要求。因此，攻击成功率不是可以脱离任务直接比较的统一指标。不同研究的分母、攻击者权限和成功标准可能不同。

安全评测的统计问题 安全攻击可能具有较低发生率或较强随机性。例如，某种攻击在 100 次测试中成功 5 次。这一结果不能被简单理解为攻击在所有真实条件下都有精确的 5% 成功概率。因此，研究者需要报告样本数量和测试条件。对于随机模型，可以考虑重复运行。对于同一基础样本衍生出的多个攻击，也需要注意样本相关性。此外，

如果研究者反复修改攻击直到成功，只报告最后一次结果，就可能形成选择偏差。因此，安全测试应当明确攻击预算和实验选择过程。

个人信息保护的法律边界 《中华人民共和国个人信息保护法》于 2021 年 11 月 1 日施行。该法对个人信息处理原则、敏感个人信息、个人权利和处理者义务作出了规定。例如，个人信息处理需要具有适当的合法依据，并遵循相关处理规则。在多模态系统中，涉及人脸、声音、位置和医疗信息等数据时，应当根据具体情况判断其法律属性和处理要求。个人信息处理的合法性不能仅根据用户是否主动上传某个文件作出全部判断。

例如，上传的视频中可能包含其他人的个人信息。因此，系统需要结合数据来源、处理目的、授权关系和相关法律要求进行分析。

责任边界的基本含义 多模态系统通常涉及多个参与主体。例如：基础模型开发者、应用系统开发者、数据提供方、部署和运营方、最终使用者。第三方工具服务商。不同主体可能控制系统中的不同环节。例如，基础模型开发者通常负责模型训练与部分能力和安全测试。应用开发者负责将模型接入特定业务流程，并管理工具权限和输入输出。部署方可能负责运行环境、访问控制和持续监测。使用者则需要按照适用规则合理使用系统。因此，发生错误时，不能简单将全部责任归于模型本身。

需要根据具体行为、合同关系、适用法律和各主体实际控制的环节分析。

责任边界与自动化程度 随着多模态智能体获得更多工具和执行能力，系统的自动化程度不断提高。例如，一个模型可以从单纯回答问题，逐渐发展到自动修改文件、调用外部服务或控制物理设备。不同自动化程度下，错误可能造成不同后果。例如：只生成建议。生成可供人工确认的操作计划。在受限环境中自动执行。在真实环境中自主执行。因此，系统需要根据执行权限和潜在影响设置不同的安全要求。例如，普通信息查询可能允许较高自动化程度。

但重要文件删除或涉及真实物理风险的操作，通常需要更加严格的权限和确认机制。具体是否必须人工确认，需要根据适用法律、实际风险和系统设计判断。

习题与讨论

1. 同一图像中有未被标注的对象。说明为什么仅依据标注词表定义幻觉可能误判，并提出人工复核规则。
2. 构造两个准确率相同、置信度不同的分类器，比较校准与任务性能。
3. 证明覆盖率为零时选择性风险的条件定义不成立。为什么不能把全拒答系统记为零风险高性能模型？
4. 设计同时包含输入扰动、模态错配与提示注入的压力测试，并分别规定模型层和执行层的失败判据。

文献导读　CHAIR 与 POPE 采用不同的幻觉评价设置，应先核对问题构造及分母；校准和保形预测的保证应连同统计条件阅读。[33, 25, 34, 35]

第五部分

系统与研究训练

第十六章　推理效率与模型部署

性能优化先要定位关键路径。视觉编码、连接、预填充、逐 Token 解码、排队、传输与工具等待，可能由不同资源限制。参数减少、FLOPs 减少、峰值显存降低和实际延迟缩短是不同测量结果，应分别验证。

> **学习目标**　分解端到端延迟；计算上下文与 KV 缓存需求；区分压缩、低精度与调度的作用；在目标硬件上报告速度与能力的变化。

16.1　端到端成本与关键路径



图 16.1　请求中的成本来源。图只列出阶段，实际系统可以并行或交叠，端到端延迟应按关键路径测量。

16.1.1　推理过程中的主要计算瓶颈

端到端推理延迟的组成　多模态模型的推理时间通常由多个阶段共同构成。例如：输入读取与解码、模态预处理、视觉或音频编码、跨模态特征转换。语言模型 Prefill。自回归 Decode。输出后处理。对于非重叠的简化执行流程，可以表示为：

$$T_{\text{total}} \approx T_{\text{input}} + T_{\text{encode}} + T_{\text{project}} + T_{\text{prefill}} + T_{\text{decode}} + T_{\text{output}} \tag{16.1}$$

其中：

- T_{input} 表示输入处理耗时；
- T_{encode} 表示模态编码耗时；
- T_{project} 表示跨模态特征处理耗时；
- T_{prefill} 表示语言模型预填充耗时；
- T_{decode} 表示输出生成耗时；
- T_{output} 表示输出解析或解码耗时。

实际服务系统还可能包含队列等待、网络传输和调度时间。

例如：

$$T_{\text{total}} = T_{\text{queue}} + T_{\text{compute}} + T_{\text{network}} \tag{16.2}$$

实际系统可能采用流水化或异步并行执行，因此各阶段耗时的直接相加只是一种近似。因此，性能分析需要以实际端到端测量为准。

首 Token 延迟 首 Token 延迟（Time to First Token，TTFT）是语言模型服务中的重要指标。其主要表示从请求开始到首个输出 Token 可用之间的时间。在实际部署中，TTFT 可能包含：排队等待、输入处理、模态编码、跨模态特征准备。语言模型 Prefill。首个输出 Token 选择及传输。因此，多模态系统中的 TTFT 可能显著受到视觉和音频编码影响。

例如，一张高分辨率图像需要较长的视觉编码时间，即使最终答案只有几个字，用户也可能需要等待。不同评测工具可能从不同时间点开始计时。例如，有的统计从服务器接收请求开始，有的则从模型实际开始计算时计时。因此，比较 TTFT 时必须明确计时边界。

每输出 Token 延迟 每输出 Token 延迟（Time Per Output Token，TPOT）用于描述生成阶段的平均速度。一种常见定义为：

$$\mathrm{TPOT} = \frac{T_{\mathrm{last}} - T_{\mathrm{first}}}{N_{\mathrm{output}} - 1} \tag{16.3}$$

其中：

- T_{first} 表示首个输出 Token 的时间；
- T_{last} 表示最后一个输出 Token 的时间；
- N_{output} 表示输出 Token 数量。

该表达适用于输出 Token 数量大于 1 的情况。

TPOT 越低，通常表示后续生成速度越快。但 TPOT 是平均值，可能掩盖生成过程中某些明显停顿。因此，对于实时交互系统，还可以关注相邻 Token 之间的时间间隔及其尾部延迟。例如，某个系统平均生成速度较快，但偶尔出现较长停顿，仍然可能影响用户体验。

注意力计算与显存访存 注意力模块需要处理 Token 之间的关系。对于普通密集注意力，输入长度增加可能带来明显计算成本。此外，GPU 计算性能不仅取决于浮点运算数量，也取决于数据搬运。例如，部分注意力实现需要反复读取和写入较大的中间张量。因此，显存带宽可能成为性能瓶颈。FlashAttention[19] 的研究强调了 IO 感知计算的重要性。这说明，减少不必要的显存读写，有时比单纯减少少量算术运算更加有效。

但具体收益依赖模型结构、序列长度、硬件和算子实现。因此，注意力优化需要通过实际性能测量判断。

模型权重与显存占用 模型权重本身通常占用大量存储。例如，一个具有 P 个参数的模型，如果使用每参数 b 字节的表示，其权重大小近似为：$M_{\mathrm{weight}} = Pb$，例如，一个 7B 参数模型，如果全部使用 FP16 存储，其理论权重大小约为：

$$M_{\text{weights}} = 14 \times 10^9 \text{ bytes} \tag{16.4}$$

约为 14 GB（十进制）。实际模型权重占用可能受到参数共享、量化格式、对齐填充和其他实现因素影响。此外，多模态模型还可能包含视觉编码器、音频编码器和跨模态连接模块。因此，总权重规模应当包含全部需要加载的组件。对于具有多个生成分支的模型，还可能需要额外的声学或视觉解码器。因此，不能仅根据语言模型参数规模判断完整多模态系统的显存需求。

推理性能与任务准确率的关系 推理优化可能改变模型处理的信息量和生成方式。例如：减少视频采样帧数。降低图像分辨率。压缩视觉 Token。限制输出 Token 数量。降低数值精度。这些方法可能减少计算成本，但也可能影响模型准确率。例如，减少视频帧数可能遗漏关键事件。压缩视觉 Token 可能丢失细小文字。降低图像分辨率可能影响对象定位。因此，推理优化不能只报告速度提升。还需要检查相同任务协议下的性能变化。例如，可以同时报告：

$$(\text{Accuracy}, \text{Latency}, \text{Memory}) \tag{16.5}$$

或者根据任务使用：

$$(\text{mIoU}, \text{Throughput}, \text{Memory}) \tag{16.6}$$

等多维指标。这样才能判断某种优化是否具有实际价值。例如，某方法将延迟降低 30%，但准确率下降 10 个百分点，是否值得采用，需要结合实际应用要求判断。

16.1.2 延迟、吞吐量、显存与成本权衡

部署优化是多目标问题 多模态模型部署通常需要同时考虑多个目标。例如：任务准确率、端到端延迟、系统吞吐量、峰值内存、设备功耗、云端计算费用、网络通信成本。隐私和可用性要求。这些目标之间可能存在冲突。例如，较大的模型可能具有更强的任务能力，却需要更多显存和计算资源。较小的模型能够降低成本，但可能在复杂视觉推理中表现不足。云端可能具有更高计算性能，但需要网络通信。端侧可以减少网络依赖，却受到内存和功耗约束。

因此，部署方案通常需要在多个指标之间进行权衡。

端到端延迟 对于云端、端侧和端云协同三种部署方式，可以分别建立简化延迟模型。完全端侧推理：

$$T_{\text{local}} = T_{\text{preprocess}} + T_{\text{encode}} + T_{\text{model}} + T_{\text{output}} \tag{16.7}$$

云端推理：

$$T_{\text{upload}} + T_{\text{network}} + T_{\text{queue}} + T_{\text{cloud infer}} + T_{\text{return}} \tag{16.8}$$

端云协同推理：

$$T_{\text{split}} = T_{\text{edge}} + T_{\text{transfer}} + T_{\text{cloud part}} + T_{\text{other}} \tag{16.9}$$

这些公式主要用于理解不同阶段的组成。实际系统可能采用流水线和并行计算，因此需要根据关键路径测量真实延迟。此外，对于流式输出任务，还需要区分首响应延迟和完整完成时间。

网络带宽的临界条件 设端侧完整推理时间为：T_{local} 云端除上传之外的总耗时为：T_{fixed} 上传数据大小为：D 有效网络带宽为：B 则：

$$T_{\text{cloud}} \approx \frac{D}{B} + T_{\text{fixed}} \tag{16.10}$$

如果希望云端比端侧更快，则需要：

$$\frac{D}{B} + T_{\text{fixed}} < T_{\text{local}} \tag{16.11}$$

当：

$$T_{\text{local}} > T_{\text{fixed}} \tag{16.12}$$

时，可以得到：

$$B > \frac{D}{T_{\text{local}} - T_{\text{fixed}}} \tag{16.13}$$

这一公式给出了简化条件下的带宽临界值。例如，媒体上传量越大，需要的带宽通常越高。但实际系统中的网络吞吐量可能变化，云端排队也可能随负载波动。因此，该公式主要用于初步建模，正式部署需要根据实测结果判断。

能耗与电池续航 对于端侧模型，能耗是重要约束。例如，某个模型每秒能够生成较多 Token，但同时消耗较高功率。如果持续运行，就可能明显影响设备续航。能耗可以表示为：

$$E = \int_0^T P(t), dt \tag{16.14}$$

其中：

- P(t) 表示瞬时功率；
- T 表示运行时间。

如果功率近似保持不变，则：$E \approx PT$，

例如，某个模型平均运行功率为 5 W，持续运行 10 秒，则消耗能量约为：$E = 50$ J，这一数值只是用于说明能耗计算的示例。实际设备中，CPU、GPU、NPU、屏幕、网

络模块和其他组件可能共同消耗能量。因此，能耗测试应明确测量边界。例如，是只测量 GPU 功率，还是测量完整设备功耗。

单位请求成本 对于在线服务，可以使用单位请求成本辅助比较。例如：

$$C_{\text{avg}} = \frac{C_{\text{total}}}{N_{\text{completed}}} \tag{16.15}$$

其中：

- C_{total} 表示统计周期内的总成本；
- $N_{\text{completed}}$ 表示完成的请求数量。

但不同请求的计算成本可能差异较大。

例如，单图问答与长视频分析通常不能视为完全相同的工作量。因此，单位请求成本最好结合任务类型统计。例如：每千次图像问答成本。每小时视频分析成本。每分钟音频处理成本。每百万输出 Token 成本。此外，还需要考虑任务是否成功完成。如果某种配置虽然成本较低，却具有较高失败率，那么单纯的单位请求成本可能具有误导性。因此，可以进一步使用单位有效任务成本进行评价。

单位有效任务成本 可以将满足预先定义质量要求的任务数量记为：N_{valid}，则单位有效任务成本可以表示为：

$$C_{\text{valid}} = \frac{C_{\text{total}}}{N_{\text{valid}}} \tag{16.16}$$

例如，两个系统完成相同数量请求。系统 A 的总成本较低，但错误率较高。系统 B 的总成本略高，但更多任务满足质量要求。此时，如果只比较每次请求的成本，可能低估系统 A 的错误代价。因此，实际部署可以根据任务使用有效任务成本、错误率和服务质量指标进行综合评价。但如何定义有效任务，需要根据实际应用确定。例如，对于视觉问答，可以使用准确率或人工评价标准。对于机器人任务，则可能需要检查实际任务是否成功完成。

因此，不能将所有任务统一为完全相同的质量标准。

具有约束的部署优化 对于实际应用，可以将部署选择表示为约束优化问题。例如：

$$\min_{a \in \mathcal{A}} \tag{16.17}$$

C(a) 满足：$T(a) \leq T_{\max}$，$M(a) \leq M_{\max}$，$Q(a) \geq Q_{\min}$，其中：

- a 表示部署方案；
- C(a) 表示综合成本；
- T(a) 表示延迟；
- M(a) 表示峰值内存；
- Q(a) 表示任务质量指标。

例如，一个实时多模态交互系统可能要求：

首响应延迟低于指定阈值。峰值内存不超过设备限制。主要任务准确率达到预先设定要求。在满足这些约束之后，再选择成本较低的方案。这种方式比单纯追求最快 Token/s 更加符合实际部署需求。

16.2 上下文、视觉预算与缓存

算例：KV 缓存的口径

对批量 b、层数 L、缓存长度 N、键值头数 h_{kv}、每头维度 d_h，若每元素占 s 字节且所有层形式相同，键和值的主体存储为

$$M_{KV} = 2bLNh_{kv}d_hs. \tag{16.18}$$

它不含分配碎片、元数据、临时缓冲和权重。共享键值头时不能以查询头数替代 h_{kv}。比较实现时，还应说明预留容量与实际使用量的区别。

16.2.1 视觉 Token 与上下文长度

视觉 Token 为什么影响推理成本 在典型视觉语言模型中，输入图像首先经过视觉编码器转换为特征表示，再通过投影、压缩或其他连接机制，使视觉信息参与语言模型计算。例如，一张图像可能产生：

$$Z_v \in \mathbb{R}^{N_v \times d_v} \tag{16.19}$$

其中，N_v 表示视觉特征数量。如果这些特征经过映射后直接进入语言模型上下文，那么视觉 Token 数量会影响语言模型的输入序列长度。对于简单的统一序列架构，可以表示为：

$$L_{\text{total}} = L_{\text{text}} + L_{\text{vision}} + L_{\text{other}} \tag{16.20}$$

其中，L_{other} 包括特殊标记和其他模态的输入表示。这一表达主要适用于视觉表示与文本共同构成语言模型上下文的架构。对于独立跨注意力结构，视觉特征可能由专门的条件记忆提供，因此需要分别计算语言自注意力与跨模态注意力的成本。但无论使用哪种结构，视觉信息的表示数量都会影响计算或缓存开销。

上下文长度与注意力计算 对于普通密集自注意力，序列长度增加会导致注意力计算规模上升。假设序列长度为 L ，隐藏维度等其他条件固定，则全序列注意力的主要计算量随序列长度近似按二次关系增长：$C_{\text{attn}} \propto L^2$，因此，如果输入序列长度从 2048 增加到 4096，那么注意力部分的相关计算量可能增大到原来的约四倍。但完整 Transformer 的计算还包含线性投影和前馈网络等操作。

例如，在固定模型结构下，部分线性层的计算量随 Token 数量近似线性增长。因此，输入长度增加一倍，不意味着整个语言模型的 Prefill 时间必然增加四倍。实际变化还受到模型规模、注意力实现、硬件利用率和批次大小影响。此外，在自回归 Decode 阶段，采用 KV Cache 的单步注意力计算通常与当前可访问的历史上下文长度相关，而不是每次重新执行完整的 $L \times L$ 全序列注意力。

因此，视觉 Token 数量对 Prefill 和 Decode 的影响形式并不相同。

分辨率与视觉 Token 数量 对于采用规则 Patch 划分的视觉编码器，可以在简化条件下估算：

$$N_{\text{patch}} = \frac{H}{P}\frac{W}{P} \tag{16.21}$$

其中：

- H,W 表示输入图像尺寸；
- P 表示正方形 Patch 的边长。

例如，一张 448×448 的图像，采用 14×14 的 Patch，可以形成：$N_{\text{patch}} = 1024$ 个 Patch。

如果长宽都扩大到原来的两倍，在 Patch 尺寸不变的情况下，Patch 数量会扩大到原来的四倍。但是，视觉编码器输出的 Patch 数量不一定等于最终输入语言模型的视觉 Token 数量。例如，模型可能使用空间池化、Patch 合并、查询压缩或多尺度特征选择。因此，分析一个具体模型的推理成本时，应当使用其实际产生的视觉 Token 数量，而不是仅根据原始图像尺寸进行推测。

压缩率与真实节省量 假设原始视觉 Token 数量为：N_v 压缩后保留：N_v' 可以定义视觉 Token 压缩率：

$$r_c = 1 - \frac{N_v'}{N_v} \tag{16.22}$$

例如：$N_v = 1024,\quad N_v' = 256$，则：$r_c = 75\%$ 表示视觉 Token 数量减少了 75%。但这并不意味着完整模型计算量减少 75%。例如，如果视觉编码器仍然处理全部 1024 个原始 Token，那么编码器成本可能没有变化。此外，语言模型仍然需要处理文本 Token 和其他输入。因此，应分别测量：视觉编码时间。语言模型 Prefill 时间。KV Cache 大小。

端到端延迟。完整系统峰值显存。只有这样才能判断压缩带来的实际收益。

视频 Token 的空间与时间压缩 视频输入同时具有空间和时间冗余。例如，相邻视频帧可能具有相似场景。同一帧中的大面积背景也可能包含冗余视觉信息。因此，视频 Token 优化可以从两个维度进行。时间压缩主要减少输入帧数，或者合并相似时间片段。空间压缩主要减少每帧的视觉 Token 数量。在简化的均匀帧采样和固定每帧 Token 数条件下，可以表示为：

$$N_{\text{video}} = FN_{\text{frame}} \tag{16.23}$$

其中：

- F 表示采样帧数；
- N_{frame} 表示每帧视觉 Token 数。

例如，原始设置使用 64 帧，每帧 256 个 Token：$N_{\text{video}} = 16384$，

如果减少到 32 帧，每帧 128 个 Token，则：$N'_{\text{video}} = 4096$，视觉 Token 数量降为原来的四分之一。但时间和空间压缩可能造成不同的信息损失。减少帧数可能遗漏短暂动作。降低每帧的视觉表示密度可能影响文字、目标和细粒度空间关系。因此，需要分别评价时间与空间压缩的效果，而不能只报告最终 Token 数量。

视觉 Token 压缩的评价方法 视觉 Token 优化需要同时评价效率和任务性能。例如，可以固定基础模型，比较不同 Token 预算：$N_v \in 128, 256, 512, 1024$，随后评价：视觉问答准确率。OCR 与文档理解、目标定位、空间关系、长视频时间推理、多轮对话。同时记录 Prefill 时间、端到端延迟和峰值显存。不同任务对视觉细节的需求不同。例如，图像场景分类可能对压缩具有较高容忍度。

但精确计数、细小文字识别和短暂视频事件可能更加敏感。因此，不能仅根据某个平均基准成绩基本不变，就认定压缩没有造成信息损失。

16.2.2 KV Cache 与缓存管理

KV Cache 为什么重要 在自回归语言模型中，模型需要根据此前的上下文逐步生成新 Token。为了避免每一步重复计算全部历史 Token 的 Key 和 Value，Transformer 通常使用 KV Cache。例如，对第 ℓ 层：

$$K_\ell^{(t)} = K_\ell^{(t-1)}; k_\ell^{(t)}] \tag{16.24}$$

$$[V_\ell^{(t-1)}; v_\ell^{(t)}] \tag{16.25}$$

其中，新的键和值被追加到历史缓存。这种方式显著减少了重复计算。但随着上下文长度增加，KV Cache 也会持续增长。例如，长文档、多图像和长视频输入可能在 Prefill 之后就产生大量缓存。如果系统还需要生成长篇回答，缓存规模可能进一步增加。因此，KV Cache 是长上下文和高并发推理中的重要显存开销。

KV Cache 显存规模 对于典型具有独立 KV 缓存的 Transformer，可以近似表示：

$$M_{\text{KV}} = 2L_{\text{layer}}N_{\text{KV}}d_hSb \tag{16.26}$$

其中：

- L_{layer} 表示层数；
- N_{KV} 表示每层 KV 头数；
- d_h 表示每个 KV 头维度；
- S 表示缓存序列长度；
- b 表示每个元素占用字节数。

这里的系数 2 对应 Key 和 Value。

例如，考虑：

$$L_{\text{layer}} = 32, \quad N_{\text{KV}} = 8, \quad d_h = 128 \tag{16.27}$$

并采用 FP16 缓存：$b = 2$，则每个缓存 Token 约占用：

$$M_{\text{KV/token}} = 131072 \text{ bytes} \tag{16.28}$$

即：

$$M_{\text{KV/token}} = 128 \text{ KiB} \tag{16.29}$$

如果序列长度为 4096：

$$M_{\text{KV}} = 512 \text{ MiB} \tag{16.30}$$

因此，单个请求就可能占用较大的 KV Cache。若同时运行多个请求，缓存总量还会进一步增加。这一公式是针对特定注意力结构的简化估算。对于 Multi-head Latent Attention、滑动窗口注意力和其他特殊缓存结构，需要按照实际存储内容进行计算。

缓存块共享与写时复制 PagedAttention 还支持不同序列之间共享部分已有缓存。例如，多个候选回答可能来自相同的输入前缀。在这些候选开始分叉之前，它们可以共享已经计算的前缀 KV Cache。当后续生成路径不同时，系统再为需要修改的部分分配独立缓存。这种思想与写时复制（Copy-on-Write）具有联系。例如，束搜索或多候选采样可能生成多个后续序列。如果每个候选都复制完整输入前缀缓存，就会产生较高显存开销。

通过共享已有前缀，可以降低部分重复存储。但缓存共享要求相应前缀的计算条件确实相同。例如，模型权重、位置编码、输入内容和其他相关配置应保持一致。否则，直接复用缓存可能导致错误预测。

多模态前缀缓存的特殊要求 对于多模态模型，前缀可能包含图像或音频内容。例如，用户连续围绕同一张图像提问：“图中有几个人？”“最左侧的人穿什么颜色的衣服？”如果视觉输入和相关模型计算保持相同，理论上可以复用部分视觉编码结果或 Prefill 缓存。但多模态缓存键不能只根据可见文本判断。例如，两个请求可能具有完全相同

的文字：“请描述这张图片。”但实际图像内容不同。显然不能复用不同图像对应的视觉状态。

因此，多模态前缀缓存需要正确识别图像、音频或视频内容及其预处理条件。此外，还应考虑模型版本、位置编码、图像分辨率和其他会影响隐藏状态的配置。对于多用户系统，缓存复用还必须符合数据隔离和隐私要求。因此，缓存命中率不是唯一目标，正确性与访问权限同样重要。

KV Cache 量化 除了改善存储管理，还可以直接降低 KV Cache 的表示精度。例如，将 FP16 缓存转换为更低位宽表示。在其他条件保持不变时，缓存元素字节数减少，可以直接降低理论存储规模。例如，从 FP16 的 16 bit 转换为 FP8 的 8 bit，原始元素存储量理论上可以减少约一半。但实际缓存还可能包含量化缩放参数、元数据和其他开销，因此完整显存节省不一定恰好为 50%。此外，低精度缓存会引入量化误差。

例如，注意力中的 Key 和 Value 误差可能影响后续 Token 的概率分布。因此，需要验证量化对长上下文、复杂推理和多模态任务的影响。

GQA 与 MLA 的缓存差异 KV Cache 的规模不仅取决于缓存精度和长度，也取决于模型架构。例如，Grouped-Query Attention（GQA）通过减少 KV 头数量，降低缓存规模。DeepSeek-V2 则提出 Multi-head Latent Attention（MLA），通过低秩压缩形式减少推理中的键值缓存开销。这类方法与推理时临时压缩缓存不同。它们通常属于模型注意力架构设计，需要与模型训练过程共同考虑。

因此，不能简单地把某个已经训练完成的普通 MHA 模型，直接替换成 MLA 结构而不改变相关权重或计算方式。在部署分析中，应根据具体模型已经采用的注意力架构估算缓存，而不是假设所有模型都使用相同的 KV 头配置。

缓存卸载与分层存储 当 GPU 显存不足时，还可以考虑将部分缓存移动到 CPU 内存或其他存储层级。例如：GPU 显存保存当前活跃请求的缓存。CPU 内存保存暂时不活跃的部分状态。必要时再将缓存加载回 GPU。这种方式可以缓解部分显存容量限制。但不同存储层级具有不同访问延迟和带宽。例如，从 CPU 内存向 GPU 显存传输大量 KV Cache，可能产生明显通信开销。

因此，缓存卸载可能提高可处理的上下文长度或请求数量，但不一定降低推理延迟。如果系统频繁在 GPU 和 CPU 之间交换缓存，还可能出现性能下降。因此，需要结合访问频率和硬件带宽判断是否值得使用。

16.3 低精度与批处理

16.3.1 量化与低精度推理

量化的基本概念 量化（Quantization）是指将模型中的部分数值从较高精度表示转换为较低精度表示，以减少存储或计算成本。例如，模型权重最初可能使用 FP16 或 BF16 表示。在某些部署条件下，可以将部分权重转换为 INT8、INT4 或其他低精度格式。其

主要动机包括：减少模型权重的显存占用。降低权重读取所需带宽。利用硬件支持的低精度矩阵计算。提高单设备可部署的模型规模。但量化不仅改变存储形式，也可能影响数值计算。

因此，量化后的模型不一定与原始高精度模型具有完全相同的输出。

量化的基本数学形式 对于一种典型的均匀量化方式，可以表示为：

$$q = \text{clip}\left(\text{round}\left(\frac{x}{s}\right) + z, q_{\min}, q_{\max}\right) \tag{16.31}$$

其中：

- x 表示原始浮点数；
- q 表示量化后的整数；
- s 表示缩放系数；
- z 表示零点；
- $q_{\min}, q_{\max}$ 表示量化范围。

对应的反量化形式为：$\hat{x} = s(q - z)$ 量化误差为：$e = x - \hat{x}$，

这只是常见的整数仿射量化形式。对于 FP8、FP4 等低精度浮点格式，其表示方式并不等同于上述整数均匀量化公式。因此，不同量化格式需要根据实际数值结构分析。

权重量化的存储收益 假设模型具有：P 个需要量化的参数。原始参数位宽为：b_{old} 目标位宽为：b_{new}，在忽略量化元数据和其他开销的理想条件下，权重存储比例近似为：

$$\rho = \frac{b_{\text{new}}}{b_{\text{old}}} \tag{16.32}$$

例如，将 FP16 权重转换为 INT4：

$$\frac{4}{16} = \frac{1}{4} \tag{16.33}$$

理论上，原始权重数据所需存储量可以降低到四分之一。例如，一个 7B 参数模型，若全部参数采用 FP16 存储，理论权重大小约为：14 GB，如果全部使用理想的 4 bit 表示，则理论权重数据约为：3.5 GB，但实际量化模型还需要保存缩放系数、零点、部分高精度层以及其他元数据。因此，实际存储通常大于上述理想值。此外，这里只讨论模型权重。

KV Cache、运行时激活和视觉编码器等其他组件，可能仍然使用不同精度。

量化校准数据 后训练量化通常需要适当的校准输入。例如，利用一批代表性样本统计激活范围或估计量化误差。对于多模态模型，校准数据具有特殊重要性。例如，模型在纯文本任务中的激活分布，可能与图像问答和长视频输入中的分布存在差异。因此，可以考虑使用不同任务构建校准集合：普通图像问答。高分辨率 OCR。多图推理、视

频理解、音频问答。校准数据不应使用正式测试集的标签或反复根据测试成绩进行选择。否则，可能产生测试集适配和数据泄漏问题。

因此，量化校准同样需要合理的数据划分。

多模态模型的分模块量化 完整多模态模型通常包含多个组件。例如：视觉编码器、音频编码器、跨模态连接模块、语言模型、音频解码器。不同组件具有不同计算特征和量化敏感性。因此，一种实际方案是采用分模块精度策略。例如：语言模型主要线性层使用 INT4 或 INT8。部分敏感连接层保留 BF16。视觉编码器使用 FP16 或其他经过验证的精度。KV Cache 使用独立的低精度格式。这只是可能的部署配置，并不是统一推荐方案。

例如，对于高分辨率文档理解，视觉编码器或连接层的低精度误差可能影响细小文字和数值识别。因此，需要通过分模块实验决定哪些部分可以量化。

量化与计算效率的区别 量化可以减少权重存储，但不一定带来相同比例的计算加速。例如，将 FP16 权重存储为 INT4，可以降低原始权重数据体积。但推理时可能需要执行反量化操作。如果硬件缺乏高效 INT4 矩阵计算路径，部分计算仍然可能使用较高精度完成。此外，量化数据的打包、解包和缩放操作也会产生额外开销。因此，实际速度取决于：硬件低精度计算能力、量化内核实现、批次大小、模型权重读取量、激活数据规模、反量化开销。

因此，不能简单认为 INT4 权重就一定比 FP16 快四倍。量化比例与端到端加速比是两个不同概念。

量化对多模态能力的影响 量化后的模型可能在不同任务中表现出不同敏感性。例如，普通对象分类可能具有一定的误差容忍度。但以下任务可能更加依赖精细数值信息：OCR。图表数值读取、对象计数、复杂空间关系。多步视觉数学推理。长视频时间定位。因此，量化评价不能只使用一个综合问答分数。例如，某个量化模型总体准确率只下降 0.5 个百分点，却可能在特定小字识别任务中下降明显。因此，建议报告任务分组结果。例如：

这样可以判断量化是否在特定能力上产生不成比例的损失。

16.3.2 批处理、并发与推理加速

单请求性能与服务性能 模型推理优化通常需要区分单请求和多请求两种场景。例如，在单用户本地部署中，研究者可能主要关注一次请求的完成时间。在在线服务中，系统可能需要同时处理大量用户请求。因此，除了单次延迟，还需要关注吞吐量与服务容量。例如：

$$\text{Throughput} = \frac{N_{\text{completed}}}{T} \tag{16.34}$$

其中：

- $N_{\text{completed}}$ 表示测量时间内完成的请求数；

- T 表示测量时间。

也可以使用输出 Token/s 统计生成吞吐量。

但不同请求可能具有不同输入长度和输出长度。因此，多模态服务的吞吐量必须结合请求分布解释。例如，每秒完成 10 个短文本请求，与每秒完成 10 个长视频请求，通常具有完全不同的计算负载。

静态批处理 静态批处理（Static Batching）是指将多个请求组成固定批次，再共同执行模型计算。例如：$(X_1, X_2, \ldots, X_B)$，其中，B 为批次大小。批处理能够提高 GPU 矩阵计算的并行效率。但不同请求可能具有不同序列长度。例如：请求 A：256 个输入 Token。请求 B：1024 个输入 Token。请求 C：4096 个输入 Token。

如果简单将所有请求填充到最大长度，可能产生大量无效计算或内存占用。因此，实际服务通常需要更加灵活的批次组织方式。

服务吞吐量与有效吞吐量 高吞吐量不一定意味着服务满足实际需求。例如，系统每秒可以完成大量请求，但部分用户等待时间过长。因此，可以引入有效吞吐量（Goodput）概念。其基本思想是，只统计满足预先定义服务质量要求的有效工作量。例如：

$$\text{Goodput} = \frac{N_{\text{completed within SLO}}}{T} \tag{16.35}$$

其中：

- $N_{\text{completed within SLO}}$ 表示满足服务等级目标的完成请求数；
- T 表示统计时间。

不同研究可能使用不同 Goodput 定义。

例如，有的要求同时满足 TTFT 和 TPOT 阈值。有的还会包含完整请求延迟或其他质量约束。因此，正式比较时需要明确 SLO 定义。DistServe 重点研究了同时满足不同阶段延迟要求的服务能力。这说明，实际推理服务不应只追求没有延迟约束的最大吞吐量。

优化方法的系统性比较 这一比较说明，不同方法具有不同作用对象。例如，如果主要瓶颈是视觉编码器，优先优化 KV Cache 可能收益有限。如果主要瓶颈是 Decode 访存，单纯减少图像输入尺寸可能无法充分解决问题。因此，需要先识别实际瓶颈，再选择对应方法。

端到端优化中的阿姆达尔定律 阿姆达尔定律（Amdahl's Law）说明，系统整体加速受到未优化部分的限制。假设某个阶段占原始总时间的比例为：p 该阶段加速倍数为：s 在其他部分不变、且阶段耗时可加的简化条件下，整体加速比为：

$$\text{Speedup} = \frac{1}{(1-p) + \frac{p}{s}} \tag{16.36}$$

例如，某个多模态系统中，Decode 占总时间的 40%。假设通过某种方法将 Decode

加速两倍。则：

$$\frac{1}{0.6 + 0.4/2} = 1.25 \tag{16.37}$$

即整体速度仅提高到原来的 1.25 倍。这一例子说明，局部模块加速并不等于端到端加速。例如，某种推测解码方法使 Decode 速度提升两倍，但如果主要时间消耗来自视频解码和视觉编码，那么完整系统的加速可能有限。因此，优化研究应当报告真实端到端结果，而不是仅根据单模块加速倍数作出结论。

建议的优化顺序 在实际部署工作中，可以采用逐步优化的研究流程。建立一个具有明确输入和输出协议的高精度基线。例如，使用原始模型配置和经过验证的推理框架，测量完整的任务表现、端到端延迟和峰值显存。分析主要瓶颈。例如，确定瓶颈来自视觉编码、Prefill、Decode、权重加载还是 KV Cache。随后，根据瓶颈选择最小必要改动。例如：视觉 Token 过多时，研究适当的压缩策略。

KV Cache 占用较高时，优先检查缓存管理和缓存精度。权重占用过大时，考虑合理的量化方案。并发吞吐量不足时，检查批处理和调度策略。完成每项重要优化后，应在相同任务协议下检查性能变化。再考虑组合优化。这种方式有助于避免同时修改大量组件，导致难以判断收益来源。

优化结果的解释边界 推理优化研究需要避免将特定测试条件下的收益推广为普遍结论。例如，某种方法在一张 GPU 上加速明显，不意味着在所有硬件上都具有相同收益。某种视觉 Token 压缩方法在普通 VQA 上保持准确率，不意味着在 OCR、复杂空间关系或长视频任务中没有损失。某种量化方法在短上下文下表现良好，也不意味着它在长上下文或高并发条件下完全没有影响。此外，模型参数量、输入长度和硬件峰值算力，并不能单独决定实际推理性能。

因此，科研报告中应明确优化效果的适用范围。例如，可以说明：“在指定模型、硬件、输入长度和并发设置下，该优化降低了端到端延迟，同时保持所测试任务的性能。”这样的结论比笼统地宣称“实现了全面推理加速”更加严谨。多模态推理效率优化的关键，是减少不必要的信息处理和数据搬运，合理利用低精度计算与缓存资源，并通过服务调度提高硬件利用率，同时确保优化后的模型仍然满足目标任务的准确性与可靠性要求。

16.4 服务接口、调度与监控

16.4.1 API 接口与请求处理

API 接口的作用 API（Application Programming Interface）用于规定外部程序如何调用模型服务，以及服务如何返回结果。例如，一个网页应用可以通过 HTTP 请求向多模态模型发送图片和问题。模型服务完成推理之后，再返回文本答案。API 的作用不仅是传输数据，还需要明确输入格式、参数含义和错误处理方式。例如，系统需要规

定：请求包含哪一个模型名称。用户输入采用什么结构。如何表示图像、音频或视频。

最大允许输入长度是多少。是否使用流式输出。如何限制生成长度。服务发生错误时如何返回状态。因此，API 协议是模型服务化的重要组成部分。

模型服务的接口类型 不同模型可能提供不同接口。例如：文本补全接口、多轮对话接口、图像问答接口、语音转写接口、音频生成接口、向量嵌入接口。对于采用统一语言模型输出的多模态系统，图像和文本可以通过同一个对话接口提交。而对于纯语音转写模型，可能使用独立的音频转写接口。不同推理框架支持的接口集合并不完全相同。即使两个框架都声称兼容某种 API，具体参数、模型类型和多模态输入支持范围也可能不同。因此，服务接口的兼容性应以实际功能测试为准。

请求参数与计算预算 模型服务通常允许客户端设置部分生成参数。例如：最大输出 Token 数。采样温度。Top-p。停止条件。是否流式生成。但这些参数可能影响计算成本。例如，用户要求模型生成非常长的回答，就可能持续占用 Decode 资源和 KV Cache。类似地，用户上传大量图像或长视频，可能造成较高的视觉编码和 Prefill 开销。因此，服务不能完全依赖用户自行控制资源预算。例如，可以设置：最大输入 Token 数。

最大视觉 Token 数。单次最大图像数量、最大视频帧数、最大音频时长。最大输出 Token 数。这些限制应当结合模型能力和硬件资源确定。不同模态的资源成本不能完全由同一个 Token 数指标概括。例如，视频解码和视觉编码本身也会消耗计算资源。因此，多模态请求需要更加全面的资源预算。

请求处理的基本流程 一个多模态 API 请求可以经历以下过程。网关接收请求并进行身份认证。随后，系统检查请求结构和资源限制。接着，读取媒体并执行必要预处理。然后，根据模型类型将请求交给对应推理实例。推理引擎完成模态编码和模型生成。输出处理模块将结果转换为协议规定的形式，并返回客户端。可以将请求延迟表示为：

$$T_{\text{total}} = T_{\text{gateway}} + T_{\text{validation}} + T_{\text{media}} + T_{\text{queue}} + T_{\text{inference}} + T_{\text{return}} \tag{16.38}$$

这一表达适用于各阶段主要串行执行的简化情况。实际系统可以通过异步下载、流水线处理或并行编码使部分阶段重叠。因此，真实端到端延迟应以客户端和服务器测量结果为准。

请求标识与可追踪性 在线服务通常为每个请求分配唯一标识。例如：request$_{\text{i}}$d，该标识可以用于关联不同处理阶段。例如：API 网关日志、媒体解析记录、推理引擎状态、工具调用记录、最终响应状态。这样，当某个请求发生异常时，研究者可以定位错误发生在哪个组件。但请求标识不应直接使用包含敏感信息的原始用户内容。此外，跨服务追踪通常还需要维护 Trace ID 等信息，以便观察请求在不同服务之间的传播路径。

认证、授权与租户隔离 多用户服务需要区分不同调用者的权限。例如，某个用户可能只能访问指定模型。另一个用户可能被允许上传较长视频。部分用户可能具有专用 GPU 资源或更高的服务配额。因此，系统需要进行身份认证和授权。例如，可以使用

API Key、身份令牌或其他适当机制识别客户端。但身份认证只证明调用者的身份或凭证状态，并不自动限制其能够访问的全部资源。因此，还需要建立模型访问控制、文件权限和资源配额。

对于多租户服务，尤其需要避免不同用户的媒体缓存、会话状态或外部文件被错误共享。这说明，多模态 API 安全不仅涉及模型生成内容，也涉及数据访问和缓存隔离。

16.4.2 动态批处理与并发控制

多请求调度为什么重要 在模型服务中，多个请求可能同时到达。例如，一个 GPU 服务器需要处理：用户 A 的图像描述。用户 B 的文档问答。用户 C 的长视频总结。用户 D 的语音理解。这些请求具有不同输入长度、模态编码开销和预计输出时间。因此，服务器需要决定哪些请求可以同时执行，哪些需要等待，以及如何控制显存占用。如果同时接收过多请求，可能发生显存不足。如果请求数量过少，又可能使 GPU 利用率较低。

因此，并发控制需要在系统吞吐量、响应延迟和运行稳定性之间进行权衡。

动态批处理的机制 动态批处理（Dynamic Batching）是指服务器根据当前请求队列和运行条件，动态组合多个推理请求。例如，短时间内有多个图像分类请求到达。系统可以将它们合并成一个批次，再共同执行模型前向计算。这样有助于提高 GPU 的矩阵计算效率。NVIDIA Triton Inference Server 提供了面向无状态模型的动态批处理机制，可配置偏好批次大小、排队等待时间和相关队列策略。

例如，一个模型可以在有限等待时间内收集多个请求，再形成批次。但普通无状态动态批处理与自回归语言模型连续批处理并不完全相同。前者通常将一组请求作为一次模型执行任务。后者则需要支持不同请求在多轮生成之间动态进入和退出。

无状态请求与有状态序列 在服务调度中，需要区分无状态模型调用与有状态序列处理。例如，一个普通图像分类任务通常可以被视为独立请求。但一个持续进行的视频对话可能需要保留历史上下文。类似地，一个自回归生成请求需要在多轮 Decode 中保留 KV Cache。因此，调度器需要维护请求状态。Triton 提供了序列批处理（Sequence Batching）及相应的迭代序列处理机制，用于支持具有状态关联的模型。

这说明，不同请求是否能够组合执行，不仅取决于其张量形状，也取决于模型是否具有持续状态。例如，如果某个视频处理模型要求相邻时间片段共享递归状态，就需要确保对应状态不会在不同用户或不同视频之间混淆。因此，多模态服务需要明确会话与模型状态之间的关系。

请求队列与背压 当请求到达速度高于系统处理能力时，请求会积累。例如：$\lambda > \mu$，其中：

- λ 表示请求平均到达率；
- μ 表示系统平均服务能力。

在持续超载且没有有效限流或丢弃机制的情况下，等待队列可能不断增长。

这会导致响应延迟增加。因此，在线服务通常需要背压（Backpressure）机制。例如：限制最大等待队列长度。限制单用户同时请求数量。限制总并发请求数。超载时返回明确的资源不足状态。这样可以避免系统无限制接收请求，最终造成整体不可用。背压的目的不是提高模型本身的准确率，而是保持服务在有限资源下的稳定性。

Little 定律与并发关系 在满足稳定运行及相应统计条件的排队系统中，Little 定律可以表示为：$L = \lambda W$，其中：

- L 表示系统内平均请求数量；
- λ 表示长期平均完成或到达率；
- W 表示请求在系统中的平均停留时间。

例如，如果一个稳定系统每秒完成 10 个请求，每个请求在系统中平均停留 2 秒，则平均在系统中的请求数量约为：$L = 10 \times 2 = 20$，

这一关系并不意味着系统只要支持 20 个并发就一定能够达到对应吞吐量。实际吞吐量还取决于模型计算能力、请求长度和缓存资源。Little 定律主要帮助理解到达率、并发数量和等待时间之间的关系。对于多模态服务，请求成本差异较大，因此还需要结合具体工作负载进行分析。

模型实例与负载均衡 当单个模型实例无法满足请求负载时，可以部署多个实例。例如：

$$\mathcal{M}_1, \mathcal{M}_2, \ldots, \mathcal{M}_K \tag{16.39}$$

每个实例运行相同或兼容的模型版本。请求路由器根据当前负载和资源情况选择目标实例。例如，可以考虑：当前等待请求数。GPU 显存使用情况。预计请求计算成本。实例是否已经缓存相应前缀。模型版本是否匹配。Ray Serve 等框架支持多实例请求路由和弹性扩展。简单轮询不一定适用于所有多模态服务。例如，一个实例可能正在处理长视频，另一个实例只处理短文本。即使两者当前请求数量相同，其实际负载也可能存在明显差异。

因此，需要考虑资源感知的路由方式。

服务质量与 Goodput 对于在线服务，最大吞吐量不一定等于有实际价值的吞吐量。例如，一个系统每秒可以完成 100 个请求，但其中大量请求超过用户可接受的延迟。这种情况下，虽然吞吐量较高，服务体验可能并不理想。因此，可以使用 Goodput 衡量满足预先定义服务质量要求的请求完成能力。例如：

$$\text{Goodput} = \frac{N_{\text{SLO satisfied}}}{T} \tag{16.40}$$

其中：

- $N_{\text{SLO satisfied}}$ 表示满足服务等级目标的请求数量；
- T 表示测量时间。

不同研究可能对 SLO 设置不同要求。

例如，有的同时约束 TTFT 和 TPOT。有的还包括完整请求延迟。因此，报告 Goodput 时必须说明实际采用的条件。

16.4.3 日志、监控与异常处理

为什么服务需要可观测性 模型在单机测试中能够正常运行，并不意味着其在长期在线服务中不会发生异常。例如：某个请求导致显存峰值过高。某种视频格式无法正确解码。模型实例发生意外退出。部分请求出现异常长的排队时间。网络连接中断导致生成结果无法返回。因此，服务需要持续记录必要的运行信息，并能够根据这些信息判断系统是否正常。可观测性（Observability）通常涉及日志、指标和分布式追踪等方面。

这些信息可以帮助研究者理解系统状态，并在发生故障时定位问题。监控并不等于简单地记录尽可能多的数据。对于多模态模型，输入可能包含个人图像、语音和文档，因此需要在可观测性与隐私保护之间取得平衡。

日志的基本作用 日志（Logging）用于记录系统在运行过程中发生的事件。例如：模型加载开始、模型加载成功、收到推理请求、请求校验失败、媒体解析异常、推理完成、请求超时、用户取消请求。可以使用不同日志级别区分事件。例如：INFO 用于记录正常运行状态。WARNING 用于记录可能需要关注但尚未导致服务失败的问题。ERROR 用于记录处理失败。DEBUG 用于更详细的调试信息。日志级别的具体使用方式取决于系统规范。

对于正式部署，通常不应长期启用包含大量敏感细节的调试日志。

分布式追踪 在多组件系统中，一个请求可能经过多个服务。例如：API 网关、对象存储、媒体解码服务、视觉编码服务、语言模型服务、输出处理服务。如果只记录完整请求耗时，就难以确定延迟来源。因此，可以使用分布式追踪（Distributed Tracing）记录请求在不同组件之间的执行过程。例如，一个请求对应一个 Trace。Trace 中可以包含多个 Span。例如：媒体下载 Span。图像预处理 Span。视觉编码 Span。

语言模型 Prefill Span。Decode Span。输出返回 Span。OpenTelemetry 提供了用于日志、指标和追踪的开放标准及相关语义约定，并持续完善生成式 AI 系统的观测规范。相关生成式 AI 语义约定仍可能随着版本发展而变化。因此，实际系统应结合使用的 OpenTelemetry 版本和框架能力确定字段。

端到端延迟分析 为了定位性能问题，可以将请求总延迟拆分为不同阶段。例如：

$$T_{\text{total}} = T_{\text{queue}} + T_{\text{media}} + T_{\text{encode}} + T_{\text{prefill}} + T_{\text{decode}} + T_{\text{return}} \tag{16.41}$$

这一表达适用于简化的串行处理流程。如果实际系统存在并行操作，则需要结合 Trace 中的时间区间分析关键路径。例如，图像下载和部分文本预处理可能同时进行。此时，直接相加各模块耗时可能高于真正的端到端时间。因此，分布式追踪不仅用于记录模块耗时，还可以帮助发现实际关键路径。例如：媒体下载是否过慢。视觉编码是

否存在排队。语言模型 Prefill 是否受到长视频请求影响。Decode 是否因为 KV Cache 压力而减速。

TTFT 与 TPOT 的监控边界 对于自回归生成服务，TTFT 和 TPOT 是常见的性能指标。但不同系统对计时范围的定义可能存在差异。例如，有的 TTFT 从客户端请求发出开始计算。有的则从推理引擎接收请求开始计算。对于多模态服务，前者可能包含媒体上传与解码时间，后者可能不包含。因此，监控时应明确客户端端到端指标与服务器内部指标的区别。类似地，流式响应中的 Chunk 不一定严格对应单个生成 Token。

因此，如果某个框架报告的是 Chunk 间隔，就不能直接将其视为严格的逐 Token 延迟。对于重要性能比较，建议同时记录原始输出 Token 数量和相应计时边界。

16.5 端侧部署与任务拆分

端云拆分必须计算通信与等待。把视觉编码移到端侧可能减少原始媒体传输，也可能增加特征传输、版本耦合或端侧负载。只有在明确任务、网络和隐私约束后，才能比较系统成本。

16.5.1 端侧模型与资源约束

端侧推理的基本概念 端侧推理（On-device Inference）是指在直接使用或产生数据的设备上完成模型计算。例如，智能手机可以在本地完成图像问答。可穿戴设备可以在本地处理部分声音事件。工业摄像设备可以在本地完成目标检测和异常识别。机器人也可以通过本地计算模块完成部分多模态感知和控制。端侧推理的主要特点，是不必为每一次模型计算都与远程云端通信。因此，它能够在部分情况下减少网络依赖，并支持离线运行。

但端侧推理的可行性取决于模型规模和设备资源。例如，同样具有数十亿参数的模型，在高性能笔记本电脑上可能运行良好，却未必适合低功耗嵌入式设备。因此，端侧模型需要根据目标设备的实际能力设计。

端侧计算资源 端侧设备通常具有多种计算单元。例如：CPU。GPU。NPU（Neural Processing Unit）。DSP（Digital Signal Processor）。专用图像处理模块。这些硬件具有不同的计算特点。例如，CPU 具有较好的通用性。GPU 适合高度并行的计算。NPU 通常针对特定神经网络算子和数值格式进行优化。DSP 则可能适合部分音频和信号处理任务。

因此，端侧部署需要根据模型算子和硬件支持情况选择执行后端。某个设备标称具有较高 NPU 算力，不意味着任意多模态大模型都能够充分利用该算力。例如，模型可能包含 NPU 不支持的算子，导致部分计算回退到 CPU。这种跨设备执行和数据搬运可能影响整体性能。

端侧内存与显存限制 端侧设备通常具有较严格的内存预算。例如，智能手机的内存需要同时供操作系统、应用程序和其他服务使用。因此，即使设备总内存较大，也不意味着可以全部分配给模型。对于多模态模型，内存占用通常包括：语言模型权重。视觉或音频编码器权重。运行时激活。KV Cache。输入媒体缓冲。输出解码缓冲。例如，一个 3B 参数模型，如果采用 FP16 存储，其理论权重数据约为：6 GB，

如果采用理想的 INT4 权重表示，则原始权重数据约为：1.5 GB，但实际运行仍然需要额外内存。因此，不能仅根据量化后的权重文件大小判断模型是否能够端侧运行。还需要检查完整推理过程中的峰值内存占用。

端侧推理框架 为了在不同设备上运行模型，需要使用适当的轻量化推理框架。例如，ExecuTorch 是 PyTorch 生态中面向移动设备、嵌入式设备和其他边缘平台的推理运行时。它支持根据目标硬件和模型结构，选择相应的执行后端。Google AI Edge 的 LiteRT 与 LiteRT-LM 则提供面向端侧机器学习和生成式模型的执行能力。

此外，llama.cpp 提供了面向多种硬件环境的轻量化语言模型推理实现，其多模态支持模块也可用于符合要求的视觉和其他输入模型。这些框架能够降低端侧模型部署的工程门槛。但框架支持某种计算设备，不意味着所有模型结构都可以无修改运行。例如，某些自定义视觉算子或动态输入流程，可能需要额外转换或适配。因此，应当在目标设备上验证模型正确性、算子支持和实际性能。

本地存储与模型加载 端侧设备通常使用闪存等非易失性存储保存模型文件。运行时，模型权重和必要的状态需要进入内存或被相应执行单元访问。如果模型权重较大，模型加载时间可能明显影响启动体验。例如，一个应用在首次启动时需要读取数 GB 权重。即使模型单次前向计算较快，加载过程仍然可能需要较长时间。Alizadeh 等人在《LLM in a flash》中研究了内存受限设备中的模型权重加载问题。

该研究针对无法完整放入 DRAM 的语言模型，提出通过选择性加载和更加高效的闪存访问组织，降低反复读取模型权重的成本。这一研究说明，端侧推理不仅涉及计算算子优化，还涉及存储层次和数据搬运。但闪存中的模型权重不等于已经驻留在运行内存中。如果需要频繁从闪存读取权重，实际推理速度可能受到存储带宽限制。因此，模型能够保存到设备中，并不意味着一定能够高效运行。

端侧隐私与离线能力 端侧推理的重要潜在优势，是可以在本地处理数据。例如，某个语音助手可以在设备上完成部分语音识别。用户的原始录音不必为该任务持续上传云端。类似地，某些相册搜索功能可以在本地建立图像特征索引。但端侧运行并不自动意味着绝对隐私安全。例如，设备本身可能存在不安全的存储权限。应用程序也可能将结果或运行日志上传到外部服务。此外，设备丢失或恶意软件也可能形成信息泄露风险。

因此，端侧部署能够减少某些远程传输需求，但仍然需要结合权限管理、数据加密和应用安全设计。

端侧部署的典型适用场景 端侧推理通常适合对网络依赖、隐私和响应时间具有较严格要求的任务。例如：本地语音识别、设备内图像搜索、基础视觉问答、智能家居控制、

部分机器人感知、离线文档摘要。但对于复杂任务，端侧小模型可能无法达到较大云端模型的能力。因此，一些系统会采用本地模型处理常见任务，将复杂请求交给远程模型。这就引出了端云协同推理问题。

16.5.2 模型压缩与任务拆分

模型压缩与任务拆分的区别 面对端侧资源限制，可以采用两类不同思路。第一类是模型压缩，即减少模型所需的参数、计算或缓存资源，使其能够在目标设备上运行。第二类是任务拆分，即将完整任务划分为多个部分，由不同模型或不同计算节点共同完成。例如，一套视觉问答系统可以通过量化，将完整模型部署到手机上。也可以让手机只运行视觉编码器，再将部分特征传输到云端执行复杂语言推理。但这两种方案具有不同的资源和系统代价。前者主要依靠模型本身的小型化。

后者则需要解决计算分配与通信开销问题。因此，模型压缩和任务拆分可以结合，但不能视为同一概念。

量化与剪枝 模型量化通过降低部分权重、激活或缓存的表示精度，减少存储与访存成本。例如，将 FP16 权重转换为 INT4，可以显著降低原始权重数据体积。模型剪枝则尝试移除部分参数、通道、注意力头或其他计算结构。例如，结构化剪枝可以减少某些网络层中的通道数量。但参数减少并不总能带来相同比例的实际加速。例如，非结构化稀疏权重可能需要专门的稀疏计算内核，才能有效减少计算时间。同样，低比特权重如果缺少硬件支持，也可能需要较多反量化操作。

因此，端侧压缩方法应当结合目标硬件评价，而不是仅比较模型文件大小。

轻量化模型结构 除了压缩已有大模型，还可以从架构设计阶段构建轻量化模型。例如，MobileNetV2 利用倒残差和线性瓶颈等设计，降低视觉网络的计算开销。这种思路后来成为移动视觉模型设计的重要基础之一。对于视觉语言模型，可以进一步优化：视觉编码器、跨模态连接模块、语言模型规模。视觉 Token 数量。

例如，MobileVLM 和 TinyLLaVA 等研究说明，模型结构、训练数据和训练策略的联合设计，可以使较小的多模态模型获得具有竞争力的任务表现。但传统移动视觉网络与多模态大模型具有不同的任务结构。例如，轻量化对象检测器可以直接输出类别和边界框。视觉语言模型则需要进一步进行语言理解和自回归生成。因此，端侧多模态部署需要同时控制视觉编码和语言生成成本。

模型划分点与通信开销 假设模型在第 k 层之后划分。端侧执行前 k 层。云端执行剩余层。可以将端云协同总延迟近似表示为：

$$T(k) = T_{\text{edge}}(k) + T_{\text{transfer}}(k) + T_{\text{cloud}}(k) + T_{\text{other}} \tag{16.42}$$

其中：

- $T_{\text{edge}}(k)$ 表示端侧计算时间；
- $T_{\text{transfer}}(k)$ 表示中间特征传输时间；

- $T_{\text{cloud}}(k)$ 表示云端剩余计算时间；
- T_{other} 表示必要的网络往返、调度和其他开销。

如果第 k 层的中间特征包含：N_k 个元素，每个元素使用：b_k

bit 表示，则其原始传输数据量近似为：

$$D_k = \frac{N_k b_k}{8} \tag{16.43}$$

字节。实际传输还可能包含额外元数据、协议开销和压缩信息。因此，选择划分点时，不仅需要考虑前后两部分的计算成本，还需要考虑中间特征的数据量。例如，某个网络中间层具有较大的特征图。即使把前几层放到端侧运行，也可能产生较高的通信开销。

任务级协作与模型路由 相比于拆分同一个模型，另一种思路是使用端侧小模型和云端大模型协作。例如，端侧小模型首先处理用户请求。如果任务比较简单，可以直接在本地完成。如果任务复杂，则将请求发送到云端大模型。例如：识别常见对象可以优先使用端侧模型。复杂多页文档分析可以调用云端模型。需要大量知识或长上下文推理的任务，也可能更适合使用云端。这种方式通常不需要在同一个 Token 的生成过程中反复进行端云通信。因此，工程实现可能比层级拆分更加简单。

但系统需要判断哪些任务能够由端侧模型可靠完成。如果路由器错误地将复杂任务交给能力不足的小模型，就可能造成性能下降。如果大量简单任务都被发送到云端，也可能失去端侧部署的成本优势。因此，任务路由需要进行独立评价。

基于风险的云端回退 任务级协作还可以结合不确定性估计和风险检测。例如，端侧模型完成初步预测，并给出经过验证的风险评分。如果风险较低，系统可以直接采用本地结果。如果风险较高，则调用云端模型进行进一步分析。可以概念性地表示为：

$$Y = \begin{cases} f_{\text{edge}}(X), & r(X) < \tau \\ f_{\text{cloud}}(X), & r(X) \geq \tau \end{cases} \tag{16.44}$$

其中：

- r(X) 表示风险分数；
- τ 表示路由阈值。

这种机制可以减少部分不必要的云端调用。

但风险评分本身可能发生错误。例如，端侧模型可能对某个错误答案具有较高置信度。因此，基于风险的模型路由不能只根据结构设计判断有效性。需要同时评价路由准确性、最终任务表现和云端调用比例。

端云协同的主要局限 端云协同虽然能够利用不同设备的资源，但也会引入新的复杂性。例如：中间特征传输、模型版本一致性、网络连接中断、跨设备缓存管理、任务状态恢复、隐私和数据权限。不同模型结果的一致性。因此，端云协同不一定比云端或端

侧独立部署更加高效。例如，如果传输中间特征的成本很高，协同推理可能反而增加延迟。类似地，如果端侧小模型已经能够满足任务要求，额外云端调用可能没有必要。因此，协同方案应当与完全本地和完全云端两种基线进行公平比较。

16.5.3 大小模型协同与任务分配

大小模型协同的基本动机 大型多模态模型通常具有较强的开放式理解和复杂任务处理能力。但其推理资源需求可能较高。相比之下，小模型具有较低的计算和存储成本，但能力可能相对有限。因此，可以根据任务特点，让不同规模的模型承担不同工作。例如，简单的图像分类可以由小模型完成。复杂的多图推理则可以交给较大的多模态模型。这种设计并不要求所有任务都调用最强模型，而是根据实际需要分配计算资源。例如，可以将一个任务的模型选择表示为：$m^* = R(x)$，其中：

- x 表示当前任务；
- R 表示任务路由策略；
- m^* 表示选择的模型。

这种框架为大小模型协同提供了基本形式。

固定分工与动态分工 大小模型协同可以采用不同策略。

固定分工主要根据任务类型预先决定使用哪个模型。

例如，OCR 使用专用模型，开放式视觉问答使用大型 VLM。

动态分工则根据每次输入的复杂度或其他信息选择模型。

例如，对于较简单的问题，优先使用小模型。如果任务较复杂或需要额外证据，再调用大模型。但动态分工需要可靠的路由机制。例如，如果路由器错误地将困难任务交给小模型，可能降低最终任务表现。因此，任务路由本身也需要独立评价。

基于置信度的模型级联 一种常见设计是根据小模型的置信度决定是否调用大模型。例如：

$$R(x) = \begin{cases} m_s, & c_s(x) \geq \tau \\ m_l, & c_s(x) < \tau \end{cases} \tag{16.45}$$

其中：

- m_s 表示小模型；
- m_l 表示大模型；
- $c_s(x)$ 表示小模型的置信度；
- τ 表示阈值。

但模型置信度不一定与真实正确率一致。

例如，小模型可能对错误的视觉判断具有很高置信度。如果直接将高置信度视为

可靠，系统可能不会将这些任务转交大模型。因此，需要在目标数据上评价置信度校准和路由错误。对于高风险任务，还需要结合业务约束决定是否允许小模型独立完成。

路由的计算成本 如果采用小模型优先、大模型兜底的级联方式，总成本可以表示为：

$$C_{\text{cascade}} = C_s + p_{\text{route}} C_l + C_r \tag{16.46}$$

其中：

- C_s 表示小模型处理成本；
- C_l 表示大模型调用成本；
- p_{route} 表示需要调用大模型的比例；
- C_r 表示额外路由成本。

这一表达假设小模型始终先执行，并且大模型只在部分任务中调用。

如果系统能够在运行小模型之前直接决定调用哪一个模型，成本表达则不同。因此，需要根据具体路由结构计算。例如，不能仅根据大模型调用比例下降，就认为总成本一定下降。还需要考虑小模型、路由器和数据传输所产生的开销。

边缘推理与网络状态 端云协同的效率还取决于网络条件。例如，云端模型可能具有较低的纯推理延迟。但如果设备网络不稳定，数据上传和结果返回可能占用较长时间。可以将云端执行时间概念性地表示为：

$$T_{\text{cloud}} = T_{\text{upload}} + T_{\text{infer}} + T_{\text{download}} \tag{16.47}$$

其中，真实系统还可能包含排队与调度时间。因此，不能只根据云端模型的计算速度判断其是否适合实时任务。例如，对于机器人低层控制，网络不稳定可能造成较大风险。因此，具有严格执行要求的功能通常需要在适当的本地控制机制下运行。

动态计算与任务难度 未来多模态系统还可以根据输入动态调整计算量。例如：简单图像使用较少视觉 Token。复杂图像使用较高分辨率。短问题调用小模型。复杂推理调用大模型。实时视频根据事件变化调整处理频率。这种设计的目标，是避免对全部输入使用相同计算预算。但动态计算策略需要能够识别真正重要的信息。例如，在工业缺陷检测中，一个微小异常可能只占极少像素，却具有较高业务价值。因此，不能仅根据视觉面积或注意力强度决定是否丢弃某些信息。

习题与讨论

1. 若某阶段只占原端到端延迟的 20%，即使该阶段耗时趋于零，总体理想加速上限是多少？说明串行假设。
2. 从张量形状推导本章 KV 缓存表达，分析 GQA、长度增加和批量增加分别改变哪一项。

3. 量化后模型权重变小但实际延迟未改善。列出算子、带宽、反量化、批量和硬件支持方面的可能解释，并设计测量。
4. 设计一个服务化基准，同时固定请求长度分布、并发、输出长度和热身方式，报告延迟分位数及吞吐量。

文献导读 FlashAttention 用于核对算术量、内存访问与实际效率的区别。不能仅根据注意力矩阵规模推断实现的峰值显存。[19]

第十七章　多模态智能体、持续交互与具身行动

智能体把模型输出接到持续执行的过程。观察、内部状态、动作、环境反馈和终止条件需要明确接口；一次看似正确的调用可能在后续步骤中放大错误。本章先讨论数字工具，再扩展到流式观测、具身策略和环境模型。

> **学习目标**　构造工具执行闭环；追踪检索证据与状态版本；满足在线信息可见性；区分离线预测、模拟闭环与真实任务结果。

17.1　闭环、状态与动作接口

模型提出动作，执行器检查格式与权限，环境产生结果，状态管理器决定哪些信息写入上下文。任务是否结束应由可验证条件判断。模型自己声称“已经完成”，不能替代文件、工具结果或环境状态的检查。

17.1.1　感知、规划、行动与反馈

智能体的基本运行循环　智能体的重要特征之一，是能够根据环境变化持续调整行为。例如，一个机器人需要将桌面上的杯子移动到指定区域。它需要观察桌面上的对象。随后，确定目标杯子的位置。接着，规划抓取和移动方式。然后执行动作。通过新的传感器观测判断杯子是否已经到达目标位置。如果抓取失败，系统可能需要重新定位杯子并调整动作。因此，完整过程不仅包含感知与行动，还包含执行之后的反馈。可以将其概括为：

感知（Perception）→ 规划（Planning）→ 行动（Action）→ 反馈（Feedback）→ 状态更新。这一循环可以重复执行，直到任务完成或触发终止条件。

这是一种概念性的功能分解，并不意味着所有智能体内部都必须具有四个彼此独立的神经网络模块。例如，某些端到端机器人策略可以直接根据观测输出动作。但从系统行为角度，仍然可以分析其观测获取、行动选择和环境反馈过程。

感知：获取与任务有关的环境信息　感知是智能体与外部环境建立联系的基础。对于多模态智能体，感知输入可以包含：图像、视频、音频、文本、深度或点云、设备状态、工具返回结果。例如，一个桌面智能体可以通过屏幕截图观察应用界面。一个机器人可以通过 RGB-D 摄像头获取场景外观和空间距离。一个音视频智能体可以利用视频帧和音频事件判断环境变化。因此，多模态感知的优势在于不同模态能够提供互补信息。但不同模态的输入质量和时间信息可能不同。

例如，机器人摄像头提供的是某个时刻的视觉观测，而其他传感器可能以不同频率更新。因此，智能体需要根据时间戳和实际任务判断信息是否仍然有效。不能无条件将历史观测当作当前环境状态。

观测与真实状态的区别 在实际环境中，智能体通常无法直接获得全部真实状态。例如，摄像头可能无法看到被遮挡物体。网页截图可能无法显示页面中尚未展开的内容。机器人也可能无法直接观测物体内部状态。因此，需要区分环境真实状态：s_t，与智能体实际观测：o_t 可以表示为：

$$o_t \sim O(o_t \mid s_t) \tag{17.1}$$

其中，O 表示观测模型。这一表示体现了部分可观测环境的基本思想。例如，智能体看到一个关闭的抽屉，但不能仅根据外观确定抽屉内部是否存在目标物体。因此，智能体需要在信息不完整的条件下推理。某些任务可能通过主动观察获得更多信息，例如打开抽屉或切换网页标签。但这些行动是否被允许，还取决于任务授权和环境约束。

状态估计与历史记忆 由于单次观测可能不完整，智能体往往需要利用历史信息。例如，一个机器人在移动过程中暂时看不到目标对象，可以利用此前观测辅助判断其可能位置。类似地，网页智能体可能需要记住已经填写过哪些字段。因此，可以引入内部状态或记忆：

$$h_t = U(h_{t-1}, o_t, a_{t-1}, f_{t-1}) \tag{17.2}$$

其中：

- h_t 表示当前内部状态；
- U 表示状态更新机制；
- o_t 表示当前观测；
- a_{t-1} 表示此前行动；
- f_{t-1} 表示此前行动反馈。

这一表达是一种一般性抽象。

实际智能体的记忆可能采用不同形式。例如：完整交互历史。压缩后的任务摘要、结构化变量、环境对象状态表。数据库中的持久记录。对于多模态智能体，还可能保存视觉特征或带时间戳的观察结果。保存历史不意味着系统能够准确恢复所有过去信息。例如，压缩摘要可能遗漏细节。历史视觉特征也可能随着环境变化而过期。因此，记忆机制需要根据任务要求进行维护和验证。

行动：从决策输出到环境执行 智能体规划完成后，需要将行动转换为外部环境能够识别的操作。例如，网页智能体可以执行：点击、输入文字、滚动页面、选择菜单、提交表单。机器人则可以执行：移动、抓取、释放、调整姿态。对于软件智能体，行动通常通过工具 API 执行。例如：$r_t = \text{Tool}(a_t)$，其中，a_t 表示工具调用参数，r_t 表示工具

返回结果。工具返回结果和环境真实变化不一定完全相同。

例如，某个 API 返回了请求已接受的信息，但后台任务可能尚未完成。因此，智能体需要根据任务确认工具返回的状态含义。对于具有副作用的操作，还可能需要读取最终环境状态进行验证。

反馈驱动的重新规划 在复杂任务中，初始计划可能无法完全适应实际环境。例如，一个智能体计划通过某个网页按钮打开设置菜单。但实际页面布局发生变化，原来的按钮位置已经改变。如果系统继续按照旧坐标操作，可能发生错误。因此，智能体需要重新观察页面，并根据新状态调整计划。可以将其表示为：

$$\Pi_{t+1} = \text{Replan}(G, h_{t+1}) \tag{17.3}$$

其中：

- P_{t+1} 表示更新后的计划；
- G 表示任务目标；
- h_{t+1} 表示最新内部状态。

并不是每个行动之后都必须重新生成完整计划。

对于稳定和确定的子任务，可以使用固定执行流程。只有当环境状态发生明显变化、原计划失效或出现新的约束时，才需要进行相应调整。因此，实际智能体可以结合固定计划与动态重新规划。

17.1.2 多步骤任务与执行闭环

多步骤任务的基本特点 多步骤任务（Multi-step Task）是指最终目标需要通过多个具有依赖关系的操作才能实现。例如，一个智能体需要根据用户提供的图像和文档，生成一份整理后的分析文件。系统可能需要：理解用户要求、定位相关资料、提取图表数据、完成数值计算、生成结果文件、检查输出内容。这些步骤共同构成任务执行过程。与单次模型调用不同，多步骤任务通常需要维护状态，并根据中间结果决定后续操作。

例如，如果某张图片中的文字识别失败，系统就可能需要重新放大图像，而不是直接继续进行数值计算。因此，多步骤执行的关键在于处理步骤之间的依赖关系和不确定性。

任务目标与完成条件 智能体执行任务之前，需要明确什么结果才算完成。例如，用户要求：“将图像中的实验数据整理为电子表格。”如果模型只生成了表格内容的文字描述，可能还没有满足用户目标。真正完成任务可能需要创建一个文件，并确认其中的数据符合要求。因此，可以定义任务目标集合：

$$G = s : \text{GoalSatisfied}(s) = 1 \tag{17.4}$$

其中，s 表示环境状态。任务完成要求最终状态属于目标集合：$s_T \in G$，例如，目

标文件已经创建，并且包含正确的数据。在真实部署中，智能体可能无法直接观察完整环境状态。因此，任务完成通常需要通过可获得的观测或工具结果进行验证。例如，重新读取文件内容、检查数据库记录，或者查询外部系统的任务状态。

任务分解 多步骤执行通常首先需要将总体目标分解为多个子目标。例如：

$$G \to g_1, g_2, \ldots, g_K \tag{17.5}$$

其中，g_i 表示子任务。例如，对于文档分析：g_1 = 读取文档，g_2 = 提取相关数据，g_3 = 执行比较，g_4 = 生成结果文件，g_5 = 验证最终结果，但任务分解不一定是简单的线性顺序。例如，多个文档页面可以独立解析。某些步骤可以并行执行。而最终汇总需要等待所有必要结果完成。

因此，复杂任务可以使用具有依赖关系的任务图表示。

误差累积 多步骤任务可能出现误差累积。例如，假设每一步独立成功的概率均为：p 如果一个任务需要连续完成：K 个必要步骤，并且必须全部成功，则在独立性假设下，整体成功概率为：$P_{\text{success}} = p^K$，例如：$p = 0.95, \qquad K = 10$，则：

$$0.95^{10} \approx 0.599 \tag{17.6}$$

即整体成功概率约为 59.9%。这只是说明误差累积现象的简化例子。真实任务中的步骤错误通常并不独立。而且智能体可能通过反馈检测和恢复部分失败。因此，不能直接使用这一公式预测真实智能体的任务成功率。但它体现了一个重要问题：即使每个子步骤看起来具有较高成功率，长任务仍然可能因为多个环节的累积错误而明显降低最终成功率。

长任务中的状态保持 复杂智能体任务可能持续较长时间。例如，系统需要分析数十份文档。或者机器人需要完成多个连续操作。因此，系统需要记录已经完成的步骤和当前工作状态。例如：任务目标、当前子任务、已获取的证据、已经执行的动作、外部工具结果、尚未解决的问题。这些信息可以保存在结构化状态中，而不一定全部放入语言模型上下文。例如，某个文档处理智能体可以将已经提取的数据写入临时表格。后续模型只需要访问相关字段，而不必始终保留全部原始文档文本。

但状态压缩可能造成信息丢失。因此，对于重要中间结果，应保留适当的原始证据或可追溯引用。

任务成功率的定义 对于具有明确目标检查器的任务，可以定义：

$$\text{SuccessRate} = \frac{N_{\text{successful episodes}}}{N_{\text{valid episodes}}} \tag{17.7}$$

其中：

- $N_{\text{successful episodes}}$ 表示最终目标得到满足的任务数量；
- $N_{\text{valid episodes}}$ 表示符合评价协议的有效任务数量。

任务是否成功，应根据预先定义的目标检查规则判断。

例如，一个网页智能体可能正确填写了部分字段，但没有提交表单。如果任务要求完整提交，则不能视为成功。类似地，对于机器人任务，完成抓取但没有成功放置对象，也不一定满足最终目标。因此，最终任务成功率通常比单步动作成功率更加直接地反映智能体实际完成任务的能力。

智能体执行预算 多步骤任务可能包含反复观察、调用工具和重新规划。如果缺少终止条件，系统可能持续消耗计算资源。例如，网页智能体反复尝试寻找不存在的按钮。或者文档智能体不断重新检索相同资料。因此，需要为任务设置合理预算。例如：最大执行步数。最大工具调用次数、最大运行时间、最大计算资源。最大允许外部操作范围。可以将任务约束表示为：

$$T_{\text{episode}} \leq T_{\max} \tag{17.8}$$

$$N_{\text{actions}} \leq N_{\max} \tag{17.9}$$

这些约束不只是性能管理措施，也可以防止智能体在异常情况下无限执行。但达到预算限制通常意味着任务中止或未完成，并不等于目标已经成功实现。因此，系统需要区分正常完成、失败终止、主动取消和预算耗尽等状态。

17.2 工具调用与结果验证

17.2.1 函数调用与结构化输出

函数调用的基本概念 函数调用（Function Calling）是指模型根据当前任务，生成符合某个预定义函数或工具接口要求的调用请求，再由外部执行系统完成相应操作。例如，用户询问：“请计算这份实验报告中两个模型的准确率差异。”模型可以根据报告中的数值，决定调用一个数值计算函数。假设系统提供：

```
calculate_difference(a, b)
```

模型需要生成适当参数：

```
{
  "a": 86.3,
  "b": 82.7
}
```

外部程序接收到参数后，实际执行计算，再将结果返回模型或应用程序。

模型生成函数调用请求，并不意味着函数已经执行。例如，模型可以生成：

```
{
  "name": "calculate_difference",
  "arguments": {
    "a": 86.3,
    "b": 82.7
  }
}
```

但只有执行器接收到该请求，并完成相应程序调用，才能得到实际计算结果。

因此，函数调用通常涉及模型和外部执行系统之间的协作。

模型输出与外部执行的区别 普通语言模型主要输出文本 Token。但在函数调用场景中，模型可以按照特定接口协议，生成一个具有明确工具名称和参数的结构化消息。例如：$c_t = (n_t, \mathbf{a}_t)$，其中：

- n_t 表示工具名称；
- $\mathbf{a}_t$ 表示调用参数；
- c_t 表示模型提出的工具调用。

随后由外部执行器完成：

$$r_t = \text{Execute}(n_t, \mathbf{a}_t) \tag{17.10}$$

其中，r_t 表示工具执行结果。模型还可以根据工具结果继续推理：$y_{t+1} = f_\theta(Q, h_t, r_t)$，这里，Q 表示用户任务，$h_t$ 表示已有交互状态。这一过程体现了模型推理与程序执行之间的分工。模型主要负责理解任务并构造调用请求。执行器则负责按照工具接口和实际权限进行操作。因此，模型的自然语言生成能力不能替代执行器的程序逻辑与权限检查。

工具接口的基本组成 为了使模型能够使用工具，系统需要向模型提供适当的工具描述。一个典型工具接口可以包含：工具名称、功能说明、输入参数名称、参数数据类型、必要参数、可选参数、参数取值范围、返回结果结构、可能的错误类型。例如，一个用于查询图像对象位置的工具，可以定义为：

```
{
  "name": "locate_object",
  "description": " 根据文本描述定位图像中的目标对象",
  "parameters": {
    "type": "object",
    "properties": {
      "image_id": {
        "type": "string",
        "description": " 已授权的图像资源标识"
      },
```

```
      "query": {
        "type": "string",
        "description": " 目标对象的文本描述"
      },
      "max_results": {
        "type": "integer",
        "minimum": 1,
        "maximum": 10
      }
    },
    "required": [
      "image_id",
      "query",
      "max_results"
    ],
    "additionalProperties": false
  }
}
```

上述示例主要说明工具参数可以采用 JSON Schema 描述。

例如，$image_id$ 用于指定图像资源。query 用于描述需要定位的对象。$max_results$ 用于限制返回目标数量。这里的接口只是一种教学示例。实际系统还需要确定图像资源如何注册、调用方是否具有访问权限，以及目标定位工具的具体返回格式。

JSON Schema 与结构化约束 JSON Schema 是用于描述和验证 JSON 数据结构的规范。例如，可以规定某个字段必须为字符串，另一个字段必须为整数。也可以限制允许的枚举值或参数范围。对于工具调用，结构化约束能够减少不合法的参数组合。例如，如果工具要求图像资源标识为字符串，就不应接受一个任意数组作为该字段的输入。类似地，如果某个参数只能取：

```
"image"
"video"
"audio"
```

则可以通过枚举约束明确允许的模态类型。

但结构合法不等于语义正确。例如，模型可以生成一个完全符合 Schema 的 $image_id$，但这个标识可能不存在。模型也可以生成符合数值范围的图像坐标，却指向错误对象。因此，结构化验证只能解决部分接口正确性问题。

多模态工具的参数表示 对于纯文本工具，参数可能只包含字符串和数值。但多模态工具还可能需要处理图像、视频、音频和空间信息。例如，视觉定位工具可能需要：图像资源标识。目标文本描述。坐标系或输入尺寸。返回数量。视频事件检索工具可能需要：视频资源标识、查询事件、开始和结束时间、时间单位。音频工具可能需要：音频

资源标识、时间片段、采样率、处理任务类型。因此，多模态工具接口设计需要明确参数的物理和语义含义。

例如，视频时间 12.5 可能表示第 12.5 秒，也可能表示某种经过转换的时间单位。类似地，图像边界框可能使用原始像素坐标，也可能使用相对于图像尺寸的归一化坐标。如果这些约定不明确，即使模型生成的参数符合 JSON Schema，也可能调用错误。

工具调用中的类型安全与语义安全 在工具调用中，可以区分不同层次的正确性。语法正确性：生成结果可以按照规定格式解析。结构正确性：参数类型和字段符合工具 Schema。语义正确性：参数对应用户真实目标及其适用条件。权限正确性：调用者被授权访问对应资源或执行操作。例如，一个工具调用可以具有正确的 JSON 格式。但文件标识可能指向错误文件。或者用户没有权限修改该文件。因此，形式化参数约束只能覆盖部分错误。

对于具有实际副作用的工具，还需要执行器独立检查权限、资源状态和业务规则。

函数调用的研究价值与局限 函数调用使多模态模型能够与外部程序建立明确接口。例如，模型不必用自然语言模拟复杂数值计算，而可以调用计算程序。对于文档处理和图像分析，也可以利用专门的解析或定位工具提高部分操作的可检查性。但函数调用机制不能直接解决所有智能体可靠性问题。例如，模型可能选择错误工具，或者根据错误的视觉理解生成参数。因此，函数调用提供的是能力扩展和执行接口，并不等于自动获得可靠规划与正确行动。

17.2.2 工具选择与参数生成

工具选择是一个决策问题 当智能体具备多个工具时，需要判断当前任务应该使用哪一个工具。例如，用户要求：“比较两份报告中的实验结果，并生成汇总表格。”系统可能需要从以下工具中选择：文档读取工具。OCR 工具、图像定位工具、计算工具、文件创建工具、搜索工具。不同工具具有不同功能。因此，工具选择不仅是根据关键词匹配工具名称，还需要理解任务目标和工具能力之间的关系。例如，如果原始 PDF 已经包含可提取的结构化表格，系统可能不需要先执行 OCR。

如果表格以扫描图像形式存在，则可能需要视觉解析。因此，工具选择应结合实际输入状态，而不是采用固定的工具调用顺序。

工具选择的形式化表示 设当前可用工具集合为：

$$\mathcal{T} = \tau_1, \tau_2, \ldots, \tau_M \tag{17.11}$$

智能体可以根据用户任务、当前状态和工具描述，选择下一步工具：

$$\tau^* = \arg\max_{\tau \in \mathcal{T} \cup \varnothing} U(\tau \mid G, h_t) \tag{17.12}$$

其中：

- G 表示任务目标；
- h_t 表示当前任务状态；
- U 表示工具在当前状态下的预期效用；
- $\varnothing$ 表示不调用工具。

这一表达用于说明工具选择的决策性质，并不意味着所有智能体都显式计算这样的效用函数。

例如，某些模型通过下一 Token 预测生成工具名称。另一些系统可能使用规则或检索器先筛选候选工具。但无论采用哪种实现，工具选择都应允许“不调用任何工具”这一结果。

工具能力发现 对于工具数量较少的系统，可以直接向模型提供全部工具定义。但当系统具有大量工具时，完整工具描述可能占用较长上下文。例如，一个系统连接了文件、数据库、日程、检索和媒体处理等多个服务。如果将全部工具参数都放入每次模型请求，可能增加 Prefill 成本，并使模型在相似工具之间产生混淆。因此，可以先根据任务检索候选工具。例如：

$$\mathcal{T}_{\text{cand}} = \text{RetrieveTools}(Q, \mathcal{T}, K) \tag{17.13}$$

其中，$\mathcal{T}_K$ 表示选出的候选工具集合。随后，模型仅在候选工具中进行选择。ToolLLM 采用工具检索与工具调用学习相结合的研究路线，其 ToolBench 数据集覆盖大量真实 API。这类方法说明，随着工具集合扩展，工具发现本身也成为需要优化的问题。但候选工具检索可能发生漏召回。如果真正需要的工具没有进入候选集合，后续模型即使具有较强推理能力，也可能无法完成任务。

因此，需要同时评价工具召回率和最终调用正确率。

工具描述的清晰性 工具描述对模型选择具有重要影响。例如，系统提供两个工具：

```
search_documents
search_images
```

如果工具描述仅包含名称，模型可能无法明确区分两者。

更合理的方式是说明：$search_documents$ 用于检索文档文本和结构化内容。$search_images$ 用于根据语义描述检索图像或视觉资源。此外，还需要说明各工具返回什么信息。例如，图像检索工具可能返回文件标识、缩略图和相似度分数。文档检索工具可能返回文本片段、页码和来源位置。这种明确的能力边界，有助于模型正确选择工具。但描述越长并不意味着越有效。过于冗长或相互矛盾的描述可能增加模型理解负担。

因此，应当使用清晰、一致且具有区分度的工具说明。

多模态参数中的空间对应 视觉工具参数经常涉及图像坐标。例如，某个工具返回边界框：$B = (x_1, y_1, x_2, y_2)$，但这一边界框可能使用原始图像坐标，也可能使用模型预处理

后的图像坐标。例如，原始图像尺寸为：1920×1080 模型实际处理的图像可能经过缩放。如果系统直接将缩放后坐标用于原始图像，就可能发生定位错误。因此，工具接口需要明确坐标系及其转换关系。例如，设原始图像和处理图像之间存在可逆或已知的几何变换：

$$p' = \mathcal{T}(p) \tag{17.14}$$

则将处理结果映射回原始图像时，需要使用相应的逆变换或明确的坐标映射规则。但如果预处理包含裁剪、填充或复杂几何变换，不能简单只按照一个缩放比例恢复。因此，多模态参数生成和结果处理应保留必要的预处理元数据。

工具调用的串行与并行 多步骤任务中，不同工具可能具有依赖关系。例如，先读取文档，再计算其中的指标。这两个步骤不能随意交换。但也有一些操作可以并行执行。例如，同时读取两份独立报告。随后再将结果汇总。可以将工具调用表示为任务依赖图：

$$\mathcal{G} = (V, E) \tag{17.15}$$

其中，节点表示工具操作，边表示执行依赖。如果两个操作之间不存在数据或状态依赖，就可以考虑并行。例如：$r_1 = \tau_1(a_1)$，$r_2 = \tau_2(a_2)$ 两个调用可以在适当条件下同时执行。但并行工具调用可能产生资源竞争。此外，如果多个工具同时修改同一文件或数据库记录，还可能出现写入冲突。因此，并行执行需要结合实际依赖和资源约束。

工具调用中的动态规划 在实际任务中，工具调用过程可能随着返回结果变化。例如，智能体最初希望直接读取某个文档中的表格。如果文档解析工具返回文本不可提取，系统可能改用页面图像和 OCR 工具。因此，工具调用不一定是预先固定的线性序列。可以将当前行动选择表示为：

$$a_t = \pi(G, h_t, \mathcal{T}_t) \tag{17.16}$$

其中：

- G 表示任务目标；
- h_t 表示当前任务状态；
- $\mathcal{T}_t$ 表示当前可用工具集合。

执行之后，根据返回结果更新状态：$h_{t+1} = U(h_t, a_t, r_t)$，

随后选择下一步工具或结束任务。这种方式体现了工具使用与智能体反馈闭环之间的联系。但不是所有简单任务都需要复杂的动态规划。对于具有明确执行顺序的任务，经过验证的固定流程可能更加高效和稳定。

17.2.3 工具执行结果的验证

工具执行为什么需要验证 工具调用并不保证任务完成。例如，一个智能体要求数据库更新某个字段。工具可能返回成功。但如果模型选择了错误记录，最终任务仍然可能失败。类似地，OCR 工具可能成功读取文档，却错误识别某个数字。计算器可能正确计算输入的数值，但这些数值可能来自错误的表格单元格。因此，工具执行结果需要根据任务目标进行验证。这种验证不仅需要检查工具是否正常运行，还需要检查返回信息是否能够支持最终结论。

调用正确、执行成功与任务成功 可以将工具使用的成功过程区分为三个层次。调用正确性主要关注工具名称和参数是否符合任务要求。执行成功性主要关注工具是否真正完成对应程序操作。任务成功性主要关注最终环境状态或回答是否满足用户目标。例如，用户要求："将某份文件中的模型准确率更新为指定数值。"模型可能生成正确的写入工具调用。但如果文件系统权限不足，工具执行可能失败。即使工具执行成功，如果修改了错误的文件，也不满足用户目标。因此：

$$S_{\text{call}} \neq S_{\text{execution}} \neq S_{\text{task}} \tag{17.17}$$

这一表达用于强调不同成功条件不等价。实际研究需要明确每一项指标的判断方式。

任务状态验证 对于具有外部副作用的工具，最终环境状态通常比工具返回的自然语言描述更加重要。例如，一个智能体执行文件创建任务。模型输出："文件已经创建完成。"这并不能直接证明实际文件存在。更合理的验证方式是，通过文件系统检查目标文件。例如，可以定义：

$$\text{Success} = \mathbb{I}[s_T \in G] \tag{17.18}$$

其中：

- s_T 表示任务执行后的环境状态；
- G 表示满足目标和必要约束的状态集合。

例如，目标文件确实存在，并且包含正确的数据，才满足对应任务。

实际智能体可能无法直接观察完整状态。因此，需要根据任务设计可执行的验证器或状态查询接口。

τ-bench：通过最终数据库状态评价工具智能体 τ-bench 是面向工具智能体与用户交互的重要评价基准。该研究构建了模拟零售和航空服务等业务场景的交互任务，要求智能体在遵守领域规则的条件下完成用户目标。与只比较工具调用文本的评价方式不同，τ-bench 重点检查最终数据库状态是否满足预先定义的目标。例如，智能体可能需要修改某项业务记录。即使过程中生成了多个看似合理的工具调用，最终记录没有达到目标状态，仍然不能视为成功。

此外，τ-bench 还引入 $pass^k$ 等指标分析多次运行中的一致性。这说明，工具型智能体的评价不仅需要关注一次成功，还需要考虑重复执行时的稳定性。但具体指标必须结合基准定义解释，不能把不同论文中的 $pass^k$，与普通单次成功率直接混用。

ToolSandbox：**有状态工具使用** ToolSandbox 是面向有状态、多轮交互工具任务的评价基准。其特点之一是能够在具有状态依赖的工具环境中，检查智能体的中间行为与最终结果。例如，某个操作可能改变后续工具调用能够访问的数据。因此，工具调用的正确性不仅取决于当前参数，也取决于此前执行的操作。这种设置更加接近实际智能体任务。例如，先创建文件，再修改文件内容，与直接修改一个尚不存在的文件，具有不同前置条件。

因此，工具执行评价需要考虑状态依赖，而不能只判断每次调用是否符合固定的静态答案。

工具执行结果的统计评价 可以根据任务分别定义工具使用指标。例如，工具选择准确率：

$$\text{ToolSelectionAcc} = \frac{N_{\text{correct tool}}}{N_{\text{eligible}}} \tag{17.19}$$

其中，分母应明确是否包含无需调用工具的样本。参数正确率：

$$\frac{N_{\text{correct arguments}}}{N_{\text{evaluated calls}}} \tag{17.20}$$

工具执行成功率：

$$\text{ExecutionSuccess} = \frac{N_{\text{executed successfully}}}{N_{\text{attempted}}} \tag{17.21}$$

最终任务成功率：

$$\frac{N_{\text{successful tasks}}}{N_{\text{valid tasks}}} \tag{17.22}$$

这些指标具有不同的统计对象。例如，参数正确率可以采用严格结构匹配，也可以采用语义等价判断。实际定义取决于任务和基准。因此，比较不同研究结果时，应检查指标分母和成功条件是否一致。

17.3 检索、记忆与执行恢复

17.3.1 文档切分、嵌入与向量检索

外部知识库的基本作用 外部知识库用于保存模型推理过程中可能需要访问的信息。例如：技术文档、论文与研究报告、产品说明书、数据库记录、网页资料、图像与视频资源、历史任务结果。与模型参数中的知识不同，外部知识库通常可以在不重新训练完

整模型的条件下更新。例如，当某个产品发布新的技术说明书时，可以将新版文档加入知识库，并更新相应索引。这样，模型在后续查询中就有机会获得新信息。但知识库中存在正确资料，并不意味着模型一定能够检索到这些资料。

此外，检索到相关内容，也不意味着生成模型一定能够正确理解。因此，知识库构建和检索质量是 RAG 系统的重要基础。

文档解析 在建立检索索引之前，通常需要将原始文档转换为可处理的数据形式。例如，一份 PDF 文档可能包含：章节标题、正文段落、公式、表格、插图、脚注、参考文献。对于文本型 PDF，可以通过文档解析程序提取文字和部分版面结构。对于扫描型 PDF，则可能需要 OCR 或视觉文档理解模型。但文档解析可能改变原始信息。例如，一个复杂表格在转换为纯文本之后，可能丢失行列对应关系。某张图表中的曲线走势，也可能无法通过 OCR 完整表示。

因此，对于结构复杂的多模态文档，需要考虑保留页面图像、表格结构和原始资源位置，而不是只保存提取后的文字。

重叠切分 固定长度切分可能将完整语义内容截断。例如，一个实验结果的描述可能跨越两个相邻片段。如果只检索其中一个片段，就可能缺少必要上下文。因此，可以在相邻片段之间保留一定重叠。例如：$c_1 = [1, 500]$，$c_2 = [401, 900]$，这意味着两个片段共享部分文本。重叠切分有助于降低边界截断造成的信息损失。但重叠过多也可能产生问题。例如：索引数据量增加。重复片段较多。检索结果缺乏多样性。模型上下文包含重复信息。

因此，重叠比例需要根据实际任务确定。不能简单认为重叠越多，检索性能就一定越好。

文本嵌入 完成切分之后，需要将文档片段转换为检索表示。文本嵌入（Text Embedding）是其中一种常见方法。例如：$z_i = f_{\text{emb}}(c_i)$，其中：

- c_i 表示文档片段；
- f_{emb} 表示文本嵌入模型；
- z_i 表示片段的向量表示。

对于一个固定维度的嵌入模型，可以得到：$z_i \in \mathbb{R}^d$，

其中，d 为嵌入维度。Sentence-BERT 是语义文本嵌入的重要代表性研究。该方法通过孪生网络和相关训练目标，使句子能够转换为适合语义相似度比较的向量表示。这种方式为后续密集语义检索提供了技术基础。

查询嵌入 当用户提出问题时，系统同样可以将查询转换为向量。例如：$z_q = f_q(q)$，其中，q 表示当前问题。随后根据查询向量与文档向量之间的相似度，选择相关片段。例如：

$$c^* = \arg\max_{c_i} \text{sim}(z_q, z_i) \tag{17.23}$$

查询编码器和文档编码器不一定完全相同。例如，双编码器检索模型可以分别使用查询编码器和文档编码器。但二者需要经过适当训练，使其输出能够进行有效相似度比较。因此，不能将任意两个没有对齐关系的嵌入模型直接组合为一个可靠检索系统。

余弦相似度 余弦相似度是向量检索中常见的相似度计算方式。例如：

$$\mathrm{sim}(z_q, z_i) = \frac{z_q^\top z_i}{\|z_q\|_2 \|z_i\|_2} \tag{17.24}$$

其主要衡量两个向量之间的方向相似程度。如果向量已经进行 L2 归一化，则余弦相似度等于向量内积。但不同嵌入模型可能使用不同的距离或相似度定义。例如，有些模型针对内积检索进行训练。另一些模型可能适合使用欧氏距离或其他度量。因此，应根据嵌入模型的训练方式与官方使用要求选择检索度量。

混合检索 混合检索（Hybrid Retrieval）是指结合不同检索方法的优势。例如，同时使用 BM25 和密集向量检索。BM25 可以发现关键词精确匹配的资料。密集检索则可以补充语义表达不同但内容相关的资料。随后，系统将不同候选集合合并，并通过适当方式生成最终排序。例如，可以使用 Reciprocal Rank Fusion（RRF）融合多个检索器的排名。其基本形式为：

$$\mathrm{RRF}(d) = \sum_{j=1}^{M} \frac{1}{k + \mathrm{rank}_j(d)} \tag{17.25}$$

其中：

- $\mathrm{rank}_j(d)$ 表示文档 d 在第 j 个检索器中的排名；
- k 为控制排名贡献的常数；
- M 表示检索器数量。

Cormack 等人在 2009 年提出了这一融合方法。

RRF 依赖排名位置，而不是要求不同检索器输出完全可比较的原始分数。但融合策略是否优于单一检索，需要通过具体数据验证。

多向量检索与 ColBERT 传统密集检索通常将一个段落压缩为单个向量。这种方式计算和存储较为简单，但可能丢失部分细粒度匹配信息。例如，一段文字同时包含多个不同概念。单个向量未必能够清晰表示每个关键词与查询之间的关系。ColBERT 提出基于上下文化 Token 表示和后期交互（Late Interaction）的检索方式。其基本思想是分别编码查询和文档，在匹配阶段通过多个 Token 向量之间的相似度计算相关分数。

例如，可以采用 MaxSim 形式：

$$\mathrm{Score}(q, d) = \sum_{i=1}^{|q|} \max_j \mathrm{sim}(q_i, d_j) \tag{17.26}$$

其中，q_i 表示查询 Token 向量，d_j 表示文档 Token 向量。这种方式能够保留更细粒度的匹配关系。但多向量检索通常需要存储更多表示，并具有相应的匹配计算成本。因此，实际系统需要根据知识库规模和检索精度要求选择。

检索质量的基本评价 对于检索系统，可以使用 Recall@K、Precision@K、MRR 和 nDCG 等指标。例如，Recall@K 可以定义为：

$$\text{Recall@}K = \frac{|\text{Relevant} \cap \text{TopK}|}{|\text{Relevant}|} \tag{17.27}$$

其中，Relevant 表示预先标注的相关文档集合。但检索评价中的“相关”具有不同层次。例如，一段文字可能与问题主题相关，却没有提供回答所需的事实。因此，RAG 检索评价最好进一步检查关键证据是否被找到。例如，某个问题需要使用实验表格中的具体数值。如果系统检索到论文的背景介绍，却没有检索到对应表格，就不应认为已经成功找到必要证据。

17.3.2 文本、图像与多模态检索

从文本检索到多模态检索 传统 RAG 主要围绕文本知识库构建。例如，先将文档中的段落转换为文本嵌入，再根据用户查询寻找相关内容。但实际多模态知识库可能包含多种数据形式。例如：图片、图表、扫描文档、视频片段、语音录音、环境声音。这些数据不一定能够通过纯文本完整表达。例如，一张实验曲线图可能包含多个模型随训练轮次变化的趋势。如果只通过 OCR 提取图例和坐标文字，就可能丢失曲线形状和变化关系。

类似地，一段视频中的动作顺序，不能只依靠独立帧的物体名称完整表示。因此，多模态检索需要根据输入数据特点选择合适的表示和检索方式。

多模态检索的基本形式 多模态检索是指查询与被检索内容可以属于不同模态，或者检索系统需要联合利用多种模态信息。例如：文本检索图像、图像检索图像、文本检索视频、音频检索音频。文本检索声音事件。图文联合检索文档页面。例如，用户输入：“查找包含红色汽车的图片。”系统需要根据文本查询，从图像集合中寻找对应内容。这种方式要求文本与图像在检索空间中具有可比较的语义表示。因此，跨模态表征学习成为多模态检索的重要基础。

CLIP 与图文共同表示空间 CLIP[8] 是图文跨模态对齐的重要代表性模型。其基本思想是通过大量图像—文本配对数据训练视觉编码器和文本编码器，使对应图像与文本在嵌入空间中更加接近。例如：$z_v = f_v(I)$，$z_t = f_t(Q)$，其中：

- f_v 表示图像编码器；
- f_t 表示文本编码器；
- I 表示图像；
- Q 表示文本描述。

随后可以计算：$s(v,t) = \text{sim}(z_v, z_t)$ 并根据相似度检索图像。

例如，用户输入：“一只狗正在草地上奔跑。”系统可以从图像库中寻找与描述相匹配的图片。CLIP 的相似度不应直接解释为图文内容正确匹配的严格概率。此外，CLIP 也不一定能够准确处理所有细粒度空间关系和文字内容。因此，图文检索结果仍然可能需要进一步验证。

SigLIP 与图文对齐 SigLIP 是另一种具有代表性的图文预训练方法。Zhai 等人在《Sigmoid Loss for Language Image Pre-Training》中提出使用成对 Sigmoid 损失进行图文对齐训练。与依赖批次内 Softmax 归一化的典型对比学习方式相比，SigLIP 使用针对图文配对关系的 Sigmoid 目标。该研究重点讨论了这种训练目标在不同批次规模下的性能和效率。

对于检索系统，SigLIP 可以作为图像和文本编码器的候选方法之一。但采用 SigLIP 或 CLIP 并不能自动保证特定检索任务中的最佳效果。例如，高分辨率文档检索和自然图像检索具有不同的信息需求。因此，应根据实际数据类型和评价协议选择嵌入模型。

视频检索 视频检索需要考虑时间维度。例如，用户询问：“找到视频中人物第一次打开门的片段。”系统不仅需要判断视频是否包含开门动作，还需要定位相应时间区间。因此，可以将视频表示为一组时间片段：$V = v_1, v_2, \ldots, v_M$ 每个片段具有：$v_i = (t_i^s, t_i^e, x_i)$，其中：

- t_i^s 表示开始时间；
- t_i^e 表示结束时间；
- x_i 表示视频片段内容或其表示。

系统可以为每个片段建立视觉或视频特征，并根据文本查询检索相关片段。

VideoCLIP 是视频与文本跨模态对齐的代表性研究。该方法通过视频和文本之间的对比预训练，学习用于视频理解和检索的共同表示。但视频检索的时间粒度可能影响结果。例如，一个较长片段可能包含目标动作，却不能准确指出事件开始位置。因此，视频检索需要区分片段级相关性与精确时间定位。

多模态检索结果融合 当不同检索器返回结果时，需要决定如何形成统一候选集合。例如，文本检索返回了某个段落。视觉检索返回了同一文档中的某张图表。二者可能共同支持用户问题。因此，可以采用文档标识和位置元数据，将不同结果关联起来。例如：

$$\mathcal{E}(q) = E_{\text{text}}(q) \cup E_{\text{image}}(q) \cup E_{\text{video}}(q) \tag{17.28}$$

其中，各集合分别来自不同模态检索。这种并集只表示候选信息的合并。实际排序还需要考虑相关性、证据质量和查询要求。此外，不同模态检索器输出的相似度可能不可直接比较。例如，图像模型的相似度与文本模型的检索分数，通常不具有相同的数值含义。因此，可以采用适当的排名融合或经过校准的评分融合方法。

17.3.3 检索结果与模型上下文

从检索候选到生成上下文 检索器通常返回多个候选片段。例如：$E_K = e_1, e_2, \ldots, e_K$，但这些候选不一定全部适合直接输入生成模型。例如，某些结果可能与问题只有弱相关关系。另一些结果可能重复。还有一些资料可能已经过时。因此，RAG 需要将检索候选进一步处理，形成适合模型使用的上下文。例如：

$$C_q = \text{ConstructContext}(Q, E_K) \tag{17.29}$$

随后模型生成：$Y = f_\theta(Q, C_q)$，其中，C_q 表示最终提供给模型的检索上下文。检索上下文并不一定只包含文本。对于多模态 RAG，它还可能包含图像页面、视频片段或其他模态表示。因此，上下文构建需要根据生成模型实际支持的输入形式进行设计。

检索结果的重排序 初始检索通常需要兼顾召回率和效率。例如，向量检索器可以较快返回大量候选。但这些候选的排序可能并不完全符合当前问题。因此，可以使用重排序（Reranking）模型，对候选结果进行更加精细的相关性判断。例如，先检索前 50 个候选片段。再通过重排序模型选择最相关的 5 个。重排序模型可以使用不同结构。例如，Cross-Encoder 可以联合处理查询和候选文本，直接估计相关性。

与独立编码的双编码器相比，这种方式通常具有更丰富的查询与候选交互，但计算成本也可能较高。因此，常见设计是先通过高效检索获得候选，再执行相对昂贵的精细排序。

多证据组合 某些问题需要联合多个证据。例如：“比较两篇论文在同一数据集上的实验表现。”系统可能需要分别检索两篇论文的实验表格。随后确认：数据集是否相同。评价指标是否相同。模型版本是否相同。测试协议是否一致。因此，单个检索片段通常不足以完整回答。可以将支持某个结论的证据集合表示为：$E_Y = e_1, e_2, \ldots, e_m$，其中，不同证据可能来自不同文档。多个真实证据之间也可能存在条件差异。

例如，某个准确率来自验证集，另一个来自测试集。如果忽略这些区别，就可能形成错误比较。因此，多证据组合不仅要求检索到信息，还需要正确理解证据适用范围。

上下文长度预算 生成模型通常具有上下文长度限制。例如，用户问题、对话历史和检索片段都需要占用上下文空间。因此，可以将 Token 预算表示为：

$$B_{\text{total}} = B_{\text{instruction}} + B_{\text{history}} + B_{\text{evidence}} + B_{\text{output}} \tag{17.30}$$

其中：

- $B_{\text{instruction}}$ 表示任务指令预算；
- B_{history} 表示对话历史预算；
- B_{evidence} 表示检索证据预算；
- B_{output} 表示需要保留的生成空间。

这一表达是简化的上下文分配方式。

实际多模态模型还可能将图像和视频特征转换为大量视觉 Token。因此，输入上下文预算不能只根据文字字符数量估计。例如，一张高分辨率文档页面可能占用较多视觉 Token。因此，实际系统需要根据模型的多模态 Token 计算规则管理上下文。

生成结果的证据约束 RAG 的重要目标之一，是使生成内容能够受到检索证据支持。例如，模型回答："方法 A 的准确率为 83.2%。"系统应当能够找到支持该数值的具体表格或文本。因此，可以将回答拆分为多个事实性主张：$Y = a_1, a_2, \ldots, a_M$，随后检查每个主张是否具有对应证据：$\text{Support}(a_i, E)$，其中，E 表示检索证据集合。证据支持判断可能涉及多个片段联合推理。

例如，比较两个模型时，需要分别检查它们的实验结果。因此，证据约束不只是检查回答是否包含引用编号，还需要确认引用的内容能够真正支持结论。

RAG 的端到端评价 RAG 系统通常包含多个处理阶段。因此，最终答案错误可能来自不同原因。例如：文档解析错误。切分导致必要信息分离。嵌入模型未能表达查询语义。检索索引未能召回关键证据。重排序遗漏相关文档。生成模型未能正确使用证据。因此，需要分别评价检索与生成环节。RAGAs 是面向检索增强生成评价的重要研究框架。它从检索上下文相关性和回答忠实性等不同角度，提供了自动化评价思路。但自动评价器本身也可能产生错误。

例如，某个模型评价器可能错误地认为某段文字支持某个数值结论。因此，重要实验仍然需要使用经过确认的标准答案、人工核查或其他可靠验证方式。

17.3.4 短期上下文与长期记忆

智能体为什么需要记忆 在单次视觉问答任务中，模型通常只需要根据当前图像和问题生成回答。例如，用户上传一张图片并询问："图中有几辆汽车？"模型完成回答后，当前任务就可以结束。但对于连续交互任务，当前输入可能不足以决定正确行动。例如，用户先要求智能体读取一份报告，随后又提出："请把刚才提取的数据与另一份报告进行比较。"此时，第二个问题依赖此前任务中获得的信息。如果系统没有保存前一次的数据提取结果，就可能需要重新读取原始报告。

类似地，机器人在移动过程中可能暂时看不到目标物体。此时，系统需要利用此前的观察结果，判断目标物体可能位于什么位置。因此，记忆的基本作用，是保存当前任务仍然需要、但未必能够从当前输入直接获得的信息。从功能上看，智能体记忆主要支持历史信息利用、任务状态维护、经验复用以及跨时间的环境理解。但保存大量历史信息并不自动意味着具备有效记忆能力。

例如，系统可能保存了全部聊天记录，却无法在需要时找到其中的重要数据。因此，记忆的关键不仅在于存储，还包括信息的选择、组织、检索与更新。

短期上下文与工作记忆 短期上下文通常指模型当前推理过程中能够直接访问的信息。例如，当前用户请求、近期对话、工具返回结果和已经检索到的证据。这些信息通常被组织到模型的输入上下文中。从任务功能角度，可以将当前工作状态表示为：$h_t = (G_t, O_t, P_t, R_t)$，其中：

- G_t 表示当前任务目标；
- O_t 表示最近的环境观测；
- P_t 表示正在执行的计划；
- R_t 表示当前需要使用的中间结果。

这一表示用于说明工作记忆应当包含与当前决策有关的信息，并不是对所有智能体内部结构的统一规定。

例如，一个文档处理智能体的工作记忆可能包含当前正在分析的文件、已提取的实验指标、尚未完成的任务和下一步需要调用的工具。因此，工作记忆可以理解为支持当前任务决策的一组活动信息。工作记忆并不一定等于完整聊天记录。它也可以由结构化数据和外部状态引用共同构成。

上下文窗口与记忆容量 对于基于 Transformer 的大模型，输入上下文具有一定长度限制。例如，用户问题、历史对话、工具返回结果和多模态特征，都可能占用模型上下文。当历史交互不断增加时，系统可能无法将全部内容持续放入当前输入。因此，需要决定哪些信息保留在工作上下文中，哪些信息转移到外部存储，以及哪些信息不再需要。

例如，一项复杂任务包含数十次文档读取和工具调用。如果每次都将全部历史重新输入模型，不仅会增加 Token 成本，还可能使模型难以定位当前最重要的信息。因此，长任务通常需要使用上下文管理机制。例如，可以保留当前目标、最近的执行结果和关键约束，将较早的详细记录存入外部任务存储。应区分：模型推理时的 KV Cache 与智能体的长期记忆不是同一概念。

KV Cache 主要保存 Transformer 注意力计算所需的历史状态，用于减少自回归生成中的重复计算。而智能体记忆强调信息能否在之后的任务和决策中被正确保存、检索和使用。因此，具有较大的 KV Cache，不意味着系统自动具有可靠的长期记忆能力。

多模态智能体的记忆内容 对于多模态智能体，记忆不应局限于自然语言摘要。例如，机器人可能需要记录：目标对象的视觉外观、对象所在位置、观察时间、机器人位姿、对象是否被移动。相应的传感器证据。类似地，视频智能体可能需要保存事件时间区间和关键帧。因此，一个多模态记忆单元可以概念性地表示为：$m_i = (z_i, t_i, l_i, r_i, q_i)$，其中：

- z_i 表示记忆内容或其特征；
- t_i 表示相关时间；
- l_i 表示空间或文档位置；
- r_i 表示原始证据引用；
- q_i 表示与记忆质量有关的辅助信息。

这里的质量信息可能包含来源可靠性或估计的不确定性。

质量分数不一定是严格校准的正确概率。因此，不能仅凭一个分数判断记忆内容绝对可信。

记忆检索与重要性选择 当智能体需要使用历史信息时，可以根据当前任务检索相关记忆。例如：

$$\mathcal{M}_t^{\text{relevant}} = \text{Retrieve}(G_t, h_t, \mathcal{M}) \tag{17.31}$$

其中：

- G_t 表示当前目标；
- h_t 表示当前工作状态；
- $\mathcal{M}$ 表示长期记忆集合；
- M_t^* 表示检索得到的相关记忆。

检索可以考虑多个因素。

例如：语义相关性、时间新近性、事件重要性、来源可靠性。与当前对象或场景的对应关系。例如，智能体需要判断某个杯子的当前位置。最近一次可靠的视觉观测，可能比数小时前的位置记录更加重要。但对于查询物体最初放置在哪里的问题，较早的历史记录反而可能是必要证据。因此，记忆选择不能简单地始终优先最新内容。它需要结合问题的时间语义和实际任务目标。

长期记忆的评价 长期记忆不能只通过保存的信息数量评价。例如，一个系统存储了大量历史事件，却不能正确回答关于过去状态的问题，那么其记忆能力仍然有限。因此，可以从多个方面评价：历史事实检索准确率。对象状态追踪能力。跨时间事件排序能力。长期任务完成率。过期记忆识别能力、记忆更新延迟、存储与检索成本。例如，EgoMemReason 面向长时间第一视角视频，分别评价实体记忆、事件记忆和行为模式记忆等能力。

这类研究说明，长期记忆不仅需要找回单个过去片段，还可能需要整合多个时间点的信息。因此，完整的多模态记忆评价，应当同时关注信息保留、时间关联和任务利用能力。

17.3.5 任务分解与执行规划

任务规划的基本作用 任务规划是智能体根据目标、环境状态和可用行动，决定如何完成任务的过程。例如，用户提出：“请阅读三份多模态模型实验报告，比较各方法在相同数据集上的性能，并生成分析文档。”这一目标通常不能通过一次模型调用完成。系统可能需要先读取报告，再提取数据，随后执行比较，最后生成文件并进行检查。因此，需要将总体目标转化为可执行的子任务。规划的主要作用，是明确任务顺序、前置条件、资源需求和完成标准。

但合理的自然语言计划不一定具有实际可执行性。例如，模型可能提出调用某个不存在的工具，或者要求访问没有权限的文件。因此，任务规划需要与真实执行能力相结合。

任务分解的基本形式 设总体目标为：G 可以将其分解为多个子目标：

$$G \to g_1, g_2, \ldots, g_n \tag{17.32}$$

例如，对于实验报告分析：g_1 = 读取报告，g_2 = 提取实验指标，g_3 = 统一比较条件，g_4 = 计算性能差异，g_5 = 生成分析文件，g_6 = 验证结果，这些子任务共同支持总体目标。但任务分解不仅需要列出步骤，还需要确认每一步的输入和输出。例如，计算性能差异之前，需要获得经过核查的实验指标。

因此，任务之间可能存在明确依赖。

任务依赖图 可以使用有向图表示复杂任务。例如：

$$\mathcal{G} = (V, E) \tag{17.33}$$

其中：

- V 表示任务节点；
- E 表示任务之间的依赖关系。

如果节点 v_i 必须先于节点 v_j 完成，则建立依赖关系：$v_i \to v_j$，

例如，读取三份独立报告可以并行执行。但综合比较必须等待必要数据全部提取完成。如果任务依赖关系不包含循环，可以使用有向无环图（Directed Acyclic Graph，DAG）进行组织。这种方式有助于发现能够并行执行的任务，并避免某些不合理的执行顺序。但实际智能体任务可能具有循环。例如，验证失败后重新执行数据提取。因此，DAG 通常用于描述某个阶段的任务依赖，而完整执行过程仍然可能包含重试和重新规划。

前置条件与完成条件 为了使任务计划具有可执行性，需要明确行动前置条件和预期结果。例如，一个文件写入动作可以定义为：前置条件：目标文件存在，调用者具有写入权限。行动：修改指定字段。预期结果：目标字段更新为指定值。可以将行动表示为：

$$a_i = (\mathrm{Pre}_i, \mathrm{Act}_i, \mathrm{Post}_i) \tag{17.34}$$

其中：

- Pre_i 表示前置条件；
- Act_i 表示执行操作；
- Post_i 表示预期后置条件。

例如，若某个文件不存在，就不能直接执行依赖该文件的读取操作。

因此，执行前需要检查相应条件是否满足。后置条件只是计划中的预期，不等于真实环境已经发生对应变化。执行之后仍然需要验证。

规划中的资源与风险约束 任务规划还需要考虑资源预算。例如：最大模型调用次数。最大工具调用次数。允许的计算时间。可以访问的数据范围。允许修改的资源。对于

具有外部副作用的行动，还需要考虑执行风险。例如，普通检索与删除重要文件具有不同的风险等级。因此，规划可以表示为具有约束的优化问题：

$$\min_{P \in \mathcal{P}} \tag{17.35}$$

C(P) 满足：GoalSatisfied$(P) = 1$ 以及必要的权限和资源约束。其中：

- $\mathcal{P}$ 表示候选计划集合；
- C(P) 表示计划成本。

在开放环境中，任务成功可能具有不确定性，难以在执行前严格保证。

因此，实际规划通常需要结合预期收益、风险控制和执行反馈。对于不能违反的权限约束，应由外部执行系统强制检查，而不是只通过模型的成本函数处理。

计划状态与任务检查点 长任务需要记录当前执行到哪里。例如，系统已经完成三份报告的数据提取，但尚未生成最终文件。如果模型进程意外中断，系统应当能够判断哪些步骤已经完成，哪些步骤需要继续。因此，可以保存结构化任务状态：$S_t = (G, P_t, D_t, E_t)$，其中：

- G 表示总目标；
- P_t 表示当前计划及步骤状态；
- D_t 表示已经确认的中间产物；
- E_t 表示执行记录与必要证据。

这种状态可以通过数据库或文件系统持久化。

保存计划文本本身并不足以恢复任务。例如，某个计划写着“文件已经生成”，但实际文件可能不存在。因此，恢复时仍然需要检查外部资源状态。

17.3.6 执行反馈与错误恢复

执行反馈的基本作用 智能体按照计划执行行动之后，需要判断外部环境是否发生预期变化。例如，网页智能体点击提交按钮之后，需要确认表单是否已经提交。文件智能体执行写入操作之后，需要检查文件内容是否正确。机器人执行抓取动作之后，需要判断物体是否被成功抓住。这些判断依赖执行反馈。可以将一次执行表示为：$r_t = \text{Execute}(a_t, s_t)$，其中：

- a_t 表示行动；
- s_t 表示执行前的环境状态；
- r_t 表示工具或环境返回的信息。

但工具返回信息不一定完整反映环境真实状态。

例如，某个 API 返回“请求已接受”，并不意味着后台任务已经完成。因此，执行反馈通常需要与进一步的环境状态检查结合起来。

执行成功与目标成功 在多步骤任务中，需要区分几个不同层次。首先是调用是否成功。例如，参数格式和访问权限是否满足要求。其次是执行是否成功。例如，工具是否实际完成对应操作。最后是任务是否成功。例如，最终环境是否满足用户要求。这三个层次不一定一致。例如，模型正确调用文件写入工具，但写入了错误数值。此时工具执行可能成功，最终任务却失败。因此，智能体不能只根据工具返回的成功标志结束任务。对于重要任务，应当检查最终结果是否满足明确的完成条件。

重新观察与重新规划 当环境状态发生变化时，智能体可能需要重新观察。例如，一个网页按钮原本位于页面右上角。但页面重新加载之后，布局发生变化。此时，继续使用旧坐标可能导致错误点击。因此，应当获取新的页面状态，并重新确定目标位置。可以将重新规划表示为：

$$\Pi_{t+1} = \mathrm{Replan}(G, h_{t+1}, F_t) \tag{17.36}$$

其中：

- G 表示任务目标；
- h_{t+1} 表示更新后的状态；
- F_t 表示失败或执行反馈；
- P_{t+1} 表示修订后的计划。

不是每一次工具失败都需要完全重新规划。

例如，某个独立文件读取操作短暂超时，可能只需要重试。因此，可以根据错误影响范围选择局部恢复或完整重规划。

幂等性与状态确认 幂等性是工具执行可靠性的重要概念。例如，对同一资源重复设置同一个确定值，可能具有幂等性质。而重复创建订单或发送邮件，则可能产生多次副作用。因此，智能体在进行自动重试时，需要考虑操作语义。例如，可以采用请求标识和状态查询，判断某次操作是否已经执行。但仅生成一个请求标识并不自动保证幂等。实际执行系统需要具有相应的去重和一致性机制。因此，关键操作的重复执行控制应当由程序和业务系统落实，而不能完全依赖模型自然语言判断。

反馈信息的可信性 外部反馈可能包含不可信内容。例如，网页正文、代码仓库注释和工具返回文本，可能包含试图影响智能体行为的指令。如果模型错误地将这些内容视为新的授权要求，就可能偏离用户目标。因此，反馈信息首先应当作为环境数据处理。对于执行权限和任务目标，应当保持独立的控制边界。此外，来自神经模型的反馈可能具有不确定性。例如，视觉检测器可能错误报告某个目标已经消失。因此，重要状态变化可以通过不同证据进行检查。

但多次使用同一个模型判断，并不意味着能够消除系统性错误。因此，应根据任务选择具有实际独立性的验证方式。

失败恢复的效果评价 错误恢复机制需要通过真实执行结果评价。例如，可以记录：失败检测率、恢复成功率、最终任务成功率、平均恢复次数。额外工具调用成本。恢复所需时间。是否造成不必要的外部状态改变。例如，某个智能体加入反思机制之后，可能提高最终任务成功率，但同时增加多轮模型调用。因此，需要同时报告性能收益和执行成本。此外，应区分不同错误类型。例如，重新规划可能有助于解决页面布局变化，但未必能够解决工具权限不足。

因此，恢复机制的结论应当与具体故障条件对应。

17.4 流式观测与持续状态

流式系统需要维护观测的发生时间、到达时间和可用范围。迟到观测可以修正对过去状态的估计，但不能被追记成模型当时已经使用的信息。测试时应按实际到达顺序重放，并保存每次决策的可见上下文。

17.4.1 流式音视频理解

离线理解与流式理解的区别 离线音视频理解通常假设模型能够在推理开始之前访问完整数据。例如，一段长度为五分钟的视频已经录制完成，模型可以在分析过程中访问视频的任意时间位置。因此，模型可以综合前后信息判断某个事件的含义。流式音视频理解（Streaming Audio-Visual Understanding）则需要在输入持续到来的过程中进行处理。例如，用户在视频播放到第 20 秒时提出问题。

系统原则上只能利用这一时刻已经到达且允许使用的信息，不能提前访问第 30 秒的视频内容。可以将截至时间 t 的观测历史表示为：$O_{\leq t} = o_i \leq t$，其中，t_i 表示观测对应的事件时间。

对于因果流式模型，当前回答应当基于可用历史：$Y_t = f_\theta(O_{\leq t}, q_t)$ 真实系统还存在数据传输和处理延迟。因此，实际可用观测可能只是 $O_{\leq t}$ 的一个子集。这一差异是流式理解与离线理解的重要区别。

流式处理与实时处理 流式处理（Streaming Processing）和实时处理（Real-Time Processing）具有联系，但并不完全等同。流式处理主要强调数据以连续形式到达，系统可以逐步处理新数据，而不必等待全部输入完成。实时处理则进一步要求系统在规定的时间约束内完成相应计算和输出。例如，一个系统能够逐帧处理视频，但每帧处理需要两秒。它仍然具有流式处理形式，却可能无法满足实时视频交互要求。因此，评价流式系统时需要区分：

输入是否能够逐步处理。是否需要等待完整序列。单次更新的实际耗时。输出相对于真实事件的延迟。是否能够长期持续运行。这些条件共同决定系统的实际在线处理能力。

Emformer 与分块语音编码 Emformer 是面向低延迟流式语音识别的代表性模型。其研究采用基于记忆的 Transformer 结构，对语音进行分块处理，并利用记忆机制保留

部分历史上下文。例如，连续语音可以划分为：$X = (X_1, X_2, \ldots, X_K)$，其中，$X_k$ 表示第 k 个音频块。模型在处理当前块时，可以访问适当的历史信息：$H_k = F(X_k, M_{k-1})$，

其中，M_{k-1} 表示历史记忆。这种结构有助于降低处理长语音时反复计算历史上下文的成本。但分块处理通常需要在识别质量和延迟之间进行权衡。例如，较大的音频块可以包含更多上下文，却可能增加等待时间。较小的音频块能够更早触发处理，但可能缺少判断当前语音所需的信息。因此，语音分块长度需要结合目标应用确定。

流式视频理解 视频流通常由连续图像帧构成。例如：$V = (I_1, I_2, \ldots, I_T)$，其中，$I_t$ 表示第 t 帧。传统视频语言模型可能先抽取一定数量的帧，再将这些帧一次性提供给模型。但在实时视频中，新的画面持续到达。例如，一个监控系统需要持续观察某个场景。如果每次查询都重新编码已经处理过的全部视频，就可能造成大量重复计算。因此，流式视频理解需要逐步处理新帧，并在必要时保留历史信息。

例如：$S_t = U(S_{t-1}, E_v(I_t))$，其中，$E_v$ 表示视觉特征提取过程。但不同视频片段的变化速度可能不同。例如，静止场景中连续帧之间可能差异较小，而快速动作可能在短时间内发生明显变化。因此，固定帧率采样与固定计算预算不一定适合全部场景。

流式音视频联合理解 真实交互场景通常同时包含视觉和音频。例如，视频会议中，人物说话与画面变化具有时间关系。系统需要判断某段语音与哪个画面对象对应。例如，用户说出：“请看右边这个设备。”模型需要根据语音发生时间，找到对应画面中的目标。因此，音视频联合理解需要保留时间戳和必要的跨模态关联。例如：

$$O = (x_i, m_i, t_i)_{i=1}^{N} \tag{17.37}$$

其中：

- x_i 表示观测内容；
- m_i 表示模态；
- t_i 表示时间。

但音频和视频通常具有不同的采样率和处理粒度。

例如，音频可能按照较短时间块处理，而视频可能按照帧或帧组处理。因此，系统需要根据任务组织两者之间的时间对应关系。

流式处理中的未来信息泄漏 在线系统需要遵守因果信息边界。例如，模型在时间 t 提供回答时，不应使用尚未到达的未来观测。可以将这一要求表示为：$Y_t = f(O_{\leq t}, q_t)$，而不应依赖：$O_{>t}$，但实际评价还需要区分事件时间与处理时间。例如，一帧图像的采集时间为 t，但由于传输延迟，系统可能在更晚时刻才收到它。因此，对于严格在线评价，模型的输入应当限制为回答截止时刻真实可用的数据。

不能仅根据数据文件中的时间戳判断全部观测是否已经可用。这一区分对于真实流式音视频、异步传感器和在线智能体尤其重要。

17.4.2 异步观测与增量式处理

多模态观测的异步性 真实多模态系统通常不以完全同步的方式接收数据。例如，摄像头可能按照固定帧率输出图像。麦克风持续采集声音，并按照音频块传输。工业振动传感器可能以较高频率输出测量。事件相机则根据像素亮度变化异步产生事件。因此，不同模态不仅具有不同采样频率，还可能采用不同的观测触发机制。例如，RGB 摄像头通常按照时间周期采集完整图像。事件相机则在局部亮度变化达到一定条件时产生事件。

因此，异步观测不仅是不同数据到达时间不同，还涉及不同传感器对环境变化的采样方式存在差异。

固定频率采样与事件驱动采样 固定频率采样通常按照预设时间间隔获取观测。例如：$t_k = k\Delta t$，其中，Δt 表示采样周期。这种方式具有规则的数据组织结构，便于批量处理。但当环境变化较慢时，连续观测可能包含大量重复信息。例如，静止摄像头连续拍摄的画面可能几乎相同。事件驱动采样则主要在满足某种触发条件时产生观测。例如，事件相机在像素亮度发生变化时输出事件。

这种方式可以减少对静态内容的重复采集，但也会使数据到达频率随环境变化。因此，固定频率和事件驱动采样具有不同特点。在实际系统中，可以根据任务选择其中一种方式，也可以联合使用。

真正的增量更新与重复窗口计算 在实际工程中，需要区分不同形式的增量处理。例如，一种方法使用固定长度的滑动窗口，每次对窗口内全部输入重新编码。另一种方法缓存已经处理过的历史特征，只编码新观测。还有一种方法进一步通过状态更新算子，将新信息写入已有记忆。这些方法虽然都能够处理连续输入，但计算性质不同。例如：特征缓存不一定具有真正的语义记忆能力。例如，缓存历史视觉 Token 可以避免重新提取视觉特征，却未必形成能够跨时间理解对象状态的环境表示。

因此，应当分别验证计算复用和状态理解能力。

递归更新与计算复杂度 假设处理一次观测的成本为：C_{obs}，如果每次查询都重新处理此前全部 t 个观测，则单次查询的历史处理成本可能随 t 增加。在简化的固定单观测编码成本条件下：

$$C_{\text{recompute}}(t) \propto tC_{\text{obs}} \tag{17.38}$$

如果采用固定成本的增量更新，则每次更新可能近似为：

$$C_{\text{update}}(t) \approx C_U \tag{17.39}$$

其中，C_U 表示单次状态更新成本。但这种对比依赖模型结构和实现方式。例如，如果递归状态本身很大，更新成本也可能较高。此外，某些查询仍然需要访问历史数据或执行复杂检索。因此，增量更新的优势应通过真实运行测量，而不能仅根据公式推断。

异步更新的因果一致性 多模态状态更新需要遵守真实时间顺序。例如，某个时刻的状态不应包含尚未发生的未来信息。如果数据由于网络延迟乱序到达，系统需要决定是否回滚、重放或采用其他一致性处理方式。例如，某条较早的观测在较晚时刻到达。直接将其作为最新观测写入状态，可能覆盖已经更新的当前信息。因此，系统需要明确状态更新使用的是事件时间还是处理时间。此外，还需要处理不同来源的数据是否允许并行写入状态。例如，两个模态同时更新同一对象的位置，可能存在冲突。

因此，持续多模态状态维护不仅是神经网络建模问题，也涉及流式数据处理中的时间与一致性约束。

增量更新的误差积累 递归状态更新可能产生长期误差积累。例如，状态更新模型每一步都引入少量位置偏差。随着多次更新，偏差可能逐渐增大。可以将真实状态与估计状态之间的误差表示为：$e_t = \hat{S}_t - S_t^*$，其中：

- $\hat{S}_t$ 表示模型状态；
- S_t^* 表示参考状态。

但真实环境状态并不总能直接获得。

例如，某些隐含语义和物体属性可能无法通过传感器完整测量。因此，实际评价通常需要使用特定任务中的参考数据或可验证观测。为了限制长期漂移，可以研究：定期重新观测。状态校正。历史信息重新对齐、选择性状态重置、多时间尺度记忆。但这些机制是否有效，需要通过长序列实验验证。

17.4.3 实时状态维护与长时上下文

长上下文与持续状态的区别 长上下文模型能够在一次推理中访问较长的输入序列。例如，模型可以读取较长的会议记录或视频内容。但这并不意味着模型能够在连续运行过程中自动维护有效环境状态。例如，一个具有较长上下文窗口的模型，可能能够读取过去一小时的视频帧。但如果每次查询都需要重新处理这些输入，计算成本仍然可能很高。此外，较长上下文中可能包含大量过时信息。因此，需要区分：长上下文访问能力。流式输入处理能力。历史信息缓存能力。持续环境状态维护能力。

这些能力之间具有联系，但不完全等价。

实时状态的基本含义 实时状态可以理解为系统对当前任务相关环境信息的内部估计。例如，在持续视频监测任务中，状态可能包含：当前对象及其位置、对象运动趋势、最近发生的事件、历史对象关系、重要任务背景。可以表示为：

$$S_t = O_t^{\text{objects}}, R_t^{\text{relations}}, E_t^{\text{events}}, M_t^{\text{history}} \tag{17.40}$$

其中，各项分别表示对象、关系、事件和历史信息。这是一种结构化状态的抽象描述。实际系统可以使用稠密神经特征、Token、对象表或其他形式。此外，状态内容应根据任务选择，不能假设所有应用都需要保存全部信息。

状态更新与对象持续性 实时环境中，对象可能持续存在，但其可见性不断变化。例如，一个人走入房间后短暂被遮挡。当前图像可能看不到该对象。但系统可以根据历史观测判断其可能仍然在附近。因此，需要维护对象的时间连续性。例如，可以将对象状态表示为：

$$z_i(t) = (p_i(t), v_i(t), c_i(t), \gamma_i(t)) \tag{17.41}$$

其中：

- $p_i(t)$ 表示位置；
- $v_i(t)$ 表示运动；
- $c_i(t)$ 表示语义属性；
- $\gamma_i(t)$ 表示状态质量或不确定性相关信息。

但模型不能因为历史中出现过某个对象，就无限期假设它仍然存在。

例如，该对象可能已经离开场景。因此，状态需要根据新观测更新，并适当处理对象消失、遮挡与重新出现等情况。

状态写入、读取与校正 持续状态维护通常包括多个操作。

状态写入负责将新观测转化为内部信息。

状态读取负责根据当前任务获得所需内容。

状态校正负责利用新证据修正已有错误。

状态清理负责移除过期或无用信息。

例如，可以表示为：

$$S_t = \mathrm{Write}(S_{t-1}, O_t) \tag{17.42}$$

$Y_t = \mathrm{Read}(S_t, q_t)$，但写入成功并不意味着状态一定正确。例如，视觉模型可能将两个相似对象关联错误。如果错误身份信息进入长期记忆，后续查询可能持续受到影响。因此，状态维护需要考虑错误写入和长期传播问题。

状态恢复与重新初始化 持续系统可能运行较长时间。例如，某个视频分析服务需要连续工作数小时。如果服务进程中断，系统可能需要恢复此前状态。因此，可以设计状态快照和恢复机制。例如，定期保存必要的状态表示、版本和时间信息。重启之后，系统可以根据快照恢复，并继续处理新观测。但快照恢复不仅涉及数据加载，还涉及时间和因果一致性。例如，恢复后不能重复处理已经写入状态的观测，也不能遗漏尚未处理的数据。

此外，如果历史状态已经失效，系统可能需要重新观测或初始化。因此，状态持久化与状态正确性需要共同验证。

持续状态与跨任务查询 长期运行的多模态系统可能需要支持不同任务。例如，同一个场景状态可以用于：对象定位、事件描述、运动判断、历史查询、异常检测。但不同任

务需要的信息可能不同。例如，运动估计需要较细时间尺度的位置变化。场景问答可能更加强调对象语义。因此，状态是否具有跨任务价值，需要通过实际任务评价。例如，可以训练或构建一个共享状态，然后使用不同查询模块分别完成多个任务。随后，与每个任务独立处理原始输入的方法比较。

多任务共享状态不应仅根据模块是否共用同一个张量判断。更重要的是，这个状态是否真正保留了各任务所需的信息，并在计算成本和任务效果上具有可验证的优势。

17.5 语言条件策略与具身数据

具身部分为进阶阅读。离线示范误差衡量对记录动作的拟合；闭环执行还受到状态分布变化、控制频率和执行器约束影响。输入和动作接口相同，仍需核对机器人形态、单位、坐标和归一化。

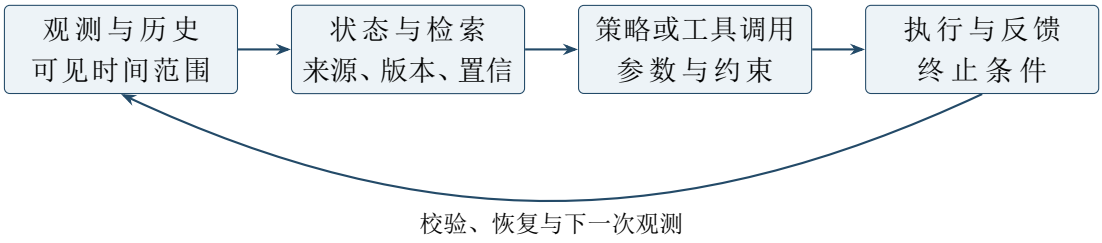


图 17.1 持续交互闭环。数字工具与物理执行共有状态和反馈结构，其动作语义与风险边界不同。

17.5.1 感知与行动的联合建模

从环境理解到行动生成 在传统多模态理解任务中，模型主要根据输入生成语义结果。例如，视觉语言模型可以将图像和文本问题映射为自然语言回答：$Y = f_\theta(I, Q)$，但机器人任务通常要求模型产生实际动作：$a_t = \pi_\theta(o_t, l, s_t)$，其中：

- o_t 表示当前环境观测；
- l 表示自然语言任务指令；
- s_t 表示机器人自身状态；
- a_t 表示机器人动作；
- π_θ 表示策略模型。

对于具有历史依赖的任务，还需要考虑此前观测和动作。

例如：

$$a_t = \pi_\theta(o_{\le t}, a_{<t}, l) \tag{17.43}$$

这一形式强调，机器人当前动作可能依赖此前的执行过程。例如，机械臂是否已经抓住某个物体，会影响下一步应该执行抬升还是重新抓取。因此，具身行动不仅是将当前视觉内容转换为动作指令，还需要建立连续环境状态与任务目标之间的联系。

感知与行动的相互依赖 机器人行动依赖感知，但行动也会改变后续感知。例如，机械臂抓取杯子之前，需要观察杯子的位置。抓取之后，杯子的位置随机械臂运动发生变化。如果杯子在过程中发生滑动，模型还需要通过新观测修正动作。因此，机器人系统具有明显的闭环特征：

$$o_t \rightarrow a_t \rightarrow s_{t+1} \rightarrow o_{t+1} \tag{17.44}$$

其中，s_{t+1} 表示环境在执行动作之后的状态。真实环境状态不一定能够直接观测。例如，机器人可能无法准确知道物体内部的质量分布或接触摩擦系数。因此，策略实际上是在部分可观测环境中，根据有限信息持续决策。这种特征决定了机器人策略不能只依赖静态图像理解能力，还需要具备状态更新和反馈调整能力。

VLA 的基本架构 视觉—语言—动作模型通常包含多个组成部分。例如：视觉编码器、语言任务表示、机器人状态编码。多模态特征融合模块。动作预测模块。可以概念性地表示为：$Z_v = E_v(o_t)$，$Z_l = E_l(l)$，$Z_s = E_s(s_t)$，随后：$H_t = F_\theta(Z_v, Z_l, Z_s)$ 最后：$A_t = D_a(H_t)$，其中，A_t 可以表示一个动作或未来一段动作序列。

但这只是一般性的架构抽象。不同 VLA 模型可能采用统一 Transformer、专门的动作专家网络或其他生成机制。因此，不能仅根据模型具有视觉、语言和动作模块，就判断其内部实现方式完全相同。

感知与行动之间的时间尺度差异 真实机器人中的不同计算环节可能具有不同更新频率。例如：语言任务可能在较长时间内保持不变。视觉观测按照摄像头帧率更新。机器人关节状态可能具有更高更新频率。底层控制器则可能需要更加严格的实时响应。因此，不能简单假设所有模块都必须在相同频率下运行。例如，可以让高层模型以较低频率更新动作目标，而由专门的控制器执行高频动作。但这种方式可能产生信息滞后。例如，机器人已经抓住物体，但高层模型尚未更新任务阶段。

因此，需要设计合理的状态同步和反馈机制。

感知与行动的联合建模挑战 综合来看，感知与行动联合建模主要面临以下问题。视觉语言表示与物理动作具有不同的信息结构。任务语义可能具有较强泛化能力，但实际动作往往与机器人形态有关。联合训练可能导致不同任务之间出现参数竞争或负迁移。此外，机器人状态随动作持续变化，因此模型需要处理长期反馈和状态更新。真实机器人对控制延迟和安全性具有明确要求。

因此，感知与行动联合建模的关键，不只是将动作加入多模态模型输出，而是建立能够被真实机器人持续执行和验证的感知—行动关系。

17.5.2 语言条件策略与动作表示

语言条件策略的机制 语言条件策略（Language-Conditioned Policy）是指根据自然语言任务指令和当前环境信息生成机器人动作的策略。例如，用户提出：“把红色杯子放到左侧托盘中。”系统需要理解：目标对象是红色杯子。目标位置是左侧托盘。任务要

求完成抓取和放置。但语言通常只指定目标，而没有完整说明如何执行。因此，策略需要结合环境状态生成动作：$a_t = \pi_\theta(o_t, s_t, l)$，

这一形式体现了语言条件策略的基本关系。但自然语言可能存在歧义。例如，场景中可能出现多个红色杯子。因此，系统可能需要进一步观察或请求用户澄清，不能无条件选择某个目标。

高层目标与低层动作 机器人任务通常可以划分为不同层次。例如：

高层目标：将杯子放到托盘中。

中层技能：接近、抓取、抬升、移动、释放。

低层动作：关节位置、关节速度或末端位姿控制。

可以将任务关系表示为：

$$l \to g_t \to a_t \tag{17.45}$$

其中，g_t 表示根据语言任务生成的子目标。这种分层方式有助于将任务理解与具体控制分离。但分层结构不一定比直接动作预测更优。例如，对于简单重复操作，端到端策略可能已经足够。而对于复杂长任务，显式子目标可能更加有利于错误定位和执行恢复。因此，需要根据任务复杂度验证不同架构的适用性。

离散动作 Token RT-2[36] 采用离散动作 Token 进行机器人动作建模。其基本思路是，将机器人动作中的数值转换为离散符号，再使用语言模型式的自回归机制进行预测。例如：

$$a_t \to (k_1, k_2, \ldots, k_m) \tag{17.46}$$

其中，k_i 表示相应动作 Token。这种方式能够将动作预测统一到已有的 Token 生成框架中。但动作离散化可能引入量化误差。例如，位置增量被划分为有限区间之后，较小的动作差异可能无法完整表示。此外，逐 Token 自回归生成可能影响动作生成速度。因此，离散 Token 的实现需要在训练便利性、动作分辨率和推理效率之间进行权衡。

连续动作表示 另一种路线是直接生成连续动作。例如：$a_t \in \mathbb{R}^d$ 模型根据当前观测预测连续数值。这种方式适合需要较高控制精度的机器人任务。但机器人行为可能具有多解性。例如，机械臂可以从不同方向接近同一个物体。如果只使用确定性回归，模型可能难以表达多个合理动作模式。因此，可以使用概率分布或生成式动作模型表示不同可能行为。例如：

$$A_{t+H-1} \sim p_\theta(A \mid o_t, s_t, l) \tag{17.47}$$

其中，A 表示未来一段动作序列。

动作分块 动作分块（Action Chunking）是机器人策略中常见的时间建模方式。例如，模型不是只预测当前一个控制步，而是一次预测未来 H 步动作：$a_t = \pi_\theta(o_t, s_t, l)$，

随后执行其中的部分动作。这种方式可以减少高层模型每个控制周期都重新推理的需求。ACT（Action Chunking with Transformers）在双臂操作研究中展示了动作分块的重要作用。其研究说明，动作分块能够帮助模型学习连续操作过程中的局部时序结构。但较长动作块可能降低对突发环境变化的响应能力。例如，目标物体在动作块执行期间发生移动，后续动作可能不再适用。因此，需要根据任务动态性选择动作块长度与重新规划频率。

动作生成中的多时间尺度 机器人任务可能包含不同时间尺度的信息。例如，语言目标可能在数十秒内保持稳定。高层动作阶段可能持续数秒。低层控制则需要更加频繁地更新。因此，可以考虑分层动作生成。例如：

$$g_t = \pi_{\text{high}}(S_t, l) \tag{17.48}$$

$$a_t = \pi_{\text{low}}(o_t, s_t, g_t) \tag{17.49}$$

其中：

- π_{high} 负责高层子目标；
- π_{low} 负责低层动作。

这种结构能够让不同模块按照适当频率运行。

但高层策略与低层控制之间可能存在状态不同步。例如，高层模型决定继续抓取，而低层系统已经检测到抓取失败。因此，需要通过反馈机制修正任务阶段。

17.5.3 具身数据、仿真与真实环境迁移

具身学习的数据需求 多模态基础模型通常可以从大规模互联网图像和文本中学习知识。但机器人不仅需要理解图像和语言，还需要学习动作对环境产生的实际影响。因此，机器人训练数据通常包含连续的观测、状态和动作。例如，可以将一段机器人轨迹表示为：

$$\tau = (o_t, s_t, a_t)_{t=1}^{T} \tag{17.50}$$

其中：

- o_t 表示传感器观测；
- s_t 表示机器人状态；
- a_t 表示执行动作。

对于语言条件任务，还可以加入：l 表示任务指令。

例如：

$$\tau_l = (l, o_1, s_1, a_1, \ldots, o_T, s_T, a_T) \tag{17.51}$$

这种数据可以用于模仿学习、行为克隆和 VLA 模型训练。机器人数据中的动作具有明确物理语义。因此，不同平台之间的数据不能仅根据数值维度直接合并。

机器人数据的获取方式 机器人数据可以通过多种方式获得。例如：人工遥操作、专家示教、预设控制程序、强化学习探索、仿真环境生成。真实机器人自主执行。不同方式具有不同特点。例如，人工遥操作可以提供相对自然的操作示范。但采集成本较高，也可能受到操作人员技能差异影响。仿真能够低成本生成大量轨迹，但可能存在现实差异。因此，机器人数据构建通常需要在规模、多样性、成本和质量之间进行权衡。

模仿学习 模仿学习（Imitation Learning）是机器人策略训练的重要方法。其基本思想是利用专家示范学习从观测到动作的映射。例如，可以通过行为克隆（Behavior Cloning，BC）最小化策略动作与示范动作之间的差异。对于连续动作，可以使用：

$$\mathcal{L}_{\mathrm{BC}} = \mathbb{E}_{(o,a)\sim D}\left[\|\pi_\theta(o) - a\|_2^2\right] \tag{17.52}$$

但这一表达只适用于某些连续动作回归设置。对于离散动作 Token、概率策略或生成式动作模型，训练目标可能不同。例如，RT-2 可以使用动作 Token 预测目标。Diffusion Policy[37] 则使用相应的去噪训练目标。因此，模仿学习是一个较广泛的学习范式，并不对应唯一损失函数。

模仿学习中的分布偏移 模仿学习可能面临训练与执行状态分布不一致的问题。例如，示范数据通常来自专家成功执行的轨迹。但部署时，策略可能因为小误差进入训练数据中较少出现的状态。例如，机械臂抓取位置偏移之后，后续视觉状态可能不同于示范轨迹。如果模型没有见过这种状态，就可能继续生成不合适动作。因此，误差可能在连续执行过程中累积。这也是机器人研究强调闭环测试和恢复能力的原因。

例如，仅在示范数据上获得较低动作预测误差，不意味着机器人能够长时间稳定完成任务。

真实机器人微调 另一种方法是先利用仿真或跨机器人数据预训练模型，再使用目标机器人上的少量真实数据进行微调。例如：

$$\theta_{\mathrm{target}} = \mathrm{FineTune}(\theta_{\mathrm{pretrain}}, D_{\mathrm{target}}) \tag{17.53}$$

其中：

- $\theta_{\mathrm{pretrain}}$ 表示预训练参数；
- D_{target} 表示目标机器人数据。

这种方式可以复用已有模型的部分知识，同时适应目标设备。

但微调可能出现过拟合。例如，模型可能只适应少数固定场景，而在新的对象位置下表现下降。因此，需要将训练场景与测试场景合理分离，并通过真实执行评价泛

化效果。

Sim-to-Real 的评价 为了评价仿真到真实环境的迁移，可以比较：仿真任务成功率。真实机器人任务成功率。不同物体条件下的成功率。不同光照和背景下的表现、控制延迟、碰撞和异常情况。例如，可以定义迁移后的真实任务成功率：

$$\text{SuccessRate}_{\text{real}} = \frac{N_{\text{real success}}}{N_{\text{real trials}}} \tag{17.54}$$

但仅比较仿真成功率与真实成功率的数值差异，还不足以完整解释迁移失败原因。例如，失败可能来自视觉识别，也可能来自接触动力学和执行器误差。因此，最好结合任务轨迹和传感器反馈进行失败分析。

具身智能的研究方向与局限 综合来看，VLA 模型和具身基础模型推动了机器人学习的发展，但仍然存在重要挑战。真实机器人数据采集成本较高，且不同机器人之间存在明显差异。机器人动作涉及连续控制和物理接触，不能直接沿用普通语言生成的全部假设。真实环境具有部分可观测性，机器人需要处理遮挡、状态变化和历史信息过期。此外，长时序任务中的误差累积和失败恢复仍然是重要问题。真实机器人运行需要考虑安全约束、推理延迟和硬件执行能力。

因此，具身智能的发展不仅依赖更大的模型，还依赖高质量数据、合理的状态表示、可靠的控制机制和系统级实验验证。

具身智能的关键，是使多模态模型能够将环境感知和任务语义转化为符合机器人形态与物理约束的行动，并通过持续反馈在真实环境中完成目标。

17.6 世界模型与闭环验证

17.6.1 世界模型与状态预测

动作条件世界模型学习状态如何随行动变化，例如 $p_\theta(s_{t+1} \mid s_t, a_t)$。仅根据历史视频预测未来画面，则可能没有可执行动作接口。两者都能研究动态信息，但支持的决策与验证不同。

潜在状态路线先编码观测，再预测状态并在需要时解码：

$$z_t = E_\theta(o_t), \qquad \widehat{z}_{t+1} = F_\phi(z_t, a_t), \qquad \widehat{o}_{t+1} = D_\psi(\widehat{z}_{t+1}). \tag{17.55}$$

编码器可能需要历史以处理部分可观测性，以上写法只展示基本接口。好的重建不保证保留动作相关状态；用于规划的表示还要接受预测和控制任务的检验。World Models 与 DreamerV3 可作为环境内部模型用于控制的阅读入口。[38, 39]

多步滚动把预测重新用作输入：

$$\widehat{z}_{t+h+1} = F_\phi(\widehat{z}_{t+h}, a_{t+h}). \tag{17.56}$$

它会进入不同于单步真实观测的状态分布。误差可能在接触等临界事件中放大，也可

能受到稳定动力学约束，不能预设其严格单调增长。评价应分别报告真实状态条件下的单步误差、自由滚动误差和使用模型规划后的闭环结果。

改变输入动作后生成不同轨迹，只说明模型对动作条件敏感。若训练数据中动作与未观测因素相关，预测差异仍可能反映相关性。把它用于反事实或因果结论前，需要在可控环境中检验真实干预的后果。

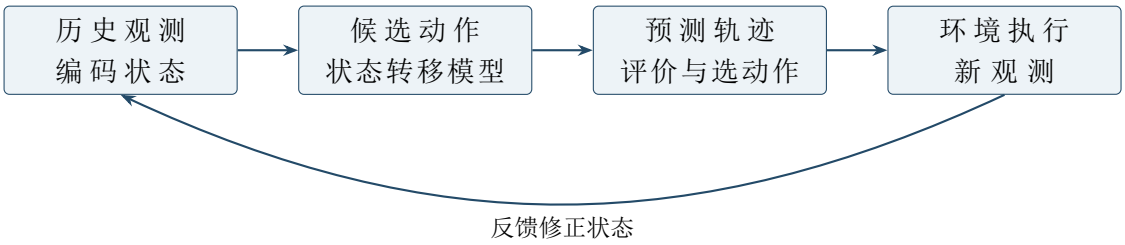


图 17.2 基于环境模型的决策示意。模型内预测与实际环境执行分开评价，闭环反馈用于修正状态。

对生成式世界模型，视觉质量、空间保持、时间连续和动作可控是不同维度。输出多个未来也不等于概率已校准；遗漏低概率而高后果的事件，可能不会明显改变平均误差。

17.6.2 从离线评测到真实环境验证

离线评测的作用与局限 离线评测通常使用已经收集完成的机器人轨迹数据。例如，输入图像、语言任务和机器人状态。模型预测对应动作。然后，将预测结果与示范动作进行比较。例如，可以使用：

$$\mathrm{MAE} = \frac{1}{Nd} \sum_{i=1}^{N} \sum_{j=1}^{d} |\hat{a}_{ij} - a_{ij}| \tag{17.57}$$

其中：

- $\hat{a}_{ij}$ 表示预测动作分量；
- a_{ij} 表示参考动作分量；
- d 表示动作维度。

这种方式能够快速比较模型的动作预测能力。

但同一任务可能存在多种有效动作。例如，机器人可以采用不同路径绕过障碍物。因此，预测动作与示范动作不完全一致，不一定意味着预测结果不可执行。反过来，离线动作误差较小，也不意味着真实执行一定成功。因此，离线评价主要用于模型开发和初步筛选，不能代替闭环任务测试。

动作预测与任务成功的区别 机器人执行具有连续反馈特征。例如，模型预测的第一个动作存在小幅位置偏差。实际执行之后，后续视觉输入可能发生变化。因此，模型进入的环境状态可能与离线示范不同。如果策略不能处理新的状态，错误可能逐步累积。

因此，需要区分：动作预测是否接近示范。执行轨迹是否满足约束。机器人是否真正完成任务。例如，一个模型可能具有较低的动作回归误差，却在真实抓取中频繁失败。因此，机器人策略评价必须包含实际任务结果。

试验次数与统计不确定性 真实机器人实验通常具有较高成本，因此试验次数可能有限。例如，某个模型在 20 次任务尝试中成功 18 次。可以计算经验成功率：0.9

但这一结果并不意味着模型在所有未来任务中的成功概率恰好为 90%。例如，有限试验可能受到对象位置、环境条件和随机初始化影响。因此，需要报告试验次数，并在适当条件下估计置信区间。对于任务具有明显分组结构的实验，还需要考虑不同任务和场景之间的相关性。例如，同一环境中连续执行的多次操作，不一定是完全独立的统计样本。

平均成功率与任务分布 机器人任务难度可能明显不同。例如：固定位置抓取、新物体抓取、复杂接触操作、多步骤整理。需要错误恢复的操作。因此，平均成功率可能掩盖某些困难任务的失败。例如，一个模型在大量简单任务中表现较好，但在少数关键任务中完全失败。此时，整体平均值可能仍然较高。因此，应当同时报告：整体成功率。不同任务类别成功率、不同对象成功率、不同场景成功率。长时序任务成功率。失败原因分布。这样能够更加准确地反映策略能力。

实时执行验证 真实机器人控制具有明确的时间约束。例如，视觉输入可能在某个时刻采集，而模型需要一定时间生成动作。如果环境在此期间发生变化，动作可能已经过期。因此，可以测量：视觉观测时间。模型推理开始时间。动作生成完成时间。实际动作执行时间。下一次反馈到达时间。例如，可以定义动作年龄：

$$t_{\text{execute}}, \qquad t_{\text{observe}} \tag{17.58}$$

其中：

- t_{observe} 表示用于生成动作的观测时间；
- t_{execute} 表示动作开始执行的时间。

这一指标能够反映动作所依赖观测的新鲜程度。

但较低动作年龄不一定保证执行正确。例如，最新观测可能受到遮挡或噪声影响。因此，需要同时评价实时性和动作准确性。

安全边界与人工监督 机器人实际执行可能影响人员、设备和环境。因此，真实实验需要合理设置安全边界。例如：限制机器人工作空间。限制执行器速度和力矩。设置独立的急停机制。监测碰撞风险。控制未经验证的动作范围。在必要情况下允许人工接管。模型在动作生成中考虑安全提示，并不能替代独立的物理安全机制。例如，一个视觉语言模型能够生成“保持安全距离”的解释，并不意味着低层控制一定遵守该要求。因此，系统安全需要通过明确的执行约束和真实测试建立证据。

17.7 执行成本与权限边界

17.7.1 任务成功率与执行成本

智能体可靠性的评价对象 智能体评价首先需要明确所评价的对象。例如，对于一个视觉语言模型，可以通过图像问答准确率评价其是否理解了输入内容。但是，对于一个能够操作网页或文件系统的智能体，仅仅检查模型生成的文字回答通常不够。例如，用户要求：“将这份实验报告中的结果整理为电子表格，并保存到指定目录。”如果模型正确描述了整理步骤，却没有创建目标文件，那么任务并未完成。即使文件已经创建，也还需要检查其内容是否符合用户要求。

因此，智能体评价的基本对象通常是完整的任务执行过程（Episode），而不仅是某一次模型输出。一个 Episode 可以包含：用户任务输入、环境初始状态、模型决策、工具调用、中间反馈以及最终环境状态。例如，可以将任务轨迹表示为：

$$\tau = (o_0, a_0, r_0, o_1, a_1, r_1, \ldots, o_T) \tag{17.59}$$

其中，o_t 表示观测，a_t 表示行动，r_t 表示执行反馈。任务轨迹可以用于分析过程，但最终评价通常还需要独立的任务完成条件。例如，工具返回“保存成功”并不必然意味着文件内容正确。

任务成功率 任务成功率（Task Success Rate，TSR）是智能体评价中的基本指标。假设评价包含 N 个有效任务，其中 N_s 个任务最终满足预先定义的目标条件，则：

$$\text{SuccessRate} = \frac{N_s}{N} \tag{17.60}$$

例如，一个智能体在 100 个有效任务中成功完成 72 个，则任务成功率为 72%。但这里的关键问题是：如何定义“成功”。例如，某个网页任务要求填写并提交表单。如果智能体只完成字段填写，而没有提交，就不能视为完整成功。对于机器人操作，成功抓取物体也不一定意味着最终放置任务已经完成。因此，任务成功条件需要在评价前确定，并尽可能通过实际环境状态进行检查。

OSWorld 是面向真实计算机环境中多模态智能体的重要评价基准。该基准通过可复现的环境初始化和执行结果检查，评价智能体是否真正完成网页、桌面应用和文件操作任务。VisualWebArena 则面向需要视觉信息理解的网页交互任务，要求智能体在实际操作过程中利用页面视觉内容完成目标。这些研究体现了从模型回答评价向环境任务评价的转变。

多步骤任务中的误差累积 多步骤任务通常包含多个相互依赖的操作。例如，一个智能体需要依次完成文档读取、表格提取、数值计算和文件生成。如果其中某一步产生错误，后续操作可能基于错误信息继续执行。为了说明这一问题，可以假设一个任务包含 K 个必要步骤，每一步独立成功的概率均为 p 。则整体成功概率为：$P_{\text{success}} = p^K$，例如：$p = 0.95, \quad K = 10$，则：

$$P_{\text{success}} \approx 0.599 \tag{17.61}$$

即在这一简化假设下，整体成功概率约为 59.9%。真实智能体的步骤通常不满足独立性假设。例如，前一步错误可能增加后续错误的概率；另一方面，反馈与恢复机制也可能修正某些失败。因此，上述公式主要用于说明长任务可靠性可能受到误差累积影响，不能直接用于预测真实系统的成功率。实际评价仍然需要完整的多步骤执行实验。

重复执行的一致性 智能体可能具有一定随机性。例如，相同任务在不同运行中，模型可能选择不同工具或行动顺序。因此，一次成功并不意味着系统能够稳定完成同类任务。τ-bench 研究了工具型智能体在模拟业务环境中的执行表现，并提出利用 $pass^k$ 评价多次运行的一致性。其核心思想是，不仅检查单次运行是否成功，还检查同一个任务在重复运行条件下能否持续成功。

例如，对于每个任务重复执行 k 次，只有全部 k 次都满足目标时，才认为该任务通过对应的一致性条件。可以表示为：

$$\text{TaskSuccess} = \frac{1}{N}\sum_{i=1}^{N}\prod_{j=1}^{k} S_{ij} \tag{17.62}$$

其中：

- $S_{ij} \in 0, 1$ 表示任务 i 在第 j 次运行中是否成功；
- N 为任务数量；
- k 为重复次数。

这一表达适用于每个任务具有 k 次实际重复运行记录的经验统计方式。

$pass^k$，与单次平均成功率不等价。其结果还受到重复次数、模型随机性和执行环境影响。因此，比较不同智能体时，需要保持重复次数和实验条件一致。

任务完成的部分正确性 某些任务不适合只使用二元成功指标。例如，一个机器人需要依次完成多个对象的整理。如果成功整理了五个对象中的四个，虽然总体目标可能没有完全实现，但已经完成了部分工作。类似地，一个文档处理智能体可能正确提取了大部分表格，只在某个复杂图表上失败。因此，可以补充部分完成指标。例如：

$$\text{SubgoalRate} = \frac{N_{\text{completed subgoals}}}{N_{\text{required subgoals}}} \tag{17.63}$$

但不同子任务可能具有不同重要性。例如，某些步骤是最终任务的必要前置条件。因此，简单计算子任务数量比例可能不能准确表示实际任务价值。对于重要应用，应当根据任务语义设计合理的阶段评价和整体成功条件。

单位成功任务成本 为了同时考虑效率与任务完成质量，可以引入单位成功任务成本。例如：

$$C_{\text{success}} = \frac{C_{\text{total}}}{N_{\text{successful}}} \tag{17.64}$$

其中，C_{total} 表示整个评价期间的实际成本。该指标包含成功和失败任务所消耗的资源，有助于反映系统取得有效结果所需的平均代价。例如，某个智能体单次调用成本较低，但经常失败并需要重复执行，最终单位成功任务成本可能较高。如果成功任务数为零，该指标无法直接计算。此外，不同任务的复杂度可能差异很大，因此最好在相同任务分布下比较。例如，单图视觉问答与长视频自动分析，通常不具有相同的资源需求。

延迟与执行预算 智能体任务可能需要较长时间完成。例如，一个多模态智能体需要分析数十份报告，或者反复操作复杂网页。因此，系统通常需要设置执行预算。例如：

$$T_{\text{episode}} \leq T_{\max} \tag{17.65}$$

$$N_{\text{tool}} \leq B_{\text{tool}} \tag{17.66}$$

$$C_{\text{episode}} \leq B_{\text{cost}} \tag{17.67}$$

其中，$T_{\max}$、B_{tool} 和 B_{cost} 分别表示时间、调用次数和成本预算。这些限制可以防止系统在异常情况下无限执行。例如，一个智能体可能因为始终无法定位某个按钮，而反复执行截图和点击操作。如果缺少终止条件，就可能持续消耗资源。预算耗尽不意味着任务成功。因此，系统需要区分正常完成、主动取消、执行失败和预算终止等不同结束状态。

任务可靠性与安全约束的联合评价 单纯提高任务成功率，并不一定意味着智能体更加可靠。例如，一个智能体可能通过访问超出授权范围的文件，提高某些任务的成功率。但这种行为显然不应被视为合理的能力提升。因此，可以定义满足安全约束的任务成功率。例如：

$$\text{SafeSuccessRate} = \frac{N_{\text{success}\wedge\text{compliant}}}{N_{\text{valid}}} \tag{17.68}$$

其中，分子表示同时满足最终目标和预先定义的安全约束的任务数量。安全约束需要具有明确的操作定义。例如，是否访问了非授权文件、是否执行了禁止的写操作，或者是否违反了机器人运动限制。这一指标并不能涵盖全部潜在风险，但能够防止仅根据最终任务成功率掩盖不适当的执行行为。因此，智能体可靠性评价应当同时考虑任务完成与约束遵守。

17.7.2 工具误用与错误传播

工具误用的基本概念 工具误用（Tool Misuse）是指智能体在工具选择、参数生成或实际执行过程中，使用了不适当的工具或产生了不符合目标的操作。例如，一个智能体可能把只需要读取文件的任务，错误地转换为修改文件。也可能使用正确的读取工具，却访问了错误的资源。对于多模态智能体，工具误用还可能来自视觉或音频信息理解错误。例如，模型将屏幕上的“删除”按钮误识别为“保存”按钮。随后执行点击操作，造成非预期状态变化。

因此，工具误用既可能源于模型的任务理解错误，也可能源于输入感知、参数映射或环境状态判断错误。并不是所有工具误用都由恶意攻击造成。例如，普通的视觉定位误差、过期页面状态和错误的参数单位，都可能导致不适当的执行行为。因此，需要区分普通可靠性失败和对抗性攻击。

工具错误的主要来源 工具误用可以发生在多个环节。首先是工具选择阶段。例如，模型选择了功能不匹配的工具。其次是参数生成阶段。例如，模型生成了错误文件路径或目标对象标识。再次是工具执行阶段。例如，外部服务状态变化，使原本正确的操作无法完成。最后是结果解释阶段。例如，工具返回的是“任务已接受”，模型却将其解释为“任务已完成”。这些错误具有不同的恢复方式。例如，参数结构错误可以通过 Schema 检查发现。

而视觉对象误识别可能需要重新读取图像或使用额外视觉证据验证。因此，不能将工具可靠性简单归结为接口返回是否正常。

多模态感知错误如何转化为行动错误 多模态智能体的一个重要风险，是感知错误可能直接影响后续行动。例如，用户要求机器人抓取红色杯子。视觉模型可能错误地将旁边的红色容器识别为目标杯子。如果系统直接根据错误定位生成抓取动作，就可能操作错误对象。可以将这一过程表示为：X

$$\xrightarrow{\text{Perception}} \hat{s} \xrightarrow{\text{Decision}} \tag{17.69}$$

a

$$\xrightarrow{\text{Execution}} \tag{17.70}$$

s’ 其中：

- X 表示多模态输入；
- $\hat{s}$ 表示智能体估计的环境状态；
- a 表示基于估计状态生成的动作；
- s’ 表示执行后的环境状态。

如果 $\hat{s}$ 与真实状态存在明显偏差，后续行动就可能不符合任务目标。

感知误差并不必然造成行动失败。例如，规划器可能利用其他传感器或约束识别

异常。但如果系统缺少独立检查，就可能使感知错误继续传播。

空间与时间错配 多模态工具还需要处理空间坐标和时间戳。例如，视觉模型可能在缩放图像中输出目标边界框。执行工具则需要使用原始屏幕坐标。如果系统没有正确恢复坐标映射，就可能点击错误位置。类似地，视频智能体可能根据历史帧判断对象位置，但实际环境已经发生变化。因此，模型输出的视觉信息可能已经过期。例如，机器人在 t_0 时刻观察到目标位置：p_{t_0}，但在 t_1 时刻执行动作。

如果目标已经移动，则：$p_{t_1} \neq p_{t_0}$ 此时，直接使用旧位置可能导致失败。因此，多模态智能体需要维护观测、推理和执行之间的时间对应关系。对于具有较高实时性要求的任务，应当根据环境变化速度设置状态有效性检查。

错误传播的基本过程 多步骤任务中的错误可能沿执行链路传播。例如：错误 OCR → 错误实验数据 → 错误统计分析 → 错误报告。或者：

错误目标定位 → 错误工具参数 → 错误文件操作 → 错误任务状态。因此，可以将任务表示为具有依赖关系的图：

$$\mathcal{G} = (V, E) \tag{17.71}$$

其中，节点表示任务处理环节，边表示数据或状态依赖。如果上游节点产生错误输出，下游节点可能在未发现错误的情况下继续处理。因此，错误影响范围与任务依赖结构有关。例如，某个错误只影响单个局部子任务，可能较容易恢复。但如果错误结果被多个下游步骤共同使用，就可能造成更大影响。

错误累积与错误放大 需要区分错误累积和错误放大。错误累积主要指多个步骤分别引入误差，使最终结果偏离目标。错误放大则指一个较小的上游错误，经后续决策或工具执行后产生更加严重的结果。例如，模型错误读取了一个实验指标，可能导致最终排序错误。而在自动化文件操作中，模型将目标目录理解错误，可能进一步修改多个不相关文件。因此，工具型智能体的风险不仅取决于模型错误发生的概率，还取决于错误进入外部执行系统后可能产生的影响。可以概念性地将风险表示为：

$$\mathrm{Risk}(a) = P(\text{failure} \mid a) \cdot I(\text{failure} \mid a) \tag{17.72}$$

其中：

- P 表示失败概率；
- I 表示失败影响程度。

这一表达是简化的风险量化方式。

实际风险评价还可能需要考虑影响范围、可恢复性和不同损害类型，不能仅根据一个乘积完成全部安全判断。

错误传播的系统性评价 为了研究错误如何影响任务，可以在受控环境中引入特定扰动。例如：将某个 OCR 数值替换为错误值。使视觉工具返回轻微偏移的边界框。延迟

视频状态更新。使工具返回暂时性错误。改变某个文件的实际版本。随后观察智能体是否能够检测、限制和恢复错误。可以分别记录：错误检测率。最终任务成功率。错误影响的下游步骤数量、恢复成功率、额外执行成本。故障注入实验应当保持其他因素尽可能一致，避免同时改变多个条件造成因果判断困难。

例如，如果同时改变 OCR 质量和规划器结构，就可能难以确定性能变化来自哪个环节。因此，多模态智能体的错误传播研究应采用分层和单因素验证方式。

17.7.3 权限控制与安全边界

智能体安全边界的基本含义 当智能体能够调用外部工具时，需要明确其可以访问什么、能够修改什么，以及哪些操作必须获得进一步授权。例如，一个文档分析智能体可能只需要读取指定目录中的报告。如果同时授予它整个文件系统的删除权限，就超出了完成任务的必要范围。类似地，一个网页智能体可能只需要查询订单状态，但不应因此自动获得修改账户支付信息的权限。因此，智能体安全边界主要用于限制其行动范围，防止模型错误或不可信输入造成不适当的外部操作。

安全边界不应仅通过自然语言提示实现。例如，仅告诉模型“不要删除文件”，并不能构成可靠的文件系统访问控制。真正的限制通常需要由工具执行器、操作系统和业务服务中的授权机制落实。

最小权限原则 最小权限原则（Principle of Least Privilege）是智能体工具安全的重要基础。其基本思想是，智能体只应获得完成当前授权任务所需的最少权限。例如，一个只需要读取文件的任务，可以使用只读工具。一个需要生成报告的任务，可以只允许写入指定输出目录。对于可能删除或覆盖重要数据的操作，则应采用更加严格的执行条件。

OWASP 的 AI Agent Security 和 MCP Security 指南均强调最小权限、工具隔离、独立授权和敏感操作确认等措施。因此，工具能力不仅需要有明确的功能描述，还需要受到实际访问范围限制。

身份认证与操作授权 身份认证（Authentication）与操作授权（Authorization）具有不同作用。身份认证用于确认调用者或主体身份。操作授权则用于判断该主体是否可以对某个资源执行特定操作。例如，用户已经登录某个系统，并不意味着他能够访问所有私人文件。因此，智能体使用工具时，应根据当前用户和任务授权范围进行权限检查。例如：$\mathrm{Allow}(u, a, r, c)$，其中：

- u 表示调用主体；
- a 表示请求执行的操作；
- r 表示目标资源；
- c 表示当前上下文条件。

只有满足相应规则时，执行器才能允许操作。

授权判断应由可信程序或服务执行，而不应完全依赖模型自身生成的判断结果。

物理智能体的安全边界 对于机器人和其他物理智能体，安全边界还涉及真实执行器。例如，机械臂需要遵守运动范围、速度和力限制。移动机器人需要考虑障碍物、人员和环境约束。因此，不能只依靠语言模型判断某个动作是否安全。例如，模型生成了“向前移动一米”的行动计划。实际控制系统仍然需要根据传感器和运动控制约束判断该动作是否可以执行。对于具有安全风险的机器人系统，应根据实际应用采用适当的独立安全控制机制。例如，运动限制、碰撞检测和紧急停止等。

模型推理通常具有不确定的执行时间，而部分底层控制需要满足严格实时性要求。因此，高层语言规划和低层安全控制通常需要保持清晰的职责边界。

智能体安全的研究基准 AgentHarm 是面向具有工具能力的语言模型智能体的安全评价基准。该研究关注模型在面对不适当任务要求时，是否能够保持必要的安全边界。与只评价聊天模型是否生成不当内容不同，AgentHarm 进一步关注智能体能否通过多步骤工具调用执行有害任务。AgentDojo 则主要研究不可信工具返回内容中的提示注入问题。VPI-Bench 进一步将攻击入口扩展到视觉界面。

这些研究共同说明，智能体安全性评价需要超越单次文本回答。例如，模型可能在文字回答中没有表现出明显问题，却在后续工具调用中执行了不适当操作。因此，应当检查实际行动轨迹和环境状态变化。

安全控制的系统性评价 为了判断安全机制是否有效，需要在正常任务与风险任务中共同评价。例如，可以设置：正常任务集合。包含错误参数的任务。含不可信工具结果的任务。需要访问受保护资源的任务。包含高风险操作的任务。随后评价：正常任务成功率。权限违规阻断率。未经授权操作发生率。安全约束满足率。错误拒绝合法任务的比例。安全控制可能引入额外成本和误拒绝。例如，一个机制可能阻断了更多高风险操作，但也使部分正常任务无法完成。

因此，安全评价需要同时分析风险降低与任务可用性的变化。智能体的自主行动能力必须建立在明确的授权范围之内。模型可以提出行动建议，但真正决定外部操作是否允许执行的，应当是具有独立规则和权限约束的执行系统。

习题与讨论

1. 为一个带检索与计算器的文档助手画出闭环，标出模型、执行器和终止判据的职责。
2. 一个流式观测在事件发生后才到达。设计同时保存事件时间和到达时间的状态记录，解释在线评价时何时可见。
3. 世界模型单步预测误差低，规划却失败。提出表征缺失、滚动分布变化和目标函数不当三种诊断。
4. 离线动作预测改进后，说明还需要哪些闭环协议才能主张任务完成能力提高。

文献导读 ReAct 用于分析推理与行动的交替，RT-2 和 OpenVLA 用于核对语言条件策略的动作接口，World Models 与 DreamerV3 用于理解模型内预测与实际执行的关

系。[30, 36, 40, 38, 39]

第十八章　综合案例与研究实践

综合实践应能把前面的方法连接起来，并留下可复查的证据。以下三项案例是可按资源缩放的作业设计，不报告未经运行的效果。每项都要求先建立简单基线，再改变一个因素，最后解释失败；模型规模不作为成绩依据。

学习目标　将需求转为可检验任务；保持数据与协议隔离；完成基线、消融、成本与失败分析；写出证据支持范围内的研究结论。

18.1　案例一：多模态文档助手

实践：多模态文档助手

任务与数据。选取具有使用许可的文档，保存页面图像、文本或 OCR、页码和区域坐标。按原始文档划分集合，避免同一文档不同页跨训练与测试。问题覆盖单区域读取、跨区域关联和图表计算，答案附可核验来源。

基线。分别建立纯文本检索回答、页面图像直接回答、检索加页面证据三种路线。若使用外部 OCR，在各路线中明确其可见性和成本；不能把 OCR 输出算作模型自行识别。

消融。固定生成模型与提示，单独改变检索粒度、切块或视觉预算。测试错配页面、缺失证据及不可回答问题。记录答案、引用页面与区域，分别检查值正确和来源正确。

交付。样本与划分清单、逐题输出、评价脚本、成本记录和失败分类。结论应回答哪类问题因何种证据改善，而不只给一个总分。

18.1.1　文档理解与知识抽取

智能文档处理的基本概念　智能文档处理（Intelligent Document Processing，IDP）是指利用 OCR、版面分析、机器学习和多模态大模型等技术，从文档中提取、理解和组织信息，并将结果用于检索、分析或实际业务流程。例如，企业可能需要处理：发票、合同、财务报表、技术说明书、实验报告、申请表、扫描档案。这些文档通常不仅包含文本，还具有表格、印章、图片和复杂的排版结构。因此，文档处理不仅是文字识别问题。

例如，一份合同中的金额可能出现在多个位置。系统不仅需要识别金额，还需要判断它属于哪个条款、什么币种以及对应何种业务含义。因此，智能文档理解通常需要联合利用文字内容、视觉布局和语义关系。

文档理解的典型流程 一个典型智能文档处理系统可以包含以下阶段：

文档输入 → 文件解析 → 版面分析 → 文字和视觉内容提取 → 结构化理解 → 字段校验 → 结果输出。

例如，对于一份扫描发票，系统首先读取页面图像。随后，识别文本、表格和其他版面区域。接着，根据字段之间的关系提取发票号码、日期和金额。检查结果是否符合业务要求。但不同任务可能不需要全部阶段。例如，纯文本型 PDF 可能能够直接提取文字和部分表格信息。扫描型文档则可能需要 OCR。一些视觉文档模型也可以直接从页面图像生成目标结构，不必显式输出完整 OCR 结果。因此，实际系统应根据文件类型选择处理路径。

OCR 与视觉文档理解 OCR（Optical Character Recognition）主要将图像中的文字转换为机器可处理的文本。例如，一张发票扫描图像可能包含：“发票号码：12345678”。OCR 系统需要识别相应字符。但文档理解还需要判断这些字符代表什么。例如，识别出字符串并不意味着已经确认它属于发票号码字段。因此，OCR 与文档语义理解具有不同功能。对于典型文档处理流程，可以表示为：

$$I \xrightarrow{\text{OCR}} T \xrightarrow{\text{Understanding}} Y \tag{18.1}$$

其中：

- I 表示文档图像；
- T 表示识别得到的文字；
- Y 表示目标结构化信息。

OCR 输出可能存在错误。

例如，数字 0 与字母 O 可能被混淆。这些错误可能继续影响字段提取。因此，传统 OCR 流水线需要考虑误差传播。

结构化知识抽取 文档理解的一个重要目标，是将非结构化内容转换为结构化信息。例如，从一份发票中提取：发票号码、日期、供应商、金额、币种。可以将输出表示为：

$$Y = (k_i, v_i)_{i=1}^{M} \tag{18.2}$$

其中：

- k_i 表示字段名称；
- v_i 表示字段值。

例如：

```
{
  "invoice_number": "INV-2026-001",
  "date": "2026-10-01",
  "currency": "CNY",
```

```
  "total_amount": 1280.50
}
```

这一示例仅用于说明结构化输出形式。

实际业务系统还需要定义字段类型、缺失值处理和数据格式。例如，日期可以采用统一标准格式。金额字段应当明确币种和数值含义。模型生成合法 JSON，并不意味着提取内容一定正确。因此，结构化输出还需要结合原始证据进行验证。

文档图表的数值理解 图表中的知识可能以图形形式表达。例如，折线图通过曲线展示实验结果随时间变化的情况。柱状图通过柱体高度比较不同类别。系统可以利用视觉语言模型解释主要趋势。但如果用户要求读取精确数值，就可能需要更加严格的数据提取方法。例如，原始表格通常比图像估计更适合提供精确结果。如果无法获得原始数据，可以结合坐标轴识别、曲线定位和图表解析方法进行估计。图像中无法精确确定的数值，不应被无条件输出为精确测量结果。

因此，应根据实际证据区分直接读取、计算结果和视觉估计。

跨页文档理解 实际业务文档经常包含多个页面。例如，一份合同可能在第一页说明交易主体，在后续页面规定付款条件和违约责任。如果模型只处理单页，就可能无法建立跨页关联。因此，多页文档理解需要解决：跨页信息检索、实体与字段对应、长上下文组织、章节和表格关系。例如，MP-DocVQA 及相关多页文档问答研究，使模型能够在多个页面中寻找回答所需的信息。这种任务不仅需要理解单个页面，还需要判断答案可能出现在哪一页。

因此，可以采用页面检索与局部文档理解相结合的方式。例如，先确定相关页面，再对页面中的具体字段进行精细提取。检索到正确页面并不意味着最终答案一定正确。对于精确字段，还需要进一步定位和核查。

智能文档处理的主要错误来源 智能文档系统可能发生不同类型的错误。例如：图像模糊造成文字误读。页面旋转导致阅读顺序错误。复杂表格造成行列关联错误。多个相似字段造成语义混淆。跨页关系没有正确建立。模型生成了文档中不存在的信息。因此，文档处理系统需要根据具体任务设置验证机制。例如，对于金额、日期和产品型号等重要字段，可以要求提供原始页面和位置证据。对于可通过程序检查的字段，可以执行格式和逻辑校验。但字段格式合法并不意味着内容正确。

例如，一个日期可以符合 YYYY-MM-DD 格式，却不一定是文档中的生效日期。因此，文档知识抽取需要同时评价字符识别、语义关联和业务正确性。

18.1.2 应用评测与业务流程集成

模型基准与应用效果的区别 多模态大模型通常通过公开基准评价其视觉理解能力。例如：VQA。DocVQA。ChartQA。OCRBench。MMMU[29]。这些基准能够反映模型在不同任务上的能力，但不能直接等同于业务系统的实际效果。例如，某个模型在 DocVQA 上取得较好的回答成绩，但在企业实际发票处理中，仍然可能受到以下因素影响：扫

描质量不稳定、文件格式复杂、业务字段不一致、部分资料缺失。输出格式要求严格。

因此，实际应用需要进一步开展针对目标业务的端到端评价。

视觉助手的常用评价 对于通用视觉助手，可以使用不同类型的公开数据集。例如，VQA 评价一般图像问答能力。MMMU 涉及多学科视觉理解和推理。MathVista 关注视觉环境中的数学推理。ChartQA 主要关注图表阅读与逻辑运算。这些基准覆盖不同能力。例如，模型可能在普通对象识别中表现较好，却在复杂图表计算中出现错误。因此，视觉助手的评价应当根据实际应用选择对应任务集合。例如，如果应用主要处理设备照片，就需要增加对象属性和异常识别测试。

如果应用主要用于财务图表，则应更加关注数值和逻辑推理。因此，不能仅依据一个综合基准的排名选择全部应用模型。

字符与字段准确率 对于 OCR 和结构化抽取，可以使用字符错误率（Character Error Rate，CER）。例如：

$$\mathrm{WER} = \frac{S + D + I}{N} \tag{18.3}$$

其中：

- S 表示字符替换数量；
- D 表示字符删除数量；
- I 表示字符插入数量；
- N 表示参考文本的字符数量。

CER 越低，通常表示文字识别错误越少。

但字符识别正确不一定意味着字段抽取正确。例如，某个文档中存在多个日期。模型可以准确识别所有日期，却把“生效日期”误填为“签订日期”。因此，还需要使用字段级准确率和实体关系指标。例如，可以定义字段精确匹配率：

$$\mathrm{FieldAccuracy} = \frac{N_{\text{correct fields}}}{N_{\text{evaluated fields}}} \tag{18.4}$$

字段正确性的判定应考虑数据规范化。例如，金额 1280.50 与 1,280.50 可以在明确规则下视为同一数值。但不同币种或单位不能被简单忽略。

端到端业务成功率 一个完整业务系统通常包含多个处理阶段。例如：用户上传文件、系统解析文档、模型提取字段、程序检查数据。业务系统保存结果。最后返回执行状态。如果其中任意必要环节失败，任务就可能无法完成。因此，可以使用端到端业务成功率：

$$\mathrm{TaskSuccess} = \frac{N_{\text{successfully completed}}}{N_{\text{valid business tasks}}} \tag{18.5}$$

其中，成功条件需要根据实际业务目标预先定义。例如，发票处理任务可能要求：

字段提取正确。金额校验通过。数据成功写入指定系统。没有发生未经授权的修改。因此，业务成功率通常比单个模型的文本识别准确率更能反映系统实际价值。

业务性能与成本 企业应用还需要考虑运行效率。例如，某个系统能够准确处理文档，但单份文件需要较长时间和较高调用费用。如果企业每天需要处理大量文件，就可能产生明显运营成本。因此，可以记录：单份文档处理延迟。单位页面计算成本。GPU 或 API 费用、人工审核比例、最终任务成功率。例如，可以定义单位成功业务任务成本：

$$C_{\text{success}} = \frac{C_{\text{total}}}{N_{\text{successful tasks}}} \tag{18.6}$$

该指标能够将模型处理和失败重试成本共同纳入考虑。但如果不同任务复杂度差异较大，应当进行分组统计。例如，单页发票与百页技术合同通常具有不同处理成本。因此，不能仅使用一个平均数描述完整业务负载。

应用价值与技术边界 通用视觉助手和智能文档系统可以提高信息处理的灵活性，并降低部分非结构化资料的使用门槛。但这些系统的实际价值，仍然取决于任务正确率、证据完整性和业务流程可靠性。例如，模型能够生成流畅的报告，并不意味着其中所有实验数值都正确。模型能够输出结构化字段，也不意味着字段具有正确业务含义。检索系统能够找到相关页面，也不意味着最终结论得到了充分支持。因此，需要将模型生成能力与外部验证机制结合起来。

对于具有重要业务影响的操作，还需要设置权限控制和必要的人工监督。

通用视觉助手与智能文档应用的核心，是将视觉理解能力转化为可交互、可检索、可验证和可集成的信息处理能力，使多模态模型能够在明确的业务约束下提供实际价值。

18.2 案例二：持续音视频事件理解

实践：持续音视频事件理解

任务与可见性。按物理时间记录观测，给出事件、查询时间和可用于该次回答的窗口。离线使用完整片段与在线使用历史前缀须分开评价。对原视频分组划分数据，保留音视频偏移与采样配置。

基线。比较逐帧或分段独立回答、固定窗口历史、带状态摘要的增量处理。固定编码器与总预算，分别测量事件时间定位、答案正确率和处理延迟。不能让摘要路线看到比基线更晚的帧。

消融。改变帧采样、音视频相对偏移和历史长度。设置需要视觉、需要声音以及需要两者关联的问题，检验增益来自哪种证据。对迟到观测分别记录当时回答和事后修正，不把后者计入前者。

交付。按到达时间重放的记录、每次决策可见上下文、分组指标与错位曲线。摘

要遗漏和采样遗漏应列为不同故障。

18.2.1 实时语音与音视频交互

从文字交互到实时多模态交互 传统语言模型交互主要采用文本形式。例如，用户输入问题，模型生成文字回答。但真实人机交流通常包含语音、图像和其他环境信息。例如，用户可以一边展示某个设备，一边询问：“这个按钮有什么作用？”系统需要同时理解语音中的指代词和摄像头画面中的对象。因此，实时多模态交互要求模型在时间上联合处理多种输入，而不是只对单张图片和一段文字进行离线推理。这种交互方式涉及：连续语音感知、视觉输入更新、说话轮次判断、跨模态指代、低延迟响应。

自然语音生成。执行状态反馈。这些能力共同影响用户体验。

级联式语音交互 较早的语音助手通常采用级联式处理架构。例如：

语音输入 → 自动语音识别（ASR）→ 语言模型 → 文本回答 → 语音合成（TTS）。

可以表示为：

$$X_{\text{text}} = \text{ASR}(A_{\text{user}}) \tag{18.7}$$

$$\text{LLM}(T_{\text{input}}) \tag{18.8}$$

$$A_{\text{out}} = \text{TTS}(Y_{\text{text}}) \tag{18.9}$$

这种设计具有模块化优势。例如，ASR 和 TTS 可以分别优化，也可以独立替换。但它可能存在信息损失。例如，语音中的停顿、语调、背景声音和其他非语言信息，在转换为普通文本后可能无法完整保留。此外，多个处理模块之间还可能引入额外延迟。因此，级联式方法适合部分任务，但并不覆盖所有自然语音交互需求。

实时交互中的时间同步 实时音视频交互需要处理不同模态的时间关系。例如，用户说出：“请看这个零件。”同时将摄像头对准某个对象。如果系统将语音和视频错误对应，就可能分析错误目标。因此，可以将多模态输入表示为带有时间戳的观测序列：

$$O = (x_i, m_i, t_i)_{i=1}^{N} \tag{18.10}$$

其中：

- x_i 表示观测内容；
- m_i 表示模态类型；
- t_i 表示时间戳。

模型需要根据任务确定哪些观测之间存在关联。

例如，用户说出“这个”时，对应的视觉对象可能出现在某一时间范围内。因此，实时多模态交互不能简单地将全部音频转写文本和视频帧无差别拼接。对于连续交互，时间对齐是理解指代和事件的重要基础。

流式输入与流式输出 实时系统通常无法等待完整语音或视频结束之后才开始处理。例如，用户可能进行一段较长的口头描述。如果系统必须等待全部输入完成，交互延迟可能较高。因此，可以采用流式处理。例如，音频和视频输入被划分为连续数据块：$X = X_1, X_2, \ldots, X_K$ 模型随着新数据块到达，逐步更新内部状态。输出也可以采用流式方式生成。例如，语音模型可以先生成前一段音频，再继续生成后续内容。这种设计能够改善用户感知到的响应速度。

但流式输出可能出现前后修正问题。例如，模型在完整理解用户问题之前就开始回答，可能导致后续内容与前面输出不一致。因此，系统需要根据任务选择合适的响应时机。

端到端延迟 实时交互的延迟通常由多个部分构成。例如：语音采集和传输、语音编码、多模态模型推理、语音生成、音频传输和播放。可以概念性地表示为：

$$T_{\text{total}} = T_{\text{input}} + T_{\text{inference}} + T_{\text{generation}} + T_{\text{network}} \tag{18.11}$$

这一表达适用于简化的串行过程。实际流式系统中的多个环节可能发生重叠，因此端到端延迟需要根据真实时间测量。此外，还需要区分：首个输出音频块延迟。完整回答生成时间。用户话轮结束到系统响应的时间。交互中断后的恢复时间。例如，一个系统可以很快开始播放语音，却需要较长时间才能完成全部回答。因此，单一平均延迟并不能完整描述实时交互体验。

无障碍与辅助交互 多模态大模型可以为无障碍交互提供新的技术手段。例如：通过语音操作部分数字服务。将图片内容转换为文字描述。为视频生成字幕。将文本转换为语音。为复杂界面提供自然语言说明。这些功能有助于改善部分用户使用数字系统的便利性。但不同用户具有不同需求。例如，视觉障碍用户可能更加依赖准确的空间描述。听力障碍用户可能更加关注字幕的实时性和完整性。

因此，系统应根据目标用户开展可用性测试，而不能仅根据模型具有某项功能，就认定其已经满足无障碍应用要求。

18.3 案例三：语言条件具身操作

实践：语言条件具身操作

任务与资源。允许采用离线数据或仿真。定义语言指令、视觉与本体状态、动作空间、控制频率、成功条件和终止条件。只有离线数据时，交付结论限定为动作拟合及诊断，不能宣称完成闭环任务。

基线。以不含语言的策略、语言条件策略和加入历史状态的策略形成比较。应固

定机器人形态、观测与动作归一化、示范数量和训练预算；若其中一项改变，应单独报告。

消融与验证。按对象、指令、初始状态或场景划分泛化。仿真闭环使用统一初始状态，记录成功、失败、碰撞、动作数和恢复。世界模型加入后，分开评价单步预测、自由滚动和实际规划收益。

交付。接口说明、动作可视化、失败轨迹和可重放配置。仿真结果不直接作为真实机器人结论。

18.3.1 语言指令到动作执行

语言指令与机器人行动之间的差异 自然语言通常描述任务目标，而机器人执行需要具体动作。例如，用户说：“请把桌面上的红色杯子放进盒子里。”这条指令包含目标对象和最终位置，但没有指定完整动作过程。机器人可能需要：识别杯子和盒子。估计目标空间位置、选择抓取姿态、移动机械臂、闭合夹爪、抬起杯子、移动至盒子上方、释放杯子。因此，从语言到动作执行，需要将高层语义目标转化为可执行的运动行为。

任务规划与运动规划 机器人规划通常涉及不同层次。

任务规划主要决定需要完成哪些子任务。

例如，先打开柜门，再取出物体。

运动规划主要决定机器人如何从当前状态移动到目标状态。

例如，机械臂需要沿着什么路径运动，才能避开障碍物并到达抓取位置。因此，可以将语言指令到动作执行的过程概括为：

$$l \to G \to P \to A \tag{18.12}$$

其中：

- l 表示自然语言指令；
- G 表示任务目标；
- P 表示任务或运动计划；
- A 表示实际动作序列。

不是所有 VLA 模型都显式采用这一分层结构。

部分端到端模型可以直接根据语言和观测生成动作。但从系统功能角度，仍然需要处理任务语义与物理动作之间的对应关系。

RT-1：**语言条件下的机器人策略** RT-1 采用 Transformer 架构，将视觉观测和语言任务映射为机器人动作。其研究使用大规模真实机器人操作数据，训练能够执行不同任务的策略。例如，模型可以根据任务语言和摄像头图像，预测对应动作。与传统单任务策略相比，RT-1 更加强调多任务训练和一定程度的泛化。但模型能够执行多个训练相关任务，并不意味着它对全部未知任务具有可靠的零样本能力。此外，RT-1 的动作

表示和训练数据与具体机器人平台有关。

因此，将模型迁移到其他设备时，需要考虑机器人形态和控制接口的差异。

连续动作与多模态行为分布 同一任务可能具有多种合理动作路径。例如，机器人可以从不同方向抓住杯子。也可以绕过不同障碍物到达目标位置。因此，动作预测不一定具有唯一正确答案。这也是扩散策略和流匹配策略具有研究价值的原因之一。例如，Diffusion Policy[37] 能够对具有多个可能行为模式的动作分布进行建模。π_0 则进一步研究在通用 VLA 框架中利用流匹配生成连续动作。

但生成多个可能动作，并不意味着这些动作都满足实际安全和碰撞约束。因此，动作生成结果仍然可能需要经过控制层检查和执行约束。

语言条件的跨任务泛化 语言指令能够表达不同任务。例如：“拿起红色杯子。”“拿起最大的杯子。”“把杯子放到盒子左侧。”这些任务可能共享部分操作技能。因此，语言条件策略可以尝试学习不同任务之间的共性。例如，抓取动作可能在多个任务中复用。不同任务主要通过目标对象和放置位置加以区分。但语言泛化与物理泛化不是同一概念。例如，模型可以理解从未见过的自然语言表达，却不一定能够抓取训练中未出现过的复杂对象。因此，VLA 泛化评价需要区分：新语言表达。

新对象、新场景、新任务组合、新机器人平台。这些条件具有不同难度。

执行接口与控制约束 机器人最终需要通过控制器执行动作。例如，VLA 模型输出末端位姿增量。控制系统需要将其转换为具体关节运动。这一过程中可能涉及：逆运动学、轨迹插值、关节速度限制、碰撞检测、力矩约束、执行器状态检查。因此，模型输出需要经过适当的动作适配和安全检查。例如：

$$\tilde{a}_t = \Pi_{\mathcal{A}_{\text{valid}}}(a_t) \tag{18.13}$$

其中，Π 表示将候选动作映射到满足规定约束的动作集合的一种概念性操作。简单的数值裁剪不一定能够保证完整的运动安全。例如，一个关节角度处于允许范围内，并不意味着机器人整体没有发生碰撞。因此，真实机器人需要根据具体机械结构和任务设置执行控制机制。

18.3.2 感知、规划与行动闭环

具身智能的闭环特征 机器人执行任务时，其行动会改变环境。例如，机械臂抓住杯子之后，杯子的位置随之变化。因此，系统不能只根据任务开始时的观测完成全部动作。它需要在执行过程中持续获取新的环境信息。例如：

$$o_t \to a_t \to s_{t+1} \to o_{t+1} \tag{18.14}$$

其中，行动 a_t 改变环境状态，随后系统获得新的观测。这种持续循环构成感知—行动闭环。闭环并不意味着每个控制周期都需要调用大型语言模型。实际系统通常具有不同时间尺度的决策和控制模块。

高层规划与低层控制 机器人任务可能具有多个时间尺度。例如，用户要求："整理桌面。"高层规划需要判断哪些物体需要移动，以及移动到什么位置。低层控制则负责执行每次抓取和放置。因此，可以采用分层结构：

$$G \xrightarrow{\text{Planner}} g_t \xrightarrow{\text{Controller}} a_t \tag{18.15}$$

其中：

- G 表示总体任务；
- g_t 表示当前子目标；
- a_t 表示具体动作。

例如，高层规划可以以较低频率更新。

低层控制则根据实际设备要求进行高频执行。这种结构有助于将语义推理与连续运动控制分离。但不同机器人系统的层级划分可能不同。例如，端到端 VLA 也可以直接输出动作，而不显式生成自然语言子目标。

开环执行与闭环执行 开环执行通常在一段时间内按照预定动作序列运行，而不根据新的观测修正该序列。例如：$A = (a_1, a_2, \ldots, a_T)$ 机器人执行完整序列后再检查结果。闭环执行则根据新的观测不断调整动作。例如：$a_t = \pi(o_t, s_t, G)$ 通常情况下，闭环方式更有利于应对环境变化。例如，抓取过程中物体发生轻微移动，系统可以根据反馈调整。但闭环执行也可能受到感知噪声和反馈延迟影响。

例如，如果每次视觉观测都存在较大误差，频繁调整动作也可能产生不稳定行为。因此，闭环控制需要合理处理观测噪声和执行时延。

滚动时域控制 滚动时域控制（Receding Horizon Control）是机器人规划与控制中的重要思想。例如，系统在时刻 t 预测未来 H 步动作：A_{t+H-1}，但只执行其中的前 h 步，其中：$h \leq H$ 执行之后，根据新观测重新规划。这样可以同时利用较长时域的动作预测和较短周期的反馈更新。Diffusion Policy 等方法采用了与滚动时域控制相关的执行思想。

但如果模型推理时间较长，新动作可能无法及时生成。因此，需要协调推理时延、动作执行频率和环境变化速度。

推理延迟与控制频率 机器人控制通常具有明确的时间要求。例如，某个控制器每隔：Δt 秒更新一次动作。控制频率为：

$$f = \frac{1}{\Delta t} \tag{18.16}$$

如果模型推理时间超过控制周期，就可能无法直接在每个周期同步生成动作。因此，可以采用不同策略。例如：降低高层策略更新频率。一次生成多个动作。通过异步推理更新动作块。使用专门的低层控制器执行高频动作。但这些方法都可能改变动作反馈和控制延迟。因此，不能只报告 VLA 的模型推理速度，还需要评价实际机器人动

作执行过程中的端到端时延。

空间记忆与对象状态 在长任务中，机器人可能无法始终看到全部目标。例如，某个对象被放入抽屉后，当前摄像头可能无法直接观察。因此，系统需要保存对象的历史位置和状态。例如，可以表示为：

$$M_t = (id_i, p_i, \sigma_i, t_i) \tag{18.17}$$

其中：

- id_i 表示对象标识；
- p_i 表示估计位置；
- σ_i 表示状态或质量信息；
- t_i 表示记录时间。

但历史位置不一定仍然有效。

例如，其他人可能移动物体。因此，机器人需要根据当前观测更新记忆，并在必要时主动重新观察。

18.4 行业差异与研究问题

场景	相对通用图文的新约束	优先验证对象
驾驶与多传感器	同步、标定、变化条件与闭环	时间错位、几何误差及失效恢复
医疗	样本分组、领域含义与高后果错误	外部验证、子组与专业判读
工业	小缺陷、长尾、设备变化	细粒度召回、误报及跨设备迁移
教育与交互	学习目标、响应时间与反馈	任务完成、事实正确与可用性
内容生成	条件控制、时间保持与来源	一致性、质量及可追溯性

这些差异决定具体协议，不宜仅因场景名称不同就重复介绍同一套编码器、检索和工具机制。

18.4.1 场景需求与模态选择

从应用目标出发定义任务 多模态系统设计首先需要回答的问题，不是选择哪一个大模型，而是明确系统需要完成什么任务。例如，一个企业希望利用多模态 AI 改善生产质量管理。这一需求仍然比较宽泛，可能涉及多种具体任务：产品表面缺陷识别。设备运行异常监测。质量报告生成。历史缺陷案例检索。自动化产品分拣。不同任务需要不同的输入、输出和验证方法。例如，表面缺陷检测可能主要依赖高分辨率图像，而设备异常监测可能更加依赖振动和温度时序数据。

因此，需要将总体业务目标转换为可以明确评价的任务。例如：“根据指定相机拍摄的产品图像，在固定时间预算内判断产品是否存在规定类型的缺陷，并输出缺陷位

置及必要的复检信息。”这种任务定义比“建立一个智能工业检测大模型”更加具体。它明确了输入、输出、目标条件和运行约束，有利于后续数据准备与模型评价。

确定任务输入与输出 对于一个多模态应用，可以将其基本功能表示为：$Y = f_\theta(X, C)$，其中：

- X 表示任务输入；
- C 表示任务条件与上下文；
- Y 表示期望输出；
- f_θ 表示系统中的模型或处理过程。

例如，医学影像辅助报告任务可能接收医学图像和经过授权的相关临床信息，输出结构化影像征象及报告草稿。

企业知识问答可能接收用户问题与文档知识库，输出带有来源引用的回答。机器人操作任务则可能接收视觉观测、机器人状态和自然语言指令，输出可执行动作。输出形式直接影响技术路线。例如，如果任务只要求判断图像是否存在缺陷，可能使用分类模型。如果需要确定缺陷位置，则可能需要检测或分割模型。如果还要求解释异常原因，就可能进一步涉及视觉语言模型、设备知识和外部检索。因此，设计阶段应当明确最终结果需要达到什么粒度。

确定任务成功条件 任务定义还应当包含明确的成功条件。例如，对于智能文档处理：字段是否正确提取。表格结构是否完整。输出是否满足规定格式。结果是否保存到正确位置。对于工业异常检测：是否识别规定类型的异常。漏检率是否处于可接受范围。是否满足生产节拍。对于语音助手：是否理解用户指令。是否正确执行目标操作。响应是否满足交互要求。因此，任务成功不能只通过模型输出是否流畅来判断。

例如，一个模型可能生成结构完整的分析报告，但其中关键数值来自错误的实验版本。此时，即使文字表达正确，也不应认定任务完全成功。因此，应在模型开发前确定目标结果和判定规则，避免在获得实验结果之后再修改成功标准。

模态互补性的判断 多模态融合的一个重要动机，是不同模态能够提供互补信息。但这种互补性需要通过任务数据和实际指标验证。例如，设单模态系统的任务指标为：M_A，加入第二种模态后的指标为：M_{A+B} 可以定义增益：$\Delta M = M_{A+B} - M_A$，如果指标越大越好，则正的 ΔM 表示融合系统在该评价条件下取得了提升。但这种提升不一定全部来自新增模态。

例如，融合模型可能具有更多参数、更长训练时间或不同预训练数据。因此，如果希望分析新增模态的贡献，就需要尽可能控制其他因素。例如，在统一训练和评价协议下比较：单模态模型、简单融合模型、复杂融合模型。相同计算预算下的替代模型。此外，还可以对新增模态进行缺失或错配干预，判断融合模型是否真正利用了该模态提供的信息。

模态间的空间与时间关系 当系统使用多种模态时，需要明确它们之间的对应关系。例如，摄像头图像和激光雷达点云通常具有不同坐标系。语音与视频具有不同采样频率。

工业振动信号和设备日志可能分别按照连续采样和离散事件记录。因此，需要根据应用建立空间和时间关联。例如，可以将多模态观测表示为：$O_t = (x_i, m_i, t_i, r_i)$，其中：

- x_i 表示观测内容；
- m_i 表示模态；
- t_i 表示时间；
- r_i 表示空间位置、设备编号或来源信息。

这种表示强调，多模态数据不能只根据文件是否同时存在，就假设其对应同一个物理状态。

例如，机器人动作执行之后，如果仍然使用执行前的视觉图像，就可能错误判断当前环境。因此，模态对齐是多模态系统设计的基本要求。

形成明确的任务契约 完成场景和模态分析后，可以形成一份任务契约（Task Contract）。其内容包括：输入数据及格式、预期输出及精度、评价任务集合、运行环境、数据访问权限。允许的异常处理方式。完成和失败判定条件。例如，一个企业文档助手可以规定：输入为指定类型的技术报告。输出为经过结构化核查的实验指标及来源引用。当缺少必要证据时，系统应明确指出信息不足。未经授权不得修改原始文件。最终结果需要通过字段和来源检查。

这种任务契约能够为后续模型选型和系统集成提供共同依据。但任务契约不是对模型能力的预先保证。它描述的是系统需要满足的要求，实际能否达到仍然需要通过测试验证。

18.4.2 原型验证与系统级评估

为什么需要分阶段验证 多模态应用系统往往包含多个模块。例如：数据解析、模态编码、融合模型、检索、工具调用、结果生成、状态验证。如果在初期一次性实现全部复杂机制，就可能难以判断最终结果由哪个模块决定。例如，一个完整知识问答系统回答错误，可能来自文档解析，也可能来自检索、生成或来源引用。因此，系统开发应采用分阶段验证方法。例如，先验证输入和输出是否正确，再逐步增加必要模块。每完成一个重要阶段，都检查其是否真正解决了预期问题。

这样能够减少重复开发和难以定位的系统错误。

最小可运行原型 最小可运行原型（Minimum Viable Prototype）主要用于验证核心任务是否具有技术可行性。例如，一个科研文档助手的初始版本可以只处理少量具有代表性的论文。其功能可能仅包括：读取 PDF。提取文本和表格。根据问题检索相关片段。生成带有来源的回答。不必在第一阶段实现完整的多智能体系统和所有文件格式。这种方式能够较快发现实际问题。例如，表格是否能够正确解析。模型是否能够读取重要数值。

检索是否能够找到必要证据。最小原型能够运行，并不意味着已经达到部署标准。其主要目标是获得可观察、可评价的早期证据。

统计显著性与稳定性 模型实验可能存在随机性。例如，训练初始化、数据采样和模型推理过程可能影响结果。因此，单次实验的少量指标提升不一定具有稳定性。可以使用多次重复实验和置信区间评价结果波动。例如，针对同一测试任务集合比较两个系统时，可以采用配对分析，减少不同任务难度带来的影响。对于具有明显分组结构的数据，例如多个医院、设备或视频录制，还需要考虑样本之间的相关性。例如，同一设备中的连续图像不应在统计分析时无条件视为完全独立样本。

因此，应根据实际实验单位选择合理的统计方法。

阶段验收与停止条件 为了避免系统长期处于反复调试却缺少明确结论的状态，可以为每个阶段设置验收条件。例如：数据解析是否达到最低正确性要求。基础模型是否优于简单基线。新增模态是否提供可重复的收益。完整流程是否满足任务成功标准。推理成本是否处于可接受范围。高风险操作是否受到有效控制。如果某个关键条件没有通过，应当分析原因。例如，视觉检索没有提供明显收益，可能需要检查任务是否真正依赖页面布局。

如果任务主要基于纯文本信息，那么继续增加视觉检索模型未必合理。因此，阶段验收不仅用于决定是否继续推进，也用于判断某条技术路线是否具有必要性。

系统级评价的总体框架 综合前面的讨论，可以形成一个面向多模态应用的系统级评价框架。这一框架并不要求每个应用使用完全相同的指标。例如，实时机器人系统需要更加关注控制频率和物理安全。企业文档助手则可能更加关注证据引用、权限隔离和数据更新。因此，系统级评价应根据任务性质确定重点。

从模型优化到系统优化 多模态大模型的研究通常重视模型结构、训练方法和基准性能。但在实际应用中，最终效果可能受到多个系统环节影响。例如，模型本身能够识别表格，但文档解析程序错误地调整了页面顺序。或者模型具有较好的推理能力，但检索器没有找到必要证据。因此，实际瓶颈不一定来自基础模型。系统优化应当首先识别主要错误来源。例如：如果输入解析错误，应优先改善数据处理。如果检索召回不足，应检查索引和查询机制。

如果模型不能利用正确证据，应分析上下文组织和生成机制。如果工具执行失败，应检查接口与权限。如果系统已经达到任务要求，则不一定需要继续增加模型复杂度。这种设计思路强调，应用优化应当围绕可验证的任务收益展开。

18.5 从复现到可检验的贡献

研究报告应围绕一个可以被反驳的判断展开。先说明现有基线在哪种条件下失败，再提出针对机制的改动，并设计能够区分替代解释的实验。若只增加数据、训练或推理预算，应把预算变化作为解释的一部分。

对负结果也应记录适用范围：方法在某类问题无收益，可能意味着假设不成立、证据不充分或优化未实现；需要进一步诊断，不能直接扩大为对整条技术路线的否定。最终报告应保留逐题结果与失败样本，使读者可以检查结论由什么证据支持。

习题与讨论

1. 从三项案例中选一项，写出一页实验协议：任务、数据隔离、基线、唯一改动、指标、预算和失败判据。
2. 为所选案例提出一个能推翻预期结论的对照，并说明若对照成立，应怎样收缩主张。
3. 设计结果表，使读者能同时判断性能、成本、方差和关键子组，而不依赖正文中的形容词。
4. 写一段负结果分析，区分实现错误、统计证据不足与研究假设不成立，并列出下一项最有判别力的实验。

第六部分

附录

附录 A　先修知识与实现速查

本附录用于查阅，不承担重新教授深度学习的任务。正文出现未熟悉的符号时，应先补相应基础，再回到任务和计算图。

A.1　向量、形状与归一化

行向量按最后一维存储特征。对 $Z \in \mathbb{R}^{N\times d}$ 施加逐位置线性映射 $W \in \mathbb{R}^{d\times d'}$，输出为 $ZW \in \mathbb{R}^{N\times d'}$。改变特征维度与减少位置数量是两种操作。张量维度相容只保证运算可执行，不能证明坐标、单位、时间或语义正确。

向量的二范数为 $\|z\|_2 = (\sum_j z_j^2)^{1/2}$。L2 归一化按每个向量计算；LayerNorm、RMSNorm 及跨批次统计具有不同的归约维度与参数。实现时先明确哪些轴参与统计，尤其要区分 Token 维、通道维和批量维。

A.2　概率、交叉熵与监督位置

对于类别 Logit a_k，$p_k = \exp(a_k)/\sum_j \exp(a_j)$。单样本负对数似然为 $-\log p_y$。序列目标须进一步说明哪些位置被预测、哪些位置有效，以及按 Token 还是按样本平均。遮罩从分母中排除无效位置，与把这些位置的损失乘零后仍除以原长度不同。

对自回归序列，教师强制提供已知的真实前缀。训练可在因果遮罩下并行计算各位置；推理需要逐步生成。将真实答案前缀输入用于训练，并不表示在评价时可以向模型提供尚未生成的真实答案。

A.3　注意力、残差与梯度

令 $Q \in \mathbb{R}^{N_q\times d_k}$、$K \in \mathbb{R}^{N_k\times d_k}$、$V \in \mathbb{R}^{N_k\times d_v}$，则

$$\mathrm{Attn}(Q,K,V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V. \tag{A.1}$$

遮罩 M 的可见项为 0，屏蔽项取负无穷的数值实现。Softmax 沿键轴计算；一整行全屏蔽时应检查实现行为，避免产生无效值。自注意力的 Q/K/V 来自同一状态，交叉注意力来自不同状态；它们都可带任务规定的可见性约束。

残差保留从输入到输出的直接路径，归一化位置影响块结构。冻结参数意味着优化器不更新该参数；只要损失需要对输入求导，就仍须保留相应前向计算图。这一规则在连接器训练与冻结生成网络时尤其关键。

A.4 优化、资源与复现

梯度下降写为 $\theta_{t+1} = \theta_t - \eta_t \nabla \mathcal{L}_t$，实际优化器还可能具有动量或自适应状态。权重相同而优化器、采样或随机状态不同，通常不能保证后续更新相同。

参数存储、梯度、优化器状态、激活和临时工作区应分别估计。FLOPs 描述运算量，时间还依赖内存访问、算子、通信、调度与硬件。训练精度、权重存储精度和累计精度也可能不同，不能用一个“低精度”概括。

A.5 卷积与尺度的必要背景

卷积的局部连接、权重共享和步幅引入空间结构。多层下采样逐渐扩大感受野，也降低可直接访问的空间采样密度。多尺度特征的目的之一是同时提供局部细节与较大范围语义；全局汇聚适合整体判断，但会改变下游可用的位置接口。正文第二章讨论这些选择怎样影响多模态任务。

附录 B　符号、单位与接口约定

B.1　常见记号

记号	含义与需检查的条件
I, T, A, V	图像、文本、音频和视频；T 也可能在局部表示时长，使用处另行定义。
B, N, L, d	批量、输入位置数、目标序列长度、隐藏维度；不默认所有模态长度相同。
$Z^{(m)}$	第 m 模态编码后的状态；注明批量、时间、位置和通道轴。
q, y, h	问题或指令、目标或答案、历史上下文；不把训练标签混入测试可见条件。
θ, ϕ, ψ	局部定义的参数集合；每阶段另外列出冻结与更新范围。
τ, β	温度或目标系数；相同符号在不同损失中作用可能不同。
t_i, a_i	时间和动作；时间应区分采样索引、事件时间、到达时间与决策时间。
$T_{A\leftarrow B}$	本书解释变换方向时采用的记号：将 B 中的列坐标转换到 A。
$\mathcal{L}, \ell$	总体或批次损失、单项损失；均注明有效项和归一化分母。
$C(\tau), R(\tau)$	选择性预测的接收比例与接收条件下风险；与保形集合包含真值的概率区分。
Γ_α	保形预测集合；覆盖保证须附校准与测试的交换性等条件。

B.2　每个接口必须记录的内容

空间接口记录坐标系、原点、方向、尺度和单位；时间接口记录采样间隔、时间戳含义及可见范围；序列接口记录模态标记、位置、长度和遮罩；输出接口记录结构、量纲、允许范围与终止规则。图表、代码和正文应使用同一套约定。

附录 C　习题解答要点

本附录给出判断标准和推导入口。实验题没有预设分数，答案应由实际记录支持；合理的负结果同样可以完成题目。

C.1　第 1 章

1. 图像问答的条件含图像与问题，输出为答案；检索输出为候选排序，监督来自相关性；动作输出还需执行语义。真实答案、未来观测与测试标签不可作为测试条件。
2. 同维保证算术接口可用；语义空间要求训练得到可比较关系；联合表示由多输入共同计算。两套独立同维编码器可以没有语义对齐，双编码器对齐也可以不在编码时融合。
3. 使用无图、错配图、受控改变关键对象或属性的成对问题；固定问题和解码。检查预测是否随相关证据变化，并区分可由常识直接回答的问题。
4. 将视觉编码、连接、语言计算和回答损失连接。被冻结的语言模型仍需向连接器输入回传梯度；仅可训练参数写入优化器。

C.2　第 2 章

1. 可整除时 $N = (H/P)(W/P)$。填充增加块数并引入无效区域，裁剪减少可见区域；二者都需要保存有效范围与坐标映射。
2. 若缩放后的局部像素为 (u, v)，原图为 $(u/s + x_0, v/s + y_0)$；若还做填充，应先减去填充偏移。
3. 汇聚把多个位置组合成一个向量，输出不直接保留逐位置对应。特征可能含隐式位置线索，但恢复坐标仍需有定义的预测头和监督，不能从语义强推断位置精确。
4. 固定编码器、总 Token 和生成设置；分开报告小文字、小目标、计数、整体语义和定位。总预算相同仍需记录切块开销及视野重叠。

C.3　第 3 章

1. 完整窗口数为 $1 + \lfloor (L - 400)/160 \rfloor$，第 m 个窗口起点为 $160m/16000$ 秒，m 从 0 开始；居中补零会改变这一边界计数。
2. 连续特征保留向量值；离散表示还受码本与量化误差约束；多码本必须说明并行预测或串行展开。长度由帧率、下采样、码本组织和特殊标记共同决定。

3. 改采样可让事件进入观测，高频表示可能保留跨帧变化；增加推理预算只处理已有输入，不能恢复从未采集的证据。
4. 按同一事件保留多偏移版本，问题中包含只靠单一模态和必须同步关联两类，报告随偏移变化的指标，禁止重新训练或选择最优测试偏移。

C.4 第 4 章

1. 先有 $p_B = T_{B\leftarrow C} p_C$，再代入 $p_A = T_{A\leftarrow B} p_B$。逆变换为 $T_{C\leftarrow A} = (T_{A\leftarrow B} T_{B\leftarrow C})^{-1}$。
2. 当前关节角是状态测量，目标关节角可以是位置控制动作；动作还需控制周期、限幅与执行器解释。相同单位和维度不代表同一时间含义。
3. 事件帧常汇聚窗口内变化，体素保留分箱时间，时间表面强调最近事件。聚合中丢失的精确先后或重复次数不能由模型确定性补回。
4. 每条记录携带传感器 ID、时间戳、参考系、数值与有效标记；查询按时间区间关联，可加入时间差。无需把不同来源强行变成等长数组。

C.5 第 5 章

1. 交叉熵对行 Logit 的导数为 $(p_{ij} - \mathbf{1}\{i = j\})/B$；再乘 $1/\tau$。列损失的归一化对象不同，总梯度为两项之和再乘二分之一。
2. 四配对的大批次每个查询有三个负候选；两个二配对微批次各只有一个。跨微批次的图文负对没有被计算；梯度累积不自动建立这些项。
3. 可以对正例集合中的概率求和后取负对数，或对每个正例的对数概率求平均；二者目标不同，必须明确定义并处理多对多标注。
4. 原图与已有类别的嵌入点积可不变，但 Softmax 分母增加，原概率通常下降。变化反映候选集合，不是图像事实本身发生改变。

C.6 第 6 章

1. 线性输出 $N \times d$，含偏置时参数 $d_v d + d$；隐藏维 h 的两层 MLP 为 $d_v h + h + hd + d$。逐位置映射不改变 N。
2. $\nabla_\phi \mathcal{L} = (\partial H/\partial \phi)^\top \nabla_H \mathcal{L}$。冻结语言参数不消除损失对输入的导数；整段无梯度前向可能切断该链。
3. 文本查询读取时直接依赖当前文本状态；固定视觉查询的预压缩可在问题之前完成。若查询本身由问题条件化，则要另算其条件依赖与复用范围。
4. 同时列出固定视觉预算与固定参数预算两套对照，分别检验信息瓶颈和训练容量。保持数据、步骤、初始化与解码一致。

C.7 第 7 章

1. 输入融合先组合观测，特征融合在编码后交互，决策融合组合预测。同一系统可以先局部特征交互，再做输出级集成。
2. 共享全注意力配对数约 $(N_v + N_a + N_t)^2$；文本定向读取视听约 $N_t(N_v + N_a)$，另加文本自身及各编码器成本。
3. 路由权重由优化目标和竞争决定，未必按可靠概率训练。可在受控噪声/缺失下测权重与错误关联，并检验校准及分布变化。
4. 逐项记录视觉语义编码与生成码是否相同、主干是否共享、不同预测头及其损失。统一可以发生在主干而不发生在全部输入输出机制。

C.8 第 8 章

1. 自回归预测离散下一项；噪声预测扩散以带噪数据、时间及条件预测噪声；流匹配预测路径上的速度。损失分母、目标维度和尺度不同。
2. 先对真实图像编码解码测重建，再固定自编码器测潜在生成；两类错误分开。不能要求生成模型补回自编码器已经舍弃的全部细节。
3. 进行对象—属性绑定核验和区域定位核验，或构造交换颜色的成对提示。整体图文分数可能对细节不敏感。
4. 长度增加可能减少边界数量；共享状态帮助连续性；采样器影响更新路径但不自动提供跨段约束。用固定预算与相同前段条件比较。

C.9 第 9 章

1. 媒体 ID、版本哈希、页/帧/区域、坐标规范、消息与目标、掩码、来源组、许可和质检标记应分开记录；修改处理器版本后可追踪旧样本。
2. 可支持相近片段分布的表现，不能据此主张跨视频或跨场景泛化。应按原视频或更高层来源划分。
3. 按样本平均赋每样本相同总权重；按 Token 平均赋每有效目标位置相同权重。长序列的贡献不同，实际采样率还要共同报告。
4. 质检器与生成器尽量解耦，抽查视觉可回答性、事实、重复和分组偏差；不能只用生成器对自己的置信分数筛选。

C.10 第 10 章

1. 每阶段写明编码器、连接器、主干与头的更新范围及学习率；解冻时明确新增参数的优化器状态，迁移或重置调度需记录。

2. 全局平均为 $\sum_b n_b \bar{\ell}_b / \sum_b n_b$；微批次等权对应每批贡献相同，不等于每 Token 等权。还需处理设备间和最后一批差异。
3. 固定初始权重和数据，连续运行与保存恢复运行在下一次更新后比较参数、优化器计数和采样索引；误差阈值需考虑非确定算子及精度。
4. 先锁定首个异常批次，查媒体/标签与有效项，再逐层检查有限值、损失分母和梯度，最后检查优化器与精度缩放。每次仅改变可定位的因素。

C.11 第 11 章

1. 媒体及用户位置通常作为条件；助手目标位置参与监督。历史助手轮是否计入取决于具体训练目标，应与当前轮监督明确区分，不能由模板默认决定。
2. 参数比为 $r(d_{\text{out}} + d_{\text{in}})/(d_{\text{out}} d_{\text{in}})$。若推理仍通过完整基座或连接被合并，参数训练量的下降不会按同倍率改变全部前向成本。
3. DPO 使用当前策略与参考策略的优选/劣选对数概率差，再对差值作偏好目标；SFT 只需示范，不直接比较未被偏好的回答。
4. 固定训练预算与数据后分别测纯文本和视觉任务，加入数据回放/冻结范围消融；独立测试分布诊断过拟合，不把能力变化全归因于遗忘。

C.12 第 12 章

1. 类别输出是离散标签，定位含空间范围，表格抽取含结构及单元格归属。评价应与输出结构对应，单一准确率会丢失错误类型。
2. 答案和来源联合正确才记联合成功；另报告值正确率、来源正确率及交集。保存行列与页面坐标以复核。
3. 两个候选均满足且无额外证据是表达歧义；唯一目标存在而预测偏离是定位错误；所需属性不可观察是证据不足。
4. 原图与裁剪检查局部信息，错配检查条件敏感性，纯文本估计语言先验。需控制裁剪提供的额外提示，不能将定位提示收益全归模型能力。

C.13 第 13 章

1. 先核实读图得到的量与单位，再检查变量绑定、运算与输出。提供正确结构化读数的对照可分离部分感知误差，但属于额外信息条件。
2. 共同原因可同时引起两个先后事件；未来遮罩只阻止信息泄漏，不建立无混杂、干预或结构因果假设。
3. 限制所有候选总 Token 及验证预算，报告实际用量和停止规则；长候选减少可生成数量，固定候选数不是固定总预算。

4. 抽取数值、区域、关系或工具结果并逐项核验；中间解释只能作为待检验输出，不能用其长度或自述评价真实内部推理。

C.14 第 14 章

1. 不同预算口径回答不同问题；若只研究架构，可先固定数据和训练预算，再给资源—性能曲线。分辨率改变也会改变视觉序列与算力。
2. 同源问题可能相关，逐题重采样低估相关结构。按视频或文档整组重采样更接近来源层泛化；同时注意组数不足导致估计不稳。
3. 顺序交换测位置偏差，长度控制测风格偏差，人工子集测实际判定质量。三者不能相互替代，人工标准也需明确。
4. 同时给总体与子组样本量、差值区间、错误后果和成本；是否部署取决于目标分布及约束，不由平均分单独决定。

C.15 第 15 章

1. 未标注不等于不存在，应回看原始观测并区分漏标与模型捏造。定义允许同义、区域证据与不可判断情况。
2. 相同预测可赋不同概率；准确率相同而 ECE 不同。完美校准的低准确模型也不因此成为高能力模型。
3. 风险分母为接收概率 $C(\tau)$，零覆盖时无接收条件分布。全拒答的服务仍有不可用及回退成本，应报告覆盖和系统结果。
4. 分开记录感知退化、对应错误和指令越权。模型可能提出非法动作而执行层阻止，二者应有不同指标。

C.16 第 16 章

1. 若其余串行阶段固定，总耗时下限为原来的 0.8，理想加速为 $1/0.8 = 1.25$。存在并行或瓶颈迁移时需另算关键路径。
2. 逐层、每 Token 保存键和值，乘批量与元素字节数即得表达。GQA 改变键值头数，长度和批量线性改变主体缓存；实际分配还含额外开销。
3. 分别测算子时间、带宽、精度转换、目标批量及硬件内核。权重更小只保证某部分存储变化，未保证相同质量或总体时延。
4. 以固定请求分布重放，注明热身、队列、并发和停止；报告首 Token、逐 Token 及端到端延迟分位数，并分开冷启动。

C.17 第 17 章

1. 模型提出检索/计算调用；执行器验证参数与权限；状态记录证据；终止由结果与条件验证。工具返回视为数据，不能自动授予新的权限。
2. 保留事件时间、到达时间与修订号；决策只使用已到达信息，迟到数据可以产生新版本，但不得改写当时可见上下文。
3. 分别检查表示能否保留控制变量、用真实状态与自由滚动的误差差异、规划目标是否与实际成功一致。模型内收益不代替执行结果。
4. 在统一任务与初始状态下闭环测成功、成本和失败；仿真与真实分开，动作接口和统计试验次数明确。

C.18 第 18 章

1. 协议必须具体到样本来源、隔离单位、可见信息与停止规则；唯一改动清楚，性能和成本都能从日志复算。
2. 选择一个替代解释对应的对照，如纯文本、错配证据或等预算简单基线。对照成立时，应删除不被支持的机制归因。
3. 列出预算口径、均值/差值、区间或种子波动、关键子组和实际成本；实验未运行的单元不填写预期数值。
4. 先验证实现与目标一致，再检查效应区间是否足够窄；若证据排除预期收益，再讨论假设范围。下一实验应能区分最主要的两个解释。

附录 D　实践协议与复现记录

D.1　六项机制实践

这些实践先检验实现与数学目标是否一致，再扩展到真实模型。工程的 `labs/mechanism_checks.py` 使用 Python 标准库，可以直接运行，输出为机制核对结果，不是模型性能。

1. **视觉预算与坐标。**按输入尺寸、块大小和切块规则计算长度；建立局部到全局的坐标往返测试。再使用真实图片检查文字与对象是否在预处理后仍可见。
2. **对比损失与负例。**在小相似度矩阵上计算行列损失，用有限差分检查梯度；拆成微批次后比较缺少的负对。不要把该诊断当成真实编码器训练结果。
3. **监督掩码与归一化。**手工构造变长答案，分别计算微批次均值和全局 Token 均值；检查无效位置、零有效项及跨设备分母。
4. **低秩适配。**在小矩阵上比较显式低秩分支与合并后的权重输出，核对缩放和形状；实际训练另需检查初始化、目标模块与优化器范围。
5. **校准与选择风险。**用带真值的预测记录计算准确率、ECE、接收比例与条件风险，绘制随阈值变化的结果；全拒答时风险标记为未定义。
6. **缓存与系统成本。**根据形状估计 KV 主体存储，实际运行时测峰值显存、首 Token 及端到端时间，分析理论口径与实现的差异。

D.2　每次实验的最小记录

记录项	内容
数据	来源、许可、版本、划分组、清洗与去重、样本数与长度分布。
模型	权重版本、处理器、视觉/音频编码器、连接方式、参数与精度。
训练	可训练模块、目标、掩码、归一化、采样率、优化器、学习率、批量与预算。
推理	提示模板、可见上下文、采样、候选数、工具、终止和失败处理。
资源	硬件、软件版本、精度、并发、预热、测量范围及实际用量。
验证	基线、唯一变量、逐题记录、样本聚类、随机种子、区间与失败类别。
恢复	权重、优化器、调度、随机状态、数据进度及恢复检查。

D.3 三项综合案例的验收

文档案例要求答案与来源能够共同复核；持续音视频案例要求每次决策的可见范围可重放；具身案例要求动作接口与离线/闭环结论分开。所有案例均应提交一个简单基线、一项受控改动和一组能够推翻预期结论的测试。

D.4 来源与本版整理范围

本版技术材料来源于原 21 章讲义及其记录的文献。参考文献库对 43 项核心工作核验了原始书目信息；校准与保形预测的关键定义另对照全文。该核验范围不能推及原书所有点名模型与全部技术细节。原稿有部分公式由迁移时恢复，优化版保留计算含义并整理符号；重要恢复与删改记录存于 `editorial/`。

正文中的教学算例和机制图用于解释定义，不作为实测结果。原有按相邻文字重排的表格已从正文撤出，原表文件仍在原稿快照中；本版新表是教学归纳。未新增未经检验的排行榜或“最新最佳”结论。

参考文献

[1] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. https://arxiv.org/abs/2010.11929, 2020. arXiv 预印本；年份为初次提交年.

[2] Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross Girshick. Masked Autoencoders Are Scalable Vision Learners. https://arxiv.org/abs/2111.06377, 2021. arXiv 预印本；年份为初次提交年.

[3] Mathilde Caron, Hugo Touvron, Ishan Misra, Hervé Jégou, Julien Mairal, Piotr Bojanowski, and Armand Joulin. Emerging Properties in Self-Supervised Vision Transformers. https://arxiv.org/abs/2104.14294, 2021. arXiv 预印本；年份为初次提交年.

[4] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, Mahmoud Assran, Nicolas Ballas, Wojciech Galuba, Russell Howes, Po-Yao Huang, Shang-Wen Li, Ishan Misra, Michael Rabbat, Vasu Sharma, Gabriel Synnaeve, Hu Xu, Hervé Jegou, Julien Mairal, Patrick Labatut, Armand Joulin, and Piotr Bojanowski. DINOv2: Learning Robust Visual Features without Supervision. https://arxiv.org/abs/2304.07193, 2023. arXiv 预印本；年份为初次提交年.

[5] Oriane Siméoni, Huy V. Vo, Maximilian Seitzer, Federico Baldassarre, Maxime Oquab, Cijo Jose, Vasil Khalidov, Marc Szafraniec, Seungeun Yi, Michaël Ramamonjisoa, Francisco Massa, Daniel Haziza, Luca Wehrstedt, Jianyuan Wang, Timothée Darcet, Théo Moutakanni, Leonel Sentana, Claire Roberts, Andrea Vedaldi, Jamie Tolan, John Brandt, Camille Couprie, Julien Mairal, Hervé Jégou, Patrick Labatut, and Piotr Bojanowski. DINOv3. https://arxiv.org/abs/2508.10104, 2025. arXiv 预印本；年份为初次提交年.

[6] Wei-Ning Hsu, Benjamin Bolte, Yao-Hung Hubert Tsai, Kushal Lakhotia, Ruslan Salakhutdinov, and Abdelrahman Mohamed. HuBERT: Self-Supervised Speech Representation Learning by Masked Prediction of Hidden Units. https://arxiv.org/abs/2106.07447, 2021. arXiv 预印本；年份为初次提交年.

[7] Alexei Baevski, Henry Zhou, Abdelrahman Mohamed, and Michael Auli. wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations. https://arxiv.org/abs/2006.11477, 2020. arXiv 预印本；年份为初次提交年.

[8] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning Transferable Visual Models From Natural Language Supervision. https://arxiv.org/abs/2103.00020, 2021. arXiv 预印本；年份为初次提交年.

[9] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust Speech Recognition via Large-Scale Weak Supervision. https://arxiv.org/abs/2212.04356, 2022. arXiv 预印本；年份为初次提交年.

[10] Junnan Li, Ramprasaath R. Selvaraju, Akhilesh Deepak Gotmare, Shafiq Joty, Caiming Xiong, and Steven Hoi. Align before Fuse: Vision and Language Representation Learning with Momentum Distillation. https://arxiv.org/abs/2107.07651, 2021. arXiv 预印本；年份为初次提交年.

[11] Junnan Li, Dongxu Li, Caiming Xiong, and Steven Hoi. BLIP: Bootstrapping Language-Image Pre-training for Unified Vision-Language Understanding and Generation. https://arxiv.org/abs/2201.12086, 2022. arXiv 预印本；年份为初次提交年.

[12] Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models. https://arxiv.org/abs/2301.12597, 2023. arXiv 预印本；年份为初次提交年.

[13] Jean-Baptiste Alayrac, Jeff Donahue, Pauline Luc, Antoine Miech, Iain Barr, Yana Hasson, Karel Lenc, Arthur Mensch, Katie Millican, Malcolm Reynolds, Roman Ring, Eliza Rutherford, Serkan Cabi, Tengda Han, Zhitao Gong, Sina Samangooei, Marianne Monteiro, Jacob Menick, Sebastian Borgeaud, Andrew Brock, Aida Nematzadeh, Sahand Sharifzadeh, Mikolaj Binkowski, Ricardo Barreira, Oriol Vinyals, Andrew Zisserman, and Karen Simonyan. Flamingo: a Visual Language Model for Few-Shot Learning. https://arxiv.org/abs/2204.14198, 2022. arXiv 预印本；年份为初次提交年.

[14] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual Instruction Tuning. https://arxiv.org/abs/2304.08485, 2023. arXiv 预印本；年份为初次提交年.

[15] Chameleon Team. Chameleon: Mixed-Modal Early-Fusion Foundation Models. https://arxiv.org/abs/2405.09818, 2024. arXiv 预印本；年份为初次提交年.

[16] Rohit Girdhar, Alaaeldin El-Nouby, Zhuang Liu, Mannat Singh, Kalyan Vasudev Alwala, Armand Joulin, and Ishan Misra. ImageBind: One Embedding Space To Bind Them All. https://arxiv.org/abs/2305.05665, 2023. arXiv 预印本；年份为初次提交年.

[17] Chengyue Wu, Xiaokang Chen, Zhiyu Wu, Yiyang Ma, Xingchao Liu, Zizheng Pan, Wen Liu, Zhenda Xie, Xingkai Yu, Chong Ruan, and Ping Luo. Janus: Decoupling Visual Encoding for Unified Multimodal Understanding and Generation. https://arxiv.org/abs/2410.13848, 2024. arXiv 预印本；年份为初次提交年.

[18] Chunting Zhou, Lili Yu, Arun Babu, Kushal Tirumala, Michihiro Yasunaga, Leonid Shamis, Jacob Kahn, Xuezhe Ma, Luke Zettlemoyer, and Omer Levy. Transfusion: Predict the Next Token and Diffuse Images with One Multi-Modal Model. https://arxiv.org/abs/2408.11039, 2024. arXiv 预印本；年份为初次提交年.

[19] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. https://arxiv.org/abs/2205.14135, 2022. arXiv 预印本；年份为初次提交年.

[20] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising Diffusion Probabilistic Models. https://arxiv.org/abs/2006.11239, 2020. arXiv 预印本；年份为初次提交年.

[21] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-Resolution Image Synthesis with Latent Diffusion Models. https://arxiv.org/abs/2112.10752, 2021. arXiv 预印本；年份为初次提交年.

[22] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow Matching for Generative Modeling. https://arxiv.org/abs/2210.02747, 2022. arXiv 预印本；年份为初次提交年.

[23] Wenliang Dai, Junnan Li, Dongxu Li, Anthony Meng Huat Tiong, Junqi Zhao, Weisheng Wang, Boyang Li, Pascale Fung, and Steven Hoi. InstructBLIP: Towards General-purpose Vision-Language Models with Instruction Tuning. https://arxiv.org/abs/2305.06500, 2023. arXiv 预印本；年份为初次提交年.

[24] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-Rank Adaptation of Large Language Models. https://arxiv.org/abs/2106.09685, 2021. arXiv 预印本；年份为初次提交年.

[25] Yifan Li, Yifan Du, Kun Zhou, Jinpeng Wang, Wayne Xin Zhao, and Ji-Rong Wen. Evaluating Object Hallucination in Large Vision-Language Models. https://arxiv.org/abs/2305.10355, 2023. arXiv 预印本；年份为初次提交年.

[26] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient Finetuning of Quantized LLMs. https://arxiv.org/abs/2305.14314, 2023. arXiv 预印本；年份为初次提交年.

[27] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal Policy Optimization Algorithms. https://arxiv.org/abs/1707.06347, 2017. arXiv 预印本；年份为初次提交年.

[28] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. https://arxiv.org/abs/2305.18290, 2023. arXiv 预印本；年份为初次提交年.

[29] Xiang Yue, Yuansheng Ni, Kai Zhang, Tianyu Zheng, Ruoqi Liu, Ge Zhang, Samuel Stevens, Dongfu Jiang, Weiming Ren, Yuxuan Sun, Cong Wei, Botao Yu, Ruibin Yuan, Renliang Sun, Ming Yin, Boyuan Zheng, Zhenzhu Yang, Yibo Liu, Wenhao Huang, Huan Sun, Yu Su, and Wenhu Chen. MMMU: A Massive Multi-discipline Multimodal Understanding and Reasoning Benchmark for Expert AGI. https://arxiv.org/abs/2311.16502, 2023. arXiv 预印本；年份为初次提交年.

[30] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing Reasoning and Acting in Language Models. https://arxiv.org/abs/2210.03629, 2022. arXiv 预印本；年份为初次提交年.

[31] Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter. GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium. https://arxiv.org/abs/1706.08500, 2017. arXiv 预印本；年份为初次提交年.

[32] Yuan Liu, Haodong Duan, Yuanhan Zhang, Bo Li, Songyang Zhang, Wangbo Zhao, Yike Yuan, Jiaqi Wang, Conghui He, Ziwei Liu, Kai Chen, and Dahua Lin. MMBench: Is Your Multi-modal Model an All-around Player? https://arxiv.org/abs/2307.06281, 2023. arXiv 预印本；年份为初次提交年.

[33] Anna Rohrbach, Lisa Anne Hendricks, Kaylee Burns, Trevor Darrell, and Kate Saenko. Object Hallucination in Image Captioning. https://arxiv.org/abs/1809.02156, 2018. arXiv 预印本；年份为初次提交年.

[34] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On Calibration of Modern Neural Networks. https://arxiv.org/abs/1706.04599, 2017. arXiv 预印本；年份为初次提交年.

[35] Anastasios N. Angelopoulos and Stephen Bates. A Gentle Introduction to Conformal Prediction and Distribution-Free Uncertainty Quantification. https://arxiv.org/abs/2107.07511, 2021. arXiv 预印本；年份为初次提交年.

[36] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, Pete Florence, Chuyuan Fu, Montse Gonzalez Arenas, Keerthana Gopalakrishnan, Kehang Han, Karol Hausman, Alexander Herzog, Jasmine Hsu, Brian Ichter, Alex Irpan, Nikhil Joshi, Ryan Julian, Dmitry Kalashnikov, Yuheng Kuang, Isabel Leal, Lisa Lee, Tsang-Wei Edward Lee, Sergey Levine, Yao Lu, Henryk Michalewski, Igor Mordatch, Karl Pertsch, Kanishka Rao, Krista Reymann, Michael Ryoo, Grecia Salazar, Pannag Sanketi, Pierre Sermanet, Jaspiar Singh, Anikait Singh, Radu Soricut, Huong Tran, Vincent Vanhoucke, Quan Vuong, Ayzaan Wahid, Stefan Welker, Paul Wohlhart, Jialin Wu, Fei Xia, Ted Xiao, Peng Xu, Sichun Xu, Tianhe Yu, and Brianna Zitkovich. RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control. https://arxiv.org/abs/2307.15818, 2023. arXiv 预印本；年份为初次提交年.

[37] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion Policy: Visuomotor Policy Learning via Action Diffusion. https://arxiv.org/abs/2303.04137, 2023. arXiv 预印本；年份为初次提交年.

[38] David Ha and Jürgen Schmidhuber. World Models. https://arxiv.org/abs/1803.10122, 2018. arXiv 预印本；年份为初次提交年.

[39] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering Diverse Domains through World Models. https://arxiv.org/abs/2301.04104, 2023. arXiv 预印本；年份为初次提交年.

[40] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. OpenVLA: An Open-Source Vision-Language-Action Model. https://arxiv.org/abs/2406.09246, 2024. arXiv 预印本；年份为初次提交年.